

Galbot SDK 开发文档

G1 系列机器人 · SDK 1.8.1

来源: developer.galbot.com/docs/SDK/1.8.1/g1/zh/

生成时间: 2026-08-24 10:36:21

目录

目录.....	2
1. 产品概述.....	30
核心功能	30
1. 基础能力	30
2. 高级能力	30
3. 业务能力	31
应用场景	31
服务与支持	32
阅读建议	33
2. 系统要求.....	34
3. 术语说明.....	35
1. 机器人本体硬件	35
2. 机器人学基础	36
3. 控制与接口	37
4. 版本兼容性检查.....	40
快速使用	40
5. 环境部署.....	41
6. 机器人端部署.....	42
1. 连接机器人网络	42
2. 部署 SDK 到机器人	47
3. 运行程序	47
3.1 Python 程序	47

3.2 C++程序	48
7. PC 端部署	49
8. PC 端部署 - Ubuntu	50
1. 物理连接	50
2. 配置网络	52
2.1 配置 PC 网口 IP	52
2.2 配置 PC 端 IP 配置文件	53
2.3 配置 Orin 端 IP 配置文件	54
2.4 配置 XCU 端 IP 配置文件	54
2.5 验证网络连接	55
3. 在 PC 上安装 SDK	55
4. 编译程序	56
5. 运行程序	56
9. PC 端部署 - WSL2	57
1. 物理连接	57
2. 配置网络	59
2.1 配置 PC 网口 IP	59
2.2 配置 PC 端 IP 配置文件	61
2.3 配置 Orin 端 IP 配置文件	61
2.4 配置 XCU 端 IP 配置文件	62
2.5 验证网络连接	62
3. 在 PC 上安装 SDK	63
4. 编译程序	63
5. 运行程序	63

10. PC 端部署 - VMWare.....	65
1. 物理连接	65
2. 配置网络	67
2.1 配置 PC 网口 IP	67
2.2 配置 PC 端 IP 配置文件	72
2.3 配置 Orin 端 IP 配置文件	73
2.4 配置 XCU 端 IP 配置文件	73
2.5 验证网络连接	74
3. 在 PC 上安装 SDK	74
4. 编译程序	74
5. 运行程序	74
11. Docker 部署.....	76
1. 环境配置	76
2. 获取镜像	76
3. 启动容器	76
12. 常规操作.....	77
地图引擎 - 建图与定位	77
建图流程	77
定位流程	86
13. 入门教程.....	88
Example1. 机器人的基础控制	88
Example2. 手臂操控	91
Example3. 机器人导航	97
Example4. 传感器数据获取	102

Example5. 抓取和放置物体	109
Example6. 执行模拟 VLA 结果	118
Example7. 双臂协同控制	126
Example8. 实时控制循环	129
Example9. 复杂多路点轨迹规划	132
Example10. 综合任务：导航+感知+抓取+放置	136
14. Python 示例	147
机器人关节名称 (G1 2.2)	147
关节组列表	147
各关节组详细信息	147
如何正确使用 joint_groups	147
G1 关节组典型场景	148
传感器类型与 Frame 说明	148
类：GalbotRobot	149
获取实例并初始化 (get_instance && init)	149
日志函数	149
设置关节角度 (set_joint_positions)	150
设置夹爪指令 (set_gripper_command)	151
设置吸盘指令 (set_suction_cup_command)	152
停止轨迹执行 (stop_trajectory_execution)	152
设置轨迹 (execute_joint_trajectory)	153
设置关节指令 (set_joint_commands)	154
批量设置关节指令 (set_joint_commands_batch)	156
直接发布原始目标 (publish_target)	158

直接请求原始目标 (request_target)	163
设置底盘速度 (set_base_velocity)	169
获取关节状态 (get_joint_states)	169
获取关节位置 (get_joint_positions)	170
获取关节名称 (get_joint_names)	171
获取夹爪状态 (get_gripper_state)	171
获取吸盘状态 (get_suction_cup_state)	172
获取坐标变换 (get_transform)	173
获取 IMU 数据 (get_imu_data)	174
获取设备信息 (get_device_information)	175
获取电池信息 (get_bms_information)	176
获取相机图像数据 (get_rgb_data && get_depth_data)	177
获取传感器内外参 (get_camera_intrinsic && get_sensor_extrinsic)	179
获取雷达数据 (get_lidar_data)	181
一键回零-odom 坐标系 (whole_body_reset_zero_odom)	185
一键回零-map 坐标系 (whole_body_reset_zero_map)	186
类: GalbotMotion	187
获取实例并初始化	187
正运动学 (使用当前状态或指定 RobotStates)	187
逆运动学 (基础与基于 RobotStates)	190
获取与设置末端位姿	193
单点运动规划 (关节空间与笛卡尔空间)	195
多点轨迹规划	198
碰撞检测	200

附加工具	202
卸载工具	203
获取连杆名称列表	204
加载/移除环境碰撞体	205
类: GalbotNavigation	207
获取实例 / 初始化	207
重定位 / 是否已定位 / 获取当前位姿	207
检查路径可达性并阻塞导航到目标	208
非阻塞导航 + 轮询判断到达	208
直线移动到目标 / 停止导航	209
获取导航状态 + 轮询 SUCCESS/FAILED 或超时	210
完整运行示例 (简单流程)	211
类: GalbotPerception	212
高精度双目深度估计, 单次推理并保存伪彩色深度图	212
类: Parameter	214
辅助函数	214
check_motion_status	214
create_joint_state / create_pose_state	214
15. C++ 示例	216
机器人关节名称 (G1 2.2)	216
关节组列表	216
各关节组详细信息	216
如何正确使用 joint_groups	216
G1 关节组典型场景	217

传感器类型与 Frame 说明	217
类: GalbotRobot	218
获取实例并初始化 (get_instance && init)	218
释放 SDK 相关资源并退出程序	219
日志函数	219
设置关节角度 (set_joint_positions)	221
设置夹爪指令 (set_gripper_command)	223
设置吸盘指令 (set_suction_cup_command)	224
停止轨迹执行 (stop_trajectory_execution)	225
设置轨迹 (execute_joint_trajectory)	226
设置关节指令 (set_joint_commands)	228
批量设置关节指令 (set_joint_commands_batch)	230
直接发布原始目标 (PublishTarget)	232
直接请求原始目标 (RequestTarget)	239
停止底盘运动 (stop_base)	244
设置底盘速度 (set_base_velocity)	245
获取关节状态 (get_joint_states)	246
获取关节位置 (get_joint_positions)	247
获取关节名称 (get_joint_names)	248
获取夹爪状态 (get_gripper_state)	249
获取吸盘状态 (get_suction_cup_state)	250
获取坐标变换 (get_transform)	251
获取 IMU 数据 (get_imu_data)	252
获取电池信息 (get_bms_information)	253

获取设备信息 (get_device_information)	254
获取相机图像数据 (get_rgb_data && get_depth_data) (get_camera_data)	256
获取传感器内外参 (get_camera_intrinsic && get_sensor_extrinsic)	258
获取雷达数据 (get_lidar_data)	261
一键回零-odom 坐标系 (whole_body_reset_zero_odom)	264
一键回零-map 坐标系 (whole_body_reset_zero_map)	265
类: GalbotMotion	266
获取实例并初始化 (get_instance && init)	266
正运动学 (使用当前状态或指定 RobotStates) (forward_kinematics)	267
获取连杆名称 (get_link_names)	269
逆运动学 (基础与基于 RobotStates) (inverse_kinematics)	270
获取与设置末端位姿(get_set_end_effort_pos)	274
单点运动规划 (关节空间与笛卡尔空间) (single_point_planning)	276
多点轨迹规划(multi_point_planning)	279
碰撞检测(collision_detection)	281
附加工具(attach_tool)	283
卸载工具(detach_tool)	284
加载/移除环境碰撞体(env_colider_operation)	285
类: GalbotNavigation	287
获取实例 / 初始化(get_instance_init)	287
重定位 / 是否已定位 / 获取当前位姿(relocation)	288
检查路径可达性并阻塞导航到目标(blocked_navigation)	289
非阻塞导航 + 轮询判断到达(non_blocking_navigation)	291
获取导航状态 + 轮询 SUCCESS/FAILED 或超时(get_navigation_status_example)	294

直线移动到目标 / 停止导航(linear_movement)	295
完整运行示例（简单流程）(complete_running_example)	298
类: GalbotPerception	300
高精度双目深度估计，单次推理并保存伪彩色深度图	300
类: Parameter	302
创建并使用 Parameter(parameter_use)	302
辅助函数	302
status_to_string / JointStates / PoseState (auxi_fun_use)	302
16. Python API 参考 - G1 机器	304
核心机器人控制	304
获取控制器权限	304
检查轨迹执行状态	305
清除末端执行器命令	305
销毁	305
执行关节轨迹	306
获取当前控制器	306
获取电池管理信息	307
获取相机内参	307
获取深度图像数据	308
获取设备信息	308
获取灵巧手状态	308
获取力传感器数据	309
获取坐标系名称	309
获取夹爪状态	310

获取 IMU 数据	310
获取关节组名称	311
获取关节名称	311
获取关节位置	311
获取关节状态	312
获取激光雷达数据	313
获取日志信息	313
获取里程计数据	313
获取 RGB 图像数据	314
获取传感器外参	314
获取吸盘状态	315
获取坐标变换	315
获取超声波数据	316
获取音量	316
获取 WBC 末端执行器位姿	316
初始化	317
检查是否运行中	317
发布目标	318
释放控制器权限	318
重新加载控制器	318
请求关机	319
请求目标	319
设置底盘姿态	319
设置底盘姿态	320

设置底盘姿态	321
设置底盘速度	321
设置灵巧手命令	322
设置末端执行器命令	323
设置夹爪命令	323
设置关节命令	324
批量设置关节命令	325
设置关节位置	326
设置吸盘命令	327
设置音量	327
启动控制器	328
启动麦克风流输入	328
停止音频流输出	329
紧急停止底盘	329
停止控制器	329
停止麦克风流输入	330
停止轨迹执行	330
切换控制器	330
等待关机	330
写入音频流输出	331
全身与底盘归零	331
全身与底盘归零	332
运动规划与执行	332
添加障碍物	333

附加目标对象	335
附加工具	336
检查碰撞	337
清空障碍物	338
分离目标对象	338
分离工具	339
正向运动学	339
基于状态的正向运动学	340
获取已构建障碍物列表	340
获取链关节状态	341
获取末端执行器姿态	341
获取指定链末端执行器姿态	342
获取雅可比矩阵	342
通过状态获取雅可比矩阵	343
获取连杆名称	344
获取运动规划配置	344
获取机器人状态	345
获取支持的链	345
获取支持的末端执行器坐标系	345
获取支持的坐标系	346
获取支持的连杆	346
获取支持的障碍物类型	346
获取支持的工具列表	346
初始化	347

逆向运动学	347
基于状态的逆向运动学	348
轨迹规划	349
多路点轨迹规划	350
多路点轨迹规划	350
移动全身关节到零位	351
移除障碍物	352
设置末端执行器姿态	352
设置运动规划配置	353
状态转字符串	354
移动导航	354
检查目标到达	355
检查路径可达性	355
获取当前姿态	356
获取导航状态	356
初始化	357
检查是否已定位	357
直线移动到目标	358
导航到目标	359
重定位	360
停止导航	360
感知模块接口	361
获取模块的最新缓存结果	361
初始化感知模块并加载模型	361

为指定模块运行一次推理	362
等待模块产生新结果或超时	362
类型与枚举	363
音频数据	363
碰撞检查选项	363
控制状态	364
深度数据	365
检测与分割结果	365
检测结果	366
灵巧手状态	367
灵巧手类型	367
力矩信息	367
错误	368
错误信息	368
力数据	368
坐标系	368
G1 控制器名称	369
G1 关节组	369
Galbot One Foxtrot 传感器类型	370
夹爪状态	370
关节组命令	371
消息头	371
IK 求解器配置	371
IMU 数据	375

关节命令	375
单关节状态	375
关节状态消息	376
关节状态	376
运动学边界	377
激光雷达数据	382
直线轨迹碰撞检测基元	383
日志级别	385
机器类型	385
运动规划配置	385
运动状态	395
导航任务状态	395
里程计数据	396
参数	396
感知模块	400
点	400
点云字段	400
点云字段数据类型	401
姿态	401
姿态状态	402
几何基元类型	402
四元数	402
RGB 数据	402
机器人状态	403

机器人状态类型	404
采样器配置	404
随机种子类型	408
传感器类型	409
SingoriX 目标	409
状态检查类型	409
吸盘动作状态	410
吸盘状态	410
目标配置	410
目标关节组轨迹	410
目标采样	411
目标任务轨迹	411
任务命令	411
终止条件类型	412
时间戳	412
轨迹	412
轨迹控制状态	413
轨迹可行性检查选项	413
轨迹规划配置	415
轨迹点	417
扭曲速度	417
超声波数据	417
超声波类型	418
三维向量	418

全身控制异常	418
力与力矩	418
17. C++ API 参考 - G1 机器人.....	421
核心机器人控制	421
析构函数	421
初始化	421
发布目标	422
请求目标	422
设置关节命令	423
设置关节位置	424
检查轨迹执行状态	425
执行关节轨迹	425
批量设置关节命令	426
设置吸盘命令	426
设置夹爪命令	427
获取夹爪状态	428
获取吸盘状态	428
获取灵巧手状态	428
设置灵巧手命令	429
获取关节位置	430
获取关节组名称	430
获取关节名称	430
获取关节状态	431
设置底盘速度	431

设置底盘姿态	432
设置底盘姿态	433
设置底盘姿态	433
设置末端执行器命令	434
清除末端执行器命令	435
获取 WBC 末端执行器位姿	435
紧急停止底盘	435
全身与底盘归零	435
全身与底盘归零	436
停止轨迹执行	437
重新加载控制器	437
切换控制器	437
获取当前控制器	438
获取控制器权限	438
释放控制器权限	439
启动控制器	439
停止控制器	439
获取 IMU 数据	440
获取里程计数据	440
获取设备信息	441
获取 RGB 图像数据	441
获取深度图像数据	441
获取激光雷达数据	442
获取超声波数据	443

获取相机内参	443
获取坐标变换	444
获取坐标系名称	444
获取传感器外参	444
获取力传感器数据	445
启动麦克风流输入	446
停止麦克风流输入	446
写入音频流输出	446
停止音频流输出	447
获取音量	447
设置音量	447
检查是否运行中	448
请求关机	448
等待关机	448
销毁	448
注册退出回调	449
获取电池管理信息	449
获取日志信息	449
获取实例	450
运动规划与执行	450
析构函数	450
初始化	450
检查是否有效	451
正向运动学	451

基于状态的正向运动学	452
逆向运动学	453
基于状态的逆向运动学	454
获取雅可比矩阵	454
通过状态获取雅可比矩阵	455
获取末端执行器姿态	456
获取指定链末端执行器姿态	457
设置末端执行器姿态	457
轨迹规划	458
单链多路点轨迹规划	459
多路点轨迹规划	460
移动全身关节到零位	461
检查碰撞	462
附加工具	462
分离工具	463
添加障碍物	464
移除障碍物	465
清空障碍物	466
附加目标对象	466
分离目标对象	468
设置运动规划配置	468
获取运动规划配置	469
获取连杆名称	469
获取支持的连杆	470

获取支持的链	470
获取支持的坐标系	470
获取支持的末端执行器坐标系	471
获取支持的工具列表	471
获取支持的障碍物类型	471
获取已构建对象列表	472
获取附加对象列表	472
检查连杆名称是否有效	472
检查链名称是否有效	473
检查坐标系名称是否有效	473
检查末端执行器坐标系是否有效	474
检查工具名称是否有效	475
检查障碍物类型是否有效	475
检查全身状态是否有效	476
检查 7 维姿态是否有效	476
检查链关节状态是否有效	477
获取机器人自由度	477
获取机器人状态	478
获取全身状态	478
获取底盘状态	478
获取链关节状态	479
获取链姿态状态	479
替换关节状态	480
状态转字符串	480

获取实例	481
移动导航	481
析构函数	481
初始化	481
重定位	482
检查是否已定位	482
获取当前姿态	483
导航到目标	483
直线移动到目标	484
停止导航	485
检查路径可达性	486
检查目标到达	487
获取导航状态	487
获取实例	488
感知模块接口	488
析构函数	488
初始化感知模块并加载模型	488
为指定模块运行一次推理	489
等待模块产生新结果或超时	489
获取模块的最新缓存结果	489
获取感知模块单例实例	490
类型与枚举	490
驱动类型	490
音频数据	490

轴对齐 2D 边界框	491
电池管理信息	492
相机信息	492
运动链类型	493
碰撞检查选项	493
控制状态	495
深度数据	495
检测与分割结果	496
检测结果	497
设备信息	500
灵巧手状态	501
灵巧手类型	501
力矩信息	501
错误	502
错误信息	502
力数据	502
坐标系	503
G1 控制器名称	503
G1 关节组	504
Galbot One Foxtrot 传感器类型	505
SDK 日志流	505
垃圾检测结果	506
夹爪状态	507
关节组命令	507

消息头	507
IK 求解器配置	508
IMU 数据	511
关节命令	512
单关节状态	512
关节状态消息	513
关节状态	513
运动学边界	514
激光雷达数据	519
直线轨迹碰撞检测基元	520
日志配置	522
日志信息	523
日志级别	523
机器类型	523
运动规划配置	524
运动状态	535
导航状态	536
导航任务状态	536
里程计数据	537
参数	537
感知模块	542
规划器配置	542
点	543
点云字段	544

姿态	545
姿态状态	546
四元数	546
参考坐标系	546
感兴趣区域	547
RGB 数据	547
机器人状态	548
采样器配置	550
随机种子类型	554
传感器状态	555
传感器类型	555
SingoriX 目标	556
吸盘动作状态	556
吸盘状态	556
目标配置	556
目标数据位	557
目标关节组轨迹	557
目标采样	557
目标任务轨迹	558
目标类型位	558
任务命令	558
时间戳	559
轨迹	559
轨迹控制状态	560

轨迹可行性检查选项	560
轨迹规划配置	562
轨迹点	564
坐标变换消息	564
扭曲速度	564
超声波数据	565
超声波类型	565
三维向量	565
力与力矩	566
18. 故障排除	567
快速定位	567
开始前先确认	568
通用检查项（建议先做）	568
控制与运动	569
发送控制指令无响应	569
机器人上运行 SDK 程序可用，但有线连接不行	569
机器人关节无法运动	570
机器人进入异常模式：定位服务异常	572
机器人进入异常模式，重启服务	573
机器人底盘无法移动	573
末端执行器控制失败	573
机器人扬声器无声音/启用音频服务 galbot_vtn	574
系统与配置	574
如何查看当前工作模式	574

如何查看机器人系统版本	576
如何检查机器人系统版本与 SDK 兼容性	577
机器人紧急开关（急停按钮）说明	578
机器人机载算力单元介绍（HPU + XCU）	580
如何进入 HPU 与 XCU	580
如何查看机器 SN 号	581
如何检查关节是否正常	581
如何启用末端实时控制指令	581
关节限位在哪里配置	582
启用/禁用某个系统服务	582
重启后 embosa IP 配置被改回去	582
相机与标定信息	583
如何修改相机分辨率	583
如何查看相机帧率	584
如何查看相机内外参和标定信息	585
日志与调试	586
底层服务日志在哪里	586
SDK log 在哪里	586
如何开启 core 文件并调试	586
在 Robot 上执行 Python 找不到 module	587
执行 Python 示例报错：numpy 没有 typeDict	587
安装与环境	587
SDK Docker 启动方式	587
如何使用感知服务（GBS 1.15.16+）	587

仍未解决?	588
19. Galbot SDK 许可证.....	589

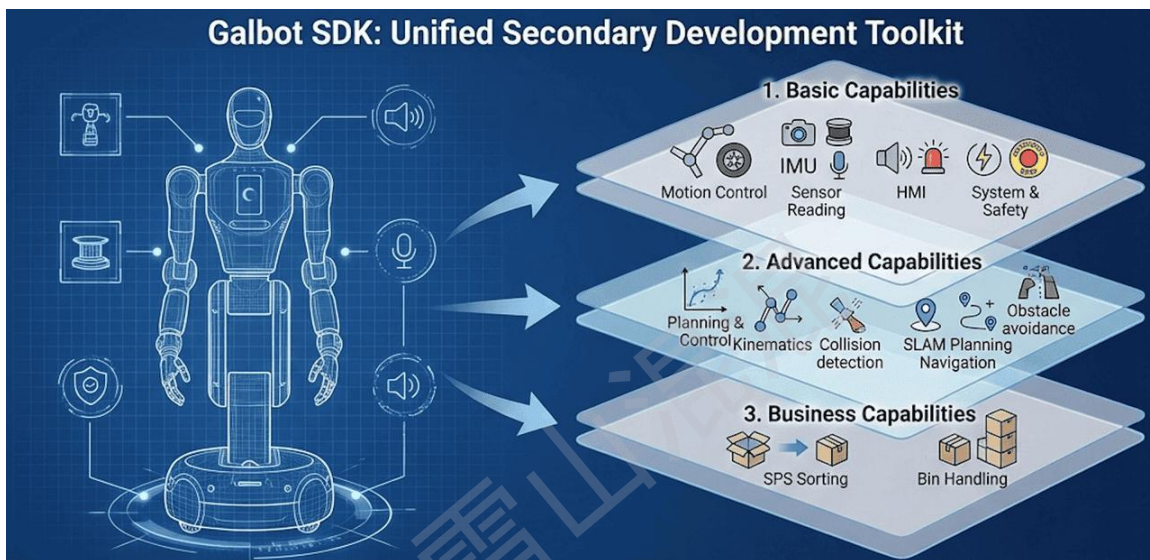
雪山混混

1. 产品概述

Galbot SDK 是专为 Galbot 系列机器人平台提供的统一二次开发工具包。通过封装底层硬件通信与算法细节，提供原子化控制与感知能力，以及可选的业务级能力。适用于工业与零售集成、学术研究等场景，帮助开发者通过简洁的 API 快速实现运动控制、感知交互与自主导航。

核心功能

Galbot SDK 采用分层解耦设计，将机器人能力划分为三大核心层次，覆盖从底层控制到高层业务的全场景开发需求。



1. 基础能力

提供硬件直接控制与原始数据访问能力，实现对机器人的操控与传感器数据获取。

- **运动控制：**支持关节空间位置/轨迹控制、末端笛卡尔空间位姿/轨迹控制，以及底盘的速度开环控制与紧急停止。
- **传感器读取：**提供彩色/深度视觉流、雷达点云、IMU 惯性数据、力传感器及超声波等全方位环境感知数据。
- **人机交互：**包含麦克风流采集、音频播放控制、RGB 指示灯效果设置。
- **系统与安全：**支持电源/日志系统监控、软件急停机机制、碰撞保护参数配置及实时资源销毁管理。

2. 高级能力

提供内置算法服务访问能力，支持自定义算法推理与执行。开发者可根据需求使用内置算法服务，或自行设计算法。

• 规划与控制接口

- 实时控制：开放高频实时关节/末端指令接口，支持自定义闭环控制算法及 VLA、RL 等推理执行。
- 正逆运动学：提供正向运动学与逆向运动学计算接口，支持约束优化。
- 运动规划：支持单链与全身多链、单路点轨迹与多路点连续轨迹生成，并具备自碰撞与环境碰撞检测功能。
- 环境管理：支持动态添加/删除障碍物，以及附加末端工具与目标物体。

• 导航接口

- 自主导航：具备 SLAM 建图定位、全局/局部路径规划及动态避障能力，支持导航任务的精准下发与状态监控。

3. 业务能力

提供场景化业务逻辑封装，包含料箱搬运、SPS 分拣等业务场景能力。用户可根据实际场景进行参数配置和流程修改，快速实现特定业务功能。该部分能力需联系商务解锁。

- **SPS 分拣模块**：针对分拣场景的自动化拾取与放置逻辑。
- **料箱搬运模块**：料箱识别、堆叠及搬运流程的快速实现。
- **更多业务模块持续开发中。**

应用场景

Galbot SDK 可根据用户实际需求，应用于包括但不限于以下场景：

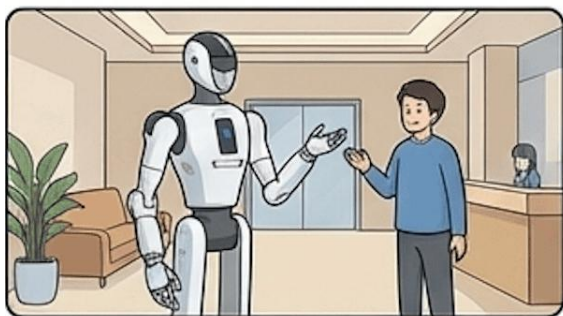
- 工业场景：物料搬运与分拣、智能仓储管理
- 零售场景：商品分拣与配送
- 服务场景：迎宾导览服务
- 研究场景：机器人算法研究、人机交互实验



Industrial Scenarios



Retail Scenarios



Service Scenarios

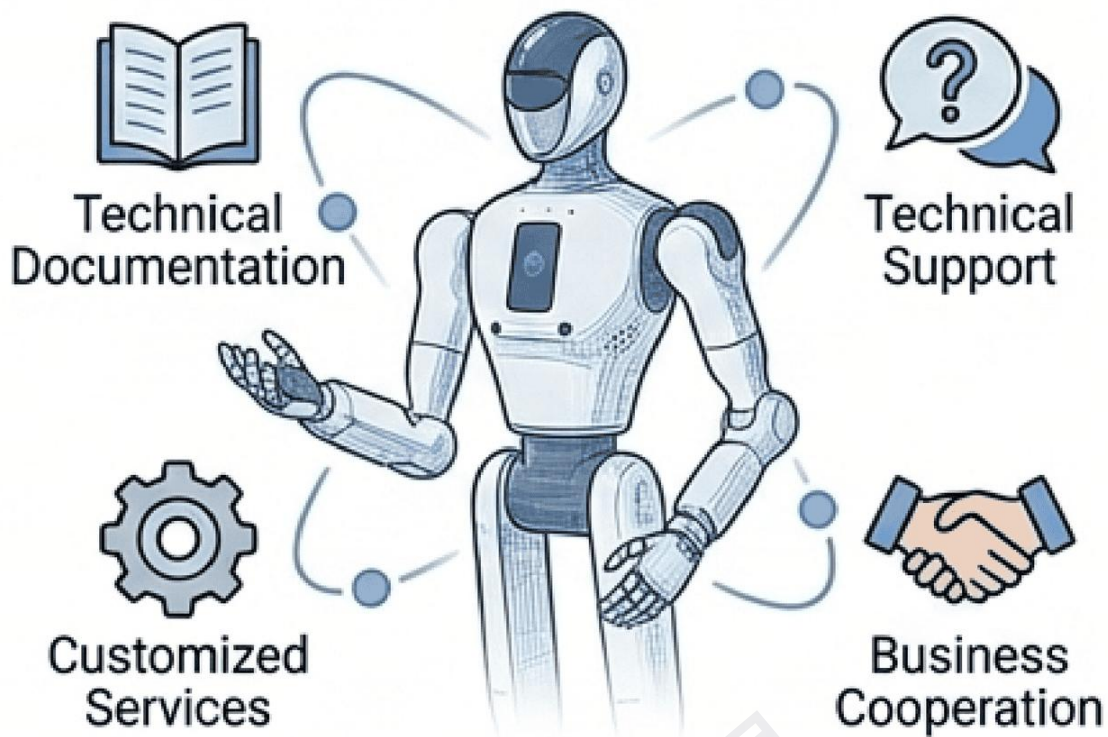


Research Scenarios

服务与支持

Galbot SDK 提供以下服务与支持：

- 技术文档：API 参考、开发示例与最佳实践
- 技术支持：技术咨询与问题解答群
- 定制服务：根据客户需求，提供 SDK 使用培训与开发指导，提供业务能力模块授权，细节请联系商务



Galbot SDK Services & Support

阅读建议

本文档中涉及 Galbot 机器人硬件术语（如 HPU、XCU）以及机器人学通用专业术语（如 Frame、IK/FK 等）。如果您是首次接触机器人开发，建议先完成 **安装与配置** 章节的部署，然后查阅其中的 **术语说明** 部分了解相关术语定义，再开始实际开发。

2. 系统要求

对象	推荐	支持范围
操作系统	Ubuntu 22.04	Ubuntu 20.04 - 24.04
架构 (PC)	x86_64	x86_64
架构 (机器人)	aarch64	aarch64
Python	3.8+	3.8 - 3.14
存储空间	16GB	3GB+

雪山混混

3. 术语说明

配置完成后可以开始探索机器人的示例和接口了，但在正式使用之前，如果不了解相关术语，可以参照下面的内容熟悉：

1. 机器人本体硬件

机器人的物理组成部分和相关硬件概念。

术语	英文	说明
XCU	X Computing Unit	基础运算单元，机器人的"小脑"，负责底层电机控制、运动控制等实时任务
HPU / Orin	High-Performance Unit / NVIDIA Jetson Orin	高算力运算单元，机器人的"大脑"，负责图像处理、AI 推理、运动规划等计算密集型任务
基座 / Base	Base / Chassis	机器人的底盘，包含移动轮子和电源系统
躯干 / Torso	Torso	机器人身体的主体部分，连接头部、手臂和基座
头部 / Head	Head	机器人的头部
手臂 / Arm	Arm	机器人的手臂，由多个连杆和关节组成，如左臂 (left_arm)、右臂 (right_arm)
腿部 / Leg	Leg	机器人的腿部结构，用于支撑身体上下运动和腰部转动
关节 / Joint	Joint	连接两个连杆的可动部分，由电机驱动
连杆 / Link	Link	连接两个关节的刚性部件
关节组 / Joint Group	Joint Group	一组相关关节的集合，如 left_arm 包含左臂所有关节，用于协同控制
末端执行器 / End-Effector	End-Effector	机械臂末端的工具，如夹爪 (Gripper)、吸盘 (Suction Cup)
自由度 / DOF	Degrees of Freedom	描述关节可独立运动的方向数量。如手臂有 7-DOF
工具中心点 / TCP	Tool Center Point	末端工具的工作点，如夹爪的中心点，用于精确控制操作位置
传感器 / Sensor	Sensor	感知环境的设备，包括相机、LiDAR、IMU、力传感器等
RGB 相机	RGB Camera	彩色相机，捕捉可见光图像
深度相机	Depth Camera	可输出深度图（每个像素的距离），用于三维感知

LiDAR / 激光雷达	Light Detection and Ranging	通过激光测距生成周围环境的点云地图
IMU / 惯性测量单元	Inertial Measurement Unit	包含加速度计和陀螺仪，测量机器人的加速度、角速度和姿态
点云 / Point Cloud	Point Cloud	三维空间中大量点的集合，每个点有 (x,y,z) 坐标，由 LiDAR 或深度相机生成

2. 机器人学基础

描述机器人运动和空间关系的核心理论概念。

术语	英文	说明
坐标系 / Frame	Reference Frame	描述位置和姿态的参考坐标系。想象成固定在空间某处的"标尺"
世界坐标系 / World	World Frame	全局固定的坐标系，通常以地图原点或初始位置为基准
基座坐标系 / Base	Base Frame	固定在机器人基座上的坐标系，随机器人移动而移动
末端坐标系	End-Effector Frame	固定在机械臂末端的坐标系，随手臂运动而运动
关节空间	Joint Space	由各个关节角度定义的空间，是机器人"原生"的控制方式
笛卡尔空间	Cartesian Space	用 (x, y, z) 坐标描述的三维空间，人类更直观的理解方式
位姿 / Pose	Pose	位置 (Position) + 姿态 (Orientation) 的完整描述，即"在哪里"+"朝哪边"
位置 / Position	Position	物体在 3D 空间中的坐标 (x, y, z)，单位通常是米 (m)
姿态 / Orientation	Orientation	物体的旋转状态，描述朝向哪个方向
四元数 / Quaternion	Quaternion	用 4 个数 (qx, qy, qz, qw) 表示旋转，避免万向锁问题，适合计算机处理
欧拉角 / Euler Angles	Euler Angles	用三个角度 (roll, pitch, yaw) 即 (横滚, 俯仰, 偏航) 表示旋转，更直观
正运动学 / FK	Forward Kinematics	已知关节角度，计算末端位姿（手在哪里）— 有唯一解
逆运动学 / IK	Inverse Kinematics	已知目标位姿，计算所需关节角度（手要到那里，关节怎么转）— 可能多解或无解
轨迹 / Trajectory	Trajectory	机器人从起点到终点的完整运动路径，包含每个时刻的位置、速度、加速度
路点 / Waypoint	Waypoint	轨迹中的关键节点，机器人会依次经

		过这些点，类似导航的“途经点”
插值 / Interpolation	Interpolation	在路点之间计算平滑过渡，生成连续的运动轨迹
SLAM	Simultaneous Localization and Mapping	同时定位与建图：机器人一边移动一边建地图，同时确定自己在地图中的位置
自碰撞	Self-Collision	机器人不同部位之间发生碰撞，如左手碰到右手
环境碰撞	Environment Collision	机器人与周围障碍物发生碰撞

3. 控制与接口

SDK 中使用的常见参数和概念说明：

术语	英文/参数名	说明
SDK	Software Development Kit	软件开发工具包，包含库、工具和文档
API	Application Programming Interface	应用程序接口，定义程序之间交互的规则
SSH	Secure Shell	远程登录协议，用于从 PC 连接到机器人
交叉编译	Cross Compilation	在 PC (x86) 上编译出机器人 (ARM) 可运行的程序
单例模式 / Singleton	Singleton	设计模式，确保全局只有一个机器人实例，通过 <code>get_instance()</code> 获取
阻塞模式 / Blocking	<code>is_blocking=True</code>	函数调用后等待操作完成才返回。例如： <code>navigate_to_goal(..., is_blocking=True)</code> 会等待导航完成才返回，期间无法执行其他操作
非阻塞模式 / Non-blocking	<code>is_blocking=False</code>	函数调用后立即返回，不等待操作完成。例如： <code>navigate_to_goal(..., is_blocking=False)</code> 立即返回，可通过 <code>is_localized()</code> 等函数查询状态
<code>base_pose</code>	<code>base_pose: [x, y, yaw]</code>	机器人基座的位姿，包含 (x, y) 位置（米）和 yaw 偏航角（弧度）
<code>frame_id</code>	<code>frame_id</code>	目标坐标系 ID，可选值："base_link"（机器人基座）、"odom"（里程计）、"map"（地图）
<code>reference_frame_id</code>	<code>reference_frame_id</code>	参考坐标系 ID，指定位姿所属的坐标系
<code>goal_pose</code>	<code>goal_pose: [x, y, z, qx, qy, qz, qw]</code>	目标位姿，包含位置 (x, y, z)（米）和四元数 (qx, qy, qz, qw) 表示 Orientation
<code>start_pose</code>	<code>start_pose</code>	起始位姿，格式同 <code>goal_pose</code>

init_pose	init_pose	初始位姿估计，用于重定位
reference_frame	reference_frame	相对坐标系名称，如 "base_link"、 "world"、运动链名称
target_frame	target_frame	目标坐标系，用于坐标变换
base_link	base_link	机器人基座坐标系，所有机械臂/头部等部件都以此为参考
odom	odom	里程计坐标系，基于轮式编码器的相对位姿估计
map	map	地图坐标系，SLAM 建立的全局一致坐标系
joint_groups	joint_groups	关节组名称列表，如 ["left_arm", "right_arm", "chassis"]
joint_names	joint_names	具体关节名列表，优先于 joint_groups，如 ["left_arm_j1", "left_arm_j2"]
joint_positions	joint_positions	关节位置数组（弧度），顺序与 get_joint_names() 返回一致
waypoints	waypoints	轨迹路点列表，每个路点包含目标位置 和到达时间
time_from_start_s	time_from_start_s	从轨迹开始到该路点的时间（秒）， 用于控制运动节奏
timeout_s	timeout_s	超时时间（秒），阻塞模式下等待操作 完成的最大时间
timestamp_ns	timestamp_ns	纳秒级时间戳，用于数据同步
safe_margin	safe_margin	碰撞检测安全距离（米），如 0.1 表示 与障碍物保持 10cm
enable_collision_check	enable_collision_check	是否启用碰撞检测，True 表示启用
obstacle_id	obstacle_id	障碍物唯一标识符，添加/删除障碍物时 使用
ignore_collision_link_names	ignore_collision_link_names	碰撞检测忽略的连杆列表
pose	pose	位姿数据，长度 7 的数组 [x, y, z, qx, qy, qz, qw]
reference_base_pose	reference_base_pose	机器人基座在地图坐标系中的位姿
linear_velocity	linear_velocity: [vx, vy, vz]	线速度数组（米/秒），vx 前进速度， vy 侧向速度，vz 垂直速度
angular_velocity	angular_velocity: [wx, wy, wz]	角速度数组（弧度/秒），wz 为偏航角 速度
status_string	status_string	状态描述字符串，如 "SUCCESS"、 "FAIL"、"TIMEOUT"、"MOVING"
is_localized	is_localized()	导航状态检查函数，返回 True 表示 机器人已在地图中定位
get_frame_names()	get_frame_names()	获取所有可用坐标系名称列表
quaternion	[qx, qy, qz, qw]	四元数表示旋转，单位四元数满足 $\sqrt{qx^2+qy^2+qz^2+qw^2} = 1$

euler	[roll, pitch, yaw]	欧拉角表示旋转，单位弧度，顺序为 横滚-俯仰-偏航
-------	--------------------	------------------------------

雪山混混

4. 版本兼容性检查

SDK 版本：安装在开发电脑或机器人上的 Galbot SDK 软件包版本。

机器人 GBS 版本：运行在 Galbot 机器人硬件上的系统固件版本。这两个版本必须兼容才能确保正常运行。

快速使用

运行兼容性检查工具验证 SDK 版本与机器人 GBS 版本是否匹配：

```
galbot_sdk check-version
```

该工具会自动检测两个版本并指示是否兼容。如果发现不匹配，会提供升级或降级建议。

更多选项请运行 `galbot_sdk check-version --help`。

雪山混混

5. 环境部署

请选择与当前开发场景匹配的部署模式。每种模式的配置步骤互相独立，进入对应页面后只会看到该模式的部署内容。

雪山混沌

6. 机器人端部署

适用场景： 生产环境部署，实时性要求高的应用

特点： 无网络延迟，通信开销最小

1. 连接机器人网络

1.1 开机后在网络选择界面连接 WiFi

雪山混混

马帆 7263

Wi-Fi

Network

马帆 7263



GalaxyGen



User Password

☐ Show Password

马帆 7263

Connect to Wi-Fi

马帆 7263

马帆 7263

马帆 7263

Wi-Fi

Network

马帆 7263



GalaxyGen



GalaxyGen66666



GalaxyGen77777



GalaxyGen66666



GalaxyGen77777



GalaxyGen77777



GalaxyGen77777



GalaxyGen77777



Get More

马帆 7263

1.2 在 Network 界面查看机器人 IP 地址

雪山混混

Wi-Fi

Network

XCU:

wlan0: inet 192.168.38.66 netmask 255.255.252.0 broadcast 192.168.39.255

eth0: inet 192.168.1.66 netmask 255.255.255.0 broadcast 192.168.1.255

eth1: inet 192.168.100.66 netmask 255.255.255.0 broadcast 192.168.100.255

eth2: inet 192.168.200.66 netmask 255.255.255.0 broadcast 192.168.200.255

orin:

wlan0: inet 192.168.38.67 netmask 255.255.252.0 broadcast 192.168.39.255

etnu: inet 192.168.100.88 netmask 255.255.255.0 broadcast 192.168.100.255

eth1: inet 192.168.1.88 netmask 255.255.255.0 broadcast 192.168.1.255

当前场景

base

切换

左臂工具

HUILING_GRAPPER

右臂工具

GSUCKER

左臂关节锁定

☐ 1关节

☐ 2关节

☐ 3关节

☐ 4关节

☐ 5关节

☐ 6关节

☐ 7关节

右臂关节锁定

☐ 1关节

☐ 2关节

☐ 3关节

☐ 4关节

☐ 5关节

☐ 6关节

☐ 7关节

腿部关节锁定

☐ 1关节

☐ 2关节

☐ 3关节

☐ 4关节

☐ 5关节

修改

全部模块

仅上传此次日志

日志大小:63.15 MB

预估压缩后大小:6.31 MB

上传日志

信息

此 IP 为后续部署 SDK 到机器人时需要输入的 IP 地址，外部也可通过 ssh 登录此 IP 在机器人上运行程序。同时，在进行 SDK 与机器人系统版本兼容性检查时，也可以通过该 IP 指定目标机器人：

```
galbot_sdk check-version [--robot-ip <IP>]
```

2. 部署 SDK 到机器人

2.1 在 PC 上执行部署脚本

```
cd GalbotSDK
./deploy_to_robot.sh
```

2.2 根据提示输入连接信息

- 用户名：galbot
- Orin IP 地址：例如 192.168.120.180
- 密码：gb@2023

success

动态库自动安装到：/data/galbot/lib

2.3 验证安装

```
ssh galbot@<机器人 IP>
ls /data/galbot/lib | grep galbot_sdk
```

success

显示 libgalbot_sdk.so 和 libgalbot_sdk.so.1.7.0

2.4 环境变量配置

注意

使用 deploy_to_robot.sh 部署 SDK 时，安装脚本已自动配置环境变量并写入 ~/.bashrc，重新登录或运行 source ~/.bashrc 即可生效。

如果运行程序时找不到 galbot_sdk 库，请执行：

```
export LD_LIBRARY_PATH=/data/galbot/lib:$LD_LIBRARY_PATH
export PYTHONPATH=/data/galbot/lib:$PYTHONPATH
export PATH=/data/galbot/lib/galbot_sdk/bin:$PATH
echo 'export LD_LIBRARY_PATH=/data/galbot/lib:$LD_LIBRARY_PATH' >> ~/.bashrc
echo 'export PYTHONPATH=/data/galbot/lib:$PYTHONPATH' >> ~/.bashrc
echo 'export PATH=/data/galbot/lib/galbot_sdk/bin:$PATH' >> ~/.bashrc
```

3. 运行程序

3.1 Python 程序

3.1.1 传输 python 源码到机器人的 Orin 上

```
scp your_app.py galbot@<机器人 IP>:/userdata
```

3.1.2 运行 PYTHON 程序

```
python your_app.py
```

3.2 C++ 程序

3.2.1 在 PC 上安装 SDK

进入到 SDK 目录下执行安装脚本

```
cd GalbotSDK  
./install.sh
```

3.2.2 在 PC 上交叉编译 (aarch64)

```
cd examples/cpp/  
mkdir -p build  
cd build  
cmake ../ -DCMAKE_TOOLCHAIN_FILE=../cmake/linux-aarch64-gcc940.cmake  
make
```

3.2.3 传输可执行文件到机器人的 Orin 上

```
scp your_app galbot@<机器人 IP>:/userdata
```

3.2.4 运行程序

```
ssh galbot@<机器人 IP>  
cd /userdata  
./your_app
```

7. PC 端部署

适用场景： 开发调试，快速迭代

特点： 开发便捷，支持局域网远程控制

根据您运行的 PC 环境进行安装部署选择。

雪山混混

8. PC 端部署 - Ubuntu

适用场景： 开发调试，快速迭代

特点： 开发便捷，支持局域网远程控制

1. 物理连接

PC 通过以太网转接板与机器人相连。转接板用于把标准 RJ45 网线接口转换为机器人专用的以太网接口，使用步骤如下：

1.1 将网线一端插入转接板的 RJ45 接口，另一端连接到 PC 网口

1.2 将转接板的另一端插入机器人背部对应的以太网接口

雪山混混



连接完成后整体走线如下图所示：



2. 配置网络

2.1 配置 PC 网口 IP

- IP 地址：192.168.1.99（或同网段其他地址）
- 子网掩码：255.255.255.0

设置路径：**设置** → **网络** → **有线设置** → **IPv4** → **手动**

取消(C)

有线

应用(A)

详细信息

身份

IPv4

IPv6

安全

IPv4 方式

☐ 自动 (DHCP)

☐ 仅本地链路

☒ 手动

☐ 禁用

☐ 与其他计算机共享

地址

地址	子网掩码	网关
192.168.1.99	255.255.255.0	

验证配置：

```
ifconfig
```

success

显示 inet 192.168.1.99

```
fanma@fanma-OMEN-Gaming-Laptop-16-ae0xxx:~/桌面$ ifconfig
docker0: flags=4099<UP,BROADCAST,MULTICAST> mtu 1500
    inet 172.17.0.1 netmask 255.255.0.0 broadcast 172.17.255.255
    ether 02:42:9b:cc:10:79 txqueuelen 0 (以太网)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

eno1: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.1.99 netmask 255.255.255.0 broadcast 192.168.1.255
    inet6 fe80::1a16:2c00:ea65:688a prefixlen 64 scopeid 0x20<link>
    ether 6c:0b:5e:a3:5a:15 txqueuelen 1000 (以太网)
    RX packets 64 bytes 4452 (4.4 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 55 bytes 9754 (9.7 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (本地环回)
    RX packets 1850119 bytes 597084135 (597.0 MB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 1850119 bytes 597084135 (597.0 MB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

wlo1: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.97.46 netmask 255.255.252.0 broadcast 192.168.99.255
    inet6 fe80::c7f7:c33b:839:a745 prefixlen 64 scopeid 0x20<link>
    ether ec:4c:8c:19:84:f3 txqueuelen 1000 (以太网)
    RX packets 5283247 bytes 3772073591 (3.7 GB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 5591553 bytes 9218841756 (9.2 GB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

2.2 配置 PC 端 IP 配置文件

文件路径：/data/config/embosa_ip_config.json

配置示例（假设 PC: 192.168.1.99, XCU: 192.168.1.66, Orin: 192.168.1.88）：

```
{
  "embosa_ip": {
    "local_interface": [
      "192.168.1.99"
    ],
    "peer_lists": [
      "192.168.1.66",
      "192.168.1.88"
    ]
  }
}
```

2.3 配置 Orin 端 IP 配置文件

警告

需先通过 WiFi 连接 Orin，参考机器人端部署步骤 1

2.3.1 登录 Orin

```
ssh galbot@<Orin 无线 IP>
# 密码: gb@2023
```

2.3.2 编辑配置文件

```
vi /data/config/embosa_ip_config.json
```

2.3.3 配置内容

危险

192.168.100.88 和 192.168.100.66 是 Orin 与 XCU 内部直连 IP，必须保留

```
{
  "embosa_ip": {
    "local_interface": [
      "192.168.100.88",
      "192.168.1.88"
    ],
    "peer_lists": [
      "192.168.100.66",
      "192.168.1.99"
    ]
  }
}
```

2.4 配置 XCU 端 IP 配置文件

2.4.1 登录 XCU

```
ssh root@<XCU 无线 IP>
# 密码: 12345678
```

2.4.2 编辑配置文件

```
vi /data/config/embosa_ip_config.json
```

2.4.3 配置内容

```
{
  "embosa_ip": {
    "local_interface": [
      "192.168.100.66",
      "192.168.1.66"
    ],
    "peer_lists": [
      "192.168.100.88",
      "192.168.1.99"
    ]
  }
}
```

2.5 验证网络连接

警告

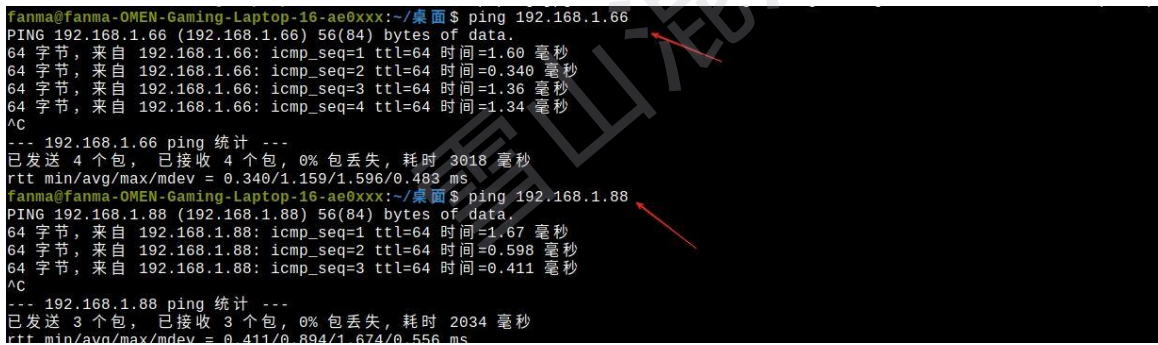
配置修改后必须重启机器人

重启后测试连接：

```
ping 192.168.1.66 # 测试 XCU
ping 192.168.1.88 # 测试 Orin
```

success

正常返回 ping 响应



```
fanma@fanma-OMEN-Gaming-Laptop-16-ae0xxx:~/桌面$ ping 192.168.1.66
PING 192.168.1.66 (192.168.1.66) 56(84) bytes of data.
64 字节，来自 192.168.1.66: icmp_seq=1 ttl=64 时间=1.60 毫秒
64 字节，来自 192.168.1.66: icmp_seq=2 ttl=64 时间=0.340 毫秒
64 字节，来自 192.168.1.66: icmp_seq=3 ttl=64 时间=1.36 毫秒
64 字节，来自 192.168.1.66: icmp_seq=4 ttl=64 时间=1.34 毫秒
^C
--- 192.168.1.66 ping 统计 ---
已发送 4 个包，已接收 4 个包，0% 包丢失，耗时 3018 毫秒
rtt min/avg/max/mdev = 0.340/1.159/1.596/0.483 ms
fanma@fanma-OMEN-Gaming-Laptop-16-ae0xxx:~/桌面$ ping 192.168.1.88
PING 192.168.1.88 (192.168.1.88) 56(84) bytes of data.
64 字节，来自 192.168.1.88: icmp_seq=1 ttl=64 时间=1.67 毫秒
64 字节，来自 192.168.1.88: icmp_seq=2 ttl=64 时间=0.598 毫秒
64 字节，来自 192.168.1.88: icmp_seq=3 ttl=64 时间=0.411 毫秒
^C
--- 192.168.1.88 ping 统计 ---
已发送 3 个包，已接收 3 个包，0% 包丢失，耗时 2034 毫秒
rtt min/avg/max/mdev = 0.411/0.894/1.674/0.556 ms
```

3. 在 PC 上安装 SDK

进入到 SDK 目录下执行安装脚本

```
cd GalbotSDK
./install.sh
```

注意

安装脚本已自动在 PC 端配置环境变量，将以下内容写入 ~/.bashrc：

```
source /opt/galbot/galbot_sdk/linux-x86_64-gcc940/setup.sh
```

重新登录或运行 source ~/.bashrc 即可生效。如果找不到 galbot_sdk 库，请重新执行安装脚本或检查 ~/.bashrc 中的配置。

4. 编译程序

在 PC 上编译 (x86_64)

```
cd examples/cpp/  
mkdir -p build  
cd build  
cmake ../ -DCMAKE_TOOLCHAIN_FILE=../cmake/linux-x86_64-gcc940.cmake  
make
```

5. 运行程序

5.1 运行 C++ 程序

```
cd /userdata # 或程序所在目录  
./your_app
```

5.2 运行 Python 程序

```
python your_app.py
```

提示

在运行 python example 时依赖一些库，所以请先运行下面的脚本保证相关库已安装：

```
cd GalbotSDK  
./install_python_deps.sh
```

9. PC 端部署 - WSL2

适用场景： 开发调试，快速迭代

特点： 开发便捷，支持局域网远程控制

1. 物理连接

PC 通过以太网转接板与机器人相连。转接板用于把标准 RJ45 网线接口转换为机器人专用的以太网接口，使用步骤如下：

- 1.1 将网线一端插入转接板的 RJ45 接口，另一端连接到 PC 网口
- 1.2 将转接板的另一端插入机器人背部对应的以太网接口

雪山混混



连接完成后整体走线如下图所示：

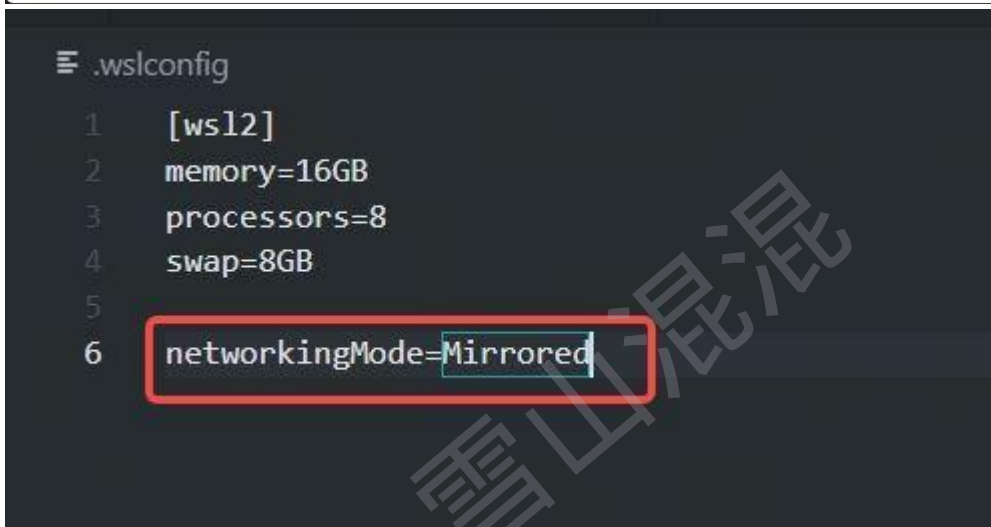
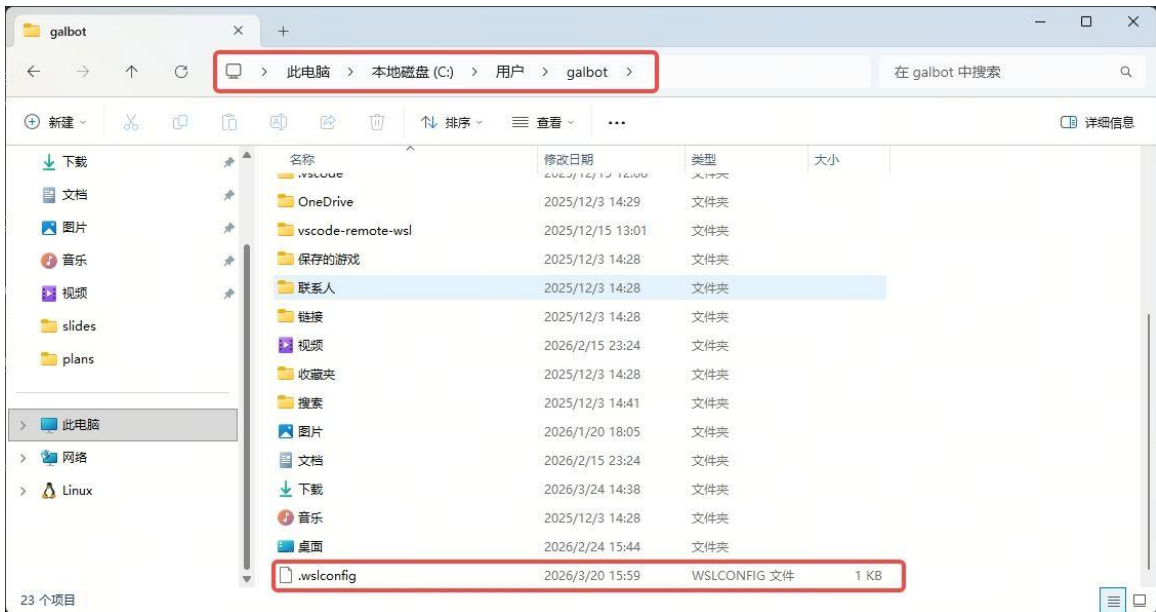


2. 配置网络

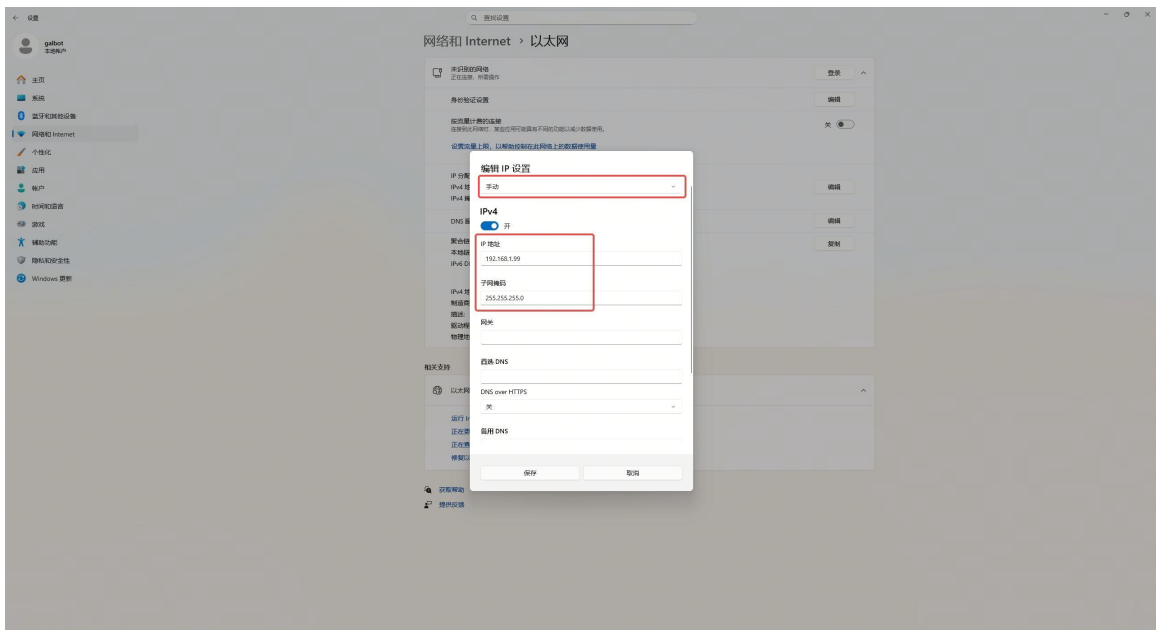
2.1 配置 PC 网口 IP

- IP 地址：192.168.1.99（或同网段其他地址）
- 子网掩码：255.255.255.0

使用 WSL2 时，需要在 Windows 系统的当前用户目录下创建.wslconfig 文件，并新增 networkingMode=Mirrored，示例：



IP 在 Windows 系统进行设置



验证配置：

ifconfig

success

显示 inet 192.168.1.99

```
fanma@fanma-OMEN-Gaming-Laptop-16-ae0xxx:~/桌面$ ifconfig
docker0: flags=4099<UP,BROADCAST,MULTICAST> mtu 1500
    inet 172.17.0.1 netmask 255.255.0.0 broadcast 172.17.255.255
    ether 02:42:9b:cc:10:79 txqueuelen 0 (以太网)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

eno1: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.1.99 netmask 255.255.255.0 broadcast 192.168.1.255
    inet6 fe80::1a16:2c00:ea65:688a prefixlen 64 scopeid 0x20<link>
    ether 6c:0b:5e:a3:5a:15 txqueuelen 1000 (以太网)
    RX packets 64 bytes 4452 (4.4 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 55 bytes 9754 (9.7 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (本地环回)
    RX packets 1850119 bytes 597084135 (597.0 MB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 1850119 bytes 597084135 (597.0 MB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

wlo1: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.97.46 netmask 255.255.252.0 broadcast 192.168.99.255
    inet6 fe80::c7f7:c33b:839:a745 prefixlen 64 scopeid 0x20<link>
    ether ec:4c:8c:19:84:f3 txqueuelen 1000 (以太网)
    RX packets 5283247 bytes 3772073591 (3.7 GB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 5591553 bytes 9218841756 (9.2 GB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

警告

如果测试 SDK 接口失败，请在 **管理员 PowerShell** 执行：

```
Set-NetFirewallProfile -Profile Domain,Private,Public -Enabled False
```

联调完成后建议立即恢复：

```
Set-NetFirewallProfile -Profile Domain,Private,Public -Enabled True
```

2.2 配置 PC 端 IP 配置文件

文件路径：/data/config/embosa_ip_config.json

配置示例（假设 PC: 192.168.1.99, XCU: 192.168.1.66, Orin: 192.168.1.88）：

```
{
  "embosa_ip": {
    "local_interface": [
      "192.168.1.99"
    ],
    "peer_lists": [
      "192.168.1.66",
      "192.168.1.88"
    ]
  }
}
```

2.3 配置 Orin 端 IP 配置文件

警告

需先通过 WiFi 连接 Orin，参考机器人端部署步骤 1

2.3.1 登录 Orin

```
ssh galbot@<Orin 无线 IP>  
# 密码: gb@2023
```

2.3.2 编辑配置文件

```
vi /data/config/embosa_ip_config.json
```

2.3.3 配置内容

危险

192.168.100.88 和 192.168.100.66 是 Orin 与 XCU 内部直连 IP，必须保留

```
{  
  "embosa_ip": {  
    "local_interface": [  
      "192.168.100.88",  
      "192.168.1.88"  
    ],  
    "peer_lists": [  
      "192.168.100.66",  
      "192.168.1.99"  
    ]  
  }  
}
```

2.4 配置 XCU 端 IP 配置文件

2.4.1 登录 XCU

```
ssh root@<XCU 无线 IP>  
# 密码: 12345678
```

2.4.2 编辑配置文件

```
vi /data/config/embosa_ip_config.json
```

2.4.3 配置内容

```
{  
  "embosa_ip": {  
    "local_interface": [  
      "192.168.100.66",  
      "192.168.1.66"  
    ],  
    "peer_lists": [  
      "192.168.100.88",  
      "192.168.1.99"  
    ]  
  }  
}
```

2.5 验证网络连接

警告

配置修改后必须重启机器人

重启后测试连接：

```
ping 192.168.1.66 # 测试 XCU
ping 192.168.1.88 # 测试 Orin
```

success

正常返回 ping 响应

```
fanma@fanma-OMEN-Gaming-Laptop-16-ae0xxx:~/桌面$ ping 192.168.1.66
PING 192.168.1.66 (192.168.1.66) 56(84) bytes of data.
64 字节, 来自 192.168.1.66: icmp_seq=1 ttl=64 时间=1.60 毫秒
64 字节, 来自 192.168.1.66: icmp_seq=2 ttl=64 时间=0.340 毫秒
64 字节, 来自 192.168.1.66: icmp_seq=3 ttl=64 时间=1.36 毫秒
64 字节, 来自 192.168.1.66: icmp_seq=4 ttl=64 时间=1.34 毫秒
^C
--- 192.168.1.66 ping 统计 ---
已发送 4 个包, 已接收 4 个包, 0% 包丢失, 耗时 3018 毫秒
rtt min/avg/max/mdev = 0.340/1.159/1.596/0.483 ms
fanma@fanma-OMEN-Gaming-Laptop-16-ae0xxx:~/桌面$ ping 192.168.1.88
PING 192.168.1.88 (192.168.1.88) 56(84) bytes of data.
64 字节, 来自 192.168.1.88: icmp_seq=1 ttl=64 时间=1.67 毫秒
64 字节, 来自 192.168.1.88: icmp_seq=2 ttl=64 时间=0.598 毫秒
64 字节, 来自 192.168.1.88: icmp_seq=3 ttl=64 时间=0.411 毫秒
^C
--- 192.168.1.88 ping 统计 ---
已发送 3 个包, 已接收 3 个包, 0% 包丢失, 耗时 2034 毫秒
rtt min/avg/max/mdev = 0.411/0.894/1.674/0.556 ms
```

3. 在 PC 上安装 SDK

进入到 SDK 目录下执行安装脚本

```
cd GalbotSDK
./install.sh
```

4. 编译程序

在 PC 上编译 (x86_64)

```
cd examples/cpp/
mkdir -p build
cd build
cmake ../ -DCMAKE_TOOLCHAIN_FILE=../cmake/linux-x86_64-gcc940.cmake
make
```

5. 运行程序

5.1 配置环境变量

提示

/opt/galbot/为默认安装路径，可根据自己的安装路径进行修改

```
source /opt/galbot/galbot_sdk/linux-x86_64-gcc940/setup.sh
```

提示

```
echo 'source /opt/galbot/galbot_sdk/linux-x86_64-gcc940/setup.sh' >> ~/.bashrc
source ~/.bashrc
```

5.2 运行 C++ 程序

```
cd /userdata # 或程序所在目录
./your_app
```

5.3 运行 Python 程序

```
python your_app.py
```

■ 提示

在运行 python example 时依赖一些库，所以请先运行下面的脚本保证相关库已安装：

```
cd GalbotSDK
./install_python_deps.sh
```

雪山混混

10. PC 端部署 - VMWare

适用场景： 开发调试，快速迭代

特点： 开发便捷，支持局域网远程控制

1. 物理连接

PC 通过以太网转接板与机器人相连。转接板用于把标准 RJ45 网线接口转换为机器人专用的以太网接口，使用步骤如下：

- 1.1 将网线一端插入转接板的 RJ45 接口，另一端连接到 PC 网口
- 1.2 将转接板的另一端插入机器人背部对应的以太网接口

雪山混混



连接完成后整体走线如下图所示：



2. 配置网络

2.1 配置 PC 网口 IP

- IP 地址：192.168.1.99（或同网段其他地址）
- 子网掩码：255.255.255.0

设置 WINDOWS 有线网卡 IP 地址，和机器人同一个网段，例如:192.168.100.100

编辑 IP 设置

手动



IPv4



开

IP 地址

192.168.1.100

子网掩码

255.255.255.0

网关

首选 DNS

设置 VMWare WorkStation VMnet0 网卡桥接到 Windows 系统的有线网卡，设置路径：**编辑** → **虚拟网络编辑器**

虚拟网络编辑器

✕

名称	类型	外部连接	主机连接	DHCP	子网地址
VMnet0	自定义...	-	-	-	192.168.106.0
VMnet8	NAT 模式	NAT 模式	已连接	已启用	192.168.60.0

添加网络(E)...

移除网络(Q)

重命名网络(W)...

VMnet 信息

☐ 桥接模式(将虚拟机直接连接到外部网络)(B)

已桥接至(G):

▼

自动设置(U)...

☐ NAT 模式(与虚拟机共享主机的 IP 地址)(N)

NAT 设置(S)...

☒ 仅主机模式(在专用网络内连接虚拟机)(H)

☐ 将主机虚拟适配器连接到此网络(V)

主机虚拟适配器名称: VMware 网络适配器 VMnet0

☐ 使用本地 DHCP 服务将 IP 地址分配给虚拟机(D)

DHCP 设置(P)...

子网 IP (I):

192 . 168 . 106 . 0

子网掩码(M):

255 . 255 . 255 . 0

⚠ 需要具备管理员特权才能修改网络配置。

更改设置(C)

还原默认设置(R)

导入(I)...

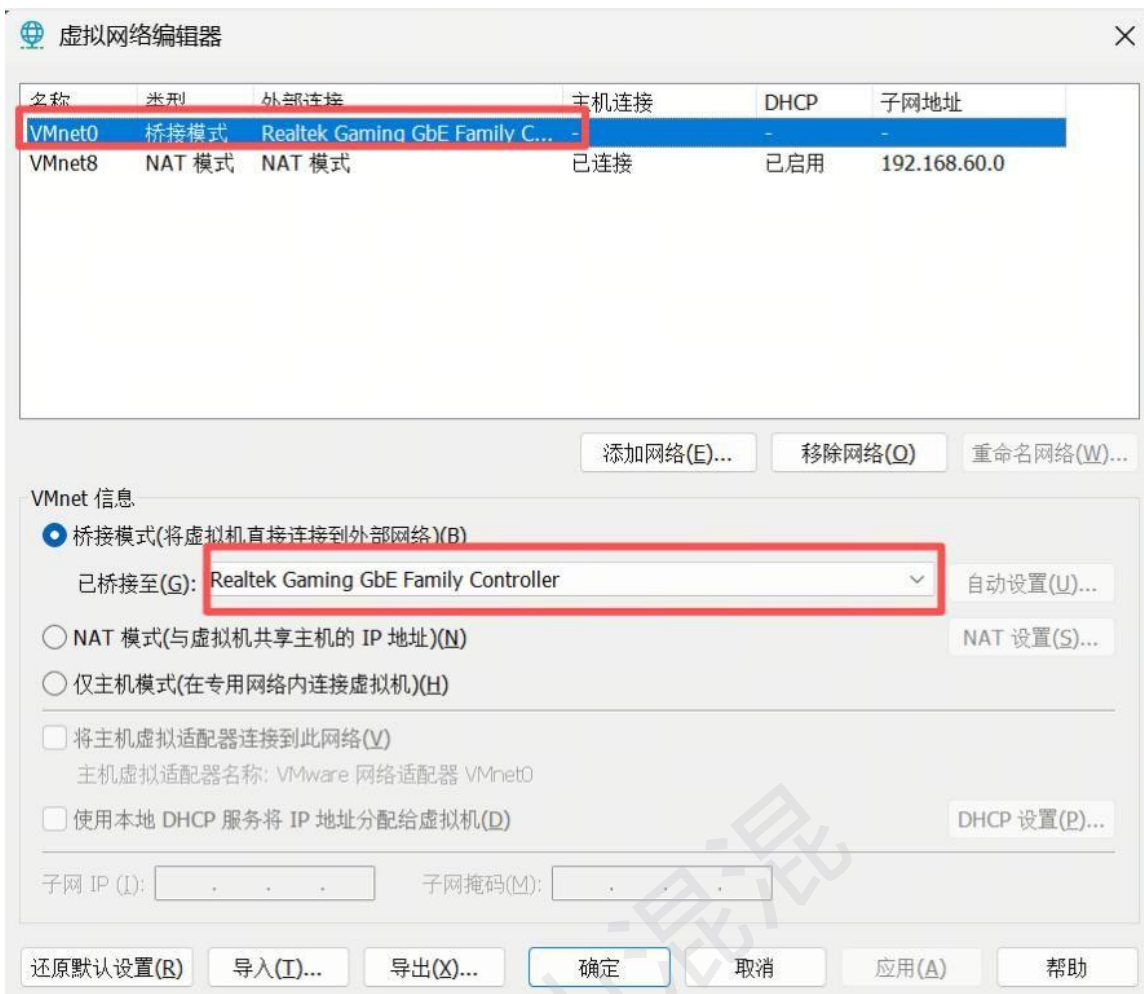
导出(X)...

确定

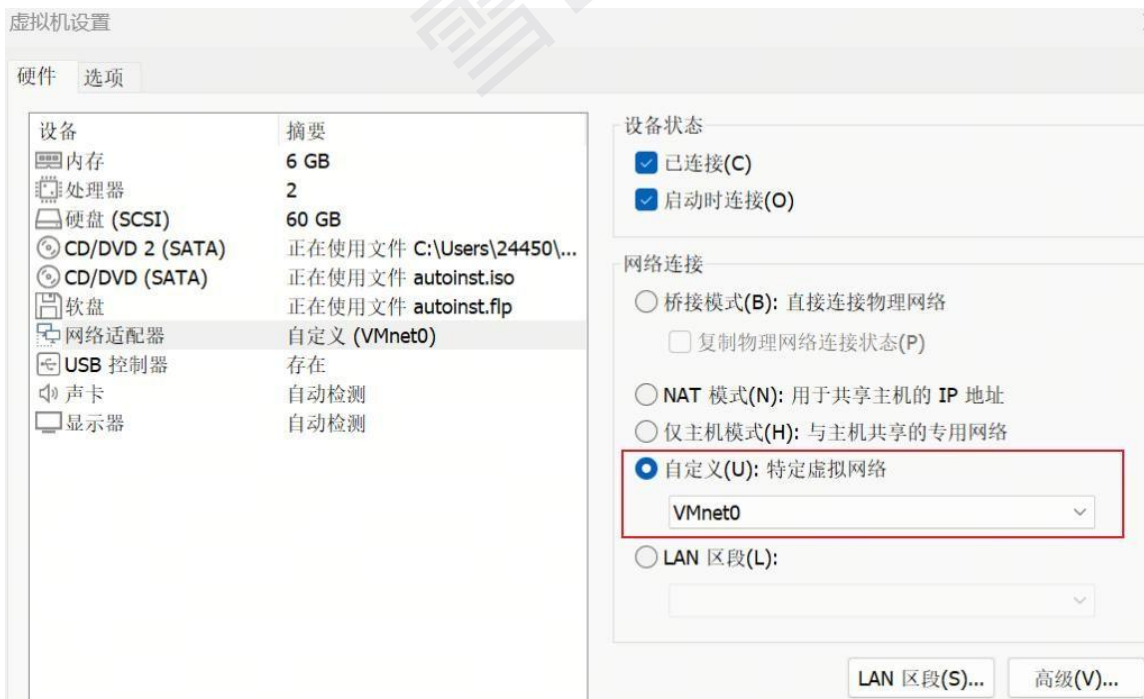
取消

应用(A)

帮助



设置 VMWare WorkStation 网络适配器，设置路径：虚拟机 → 设置



设置 ubuntu 的网口 IP

Cancel
Wired
Apply

Details
Identity
IPv4
IPv6
Security

IPv4 Method

☐ Automatic (DHCP)
☐ Link-Local Only

☒ Manual
☐ Disable

☐ Shared to other computers

Addresses

Address	Netmask	Gateway	
192.168.1.99	255.255.255.0		

DNS

Automatic ☒

提示

验证配置

ubuntu ip 地址为 192.168.1.99,并且 windows 系统可以 ping 通机器人以及虚拟机中的 ubuntu 系统

```

ens33: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.1.99 netmask 255.255.255.0 broadcast 192.168.1.255
    inet6 fe80::1dfc:26cf:359:8a55 prefixlen 64 scopeid 0x20<link>
    ether 00:0c:29:14:cc:8b txqueuelen 1000 (Ethernet)
    RX packets 1404907 bytes 895600486 (895.6 MB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 6059 bytes 3310225 (3.3 MB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 14886 bytes 1060591 (1.0 MB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 14886 bytes 1060591 (1.0 MB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
  
```

```
C:\Users\24450>ping 192.168.1.66
```

正在 Ping 192.168.1.66 具有 32 字节的数据:

来自 192.168.1.66 的回复: 字节=32 时间<1ms TTL=64

来自 192.168.1.66 的回复: 字节=32 时间<1ms TTL=64

来自 192.168.1.66 的回复: 字节=32 时间<1ms TTL=64

来自 192.168.1.66 的回复: 字节=32 时间<1ms TTL=64

192.168.1.66 的 Ping 统计信息:

数据包: 已发送 = 4, 已接收 = 4, 丢失 = 0 (0% 丢失),
往返行程的估计时间(以毫秒为单位):

最短 = 0ms, 最长 = 0ms, 平均 = 0ms

```
C:\Users\24450>ping 192.168.1.88
```

正在 Ping 192.168.1.88 具有 32 字节的数据:

来自 192.168.1.88 的回复: 字节=32 时间<1ms TTL=64

来自 192.168.1.88 的回复: 字节=32 时间<1ms TTL=64

来自 192.168.1.88 的回复: 字节=32 时间<1ms TTL=64

来自 192.168.1.88 的回复: 字节=32 时间=1ms TTL=64

192.168.1.88 的 Ping 统计信息:

数据包: 已发送 = 4, 已接收 = 4, 丢失 = 0 (0% 丢失),
往返行程的估计时间(以毫秒为单位):

最短 = 0ms, 最长 = 1ms, 平均 = 0ms

```
C:\Users\24450>ping 192.168.1.99
```

正在 Ping 192.168.1.99 具有 32 字节的数据:

来自 192.168.1.99 的回复: 字节=32 时间=2ms TTL=64

来自 192.168.1.99 的回复: 字节=32 时间=1ms TTL=64

来自 192.168.1.99 的回复: 字节=32 时间=1ms TTL=64

来自 192.168.1.99 的回复: 字节=32 时间=1ms TTL=64

192.168.1.99 的 Ping 统计信息:

数据包: 已发送 = 4, 已接收 = 4, 丢失 = 0 (0% 丢失),
往返行程的估计时间(以毫秒为单位):

最短 = 1ms, 最长 = 2ms, 平均 = 1ms

2.2 配置 PC 端 IP 配置文件

文件路径: /data/config/embosa_ip_config.json

配置示例 (假设 PC: 192.168.1.99, XCU: 192.168.1.66, Orin: 192.168.1.88) :

```
{
  "embosa_ip": {
    "local_interface": [
      "192.168.1.99"
    ],
    "peer_lists": [
```

```
        "192.168.1.66",  
        "192.168.1.88"  
    ]  
}  
}
```

2.3 配置 Orin 端 IP 配置文件

警告

需先通过 WiFi 连接 Orin，参考机器人端部署步骤 1

2.3.1 登录 Orin

```
ssh galbot@<Orin 无线 IP>  
# 密码: gb@2023
```

2.3.2 编辑配置文件

```
vi /data/config/embosa_ip_config.json
```

2.3.3 配置内容

危险

192.168.100.88 和 192.168.100.66 是 Orin 与 XCU 内部直连 IP，必须保留

```
{  
  "embosa_ip": {  
    "local_interface": [  
      "192.168.100.88",  
      "192.168.1.88"  
    ],  
    "peer_lists": [  
      "192.168.100.66",  
      "192.168.1.99"  
    ]  
  }  
}
```

2.4 配置 XCU 端 IP 配置文件

2.4.1 登录 XCU

```
ssh root@<XCU 无线 IP>  
# 密码: 12345678
```

2.4.2 编辑配置文件

```
vi /data/config/embosa_ip_config.json
```

2.4.3 配置内容

```
{  
  "embosa_ip": {  
    "local_interface": [  
      "192.168.100.66",  
      "192.168.1.66"  
    ]  
  }  
}
```



```
    ],
    "peer_lists": [
        "192.168.100.88",
        "192.168.1.99"
    ]
}
```

2.5 验证网络连接

警告

配置修改后必须重启机器人

重启后测试连接：

```
ping 192.168.1.66 # 测试 XCU
ping 192.168.1.88 # 测试 Orin
```

success

正常返回 ping 响应

```
fanma@fanma-OMEN-Gaming-Laptop-16-ae0xxx:~/桌面$ ping 192.168.1.66
PING 192.168.1.66 (192.168.1.66) 56(84) bytes of data.
64 字节，来自 192.168.1.66: icmp_seq=1 ttl=64 时间=1.60 毫秒
64 字节，来自 192.168.1.66: icmp_seq=2 ttl=64 时间=0.340 毫秒
64 字节，来自 192.168.1.66: icmp_seq=3 ttl=64 时间=1.36 毫秒
64 字节，来自 192.168.1.66: icmp_seq=4 ttl=64 时间=1.34 毫秒
^C
--- 192.168.1.66 ping 统计 ---
已发送 4 个包，已接收 4 个包，0% 包丢失，耗时 3018 毫秒
rtt min/avg/max/mdev = 0.340/1.159/1.596/0.483 ms
fanma@fanma-OMEN-Gaming-Laptop-16-ae0xxx:~/桌面$ ping 192.168.1.88
PING 192.168.1.88 (192.168.1.88) 56(84) bytes of data.
64 字节，来自 192.168.1.88: icmp_seq=1 ttl=64 时间=1.67 毫秒
64 字节，来自 192.168.1.88: icmp_seq=2 ttl=64 时间=0.598 毫秒
64 字节，来自 192.168.1.88: icmp_seq=3 ttl=64 时间=0.411 毫秒
^C
--- 192.168.1.88 ping 统计 ---
已发送 3 个包，已接收 3 个包，0% 包丢失，耗时 2034 毫秒
rtt min/avg/max/mdev = 0.411/0.894/1.674/0.556 ms
```

3. 在 PC 上安装 SDK

进入到 SDK 目录下执行安装脚本

```
cd GalbotSDK
./install.sh
```

4. 编译程序

在 PC 上编译 (x86_64)

```
cd examples/cpp/
mkdir -p build
cd build
cmake ../ -DCMAKE_TOOLCHAIN_FILE=../cmake/linux-x86_64-gcc940.cmake
make
```

5. 运行程序

5.1 配置环境变量

提示

/opt/galbot/为默认安装路径，可根据自己的安装路径进行修改

```
source /opt/galbot/galbot_sdk/linux-x86_64-gcc940/setup.sh
```

提示

```
echo 'source /opt/galbot/galbot_sdk/linux-x86_64-gcc940/setup.sh' >> ~/.bashrc  
source ~/.bashrc
```

5.2 运行 C++ 程序

```
cd /userdata # 或程序所在目录  
./your_app
```

5.3 运行 Python 程序

```
python your_app.py
```

提示

在运行 python example 时依赖一些库，所以请先运行下面的脚本保证相关库已安装：

```
cd GalbotSDK  
./install_python_deps.sh
```

11. Docker 部署

适用场景： 需要与宿主机隔离的环境

1. 环境配置

- 需安装 **Docker** 与 **Docker Compose**
- 可参考 **PC 端部署** 中第一、第二章节，完成网线连接、修改机器人 embosa_ip_config.json 等配置

2. 获取镜像

示例：

```
docker pull galbot/galbot_sdk:1.8.0
```

可根据实际需要版本，替换 galbot_sdk:1.8.0 中版本号

3. 启动容器

进入 SDK 产出中 docker 路径，执行 run 脚本即可启动镜像

```
cd GalbotSDK/docker  
./run.sh
```

容器内已安装好 SDK 动态库与运行环境

12. 常规操作

本文档介绍运行 Galbot 机器人过程中可能进行的一些常规操作，确保各项功能正常运行。

地图引擎 - 建图与定位

地图引擎是导航功能的核心基础。地图引擎不是可选附加功能，而是导航功能的前置必须步骤。在使用导航功能前，必须完成建图并确保定位正常运行。未正确完成建图和定位将导致导航功能无法正常工作。

当机器人开启定位导航服务时（机器人工作模式可能默认开启导航服务），必须先完成建图。新的机器人首次使用前需要进行建图操作，否则定位服务无法正常启动，机器人可能进入异常模式。

建图流程

建图是导航的前提条件。在开始使用导航功能前，必须先完成环境地图的构建。

1. 前置条件检查 (必选)

检查雷达服务是否已启动

建图需要雷达服务处于运行状态。在 HPU 终端输入以下命令查看雷达进程：

```
top
```

如果雷达已被自动拉起，可以在进程列表中看到名为 service_livox_capture 的进程，如下图所示：

top - 11:19:02 up 17 days, 4:05, 1 user, load average: 0.40, 0.58, 1.17
任务: 472 total, 2 running, 469 sleeping, 0 stopped, 1 zombie
%Cpu(s): 3.7 us, 3.0 sy, 0.0 ni, 92.6 id, 0.0 wa, 0.5 hi, 0.2 si, 0.0 st
MiB Mem : 62802.3 total, 16506.2 free, 13965.7 used, 32330.4 buff/cache
MiB Swap: 31401.2 total, 30768.4 free, 632.8 used. 47376.0 avail Mem

进程号	USER	PR	NI	VIRT	RES	SHR	%CPU	%MEM	TIME+	COMMAND
5821	galbot_+	0	-20	3631580	397940	111624	S	16.5	0.6	1704:47 nxnode.bin
5041	root	20	0	24.3g	176488	92340	S	7.9	0.3	1317:50 Xorg
5429	galbot_+	20	0	2512992	218492	51320	S	6.9	0.3	1406:17 ToDesk
4794	galbot_+	20	0	4636180	22664	14912	S	5.0	0.0	554:30.46 pulseaudio
2043234	galbot_+	0	-20	269388	93584	68244	S	3.6	0.1	29:45.56 nxcodemc.bin
1485	root	20	0	1624376	172388	14472	S	3.3	0.3	734:46.70 ToDesk_Service
3888867	galbot_+	20	0	1157.5g	538216	77384	S	2.3	0.8	3:13.43 code
174	root	-51	0	0	0	0	S	2.0	0.0	320:02.39 irq/134-host_sy
1346	root	20	0	0	0	0	S	1.3	0.0	225:21.73 nvgpu_channel_p
3897576	root	20	0	0	0	0	I	1.0	0.0	0:07.21 kworker/u24:2-dev+
3905925	galbot_+	20	0	1918188	128724	31232	S	1.0	0.2	0:01.51 service_livox_c
1078	root	20	0	543412	21536	16036	S	0.7	0.0	267:51.52 sunloginclnt
2902	nx	20	0	1049224	8688	6476	S	0.7	0.0	50:34.62 nxd
2660676	galbot_+	20	0	2744744	232056	117808	S	0.7	0.4	15:47.02 Isolated Web Co
3798400	galbot_+	20	0	2494808	106952	86940	S	0.7	0.2	8:13.65 Isolated Web Co
3906016	galbot_+	20	0	15132	3896	2908	R	0.7	0.0	0:01.83 top

如果雷达未启动，使用以下命令启动雷达（注意：不要重复启动）：

```
/data/galbot/bin/service_livox_capture
```

确保地图路径具有可写权限

默认地图路径存放在 `/var/maps/` 目录下。如果该路径不存在或没有写入权限，请创建路径并赋予可写权限：

```
sudo mkdir -p /var/maps/  
sudo chmod 777 -R /var/maps/
```

2. 开始建图 (必选)

启动建图程序

1. 拍下机器人急停按钮，将机器人推至建图起点（可以从任何位置开始建图）

1. 执行以下命令启动建图：

```
/data/galbot/bin/mapping_server
```

正常建图时会显示以下信息：

- 关键帧数量 (keyframe num)
- 位姿延时 (time delay)
- 当前机器人位姿 (x y z qx qy qz qw)

关键帧数量会随着机器人移动而增加，机器人静止时关键帧数量保持不变，如下图所示：

```
keyframe num: 1 time delay: 0.087615 pose.x: 0.00809675 pose.y: -0.0123061 pose.z: -0.00137215 pose.qx: 0.000518241 pose.qy: 0.999999  
keyframe num: 1 time delay: 0.088187 pose.x: 0.010443 pose.y: -0.0123492 pose.z: -0.00125673 pose.qx: 0.000118427 pose.qy: 1  
keyframe num: 1 time delay: 0.0884869 pose.x: 0.00907572 pose.y: -0.0117949 pose.z: -0.0014445 pose.qx: 0.000806805 pose.qy: 0.999999  
keyframe num: 1 time delay: 0.0891619 pose.x: 0.00719585 pose.y: -0.0117713 pose.z: -0.00134079 pose.qx: 0.00111135 pose.qy: 0.999999  
keyframe num: 1 time delay: 0.0895579 pose.x: 0.00719589 pose.y: -0.0120172 pose.z: -0.00107829 pose.qx: 0.00102897 pose.qy: 0.999999  
keyframe num: 1 time delay: 0.0902181 pose.x: 0.00569084 pose.y: -0.00926068 pose.z: -0.00143298 pose.qx: 0.00109368 pose.qy: 0.999999  
keyframe num: 1 time delay: 0.0915489 pose.x: 0.00342654 pose.y: -0.0106243 pose.z: -0.000845056 pose.qx: 0.000325046 pose.qy: 1  
keyframe num: 1 time delay: 0.0920222 pose.x: 0.00458235 pose.y: -0.0103369 pose.z: -0.00045693 pose.qx: 0.000920564 pose.qy: 0.999999  
keyframe num: 1 time delay: 0.0925741 pose.x: 0.00442366 pose.y: -0.0103272 pose.z: -0.000227458 pose.qx: 0.00141885 pose.qy: 0.999999  
keyframe num: 1 time delay: 0.092972 pose.x: 0.00520872 pose.y: -0.00961502 pose.z: -0.00182349 pose.qx: -1.74964e-05 pose.qy: 1  
keyframe num: 1 time delay: 0.0940652 pose.x: 0.00597258 pose.y: -0.00978803 pose.z: -0.00153781 pose.qx: 0.00122323 pose.qy: 0.999999  
keyframe num: 1 time delay: 0.0945451 pose.x: 0.00841151 pose.y: -0.0123826 pose.z: -0.00239345 pose.qx: 0.0010669 pose.qy: 0.999999  
keyframe num: 1 time delay: 0.0952082 pose.x: 0.00950637 pose.y: -0.0137458 pose.z: -0.00303864 pose.qx: 0.000938807 pose.qy: 0.999999  
keyframe num: 1 time delay: 0.0969009 pose.x: 0.00905055 pose.y: -0.0154099 pose.z: -0.00179241 pose.qx: 0.000425681 pose.qy: 0.999999  
keyframe num: 1 time delay: 0.0971839 pose.x: 0.0102936 pose.y: -0.0149759 pose.z: -0.00254112 pose.qx: 0.000559082 pose.qy: 1  
keyframe num: 1 time delay: 0.0977478 pose.x: 0.00892673 pose.y: -0.0142835 pose.z: -0.000914829 pose.qx: -0.000222805 pose.qy: 0.999999  
keyframe num: 1 time delay: 0.098315 pose.x: 0.00804138 pose.y: -0.0119794 pose.z: -0.0011842 pose.qx: 0.000857199 pose.qy: 0.999999  
keyframe num: 1 time delay: 0.098774 pose.x: 0.00444347 pose.y: -0.0118939 pose.z: -0.000689109 pose.qx: 0.00106696 pose.qy: 0.999999  
keyframe num: 1 time delay: 0.0993061 pose.x: 0.00397314 pose.y: -0.0100379 pose.z: -0.00201442 pose.qx: 0.000628611 pose.qy: 1  
keyframe num: 1 time delay: 0.100149 pose.x: 0.00437601 pose.y: -0.0123444 pose.z: -0.00277242 pose.qx: 0.000365016 pose.qy: 1  
keyframe num: 1 time delay: 0.102378 pose.x: 0.00282276 pose.y: -0.011793 pose.z: -0.00272449 pose.qx: 0.000950535 pose.qy: 0.999999  
keyframe num: 1 time delay: 0.102906 pose.x: 0.00280599 pose.y: -0.0104429 pose.z: -0.00383237 pose.qx: 0.000315153 pose.qy: 1
```

1. 推动机器人遍历环境一周以构建完整地图。推动时应避免过大的加速度和角速度（接近人行走速度即可）

3. 保存地图 (必选)

保存构建好的地图

直接结束建图程序不会自动保存地图。如需保存地图，请先执行保存操作，然后再关闭建图程序。

执行保存地图工具：

```
/data/galbot/bin/engine_tools
```

启动工具后，输入数字 1，然后按 Enter 回车选择保存地图功能。地图默认保存路径为

`/var/maps/room1102`，如下图所示：

```
(base) galbot_orin_1225@ubuntu:~/workspace/代码整理后/mes_eea/bin$ ./engine_tools

=====
          地图引擎测试工具
=====
请输入数字选择功能：
1 保存地图
2 发送一次初始定位
3 pcd点云地图转osm文件
4 开始更新地图
5 结束地图更新
6 录制位姿估计轨迹
7 录制轮式odom轨迹
8 打印电子围栏
9 调用一次重定位服务
请输入数字选择 (1-9)，输入88退出当前程序
: 
```

保存地图后，地图目录下必须存在 `global_cloud_cleaned.pcd`，供后续定位使用。

- 如果已经完成点云后处理，请确保清理后的地图文件命名为 `global_cloud_cleaned.pcd`
- 如果暂时不做杂点擦除，请在地图目录下执行以下命令：

```
cp global_cloud.pcd global_cloud_cleaned.pcd
```

4. 编辑地图 (可选)

本步骤用于优化地图质量，提高导航精度和安全性。

■ 信息

- **擦除地图杂点**：地图中的空中杂点可能导致导航系统误判障碍物，影响路径规划精度
- **绘制电子围栏**：没有电子围栏限制，机器人可能会进入非预期或危险区域

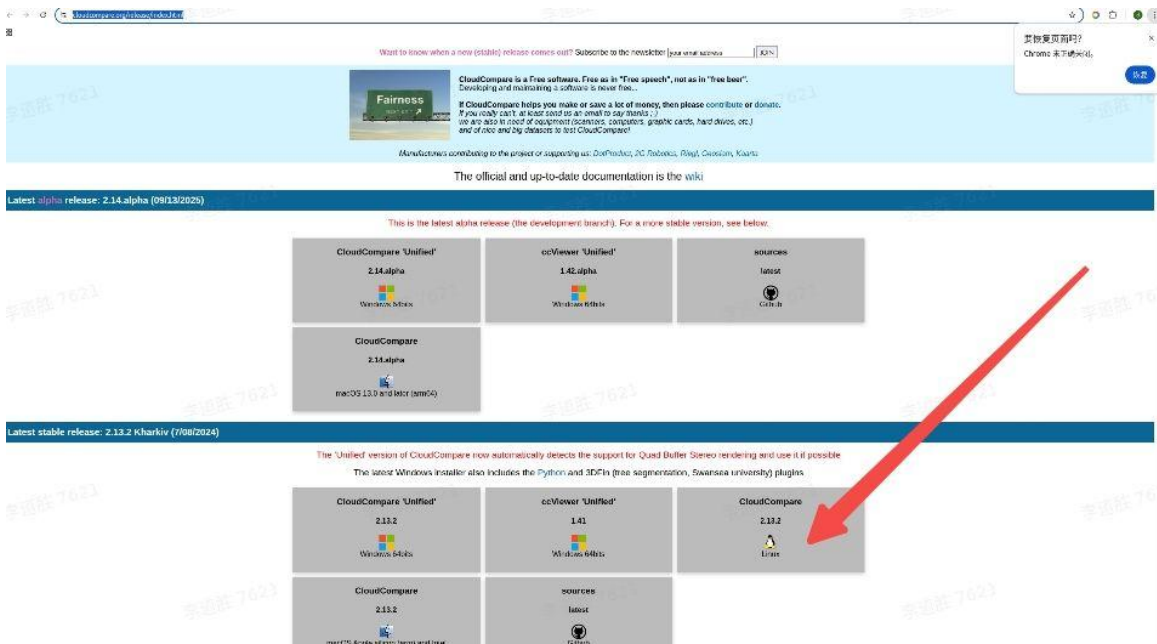
4.1 擦除地图杂点 (可选)

推荐进行本步骤。地图中通常可能存在少量杂点，擦除后可以提高导航成功率。尤其当房间中存在镜面、毛玻璃等材质物体时，点云地图中更容易出现杂点，建议使用 CloudCompare 软件进行后处理。

准备工作：下载 CloudCompare 软件

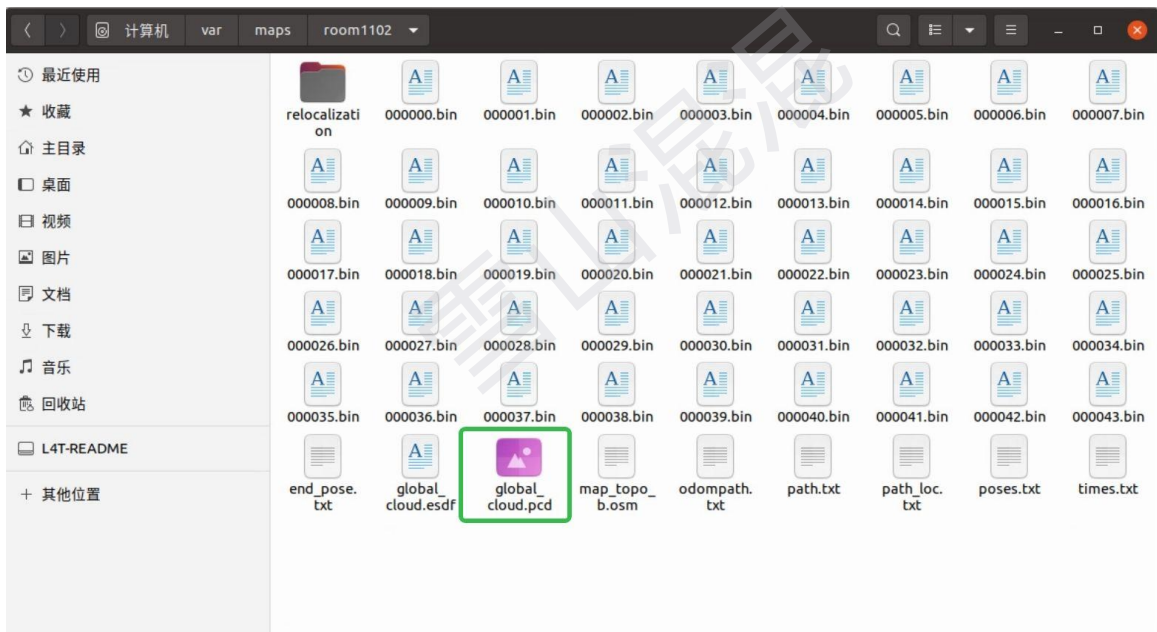
下载链接：<https://www.cloudcompare.org/release/index.html>

如图所示：



操作步骤：

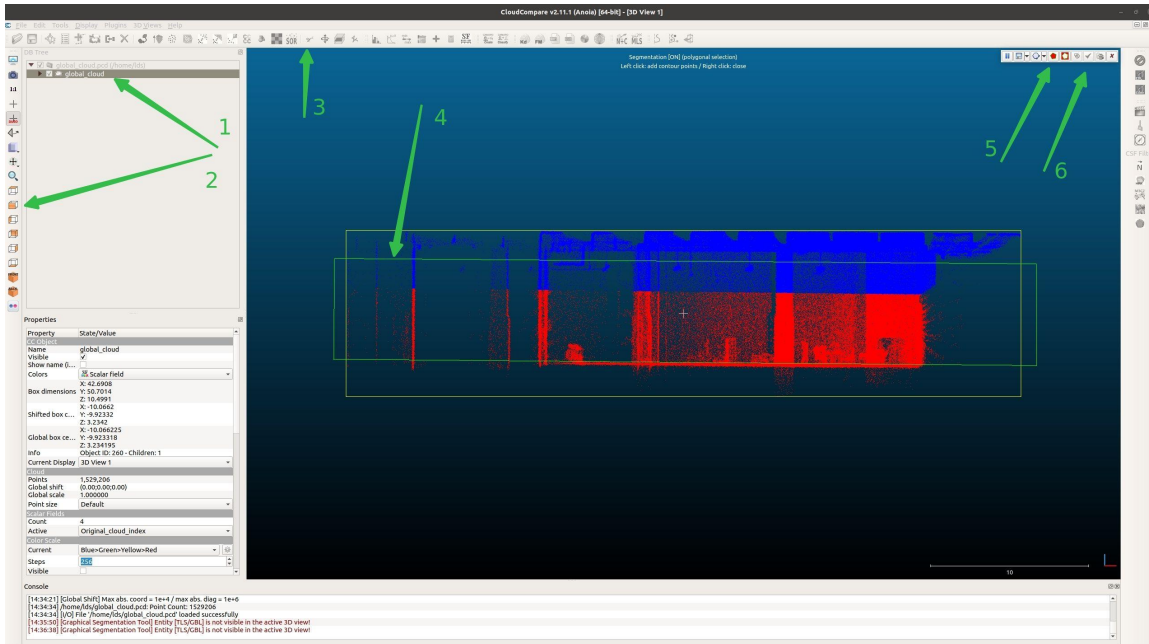
1. 打开 CloudCompare 软件，将点云地图 `global_cloud.pcd` 拖入软件工作区



1. 按照下图绿色箭头编号依次操作：

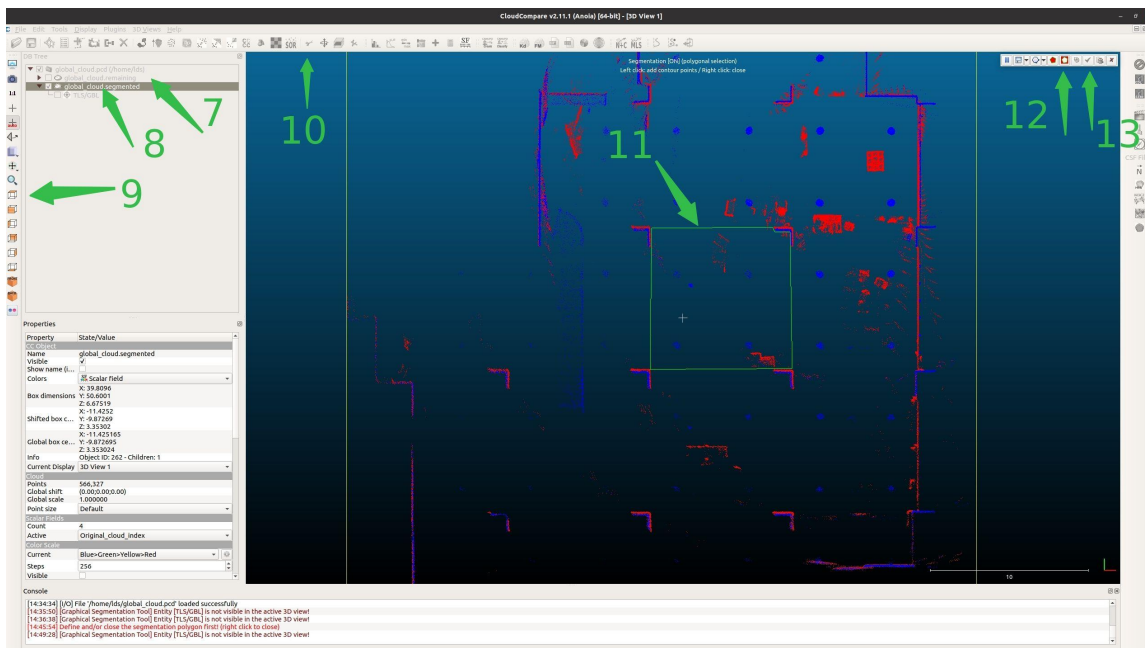
- **步骤 1：**勾选文件
- **步骤 2：**选择正视图
- **步骤 3：**选择裁剪功能（图标为剪刀）
- **步骤 4：**框选目标区域（地面和天花板之间的机器人活动区域）
- **步骤 5：**保留内部点云
- **步骤 6：**确认操作（图标为对勾）

此时点云被分割为两部分：框选内的 `global_cloud.segmented` 和框选外的 `global_cloud.remaining`：



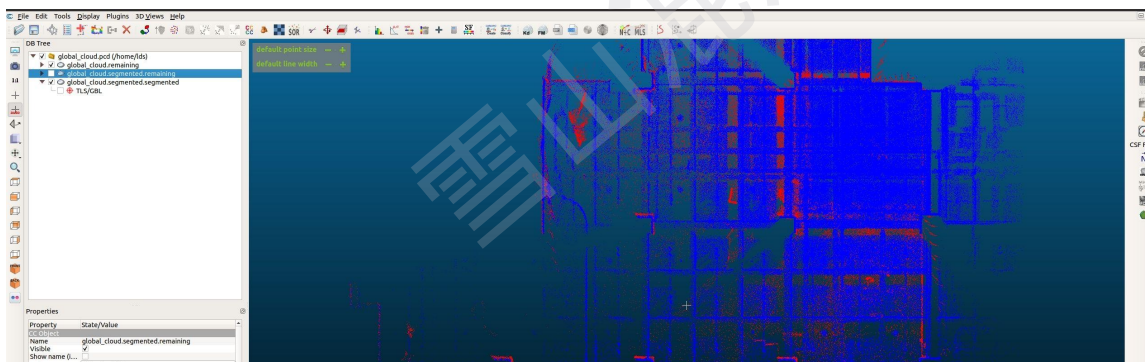
1. 继续以下步骤：

- **步骤 7：**取消勾选 `global_cloud.remaining`
- **步骤 8：**勾选 `global_cloud.segmented`
- **步骤 9：**选择俯视图
- **步骤 10：**选择裁剪功能（剪刀图标）
- **步骤 11：**框选机器人活动区域（注意不要选中真实障碍物，如料垛、桌子等）
- **步骤 12：**点击框外图标
- **步骤 13：**确认操作



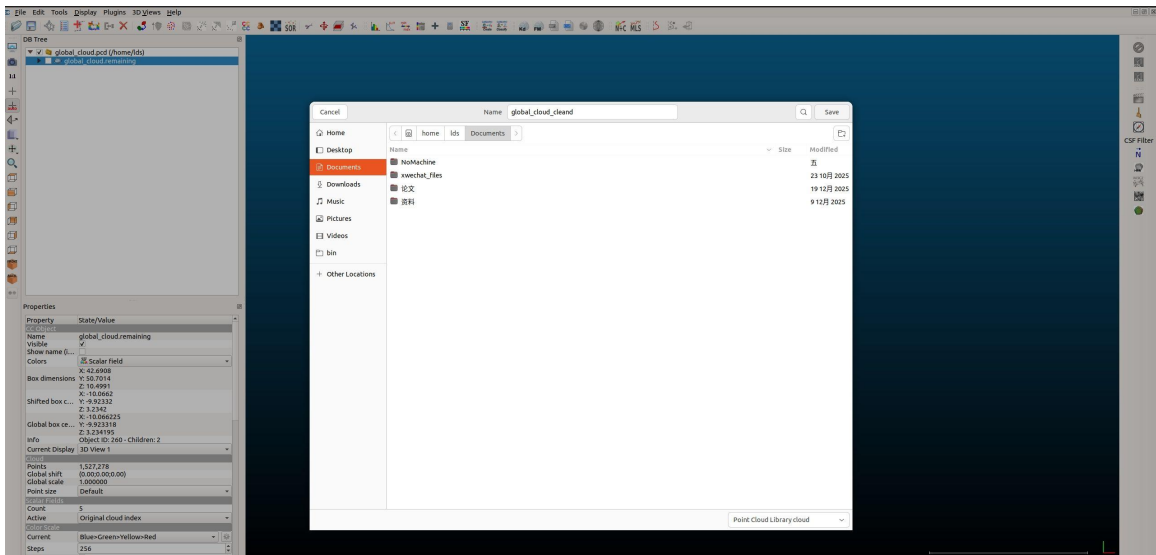
1. 删除杂点并合并：

- 选中 `global_cloud.segmented.remaining` (杂点部分) ， 右键删除
- 按住 `Ctrl` 键多选 `global_cloud.remaining` 和 `global_cloud.segmented.remaining`



1. 保存清理后的地图：

- 选择 `Edit` → `Merge` 合并两部分点云
- 选择 `File` → `Save` 保存点云
- 必须命名为 `global_cloud_cleaned.pcd`
- 移动擦除后的地图 `global_cloud_cleaned.pcd` 到地图文件夹 `/var/maps/room1102/`



4.2 编辑 OSM 文件 (可选)

通过绘制电子围栏来限制机器人的活动范围，防止机器人进入危险或非预期区域。

信息

- **绘制电子围栏：**在地图上设置虚拟边界，禁止机器人穿越围栏进入特定区域（如楼梯、危险区域、非工作区域等）
- **不做的影响：**机器人可以自由移动到地图中的任何位置，可能进入危险区域或非预期的工作区域，存在安全风险

在建图完成后（假设地图保存路径为 `/var/maps/room1102/`），使用 `engine_tools` 工具转换点云地图为 OSM 文件：

1. 执行 `engine_tools`，输入数字 3，选择点云地图转 OSM 文件功能

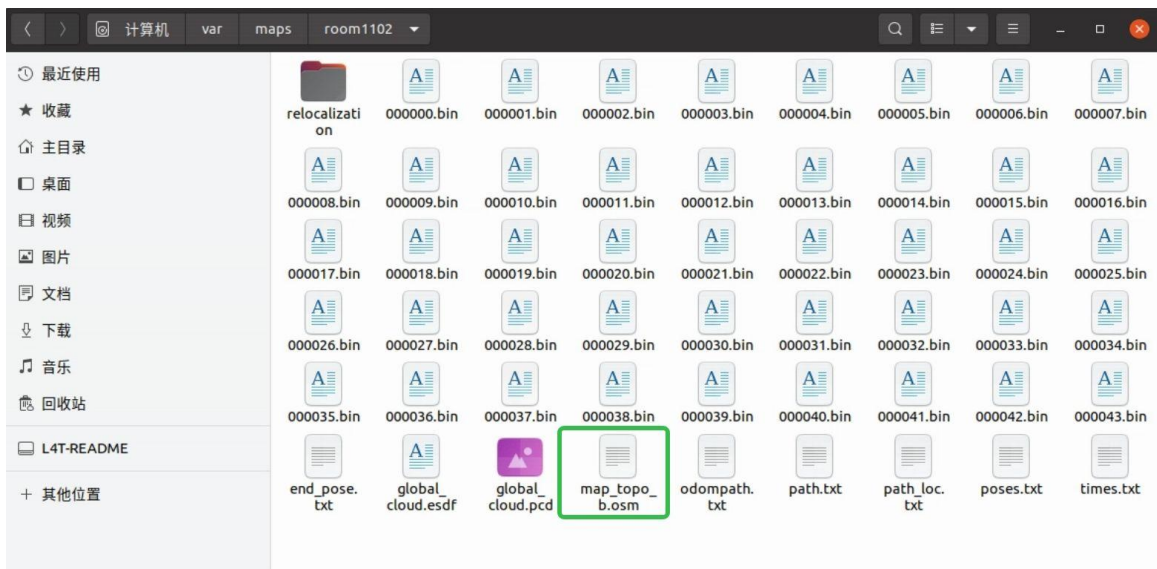
1. 直接回车使用默认地图。当前示例地图存放在 `/var/maps/room1102/`，所以需要输入地图的完整路径 `/var/maps/room1102/global_cloud.pcd`，然后回车转换：

```
(base) galbot_orin_1225@ubuntu:~/workspace/代码整理后/mes_eea_double_lidar/bin$ ./engine_tools

=====
地图引擎测试工具
=====
请输入数字选择功能：
1 保存地图
2 发送一次初始定位
3 pcd点云地图转osm文件
4 开始更新地图
5 结束地图更新
6 录制位姿估计轨迹
7 录制轮式odom轨迹
8 打印电子围栏
9 调用一次重定位服务
请输入数字选择（1-9），输入88退出当前程序
: 3

=== 你选择了pcd转osm ===
默认使用的PCD文件是: /var/maps/cur/global_cloud.pcd
如过不使用默认地图请输入完整的pcd文件路径如/var/maps/cur/cloudmap.pcd 可以回车使用默认PCD地图
: /var/maps/cur/global_cloud.pcd
```

1. 转换后的文件如下图绿框所示，使用 JOSM 软件打开该 OSM 文件，如图所示：



安装 JOSM (如未安装) :

```
sudo apt install josm
```

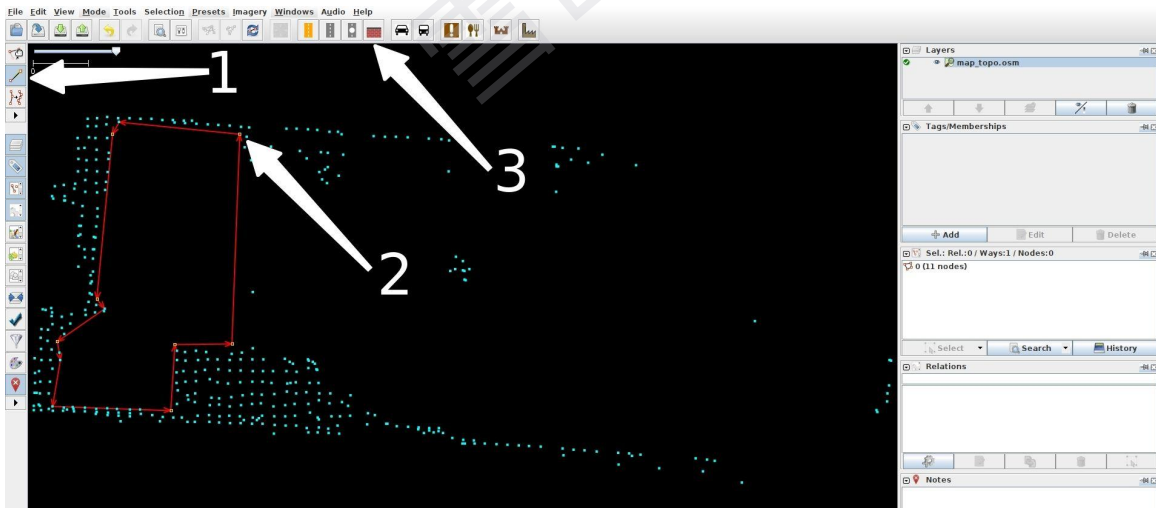
安装完成后, 终端输入 `josm` 打开软件。

绘制电子围栏步骤:

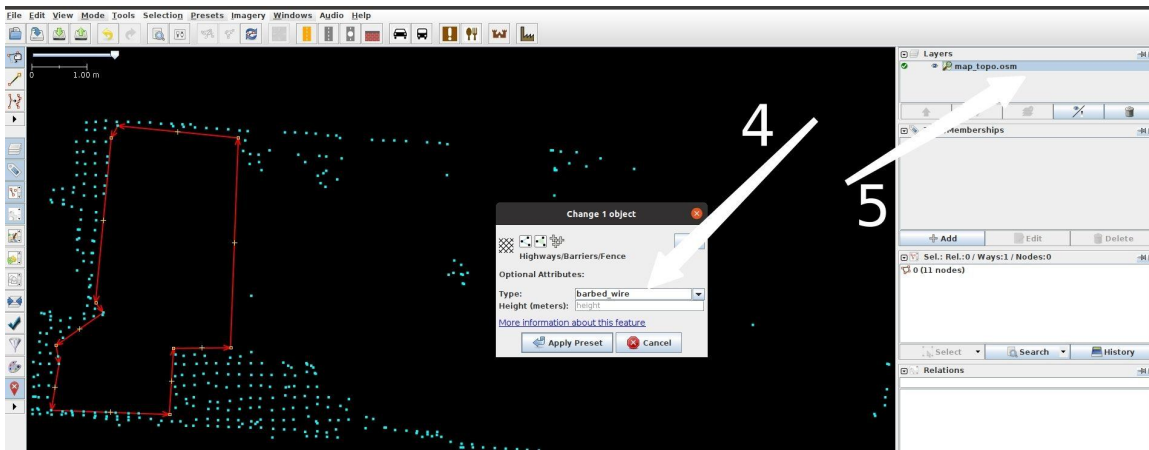
1. 选择左上角文件 → 打开, 找到 `/var/maps/room1102/map_topo.osm`

1. 选择下图白色箭头 1 指向的图标, 鼠标左键单击绘制图形 (红色细线), 如白色箭头 2 所示

1. 绘制结束后 (封闭图形自动结束, 非封闭图形按 `Esc` 结束), 单击白色箭头 1 上方的图标, 选择刚刚绘制的线 (被选中的线呈红色, 按住 `Ctrl` 可选择多条线), 然后点击白色箭头 3:



1. 在弹窗里选择 Fence 栏, 设置类型为 `barbed_wire` (如白色箭头 4 所示), 点击 Apply Preset, 最后鼠标右击白色箭头 5, 选择 Save 保存:



5. 地图更新 (可选)

当环境发生变化时（如新增家具、障碍物移动等），可以通过地图更新功能来优化地图。

信息

如果环境发生变化后不更新地图，机器人仍使用旧地图进行导航，可能导致：

- 定位精度下降
- 路径规划不准确
- 机器人可能误判障碍物位置

为节省计算资源，地图更新功能默认关闭。可以手动开启更新开关并由人工操作保存更新后的地图：

步骤 1： 在定位良好的情况下（当前评分大于 0.9）打开地图更新开关

```
vim /data/galbot/config/mes/manager.config
```

将 `manager_update_map=0` 改为 `manager_update_map=1`

步骤 2： 原地重启定位，在评分恢复正常后（当前评分大于 0.9），执行 `engine_tools` 选择开始更新地图

步骤 3： 推动机器人绕环境一周后，执行 `engine_tools` 选择结束更新地图

步骤 4： 更新后的地图不会覆盖当前定位地图（`/var/maps/cur`），而是保存在 `/var/maps/update_map`

步骤 5： 地图更新结束后，将参数 `manager_update_map=1` 改回 0，然后可以替换新地图了

警告

1. 修改参数和替换地图都需要重启定位服务才能生效

1. 当前示例写的是 `engine_tools`，如果没有 `engine_tools`，使用对应的 `/data/galbot/bin/test_start_update_map` 和 `/data/galbot/bin/test_stop_update_map`

定位流程

定位是使用导航功能的前提。在开始导航前，必须确保定位服务正常运行且定位状态良好。

1. 准备地图 (必选)

定位程序需要启动雷达（定位程序不依赖建图程序，建图结束后即可关闭建图程序）。

定位使用的地图路径是 `/var/maps/cur`，将目标地图重命名为 `cur`，例如将刚刚新建的地图 `room1102` 直接重命名为 `cur`，如图所示：



2. 启动定位 (必选)

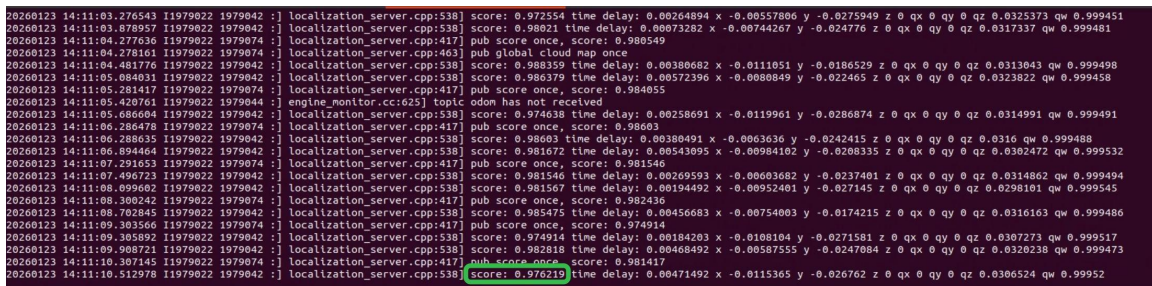
启动定位服务：

```
/data/galbot/bin/localization_server
```

3. 查看定位状态 (必选)

查看定位状态：

```
tail -f /userdata/log/localization_server/localization_server.INFO
```



如果分数低于 0.75 或者没有分数和位姿发布，请将机器推至建图起点位姿，下发一次初始位姿（使用 `engine_tools` 输入数字 2，Enter 回车发送一次初始位姿），如图所示：

```
(base) galbot_orin_1225@ubuntu:~/workspace/代码整理后/mes_eea/bin$ ./engine_tools

=====
          地图引擎测试工具
=====
请输入数字选择功能：
1 保存地图
2 发送一次初始定位
3 pcd点云地图转osm文件
4 开始更新地图
5 结束地图更新
6 录制位姿估计轨迹
7 录制轮式odom轨迹
8 打印电子围栏
9 调用一次重定位服务
请输入数字选择 (1-9)，输入88退出当前程序
: █
```

4. 验证定位就绪 (必选)

在开始导航前，请确认以下检查项：

- 雷达服务正在运行
- 定位服务已启动
- 定位分数大于 0.75
- 当前位姿正常发布

success

完成以上所有必选步骤后，您可以开始使用机器人导航功能。详情请参考 入门教程 - Example 3. 机器人导航。

13. 入门教程

接下来将会通过几个简单示例的介绍，来快速了解 Galbot 机器人的基础开发应用。运行接下来的所有示例，您需要首先下载安装 Galbot SDK，并参考安装与配置说明进行配置。

Example1. 机器人的基础控制

本示例展示了 GALBOT 机器人的基础控制，执行本程序后，机器人的两臂将缓慢举起到头顶，形成一个类似比心的姿势。通过初始化机器人实例、检查安全状态后，让机器人的左右手臂执行预设的比心动作序列，先将手臂移动到比心姿势保持一段时间，然后恢复到原始姿态，整个过程包含了关节位置获取、关节角度设置、阻塞等待以及异常处理等基本控制流程。

警告

```
examples/python/tutorials/example1_galbot_control_started.py
"""
Note: When running this example, please ensure the robot's `emergency stop button` is
released;
"""
import time

try:
    from galbot_sdk.g1 import GalbotRobot
    from galbot_sdk.g1 import ControlStatus
except ImportError:
    print("Failed to import galbot_sdk, please install it first or check if it is in
PYTHONPATH")
    exit(1)

def demo_heart_pose(robot: GalbotRobot,
                    joint_group_names: list,
                    position_seq: list,
                    is_blocking: bool,
                    max_speed: float,
                    timeout_s: float,
                    retry_count: int = 3
                    ):
    """
    Robot heart gesture demonstration function

    Parameters:
        robot (GalbotRobot): GalbotRobot instance
        joint_group_names (list): List of joint group names to control
        position_seq (list): List of joint group angle sequences to set
        is_blocking (bool): Whether to set angles in blocking mode
        max_speed (float): Maximum speed
        timeout_s (float): Timeout time (seconds)
        retry_count (int, optional): Number of retries, default is 3

    Returns:
```

```

None
"""

# Get current joint group angles for subsequent restoration
original_pos = robot.get_joint_positions(joint_group_names, [])
print(f"Current angles of joint group {joint_group_names}: {original_pos}")

# Start heart gesture
pos_idx = 0
print("Starting heart gesture...")
while True:
    time.sleep(1)
    pos = position_seq[pos_idx]
    control_status = robot.set_joint_positions(
        pos, joint_group_names, [], is_blocking, max_speed, timeout_s
    )

    # If setting fails, retry 3 times
    retry_cnt = retry_count
    while control_status != ControlStatus.SUCCESS and retry_cnt > 0:
        print(f"Setting angles for joint group {joint_group_names} failed, retrying
{retry_cnt}...")
        retry_cnt = retry_cnt - 1
        time.sleep(1)
        control_status = robot.set_joint_positions(
            pos, joint_group_names, [], is_blocking, max_speed, timeout_s
        )
    # If successful, switch to next pose sequence
    if control_status == ControlStatus.SUCCESS:
        print(f"Setting angles for joint group {joint_group_names} successful")
        pos_idx = pos_idx + 1
    # If failed, break the loop
    else:
        print(f"Setting angles for joint group {joint_group_names} failed")
        break

    # If all pose sequences are completed, break the loop
    if pos_idx > len(position_seq) - 1:
        break

# Get current joint group angles
print("Showing heart gesture for 15 seconds, then restoring original pose...")
time.sleep(5)

# Restore original pose joint group angles
control_status = robot.set_joint_positions(
    original_pos, joint_group_names, [], is_blocking, max_speed, timeout_s
)
# If setting fails, retry 5 times
retry_cnt = retry_count
while control_status != ControlStatus.SUCCESS and retry_cnt > 0:
    print(f"Setting angles for joint group {joint_group_names} failed, retrying
{retry_cnt}...")
    retry_cnt = retry_cnt - 1
    time.sleep(2)
    control_status = robot.set_joint_positions(
        original_pos, joint_group_names, [], is_blocking, max_speed, timeout_s
    )
# If successful, restore original pose
if control_status == ControlStatus.SUCCESS:

```

```

        print(f"Restoring angles for joint group {joint_group_names} successful")
    else:
        print(f"Restoring angles for joint group {joint_group_names} failed")

def check_robot_safety():
    """Check if the robot is safe"""
    # Prompt for precautions
    print("    Note: 1. Please ensure the robot's emergency stop button is released; 2. Please ensure there are no obstacles in front, back, left, and right of the robot to avoid unexpected situations.")
    while True:
        key = input("Please confirm that the robot's emergency stop button is released and there are no obstacles. Continue? (y/n)...")
        if key == 'y':
            print("User confirmed, continuing execution...")
            break
        elif key == 'n':
            print("User not confirmed, program exiting...")
            exit(1)
        else:
            print("Input error, please enter 'y' or 'n'")

def main():
    check_robot_safety()

    try:
        # Get robot instance
        robot = GalbotRobot()

        # Initialize robot
        state = robot.init()
        if not state:
            print(f"Initialization failed")
            exit(1)
        else:
            print(f"Initialization successful")
            print(f"Is robot running: {robot.is_running()}")

        # Wait for data preparation
        time.sleep(3)

        # Get list of joint names
        joint_names = robot.get_joint_names()
        if len(joint_names) > 0:
            print(f"List of joint names: {joint_names}")
        else:
            print(f"Failed to get list of joint names")

        # Get joint positions using joint group names, empty returns all joints by default
        joint_group_names = ["left_arm", "right_arm"]
        # Left and right arm heart gesture sequence
        position_seq = [
            [1.53, 0.36, -2.54, -1.80, 0.12, -0.82, 0.09, # left_arm
             -1.53, -0.36, 2.54, 1.80, -0.12, 0.82, -0.09] # right_arm
        ]
        # Whether to block and wait for joints to reach position
        is_blocking = True
        # Limit maximum joint speed to 0.1rad/s
        max_speed = 0.1
        # Maximum blocking wait time

```

```

        timeout_s = 20

        # Perform heartbeat gesture
        demo_heart_pose(robot, joint_group_names, position_seq,
                        is_blocking, max_speed, timeout_s)

    except Exception as e:
        print(f"An exception occurred: {e}")
    finally:
        # Actively send SIGINT shutdown signal
        robot.request_shutdown()
        # Wait to enter shutdown state
        robot.wait_for_shutdown()
        # Release SDK resources
        robot.destroy()
        print('Resource release successful')

if __name__ == "__main__":
    main()

```

Example2. 手臂操控

本示例展示了 GALBOT 机器人的手臂操控，执行本程序后，机器人左臂将缓缓抬起一定高度，然后在缓缓恢复原始姿态。主要实现了正逆运动学（IK/FK）求解、末端执行器位姿控制以及关节角度设置等功能，通过获取当前左臂末端位姿、使用逆运动学计算目标位姿对应的关节角度、设置关节角度使机器人移动到目标位置，并验证最终位姿与目标位姿的一致性，最后将机器人恢复到原始位姿。

警告

```

examples/python/tutorials/example2_arm_manipulation.py
"""
Note: When running this example, please ensure the robot's motion control service
`/data/galbot/bin/service_motion_plan`,
robot state publishing service `/data/galbot/bin/robot_state_publish`,
and hand-eye calibration publishing service `/data/galbot/bin/eyehand_calib_publish`
are loaded;
"""
try:
    import galbot_sdk.g1 as gm
    from galbot_sdk.g1 import GalbotMotion
    from galbot_sdk.g1 import GalbotRobot
    from galbot_sdk.g1 import ControlStatus
except ImportError:
    print("Failed to import galbot_sdk, please install it first or check if it is in
PYTHONPATH")
    exit(1)

import os

try:
    import numpy as np
except ImportError:
    os.system("pip install numpy")
    import numpy as np

```

```

import time
from typing import Sequence, Dict

def printStatus(status):
    if(status == gm.MotionStatus.SUCCESS):
        print("Execution result: SUCCESS, Execution successful")
    elif(status == gm.MotionStatus.TIMEOUT):
        print("Execution result: TIMEOUT, Execution timeout")
    elif(status == gm.MotionStatus.FAULT):
        print("Execution result: FAULT, Fault occurred, unable to continue execution")
    elif(status == gm.MotionStatus.INVALID_INPUT):
        print("Execution result: INVALID_INPUT, Input parameters do not meet requirements")
    elif(status == gm.MotionStatus.INIT_FAILED):
        print("Execution result: INIT_FAILED, Internal communication component creation
failed")
    elif(status == gm.MotionStatus.IN_PROGRESS):
        print("Execution result: IN_PROGRESS, Moving but not in position")
    elif(status == gm.MotionStatus.STOPPED_UNREACHED):
        print("Execution result: STOPPED_UNREACHED, Stopped but did not reach target")
    elif(status == gm.MotionStatus.DATA_FETCH_FAILED):
        print("Execution result: DATA_FETCH_FAILED, Data acquisition failed")
    elif(status == gm.MotionStatus.PUBLISH_FAIL):
        print("Execution result: PUBLISH_FAIL, Data sending failed")
    elif(status == gm.MotionStatus.COMM_DISCONNECTED):
        print("Execution result: COMM_DISCONNECTED, Connection failed")

def quat_normalize(q: np.ndarray) -> np.ndarray:
    q = np.array(q, dtype=np.float64)
    return q / np.linalg.norm(q)

def quat_conjugate(q: np.ndarray) -> np.ndarray:
    """
    Calculate the conjugate of a quaternion

    Parameters:
        q (np.ndarray): Input quaternion [x, y, z, w]

    Returns:
        np.ndarray: Conjugate of the quaternion [x, y, z, w]
    """
    qx, qy, qz, qw = q
    return np.array([-qx, -qy, -qz, qw])

def quat_multiply(q1: np.ndarray, q2: np.ndarray) -> np.ndarray:
    """
    Calculate the product of two quaternions

    Parameters:
        q1 (np.ndarray): First quaternion [x, y, z, w]
        q2 (np.ndarray): Second quaternion [x, y, z, w]

    Returns:
        np.ndarray: Product of the two quaternions [x, y, z, w]
    """
    x1, y1, z1, w1 = q1
    x2, y2, z2, w2 = q2
    return np.array([
        w1*x2 + x1*w2 + y1*z2 - z1*y2,
        w1*y2 - x1*z2 + y1*w2 + z1*x2,

```



```

        w1*z2 + x1*y2 - y1*x2 + z1*w2,
        w1*w2 - x1*x2 - y1*y2 - z1*z2
    ])

def orientation_error_angle(A: np.ndarray, B: np.ndarray) -> float:
    """
    Calculate the rotation angle error between two quaternions (radians)

    Parameters:
        A (np.ndarray): First quaternion [x, y, z, w]
        B (np.ndarray): Second quaternion [x, y, z, w]

    Returns:
        float: Rotation angle error (radians)
    """
    qA = quat_normalize(A[3:7])
    qB = quat_normalize(B[3:7])

    q_err = quat_multiply(qB, quat_conjugate(qA))
    q_err = quat_normalize(q_err)

    # Numerical stability
    qw = np.clip(q_err[3], -1.0, 1.0)

    angle = 2 * np.arccos(qw)
    return angle # Unit: radians

def calculate_error(pose1: Sequence[float], pose2: Sequence[float]) -> Dict[str, float]:
    """
    Calculate position error and rotation error between two poses (radians)

    Parameters:
        pose1 (Sequence[float]): First pose [x, y, z, qx, qy, qz, qw]
        pose2 (Sequence[float]): Second pose [x, y, z, qx, qy, qz, qw]

    Returns:
        dict: Dictionary containing position error (meters) and rotation error (radians)
    """
    A, B = np.array(pose1), np.array(pose2)
    pos_err = np.linalg.norm(A[:3] - B[:3])
    rot_err = orientation_error_angle(A, B)

    return {
        "position_error_norm": pos_err,
        "orientation_error_rad": rot_err,
        "orientation_error_deg": np.degrees(rot_err)
    }

def check_robot_safety():
    """Check if the robot is safe"""
    # Prompt for precautions
    print("    Note: 1. Please ensure the robot's emergency stop button is released; 2. Please ensure there are no obstacles in front, back, left, and right of the robot to avoid unexpected situations.")
    while True:
        key = input("Please confirm that the robot's emergency stop button is released and there are no obstacles. Continue? (y/n)...")
        if key == 'y':
            print("User confirmed, continuing execution...")

```

```

        break
    elif key == 'n':
        print("User not confirmed, program exiting...")
        exit(1)
    else:
        print("Input error, please enter 'y' or 'n'")

def main():
    check_robot_safety()
    try:
        # Get GalbotMotion singleton and initialize
        robot = GalbotRobot()
        motion = GalbotMotion()

        if motion.init():
            print("GalbotMotion initialization successful")
        else:
            print("GalbotMotion initialization failed")
        if robot.init():
            print("GalbotRobot initialization successful")
        else:
            print("GalbotRobot initialization failed")

        # Program starts immediately, wait for data readiness time
        time.sleep(3)

        # Define target pose
        chain_pose_baselink = {
            "leg": [0.0596, -0.0000, 1.0327, 0.5000, 0.5003, 0.4997, 0.5000],
            "head": [0.0599, 0.0002, 1.4098, -0.7072, 0.0037, 0.0037, 0.7069],
            "left_arm": [0.1267, 0.2342, 0.7356, 0.0220, 0.0127, 0.0343, 0.9991],
            "right_arm": [0.1267, -0.2345, 0.7358, -0.0225, 0.0126, -0.0343, 0.9991]
        }

        # set initial joint positions
        joint_pos = [0.5, 1.5, 1.0, 0.0, 0.0,
                    0.0, 0.0,
                    2.0, -1.5, -0.6, -1.7, 0.0, -0.8, 0.0,
                    -2.0, 1.5, 0.6, 1.7, 0.0, 0.8, 0.0]
        joint_groups_names = ["leg", "head", "left_arm", "right_arm"]
        joint_names = []
        is_blocking = True
        max_speed_rad_s = 0.1
        timeout_s = 30.0

        status = robot.set_joint_positions(
            joint_pos, joint_groups_names, joint_names, is_blocking, max_speed_rad_s,
            timeout_s
        )

        if status != ControlStatus.SUCCESS:
            print("set join position failed")
        else:
            print("set join position successful")

        # Define target chain name, target pose, reference pose, end link
        target_frame = "EndEffector"
        reference_frame = "base_link"
        target_chain = "left_arm"
        end_link = "left_arm_end_effector_mount_link"

```

```

# 1. Get current left_arm end-effector pose
try:
    status, original_pose = motion.get_end_effector_pose_on_chain(
        chain_name=target_chain,
        frame_id=target_frame,
        reference_frame=reference_frame
    )
    assert status == gm.MotionStatus.SUCCESS, "Failed to get end-effector pose"
    print(f"✓ Current {target_chain} end-effector pose: {original_pose}")
    time.sleep(0.8)
except Exception as e:
    print(f"✗ Exception getting end-effector pose by chain name: {e}")

# 2. Solve joint angles based on target pose IK and verify the solution
# 2.1 Solve joint angles joint_angles_ik for target pose through IK
try:
    status, joint_angles_ik = motion.inverse_kinematics(
        target_pose=chain_pose_baselink[target_chain],
        chain_names=[target_chain],
        target_frame=target_frame,
        reference_frame=reference_frame,
        enable_collision_check=False # Disable collision detection
    )
    assert status == gm.MotionStatus.SUCCESS, "IK solving failed"
    print(f"✓ Target {target_chain} IK solving successful joint_angles_ik:
{joint_angles_ik}")
    time.sleep(1)
except Exception as e:
    print(f"✗ IK solving exception: {e}")

# 2.2 Set end-effector pose to target pose tgt_pose_ik by setting joint group
angles joint_angles_ik
try:
    status = robot.set_joint_positions(
        joint_angles_ik[target_chain],
        [target_chain],
        [],
        True,
        0.1,
        20.0,
    )
    assert status == ControlStatus.SUCCESS, "Setting joint group angles failed"
    print(f"✓ Setting {target_chain} joint group angles successful.")
    time.sleep(1)
except Exception as e:
    print(f"✗ Setting {target_chain} joint group angles exception: {e}")

# 2.3 Verify whether the set joint group angles are consistent with the solved
angles
try:
    status, tgt_pose_ik = motion.get_end_effector_pose_on_chain(
        chain_name=target_chain,
        frame_id=target_frame,
        reference_frame=reference_frame
    )
    assert status == gm.MotionStatus.SUCCESS, "Failed to get end-effector pose"
    print(f"✓ Getting {target_chain} end-effector pose successful: {tgt_pose_ik}")
    time.sleep(1)

```

```

        error = calculate_error(tgt_pose_ik, chain_pose_baselink[target_chain])
        print(f"End-effector pose error: {error}")
    except Exception as e:
        print(f"✗ Getting {target_chain} end-effector pose exception: {e}")

# 2.4 Verify whether the end-effector pose tgt_pose_fk corresponding to joint
group angles joint_angles_ik solved by FK is consistent with target pose tgt_pose_ik
try:
    status, tgt_pose_fk = motion.forward_kinematics(
        target_frame=end_link,
        reference_frame=reference_frame,
        joint_state=joint_angles_ik,
        params=gm.Parameter()
    )
    assert status == gm.MotionStatus.SUCCESS, "FK solving failed"
    print(f"✓ Target {target_chain} FK solving successful: {tgt_pose_fk}")
    time.sleep(1)

    error = calculate_error(tgt_pose_fk, chain_pose_baselink[target_chain])
    print(f"FK solving error: {error}")
except Exception as e:
    print(f"✗ FK solving exception: {e}")

time.sleep(3)
print()

# 3. Restore to original pose by setting end-effector pose
# 3.1 Set end-effector pose to restore to original pose
try:
    status = motion.set_end_effector_pose(
        target_pose=original_pose,
        end_effector_frame=target_chain,
        reference_frame=reference_frame,
        enable_collision_check=False,
        is_blocking=True,
        timeout=5.0,
        params=gm.Parameter()
    )
    assert status == gm.MotionStatus.SUCCESS, "Setting end-effector pose failed"
    print(f"✓ Setting end-effector pose successful: status={status}")
    time.sleep(1)
except Exception as e:
    print(f"✗ Setting {target_chain} end-effector pose exception: {e}")

# 3.2 Get end-effector pose and verify whether it has been restored to original
pose
try:
    status, original_pose_rec = motion.get_end_effector_pose_on_chain(
        chain_name=target_chain,
        frame_id=target_frame,
        reference_frame=reference_frame
    )
    assert status == gm.MotionStatus.SUCCESS, "Failed to get end-effector pose"
    print(f"✓ Getting {target_chain} end-effector pose successful:
{original_pose_rec}")
    time.sleep(0.8)

    error = calculate_error(original_pose_rec, original_pose)
    print(f"Restore end-effector pose error: {error}")
except Exception as e:

```

```

        print(f"✗ Setting end-effector pose exception: {e}")

    except Exception as e:
        print(f"✗ Main program exception: {e}")
    finally:
        robot.request_shutdown()
        robot.wait_for_shutdown()
        robot.destroy()

if __name__=="__main__":
    main()

```

Example3. 机器人导航

在运行本示例前，请确认已完成**地图引擎建图与定位**流程：

危险

导航功能依赖地图引擎，必须先完成建图和定位才能运行本示例。

详细操作步骤请参考：[常规操作 - 地图引擎建图与定位](#)

本示例展示了 GALBOT 机器人的导航功能，执行本程序后，机器人在其左前方绕行一个正方形路径。通过初始化导航和机器人实例、检查安全状态后，让机器人执行正方形轨迹运动，每次向前移动 0.5 米并左转 90 度，重复 4 次形成一个正方形路径，并在完成后返回起始位置，整个过程包含了位姿获取、路径可达性检查、导航到目标点、目标到达检测以及异常处理等导航控制功能。

警告

```

examples/python/tutorials/example3_robot_navigation.py
"""
Note: When running this example, please confirm that the robot's navigation function
`/data/galbot/bin/service_navigation_plan` has been loaded;
"""
try:
    from galbot_sdk.g1 import GalbotNavigation
    from galbot_sdk.g1 import GalbotRobot
except ImportError:
    print("Failed to import galbot_sdk, please install it first or check if it's in PYTHONPATH")
    exit(1)

import os

try:
    import numpy as np
except ImportError:
    os.system("pip install numpy")
    import numpy as np

try:
    from scipy.spatial.transform import Rotation as R
except ImportError:
    os.system("pip install scipy")
    from scipy.spatial.transform import Rotation as R

```

```

import time
from typing import Sequence, Dict

def quat_normalize(q: np.ndarray) -> np.ndarray:
    q = np.array(q, dtype=np.float64)
    return q / np.linalg.norm(q)

def quat_conjugate(q: np.ndarray) -> np.ndarray:
    """
    Calculate the conjugate of a quaternion

    Parameters:
        q (np.ndarray): Input quaternion [x, y, z, w]

    Returns:
        np.ndarray: Conjugate of the quaternion [x, y, z, w]
    """
    qx, qy, qz, qw = q
    return np.array([-qx, -qy, -qz, qw])

def quat_multiply(q1: np.ndarray, q2: np.ndarray) -> np.ndarray:
    """
    Calculate the product of two quaternions

    Parameters:
        q1 (np.ndarray): First quaternion [x, y, z, w]
        q2 (np.ndarray): Second quaternion [x, y, z, w]

    Returns:
        np.ndarray: Product of the two quaternions [x, y, z, w]
    """
    x1, y1, z1, w1 = q1
    x2, y2, z2, w2 = q2
    return np.array([
        w1*x2 + x1*w2 + y1*z2 - z1*y2,
        w1*y2 - x1*z2 + y1*w2 + z1*x2,
        w1*z2 + x1*y2 - y1*x2 + z1*w2,
        w1*w2 - x1*x2 - y1*y2 - z1*z2
    ])

def orientation_error_angle(A: np.ndarray, B: np.ndarray) -> float:
    """
    Calculate the rotation angle error between two quaternions (in radians)

    Parameters:
        A (np.ndarray): First quaternion [x, y, z, w]
        B (np.ndarray): Second quaternion [x, y, z, w]

    Returns:
        float: Rotation angle error (in radians)
    """
    qA = quat_normalize(A[3:7])
    qB = quat_normalize(B[3:7])

    q_err = quat_multiply(qB, quat_conjugate(qA))
    q_err = quat_normalize(q_err)

    # Numerically stable
    qw = np.clip(q_err[3], -1.0, 1.0)

```

```

    angle = 2 * np.arccos(qw)
    return angle # Unit: radians

def calculate_error(pose1: Sequence[float], pose2: Sequence[float]) -> Dict[str, float]:
    """
    Calculate the position error and rotation error between two poses (in radians)

    Parameters:
        pose1 (Sequence[float]): First pose, [x, y, z, qx, qy, qz, qw]
        pose2 (Sequence[float]): Second pose, [x, y, z, qx, qy, qz, qw]

    Returns:
        dict: Dictionary containing position error (in meters) and rotation error (in
radians)
    """
    A, B = np.array(pose1), np.array(pose2)
    pos_err = np.linalg.norm(A[:3] - B[:3])
    rot_err = orientation_error_angle(A, B)

    return {
        "position_error_norm": pos_err,
        "orientation_error_rad": rot_err,
        "orientation_error_deg": np.degrees(rot_err)
    }

def local_pose_to_global(start_pose: Sequence[float], local_pose: Sequence[float]):
    """
    Convert local pose to global pose

    Parameters:
        start_pose (Sequence[float]): Start pose, [x, y, z, qx, qy, qz, qw]
        local_pose (Sequence[float]): Local pose, [x, y, z, qx, qy, qz, qw]

    Returns:
        Sequence[float]: Global pose, [x, y, z, qx, qy, qz, qw]
    """
    start_mat = np.eye(4)
    start_mat[:3, :3] = R.from_quat([start_pose[3], start_pose[4], start_pose[5],
start_pose[6]]).as_matrix()
    start_mat[:3, 3] = [start_pose[0], start_pose[1], start_pose[2]]

    local_mat = np.eye(4)
    local_mat[:3, :3] = R.from_quat([local_pose[3], local_pose[4], local_pose[5],
local_pose[6]]).as_matrix()
    local_mat[:3, 3] = [local_pose[0], local_pose[1], local_pose[2]]

    global_mat = start_mat @ local_mat

    return global_mat[:3, 3].tolist() +
R.from_matrix(global_mat[:3, :3]).as_quat().tolist()

def demo_square_move(robot: GalbotRobot, nav: GalbotNavigation):
    """
    Demonstrate robot moving in a square pattern in navigation environment

    Parameters:
        robot (GalbotRobot): Robot instance
        nav (GalbotNavigation): Navigation instance

```

```

"""
try:
    start_pose = nav.get_current_pose()
except Exception as e:
    print(f"Failed to get current pose: {e}")
    return

# Move forward 0.5m, turn left 90 degrees
local_pose = [0.5, 0.0, 0.0, 0.0, 0.0, 0.707, 0.707]

try:
    # Move forward 0.5m each time, turn left 90 degrees, repeat 4 times to form a
square
    for _ in range(4):
        # Calculate target pose
        cur_pose = nav.get_current_pose()
        goal_pose = local_pose_to_global(cur_pose, local_pose)

        # Check if path is reachable
        if nav.check_path_reachability(goal_pose, cur_pose):
            # Navigate to target pose
            retry_cnt = 3
            while True:
                status = nav.navigate_to_goal(goal_pose, enable_collision_check=True,
is_blocking=True, timeout=30)
                time.sleep(0.5)
                retry_cnt -= 1
                if nav.check_goal_arrival() or retry_cnt < 0:
                    break
            else:
                print(f"Navigation failed, retrying...{retry_cnt}")
                print("navigate_to_goal return status:", status)
                print("Has arrived:", nav.check_goal_arrival())
        else:
            print("Path unreachable or unsafe")

        cur_pose = nav.get_current_pose()
        print(f"Current pose: {cur_pose}, Error from start pose: {calculate_error(cur_pose,
start_pose)}")
    except Exception as e:
        print(f"Exception occurred during navigation: {e}")

def move_to_original(robot: GalbotRobot, nav: GalbotNavigation):
    """
    Demonstrate robot returning to start pose in navigation environment

    Parameters:
        robot (GalbotRobot): Robot instance
        nav (GalbotNavigation): Navigation instance
    """
    cur_pose = nav.get_current_pose()
    goal_pose = [0, 0, 0, 0, 0, 0, 1]

    try:
        if nav.check_path_reachability(goal_pose, cur_pose):
            retry_cnt = 3
            while True:
                status = nav.navigate_to_goal(goal_pose, enable_collision_check=True,
is_blocking=True, timeout=30)
                time.sleep(0.5)

```



```

        retry_cnt -= 1
        if nav.check_goal_arrival() or retry_cnt < 0:
            break
        else:
            print(f"Navigation failed, retrying...{retry_cnt}")
            print("navigate_to_goal return status:", status)
            print("Has arrived:", nav.check_goal_arrival())
    else:
        print("Path unreachable or unsafe")
except Exception as e:
    print(f"Exception occurred during navigation: {e}")

def check_robot_safety():
    """Check if robot is safe"""
    # Prompt important notes
    print("    Note: 1. Please ensure the emergency stop button of the robot is released; 2. Please ensure there are no obstructions around the robot to avoid unexpected situations; 3. Please ensure the area around the robot is clear of obstacles.")
    while True:
        key = input("Please confirm that the robot's emergency stop button is released and there are no obstructions, continue? (y/n)...")
        if key == 'y':
            print("User confirmed, continuing...")
            break
        elif key == 'n':
            print("User did not confirm, exiting program...")
            exit(1)
        else:
            print("Invalid input, please enter 'y' or 'n'")

def main():
    check_robot_safety()
    try:
        # Get robot instance
        robot = GalbotRobot()
        # Get navigation instance
        nav = GalbotNavigation()

        # Initialize robot
        if robot.init():
            print("Robot initialization successful")
        else:
            print("Robot initialization failed")
        # Initialize navigation
        if nav.init():
            print("Navigation initialization successful")
        else:
            print("Navigation initialization failed")

        # Wait for data preparation
        time.sleep(1)

        # Check initial localization status
        is_localized = nav.is_localized()
        if not is_localized:
            print("Localization failed, attempting to re-localize: Please move the robot to the origin of the map!")
            time.sleep(3)
            while not is_localized:
                nav.relocalize([0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 1.0])

```

```

        time.sleep(0.5)
        is_localized = nav.is_localized()

    # square_move
    demo_square_move(robot, nav)

except Exception as e:
    print(f"Exception occurred: {e}")
finally:
    robot.request_shutdown()
    robot.wait_for_shutdown()
    robot.destroy()

if __name__ == "__main__":
    main()

```

Example4. 传感器数据获取

本示例展示了 GALBOT 机器人的传感器数据采集功能。通过初始化机器人并启用左臂相机、深度相机、底座激光雷达和躯干 IMU 等传感器，程序获取并处理 RGB 图像、深度图像、激光雷达点云数据和 IMU 数据，对图像进行解码和可视化，将点云数据转换并保存为 PCD 格式文件，同时实现深度图与 RGB 图像融合生成彩色点云的功能。

警告

```

examples/python/tutorials/example4_sensor_data_collect.py
"""
Note: When running this example, please confirm that the robot's left arm camera driver
`/data/galbot/bin/left_arm_camera_capture`
and radar driver `/data/galbot/bin/service_livox_capture` have been loaded;
"""
try:
    from galbot_sdk.g1 import GalbotRobot
    from galbot_sdk.g1 import SensorType
except ImportError:
    print("import galbot_sdk failed, please install it first or check if it is in the
PYTHONPATH")
    exit(1)

import os

try:
    import open3d as o3d
except ImportError:
    os.system("pip install open3d")
    import open3d as o3d

try:
    import cv2
except ImportError:
    os.system("pip install opencv-python")
    import cv2

try:
    import numpy as np

```

```

except ImportError:
    os.system("pip install numpy")
    import numpy as np

import time
from typing import Dict

def convert_pointcloud(cloud):
    """
    Convert cloud dict to NumPy array dictionary

    Parameters:
        cloud (dict): PointCloud2 protobuf message object

    Returns:
        Dictionary: {field_name: NumPy array}
        - Single-element fields: shape (N,)
        - Multi-element fields: shape (N, count) or (N,)
        - N = width * height (total number of points)
    """

    if not cloud:
        return {}

    num_points = cloud["height"] * cloud["width"]
    if num_points == 0:
        return {}

    DTYPE_MAP = {
        1: np.int8,
        2: np.uint8,
        3: np.int16,
        4: np.uint16,
        5: np.int32,
        6: np.uint32,
        7: np.float32,
        8: np.float64
    }
    dtype_list = []
    for field in cloud["fields"]:
        # Get base data type
        np_dtype_class = DTYPE_MAP.get(field["datatype"])
        if np_dtype_class is None:
            raise ValueError(f"Unsupported data type: {field['datatype']}")

        dtype_inst = np.dtype(np_dtype_class)

        # Handle byte order (endianness)
        if dtype_inst.itemsize > 1:
            byteorder = '>' if cloud["is_bigendian"] else '<'
            dtype_inst = dtype_inst.newbyteorder(byteorder)

        # Add to dtype list
        if field["count"] == 1:
            dtype_list.append((field["name"], dtype_inst))
        else:
            # Multi-element fields (e.g., rgb)
            dtype_list.append((field["name"], dtype_inst, field["count"]))

```

```

# Create structured dtype
dtype = np.dtype(dtype_list)

# Data integrity check
expected_size = num_points * cloud["point_step"]
if len(cloud["data"]) < expected_size:
    raise ValueError(
        f"Insufficient data length: expected {expected_size} bytes, "
        f"actual {len(cloud['data'])} bytes"
    )

# Create NumPy structured array from binary data
# count parameter ensures only expected number of points are read
arr = np.frombuffer(cloud["data"], dtype=dtype, count=num_points)

# Convert to regular dictionary (copy data to avoid modifying original)
result = {}
for field in cloud["fields"]:
    field_data = arr[field["name"]]

    # Handle shape of multi-element fields
    if field["count"] == 1:
        result[field["name"]] = field_data.copy()
    else:
        # Keep original shape or flatten, choose according to needs
        result[field["name"]] = field_data.copy()

return result

def get_xyz_array(pointcloud_dict: Dict[str, np.ndarray],
                  remove_nan: bool = False) -> np.ndarray:
    """
    Extract XYZ coordinate array from converted point cloud dictionary

    Parameters:
        pointcloud_dict (Dict[str, np.ndarray]): Dictionary returned by
        pointcloud2_to_numpy()
        remove_nan (bool, optional): Whether to remove points containing NaN (for
        FLOAT32/FLOAT64 types). Defaults to False.

    Returns:
        Nx3 point coordinate array
    """
    required = ['x', 'y', 'z']
    if not all(k in pointcloud_dict for k in required):
        raise ValueError("Point cloud data missing required xyz fields")

    points = np.stack([pointcloud_dict['x'],
                       pointcloud_dict['y'],
                       pointcloud_dict['z']], axis=1)

    if remove_nan:
        mask = ~np.isnan(points).any(axis=1)
        points = points[mask]

    return points

def save_xyz_to_pcd(xyz_array: np.ndarray, filename: str, binary: bool = False) -> None:
    """

```

Save XYZ coordinates to PCD file format (simplest option for coordinate-only data)

Parameters:

xyz_array (np.ndarray): Nx3 array of XYZ coordinates

filename (str): Output PCD file path

binary (bool, optional): If True, saves in binary format; otherwise ASCII.

Defaults to False.

"""

if xyz_array.ndim != 2 or xyz_array.shape[1] != 3:

raise ValueError(f"xyz_array must have shape (N, 3), got {xyz_array.shape}")

num_points = xyz_array.shape[0]

header = [

"# .PCD v0.7 - Point Cloud Data file format",

"VERSION 0.7",

"FIELDS x y z",

"SIZE 4 4 4",

"TYPE F F F", # F = float32

"COUNT 1 1 1",

f"WIDTH {num_points}",

"HEIGHT 1",

"VIEWPOINT 0 0 0 1 0 0 0",

f"POINTS {num_points}",

f"DATA {'binary' if binary else 'ascii'}"

]

if binary:

with open(filename, 'wb') as f:

f.write(('\\n'.join(header) + '\\n').encode('ascii'))

f.write(xyz_array.astype(np.float32).tobytes())

else:

with open(filename, 'w') as f:

f.write('\\n'.join(header) + '\\n')

np.savetxt(f, xyz_array, fmt='%f')

def decode_compressed_image(compressed_image):

"""

decode CompressedImage image

Parameters:

compressed_image (dict): image dict, keys:[header, format, data, "depth_scale"]

Returns:

numpy.ndarray: decoded image

"""

image_data = compressed_image["data"]

if compressed_image["format"] == "rgb8":

return decode_rgb_image(image_data)

elif compressed_image["format"] == "16UC1":

return decode_depth_image(compressed_image)

else:

raise ValueError(f"Unsupport data format: {compressed_image['format']}")

def decode_rgb_image(image_data):

"""decode rgb image"""

nparr = np.frombuffer(image_data, np.uint8)

img = cv2.imdecode(nparr, cv2.IMREAD_COLOR)

if img is None:

raise ValueError("Fail to Decode RGB Image")

```

    return img

def decode_depth_image(image_data):
    """decode depth image"""
    depth_img = np.frombuffer(image_data["data"], dtype=np.uint16).copy()

    # Check if height and width exist
    if "height" not in image_data or "width" not in image_data:
        raise ValueError("Missing 'height' or 'width' in depth image metadata.")
    if image_data["height"] == 0 or image_data["width"] == 0:
        raise ValueError(f"Invalid 'height' ({image_data['height']}) or 'width' ({image_data['width']}) in depth image metadata.")

    # Parse depth image
    depth_img = depth_img.reshape((image_data["height"], image_data["width"]))
    depth_img = depth_img.astype(np.float32) / image_data["depth_scale"]

    return depth_img

def depth_rgb_to_pointcloud(depth, rgb, fx, fy, cx, cy, depth_scale=1.0):
    """
    Convert depth map and RGB image to point cloud

    Parameters:
        depth: (H, W) depth map
        rgb: (H, W, 3) RGB image
        depth_scale: Depth unit scaling (use 0.001 for mm->m conversion)

    Returns:
        points: (N, 3) point cloud coordinate array
        colors: (N, 3) point cloud color array (0-1 range)
    """
    assert depth.shape[:2] == rgb.shape[:2]

    H, W = depth.shape

    # Pixel coordinates
    u, v = np.meshgrid(np.arange(W), np.arange(H))

    # Depth (converted to meters)
    Z = depth.astype(np.float32) * depth_scale

    # Filter invalid depths
    valid = Z > 0

    X = (u - cx) * Z / fx
    Y = (v - cy) * Z / fy

    points = np.stack((X, Y, Z), axis=-1)
    colors = rgb.astype(np.float32) / 255.0

    # Keep only valid points
    points = points[valid]
    colors = colors[valid]

    return points, colors

def check_robot_safety():
    """Check if robot is safe"""
    # Prompt important notes

```

```

    print("    Note: 1. Please ensure the emergency stop button of the robot is released; 2.
Please ensure there are no obstructions around the robot to avoid unexpected situations.")
    while True:
        key = input("Please confirm that the robot's emergency stop button is released and
there are no obstructions, continue? (y/n)...")
        if key == 'y':
            print("User confirmed, continuing...")
            break
        elif key == 'n':
            print("User did not confirm, exiting program...")
            exit(1)
        else:
            print("Invalid input, please enter 'y' or 'n'")

def main():
    SHOW_IMAGE = False
    check_robot_safety()
    try:
        # Get and initialize the GalbotRobot singleton
        robot = GalbotRobot()

        # Get RGB and depth images from the left arm, depth images from the right arm,
        # base LiDAR data, and torso IMU data
        enable_sensor_set = {SensorType.LEFT_ARM_CAMERA, # Left arm depth camera
                             SensorType.LEFT_ARM_DEPTH_CAMERA, # Left arm RGB camera
                             SensorType.BASE_LIDAR,} # Base LiDAR

        # To save resource overhead, only cameras and radar sensors passed during
initialization can acquire data
        robot.init(enable_sensor_set)
        print("Initialization successful")
        # Program starts immediately, wait for data readiness
        time.sleep(5)

        # Get RGB image from the left arm
        rgb_image_data = robot.get_rgb_data(SensorType.LEFT_ARM_CAMERA)
        if not rgb_image_data:
            print("No rgb image data!")
        else:
            print("get rgb image suceess")
            print(rgb_image_data['header'])
            img = decode_compressed_image(rgb_image_data)

            # Save RGB image
            cv2.imwrite("rgb_image_data.jpg", img)
            # Visualize RGB image
            if SHOW_IMAGE:
                cv2.namedWindow("rgb image", cv2.WINDOW_NORMAL)
                cv2.imshow("rgb image", img)
                cv2.waitKey(0)
                cv2.destroyAllWindows()

        # Get depth image from the left arm
        depth_data = robot.get_depth_data(SensorType.LEFT_ARM_DEPTH_CAMERA)
        if not depth_data or "data" not in depth_data:
            print("Depth camera not ready")
        else:
            print("get depth data suceess")
            print(depth_data['header'])
            depth_img_raw = decode_compressed_image(depth_data)

```

```

        depth_img = cv2.normalize(depth_img_raw, None, 0, 255, cv2.NORM_MINMAX) #
Normalize, mapping depth values to 0-1 range
        depth_img = depth_img.astype(np.uint8)

    # Save depth image
    cv2.imwrite("depth_data.jpg", depth_img)
    # Visualize depth image
    if SHOW_IMAGE:
        cv2.namedWindow("depth image", cv2.WINDOW_NORMAL)
        cv2.imshow("depth image", depth_img)
        cv2.waitKey(0)
        cv2.destroyAllWindows()

# Get base LiDAR data
lidar_data = robot.get_lidar_data(SensorType.BASE_LIDAR)
if not lidar_data:
    print("No lidar data!")
else:
    pointcloud_dict = convert_pointcloud(lidar_data)
    xyz_points = get_xyz_array(pointcloud_dict)
    save_xyz_to_pcd(xyz_points, "output_xyz.pcd")
    print(pointcloud_dict)
    print("get lidar data success")

# Visualize LiDAR point cloud
if SHOW_IMAGE:
    vis = o3d.visualization.Visualizer()
    vis.create_window()
    pcd = o3d.geometry.PointCloud()
    pcd.points = o3d.utility.Vector3dVector(xyz_points)
    vis.add_geometry(pcd)
    vis.run()
    vis.destroy_window()

# Get torso IMU data
imu_data = robot.get_imu_data(SensorType.TORSO_IMU)
if not imu_data:
    print("No imu data!")
else:
    print("get imu data suceess")

try:
    camera_info = robot.get_camera_intrinsic(SensorType.LEFT_ARM_DEPTH_CAMERA)
    if not camera_info:
        print("No camera info!")
    else:
        print(camera_info)
except Exception as e:
    camera_info = {
        "width": 1280,
        "height": 720,
        "distortion_model": "plumb_bob",
        "camera_type": "D405",
        "K": [653.4349365234375, 0.0, 639.95159912109375,
              0.0, 652.48858642578125, 365.29425048828125,
              0.0, 0.0, 1.0],
    }

# Convert depth map and RGB image to point cloud and save
if depth_data and rgb_image_data:

```



```

        points, colors = depth_rgb_to_pointcloud(
            depth_img_raw,
            img,
            fx=camera_info['K'][0],
            fy=camera_info['K'][4],
            cx=camera_info['K'][2],
            cy=camera_info['K'][5],
            depth_scale=0.1 # If depth is in mm, set to 0.001
        )
        save_xyz_to_pcd(points, "left_arm_camera_pointcloud.pcd", binary=True)
        print(f"RGB fused depth map point cloud saved to
left_arm_camera_pointcloud.pcd, number of points: {points.shape[0]}")
    except Exception as e:
        print(f"✖ Exception occurred: {e}")
    finally:
        # Actively send SIGINT exit signal
        robot.request_shutdown()
        # Wait to enter shutdown state
        robot.wait_for_shutdown()
        # Release SDK resources
        robot.destroy()
        print('Resource release successful')

if __name__=="__main__":
    main()

```

Example5. 抓取和放置物体

本示例展示了机器人的抓取和放置物体功能，执行本程序后机器人将移动至其左后方约 1 米位置处，尝试虚空抓取物体。通过初始化机器人、导航、运动控制实例以及多个传感器，程序首先导航到目标区域，然后抬起相机到合适高度，利用视觉系统检测目标物体并将其位姿转换到底盘坐标系，接着控制机械臂移动到目标位置进行抓取，完成抓取后导航回到初始位置并将物体放下，整个过程集成了导航、视觉识别、机械臂控制和夹爪操作等多种功能。

警告

examples/python/tutorials/example5_pick_and_place.py

"""

Note: When running this example, please confirm that the robot's motion control service `/data/galbot/bin/service\_motion\_plan`, robot state publishing service `/data/galbot/bin/robot\_state\_publish`, navigation service `/data/galbot/bin/service\_navigation\_plan` and hand-eye calibration publishing service `/data/galbot/bin/eyehand\_calib\_publish` have been loaded;

"""

try:

```

import galbot_sdk.g1 as gm
from galbot_sdk.g1 import GalbotNavigation
from galbot_sdk.g1 import GalbotRobot
from galbot_sdk.g1 import GalbotMotion
from galbot_sdk.g1 import G1JointGroup, SensorType
from galbot_sdk.g1 import ControlStatus

```

except ImportError:

```

    print("Import galbot_sdk failed, please install it first or check if it is in the
PYTHONPATH")
    exit(1)

import os

try:
    import numpy as np
except ImportError:
    os.system("pip install numpy")
    import numpy as np

try:
    from scipy.spatial.transform import Rotation as R
except ImportError:
    os.system("pip install scipy")
    from scipy.spatial.transform import Rotation as R

try:
    import cv2
except ImportError:
    os.system("pip install opencv-python")
    import cv2

import time
from typing import Sequence

def robot_default(robot: GalbotRobot):
    # set initial joint positions
    joint_pos = [0.5, 1.5, 1.0, 0.0, 0.0,
                 0.0, 0.0,
                 2.0, -1.5, -0.6, -1.7, 0.0, -0.8, 0.0,
                 -2.0, 1.5, 0.6, 1.7, 0.0, 0.8, 0.0]
    joint_groups_names = ["leg", "head", "left_arm", "right_arm"]
    joint_names = []
    is_blocking = True
    max_speed_rad_s = 0.1
    timeout_s = 30.0

    status = robot.set_joint_positions(
        joint_pos, joint_groups_names, joint_names, is_blocking, max_speed_rad_s,
        timeout_s
    )

    if status != ControlStatus.SUCCESS:
        print("set join position failed")
    else:
        print("set join position successful")

def decode_compressed_image(compressed_image, camera_info={}):
    """
    decode CompressedImage image

    Parameters:
        compressed_image (dict): image dict, keys:[header, format, data, "depth_scale"]

    Returns:
        numpy.ndarray: decoded image
    """
    image_data = compressed_image["data"]

```

```

    if compressed_image["format"] == "rgb8":
        return decode_rgb_image(image_data)
    elif compressed_image["format"] == "16UC1":
        return decode_depth_image(image_data, compressed_image["depth_scale"], camera_info)
    else:
        raise ValueError(f"Unsupport data format: {compressed_image['format']}")

def decode_rgb_image(image_data):
    """decode rgb image"""
    nparr = np.frombuffer(image_data, np.uint8)
    img = cv2.imdecode(nparr, cv2.IMREAD_COLOR)
    if img is None:
        raise ValueError("Fail to Decode RGB Image")
    return img

def decode_depth_image(image_data, depth_scale, camera_info):
    """decode depth image"""
    depth_img = np.frombuffer(image_data, dtype=np.uint16).copy()

    if not camera_info:
        depth_img = depth_img.reshape((720, 1280))
    else:
        depth_img = depth_img.reshape((camera_info["height"], camera_info["width"]))
    depth_img = depth_img.astype(np.float32) / depth_scale

    return depth_img

def print_gripper_state(joint_group, gripper_state):
    """
    Print gripper state

    Parameters:
        joint_group (G1JointGroup): G1JointGroup enum
        gripper_state (object): Contains timestamp_ns, width, velocity, effort, is_moving
    """
    print(f"Timestamp (ns): {gripper_state.timestamp_ns}")
    print(
        f"width {gripper_state.width} "
        f"velocity {gripper_state.velocity} "
        f"effort {gripper_state.effort} "
        f"is moving {gripper_state.is_moving}"
    )

def get_navigation_pose(object_goal_pose: Sequence[float], motion: GalbotMotion, arm: str = "left_arm"):
    """
    Get navigation target pose

    Parameters:
        object_goal_pose (Sequence[float]): Target pose [x, y, z, qx, qy, qz, qw]
        motion (GalbotMotion): Motion control instance
        arm (str, optional): End effector name. Defaults to "left_arm".

    Returns:
        Sequence[float]: Navigation target pose [x, y, z, qx, qy, qz, qw]
    """
    assert arm in ["left_arm", "right_arm"], "arm must be left_arm or right_arm"

    try:
        status, ee_pose_in_base = motion.get_end_effector_pose(

```

```

        end_effector_frame=f"{arm}_end_effector_mount_link",
        reference_frame="base_link"
    )
    if status != gm.MotionStatus.SUCCESS:
        print(f"Failed to get end effector pose: status={status}")
        offset_y = ee_pose_in_base[1]
    else:
        print(f"Successfully got end effector pose: pose={ee_pose_in_base}")
        offset_y = 0.3

    # Set chassis pose to the same z coordinate as the target pose
    base_goal_pose_mat = np.eye(4)
    base_goal_pose_mat[:3, :3] = R.from_quat(object_goal_pose[3:]).as_matrix()
    base_goal_pose_mat[:3, 3] = np.array([object_goal_pose[0], object_goal_pose[1], 0])

    # According to the relative position of the chassis and camera, move the camera
    navigation target 0.6m backward in the local coordinate to leave observation space for the
    camera
    base_goal_pose_mat = base_goal_pose_mat @ np.array([[1,0,0,-0.6],[0,1,0,-
offset_y],[0,0,1,0],[0,0,0,1]])
    print(base_goal_pose_mat)

    base_goal_pose_quat = R.from_matrix(base_goal_pose_mat[:3, :3]).as_quat()
    base_goal_pose_pos = base_goal_pose_mat[:3, 3]

    return base_goal_pose_pos.tolist() + base_goal_pose_quat.tolist()

except Exception as e:
    print("Failed to get navigation target pose:", e)

def navigation_to_goal(nav: GalbotNavigation, goal_pose: Sequence[float], retry_cnt: int =
3):
    """
    Navigate to target pose

    Parameters:
        nav (GalbotNavigation): Navigation instance
        goal_pose (Sequence[float]): Target pose [x, y, z, qx, qy, qz, qw]
        retry_cnt (int, optional): Number of retries. Defaults to 3.
    """
    try:
        cur_pose = nav.get_current_pose()
        print(f"Current pose: {cur_pose}")
        if nav.check_path_reachability(goal_pose, cur_pose):
            retry_cnt = 3
            while True:
                status = nav.navigate_to_goal(goal_pose, enable_collision_check=True,
is_blocking=True, timeout=20)
                time.sleep(0.5)
                retry_cnt -= 1
                if nav.check_goal_arrival() or retry_cnt < 0:
                    break
            else:
                print(f"Navigation failed: status={status}, retrying: {retry_cnt}")
                print("navigate_to_goal return status:", status)
                print("Has arrived:", nav.check_goal_arrival())
        else:
            print("Path unreachable or unsafe")
    except Exception as e:

```

```

        print(f"Exception occurred during navigation: {e}")

def lift_camera_up(motion: GalbotMotion, target_pose: Sequence[float], target_chain: str,
reference_frame: str):
    """
    Lift camera to target height

    Parameters:
        motion (GalbotMotion): Motion control instance
        target_pose (Sequence[float]): Target pose [x, y, z, qx, qy, qz, qw]
        target_chain (str): End effector name
        reference_frame (str): Reference frame
    """
    try:
        retry_cnt = 3
        while True:
            status, cur_ee_pose = motion.get_end_effector_pose_on_chain(
                chain_name=target_chain,
                frame_id="EndEffector",
                reference_frame=reference_frame
            )
            time.sleep(0.5)
            retry_cnt -= 1

            if status == gm.MotionStatus.SUCCESS or retry_cnt < 0:
                print(f"Current end effector pose: {cur_ee_pose}")
                break
            else:
                print(f"Failed to get end effector pose: status={status}, retrying:
{retry_cnt}")

            tgt_ee_pose = cur_ee_pose.copy()
            tgt_ee_pose[2] = target_pose[2] - 0.1
            print(f"Target end effector pose: {tgt_ee_pose}")

            retry_cnt = 3
            while True:
                status = motion.set_end_effector_pose(
                    target_pose=tgt_ee_pose,
                    end_effector_frame=target_chain,
                    reference_frame=reference_frame,
                    enable_collision_check=False,
                    is_blocking=True,
                    timeout=5.0,
                    params=gm.Parameter()
                )
                time.sleep(0.5)
                retry_cnt -= 1
                if status == gm.MotionStatus.SUCCESS or retry_cnt < 0:
                    print(f"✔ Successfully set end effector pose: status={status}")
                    break
                else:
                    print(f"Failed to set end effector pose: status={status}, retrying:
{retry_cnt}")
            except Exception as e:
                print(f"✖ Failed to lift camera: {e}")

def detect_target(img: np.ndarray, depth_img: np.ndarray) -> Sequence[float]:
    """
    Detection target function. Input RGB image and depth image, output target pose.

```

```

Parameters:
    img (np.ndarray): RGB image
    depth_img (np.ndarray): Depth image

Returns:
    Sequence[float]: Target pose [x, y, z, qx, qy, qz, qw]
"""
try:

    ##### NOTE #####
    # This function is a placeholder. In a real-world scenario, you would implement
    # target detection using computer vision techniques. For this example, we assume
    # a default pose.
    #####

    # Assume detected target pose is [-0.05, -0.1, 0.12, 0.0, 0.0, 0.0, 1.0]
    # Indicates the target is 0.12m in front of the camera, 0.05m to the left, 0.1m in
height, facing the default camera direction
    default_pose = [-0.05, -0.1, 0.12, 0.0, 0.0, 0.0, 1.0]

    return default_pose
except Exception as e:
    print(f"Target detection exception: {e}")
    return None

def pose_camera_to_base(robot: GalbotRobot, pose_camera: Sequence[float]) ->
Sequence[float]:
    """
    Transform camera pose to chassis coordinate system

    Parameters:
        robot (GalbotRobot): Robot instance
        pose_camera (Sequence[float]): Camera pose [x, y, z, qx, qy, qz, qw]

    Returns:
        Sequence[float]: Chassis pose [x, y, z, qx, qy, qz, qw]
    """
    source_frame="left_arm_camera_color_optical_frame"
    target_frame="base_link"
    base_to_cam = robot.get_transform(target_frame, source_frame)[0]
    if base_to_cam is None:
        print("Failed to get transform from camera to chassis")
        return None
    else:
        print("base_to_cam: ", base_to_cam)

    base_to_cam_mat = np.eye(4)
    base_to_cam_mat[:3, :3] = R.from_quat(base_to_cam[3:]).as_matrix()
    base_to_cam_mat[:3, 3] = np.array(base_to_cam[:3])

    pose_camera_mat = np.eye(4)
    pose_camera_mat[:3, :3] = R.from_quat(pose_camera[3:]).as_matrix()
    pose_camera_mat[:3, 3] = np.array(pose_camera[:3])

    pose_base_mat = base_to_cam_mat @ pose_camera_mat[:, 3:]
    print("pose_base_mat: ", pose_base_mat)

    return pose_base_mat.flatten()[:3].tolist() + [0, 0, 0, 1]

```

```

def detect_object(robot: GalbotRobot, arm: str = "left_arm"):
    try:
        # Get camera image data
        if arm == "left_arm":
            rgb_image_data = robot.get_rgb_data(SensorType.LEFT_ARM_CAMERA)
            depth_data = robot.get_depth_data(SensorType.LEFT_ARM_DEPTH_CAMERA)
        elif arm == "right_arm":
            rgb_image_data = robot.get_rgb_data(SensorType.RIGHT_ARM_CAMERA)
            depth_data = robot.get_depth_data(SensorType.RIGHT_ARM_DEPTH_CAMERA)
        else:
            raise ValueError("arm must be left_arm or right_arm")

        # Decode image data
        if not rgb_image_data:
            print("No rgb image data!")
        else:
            print("get rgb image suceess")
            img = decode_compressed_image(rgb_image_data)

        if not depth_data:
            print("No depth_data!")
        else:
            depth_img = decode_compressed_image(depth_data)
            print("get depth data suceess")

        # Detect target
        object_pose_camera = detect_target(img, depth_img)
        if object_pose_camera is None:
            print("Target detection failed")
            return None
        else:
            print(f"object_pose_camera: {object_pose_camera}")

        # Calculate target pose in chassis coordinate system
        object_pose_base = pose_camera_to_base(robot, object_pose_camera)
        print(f"Target pose in chassis coordinate system: {object_pose_base}")

    except Exception as e:
        print(f"Target detection exception: {e}")

    return object_pose_base

def check_robot_safety():
    """Check if robot is safe"""
    # Prompt important notes
    print("    Note: 1. Please ensure the emergency stop button of the robot is released; 2. Please ensure there are no obstructions around the robot to avoid unexpected situations. 3. Please ensure the area around the robot is clear of obstacles.")
    while True:
        key = input("Please confirm that the robot's emergency stop button is released and there are no obstructions, continue? (y/n)...")
        if key == 'y':
            print("User confirmed, continuing...")
            break
        elif key == 'n':
            print("User did not confirm, exiting program...")
            exit(1)
        else:
            print("Invalid input, please enter 'y' or 'n'")

```

```

def pick_and_place(robot: GalbotRobot,
                   nav: GalbotNavigation,
                   motion: GalbotMotion,
                   object_pose_base: Sequence[float],
                   target_chain: str,
                   reference_frame: str):
    try:
        # Open left gripper
        # Set left gripper width to 0.1m, speed to 0.05m, force to 10N, will block until
gripper reaches position
        status = robot.set_gripper_command(
            G1JointGroup.left_gripper, 0.1, 0.05, 10, False
        )
        time.sleep(0.5)

        print("object_pose_base: ", object_pose_base)

        # Reach to target position
        retry_cnt = 3
        while True:
            status = motion.set_end_effector_pose(
                target_pose=object_pose_base,
                end_effector_frame=target_chain,
                reference_frame=reference_frame,
                enable_collision_check=False,
                is_blocking=True,
                timeout=5.0,
                params=gm.Parameter()
            )
            time.sleep(1)
            retry_cnt -= 1
            if status == gm.MotionStatus.SUCCESS or retry_cnt < 0:
                break
            else:
                print(f"Failed to set end effector pose: status={status}, retry count:
{retry_cnt}")

        assert status == gm.MotionStatus.SUCCESS, "Failed to set end effector pose"
        print(f"✔ Successfully set end effector pose: status={status}")

        # Close gripper to grasp object
        status = robot.set_gripper_command(
            G1JointGroup.left_gripper, 0.02, 0.05, 10, False
        )
        time.sleep(0.5)

        # Return to initial position
        navigation_to_goal(nav, [0, 0, 0, 0, 0, 0, 1])
        time.sleep(2)

        # Release target
        status = robot.set_gripper_command(
            G1JointGroup.left_gripper, 0.1, 0.05, 10, False
        )
        time.sleep(0.5)
    except Exception as e:
        print(f"Exception occurred during pick_and_place: {e}")
        return None

```



```

def main():
    check_robot_safety()
    try:
        # Get robot instance
        robot = GalbotRobot()
        # Get GalbotMotion instance
        motion = GalbotMotion()
        # Get navigation instance
        nav = GalbotNavigation()

        # Get RGB and depth images from left arm, depth images from right arm,
        # base LiDAR data, and torso IMU data
        enable_sensor_set = {SensorType.LEFT_ARM_CAMERA, # Left arm depth camera
                             SensorType.LEFT_ARM_DEPTH_CAMERA, # Left arm RGB camera
                             SensorType.BASE_LIDAR, # Base LiDAR
                             SensorType.TORSO_IMU} # Torso IMU sensor

        # Initialize robot
        if robot.init(enable_sensor_set):
            print("GalbotRobot initialization successful")
        else:
            print("GalbotRobot initialization failed")
        if motion.init():
            print("GalbotMotion initialization successful")
        else:
            print("GalbotMotion initialization failed")
        if nav.init():
            print("GalbotNavigation initialization successful")
        else:
            print("GalbotNavigation initialization failed")

        # Program starts immediately, wait for data readiness
        time.sleep(1)

        robot_default(robot);

        # Calculate navigation target pose
        object_goal_pose = [-1, 0.33, 0.90, 0, 0, 1, 0]
        base_goal_pose = [0, 0, 0, 0, 0, 0, 1]
        base_goal_pose = get_navigation_pose(object_goal_pose, motion)

        # Navigate to target pose
        navigation_to_goal(nav, base_goal_pose)

        target_chain = "left_arm"
        reference_frame = "base_link"

        # Get current end effector pose for subsequent restoration
        try:
            status, original_pose = motion.get_end_effector_pose_on_chain(
                chain_name=target_chain,
                frame_id="EndEffector",
                reference_frame=reference_frame
            )
            assert status == gm.MotionStatus.SUCCESS, "Failed to get end effector pose"
            print(f"✓ Successfully got {target_chain} end effector pose: {original_pose}")
            time.sleep(1)
        except Exception as e:
            print(f"✗ {target_chain} end effector pose exception: {e}")

```

```

# Lift camera
lift_camera_up(motion, object_goal_pose, "left_arm", "base_link")

# Detect target
object_pose_base = detect_object(robot, arm="left_arm")
if object_pose_base is None:
    print("Target detection failed")
    return None
else:
    print(f"Detected target pose: {object_pose_base}")

# Grasp and return to initial position
pick_and_place(robot, nav, motion, object_pose_base, "left_arm", "base_link")

# After grasping target, restore posture
try:
    time.sleep(2)
    status = motion.set_end_effector_pose(
        target_pose=original_pose,
        end_effector_frame=target_chain,
        reference_frame=reference_frame,
        enable_collision_check=False,
        is_blocking=True,
        timeout=5.0,
        params=gm.Parameter()
    )
    assert status == gm.MotionStatus.SUCCESS, "Failed to set end effector pose"
    print(f"✓ Successfully set end effector pose: status={status}")
    time.sleep(1)
except Exception as e:
    print(f"✗ {target_chain} end effector pose exception: {e}")

except Exception as e:
    print(f"Exception occurred: {e}")
finally:
    # Actively send SIGINT exit signal
    robot.request_shutdown()
    # Wait to enter shutdown state
    robot.wait_for_shutdown()
    # Release SDK resources
    robot.destroy()
    print('Resource release successful')

if __name__ == "__main__":
    main()

```

Example6. 执行模拟 VLA 结果

本示例是一个 VLA（视觉-语言-动作）模型的执行示例，旨在演示如何利用机器人视觉传感器数据驱动机器人执行相应动作。脚本首先初始化机器人系统并启用多个摄像头传感器，然后获取并解码视觉数据，通过一个模拟的 VLA 模型（fake_vla 函数）生成机器人各部分（腿部、头部、双臂）的运动轨迹，接着进行碰撞检测确保安全后执行这些轨迹。运行此脚本时，机器人会先进行安全确认，然后各关节按照预定轨迹移动，在执行过程中可以观察到机器人各部分按预定路径运动的现象，最终完成一

系列协调的动作序列，同时输出执行状态和关节状态信息。

警告

```
examples/python/tutorials/example6_execute_vla.py
"""
Note: When running this example, please ensure the robot's `emergency stop button` is
released;
"""
import time
import os
from typing import List, Dict, Any

try:
    import galbot_sdk.g1 as gm
    from galbot_sdk.g1 import GalbotRobot
    from galbot_sdk.g1 import GalbotMotion
    from galbot_sdk.g1 import GalbotNavigation
    from galbot_sdk.g1 import (
        SensorType, G1JointGroup, ControlStatus,
        Trajectory, TrajectoryPoint, JointCommand
    )
except ImportError:
    print("Failed to import galbot_sdk, please install it first or check if it is in
PYTHONPATH")
    exit(1)

try:
    import cv2
except ImportError:
    os.system("pip install opencv-python")
    import cv2

try:
    import numpy as np
except ImportError:
    os.system("pip install numpy")
    import numpy as np

def decode_compressed_image(compressed_image: Dict[str, Any]) -> np.ndarray:
    """
    Decode CompressedImage image.

    Parameters:
        compressed_image (dict): image dict, keys: [header, format, data, "depth_scale"]

    Returns:
        numpy.ndarray: decoded image
    """
    image_data = compressed_image["data"]
    if compressed_image["format"] == "rgb8":
        return decode_rgb_image(image_data)
    elif compressed_image["format"] == "16UC1":
        return decode_depth_image(image_data)
    else:
        raise ValueError(f"Unsupported data format: {compressed_image['format']}")
```

```

def decode_rgb_image(image_data: bytes) -> np.ndarray:
    """
    Decode RGB image.

    Parameters:
        image_data (bytes): Raw image data

    Returns:
        numpy.ndarray: Decoded RGB image
    """
    nparr = np.frombuffer(image_data, np.uint8)
    img = cv2.imdecode(nparr, cv2.IMREAD_COLOR)
    if img is None:
        raise ValueError("Failed to decode RGB Image")
    return img


def decode_depth_image(image_data: Dict[str, Any]) -> np.ndarray:
    """
    Decode depth image.

    Parameters:
        image_data (dict): Depth image data with metadata

    Returns:
        numpy.ndarray: Decoded depth image
    """
    depth_img = np.frombuffer(image_data["data"], dtype=np.uint16).copy()

    # Check if height and width exist
    if "height" not in image_data or "width" not in image_data:
        raise ValueError("Missing 'height' or 'width' in depth image metadata.")
    if image_data["height"] == 0 or image_data["width"] == 0:
        raise ValueError(
            f"Invalid 'height' ({image_data['height']}) or 'width' ({image_data['width']})"
            "in depth image metadata."
        )

    # Parse depth image
    depth_img = depth_img.reshape((image_data["height"], image_data["width"]))
    depth_img = depth_img.astype(np.float32) / image_data["depth_scale"]

    return depth_img


def fake_vla(rgb_data_dict: dict) -> Dict[str, np.ndarray]:
    """
    Fake VLA (Vision-Language-Action) model implementation.

    This function is a placeholder. In a real-world scenario, you would implement
    target detection using computer vision techniques. For this example, we assume
    a default pose.

    Parameters:
        rgb_data_dict (dict): Dictionary containing RGB image data from various sensors

    Returns:
        dict: Trajectories for different robot components (legs, head, arms)
    """

```

```

print("Fake VLA executing...")

# Define trajectories for different robot parts
right_arm_traj = np.linspace(
    [-2.0, 1.59, 0.6, 1.7, 0.0, 0.8, 0.0],
    [-1.5, 1.59, 0.6, 1.5, 0.0, 0.6, 0.0],
    num=200,
)

left_arm_traj = np.linspace(
    [1.9999, -1.6000, -0.5999, -1.6999, 0.0000, -0.7999, 0.0000],
    [1.9999, -1.6000, -0.5999, -1.6999, 0.0000, -0.7999, 0.0000],
    num=200,
)

head_traj = np.linspace(
    [0.0, 0.0],
    [0.0, 0.0],
    num=200,
)

leg_traj = np.linspace(
    [0.299, 1.199, 0.849, 0.0000, 0.0],
    [0.299, 1.199, 0.849, 0.0000, 0.0],
    num=200,
)

return {
    "leg": leg_traj,
    "head": head_traj,
    "left_arm": left_arm_traj,
    "right_arm": right_arm_traj,
}

def estimate_vla(robot: 'GalbotRobot', enable_sensor_set: set) -> Dict[str, np.ndarray]:
    """
    Estimate VLA (Vision-Language-Action) model outputs based on sensor data.

    Parameters:
        robot (GalbotRobot): GalbotRobot instance
        enable_sensor_set (set): Set of enabled sensor types

    Returns:
        dict: Joint positions for each joint group
    """
    # Get RGB images from enabled cameras
    rgb_data_dict = {}
    for sensor_type in enable_sensor_set:
        if sensor_type in [
            SensorType.LEFT_ARM_CAMERA,
            SensorType.RIGHT_ARM_CAMERA,
            SensorType.HEAD_LEFT_CAMERA,
            SensorType.HEAD_RIGHT_CAMERA
        ]:
            # Get RGB data of sensor type
            rgb_data = robot.get_rgb_data(sensor_type)
            time.sleep(1)

            # Decode RGB image

```

```

        if rgb_data:
            rgb_image = decode_compressed_image(rgb_data)
            rgb_data_dict[sensor_type] = rgb_image
        else:
            print(f"Failed to get RGB data from {sensor_type}")
    else:
        print(f"Unsupported sensor type: {sensor_type}")

    print("RGB data dictionary keys:", rgb_data_dict.keys())

    # Get joint position list
    joint_positions = fake_vla(rgb_data_dict)
    return joint_positions

def generate_target_point(q: List[float], target_time: float = 10.0) -> 'TrajectoryPoint':
    """
    Generate target points for trajectory execution.

    Parameters:
        q (List[float]): Joint positions
        target_time (float): Time from start in seconds (default: 10.0)

    Returns:
        TrajectoryPoint: Target trajectory point with specified joint commands
    """
    joint_position = TrajectoryPoint()
    joint_position.time_from_start_second = target_time
    joint_command_vec = []
    for joint in q:
        joint_cmd = JointCommand()
        joint_cmd.position = joint
        joint_command_vec.append(joint_cmd)
    joint_position.joint_command_vec = joint_command_vec
    return joint_position

def generate_target_trajectory(
    trajectory: np.ndarray,
    joint_groups: List[str] = None,
    joint_names: List[str] = None,
    dt: float = 0.008
) -> 'Trajectory':
    """
    Generate trajectory for joints.

    Parameters:
        trajectory (np.ndarray): 2D array of joint positions over time
        joint_groups (List[str]): List of joint groups
        joint_names (List[str]): List of joint names
        dt (float): Time step between trajectory points (default: 0.008)

    Returns:
        Trajectory: Generated trajectory for execution
    """
    if joint_groups is None:
        joint_groups = []
    if joint_names is None:
        joint_names = []

```

```

if trajectory is None or np.ndim(trajectory) != 2 or len(trajectory) == 0:
    return None

# Create Trajectory
traj = Trajectory()
traj.joint_groups = joint_groups
traj.joint_names = joint_names

current_time = 0.0
points = []
for state in trajectory:
    current_time += dt
    # Generate target point for each joint
    traj_point = generate_target_point(state, current_time)
    points.append(traj_point)

traj.points = points
return traj

def print_joint_states(joint_states: List['JointState']) -> None:
    """
    Print joint states in a readable format.

    Parameters:
        joint_states (List[JointState]): List of joint states
    """
    for js in joint_states:
        print(
            f" : position = {js.position} , velocity = {js.velocity} "
            f", acceleration = {js.acceleration} , effort = {js.effort} , current = "
            f"{js.current}"
        )

def check_robot_safety() -> None:
    """
    Check if the robot is safe to operate.
    Prompts the user to confirm safety conditions before proceeding.
    """
    # Prompt for precautions
    print(
        "    Note: 1. Please ensure the robot's emergency stop button is released; "
        "2. Please ensure there are no obstacles in front, back, left, and right "
        "of the robot to avoid unexpected situations."
    )

    while True:
        key = input(
            "Please confirm that the robot's emergency stop button is released "
            "and there are no obstacles. Continue? (y/n)..."
        ).lower()

        if key == 'y':
            print("User confirmed, continuing execution...")
            break
        elif key == 'n':
            print("User not confirmed, program exiting...")
            exit(1)
        else:

```

```

        print("Input error, please enter 'y' or 'n'")

def main() -> None:
    """
    Main function to execute the VLA (Vision-Language-Action) example.
    Initializes the robot, estimates actions based on sensor data, and executes
    trajectories.
    """
    check_robot_safety()

    robot = None # Initialize robot variable for cleanup in finally block

    try:
        # Get robot instances
        robot = GalbotRobot()
        motion = GalbotMotion()
        navi = GalbotNavigation()

        # Enable required sensors
        enable_sensor_set = {
            SensorType.LEFT_ARM_CAMERA,
            SensorType.RIGHT_ARM_CAMERA,
            SensorType.HEAD_LEFT_CAMERA,
            SensorType.HEAD_RIGHT_CAMERA
        }

        # Initialize robot components
        if not robot.init(enable_sensor_set):
            print("Robot initialization failed")
            exit(1)
        else:
            print("Robot initialization successful")

        if not motion.init():
            print("Motion initialization failed")
            exit(1)
        else:
            print("Motion initialization successful")

        if not navi.init():
            print("Navigation initialization failed")
            exit(1)
        else:
            print("Navigation initialization successful")

        # Wait for data preparation
        time.sleep(3)

        # Estimate VLA actions
        joint_positions = estimate_vla(robot, enable_sensor_set)

        # Generate target trajectory
        joint_groups = list(joint_positions.keys())
        joint_traj = np.concatenate(list(joint_positions.values()), axis=-1)

        # Final joint position check state
        whole_body_joint = joint_traj[-1]
        base_state = [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 1.0]
        check_state = gm.RobotStates()

```



```

check_state.whole_body_joint = whole_body_joint
check_state.base_state = base_state
print(f"✓ Final joint position check state: {whole_body_joint}")

# Check collision
status, collision_res = motion.check_collision(
    start=[check_state],
    enable_collision_check=True
)
time.sleep(1)

if status == gm.MotionStatus.SUCCESS:
    print(f"✓ OK: collision check finished: {collision_res} (False=no collision)")

    # Execute trajectory
    trajectory = generate_target_trajectory(joint_traj, joint_groups, [])
    if trajectory is not None:
        status = robot.execute_joint_trajectory(trajectory, True)
        time.sleep(1)

        if status == ControlStatus.SUCCESS:
            print("✓ Joint trajectory execution successful.")
        else:
            print("✗ Joint trajectory execution failed.")

        # Check joint state
        joint_states = robot.get_joint_states(joint_groups, [])
        print_joint_states(joint_states)
        final_positions = [getattr(js, "position", js) for js in joint_states]
        print(f"✓ Final joint position check state after execution (positions):
{final_positions}")
    else:
        print("✗ Generated trajectory is invalid, cannot execute.")
else:
    print("✗ Collision check failed, will not execute the joint trajectory.")

except Exception as e:
    print(f"An exception occurred: {e}")
    import traceback
    traceback.print_exc()
finally:
    # Ensure proper cleanup
    if robot is not None:
        # Actively send SIGINT shutdown signal
        robot.request_shutdown()
        # Wait to enter shutdown state
        robot.wait_for_shutdown()
        # Release SDK resources
        robot.destroy()
    print('Resource release successful')

if __name__ == "__main__":
    main()

```

Example7. 双臂协同控制

本示例是一个机器人臂操控演示程序，旨在展示如何通过设置关节角度序列来控制机器人的左右臂执行特定的动作序列。脚本首先进行安全检查并初始化机器人系统，然后获取当前关节位置作为原始姿态，接着让机器人双臂执行预设的抬升动作序列（将关节移动到特定角度），在完成动作展示后自动恢复到原始姿态。运行此脚本时，可以看到机器人双臂按照预定的角度序列执行抬升动作，在保持动作 15 秒后恢复初始状态，同时终端会输出执行过程中的状态信息、关节位置数据以及成功或失败的提示。

警告

```
examples/python/tutorials/example7_arm_manipulation2.py
"""
Note: When running this example, please ensure the robot's `emergency stop button` is
released;
"""
import time

try:
    from galbot_sdk.g1 import GalbotRobot
    from galbot_sdk.g1 import ControlStatus
except ImportError:
    print("Failed to import galbot_sdk, please install it first or check if it is in
PYTHONPATH")
    exit(1)

def set_joint_positions(robot: GalbotRobot,
                        joint_group_names: list,
                        position_seq: list,
                        is_blocking: bool,
                        max_speed: float,
                        timeout_s: float,
                        retry_count: int = 3
                        ):
    """
    Robot arm manipulation demonstration function

    Parameters:
        robot (GalbotRobot): GalbotRobot instance
        joint_group_names (list): List of joint group names to control
        position_seq (list): List of joint group angle sequences to set
        is_blocking (bool): Whether to set angles in blocking mode
        max_speed (float): Maximum speed
        timeout_s (float): Timeout time (seconds)
        retry_count (int, optional): Number of retries, default is 3

    Returns:
        None
    """

    # Get current joint group angles for subsequent restoration
    original_pos = robot.get_joint_positions(joint_group_names, [])
    print(f"Current angles of joint group {joint_group_names}: {original_pos}")
```

```

# Start arm manipulation gesture
pos_idx = 0
print("Starting arm manipulation gesture...")
while True:
    time.sleep(1)
    pos = position_seq[pos_idx]
    control_status = robot.set_joint_positions(
        pos, joint_group_names, [], is_blocking, max_speed, timeout_s
    )

    # If setting fails, retry 3 times
    retry_cnt = retry_count
    while control_status != ControlStatus.SUCCESS and retry_cnt > 0:
        print(f"Setting angles for joint group {joint_group_names} failed, retrying
{retry_cnt}...")
        retry_cnt = retry_cnt - 1
        time.sleep(1)
        control_status = robot.set_joint_positions(
            pos, joint_group_names, [], is_blocking, max_speed, timeout_s
        )
    # If successful, switch to next pose sequence
    if control_status == ControlStatus.SUCCESS:
        print(f"✓ Setting angles for joint group {joint_group_names} successful")
        pos_idx = pos_idx + 1
    # If failed, break the loop
    else:
        print(f"✗ Setting angles for joint group {joint_group_names} failed")
        break

    # If all pose sequences are completed, break the loop
    if pos_idx > len(position_seq) - 1:
        break

# Get current joint group angles
print("Showing arm manipulation gesture for 15 seconds, then restoring original
pose...")
time.sleep(5)

# Restore original pose joint group angles
control_status = robot.set_joint_positions(
    original_pos, joint_group_names, [], is_blocking, max_speed, timeout_s
)
# If setting fails, retry 5 times
retry_cnt = retry_count
while control_status != ControlStatus.SUCCESS and retry_cnt > 0:
    print(f"Setting angles for joint group {joint_group_names} failed, retrying
{retry_cnt}...")
    retry_cnt = retry_cnt - 1
    time.sleep(2)
    control_status = robot.set_joint_positions(
        original_pos, joint_group_names, [], is_blocking, max_speed, timeout_s
    )
# If successful, restore original pose
if control_status == ControlStatus.SUCCESS:
    print(f"✓ Restoring angles for joint group {joint_group_names} successful")
else:
    print(f"✗ Restoring angles for joint group {joint_group_names} failed")

def print_joint_positions(joint_positions):

```

```

"""
joint_positions: List of joint positions

Parameters:
    joint_positions (List[float]): List of joint positions
"""
print(f"pos count is {len(joint_positions)}")
for pos in joint_positions:
    print(pos)

def check_robot_safety():
    """Check if the robot is safe"""
    # Prompt for precautions
    print("    Note: 1. Please ensure the robot's emergency stop button is released; 2. Please ensure there are no obstacles in front, back, left, and right of the robot to avoid unexpected situations.")
    while True:
        key = input("Please confirm that the robot's emergency stop button is released and there are no obstacles. Continue? (y/n)...")
        if key == 'y':
            print("User confirmed, continuing execution...")
            break
        elif key == 'n':
            print("User not confirmed, program exiting...")
            exit(1)
        else:
            print("Input error, please enter 'y' or 'n'")

def main():
    check_robot_safety()

    try:
        # Get robot instance
        robot = GalbotRobot()

        # Initialize robot
        state = robot.init()
        if not state:
            print(f"Initialization failed")
            exit(1)
        else:
            print(f"Initialization successful")
            print(f"Is robot running: {robot.is_running()}")

        # Wait for data preparation
        time.sleep(3)

        # Get list of joint names
        joint_names = robot.get_joint_names()
        if len(joint_names) > 0:
            print(f"List of joint names: {joint_names}")
        else:
            print(f"Failed to get list of joint names")

        # Get joint positions using joint group names, empty returns all joints by default
        joint_group_names = ["left_arm", "right_arm"]
        # Left and right arm arm manipulation gesture sequence
        position_seq = [
            [1.53, 0.36, -2.54, -1.80, 0.12, -0.82, 0.09, # left_arm
             -1.53, -0.36, 2.54, 1.80, -0.12, 0.82, -0.09] # right_arm

```

```

]
# Whether to block and wait for joints to reach position
is_blocking = True
# Limit maximum joint speed to 0.1rad/s
max_speed = 0.1
# Maximum blocking wait time
timeout_s = 20

# The arms perform lift-up gesture
set_joint_positions(robot, joint_group_names, position_seq,
                    is_blocking, max_speed, timeout_s)

# Get joint positions again to verify the lift-up gesture
joint_positions = robot.get_joint_positions(joint_group_names, [])
print_joint_positions(joint_positions)

except Exception as e:
    print(f"An exception occurred: {e}")
finally:
    # Actively send SIGINT shutdown signal
    robot.request_shutdown()
    # Wait to enter shutdown state
    robot.wait_for_shutdown()
    # Release SDK resources
    robot.destroy()
    print('Resource release successful')

if __name__ == "__main__":
    main()

```

Example8. 实时控制循环

Waiting...

警告

```

examples/python/tutorials/example8_real_time_control_loop.py
"""
Note: When running this example, please ensure the robot's `emergency stop button` is
released;
"""
import time

try:
    from galbot_sdk.g1 import GalbotRobot
    from galbot_sdk.g1 import ControlStatus
except ImportError:
    print("Failed to import galbot_sdk, please install it first or check if it is in
PYTHONPATH")
    exit(1)

def demo_heart_pose(robot: GalbotRobot,
                    joint_group_names: list,
                    position_seq: list,
                    is_blocking: bool,
                    max_speed: float,

```

```

        timeout_s: float,
        retry_count: int = 3
    ):
"""
Robot heart gesture demonstration function

Parameters:
    robot (GalbotRobot): GalbotRobot instance
    joint_group_names (list): List of joint group names to control
    position_seq (list): List of joint group angle sequences to set
    is_blocking (bool): Whether to set angles in blocking mode
    max_speed (float): Maximum speed
    timeout_s (float): Timeout time (seconds)
    retry_count (int, optional): Number of retries, default is 3

Returns:
    None
"""

# Get current joint group angles for subsequent restoration
original_pos = robot.get_joint_positions(joint_group_names, [])
print(f"Current angles of joint group {joint_group_names}: {original_pos}")

# Start heart gesture
pos_idx = 0
print("Starting heart gesture...")
while True:
    time.sleep(1)
    pos = position_seq[pos_idx]
    control_status = robot.set_joint_positions(
        pos, joint_group_names, [], is_blocking, max_speed, timeout_s
    )

    # If setting fails, retry 3 times
    retry_cnt = retry_count
    while control_status != ControlStatus.SUCCESS and retry_cnt > 0:
        print(f"Setting angles for joint group {joint_group_names} failed, retrying
{retry_cnt}...")
        retry_cnt = retry_cnt - 1
        time.sleep(1)
        control_status = robot.set_joint_positions(
            pos, joint_group_names, [], is_blocking, max_speed, timeout_s
        )

    # If successful, switch to next pose sequence
    if control_status == ControlStatus.SUCCESS:
        print(f"Setting angles for joint group {joint_group_names} successful")
        pos_idx = pos_idx + 1
    # If failed, break the loop
    else:
        print(f"Setting angles for joint group {joint_group_names} failed")
        break

    # If all pose sequences are completed, break the loop
    if pos_idx > len(position_seq) - 1:
        break

# Get current joint group angles
print("Showing heart gesture for 15 seconds, then restoring original pose...")
time.sleep(5)

```

```

# Restore original pose joint group angles
control_status = robot.set_joint_positions(
    original_pos, joint_group_names, [], is_blocking, max_speed, timeout_s
)
# If setting fails, retry 5 times
retry_cnt = retry_count
while control_status != ControlStatus.SUCCESS and retry_cnt > 0:
    print(f"Setting angles for joint group {joint_group_names} failed, retrying
{retry_cnt}...")
    retry_cnt = retry_cnt - 1
    time.sleep(2)
    control_status = robot.set_joint_positions(
        original_pos, joint_group_names, [], is_blocking, max_speed, timeout_s
    )
# If successful, restore original pose
if control_status == ControlStatus.SUCCESS:
    print(f"Restoring angles for joint group {joint_group_names} successful")
else:
    print(f"Restoring angles for joint group {joint_group_names} failed")

def check_robot_safety():
    """Check if the robot is safe"""
    # Prompt for precautions
    print("    Note: 1. Please ensure the robot's emergency stop button is released; 2.
Please ensure there are no obstacles in front, back, left, and right of the robot to avoid
unexpected situations.")
    while True:
        key = input("Please confirm that the robot's emergency stop button is released and
there are no obstacles. Continue? (y/n)...")
        if key == 'y':
            print("User confirmed, continuing execution...")
            break
        elif key == 'n':
            print("User not confirmed, program exiting...")
            exit(1)
        else:
            print("Input error, please enter 'y' or 'n'")

def main():
    check_robot_safety()

    try:
        # Get robot instance
        robot = GalbotRobot()

        # Initialize robot
        state = robot.init()
        if not state:
            print(f"Initialization failed")
            exit(1)
        else:
            print(f"Initialization successful")
            print(f"Is robot running: {robot.is_running()}")

        # Wait for data preparation
        time.sleep(3)

        # Get list of joint names
        joint_names = robot.get_joint_names()
        if len(joint_names) > 0:

```

```

        print(f"List of joint names: {joint_names}")
    else:
        print(f"Failed to get list of joint names")

    # Get joint positions using joint group names, empty returns all joints by default
    joint_group_names = ["left_arm", "right_arm"]
    # Left and right arm heart gesture sequence
    position_seq = [
        [1.53, 0.36, -2.54, -1.80, 0.12, -0.82, 0.09, # left_arm
         -1.53, -0.36, 2.54, 1.80, -0.12, 0.82, -0.09] # right_arm
    ]
    # Whether to block and wait for joints to reach position
    is_blocking = True
    # Limit maximum joint speed to 0.1rad/s
    max_speed = 0.1
    # Maximum blocking wait time
    timeout_s = 20

    # Perform heartbeat gesture
    demo_heart_pose(robot, joint_group_names, position_seq,
                    is_blocking, max_speed, timeout_s)

except Exception as e:
    print(f"An exception occurred: {e}")
finally:
    # Actively send SIGINT shutdown signal
    robot.request_shutdown()
    # Wait to enter shutdown state
    robot.wait_for_shutdown()
    # Release SDK resources
    robot.destroy()
    print('Resource release successful')

if __name__ == "__main__":
    main()

```

Example9. 复杂多路点轨迹规划

本示例是演示 Galbot 机器人多路径点规划功能，通过在笛卡尔空间和关节空间中设置多个中间路径点来实现复杂轨迹规划；脚本主要进行了初始化机器人系统、添加和清除障碍物、执行笛卡尔空间多路径点规划（左臂末端执行器沿直线移动）和关节空间多路径点规划（左臂关节角度平滑变化）等操作；运行时会观察到机器人左臂按照预设路径点逐步移动，在终端显示各种状态信息和执行结果；最终会输出规划成功与否的结果信息，并清理所有障碍物后安全关闭机器人系统。

警告

examples/python/tutorials/example9_multiple_waypoints_planning.py

```
import time
```

```
try:
```

```

    import galbot_sdk.g1 as gm
    from galbot_sdk.g1 import GalbotRobot
    from galbot_sdk.g1 import GalbotMotion

```



```

from galbot_sdk.g1 import GalbotNavigation
from galbot_sdk.g1 import SensorType, G1JointGroup, ControlStatus, Trajectory,
TrajectoryPoint, JointCommand
except ImportError:
    print("Failed to import galbot_sdk, please install it first or check if it is in
PYTHONPATH")
    exit(1)

def printStatus(status):
    if(status == gm.MotionStatus.SUCCESS):
        print("Execute result: SUCCESS, completed successfully")
    elif(status == gm.MotionStatus.TIMEOUT):
        print("Execute result: TIMEOUT, timeout occurred")
    elif(status == gm.MotionStatus.FAULT):
        print("Execute result: FAULT, fault occurred, cannot continue")
    elif(status == gm.MotionStatus.INVALID_INPUT):
        print("Execute result: INVALID_INPUT, input parameters do not meet
requirements")
    elif(status == gm.MotionStatus.INIT_FAILED):
        print("Execute result: INIT_FAILED, internal communication component creation
failed")
    elif(status == gm.MotionStatus.IN_PROGRESS):
        print("Execute result: IN_PROGRESS, motion is in progress but not yet reached")
    elif(status == gm.MotionStatus.STOPPED_UNREACHED):
        print("Execute result: STOPPED_UNREACHED, stopped but not yet reached target")
    elif(status == gm.MotionStatus.DATA_FETCH_FAILED):
        print("Execute result: DATA_FETCH_FAILED, data fetch failed")
    elif(status == gm.MotionStatus.PUBLISH_FAIL):
        print("Execute result: PUBLISH_FAIL, data publish failed")
    elif(status == gm.MotionStatus.COMM_DISCONNECTED):
        print("Execute result: COMM_DISCONNECTED, communication disconnected")

def check_robot_safety() -> None:
    """
    Check if the robot is safe to operate.
    Prompts the user to confirm safety conditions before proceeding.
    """
    # Prompt for precautions
    print(
        "    Note: 1. Please ensure the robot's emergency stop button is released; "
        "2. Please ensure there are no obstacles in front, back, left, and right "
        "of the robot to avoid unexpected situations."
    )

    while True:
        key = input(
            "Please confirm that the robot's emergency stop button is released "
            "and there are no obstacles. Continue? (y/n)..."
        ).lower()

        if key == 'y':
            print("User confirmed, continuing execution...")
            break
        elif key == 'n':
            print("User not confirmed, program exiting...")
            exit(1)
        else:
            print("Input error, please enter 'y' or 'n'")

def main():

```

```

check_robot_safety()
try:
    # Get GalbotMotion and GalbotRobot singletons
    robot = GalbotRobot()
    motion = GalbotMotion()
    nav = GalbotNavigation()

    if robot.init():
        print("GalbotRobot initialized successfully")
    else:
        print("GalbotRobot initialization failed")
    if motion.init():
        print("GalbotMotion initialized successfully")
    else:
        print("GalbotMotion initialization failed")
    if nav.init():
        print("GalbotNavigation initialized successfully")
    else:
        print("GalbotNavigation initialization failed")

    # Add a box collision object into Motion environment.
    # This affects Motion-side collision checking (e.g., motion_plan/check_collision).
    try:
        obstacle_id = "box_test_1"
        obj_type = "box"
        obj_pose = [1.0, 0.0, 1.0, 0,0,0,1]
        obj_size = [1.0, 1.0, 1.0]
        target_frame = "world"
        status = motion.add_obstacle(
            obstacle_id=obstacle_id,
            obstacle_type=obj_type,
            pose=obj_pose,
            scale=obj_size,
            target_frame=target_frame
        )
        printStatus(status)
        print(f"✓ Obstacle {obstacle_id} added successfully")
        motion.clear_obstacle()
        print(f"✓ Obstacle {obstacle_id} cleared successfully")
    except Exception as e:
        print(f"Failed to add obstacle {obstacle_id}: {e}")

    # Wait for data to be ready
    time.sleep(2)

    chain_joints = {
        "leg": [0.4992,1.4991,1.0005,0.0000,-0.0004],
        "head": [0.0000,0.0],
        "left_arm": [1.9999,-1.6000,-0.5999,-1.6999,0.0000,-0.7999,0.0000],
        "right_arm": [-2.0000,1.6001,0.6001,1.7000,0.0000,0.8000,0.0000]
    }
    chain_pose_baselink = {
        "leg": [0.0596,-0.0000,1.0327,0.5000,0.5003,0.4997,0.5000],
        "head": [0.0599,0.0002,1.4098,-0.7072,0.0037,0.0037,0.7069],
        "left_arm": [0.1267,0.2342,0.7356,0.0220,0.0127,0.0343,0.9991],
        "right_arm": [0.1267,-0.2345,0.7358,-0.0225,0.0126,-0.0343,0.9991]
    }
    whole_body_joint = [
        num for key in ["leg", "head", "left_arm", "right_arm"]
        for num in chain_joints[key]

```

```

]
base_state = [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 1.0]
custom_param = gm.Parameter()

# Scene 1: Multi-path Point Planning in Cartesian Space (PoseState Target)
try:
    # Construct target pose
    target_pose_state = gm.PoseState()
    target_pose_state.chain_name = "left_arm"

    # Construct waypoints (3 intermediate poses)
    waypoint_poses = [
        [0.1267,0.2342,0.7356,0.0220,0.0127,0.0343,0.9991],
        [0.2267,0.2342,0.7356,0.0220,0.0127,0.0343,0.9991],
        [0.3267,0.2342,0.7356,0.0220,0.0127,0.0343,0.9991],
        [0.4267,0.2342,0.7356,0.0220,0.0127,0.0343,0.9991],
    ]

    status, traj = motion.motion_plan_multi_waypoints(
        target=target_pose_state,
        waypoint_poses=waypoint_poses,
        enable_collision_check=False,
        params=custom_param
    )
    time.sleep(1)
    printStatus(status)
    if status == gm.MotionStatus.SUCCESS:
        if traj != {}:
            print(f"✓ Multi-path Point Planning in Cartesian Space (PoseState Target) Success: trajectory points={len(traj[target_pose_state.chain_name])}")
        else:
            print(f" Multi-path Point Planning in Cartesian Space (PoseState Target) Success: trajectory is empty, possibly already at target; check whether the target matches current state or is within tolerance")
    else:
        print(f"✗ Multi-path Point Planning in Cartesian Space (PoseState Target) Failed: {status}")
        status = robot.stop_trajectory_execution()
        printStatus(status)

except Exception as e:
    print(f"✗ Multi-path Point Planning in Cartesian Space (PoseState Target) Exception: {e}")

# Scene 2: Multi-path Point Planning in Joint Space (JointStates Target)
try:
    # Construct target joint
    target_joint = gm.JointStates()
    target_joint.chain_name = "left_arm"

    # Construct waypoints (3 intermediate joint states)
    waypoints = [
        [0.1267,0.2342,0.7356,0.0220,0.0127,0.0343,0.9991],
        [0.2267,0.4342,0.7356,0.0220,0.0127,0.0343,0.9991],
        [0.3267,0.6342,0.7356,0.0220,0.0127,0.0343,0.9991],
        [0.4267,0.8342,0.7356,0.0220,0.0127,0.0343,0.9991]
    ]

    status, traj = motion.motion_plan_multi_waypoints(
        target=target_joint,

```

```

        waypoint_poses=waypoints,
        enable_collision_check=False,
        params=custom_param
    )
    printStatus(status)

    if status == gm.MotionStatus.SUCCESS:
        if traj != {}:
            print(f"✔ Multi-path Point Planning in Joint Space (JointStates Target) Success: trajectory points={len(traj[target_joint.chain_name])}")
        else:
            print(f" Multi-path Point Planning in Joint Space (JointStates Target) Success: trajectory is empty, possibly already at target; check whether the target matches current state or is within tolerance")
        else:
            print(f"✘ Multi-path Point Planning in Joint Space (JointStates Target) Failed: {status}")
            status = robot.stop_trajectory_execution()
            printStatus(status)
    except Exception as e:
        print(f"✘ Multi-path Point Planning in Joint Space (JointStates Target) Exception: {e}")

    # Clear all obstacles
    motion.clear_obstacle()
    print(f"✔ Clear all obstacles successfully")

    # Check trajectory execution status
    status = robot.check_trajectory_execution_status(chain_joints.keys())
    print(f"✔ Check trajectory execution status: {status}")

    printStatus(status)

except Exception as e:
    print(f"An exception occurred: {e}")
finally:
    # Actively send SIGINT shutdown signal
    robot.request_shutdown()
    # Wait to enter shutdown state
    robot.wait_for_shutdown()
    # Release SDK resources
    robot.destroy()
    print('Resource release successful')

if __name__ == "__main__":
    main()

```

Example10. 综合任务：导航+感知+抓取+放置

本示例是一个综合性的机器人任务示例，实现了从导航到感知再到抓取和放置的完整工作流程。其目的是演示如何结合使用 Galbot 机器人的导航、视觉感知和机械臂控制功能来完成复杂的自主操作任务。脚本首先进行安全检查和系统初始化，然后让机器人导航至目标区域，接着抬起摄像头以便更好地观察环境，利用左臂摄像头获取 RGB 和深度图像进行目标检测，将检测到的目标位置从相机坐标系转换到机器人基座坐标系，最后执行抓取操作并将物体放回起始位置。整个过程中会显示详细的执行

状态信息，包括导航状态、关节位置、抓取状态等，最终安全关闭机器人系统。

警告

```
examples/python/tutorials/example10_final_task.py
"""
Note: When running this example, please confirm that the robot's motion control service
`/data/galbot/bin/service_motion_plan`,
robot state publishing service `/data/galbot/bin/robot_state_publish`,
navigation service `/data/galbot/bin/service_navigation_plan`
and hand-eye calibration publishing service `/data/galbot/bin/eyehand_calib_publish`
have been loaded;
"""
try:
    import galbot_sdk.g1 as gm
    from galbot_sdk.g1 import GalbotNavigation
    from galbot_sdk.g1 import GalbotRobot
    from galbot_sdk.g1 import GalbotMotion
    from galbot_sdk.g1 import G1JointGroup, SensorType
except ImportError:
    print("Import galbot_sdk failed, please install it first or check if it is in the
PYTHONPATH")
    exit(1)

import os

try:
    import numpy as np
except ImportError:
    os.system("pip install numpy")
    import numpy as np

try:
    from scipy.spatial.transform import Rotation as R
except ImportError:
    os.system("pip install scipy")
    from scipy.spatial.transform import Rotation as R

try:
    import cv2
except ImportError:
    os.system("pip install opencv-python")
    import cv2

import time
from typing import Sequence

def decode_compressed_image(compressed_image, camera_info={}):
    """
    decode CompressedImage image

    Parameters:
        compressed_image (dict): image dict, keys:[header, format, data, "depth_scale"]

    Returns:
        numpy.ndarray: decoded image
    """
    image_data = compressed_image["data"]
```

```

    if compressed_image["format"] == "rgb8":
        return decode_rgb_image(image_data)
    elif compressed_image["format"] == "16UC1":
        return decode_depth_image(image_data, compressed_image["depth_scale"], camera_info)
    else:
        raise ValueError(f"Unsupport data format: {compressed_image['format']}")

def decode_rgb_image(image_data):
    """decode rgb image"""
    nparr = np.frombuffer(image_data, np.uint8)
    img = cv2.imdecode(nparr, cv2.IMREAD_COLOR)
    if img is None:
        raise ValueError("Fail to Decode RGB Image")
    return img

def decode_depth_image(image_data, depth_scale, camera_info):
    """decode depth image"""
    depth_img = np.frombuffer(image_data, dtype=np.uint16).copy()

    if not camera_info:
        depth_img = depth_img.reshape((720, 1280))
    else:
        depth_img = depth_img.reshape((camera_info["height"], camera_info["width"]))
    depth_img = depth_img.astype(np.float32) / depth_scale

    return depth_img

def print_gripper_state(joint_group, gripper_state):
    """
    Print gripper state

    Parameters:
        joint_group (G1JointGroup): G1JointGroup enum
        gripper_state (object): Contains timestamp_ns, width, velocity, effort, is_moving
    """
    print(f"Timestamp (ns): {gripper_state.timestamp_ns}")
    print(
        f"width {gripper_state.width} "
        f"velocity {gripper_state.velocity} "
        f"effort {gripper_state.effort} "
        f"is moving {gripper_state.is_moving}"
    )

def get_navigation_pose(object_goal_pose: Sequence[float], motion: GalbotMotion, arm: str = "left_arm"):
    """
    Get navigation target pose

    Parameters:
        object_goal_pose (Sequence[float]): Target pose [x, y, z, qx, qy, qz, qw]
        motion (GalbotMotion): Motion control instance
        arm (str, optional): End effector name. Defaults to "left_arm".

    Returns:
        Sequence[float]: Navigation target pose [x, y, z, qx, qy, qz, qw]
    """
    assert arm in ["left_arm", "right_arm"], "arm must be left_arm or right_arm"

    try:
        status, ee_pose_in_base = motion.get_end_effector_pose(

```

```

        end_effector_frame=f"{arm}_end_effector_mount_link",
        reference_frame="base_link"
    )
    if status != gm.MotionStatus.SUCCESS:
        print(f"✗ Failed to get end effector pose: status={status}")
        offset_y = ee_pose_in_base[1]
    else:
        print(f"✓ Successfully got end effector pose: pose={ee_pose_in_base}")
        offset_y = 0.3

    # Set chassis pose to the same z coordinate as the target pose
    base_goal_pose_mat = np.eye(4)
    base_goal_pose_mat[:3, :3] = R.from_quat(object_goal_pose[3:]).as_matrix()
    base_goal_pose_mat[:3, 3] = np.array([object_goal_pose[0], object_goal_pose[1], 0])

    # According to the relative position of the chassis and camera, move the camera
    # navigation target 0.6m backward in the local coordinate to leave observation space for the
    # camera
    base_goal_pose_mat = base_goal_pose_mat @ np.array([[1,0,0,-0.6],[0,1,0,-
offset_y],[0,0,1,0],[0,0,0,1]])

    base_goal_pose_quat = R.from_matrix(base_goal_pose_mat[:3, :3]).as_quat()
    base_goal_pose_pos = base_goal_pose_mat[:3, 3]

    return base_goal_pose_pos.tolist() + base_goal_pose_quat.tolist()

except Exception as e:
    print("Failed to get navigation target pose:", e)
    return [1, -1, 0, 0, 0, 0, 1]

def navigation_to_goal(nav: GalbotNavigation, goal_pose: Sequence[float], retry_cnt: int =
3):
    """
    Navigate to target pose

    Parameters:
        nav (GalbotNavigation): Navigation instance
        goal_pose (Sequence[float]): Target pose [x, y, z, qx, qy, qz, qw]
        retry_cnt (int, optional): Number of retries. Defaults to 3.
    """
    try:
        cur_pose = nav.get_current_pose()
        print(f"Current pose: {cur_pose}")
        if nav.check_path_reachability(goal_pose, cur_pose):
            retry_cnt = 3
            while True:
                status = nav.navigate_to_goal(goal_pose, enable_collision_check=True,
is_blocking=True, timeout=20)
                time.sleep(0.5)
                retry_cnt -= 1
                if nav.check_goal_arrival() or retry_cnt < 0:
                    break
            else:
                print(f"Navigation failed: status={status}, retrying: {retry_cnt}")
                print("navigate_to_goal return status:", status)
                print("Has arrived:", nav.check_goal_arrival())
        else:
            print("Path unreachable or unsafe")
    except Exception as e:

```

```

        print(f"Exception occurred during navigation: {e}")

def lift_camera_up(motion: GalbotMotion, target_pose: Sequence[float], target_chain: str,
reference_frame: str):
    """
    Lift camera to target height

    Parameters:
        motion (GalbotMotion): Motion control instance
        target_pose (Sequence[float]): Target pose [x, y, z, qx, qy, qz, qw]
        target_chain (str): End effector name
        reference_frame (str): Reference frame
    """
    try:
        retry_cnt = 3
        while True:
            status, cur_ee_pose = motion.get_end_effector_pose_on_chain(
                chain_name=target_chain,
                frame_id="EndEffector",
                reference_frame=reference_frame
            )
            time.sleep(0.5)
            retry_cnt -= 1

            if status == gm.MotionStatus.SUCCESS:
                print(f"✔ Successfully got end effector pose: pose={cur_ee_pose}")
                break
            elif retry_cnt < 0:
                cur_ee_pose = [0.1267,0.2342,0.7356,0.0220,0.0127,0.0343,0.9991]
                print(f"✗ Failed to get end effector pose: status={status}, retrying:
{retry_cnt}")
                break
            else:
                print(f"Failed to get end effector pose: status={status}, retrying:
{retry_cnt}")

            tgt_ee_pose = cur_ee_pose.copy()
            tgt_ee_pose[2] = target_pose[2] - 0.1
            print(f"Target end effector pose: {tgt_ee_pose}")

            retry_cnt = 3
            while True:
                status = motion.set_end_effector_pose(
                    target_pose=tgt_ee_pose,
                    end_effector_frame=target_chain,
                    reference_frame=reference_frame,
                    enable_collision_check=False,
                    is_blocking=True,
                    timeout=5.0,
                    params=gm.Parameter()
                )
                time.sleep(0.5)
                retry_cnt -= 1
                if status == gm.MotionStatus.SUCCESS or retry_cnt < 0:
                    print(f"✔ Successfully set end effector pose: status={status}")
                    break
                else:
                    print(f"Failed to set end effector pose: status={status}, retrying:
{retry_cnt}")
            except Exception as e:

```



```

        print(f"✗ Failed to lift camera: {e}")

def detect_target(img: np.ndarray, depth_img: np.ndarray) -> Sequence[float]:
    """
    Detection target function. Input RGB image and depth image, output target pose.

    Parameters:
        img (np.ndarray): RGB image
        depth_img (np.ndarray): Depth image

    Returns:
        Sequence[float]: Target pose [x, y, z, qx, qy, qz, qw]
    """
    try:

        ##### NOTE #####
        # This function is a placeholder. In a real-world scenario, you would implement
        # target detection using computer vision techniques. For this example, we assume
        # a default pose.
        #####

        # opencv frame: x right, y down, z forward
        default_pose = [0.0, 0.20, 0.29, 0.0, 0.71, 0.0, 0.71]

        return default_pose
    except Exception as e:
        print(f"Target detection exception: {e}")
        return None

def pose_camera_to_base(robot: GalbotRobot, pose_camera: Sequence[float]) ->
Sequence[float]:
    """
    Transform camera pose to chassis coordinate system

    Parameters:
        robot (GalbotRobot): Robot instance
        pose_camera (Sequence[float]): Camera pose [x, y, z, qx, qy, qz, qw]

    Returns:
        Sequence[float]: Chassis pose [x, y, z, qx, qy, qz, qw]
    """
    source_frame="left_arm_camera_color_optical_frame"
    target_frame="base_link"
    base_to_cam = robot.get_transform(target_frame, source_frame)[0]
    if base_to_cam is None:
        print("Failed to get transform from camera to chassis")
        return None
    else:
        print("base_to_cam: ", base_to_cam)

    base_to_cam_mat = np.eye(4)
    base_to_cam_mat[:3, :3] = R.from_quat(base_to_cam[:3]).as_matrix()
    base_to_cam_mat[:3, 3] = np.array(base_to_cam[:3])

    pose_base_mat = base_to_cam_mat[:3, :3] @ np.array(pose_camera[:3]).reshape(3, 1) +
    base_to_cam_mat[:3, 3:]

    return pose_base_mat.flatten()[:3].tolist() + [0, 0, 0, 1]

def detect_object(robot: GalbotRobot, arm: str = "left_arm"):

```

```

try:
    # Get camera image data
    if arm == "left_arm":
        rgb_image_data = robot.get_rgb_data(SensorType.LEFT_ARM_CAMERA)
        depth_data = robot.get_depth_data(SensorType.LEFT_ARM_DEPTH_CAMERA)
    elif arm == "right_arm":
        rgb_image_data = robot.get_rgb_data(SensorType.RIGHT_ARM_CAMERA)
        depth_data = robot.get_depth_data(SensorType.RIGHT_ARM_DEPTH_CAMERA)
    else:
        raise ValueError("arm must be left_arm or right_arm")

    # Decode image data
    if not rgb_image_data:
        print("No rgb image data!")
    else:
        print("get rgb image suceess")
        img = decode_compressed_image(rgb_image_data)

    if not depth_data:
        print("No depth_data!")
    else:
        depth_img = decode_compressed_image(depth_data)
        print("get depth data suceess")

    # Detect target
    object_pose_camera = detect_target(img, depth_img)
    if object_pose_camera is None:
        print("Target detection failed")
        return None
    else:
        print(f"object_pose_camera: {object_pose_camera}")

    # Calculate target pose in chassis coordinate system
    object_pose_base = pose_camera_to_base(robot, object_pose_camera)
    print(f"Target pose in chassis coordinate system: {object_pose_base}")

except Exception as e:
    object_pose_base = [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 1.0]
    print(f"Target detection exception: {e}")

return object_pose_base

def check_robot_safety():
    """Check if robot is safe"""
    # Prompt important notes
    print("    Note: 1. Please ensure the emergency stop button of the robot is released; 2. Please ensure there are no obstructions around the robot to avoid unexpected situations. 3. Please ensure the area around the robot is clear of obstacles.")
    while True:
        key = input("Please confirm that the robot's emergency stop button is released and there are no obstructions, continue? (y/n)...")
        if key == 'y':
            print("User confirmed, continuing...")
            break
        elif key == 'n':
            print("User did not confirm, exiting program...")
            exit(1)
        else:
            print("Invalid input, please enter 'y' or 'n'")

```

```

def pick_and_place(robot: GalbotRobot,
                    nav: GalbotNavigation,
                    motion: GalbotMotion,
                    object_pose_base: Sequence[float],
                    target_chain: str,
                    reference_frame: str):
    try:
        # Attach tool to end effector
        status = motion.attach_tool(chain="left_arm", tool="galbot_gripper")
        time.sleep(0.5)
        if status != gm.MotionStatus.SUCCESS:
            print(f"✗ Failed to attach tool: status={status}")
        else:
            print(f"✓ Successfully attached tool: status={status}")

        # Open left gripper
        # Set left gripper width to 0.1m, speed to 0.05m, force to 10N, will block until
gripper reaches position
        status = robot.set_gripper_command(
            G1JointGroup.left_gripper, 0.1, 0.05, 10, False
        )
        time.sleep(0.5)
        print(f"✓ Successfully set left gripper width to 0.1m: status={status}")

        print("object_pose_base: ", object_pose_base)

        # Reach to target position
        retry_cnt = 3
        while True:
            status = motion.set_end_effector_pose(
                target_pose=object_pose_base,
                end_effector_frame=target_chain,
                reference_frame=reference_frame,
                enable_collision_check=False,
                is_blocking=True,
                timeout=5.0,
                params=gm.Parameter()
            )
            time.sleep(1)
            retry_cnt -= 1
            if status == gm.MotionStatus.SUCCESS or retry_cnt < 0:
                break
            else:
                print(f"Failed to set end effector pose: status={status}, retry count:
{retry_cnt}")

        if status != gm.MotionStatus.SUCCESS:
            print(f"✗ Failed to set end effector pose: status={status}")
        else:
            print(f"✓ Successfully set end effector pose: status={status}")

        # Close gripper to grasp object
        status = robot.set_gripper_command(
            G1JointGroup.left_gripper, 0.02, 0.05, 10, False
        )
        time.sleep(0.5)
        print(f"✓ Successfully closed left gripper: status={status}")

        # Attach target object (box scale = length/width/height in m, must all be > 0)
        status = motion.attach_target_object(

```

```

        "grasped_box",
        "box",
        [0, 0, 0, 0, 0, 0, 1],
        [0.05, 0.05, 0.05],
        "",
        "left_arm",
        "ee_base",
    )
    time.sleep(0.5)
    if status != gm.MotionStatus.SUCCESS:
        print(f"✗ Failed to attach target object: status={status}")
    else:
        print(f"✓ Successfully attached target object: status={status}")

    # Return to initial position
    navigation_to_goal(nav, [0, 0, 0, 0, 0, 0, 1])
    time.sleep(2)
    print(f"✓ Successfully returned to initial position")

    # Release target
    status = robot.set_gripper_command(
        GJGroup.left_gripper, 0.1, 0.05, 10, False
    )
    time.sleep(0.5)
    print(f"✓ Successfully released left gripper: status={status}")

    # Detach target object
    status = motion.detach_target_object("grasped_box")
    time.sleep(0.5)
    if status != gm.MotionStatus.SUCCESS:
        print(f"✗ Failed to detach target object: status={status}")
    else:
        print(f"✓ Successfully detached target object: status={status}")

    # Detach tool from end effector
    status = motion.detach_tool(chain="left_arm")
    time.sleep(0.5)
    if status != gm.MotionStatus.SUCCESS:
        print(f"✗ Failed to detach tool: status={status}")
    else:
        print(f"✓ Successfully detached tool: status={status}")
except Exception as e:
    print(f"Exception occurred during pick_and_place: {e}")
    return None

def run_move_whole_body_joint_zero(motion: GalbotMotion, phase: str = "") -> None:
    """Call move_whole_body_joint_zero with retries (same parameters as S1 example10)."""
    retry_cnt = 3
    suffix = f" [{phase}]" if phase else ""
    while True:
        zero_status = motion.move_whole_body_joint_zero(True, 0.2, 15.0, gm.Parameter())
        time.sleep(0.5)

        if zero_status == gm.MotionStatus.SUCCESS:
            print(f"✓ Successfully moved whole body joint zero{suffix}: status={zero_status}")
            break

    retry_cnt -= 1

```

```

        if retry_cnt < 0:
            print(f"✗ Failed to move_whole_body_joint_zero after retries{suffix},
status={zero_status}")
            break
        print(f"Failed to move_whole_body_joint_zero{suffix}: retry count: {retry_cnt},
status={zero_status}")

def main():
    check_robot_safety()
    try:
        # Get robot instance
        robot = GalbotRobot()
        # Get GalbotMotion instance
        motion = GalbotMotion()
        # Get navigation instance
        nav = GalbotNavigation()

        # Get RGB and depth images from left arm, depth images from right arm,
        # base LiDAR data, and torso IMU data
        enable_sensor_set = {SensorType.LEFT_ARM_CAMERA, # Left arm depth camera
                             SensorType.LEFT_ARM_DEPTH_CAMERA, # Left arm RGB camera
                             SensorType.BASE_LIDAR, # Base LiDAR
                             SensorType.TORSO_IMU} # Torso IMU sensor

        # Initialize robot
        if robot.init(enable_sensor_set):
            print("GalbotRobot initialization successful")
        else:
            print("GalbotRobot initialization failed")
        if motion.init():
            print("GalbotMotion initialization successful")
        else:
            print("GalbotMotion initialization failed")
        if nav.init():
            print("GalbotNavigation initialization successful")
        else:
            print("GalbotNavigation initialization failed")

        # Program starts immediately, wait for data readiness
        time.sleep(1)

        # Reset whole body to SDK zero pose before navigation and perception
        run_move_whole_body_joint_zero(motion, "startup")

        # Relocalize robot if not localized
        while True:
            if nav.is_localized():
                print("✓ Robot localized successfully, proceeding to navigate to goal...")
                break
            else:
                print("✗ Navigation not localized, relocalizing...")
                nav.relocalize([0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 1.0])
                time.sleep(2)

        # Calculate navigation target pose
        object_goal_pose = [1.0, -1.0, 1.0, 0, 0, 0, 1]
        base_goal_pose = get_navigation_pose(object_goal_pose, motion)
        print(f"Navigation target pose: {base_goal_pose}")

```

```
# Navigate to target pose
navigation_to_goal(nav, base_goal_pose)
print()

# Lift camera
lift_camera_up(motion, object_goal_pose, "left_arm", "base_link")
print()

# Detect target
object_pose_base = detect_object(robot, arm="left_arm")
print()

# Grasp and return to initial position
pick_and_place(robot, nav, motion, object_pose_base, "left_arm", "base_link")
print()

# After grasping target, move the whole body back to SDK zero pose
run_move_whole_body_joint_zero(motion, "post-grasp")

except Exception as e:
    print(f"Exception occurred: {e}")
finally:
    # Actively send SIGINT exit signal
    robot.request_shutdown()
    # Wait to enter shutdown state
    robot.wait_for_shutdown()
    # Release SDK resources
    robot.destroy()
    print('Resource release successful')

if __name__ == "__main__":
    main()
```

14. Python 示例

本文件为 API 中公开的函数与类型提供简短示例，演示如何使用这些接口。

机器人关节名称 (G1 2.2)

关节组列表

机器人关节组名称包括: ["head", "left_arm", "right_arm", "leg", "left_gripper", "right_gripper", "left_suction_cup", "right_suction_cup"]

各关节组详细信息

关节组名称	关节组英文名	关节数量	关节名称列表
头部	head	2	head_joint1, head_joint2
腿部	leg	5	leg_joint1, leg_joint2, leg_joint3, leg_joint4, leg_joint5
左臂	left_arm	7	left_arm_joint1, left_arm_joint2, left_arm_joint3, left_arm_joint4, left_arm_joint5, left_arm_joint6, left_arm_joint7
右臂	right_arm	7	right_arm_joint1, right_arm_joint2, right_arm_joint3, right_arm_joint4, right_arm_joint5, right_arm_joint6, right_arm_joint7
左夹爪	left_gripper	1	left_gripper_joint1
右夹爪	right_gripper	1	right_gripper_joint1
左吸盘	left_suction_cup	1	left_suction_cup_joint1
右吸盘	right_suction_cup	1	right_suction_cup_joint1

如何正确使用 joint_groups

joint_groups 的设计单位是“关节组（运动链）”，而不是单个关节。这样设计是为了保证：

- 运动链一致性：头/臂/腿等关节按链路整体校验和控制。
- 指令顺序确定性：joint_groups 会按传入顺序展开为具体关节。
- 组级约束校验：执行前会检查关节组的 active/passive 属性及输入合法性。

传参规则：

- 1. 需要控制整条链路时，优先使用 joint_groups (head/arm/leg/gripper/suction cup) 。
- 1. 当 joint_names 非空时，优先使用 joint_names，会覆盖 joint_groups。
- 1. 当 joint_groups 和 joint_names 都为空时，SDK 默认控制所有 active body 关节组。
- 1. 建议先调用 get_joint_group_names() 和 get_joint_names(True, groups)，避免手写字符串出错。

G1 关节组典型场景

关节组	典型场景
head	头部朝向/相机对准
left_arm / right_arm	机械臂抓取与操作
leg	下肢姿态调整
left_gripper / right_gripper	夹爪开合宽度控制
left_suction_cup / right_suction_cup	吸盘吸放作业

传感器类型与 Frame 说明

获取传感器数据时 (get_rgb_data、get_depth_data、get_imu_data、get_lidar_data) ，通过 SensorType 枚举指定要查询的传感器。可用的 SensorType 枚举值详见：[Python API 参考 > 传感器类型](#)。

获取传感器外参有两种方式：

- **get_sensor_extrinsic()**: SDK 内部已映射 frame ID，直接传入 SensorType 即可
- **get_transform()**: 需要显式传入 frame 名称，表格第二列为各传感器对应的 Frame ID

SensorType	Frame ID
HEAD_LEFT_CAMERA	head_left_camera_color_optical_frame
HEAD_RIGHT_CAMERA	head_right_camera_color_optical_frame
LEFT_ARM_CAMERA	left_arm_camera_color_optical_frame
LEFT_ARM_DEPTH_CAMERA	left_arm_camera_color_optical_frame
RIGHT_ARM_CAMERA	right_arm_camera_color_optical_frame
RIGHT_ARM_DEPTH_CAMERA	right_arm_camera_color_optical_frame
BASE_LIDAR	lidar_base_link
TORSO_IMU	imu_base_link
BASE_ULTRASONIC	— (无 TF frame)
LEFT_FRONT_SURROUND_CAMERA	left_front_surround_color_optical_frame
RIGHT_FRONT_SURROUND_CAMERA	right_front_surround_color_optical_frame
LEFT_REAR_SURROUND_CAMERA	left_rear_surround_color_optical_frame
RIGHT_REAR_SURROUND_CAMERA	right_rear_surround_color_optical_frame

注意：调用 **get_frame_names()** 可获取当前所有可用的坐标系名称。

类：GalbotRobot

tips: 程序启动后立刻就 get, 数据可能不会立刻就绪, 可适当 sleep 几秒

获取实例并初始化 (get_instance && init)

适用场景：程序启动阶段，获取机器人单例并完成 SDK 初始化。必须在调用其他 API 前执行。

```
examples/python/galbot_robot/get_instance.py
from galbot_sdk.g1 import GalbotRobot
import time

# Get GalbotRobot
robot = GalbotRobot()

state = robot.init()
if not state:
    print("Initialization failed")
else:
    print("Initialization succeeded")

while robot.is_running():
    # business logic
    time.sleep(1)
    break

# Send exit signal to exit the program
robot.request_shutdown()
# Wait for exit state
robot.wait_for_shutdown()
# Release SDK-related resources
robot.destroy()
```

日志函数

适用场景：使用 SDK 内置的日志系统输出日志，统一日志级别和格式。

```
examples/python/galbot_robot/logger_example.py
import galbot_sdk

cfg = {
    # Log storage directory; if empty, defaults to ~/galbot_sdk_log/user_log
    "path": "",

    # Log file name; if empty, defaults to
    <process_name>_<current_time>_<pid>_<thread_id>.log
    "file_name": "",

    # log bytes, log size,
    "file_max_size": 10 * 1024 * 1024, # 10MB

    # Maximum rotating log files; oldest file is overwritten when limit exceeded
    "file_max_num": 5,

    # Whether to output logs to console; default is False
    "console_output": True,
```

```

    # Log output level, available values: debug, info, warning, error, critical
    "level": "info",
}

galbot_sdk.init_logger(cfg)

# Write log
galbot_sdk.debug("Debug example")
galbot_sdk.info("Info example")
galbot_sdk.warning("Warning example")
galbot_sdk.error("Error example")
galbot_sdk.critical("Critical example")

```

设置关节角度 (set_joint_positions)

适用场景：单次点位移动，低频率的位置控制任务。该接口内部会对目标位置进行速度受限的插值规划，适合一次性移动到目标关节角度的场景。

WARNING: 此接口**不适合**模型推理输出的高频关节控制场景！该接口每次调用都会产生一次轨迹插值，连续调用会导致运动不连续且有延迟。

如果您在做**模型推理**场景，请使用 `set_joint_commands` 或 `set_joint_commands_batch` 直接下发关节指令。

```

examples/python/galbot_robot/set_joint_positions.py
import time
from galbot_sdk.g1 import GalbotRobot
from galbot_sdk.g1 import ControlStatus

# Get and initialize the GalbotRobot singleton
robot = GalbotRobot()
robot.init()
print('Initialization succeeded')

# Program started, waiting for data
time.sleep(2)

# Set head joints to 0.2, 0.2, block and wait for motion to complete, max timeout 10s
joint_pos = [0.2, 0.2]
# Set head joint group; if empty, defaults to whole body joints ["leg", "head", "left_arm",
"right_arm"]
joint_groups = ["head"]
# Whether to block until joints reach target
is_blocking = True
# Limit joint max speed to 0.1rad/s
max_speed = 0.1
# Maximum blocking wait time
timeout_s = 10

status = robot.set_joint_positions(
    joint_pos, joint_groups, [], is_blocking, max_speed, timeout_s
)

if status != ControlStatus.SUCCESS:
    print("Joint angle setting failed")
else:
    print('Joint angle setting succeeded')

```

```

time.sleep(1)

# Use specific joint names for control; this parameter overrides joint_groups
joint_names = ["head_joint1", "head_joint2"]
joint_pos = [0.0, 0.0]

status = robot.set_joint_positions(
    joint_pos, [], joint_names, is_blocking, max_speed, timeout_s
)

if status != ControlStatus.SUCCESS:
    print("Joint angle setting failed")
else:
    print('Joint angle setting succeeded')

# send SIGINT shutdown signal
robot.request_shutdown()
# Wait until entering shutdown state
robot.wait_for_shutdown()
# Perform SDK resource release
robot.destroy()
print('Resources released successfully')

```

设置夹爪指令 (set_gripper_command)

适用场景：控制夹爪开合。支持位置控制和力矩控制两种模式。

```

examples/python/galbot_robot/set_gripper_command.py
import time
from galbot_sdk.g1 import GalbotRobot, G1JointGroup, ControlStatus

# Get and initialize the GalbotRobot singleton
robot = GalbotRobot()
robot.init()
print('Initialization succeeded')
# Program started, waiting for data
time.sleep(2)

# Set left gripper width to 0.02m, speed 0.05m, torque 10N, block until gripper reaches
position
status = robot.set_gripper_command(
    G1JointGroup.left_gripper, 0.02, 0.05, 10, True
)
if status != ControlStatus.SUCCESS:
    print("Set gripper failed")
else:
    print('Set gripper success')

# Set left gripper width to 0.1m, speed 0.05m, torque 10N, block until gripper reaches
position
status = robot.set_gripper_command(
    G1JointGroup.left_gripper, 0.1, 0.05, 10, True
)

if status != ControlStatus.SUCCESS:
    print("Set gripper failed")
else:
    print('Set gripper success')

```

```
# send SIGINT shutdown signal
robot.request_shutdown()
# Wait until entering shutdown state
robot.wait_for_shutdown()
# Perform SDK resource release
robot.destroy()
print('Resources released successfully')
```

设置吸盘指令 (set_suction_cup_command)

适用场景：控制吸盘吸放物体，获取吸盘当前的开启/关闭状态。

```
examples/python/galbot_robot/set_suction_cup_command.py
from galbot_sdk.g1 import GalbotRobot
from galbot_sdk.g1 import G1JointGroup, ControlStatus
import time

# Get and initialize the GalbotRobot singleton
robot = GalbotRobot()
robot.init()
time.sleep(1)
print('Initialization succeeded')

# Set suction cup joint group (right suction cup)
joint_group = G1JointGroup.right_suction_cup

# Whether to activate suction cup
activate = True # True: activate suction cup, False: deactivate suction cup

# Send suction cup control command
status = robot.set_suction_cup_command(
    joint_group,
    activate
)

# Check execution results
if status != ControlStatus.SUCCESS:
    print("Set suction cup failed")
else:
    print("Set suction cup succeeded")

# send SIGINT shutdown signal
robot.request_shutdown()
# Wait until entering shutdown state
robot.wait_for_shutdown()
# Perform SDK resource release
robot.destroy()
print('Resources released successfully')
```

停止轨迹执行 (stop_trajectory_execution)

适用场景：停止当前正在执行的关节轨迹，中断正在进行的运动。

```
examples/python/galbot_robot/stop_trajectory_execution.py
from galbot_sdk.g1 import GalbotRobot
from galbot_sdk.g1 import ControlStatus
import time
```

```

# Get and initialize the GalbotRobot singleton
robot = GalbotRobot()
robot.init()
time.sleep(2)
print("Initialization succeeded")

# Stop trajectory execution
while True:
    status = robot.stop_trajectory_execution()

    # Check execution results
    if status == ControlStatus.SUCCESS:
        print('Stop trajectory execution succeeded')
        break

    print("Trajectory stop failed, retrying...")

# send SIGINT shutdown signal
robot.request_shutdown()
# Wait until entering shutdown state
robot.wait_for_shutdown()
# Perform SDK resource release
robot.destroy()
print('Resources released successfully')

```

设置轨迹 (execute_joint_trajectory)

适用场景：执行一条预定义的关节空间轨迹，包含多个路点。SDK 会按照时间节点执行整条轨迹。

```

examples/python/galbot_robot/execute_joint_trajectory.py
from galbot_sdk.g1 import GalbotRobot
from galbot_sdk.g1 import G1JointGroup, ControlStatus, Trajectory, TrajectoryPoint,
JointCommand
import time
import numpy as np
from typing import List

# Generate trajectory point with position and time info
def generate_target_point(q: List[float], target_time: float = 10):
    """Generate target for joints"""
    joint_position = TrajectoryPoint()
    joint_position.time_from_start_second = target_time
    joint_command_vec = []
    for joint in q:
        joint_cmd = JointCommand()
        joint_cmd.position = joint
        joint_command_vec.append(joint_cmd)
    joint_position.joint_command_vec = joint_command_vec
    return joint_position

def generate_target_trajectory(trajectory, joint_groups=[], joint_names=[], dt=0.008):
    """Generate trajectory for joints"""
    if trajectory is None or np.ndim(trajectory) != 2 or len(trajectory) == 0:
        return None

    # Create Trajectory
    traj = Trajectory()
    traj.joint_groups = joint_groups
    traj.joint_names = joint_names

```

```

time = 0.0
points = []
for state in trajectory:
    time += dt
    # Create single trajectory point
    traj_point = generate_target_point(state, time)
    points.append(traj_point)

traj.points = points
return traj

# Trajectory execution function
def traj_exec():
    # Get GalbotRobot singleton and initialize; only needs to be initialized once
    robot = GalbotRobot()
    robot.init()
    time.sleep(1)
    print("Initialization succeeded")

    head_traj = np.linspace(
        [0.0, 0.0],
        [0.5, 0.0],
        num=200,
    )
    # Whether to block and wait for trajectory execution to complete
    is_block = True
    # Specify which joint group trajectory to execute
    joint_groups = ["head"]
    # Execute specified joint trajectory; if provided, overrides joint_groups parameters
    joint_names = []
    status = robot.execute_joint_trajectory(generate_target_trajectory(head_traj.tolist(),
joint_groups, joint_names), is_block)

    # Check execution results
    if status != ControlStatus.SUCCESS:
        print("Trajectory execution failed")
    else:
        print("Trajectory execution succeeded")

    # send SIGINT shutdown signal
    robot.request_shutdown()
    # Wait until entering shutdown state
    robot.wait_for_shutdown()
    # Perform SDK resource release
    robot.destroy()
    print('Resources released successfully')

traj_exec()

```

设置关节指令 (set_joint_commands)

适用场景：高频关节控制，例如**模型推理输出**（每步推理输出一帧关节指令）、自定义轨迹跟踪。直接下发关节指令给底层控制器，不做额外插值。

examples/python/galbot_robot/set_joint_commands_example.py

```

import time
import math
from galbot_sdk.g1 import GalbotRobot
from galbot_sdk.g1 import JointCommand

def head_high_frequency_control():
    """
    Head high-frequency control example
    """

    control_frequency = 100.0 # Hz
    dt = 1.0 / control_frequency
    duration = 4.0 # Control for 4 seconds

    amplitude = 0.3 # Maximum oscillation amplitude (rad)
    frequency = 0.5 # Sine frequency (Hz)
    # Joint group name to control
    joint_groups = ["head"]
    # Fill this field to control specific joints, which will override joint_groups. If
    empty, controls all joints in joint_groups by default
    joint_names = []

    print("Start high-frequency head control")

    joint_commands = [JointCommand(), JointCommand()]

    start_time = time.time()

    while True:
        current_time = time.time() - start_time
        if current_time > duration:
            break

        # Generate sine trajectory
        target_position = amplitude * math.sin(
            2 * math.pi * frequency * current_time
        )

        # Set head joint angles
        joint_commands[0].position = target_position
        joint_commands[1].position = target_position
        print(f"current: {current_time:.2f}s, target: {target_position:.3f} rad")

        # Expected arrival time
        time_from_start_sec = 0.0

        execution_status = GalbotRobot().set_joint_commands(
            joint_commands,
            joint_groups,
            joint_names,
            time_from_start_sec
        )

        # Sleep at a fixed interval
        time.sleep(dt)

    print("end")

def main():
    # Get and initialize the GalbotRobot singleton

```

```

robot = GalbotRobot()

if robot.init():
    print("System initialized successfully!")
else:
    print("System initialization failed!")
    return

# Program started, waiting for data
time.sleep(2)

head_high_frequency_control()

# Exit system and release SDK resources
robot.request_shutdown()
robot.wait_for_shutdown()
robot.destroy()
print("Program exited")

if __name__ == "__main__":
    main()

```

批量设置关节指令 (set_joint_commands_batch)

适用场景：批量下发未来多帧关节指令，用于运动预测模型一次性输出多步结果的场景，可以提高控制频率和连续性。

```

examples/python/galbot_robot/set_joint_commands_batch.py
from galbot_sdk.g1 import GalbotRobot, G1JointGroup, ControlStatus, Trajectory,
TrajectoryPoint, JointCommand
import time
import numpy as np
from typing import List

# Generate trajectory point with position and time info
def generate_target_point(q: List[float], target_time: float = 10):
    """Generate target for joints"""
    joint_position = TrajectoryPoint()
    joint_position.time_from_start_second = target_time
    joint_command_vec = []
    for joint in q:
        joint_cmd = JointCommand()
        joint_cmd.position = joint
        joint_cmd.velocity = 0.0
        joint_cmd.acceleration = 0.0
        joint_cmd.effort = 0.0
        # joint_cmd.Kp = 0.0
        # joint_cmd.Kd = 0.0
        joint_command_vec.append(joint_cmd)
    joint_position.joint_command_vec = joint_command_vec
    return joint_position

def generate_batch_trajectory(trajectory, joint_groups=[], joint_names=[], dt=0.008):
    """Generate batch trajectory for joints"""
    if trajectory is None or np.ndim(trajectory) != 2 or len(trajectory) == 0:
        return None

```



```

# Create Trajectory
traj = Trajectory()
traj.joint_groups = joint_groups
traj.joint_names = joint_names

time = 0.0
points = []
for state in trajectory:
    time += dt
    # Create single trajectory point
    traj_point = generate_target_point(state, time)
    points.append(traj_point)

traj.points = points
return traj

# Batch set-joint-command function
def batch_commands_exec():
    # Get GalbotRobot singleton and initialize; only needs to be initialized once
    robot = GalbotRobot()
    robot.init()
    time.sleep(1)
    print("Initialization succeeded")

    # Generate batched trajectory data (joint commands at multiple time points)
    head_traj = np.linspace(
        [0.0, 0.0],
        [0.5, 0.0],
        num=10, # Number of batch trajectory points
    )
    # Specify which joint group trajectory to execute
    joint_groups = ["head"]
    # Execute specified joint trajectory; if provided, overrides joint_groups parameters
    joint_names = []

    # Batch set joint commands (non-blocking, returns immediately)
    status = robot.set_joint_commands_batch(generate_batch_trajectory(head_traj.tolist(),
joint_groups, joint_names))

    # Check execution results
    if status != ControlStatus.SUCCESS:
        print("Batch command submission failed")
    else:
        print("Batch commands submitted, executing in background (non-blocking)")

    # Wait for a while to let the command execute
    time.sleep(1)

    # send SIGINT shutdown signal
    robot.request_shutdown()
    # Wait until entering shutdown state
    robot.wait_for_shutdown()
    # Perform SDK resource release
    robot.destroy()
    print('Resources released successfully')

batch_commands_exec()

```

直接发布原始目标 (publish_target)

适用场景：高级用户直接构造 SingoriXTarget 并通过 publish 通道下发到底层 WBCS，适合高频原始 target 发送与 joint/task mixed target 一次性 dispatch。

```
examples/python/galbot_robot/publish_target_example.py
"""
G1 PublishTarget menu example for SDK mirror SingoriXTarget.

This example shows how to construct `SingoriXTarget` directly at the Python SDK
layer and send it to the low-level WBCS through `publish_target()`.

1. Joint-space commands are written into `target_group_trajectory_map`
2. Chassis pose / twist style task-space commands are written into
`target_task_trajectory_map`
3. One `SingoriXTarget` can contain both joint trajectory and task trajectory
4. The current SDK does not automatically switch the chassis controller, so
pose / twist / mixed scenes explicitly call `switch_controller(...)`
5. The base twist scene automatically sends a zero-twist target after the
configured duration to stop the chassis
"""

import math
import time

from galbot_sdk.g1 import (
    ControlStatus,
    G1ControllerName,
    G1JointGroup,
    GalbotRobot,
    GroupCommand,
    JointCommand,
    Pose,
    SingoriXTarget,
    TargetConfig,
    TargetGroupTrajectory,
    TargetSampling,
    TargetTaskTrajectory,
    TaskCommand,
    Twist,
    Vector3,
    FrameTriad,
    TARGET_DATA_DEFAULT,
    TARGET_DATA_FRAME_POSE,
    TARGET_DATA_FRAME_TWIST,
    TARGET_TYPE_DEFAULT,
    TARGET_TYPE_OVERRIDE,
    TARGET_TYPE_PROVERRIDE,
)

CHASSIS_TASK_NAME = "chassis"
CHASSIS_SUBTASK_POSE = "chassis_pose"
CHASSIS_SUBTASK_TWIST = "chassis_twist"

def now_ns():
    return time.time_ns()
```

```

def status_to_string(status):
    return getattr(status, "name", str(status))

def yaw_to_quaternion(yaw):
    return [0.0, 0.0, math.sin(yaw * 0.5), math.cos(yaw * 0.5)]

def make_empty_target():
    target = SingoriXTarget()
    target.header.timestamp_ns = now_ns()
    target.header.frame_id = "base_link"
    return target

def make_group_target_config():
    config = TargetConfig()
    config.target_data = TARGET_DATA_DEFAULT
    config.target_type = TARGET_TYPE_PROVERRIDE
    config.target_sampling = TargetSampling.TARGET_SAMPLING_DEFAULT
    config.target_priority = 1
    return config

def make_pose_target_config():
    config = TargetConfig()
    config.target_data = TARGET_DATA_FRAME_POSE
    config.target_type = TARGET_TYPE_PROVERRIDE
    config.target_sampling = TargetSampling.TARGET_SAMPLING_LINEAR_INTERPOLATE
    config.target_priority = 1
    return config

def make_twist_target_config():
    config = TargetConfig()
    config.target_data = TARGET_DATA_FRAME_TWIST
    config.target_type = TARGET_TYPE_OVERRIDE
    config.target_sampling = TargetSampling.TARGET_SAMPLING_DIRECT_PASS
    config.target_priority = 1
    return config

def make_joint_command(position):
    command = JointCommand()
    command.position = position
    command.velocity = 0.0
    command.acceleration = 0.0
    command.effort = 0.0
    return command

def make_vector3(x, y, z):
    vec = Vector3()
    vec.x = x
    vec.y = y
    vec.z = z
    return vec

```

```

def build_joint_target(group_name, joint_names, positions, time_from_start_s):
    target = make_empty_target()

    group_traj = TargetGroupTrajectory()
    group_traj.target_config = make_group_target_config()
    group_traj.joint_names = list(joint_names)

    command = GroupCommand()
    command.time_from_start_s = time_from_start_s
    command.joint_commands = [make_joint_command(position) for position in positions]
    group_traj.group_commands = [command]

    target.target_group_trajectory_map = {group_name: group_traj}
    return target

```

```

def build_chassis_pose_target(x, y, yaw, time_from_start_s, frame_id="odom",
reference_frame_id="odom"):
    target = make_empty_target()

    task_traj = TargetTaskTrajectory()
    task_traj.target_config = make_pose_target_config()
    task_traj.group_names = [G1JointGroup.chassis]
    task_traj.subtask_names = [CHASSIS_SUBTASK_POSE]

    triad = FrameTriad()
    triad.header.timestamp_ns = now_ns()
    triad.header.frame_id = frame_id
    triad.body_frame_id = "base_link"
    triad.reference_frame_id = reference_frame_id
    triad.pose = Pose([x, y, 0.0], yaw_to_quaternion(yaw))

    command = TaskCommand()
    command.time_from_start_s = time_from_start_s
    command.subtask_commands = [triad]
    task_traj.task_commands = [command]

    target.target_task_trajectory_map = {CHASSIS_TASK_NAME: task_traj}
    return target

```

```

def build_chassis_twist_target(vx, vy, wz, time_from_start_s):
    target = make_empty_target()

    task_traj = TargetTaskTrajectory()
    task_traj.target_config = make_twist_target_config()
    task_traj.group_names = [G1JointGroup.chassis]
    task_traj.subtask_names = [CHASSIS_SUBTASK_TWIST]

    twist = Twist()
    twist.linear = make_vector3(vx, vy, 0.0)
    twist.angular = make_vector3(0.0, 0.0, wz)

    triad = FrameTriad()
    triad.header.timestamp_ns = now_ns()
    triad.header.frame_id = "base_link"
    triad.body_frame_id = "base_link"
    triad.reference_frame_id = "base_link"
    triad.twist = twist

```

```

    command = TaskCommand()
    command.time_from_start_s = time_from_start_s
    command.subtask_commands = [triad]
    task_traj.task_commands = [command]

    target.target_task_trajectory_map = {CHASSIS_TASK_NAME: task_traj}
    return target

def build_stop_twist_target():
    return build_chassis_twist_target(0.0, 0.0, 0.0, 0.1)

def merge_targets(targets):
    merged = make_empty_target()
    group_map = {}
    task_map = {}
    for target in targets:
        group_map.update(target.target_group_trajectory_map)
        task_map.update(target.target_task_trajectory_map)
    merged.target_group_trajectory_map = group_map
    merged.target_task_trajectory_map = task_map
    return merged

def ensure_controller(robot, controller_name):
    status = robot.switch_controller(controller_name)
    print(f"switch_controller({controller_name}): {status_to_string(status)}")
    return status

def run_twist_scene(robot, scene_name, target, twist_duration_s):
    print(f"{scene_name}: start moving for {twist_duration_s} seconds")
    motion_status = robot.publish_target(target)
    print(f"{scene_name} publish_target status: {status_to_string(motion_status)}")
    if motion_status != ControlStatus.SUCCESS:
        return motion_status

    time.sleep(twist_duration_s)
    print(f"{scene_name}: send stop twist target")
    stop_status = robot.publish_target(build_stop_twist_target())
    print(f"{scene_name} stop status: {status_to_string(stop_status)}")
    return stop_status

def print_menu():
    print(
        "\nAvailable commands:\n"
        "  joint      - publish a joint-only head target\n"
        "  base_pose   - publish a chassis pose target\n"
        "  base_twist  - publish a chassis twist target with auto stop\n"
        "  mixed_pose  - publish head + left_arm + chassis pose in one target\n"
        "  mixed_twist - publish head + left_arm + chassis twist in one target\n"
        "  quit       - exit example\n"
    )

def main():
    robot = GalbotRobot()

```

```

if not robot.init():
    print("robot init failed")
    return

time.sleep(2)

head_joint_names = robot.get_joint_names(True, [G1JointGroup.head])
left_arm_joint_names = robot.get_joint_names(True, [G1JointGroup.left_arm])
if not head_joint_names or not left_arm_joint_names:
    print("failed to fetch active head/left_arm joints")
    robot.request_shutdown()
    robot.wait_for_shutdown()
    robot.destroy()
    return

head_single_joint = [head_joint_names[0]]
arm_single_joint = [left_arm_joint_names[0]]
joint_time_s = 3.0
pose_time_s = 4.0
twist_command_time_s = 0.2
twist_duration_s = 2.0

print_menu()

while True:
    command = input("Enter command: ").strip()
    if command == "quit":
        break

    if command == "joint":
        target = build_joint_target(G1JointGroup.head, head_single_joint, [0.2],
joint_time_s)
        status = robot.publish_target(target)
        print(f"joint publish_target status: {status_to_string(status)}")
        continue

    if command == "base_pose":
        if ensure_controller(robot, G1ControllerName.CHASSIS_POSE_CTRL) !=
ControlStatus.SUCCESS:
            continue
        target = build_chassis_pose_target(0.2, 0.0, 0.0, pose_time_s)
        status = robot.publish_target(target)
        print(f"base_pose publish_target status: {status_to_string(status)}")
        continue

    if command == "base_twist":
        if ensure_controller(robot, G1ControllerName.CHASSIS_TWIST_CTRL) !=
ControlStatus.SUCCESS:
            continue
        target = build_chassis_twist_target(0.05, 0.0, 0.0, twist_command_time_s)
        run_twist_scene(robot, "base_twist", target, twist_duration_s)
        continue

    if command == "mixed_pose":
        if ensure_controller(robot, G1ControllerName.CHASSIS_POSE_CTRL) !=
ControlStatus.SUCCESS:
            continue
        target = merge_targets(
            [
                build_joint_target(G1JointGroup.head, head_single_joint, [0.2],

```

```

joint_time_s),
            build_joint_target(G1JointGroup.left_arm, arm_single_joint, [0.1],
joint_time_s),
            build_chassis_pose_target(0.1, 0.0, 0.0, pose_time_s),
        ]
    )
    status = robot.publish_target(target)
    print(f"mixed_pose publish_target status: {status_to_string(status)}")
    continue

    if command == "mixed_twist":
        if ensure_controller(robot, G1ControllerName.CHASSIS_TWIST_CTRL) !=
ControlStatus.SUCCESS:
            continue
        target = merge_targets(
            [
                build_joint_target(G1JointGroup.head, head_single_joint, [-0.2],
joint_time_s),
                build_joint_target(G1JointGroup.left_arm, arm_single_joint, [-0.1],
joint_time_s),
                build_chassis_twist_target(0.05, 0.0, 0.0, twist_command_time_s),
            ]
        )
        run_twist_scene(robot, "mixed_twist", target, twist_duration_s)
        continue

    print(f"Unknown command: {command}")
    print_menu()

    robot.request_shutdown()
    robot.wait_for_shutdown()
    robot.destroy()

if __name__ == "__main__":
    main()

```

直接请求原始目标 (request_target)

适用场景：高级用户直接构造 SingorIXTarget 并通过 request 通道下发到底层 WBCS，适合需要获取服务端错误返回的 raw target 场景。

```

examples/python/galbot_robot/request_target_example.py
"""
G1 RequestTarget menu example for SDK mirror SingorIXTarget.

This example shows how to construct `SingorIXTarget` directly at the Python SDK
layer and send it to the low-level WBCS through `request_target()`.

1. Joint-space commands are written into `target_group_trajectory_map`
2. Chassis pose / twist style task-space commands are written into
`target_task_trajectory_map`
3. One `SingorIXTarget` can contain both joint trajectory and task trajectory
4. The current SDK does not automatically switch the chassis controller, so
pose / twist / mixed scenes explicitly call `switch_controller(...)`
5. The base twist scene automatically sends a zero-twist target after the
configured duration to stop the chassis
"""

```

```

import math
import time

from galbot_sdk.g1 import (
    ControlStatus,
    ErrorInfo,
    G1ControllerName,
    G1JointGroup,
    GalbotRobot,
    GroupCommand,
    JointCommand,
    Pose,
    SingoriXTarget,
    TargetConfig,
    TargetGroupTrajectory,
    TargetSampling,
    TargetTaskTrajectory,
    TaskCommand,
    Twist,
    Vector3,
    FrameTriad,
    TARGET_DATA_DEFAULT,
    TARGET_DATA_FRAME_POSE,
    TARGET_DATA_FRAME_TWIST,
    TARGET_TYPE_DEFAULT,
    TARGET_TYPE_OVERRIDE,
    TARGET_TYPE_PROVERRIDE,
)

CHASSIS_TASK_NAME = "chassis"
CHASSIS_SUBTASK_POSE = "chassis_pose"
CHASSIS_SUBTASK_TWIST = "chassis_twist"

def now_ns():
    return time.time_ns()

def yaw_to_quaternion(yaw):
    return [0.0, 0.0, math.sin(yaw * 0.5), math.cos(yaw * 0.5)]

def make_empty_target():
    target = SingoriXTarget()
    target.header.timestamp_ns = now_ns()
    target.header.frame_id = "base_link"
    return target

def make_group_target_config():
    config = TargetConfig()
    config.target_data = TARGET_DATA_DEFAULT
    config.target_type = TARGET_TYPE_PROVERRIDE
    config.target_sampling = TargetSampling.TARGET_SAMPLING_DEFAULT
    config.target_priority = 1
    return config

def make_pose_target_config():

```



```

    config = TargetConfig()
    config.target_data = TARGET_DATA_FRAME_POSE
    config.target_type = TARGET_TYPE_OVERRIDE
    config.target_sampling = TargetSampling.TARGET_SAMPLING_LINEAR_INTERPOLATE
    config.target_priority = 1
    return config

def make_twist_target_config():
    config = TargetConfig()
    config.target_data = TARGET_DATA_FRAME_TWIST
    config.target_type = TARGET_TYPE_OVERRIDE
    config.target_sampling = TargetSampling.TARGET_SAMPLING_DIRECT_PASS
    config.target_priority = 1
    return config

def make_joint_command(position):
    command = JointCommand()
    command.position = position
    command.velocity = 0.0
    command.acceleration = 0.0
    command.effort = 0.0
    return command

def make_vector3(x, y, z):
    vec = Vector3()
    vec.x = x
    vec.y = y
    vec.z = z
    return vec

def build_joint_target(group_name, joint_names, positions, time_from_start_s):
    target = make_empty_target()

    group_traj = TargetGroupTrajectory()
    group_traj.target_config = make_group_target_config()
    group_traj.joint_names = list(joint_names)

    command = GroupCommand()
    command.time_from_start_s = time_from_start_s
    command.joint_commands = [make_joint_command(position) for position in positions]
    group_traj.group_commands = [command]

    target.target_group_trajectory_map = {group_name: group_traj}
    return target

def build_chassis_pose_target(x, y, yaw, time_from_start_s, frame_id="odom",
reference_frame_id="odom"):
    target = make_empty_target()

    task_traj = TargetTaskTrajectory()
    task_traj.target_config = make_pose_target_config()
    task_traj.group_names = [G1JointGroup.chassis]
    task_traj.subtask_names = [CHASSIS_SUBTASK_POSE]

    triad = FrameTriad()

```

```

    triad.header.timestamp_ns = now_ns()
    triad.header.frame_id = frame_id
    triad.body_frame_id = "base_link"
    triad.reference_frame_id = reference_frame_id
    triad.pose = Pose([x, y, 0.0], yaw_to_quaternion(yaw))

    command = TaskCommand()
    command.time_from_start_s = time_from_start_s
    command.subtask_commands = [triad]
    task_traj.task_commands = [command]

    target.target_task_trajectory_map = {CHASSIS_TASK_NAME: task_traj}
    return target

def build_chassis_twist_target(vx, vy, wz, time_from_start_s):
    target = make_empty_target()

    task_traj = TargetTaskTrajectory()
    task_traj.target_config = make_twist_target_config()
    task_traj.group_names = [G1JointGroup.chassis]
    task_traj.subtask_names = [CHASSIS_SUBTASK_TWIST]

    twist = Twist()
    twist.linear = make_vector3(vx, vy, 0.0)
    twist.angular = make_vector3(0.0, 0.0, wz)

    triad = FrameTriad()
    triad.header.timestamp_ns = now_ns()
    triad.header.frame_id = "base_link"
    triad.body_frame_id = "base_link"
    triad.reference_frame_id = "base_link"
    triad.twist = twist

    command = TaskCommand()
    command.time_from_start_s = time_from_start_s
    command.subtask_commands = [triad]
    task_traj.task_commands = [command]

    target.target_task_trajectory_map = {CHASSIS_TASK_NAME: task_traj}
    return target

def build_stop_twist_target():
    return build_chassis_twist_target(0.0, 0.0, 0.0, 0.1)

def merge_targets(targets):
    merged = make_empty_target()
    group_map = {}
    task_map = {}
    for target in targets:
        group_map.update(target.target_group_trajectory_map)
        task_map.update(target.target_task_trajectory_map)
    merged.target_group_trajectory_map = group_map
    merged.target_task_trajectory_map = task_map
    return merged

def print_error_info(scene_name, error_info):

```

```

    if error_info is None:
        print(f"{scene_name}: request_target returned None")
        return
    if not isinstance(error_info, ErrorInfo):
        print(f"{scene_name}: unexpected response type {type(error_info)}")
        return
    if not error_info.error_vec:
        print(f"{scene_name}: request_target success, service returned no errors")
        return

    print(f"{scene_name}: request_target returned {len(error_info.error_vec)} error
entries:")
    for error in error_info.error_vec:
        print(
            f"    component={error.commpent}, code={error.error_code},
description={error.description}"
        )

def ensure_controller(robot, controller_name):
    status = robot.switch_controller(controller_name)
    print(f"switch_controller({controller_name}): {getattr(status, 'name', status)}")
    return status

def run_twist_scene(robot, scene_name, target, twist_duration_s):
    print(f"{scene_name}: start moving for {twist_duration_s} seconds")
    print_error_info(scene_name, robot.request_target(target))
    time.sleep(twist_duration_s)
    print(f"{scene_name}: send stop twist target")
    print_error_info(f"{scene_name}_stop", robot.request_target(build_stop_twist_target()))

def print_menu():
    print(
        "\nAvailable commands:\n"
        "  joint      - request a joint-only head target\n"
        "  base_pose   - request a chassis pose target\n"
        "  base_twist  - request a chassis twist target with auto stop\n"
        "  mixed_pose  - request head + left_arm + chassis pose in one target\n"
        "  mixed_twist - request head + left_arm + chassis twist in one target\n"
        "  quit       - exit example\n"
    )

def main():
    robot = GalbotRobot()
    if not robot.init():
        print("robot init failed")
        return

    time.sleep(2)

    head_joint_names = robot.get_joint_names(True, [G1JointGroup.head])
    if not head_joint_names:
        print("failed to fetch active head joints")
        robot.request_shutdown()
        robot.wait_for_shutdown()
        robot.destroy()
        return

```

```

head_single_joint = [head_joint_names[0]]
joint_time_s = 3.0
pose_time_s = 4.0
twist_command_time_s = 0.2
twist_duration_s = 2.0

print_menu()

while True:
    command = input("Enter command: ").strip()
    if command == "quit":
        break

    if command == "joint":
        target = build_joint_target(G1JointGroup.head, head_single_joint, [0.2],
joint_time_s)
        print_error_info("joint", robot.request_target(target))
        continue

    if command == "base_pose":
        if ensure_controller(robot, G1ControllerName.CHASSIS_POSE_CTRL) !=
ControlStatus.SUCCESS:
            continue
        target = build_chassis_pose_target(0.2, 0.0, 0.0, pose_time_s)
        print_error_info("base_pose", robot.request_target(target))
        continue

    if command == "base_twist":
        if ensure_controller(robot, G1ControllerName.CHASSIS_TWIST_CTRL) !=
ControlStatus.SUCCESS:
            continue
        target = build_chassis_twist_target(0.05, 0.0, 0.0, twist_command_time_s)
        run_twist_scene(robot, "base_twist", target, twist_duration_s)
        continue

    if command == "mixed_pose":
        if ensure_controller(robot, G1ControllerName.CHASSIS_POSE_CTRL) !=
ControlStatus.SUCCESS:
            continue
        target = merge_targets(
            [
                build_joint_target(G1JointGroup.head, head_single_joint, [0.2],
joint_time_s),
                build_chassis_pose_target(0.1, 0.0, 0.0, pose_time_s),
            ]
        )
        print_error_info("mixed_pose", robot.request_target(target))
        continue

    if command == "mixed_twist":
        if ensure_controller(robot, G1ControllerName.CHASSIS_TWIST_CTRL) !=
ControlStatus.SUCCESS:
            continue
        target = merge_targets(
            [
                build_joint_target(G1JointGroup.head, head_single_joint, [-0.2],
joint_time_s),
                build_chassis_twist_target(0.05, 0.0, 0.0, twist_command_time_s),
            ]
        )

```

```

    )
    run_twist_scene(robot, "mixed_twist", target, twist_duration_s)
    continue

    print(f"Unknown command: {command}")
    print_menu()

    robot.request_shutdown()
    robot.wait_for_shutdown()
    robot.destroy()

if __name__ == "__main__":
    main()

```

设置底盘速度 (set_base_velocity)

适用场景：控制移动底盘的线速度和角速度，用于导航或远程遥控。

```

examples/python/galbot_robot/set_base_velocity.py
from galbot_sdk.g1 import GalbotRobot, ControlStatus
import time

# Get GalbotRobot
robot = GalbotRobot()
robot.init()
time.sleep(1)
print("Initialization succeeded")

# Set chassis speed
linear_velocity = [0.05, 0.0, 0.0] # 0.5 m/s
angular_velocity = [0.0, 0.0, 0.1] # 0.1 rad/s

duration_s = 2.0 # Automatically stop after 2 seconds
status = robot.set_base_velocity(linear_velocity, angular_velocity, duration_s)

if status == ControlStatus.SUCCESS:
    print(f"Chassis speed set successfully; will auto-stop after {duration_s} seconds.")
else:
    print("Set chassis speed failed.")

time.sleep(duration_s + 0.5)

# send SIGINT shutdown signal
robot.request_shutdown()
# Wait until entering shutdown state
robot.wait_for_shutdown()
# Perform SDK resource release
robot.destroy()
print('Resources released successfully')

```

获取关节状态 (get_joint_states)

适用场景：获取所有关节的完整状态信息，包括位置、速度、电流等。适用于需要完整关节信息的算法（如动力学计算、状态估计）。

```

examples/python/galbot_robot/get_joint_states.py

```

```

import time
from galbot_sdk.g1 import GalbotRobot

def print_joint_states(joint_states):
    """
    joint_state_vec: List of JointState; each object has position, velocity, acceleration,
    effort, current
    """
    for js in joint_states:
        print(f" : position = {js.position} , velocity = {js.velocity} "
              f", acceleration = {js.acceleration} , effort = {js.effort} , current = "
              f"{js.current}")

# Get and initialize the GalbotRobot singleton
robot = GalbotRobot()
robot.init()
# Program started, waiting for data
time.sleep(1)
print("Initialization succeeded")
# Get joint states by joint group names; returns all joints if empty
joint_group_names = ["left_arm"]
ret = robot.get_joint_states(joint_group_names, [])
print_joint_states(ret)

# Get specified joint states; if provided, overrides joint group input
joint_names = ["left_arm_joint1", "left_arm_joint2"]
state_ret = robot.get_joint_states([], joint_names)
print_joint_states(state_ret)

# send SIGINT shutdown signal
robot.request_shutdown()
# Wait until entering shutdown state
robot.wait_for_shutdown()
# Perform SDK resource release
robot.destroy()
print('Resources released successfully')

```

获取关节位置 (get_joint_positions)

适用场景：仅获取当前关节位置。当你只需要关节位置信息时使用此接口，比 get_joint_states 更轻量。

```

examples/python/galbot_robot/get_joint_positions.py
import time
from galbot_sdk.g1 import GalbotRobot

def print_joint_positions(joint_positions):
    print(f"pos count is {len(joint_positions)}")
    for pos in joint_positions:
        print(pos)

# Get and initialize the GalbotRobot singleton
robot = GalbotRobot()
robot.init()
# Program started, waiting for data
time.sleep(1)
print("Initialization succeeded")

# Get joint positions by joint group names; returns all joints if empty

```

```

joint_group_names = ["left_arm"]
ret = robot.get_joint_positions(joint_group_names, [])
print("Left arm joint positions:")
print_joint_positions(ret)
# Get specified joint positions; if provided, overrides joint group input
joint_names = ["left_arm_joint1", "left_arm_joint2"]
state_ret = robot.get_joint_positions([], joint_names)
print("Left arm joints 1 and 2 positions:")
print_joint_positions(state_ret)

# send SIGINT shutdown signal
robot.request_shutdown()
# Wait until entering shutdown state
robot.wait_for_shutdown()
# Perform SDK resource release
robot.destroy()
print('Resources released successfully')

```

获取关节名称 (get_joint_names)

适用场景：获取机器人所有关节的名称列表，用于迭代遍历关节或配置文件生成。

```

examples/python/galbot_robot/get_joint_names.py
import time
from galbot_sdk.g1 import GalbotRobot

# Get and initialize the GalbotRobot singleton
robot = GalbotRobot()
robot.init()
# Program started, waiting for data
time.sleep(1)
print("Initialization succeeded")

# Get joint positions by joint group names; returns all joints if empty
joint_group_names = ["left_arm"]
# get joint
only_active_joint = True
ret = robot.get_joint_names(only_active_joint, joint_group_names)
print("Left joint names:")
for i, name in enumerate(ret):
    print(f"{i + 1}: {name}")

# send SIGINT shutdown signal
robot.request_shutdown()
# Wait until entering shutdown state
robot.wait_for_shutdown()
# Perform SDK resource release
robot.destroy()
print('Resources released successfully')

```

获取夹爪状态 (get_gripper_state)

适用场景：获取夹爪当前位置和状态，判断夹爪当前是打开还是闭合。

```

examples/python/galbot_robot/get_gripper_state.py
import time
from galbot_sdk.g1 import GalbotRobot, G1JointGroup

def print_gripper_state(joint_group, gripper_state):

```

```

"""
joint_group: G1JointGroup enum value
gripper_state: object including timestamp_ns, width, velocity, effort, is_moving
"""

print(f"Timestamp (ns): {gripper_state.timestamp_ns}")
print(
    f"width {gripper_state.width} "
    f"velocity {gripper_state.velocity} "
    f"effort {gripper_state.effort} "
    f"is moving {gripper_state.is_moving}"
)

# Get and initialize the GalbotRobot singleton
robot = GalbotRobot()
robot.init()

# Program started, waiting for data
time.sleep(1)
print("Initialization succeeded")

# Set gripper joint group (left gripper)
joint_group = G1JointGroup.left_gripper

# Get gripper state
gripper_state = robot.get_gripper_state(joint_group)

if gripper_state is None:
    print("get gripper state error")
else:
    print("Left gripper state is as follows:")
    print_gripper_state(joint_group, gripper_state)

# send SIGINT shutdown signal
robot.request_shutdown()
# Wait until entering shutdown state
robot.wait_for_shutdown()
# Perform SDK resource release
robot.destroy()
print('Resources released successfully')

```

获取吸盘状态 (get_suction_cup_state)

适用场景：获取吸盘当前是否处于吸合状态，以及是否成功检测到物体。

```

examples/python/galbot_robot/get_suction_cup_state.py
import time
from galbot_sdk.g1 import GalbotRobot, G1JointGroup

def print_suction_cup_state(suction_cup_state):
    """
    suction_cup_state: object including timestamp_ns, pressure, activation, action_state
    """
    group_name = joint_group.name
    print(f"Timestamp (ns): {suction_cup_state.timestamp_ns}")
    print(
        f"pressure {suction_cup_state.pressure} "
        f"activation {suction_cup_state.activation} "
        f"action state {int(suction_cup_state.action_state)}"
    )

```



```

# Get and initialize the GalbotRobot singleton
robot = GalbotRobot()
robot.init()

# Program started, waiting for data
time.sleep(1)
print("Initialization succeeded")

# Set suction cup joint group (right suction cup)
joint_group = G1JointGroup.right_suction_cup

# Get suction cup state
suction_cup_state = robot.get_suction_cup_state(joint_group)

if suction_cup_state is None:
    print("get suction cup error")
else:
    print("Right suction cup status:")
    print_suction_cup_state(suction_cup_state)

# send SIGINT shutdown signal
robot.request_shutdown()
# Wait until entering shutdown state
robot.wait_for_shutdown()
# Perform SDK resource release
robot.destroy()
print('Resources released successfully')

```

获取坐标变换 (get_transform)

适用场景：获取两个坐标系之间的变换关系（位姿），用于机械臂抓取、视觉定位等场景。

```

examples/python/galbot_robot/get_transform.py
import time
from galbot_sdk.g1 import GalbotRobot

def print_pose(pose_vec):
    """
    pose_vec: list of floats
    """
    print("pose_vec = [" + ", ".join(str(p) for p in pose_vec) + "]")

# Get and initialize the GalbotRobot singleton
robot = GalbotRobot()
robot.init()

# Program started, waiting for data
time.sleep(1)
print("Initialization succeeded")

# Set target frame and source frame
target_frame = "base_link"
source_frame = "left_arm_link1"
timestamp_ns = 0    # 0 means fetch the latest TF transform value

# Get coordinate transform
ret_val = robot.get_transform(target_frame, source_frame)

```

```

if not ret_val[0]:
    print("get_transform error")
else:
    print("tf_timestamp_ns:", ret_val[1])
    print_pose(ret_val[0])

# send SIGINT shutdown signal
robot.request_shutdown()
# Wait until entering shutdown state
robot.wait_for_shutdown()
# Perform SDK resource release
robot.destroy()
print('Resources released successfully')

```

获取 IMU 数据 (get_imu_data)

适用场景：获取机器人基座 IMU 的加速度、角速度和姿态信息，用于状态估计、运动分析。

```

examples/python/galbot_robot/get_imu_data.py
import time
from galbot_sdk.g1 import GalbotRobot, SensorType
from typing import Dict

def print_imu_data(imu_data: Dict):
    """
    imu_data: dict, includes:
    - 'timestamp_ns'
    - 'accel' : {'x', 'y', 'z'}
    - 'gyro' : {'x', 'y', 'z'}
    - 'magnet' : {'x', 'y', 'z'}
    """
    if not imu_data:
        print("IMU data is empty")
        return

    print(f"Timestamp (ns): {imu_data.get('timestamp_ns')}")

    accel = imu_data.get("accel", {})
    gyro = imu_data.get("gyro", {})
    magnet = imu_data.get("magnet", {})

    print(
        f"Accelerometer: x={accel.get('x')}, "
        f"y={accel.get('y')}, "
        f"z={accel.get('z')}"
    )
    print(
        f"Gyroscope: x={gyro.get('x')}, "
        f"y={gyro.get('y')}, "
        f"z={gyro.get('z')}"
    )
    print(
        f"Magnetometer: x={magnet.get('x')}, "
        f"y={magnet.get('y')}, "
        f"z={magnet.get('z')}"
    )

# Get and initialize the GalbotRobot singleton
robot = GalbotRobot()

```

```

robot.init({SensorType.TORSO_IMU})

# Program started, waiting for data
time.sleep(1)
print("Initialization succeeded")

imu_data = robot.get_imu_data(SensorType.TORSO_IMU)
if not imu_data:
    print("No imu data!")
else:
    print("IMU data:")
    print_imu_data(imu_data)

# send SIGINT shutdown signal
robot.request_shutdown()
# Wait until entering shutdown state
robot.wait_for_shutdown()
# Perform SDK resource release
robot.destroy()
print('Resources released successfully')

```

获取设备信息 (get_device_information)

适用场景：获取机器人硬件版本、固件版本等设备信息，用于调试和兼容性检查。

```

examples/python/galbot_robot/get_device_information.py
import time
from galbot_sdk.g1 import GalbotRobot
from typing import Dict

def print_device_info(device_info: dict):
    """
    device_info: dict, including the following fields:
    - 'model': Device model (str)
    - 'serial_number': Serial number (str)
    - 'firmware_version': Firmware version (str)
    - 'hardware_version': Hardware version (str)
    - 'manufacturer': Manufacturer (str)
    """
    if not device_info:
        print("Device information is empty")
        return

    print("Device information:")
    print(f"  Model: {device_info.get('model', 'N/A')}")
    print(f"  Serial number: {device_info.get('serial_number', 'N/A')}")
    print(f"  Firmware version: {device_info.get('firmware_version', 'N/A')}")
    print(f"  Hardware version: {device_info.get('hardware_version', 'N/A')}")
    print(f"  : {device_info.get('manufacturer', 'N/A')}")

# Get and initialize the GalbotRobot singleton
robot = GalbotRobot()
robot.init()

# Program started, waiting for data
time.sleep(1)
print("Initialization succeeded")

device_info = robot.get_device_information()

```

```

if not device_info:
    print("Failed to get device information!")
else:
    print("Device information retrieved successfully!")
    print_device_info(device_info)

# send SIGINT shutdown signal
robot.request_shutdown()
# Wait until entering shutdown state
robot.wait_for_shutdown()
# Perform SDK resource release
robot.destroy()
print('Resources released successfully')

```

获取电池信息 (get_bms_information)

适用场景：获取电池电压、电量等信息，用于低电量检测和状态监控。

examples/python/galbot_robot/get_bms_information.py

```

import time
from galbot_sdk.g1 import GalbotRobot

def print_bms_information(bms_info: dict):
    """
    bms_info: dict, includes:
    - 'voltage'          : float (V)
    - 'current'          : float (A)
    - 'battery_level'    : float (0-100)
    - 'temperature'      : float (°C)
    - 'charging_status'  : str or int
    - 'health_status'    : str or int
    - 'capacity'         : float (Ah)
    """
    if not bms_info:
        print("BMS information is empty")
        return

    print(f"Voltage (V): {bms_info.get('voltage')}")
    print(f"Current (A): {bms_info.get('current')}")
    print(f"Battery level (%): {bms_info.get('battery_level')}")
    print(f"Temperature (C): {bms_info.get('temperature')}")
    print(f"Charging status: {bms_info.get('charging_status')}")
    print(f"Health status: {bms_info.get('health_status')}")
    print(f"Capacity (Ah): {bms_info.get('capacity')}")

# Get and initialize the GalbotRobot singleton
robot = GalbotRobot()
robot.init()

# Program started, waiting for data
time.sleep(3)
print("Initialization succeeded")

bms_info = robot.get_bms_information()
print_bms_information(bms_info)

# send SIGINT shutdown signal

```

```

robot.request_shutdown()
# Wait until entering shutdown state
robot.wait_for_shutdown()
# Perform SDK resource release
robot.destroy()
print("Resources released successfully")

```

获取相机图像数据 (get_rgb_data && get_depth_data)

适用场景：获取 RGB 图像和深度图像数据，用于视觉感知、目标检测、SLAM 等任务。

```

examples/python/galbot_robot/get_camera_data.py
try:
    from galbot_sdk.g1 import GalbotRobot, SensorType
except ImportError:
    print("import galbot_sdk failed, please install it first or check if it is in the PYTHONPATH")
    exit(1)

import os

try:
    import cv2
except ImportError:
    os.system("pip install opencv-python")
    import cv2

try:
    import numpy as np
except ImportError:
    os.system("pip install numpy")
    import numpy as np

import time
from typing import Dict

def decode_compressed_image(compressed_image):
    """
    decode CompressedImage image

    Parameters:
        compressed_image (dict): image dict, keys:[header, format, data, "depth_scale"]

    Returns:
        numpy.ndarray: decoded image
    """
    image_data = compressed_image["data"]
    if compressed_image["format"] == "rgb8":
        return decode_rgb_image(image_data)
    elif compressed_image["format"] == "16UC1":
        return decode_depth_image(compressed_image)
    else:
        raise ValueError(f"Unsupport data format: {compressed_image['format']}")

def decode_rgb_image(image_data):
    """decode rgb image"""
    nparr = np.frombuffer(image_data, np.uint8)
    img = cv2.imdecode(nparr, cv2.IMREAD_COLOR)
    if img is None:

```

```

        raise ValueError("Fail to Decode RGB Image")
    return img

def decode_depth_image(image_data):
    """decode depth image"""
    depth_img = np.frombuffer(image_data["data"], dtype=np.uint16).copy()

    # Check whether height and width exist
    if "height" not in image_data or "width" not in image_data:
        raise ValueError("Missing 'height' or 'width' in depth image metadata.")
    if image_data["height"] == 0 or image_data["width"] == 0:
        raise ValueError(f"Invalid 'height' ({image_data['height']}) or 'width' ({image_data['width']}) in depth image metadata.")

    # Parse depth image
    depth_img = depth_img.reshape((image_data["height"], image_data["width"]))
    depth_img = depth_img.astype(np.float32) / image_data["depth_scale"]

    return depth_img

def main():
    SHOW_IMAGE = False
    robot = GalbotRobot()

    # Get left arm RGB and depth images, right arm depth image, chassis lidar data, torso
    # IMU data
    enable_sensor_set = {SensorType.LEFT_ARM_CAMERA, # Left arm depth camera
                        SensorType.LEFT_ARM_DEPTH_CAMERA,} # Left arm RGB camera
    robot.init(enable_sensor_set)
    print("Initialization succeeded")
    # Program started, waiting for data
    time.sleep(5)
    # Get left arm RGB image
    rgb_image_data = robot.get_rgb_data(SensorType.LEFT_ARM_CAMERA)
    if not rgb_image_data:
        print("No rgb image data!")
    else:
        print("get rgb image success")
        print(rgb_image_data['header'])
        img = decode_compressed_image(rgb_image_data)

        # Save RGB image
        cv2.imwrite("rgb_image_data.jpg", img)
        # 可视化 RGB 图像
        if SHOW_IMAGE:
            cv2.namedWindow("rgb image", cv2.WINDOW_NORMAL)
            cv2.imshow("rgb image", img)
            cv2.waitKey(0)
            cv2.destroyAllWindows()

    # Get left arm depth image
    depth_data = robot.get_depth_data(SensorType.LEFT_ARM_DEPTH_CAMERA)
    if not depth_data or "data" not in depth_data:
        print("Depth camera not ready")
    else:
        print("get depth data success")
        print(depth_data['header'])
        depth_img_raw = decode_compressed_image(depth_data)
        depth_img = cv2.normalize(depth_img_raw, None, 0, 255, cv2.NORM_MINMAX) #

```

```

Normalize depth values to 0-1 range
depth_img = depth_img.astype(np.uint8)

# Save depth image
cv2.imwrite("depth_data.jpg", depth_img)
# 可视化深度图
if SHOW_IMAGE:
    cv2.namedWindow("depth image", cv2.WINDOW_NORMAL)
    cv2.imshow("depth image", depth_img)
    cv2.waitKey(0)
    cv2.destroyAllWindows()

# send SIGINT shutdown signal
robot.request_shutdown()
# Wait until entering shutdown state
robot.wait_for_shutdown()
# Perform SDK resource release
robot.destroy()
print('Resources released successfully')

if __name__=="__main__":
    main()

```

获取传感器内外参 (get_camera_intrinsic && get_sensor_extrinsic)

适用场景：获取相机内参和传感器相对于机器人基座的外参，用于计算机视觉计算和 3D 重建。

```

examples/python/galbot_robot/get_camera_params.py
try:
    from galbot_sdk.g1 import GalbotRobot, SensorType
except ImportError:
    print("import galbot_sdk failed, please install it first or check if it is in the PYTHONPATH")
    exit(1)

import os

try:
    import cv2
except ImportError:
    os.system("pip install opencv-python")
    import cv2

try:
    import numpy as np
except ImportError:
    os.system("pip install numpy")
    import numpy as np

import time
from typing import Dict

def decode_compressed_image(compressed_image):
    """
    decode CompressedImage image

    Parameters:
        compressed_image (dict): image dict, keys:[header, format, data, "depth_scale"]

```

```

Returns:
    numpy.ndarray: decoded image
"""
image_data = compressed_image["data"]
if compressed_image["format"] == "rgb8":
    return decode_rgb_image(image_data)
elif compressed_image["format"] == "16UC1":
    return decode_depth_image(compressed_image)
else:
    raise ValueError(f"Unsupport data format: {compressed_image['format']}")

def decode_rgb_image(image_data):
    """decode rgb image"""
    nparr = np.frombuffer(image_data, np.uint8)
    img = cv2.imdecode(nparr, cv2.IMREAD_COLOR)
    if img is None:
        raise ValueError("Fail to Decode RGB Image")
    return img

def decode_depth_image(image_data):
    """decode depth image"""
    depth_img = np.frombuffer(image_data["data"], dtype=np.uint16).copy()

    # Check whether height and width exist
    if "height" not in image_data or "width" not in image_data:
        raise ValueError("Missing 'height' or 'width' in depth image metadata.")
    if image_data["height"] == 0 or image_data["width"] == 0:
        raise ValueError(f"Invalid 'height' ({image_data['height']}) or 'width' ({image_data['width']}) in depth image metadata.")

    # Parse depth image
    depth_img = depth_img.reshape((image_data["height"], image_data["width"]))
    depth_img = depth_img.astype(np.float32) / image_data["depth_scale"]

    return depth_img

def main():
    robot = GalbotRobot()

    # Get left arm RGB and depth images, right arm depth image, chassis lidar data, torso
    IMU data
    enable_sensor_set = {SensorType.LEFT_ARM_CAMERA, # Left arm depth camera
                        SensorType.LEFT_ARM_DEPTH_CAMERA,} # Left arm RGB camera
    robot.init(enable_sensor_set)
    print("Initialization succeeded")

    # Program started, waiting for data
    time.sleep(5)

    # Get left arm RGB image
    rgb_image_data = robot.get_rgb_data(SensorType.LEFT_ARM_CAMERA)
    if not rgb_image_data:
        print("No rgb image data!")
    else:
        print("get rgb image suceess")
        print(rgb_image_data['header'])
        img = decode_compressed_image(rgb_image_data)

    # Save RGB image
    cv2.imwrite("rgb_image_data.jpg", img)

```



```

# Get left arm camera intrinsics
camera_intrinsics = robot.get_camera_intrinsic(SensorType.LEFT_ARM_CAMERA)
if not camera_intrinsics:
    print("No camera intrinsics data!")
else:
    print("get camera intrinsics suceess")
    print(camera_intrinsics)

# Get left arm depth image
depth_data = robot.get_depth_data(SensorType.LEFT_ARM_DEPTH_CAMERA)
if not depth_data or "data" not in depth_data:
    print("Depth camera not ready")
else:
    print("get depth data suceess")
    print(depth_data['header'])
    depth_img_raw = decode_compressed_image(depth_data)
    depth_img = cv2.normalize(depth_img_raw, None, 0, 255, cv2.NORM_MINMAX) #
Normalize depth values to 0-1 range
    depth_img = depth_img.astype(np.uint8)

    # Save depth image
    cv2.imwrite("depth_data.jpg", depth_img)

# Get left-arm depth camera intrinsics
camera_intrinsics = robot.get_camera_intrinsic(SensorType.LEFT_ARM_DEPTH_CAMERA)
if not camera_intrinsics:
    print("No camera intrinsics data!")
else:
    print("get camera intrinsics suceess")
    print(camera_intrinsics)

# Extrinsics
time.sleep(2)
camera_extrinsics, timestamp_ns =
robot.get_sensor_extrinsic(SensorType.LEFT_ARM_DEPTH_CAMERA)
if not camera_extrinsics:
    print("No camera extrinsics data!")
else:
    print("get camera extrinsics suceess")
    print(camera_extrinsics)

# send SIGINT shutdown signal
robot.request_shutdown()
# Wait until entering shutdown state
robot.wait_for_shutdown()
# Perform SDK resource release
robot.destroy()
print('Resources released successfully')

if __name__=="__main__":
    main()

```

获取雷达数据 (get_lidar_data)

适用场景：获取激光雷达点云数据，用于导航、避障、环境建模。

examples/python/galbot_robot/get_lidar_data.py

```

from galbot_sdk.g1 import GalbotRobot, SensorType
from typing import Dict
import time
import numpy as np

def convert_pointcloud(cloud):
    """
    Convert cloud dict to NumPy array dictionary

    Args:
        pointcloud_msg: PointCloud2 protobuf message object

    Returns:
        Dictionary: {field_name: NumPy array}
        - Single-element fields: shape (N,)
        - Multi-element fields: shape (N, count) or (N,)
        - N = width * height (total number of points)
    """

    if not cloud:
        return {}

    num_points = cloud["height"] * cloud["width"]
    if num_points == 0:
        return {}

    DTYPE_MAP = {
        1: np.int8,
        2: np.uint8,
        3: np.int16,
        4: np.uint16,
        5: np.int32,
        6: np.uint32,
        7: np.float32,
        8: np.float64
    }
    dtype_list = []
    for field in cloud["fields"]:
        # Get base data type
        np_dtype_class = DTYPE_MAP.get(field["datatype"])
        if np_dtype_class is None:
            raise ValueError(f"Unsupported data type: {field['datatype']}")

        dtype_inst = np.dtype(np_dtype_class)

        # Handle byte order (endianness)
        if dtype_inst.itemsize > 1:
            byteorder = '>' if cloud["is_bigendian"] else '<'
            dtype_inst = dtype_inst.newbyteorder(byteorder)

        # Add to dtype list
        if field["count"] == 1:
            dtype_list.append((field["name"], dtype_inst))
        else:
            # Multi-element fields (e.g., rgb)
            dtype_list.append((field["name"], dtype_inst, field["count"]))

    # Create structured dtype
    dtype = np.dtype(dtype_list)

```

```

# Data integrity check
expected_size = num_points * cloud["point_step"]
if len(cloud["data"]) < expected_size:
    raise ValueError(
        f"Insufficient data length: expected {expected_size} bytes, "
        f"actual {len(cloud['data'])} bytes"
    )

# Create NumPy structured array from binary data
# count parameter ensures only expected number of points are read
arr = np.frombuffer(cloud["data"], dtype=dtype, count=num_points)

# Convert to regular dictionary (copy data to avoid modifying original)
result = {}
for field in cloud["fields"]:
    field_data = arr[field["name"]]

    # Handle shape of multi-element fields
    if field["count"] == 1:
        result[field["name"]] = field_data.copy()
    else:
        # Keep original shape or flatten, choose according to needs
        result[field["name"]] = field_data.copy()

return result

def get_xyz_array(pointcloud_dict: Dict[str, np.ndarray],
                  remove_nan: bool = False) -> np.ndarray:
    """
    Extract XYZ coordinate array from converted point cloud dictionary

    Args:
        pointcloud_dict: Dictionary returned by pointcloud2_to_numpy()
        remove_nan: Whether to remove points containing NaN (for FLOAT32/FLOAT64 types)

    Returns:
        Nx3 point coordinate array
    """
    required = ['x', 'y', 'z']
    if not all(k in pointcloud_dict for k in required):
        raise ValueError("Point cloud data missing required xyz fields")

    points = np.stack([pointcloud_dict['x'],
                       pointcloud_dict['y'],
                       pointcloud_dict['z']], axis=1)

    if remove_nan:
        mask = ~np.isnan(points).any(axis=1)
        points = points[mask]

    return points

def save_xyz_to_pcd(xyz_array: np.ndarray, filename: str, binary: bool = False) -> None:
    """
    Save XYZ coordinates to PCD file format (simplest option for coordinate-only data)

    Args:
        xyz_array: Nx3 array of XYZ coordinates
        filename: Output PCD file path
    """

```

```

        binary: If True, saves in binary format; otherwise ASCII
"""
if xyz_array.ndim != 2 or xyz_array.shape[1] != 3:
    raise ValueError(f"xyz_array must have shape (N, 3), got {xyz_array.shape}")

num_points = xyz_array.shape[0]
header = [
    "# .PCD v0.7 - Point Cloud Data file format",
    "VERSION 0.7",
    "FIELDS x y z",
    "SIZE 4 4 4",
    "TYPE F F F", # F = float32
    "COUNT 1 1 1",
    f"WIDTH {num_points}",
    "HEIGHT 1",
    "VIEWPOINT 0 0 0 1 0 0 0",
    f"POINTS {num_points}",
    f"DATA {'binary' if binary else 'ascii'}"
]

if binary:
    with open(filename, 'wb') as f:
        f.write(('\\n'.join(header) + '\\n').encode('ascii'))
        f.write(xyz_array.astype(np.float32).tobytes())
else:
    with open(filename, 'w') as f:
        f.write('\\n'.join(header) + '\\n')
        np.savetxt(f, xyz_array, fmt='%f')

# Get and initialize the GalbotRobot singleton
robot = GalbotRobot()
enable_sensor_set = {SensorType.BASE_LIDAR}
# To save resource overhead, only cameras and LiDAR sensors passed during initialization
# can retrieve data
robot.init(enable_sensor_set)
print("Initialization succeeded")
# Program started, waiting for data
time.sleep(4)

cloud = robot.get_lidar_data(SensorType.BASE_LIDAR)
if not cloud:
    print("No lidar data!")
else:
    pointcloud_dict = convert_pointcloud(cloud)
    xyz_points = get_xyz_array(pointcloud_dict)
    save_xyz_to_pcd(xyz_points, "output_xyz.pcd")
    print(pointcloud_dict)
    print("get lidar data success")

# send SIGINT shutdown signal
robot.request_shutdown()
# Wait until entering shutdown state
robot.wait_for_shutdown()
# Perform SDK resource release
robot.destroy()
print('Resources released successfully')

```

一键回零-odom 坐标系 (whole_body_reset_zero_odom)

适用场景：快速将全身上下所有关节运动到零位（初始姿态），基于 odom 坐标系。通常用于程序启动后的初始化。

```
examples/python/galbot_robot/whole_body_reset_zero_odom_example.py

from galbot_sdk.g1 import GalbotRobot
from galbot_sdk.g1 import GalbotMotion
import time

def main():
    robot = GalbotRobot()
    motion = GalbotMotion()

    if not robot.init():
        print("GalbotRobot init failed.")
        return
    if not motion.init():
        print("GalbotMotion init failed.")
        return

    time.sleep(2)

    # Whole-body joints: leg(5) + head(2) + left_arm(7) + right_arm(7)
    whole_body_joint_1 = [
        0.25, 1.1, 0.85, 0.0, 0.0,      # leg
        0.5, 0.5,                      # head
        2.0, -1.55, -0.55, -1.7, -0.0, -0.8, 0.2, # left_arm
        -2.0, 1.55, 0.55, 1.7, 0.0, 0.8, 0.2     # right_arm
    ]

    # Base pose command odom(x, y, yaw)
    base_x_1 = 0.2
    base_y_1 = 0.0
    base_yaw_1 = 0.0

    # Optional frames (frame_id: base_link/odom/map, reference_frame_id: odom/map)
    frame_id = "base_link"
    reference_frame_id = "odom"

    # Chassis pose interpolation time (seconds), used to generate a smooth chassis trajectory
    base_time_s = 15.0

    time.sleep(1)

    # reset to zero
    result = robot.zero_whole_body_and_base(
        frame_id,
        reference_frame_id,
        True,
        0.2,
        15.0,
    )
    print("Zero joint status:", result[0])
    print("Zero base status:", result[1])
```

```

robot.request_shutdown()
robot.wait_for_shutdown()
robot.destroy()

if __name__ == "__main__":
    main()

```

一键回零-map 坐标系 (whole_body_reset_zero_map)

适用场景：快速将全身上下所有关节运动到零位（初始姿态），基于 map 坐标系。

```

examples/python/galbot_robot/whole_body_reset_zero_map_example.py
from galbot_sdk.g1 import GalbotRobot
from galbot_sdk.g1 import GalbotMotion
import time

def main():
    robot = GalbotRobot()
    motion = GalbotMotion()

    if not robot.init():
        print("GalbotRobot init failed.")
        return
    if not motion.init():
        print("GalbotMotion init failed.")
        return

    time.sleep(2)

    # Whole-body joints: leg(5) + head(2) + left_arm(7) + right_arm(7)
    whole_body_joint_1 = [
        0.25, 1.1, 0.85, 0.0, 0.0,      # leg
        0.5, 0.5,                      # head
        2.0, -1.55, -0.55, -1.7, -0.0, -0.8, 0.2,  # left_arm
        -2.0, 1.55, 0.55, 1.7, 0.0, 0.8, 0.2      # right_arm
    ]

    # Base pose command map(x, y, yaw) Note: adjust based on the robot's actual
    localization in the map frame
    base_x_1 = -0.4
    base_y_1 = 0.226593
    base_yaw_1 = 0.0

    # Optional frames (frame_id: base_link/odom/map, reference_frame_id: odom/map)
    frame_id = "base_link"
    reference_frame_id = "map"

    # Chassis pose interpolation time (seconds), used to generate a smooth chassis
    trajectory
    base_time_s = 15.0

    time.sleep(1)

    # reset to zero
    result = robot.zero_whole_body_and_base(
        frame_id,
        reference_frame_id,

```

```

        True,
        0.2,
        15.0,
    )
    print("Zero joint status:", result[0])
    print("Zero base status:", result[1])

    robot.request_shutdown()
    robot.wait_for_shutdown()
    robot.destroy()

if __name__ == "__main__":
    main()

```

类：GalbotMotion

获取实例并初始化

适用场景：获取运动规划模块单例并初始化。必须在使用运动规划功能前调用。

```

examples/python/galbot_motion/get_instance.py
from galbot_sdk.g1 import GalbotMotion
from galbot_sdk.g1 import GalbotRobot

# Get and initialize the GalbotMotion singleton
motion = GalbotMotion()
robot = GalbotRobot()

if motion.init():
    print("GalbotMotion initialized successfully")
else:
    print("GalbotMotion initialization failed")

if robot.init():
    print("GalbotRobot initialized successfully")
else:
    print("GalbotRobot initialization failed")

# You can still manage the robot lifecycle through GalbotRobot
robot.request_shutdown()
robot.wait_for_shutdown()
robot.destroy()

```

正运动学（使用当前状态或指定 RobotStates）

适用场景：根据给定的关节角度计算末端执行器的位姿。用于机械臂任务验证和状态分析。

```

examples/python/galbot_motion/fk.py
import time
import galbot_sdk.g1 as gm
from galbot_sdk.g1 import GalbotMotion, GalbotRobot

# Get and initialize the GalbotMotion singleton
motion = GalbotMotion()
robot = GalbotRobot()

```

```

def printStatus(status):
    if(status == gm.MotionStatus.SUCCESS):
        print("Execution result: SUCCESS, execution successful")
    elif(status == gm.MotionStatus.TIMEOUT):
        print("Execution result: TIMEOUT, execution timed out")
    elif(status == gm.MotionStatus.FAULT):
        print("Execution result: FAULT, a fault occurred and execution cannot
continue")
    elif(status == gm.MotionStatus.INVALID_INPUT):
        print("Execution result: INVALID_INPUT, input parameters do not meet
requirements")
    elif(status == gm.MotionStatus.INIT_FAILED):
        print("Execution result: INIT_FAILED, failed to create internal communication
components")
    elif(status == gm.MotionStatus.IN_PROGRESS):
        print("Execution result: IN_PROGRESS, in motion but not yet in position")
    elif(status == gm.MotionStatus.STOPPED_UNREACHED):
        print("Execution result: STOPPED_UNREACHED, stopped but target not reached")
    elif(status == gm.MotionStatus.DATA_FETCH_FAILED):
        print("Execution result: DATA_FETCH_FAILED, failed to fetch data")
    elif(status == gm.MotionStatus.PUBLISH_FAIL):
        print("Execution result: PUBLISH_FAIL, data transmission failed")
    elif(status == gm.MotionStatus.COMM_DISCONNECTED):
        print("Execution result: COMM_DISCONNECTED, connection failed")

if motion.init():
    print("GalbotMotion initialized successfully")
else:
    print("GalbotMotion initialization failed")
if robot.init():
    print("GalbotRobot initialized successfully")
else:
    print("GalbotRobot initialization failed")

# Program started, waiting for data
time.sleep(1)

chain_joints = {
    "leg": [0.4992,1.4991,1.0005,0.0000,-0.0004],
    "head": [0.0000,0.0],
    "left_arm": [1.9999,-1.6000,-0.5999,-1.6999,0.0000,-0.7999,0.0000],
    "right_arm": [-2.0000,1.6001,0.6001,1.7000,0.0000,0.8000,0.0000]
}

chain_pose_baselink = {
    "leg": [0.0596,-0.0000,1.0327,0.5000,0.5003,0.4997,0.5000],
    "head": [0.0599,0.0002,1.4098,-0.7072,0.0037,0.0037,0.7069],
    "left_arm": [0.1267,0.2342,0.7356,0.0220,0.0127,0.0343,0.9991],
    "right_arm": [0.1267,-0.2345,0.7358,-0.0225,0.0126,-0.0343,0.9991]
}

whole_body_joint = [
    num for key in ["leg", "head", "left_arm", "right_arm"]
    for num in chain_joints[key]
]

base_state = [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 1.0]
custom_param = gm.Parameter()
reference_frame = "base_link"
target_frame = "EndEffector"
target_chain = "left_arm"
one_chain = [target_chain]
chain_with_torso = [target_chain, "torso"]

```



```

error_chains = [target_chain, "torso", "head"]
# Scenario 1: Single-chain inverse kinematics
try:
    status, joint_map = motion.inverse_kinematics(
        target_pose=chain_pose_baselink[target_chain],
        chain_names=one_chain,
        target_frame=target_frame,
        reference_frame=reference_frame,
        enable_collision_check=False # Disable collision checking for accelerated testing
    )
    printStatus(status)
    assert status == gm.MotionStatus.SUCCESS, "Inverse kinematics calculation failed"
    print(f"✔ Basic version forward kinematics successful: joint angles={joint_map}")
    time.sleep(0.8)
except Exception as e:
    print(f"✘ Basic version forward kinematics exception: {e}")

# Scenario 2: Arm chain + torso inverse kinematics
try:
    status, joint_map = motion.inverse_kinematics(
        target_pose=chain_pose_baselink[target_chain],
        chain_names=chain_with_torso,
        target_frame=target_frame,
        reference_frame=reference_frame,
        enable_collision_check=False # Disable collision checking for accelerated testing
    )
    printStatus(status)
    assert status == gm.MotionStatus.SUCCESS, "Inverse kinematics calculation failed"
    print(f"✔ Custom initial joint forward kinematics successful: joint
angles={joint_map}")
    time.sleep(0.8)
except Exception as e:
    print(f"✘ Custom initial joint forward kinematics exception: {e}")

# Scenario 3: invalid chain combination
try:
    status, joint_map = motion.inverse_kinematics(
        target_pose=chain_pose_baselink[target_chain],
        chain_names=error_chains,
        target_frame=target_frame,
        reference_frame=reference_frame,
        enable_collision_check=False # Disable collision checking for accelerated testing
    )
    printStatus(status)
    assert status == gm.MotionStatus.INVALID_INPUT, "Inverse kinematics calculation
failed"
    print(f"✔ Invalid chain-combination input check passed")
    time.sleep(0.8)
except Exception as e:
    print(f"✘ Custom initial joint forward kinematics exception: {e}")

# Scenario 4: Use reference joints
try:
    # initial_joint_positions can specify chain joints as IK reference, unspecified chain
    joints use whole-body joints
    status, joint_map = motion.inverse_kinematics(
        target_pose=chain_pose_baselink[target_chain],
        chain_names=one_chain,
        target_frame=target_frame,
        reference_frame=reference_frame,

```

```

        initial_joint_positions=chain_joints,
        enable_collision_check=False # Disable collision checking for accelerated testing
    )
    printStatus(status)
    assert status == gm.MotionStatus.SUCCESS, "Inverse kinematics calculation failed"
    print(f"✔ Custom initial joint forward kinematics successful: joint
angles={joint_map}")
    time.sleep(0.8)
except Exception as e:
    print(f"✗ Custom initial joint forward kinematics exception: {e}")

# Scenario 5: Use RobotStates
try:
    ref_robot_state = gm.RobotStates()
    ref_robot_state.chain_name = target_chain
    ref_robot_state.whole_body_joint = whole_body_joint
    ref_robot_state.base_state = base_state
    target_frame = "EndEffector"
    reference_frame = "base_link"
    status, joint_map = motion.inverse_kinematics_by_state(
        target_pose=chain_pose_baselink[target_chain],
        chain_names=one_chain,
        target_frame=target_frame,
        reference_frame=reference_frame,
        reference_robot_states=ref_robot_state
    )
    printStatus(status)
    assert status == gm.MotionStatus.SUCCESS, "Inverse kinematics calculation failed"
    print(f"✔ Based on RobotStates forward kinematics successful: joint
angles={joint_map}")
except Exception as e:
    print(f"✗ Based on RobotStatesforward kinematics exception: {e}")

robot.request_shutdown()
robot.wait_for_shutdown()
robot.destroy()

```

逆运动学（基础与基于 RobotStates）

适用场景：根据期望的末端位姿求解所需的关节角度。是机械臂抓取等操作的核心步骤。

```

examples/python/galbot_motion/ik.py
import time
import galbot_sdk.g1 as gm
from galbot_sdk.g1 import GalbotMotion, GalbotRobot

# Get and initialize the GalbotMotion singleton
motion = GalbotMotion()
robot = GalbotRobot()

def printStatus(status):
    if(status == gm.MotionStatus.SUCCESS):
        print("Execution result: SUCCESS, execution successful")
    elif(status == gm.MotionStatus.TIMEOUT):
        print("Execution result: TIMEOUT, execution timed out")
    elif(status == gm.MotionStatus.FAULT):
        print("Execution result: FAULT, a fault occurred and execution cannot
continue")
    elif(status == gm.MotionStatus.INVALID_INPUT):

```

```

        print("Execution result: INVALID_INPUT, input parameters do not meet
requirements")
    elif(status == gm.MotionStatus.INIT_FAILED):
        print("Execution result: INIT_FAILED, failed to create internal communication
components")
    elif(status == gm.MotionStatus.IN_PROGRESS):
        print("Execution result: IN_PROGRESS, in motion but not yet in position")
    elif(status == gm.MotionStatus.STOPPED_UNREACHED):
        print("Execution result: STOPPED_UNREACHED, stopped but target not reached")
    elif(status == gm.MotionStatus.DATA_FETCH_FAILED):
        print("Execution result: DATA_FETCH_FAILED, failed to fetch data")
    elif(status == gm.MotionStatus.PUBLISH_FAIL):
        print("Execution result: PUBLISH_FAIL, data transmission failed")
    elif(status == gm.MotionStatus.COMM_DISCONNECTED):
        print("Execution result: COMM_DISCONNECTED, connection failed")

if motion.init():
    print("GalbotMotion initialized successfully")
else:
    print("GalbotMotion initialization failed")
if robot.init():
    print("GalbotRobot initialized successfully")
else:
    print("GalbotRobot initialization failed")

# Program started, waiting for data
time.sleep(1)

chain_joints = {
    "leg": [0.4992,1.4991,1.0005,0.0000,-0.0004],
    "head": [0.0000,0.0],
    "left_arm": [1.9999,-1.6000,-0.5999,-1.6999,0.0000,-0.7999,0.0000],
    "right_arm": [-2.0000,1.6001,0.6001,1.7000,0.0000,0.8000,0.0000]
}
chain_pose_baselink = {
    "leg": [0.0596,-0.0000,1.0327,0.5000,0.5003,0.4997,0.5000],
    "head": [0.0599,0.0002,1.4098,-0.7072,0.0037,0.0037,0.7069],
    "left_arm": [0.1267,0.2342,0.7356,0.0220,0.0127,0.0343,0.9991],
    "right_arm": [0.1267,-0.2345,0.7358,-0.0225,0.0126,-0.0343,0.9991]
}
whole_body_joint = [
    num for key in ["leg", "head", "left_arm", "right_arm"]
    for num in chain_joints[key]
]
base_state = [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 1.0]
custom_param = gm.Parameter()
reference_frame = "base_link"
target_frame = "EndEffector"
target_chain = "left_arm"
one_chain = [target_chain]
chain_with_torso = [target_chain, "torso"]
error_chains = [target_chain, "torso", "head"]
# Scenario 1: Single-chain inverse kinematics
try:
    status, joint_map = motion.inverse_kinematics(
        target_pose=chain_pose_baselink[target_chain],
        chain_names=one_chain,
        target_frame=target_frame,
        reference_frame=reference_frame,
        enable_collision_check=False # Disable collision checking for accelerated testing

```

```

    )
    printStatus(status)
    assert status == gm.MotionStatus.SUCCESS, "Inverse kinematics calculation failed"
    print(f"✔ Basic IK succeeded: joint angles={joint_map}")
    time.sleep(0.8)
except Exception as e:
    print(f"✗ Basic IK exception: {e}")

# Scenario 2: Arm chain + torso inverse kinematics
try:
    status, joint_map = motion.inverse_kinematics(
        target_pose=chain_pose_baselink[target_chain],
        chain_names=chain_with_torso,
        target_frame=target_frame,
        reference_frame=reference_frame,
        enable_collision_check=False # Disable collision checking for accelerated testing
    )
    printStatus(status)
    assert status == gm.MotionStatus.SUCCESS, "Inverse kinematics calculation failed"
    print(f"✔ IK with custom initial joints succeeded: joint angles={joint_map}")
    time.sleep(0.8)
except Exception as e:
    print(f"✗ IK with custom initial joints exception: {e}")

# Scenario 3: invalid chain combination
try:
    status, joint_map = motion.inverse_kinematics(
        target_pose=chain_pose_baselink[target_chain],
        chain_names=error_chains,
        target_frame=target_frame,
        reference_frame=reference_frame,
        enable_collision_check=False # Disable collision checking for accelerated testing
    )
    printStatus(status)
    assert status == gm.MotionStatus.INVALID_INPUT, "Inverse kinematics calculation
failed"
    print(f"✔ Invalid chain-combination input check passed")
    time.sleep(0.8)
except Exception as e:
    print(f"✗ IK with custom initial joints exception: {e}")

# Scenario 4: Use reference joints
try:
    # initial_joint_positions can specify chain joints as IK reference; unspecified chain
    joints are filled from whole-body joints
    status, joint_map = motion.inverse_kinematics(
        target_pose=chain_pose_baselink[target_chain],
        chain_names=one_chain,
        target_frame=target_frame,
        reference_frame=reference_frame,
        initial_joint_positions=chain_joints,
        enable_collision_check=False # Disable collision checking for accelerated testing
    )
    printStatus(status)
    assert status == gm.MotionStatus.SUCCESS, "Inverse kinematics calculation failed"
    print(f"✔ IK with custom initial joints succeeded: joint angles={joint_map}")
    time.sleep(0.8)
except Exception as e:
    print(f"✗ IK with custom initial joints exception: {e}")

```

```

# Scenario 5: Use RobotStates
try:
    ref_robot_state = gm.RobotStates()
    ref_robot_state.chain_name = target_chain
    ref_robot_state.whole_body_joint = whole_body_joint
    ref_robot_state.base_state = base_state
    target_frame = "EndEffector"
    reference_frame = "base_link"
    status, joint_map = motion.inverse_kinematics_by_state(
        target_pose=chain_pose_baselink[target_chain],
        chain_names=one_chain,
        target_frame=target_frame,
        reference_frame=reference_frame,
        reference_robot_states=ref_robot_state
    )
    printStatus(status)
    assert status == gm.MotionStatus.SUCCESS, "Inverse kinematics calculation failed"
    print(f"✔ RobotStates-based IK succeeded: joint angles={joint_map}")
except Exception as e:
    print(f"✘ RobotStates-based IK exception: {e}")

robot.request_shutdown()
robot.wait_for_shutdown()
robot.destroy()

```

获取与设置末端位姿

适用场景：直接获取当前末端位姿，或将末端移动到指定位姿。集成了逆运动学计算和执行。

```

examples/python/galbot_motion/get_set_end_effector_pose.py
import time
import galbot_sdk.g1 as gm
from galbot_sdk.g1 import GalbotMotion, GalbotRobot

# Get and initialize the GalbotMotion singleton
motion = GalbotMotion()
robot = GalbotRobot()

def printStatus(status):
    if(status == gm.MotionStatus.SUCCESS):
        print("Execution result: SUCCESS, execution successful")
    elif(status == gm.MotionStatus.TIMEOUT):
        print("Execution result: TIMEOUT, execution timed out")
    elif(status == gm.MotionStatus.FAULT):
        print("Execution result: FAULT, a fault occurred and execution cannot
continue")
    elif(status == gm.MotionStatus.INVALID_INPUT):
        print("Execution result: INVALID_INPUT, input parameters do not meet
requirements")
    elif(status == gm.MotionStatus.INIT_FAILED):
        print("Execution result: INIT_FAILED, failed to create internal communication
components")
    elif(status == gm.MotionStatus.IN_PROGRESS):
        print("Execution result: IN_PROGRESS, in motion but not yet in position")
    elif(status == gm.MotionStatus.STOPPED_UNREACHED):
        print("Execution result: STOPPED_UNREACHED, stopped but target not reached")
    elif(status == gm.MotionStatus.DATA_FETCH_FAILED):
        print("Execution result: DATA_FETCH_FAILED, failed to fetch data")
    elif(status == gm.MotionStatus.PUBLISH_FAIL):

```

```

        print("Execution result: PUBLISH_FAIL, data transmission failed")
    elif(status == gm.MotionStatus.COMM_DISCONNECTED):
        print("Execution result: COMM_DISCONNECTED, connection failed")

if motion.init():
    print("GalbotMotion initialized successfully")
else:
    print("GalbotMotion initialization failed")
if robot.init():
    print("GalbotRobot initialized successfully")
else:
    print("GalbotRobot initialization failed")

# Program started, waiting for data
time.sleep(1)

chain_pose_baselink = {
    "leg": [0.0596,-0.0000,1.0327,0.5000,0.5003,0.4997,0.5000],
    "head": [0.0599,0.0002,1.4098,-0.7072,0.0037,0.0037,0.7069],
    "left_arm": [0.1267,0.2342,0.7356,0.0220,0.0127,0.0343,0.9991],
    "right_arm": [0.1267,-0.2345,0.7358,-0.0225,0.0126,-0.0343,0.9991]
}

custom_param = gm.Parameter()
target_frame = "EndEffector"
reference_frame = "base_link"
target_chain = "left_arm"
# Scenario 1: Basic version
try:
    end_ee_link = "left_arm_end_effector_mount_link"
    status, pose = motion.get_end_effector_pose(
        end_effector_frame=end_ee_link,
        reference_frame=reference_frame
    )
    printStatus(status)
    assert status == gm.MotionStatus.SUCCESS, "Failed to get end-effector pose"
    print(f"✔ Basic end-effector pose retrieval succeeded: {pose}")
    time.sleep(0.8)
except Exception as e:
    print(f"✘ Basic end-effector pose retrieval exception: {e}")

# Scenario 2: Specify chain name + custom frame
try:
    status, pose = motion.get_end_effector_pose_on_chain(
        chain_name=target_chain,
        frame_id=target_frame,
        reference_frame=reference_frame
    )
    printStatus(status)
    assert status == gm.MotionStatus.SUCCESS, "Failed to get end-effector pose"
    print(f"✔ End-effector pose retrieval by specified chain succeeded: {pose}")
    time.sleep(0.8)
except Exception as e:
    print(f"✘ End-effector pose retrieval by specified chain exception: {e}")

end_effector_frame="left_arm"
reference_frame = "base_link"
try:
    status = motion.set_end_effector_pose(
        target_pose=chain_pose_baselink[end_effector_frame],
        end_effector_frame=end_effector_frame,

```

```

        reference_frame=reference_frame,
        enable_collision_check=False,
        is_blocking=False,
        timeout=5.0,
        params=custom_param
    )
    printStatus(status)
    assert status == gm.MotionStatus.SUCCESS, "Set end-effector pose failed"
    print(f"✔ End-effector pose set succeeded: status={status}")
except Exception as e:
    print(f"✘ End-effector pose setting exception: {e}")

robot.request_shutdown()
robot.wait_for_shutdown()
robot.destroy()

```

单点运动规划（关节空间与笛卡尔空间）

适用场景：规划从当前位姿到单个目标位姿的无碰撞运动轨迹。适用于单点目标到达任务。

```

examples/python/galbot_motion/motion_plan.py
import time

import galbot_sdk.g1 as gm
from galbot_sdk.g1 import GalbotMotion, GalbotRobot

# NOTE:
# - GalbotMotion currently does NOT provide real-time obstacle perception / automatic
  environment updates.
# - Motion collision checking uses self-collision + a collision world built from objects
  you load manually via
#   add_obstacle()/attach_target_object() (including point clouds if you load them
  explicitly).

motion = GalbotMotion()
robot = GalbotRobot()

def printStatus(status):
    if(status == gm.MotionStatus.SUCCESS):
        print("Result: SUCCESS")
    elif(status == gm.MotionStatus.TIMEOUT):
        print("Result: TIMEOUT")
    elif(status == gm.MotionStatus.FAULT):
        print("Result: FAULT")
    elif(status == gm.MotionStatus.INVALID_INPUT):
        print("Result: INVALID_INPUT")
    elif(status == gm.MotionStatus.INIT_FAILED):
        print("Result: INIT_FAILED")
    elif(status == gm.MotionStatus.IN_PROGRESS):
        print("Result: IN_PROGRESS")
    elif(status == gm.MotionStatus.STOPPED_UNREACHED):
        print("Result: STOPPED_UNREACHED")
    elif(status == gm.MotionStatus.DATA_FETCH_FAILED):
        print("Result: DATA_FETCH_FAILED")
    elif(status == gm.MotionStatus.PUBLISH_FAIL):
        print("Result: PUBLISH_FAIL")
    elif(status == gm.MotionStatus.COMM_DISCONNECTED):
        print("Result: COMM_DISCONNECTED")

```

```

if motion.init():
    print("GalbotMotion init OK")
else:
    print("GalbotMotion init FAILED")
if robot.init():
    print("GalbotRobot init OK")
else:
    print("GalbotRobot init FAILED")

# Wait for data to be ready.
time.sleep(1)

chain_joints = {
    "leg": [0.4992, 1.4991, 1.0005, 0.0000, -0.0004],
    "head": [0.0000, 0.0],
    "left_arm": [1.9999, -1.6000, -0.5999, -1.6999, 0.0000, -0.7999, 0.0000],
    "right_arm": [-2.0000, 1.6001, 0.6001, 1.7000, 0.0000, 0.8000, 0.0000]
}
chain_pose_baselink = {
    "leg": [0.0596, -0.0000, 1.0327, 0.5000, 0.5003, 0.4997, 0.5000],
    "head": [0.0599, 0.0002, 1.4098, -0.7072, 0.0037, 0.0037, 0.7069],
    "left_arm": [0.1267, 0.2342, 0.7356, 0.0220, 0.0127, 0.0343, 0.9991],
    "right_arm": [0.1267, -0.2345, 0.7358, -0.0225, 0.0126, -0.0343, 0.9991]
}
whole_body_joint = [
    num for key in ["leg", "head", "left_arm", "right_arm"]
    for num in chain_joints[key]
]
base_state = [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 1.0]
custom_param = gm.Parameter()

# Scenario 1: joint-space planning, target type = joint state
try:
    # Construct target joint state

    target_joint = gm.JointStates()
    target_joint.chain_name = "left_arm"
    target_joint.joint_positions = chain_joints[target_joint.chain_name]

    status, traj = motion.motion_plan(
        target=target_joint,
        # When enable_collision_check=True, collision is checked against Motion-side
        # explicitly loaded obstacles
        # (add_obstacle/attach_target_object) and self-collision.
        enable_collision_check=False,
        params=custom_param
    )
    printStatus(status)
    assert status == gm.MotionStatus.SUCCESS, "Planning failed"
    if traj != {}:
        print(f"✓ Joint-space planning + joint-target single-chain single-point planning
succeeded: trajectory points={len(traj[target_joint.chain_name])}")
        time.sleep(0.8)
    else:
        print(f" Return status is SUCCESS, but trajectory is empty; possibly already
reached, check whether the target matches current state or is within tolerance")

except Exception as e:
    print(f"ERROR: joint-space single-point planning exception: {e}")

```



```

# Scenario 2: joint-space planning, target type = end-effector pose (Cartesian)
try:
    # Construct target pose state
    target_pose_state = gm.PoseState()
    target_pose_state.chain_name = "left_arm"
    target_pose_state.frame_id = "EndEffector"
    target_pose_state.reference_frame = "base_link"
    target_pose_state.pose = gm.Pose(chain_pose_baselink[target_pose_state.chain_name])
    # target_pose_state.pose.position.x += 0.2

    status, traj = motion.motion_plan(
        target=target_pose_state,
        enable_collision_check=False
    )
    printStatus(status)
    assert status == gm.MotionStatus.SUCCESS, "Planning failed"
    if traj != {}:
        print(f"✓ Joint-space planning + end-effector-pose-target single-chain single-
point planning succeeded: trajectory length={len(traj[target_pose_state.chain_name])}")
        time.sleep(0.8)
    else:
        print(f" Return status is SUCCESS, but trajectory is empty; possibly already
reached, check whether the target matches current state or is within tolerance")

except Exception as e:
    print(f"ERROR: Cartesian single-point planning exception: {e}")

# Scenario 3: joint-space planning with an explicit start state
try:
    # Construct target joint state

    target_joint = gm.JointStates()
    target_joint.chain_name = "left_arm"
    target_joint.joint_positions = chain_joints[target_joint.chain_name]

    start_joint = gm.JointStates()
    start_joint.chain_name = "left_arm"
    start_joint.joint_positions = [0] * 7

    status, traj = motion.motion_plan(
        target=target_joint,
        start=start_joint,
        enable_collision_check=False,
        params=custom_param
    )
    printStatus(status)
    assert status == gm.MotionStatus.SUCCESS, "Planning failed"
    if traj != {}:
        print(f"✓ Joint-space planning + joint-target single-chain single-point planning
succeeded: trajectory points={len(traj[target_joint.chain_name])}")
    else:
        print(f" Return status is SUCCESS, but trajectory is empty; possibly already
reached, check whether the target matches current state or is within tolerance")

except Exception as e:
    print(f"ERROR: joint-space single-point planning exception: {e}")

robot.request_shutdown()
robot.wait_for_shutdown()
robot.destroy()

```

多点轨迹规划

适用场景：规划经过多个路径点的连续运动轨迹，适用于需要经过多个中间点的复杂运动。

```
examples/python/galbot_motion/motion_plan_multi_waypoints.py

import time
import galbot_sdk.g1 as gm
from galbot_sdk.g1 import GalbotMotion, GalbotRobot

# Get and initialize the GalbotMotion singleton
motion = GalbotMotion()
robot = GalbotRobot()

def printStatus(status):
    if(status == gm.MotionStatus.SUCCESS):
        print("Execution result: SUCCESS, execution successful")
    elif(status == gm.MotionStatus.TIMEOUT):
        print("Execution result: TIMEOUT, execution timed out")
    elif(status == gm.MotionStatus.FAULT):
        print("Execution result: FAULT, a fault occurred and execution cannot
continue")
    elif(status == gm.MotionStatus.INVALID_INPUT):
        print("Execution result: INVALID_INPUT, input parameters do not meet
requirements")
    elif(status == gm.MotionStatus.INIT_FAILED):
        print("Execution result: INIT_FAILED, failed to create internal communication
components")
    elif(status == gm.MotionStatus.IN_PROGRESS):
        print("Execution result: IN_PROGRESS, in motion but not yet in position")
    elif(status == gm.MotionStatus.STOPPED_UNREACHED):
        print("Execution result: STOPPED_UNREACHED, stopped but target not reached")
    elif(status == gm.MotionStatus.DATA_FETCH_FAILED):
        print("Execution result: DATA_FETCH_FAILED, failed to fetch data")
    elif(status == gm.MotionStatus.PUBLISH_FAIL):
        print("Execution result: PUBLISH_FAIL, data transmission failed")
    elif(status == gm.MotionStatus.COMM_DISCONNECTED):
        print("Execution result: COMM_DISCONNECTED, connection failed")

if motion.init():
    print("GalbotMotion initialized successfully")
else:
    print("GalbotMotion initialization failed")
if robot.init():
    print("GalbotRobot initialized successfully")
else:
    print("GalbotRobot initialization failed")

# Program started, waiting for data
time.sleep(2)

chain_joints = {
    "leg": [0.4992,1.4991,1.0005,0.0000,-0.0004],
    "head": [0.0000,0.0],
    "left_arm": [1.9999,-1.6000,-0.5999,-1.6999,0.0000,-0.7999,0.0000],
    "right_arm": [-2.0000,1.6001,0.6001,1.7000,0.0000,0.8000,0.0000]
}

chain_pose_baselink = {
    "leg": [0.0596,-0.0000,1.0327,0.5000,0.5003,0.4997,0.5000],
    "head": [0.0599,0.0002,1.4098,-0.7072,0.0037,0.0037,0.7069],
```

```

    "left_arm": [0.1267,0.2342,0.7356,0.0220,0.0127,0.0343,0.9991],
    "right_arm": [0.1267,-0.2345,0.7358,-0.0225,0.0126,-0.0343,0.9991]
}
whole_body_joint = [
    num for key in ["leg", "head", "left_arm", "right_arm"]
    for num in chain_joints[key]
]
base_state = [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 1.0]
custom_param = gm.Parameter()

# Scenario 1: Multi-waypoint planning in Cartesian space (PoseState target)
try:
    # Construct target pose
    target_pose_state = gm.PoseState()
    target_pose_state.chain_name = "left_arm"

    # Construct waypoints (3 intermediate poses)
    waypoint_poses = [
        [0.1267,0.2342,0.7356,0.0220,0.0127,0.0343,0.9991],
        [0.2267,0.2342,0.7356,0.0220,0.0127,0.0343,0.9991],
        [0.3267,0.2342,0.7356,0.0220,0.0127,0.0343,0.9991],
        [0.4267,0.2342,0.7356,0.0220,0.0127,0.0343,0.9991],
    ]

    status, traj = motion.motion_plan_multi_waypoints(
        target=target_pose_state,
        waypoint_poses=waypoint_poses,
        enable_collision_check=False,
        params=custom_param
    )
    printStatus(status)
    assert status == gm.MotionStatus.SUCCESS, "Cartesian multi-waypoint single-chain
planning failed"
    if traj != {}:
        print(f"✓ Cartesian waypoint single-chain planning succeeded: trajectory
points={len(traj[target_pose_state.chain_name])}")
        time.sleep(0.8)
    else:
        print(f" Return status is SUCCESS, but trajectory is empty; possibly already
reached, check whether the target matches current state or is within tolerance")
except Exception as e:
    print(f"✗ Cartesian multi-point motion planning exception: {e}")

# Scenario 2: Multi-waypoint planning in joint space (JointStates target)
try:
    # Construct target pose
    target_joint = gm.JointStates()
    target_joint.chain_name = "left_arm"

    # Construct waypoints (3 intermediate poses)
    waypoints = [
        [0.1267,0.2342,0.7356,0.0220,0.0127,0.0343,0.9991],
        [0.2267,0.4342,0.7356,0.0220,0.0127,0.0343,0.9991],
        [0.3267,0.6342,0.7356,0.0220,0.0127,0.0343,0.9991],
        [0.4267,0.8342,0.7356,0.0220,0.0127,0.0343,0.9991]
    ]

    status, traj = motion.motion_plan_multi_waypoints(
        target=target_joint,
        waypoint_poses=waypoints,

```

```

        enable_collision_check=False,
        params=custom_param
    )
    printStatus(status)
    assert status == gm.MotionStatus.SUCCESS, "Cartesian multi-waypoint single-chain
planning failed"
    if traj != {}:
        print(f"✓ Joint-waypoint single-chain planning succeeded: trajectory
points={len(traj[target_pose_state.chain_name])}")
    else:
        print(f" Return status is SUCCESS, but trajectory is empty; possibly already
reached, check whether the target matches current state or is within tolerance")
except Exception as e:
    print(f"✗ Joint-space multi-point motion planning exception: {e}")

robot.request_shutdown()
robot.wait_for_shutdown()
robot.destroy()

```

碰撞检测

适用场景：检查给定关节状态是否会发生自碰撞或与环境障碍物碰撞。用于运动规划前的可行性检查。

```

examples/python/galbot_motion/check_collision.py
import galbot_sdk.g1 as gm
from galbot_sdk.g1 import GalbotMotion, GalbotRobot, GalbotNavigation
import time

# NOTE:
# - GalbotMotion currently does NOT provide real-time obstacle perception / automatic
environment updates.
# - Collision checking uses self-collision + the collision world built from objects you
load manually via
#   add_obstacle()/attach_target_object() (including point clouds if you load them
explicitly).

motion = GalbotMotion()
robot = GalbotRobot()
nav = GalbotNavigation()

def printStatus(status):
    if(status == gm.MotionStatus.SUCCESS):
        print("Result: SUCCESS")
    elif(status == gm.MotionStatus.TIMEOUT):
        print("Result: TIMEOUT")
    elif(status == gm.MotionStatus.FAULT):
        print("Result: FAULT")
    elif(status == gm.MotionStatus.INVALID_INPUT):
        print("Result: INVALID_INPUT")
    elif(status == gm.MotionStatus.INIT_FAILED):
        print("Result: INIT_FAILED")
    elif(status == gm.MotionStatus.IN_PROGRESS):
        print("Result: IN_PROGRESS")
    elif(status == gm.MotionStatus.STOPPED_UNREACHED):
        print("Result: STOPPED_UNREACHED")
    elif(status == gm.MotionStatus.DATA_FETCH_FAILED):
        print("Result: DATA_FETCH_FAILED")
    elif(status == gm.MotionStatus.PUBLISH_FAIL):

```

```

        print("Result: PUBLISH_FAIL")
    elif(status == gm.MotionStatus.COMM_DISCONNECTED):
        print("Result: COMM_DISCONNECTED")

if motion.init():
    print("GalbotMotion init OK")
else:
    print("GalbotMotion init FAILED")
if robot.init():
    print("GalbotRobot init OK")
else:
    print("GalbotRobot init FAILED")
if nav.init():
    print("GalbotNavigation init OK")
else:
    print("GalbotNavigation init FAILED")

# Wait for data to be ready.
time.sleep(3)

chain_joints = {
    "leg": [0.4992,1.4991,1.0005,0.0000,-0.0004],
    "head": [0.0000,0.0],
    "left_arm": [1.9999,-1.6000,-0.5999,-1.6999,0.0000,-0.7999,0.0000],
    "right_arm": [-2.0000,1.6001,0.6001,1.7000,0.0000,0.8000,0.0000]
}

whole_body_joint = [
    num for key in ["leg", "head", "left_arm", "right_arm"]
    for num in chain_joints[key]
]
base_state = [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 1.0]
custom_param = gm.Parameter()

try:
    # Build RobotStates list to check.
    check_states = [gm.RobotStates() for _ in range(2)]
    check_states[0].whole_body_joint = whole_body_joint
    check_states[0].base_state = base_state

    bad_left_arm_joint = [1.99995,-1.60004,0.599905,-1.69994,0,-0.799924,0]

    check_left_arm = gm.JointStates()
    check_left_arm.chain_name = "left_arm"
    check_left_arm.joint_positions = bad_left_arm_joint
    check_states[1] = check_left_arm

    status, collision_res = motion.check_collision(
        start=check_states,
        enable_collision_check=True
    )
    printStatus(status)
    assert status == gm.MotionStatus.SUCCESS, "Collision check failed"
    assert len(collision_res) == len(check_states), "Result size mismatch"
    print(f"OK: collision check finished: {collision_res} (False=no collision)")
except Exception as e:
    print(f"ERROR: collision check exception: {e}")

robot.request_shutdown()

```

```
robot.wait_for_shutdown()
robot.destroy()
```

附加工具

适用场景：在机器人末端添加工具（如夹具、吸盘等），更新运动规划模型，以便考虑工具质量和碰撞体积。

```
examples/python/galbot_motion/attach_tool.py

import time
import galbot_sdk.g1 as gm
from galbot_sdk.g1 import GalbotMotion, GalbotRobot

# Get and initialize the GalbotMotion singleton
motion = GalbotMotion()
robot = GalbotRobot()

def printStatus(status):
    if(status == gm.MotionStatus.SUCCESS):
        print("Execution result: SUCCESS, execution successful")
    elif(status == gm.MotionStatus.TIMEOUT):
        print("Execution result: TIMEOUT, execution timed out")
    elif(status == gm.MotionStatus.FAULT):
        print("Execution result: FAULT, a fault occurred and execution cannot
continue")
    elif(status == gm.MotionStatus.INVALID_INPUT):
        print("Execution result: INVALID_INPUT, input parameters do not meet
requirements")
    elif(status == gm.MotionStatus.INIT_FAILED):
        print("Execution result: INIT_FAILED, failed to create internal communication
components")
    elif(status == gm.MotionStatus.IN_PROGRESS):
        print("Execution result: IN_PROGRESS, in motion but not yet in position")
    elif(status == gm.MotionStatus.STOPPED_UNREACHED):
        print("Execution result: STOPPED_UNREACHED, stopped but target not reached")
    elif(status == gm.MotionStatus.DATA_FETCH_FAILED):
        print("Execution result: DATA_FETCH_FAILED, failed to fetch data")
    elif(status == gm.MotionStatus.PUBLISH_FAIL):
        print("Execution result: PUBLISH_FAIL, data transmission failed")
    elif(status == gm.MotionStatus.COMM_DISCONNECTED):
        print("Execution result: COMM_DISCONNECTED, connection failed")

if motion.init():
    print("GalbotMotion initialized successfully")
else:
    print("GalbotMotion initialization failed")
if robot.init():
    print("GalbotRobot initialized successfully")
else:
    print("GalbotRobot initialization failed")

# Program started, waiting for data
time.sleep(2)

try:
    chain_name = "left_arm"
    tool_name = "suction_cup"
    status = motion.attach_tool(
        chain=chain_name,
```

```

        tool=tool_name
    )
    printStatus(status)
    assert status == gm.MotionStatus.SUCCESS, "load failed"
    print(f"✔ Tool attached successfully")
except Exception as e:
    print(f"✗ Tool attachment exception: {e}")

robot.request_shutdown()
robot.wait_for_shutdown()
robot.destroy()

```

卸载工具

适用场景：从机器人末端移除已附加的工具，恢复原始机器人模型。

```

examples/python/galbot_motion/detach_tool.py
import time
import galbot_sdk.g1 as gm
from galbot_sdk.g1 import GalbotMotion, GalbotRobot

# Get and initialize the GalbotMotion singleton
motion = GalbotMotion()
robot = GalbotRobot()

def printStatus(status):
    if(status == gm.MotionStatus.SUCCESS):
        print("Execution result: SUCCESS, execution successful")
    elif(status == gm.MotionStatus.TIMEOUT):
        print("Execution result: TIMEOUT, execution timed out")
    elif(status == gm.MotionStatus.FAULT):
        print("Execution result: FAULT, a fault occurred and execution cannot
continue")
    elif(status == gm.MotionStatus.INVALID_INPUT):
        print("Execution result: INVALID_INPUT, input parameters do not meet
requirements")
    elif(status == gm.MotionStatus.INIT_FAILED):
        print("Execution result: INIT_FAILED, failed to create internal communication
components")
    elif(status == gm.MotionStatus.IN_PROGRESS):
        print("Execution result: IN_PROGRESS, in motion but not yet in position")
    elif(status == gm.MotionStatus.STOPPED_UNREACHED):
        print("Execution result: STOPPED_UNREACHED, stopped but target not reached")
    elif(status == gm.MotionStatus.DATA_FETCH_FAILED):
        print("Execution result: DATA_FETCH_FAILED, failed to fetch data")
    elif(status == gm.MotionStatus.PUBLISH_FAIL):
        print("Execution result: PUBLISH_FAIL, data transmission failed")
    elif(status == gm.MotionStatus.COMM_DISCONNECTED):
        print("Execution result: COMM_DISCONNECTED, connection failed")

if motion.init():
    print("GalbotMotion initialized successfully")
else:
    print("GalbotMotion initialization failed")
if robot.init():
    print("GalbotRobot initialized successfully")
else:
    print("GalbotRobot initialization failed")

```

```

# Program started, waiting for data
time.sleep(2)

# 1. Detach tool
try:
    chain_name = "left_arm"
    status = motion.detach_tool(
        chain=chain_name
    )
    printStatus(status)
    assert status == gm.MotionStatus.SUCCESS, "detach failed"
    print(f"✔ Tool detached successfully")
except Exception as e:
    print(f"✗ Tool detachment exception: {e}")

robot.request_shutdown()
robot.wait_for_shutdown()
robot.destroy()

```

获取连杆名称列表

适用场景：获取机器人模型中所有连杆的名称列表，用于碰撞检测和运动规划调试。

```

examples/python/galbot_motion/get_link_names.py
import time
import galbot_sdk.g1 as gm
from galbot_sdk.g1 import GalbotMotion, GalbotRobot

# Get and initialize the GalbotMotion singleton
motion = GalbotMotion()
robot = GalbotRobot()

if motion.init():
    print("GalbotMotion initialized successfully")
else:
    print("GalbotMotion initialization failed")
if robot.init():
    print("GalbotRobot initialized successfully")
else:
    print("GalbotRobot initialization failed")

# Program started, waiting for data
time.sleep(2)

try:
    # Get all link names
    all_link_names = motion.get_link_names(only_end_effector=False)
    print(f"\nAll link names (total {len(all_link_names)}):")
    for i, link_name in enumerate(all_link_names, 1):
        print(f"  {i}. {link_name}")

    # getend effectorexecute link
    ee_link_names = motion.get_link_names(only_end_effector=True)
    print(f"\nEnd-effector link names (total {len(ee_link_names)}):")
    for i, link_name in enumerate(ee_link_names, 1):
        print(f"  {i}. {link_name}")

    # example: link kinematics
    if ee_link_names:

```



```

        print(f"\nRun forward kinematics using end-effector link '{ee_link_names[0]}'...")
        success, fk_result = motion.forward_kinematics(ee_link_names[0])
        if success == gm.MotionStatus.SUCCESS:
            print(f"Forward-kinematics result: {fk_result}")
        else:
            print(f"Forward-kinematics computation failed: {success}")
except Exception as e:
    print(f"✗ Link-name retrieval exception: {e}")

robot.request_shutdown()
robot.wait_for_shutdown()
robot.destroy()

```

加载/移除环境碰撞体

适用场景：向运动规划环境中添加障碍物或移除已添加的障碍物，使运动规划考虑环境障碍。

examples/python/galbot_motion/add_obstacle.py

```

import time

import galbot_sdk.g1 as gm
from galbot_sdk.g1 import GalbotMotion, GalbotRobot

# NOTE:
# - GalbotMotion currently does NOT provide real-time obstacle perception / automatic
  environment updates.
# - If you want Motion collision checking to consider obstacles (including point clouds),
  you must load them
#   manually via add_obstacle()/attach_target_object().

motion = GalbotMotion()
robot = GalbotRobot()

def printStatus(status):
    if(status == gm.MotionStatus.SUCCESS):
        print("Result: SUCCESS")
    elif(status == gm.MotionStatus.TIMEOUT):
        print("Result: TIMEOUT")
    elif(status == gm.MotionStatus.FAULT):
        print("Result: FAULT")
    elif(status == gm.MotionStatus.INVALID_INPUT):
        print("Result: INVALID_INPUT")
    elif(status == gm.MotionStatus.INIT_FAILED):
        print("Result: INIT_FAILED")
    elif(status == gm.MotionStatus.IN_PROGRESS):
        print("Result: IN_PROGRESS")
    elif(status == gm.MotionStatus.STOPPED_UNREACHED):
        print("Result: STOPPED_UNREACHED")
    elif(status == gm.MotionStatus.DATA_FETCH_FAILED):
        print("Result: DATA_FETCH_FAILED")
    elif(status == gm.MotionStatus.PUBLISH_FAIL):
        print("Result: PUBLISH_FAIL")
    elif(status == gm.MotionStatus.COMM_DISCONNECTED):
        print("Result: COMM_DISCONNECTED")

if motion.init():
    print("GalbotMotion init OK")
else:
    print("GalbotMotion init FAILED")

```

```

if robot.init():
    print("GalbotRobot init OK")
else:
    print("GalbotRobot init FAILED")

# Wait for data to be ready.
time.sleep(2)

# 1) Add a box collision object into Motion environment.
# This affects Motion-side collision checking (e.g., motion_plan/check_collision).
try:
    obstacle_id = "box_test_1"
    obj_type = "box"
    obj_pose = [1.0, 0.0, 1.0, 0,0,0,1]
    obj_size = [1.0, 1.0, 1.0]
    target_frame = "world"
    status = motion.add_obstacle(
        obstacle_id=obstacle_id,
        obstacle_type=obj_type,
        pose=obj_pose,
        scale=obj_size,
        target_frame=target_frame
    )
    printStatus(status)
    motion.clear_obstacle()
    assert status == gm.MotionStatus.SUCCESS, "Failed to add obstacle"
    print(f"OK: added obstacle: {obstacle_id}")
except Exception as e:
    print(f"ERROR: add obstacle exception: {e}")

# 2) Add a duplicate ID (expected to fail).
try:
    obstacle_id = "box_test_1"
    obj_type = "box"
    obj_pose = [1.0, 0.0, 1.0, 0,0,0,1]
    obj_size = [1.0, 1.0, 1.0]
    target_frame = "world"
    status = motion.add_obstacle(
        obstacle_id=obstacle_id,
        obstacle_type=obj_type,
        pose=obj_pose,
        scale=obj_size,
        target_frame=target_frame
    )
    status = motion.add_obstacle(
        obstacle_id=obstacle_id,
        obstacle_type=obj_type,
        pose=obj_pose,
        scale=obj_size,
        target_frame=target_frame
    )
    printStatus(status)
    motion.clear_obstacle()
    assert status == gm.MotionStatus.FAULT, "Expected duplicate obstacle ID to fail"
    print("OK: duplicate obstacle ID is rejected")
except Exception as e:
    print(f"ERROR: duplicate obstacle exception: {e}")

robot.request_shutdown()

```

```
robot.wait_for_shutdown()
robot.destroy()
```

类：GalbotNavigation

获取实例 / 初始化

适用场景：获取导航模块单例并初始化。必须在使用导航功能前调用。

```
examples/python/galbot_navigation/get_instance.py
from galbot_sdk.g1 import GalbotNavigation
from galbot_sdk.g1 import GalbotRobot
import numpy as np

# Initialize system and navigation module
robot = GalbotRobot()
robot.init()

nav = GalbotNavigation()
nav.init()

print("GalbotNavigation has been initialized:", nav is not None)

# send SIGINT shutdown signal
robot.request_shutdown()
# Wait until entering shutdown state
robot.wait_for_shutdown()
# Perform SDK resource release
robot.destroy()
print('Resources released successfully')
```

重定位 / 是否已定位 / 获取当前位姿

适用场景：触发重定位，检查机器人是否已经成功定位，并获取机器人在地图中的当前位姿。

```
examples/python/galbot_navigation/relocalized.py
from galbot_sdk.g1 import GalbotNavigation
from galbot_sdk.g1 import GalbotRobot
import numpy as np
import time

nav = GalbotNavigation()
nav.init()
robot = GalbotRobot()
robot.init()

init_pose = np.array([0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 1.0])

# success
while not nav.is_localized():
    nav.relocalize(init_pose)
    time.sleep(0.5)

print("Current pose:", nav.get_current_pose())

nav.stop_navigation()
# send SIGINT shutdown signal
```

```

robot.request_shutdown()
# Wait until entering shutdown state
robot.wait_for_shutdown()
# Perform SDK resource release
robot.destroy()
print('Resources released successfully')

```

检查路径可达性并阻塞导航到目标

适用场景：检查目标点是否可达，如果可达则阻塞等待导航完成到达目标。简单场景使用方便。

```

examples/python/galbot_navigation/check_path_reachability.py
from galbot_sdk.g1 import GalbotNavigation
from galbot_sdk.g1 import GalbotRobot
from galbot_sdk.g1 import ControlStatus, G1ControllerName
import numpy as np
import sys

nav = GalbotNavigation()
nav.init()
robot = GalbotRobot()
robot.init()

start = nav.get_current_pose()
goal = np.array([1.0, 1.0, 0.0, 0, 0, 0.4794255, 0.8775826])

res = robot.switch_controller(G1ControllerName.CHASSIS_POSE_CTRL)
if res != ControlStatus.SUCCESS:
    print("Failed to switch controller!")
    sys.exit(1)
else:
    print("Controller switched successfully!")

if nav.check_path_reachability(goal, start):
    status = nav.navigate_to_goal(
        goal, enable_collision_check=True, is_blocking=True, timeout=30
    )
    print("navigate_to_goal returned status:", status)
    print("Reached or not:", nav.check_goal_arrival())
else:
    print("Path unreachable or unsafe")

nav.stop_navigation()
# send SIGINT shutdown signal
robot.request_shutdown()
# Wait until entering shutdown state
robot.wait_for_shutdown()
# Perform SDK resource release
robot.destroy()
print("Resources released successfully")

```

非阻塞导航 + 轮询判断到达

适用场景：启动导航但不阻塞当前线程，可以在导航过程中做其他处理，需要轮询判断是否到达目标。适合需要异步处理的场景。

```

examples/python/galbot_navigation/navigate_to_goal.py

```

```

from galbot_sdk.g1 import GalbotNavigation
from galbot_sdk.g1 import GalbotRobot
from galbot_sdk.g1 import ControlStatus, G1ControllerName
import numpy as np
import time
import sys

nav = GalbotNavigation()
nav.init()
robot = GalbotRobot()
robot.init()

goal = np.array([0.5, 0.0, 0.0, 0, 0, 0.0, 1.0])

res = robot.switch_controller(G1ControllerName.CHASSIS_POSE_CTRL)
if res != ControlStatus.SUCCESS:
    print("Failed to switch controller!")
    sys.exit(1)
else:
    print("Controller switched successfully!")

nav.navigate_to_goal(goal, enable_collision_check=True, is_blocking=False, timeout=20)

start_time = time.time()
reached = False

while True:
    if nav.check_goal_arrival():
        reached = True
        break
    if time.time() - start_time > 20:
        print("Navigation timed out; target not reached within 20s")
        break
    print("Navigating...")
    time.sleep(0.5)

if reached:
    print("Target reached")

nav.stop_navigation()
# send SIGINT shutdown signal
robot.request_shutdown()
# Wait until entering shutdown state
robot.wait_for_shutdown()
# Perform SDK resource release
robot.destroy()
print("Resources released successfully")

```

直线移动到目标 / 停止导航

适用场景：让机器人沿直线移动到目标点，或停止当前正在进行的导航任务。

```

examples/python/galbot_navigation/move_straight_to.py
from galbot_sdk.g1 import GalbotNavigation
from galbot_sdk.g1 import GalbotRobot
from galbot_sdk.g1 import ControlStatus, G1ControllerName
import numpy as np
import time
import sys

```

```

nav = GalbotNavigation()
nav.init()
robot = GalbotRobot()
robot.init()

target = np.array([0.2, 0.0, 0.0, 0, 0, 0.0, 1.0])

res = robot.switch_controller(G1ControllerName.CHASSIS_POSE_CTRL)
if res != ControlStatus.SUCCESS:
    print("Failed to switch controller!")
    sys.exit(1)
else:
    print("Controller switched successfully!")

nav.move_straight_to(target, is_blocking=False, timeout=10)
time.sleep(1.0)
nav.stop_navigation()

# send SIGINT shutdown signal
robot.request_shutdown()
# Wait until entering shutdown state
robot.wait_for_shutdown()
# Perform SDK resource release
robot.destroy()
print("Resources released successfully")

```

获取导航状态 + 轮询 SUCCESS/FAILED 或超时

适用场景：获取当前导航任务的执行状态，用于非阻塞导航时轮询。

```

examples/python/galbot_navigation/get_navigation_status.py
"""
example: navigation get_navigation_status, SUCCESS/FAILED timeout exit,
Avoid deadlock and execute error logic.
"""

from galbot_sdk.g1 import GalbotNavigation
from galbot_sdk.g1 import GalbotRobot
from galbot_sdk.g1 import NavigationTaskStatus, ControlStatus, G1ControllerName
import numpy as np
import time
import sys

nav = GalbotNavigation()
nav.init()
robot = GalbotRobot()
robot.init()

goal = np.array([0.5, 0.0, 0.0, 0, 0, 0.0, 1.0])
timeout_s = 20.0
poll_interval_s = 0.5

res = robot.switch_controller(G1ControllerName.CHASSIS_POSE_CTRL)
if res != ControlStatus.SUCCESS:
    print("Failed to switch controller!")
    sys.exit(1)
else:
    print("Controller switched successfully!")

```

```

# Non-blocking navigation
nav.navigate_to_goal(
    goal, enable_collision_check=True, is_blocking=False, timeout=timeout_s
)
start = time.time()

while True:
    status = nav.get_navigation_status()
    elapsed = time.time() - start

    if status == NavigationTaskStatus.SUCCESS:
        print("Target reached")
        break
    if status == NavigationTaskStatus.FAILED:
        print("Navigation failed; exit error-handling logic promptly")
        break
    if elapsed >= timeout_s:
        print("navigationtimeout, exit")
        break

    if status == NavigationTaskStatus.RUNNING:
        print(f"Navigating... Status: {status.name}, elapsed: {elapsed:.1f}s")
    else:
        print(f"Status: {status.name}, elapsed: {elapsed:.1f}s")

    time.sleep(poll_interval_s)

nav.stop_navigation()
robot.request_shutdown()
robot.wait_for_shutdown()
robot.destroy()
print("Resources released successfully")

```

完整运行示例（简单流程）

适用场景：展示导航功能的完整使用流程，包括初始化、重定位、导航到目标等步骤，供参考。

```

examples/python/galbot_navigation/complete_example.py
from galbot_sdk.g1 import GalbotRobot, GalbotNavigation
from galbot_sdk.g1 import ControlStatus, G1ControllerName
import numpy as np
import time
import sys

robot = GalbotRobot()
robot.init()
nav = GalbotNavigation()
nav.init()

init_pose = np.array([0.0, 0.0, 0.0, 0, 0, 0.0, 1.0])
goal_pose = np.array([1.0, 0.0, 0.0, 0, 0, 0.0, 1.0])

res = robot.switch_controller(G1ControllerName.CHASSIS_POSE_CTRL)
if res != ControlStatus.SUCCESS:
    print("Failed to switch controller!")
    sys.exit(1)
else:
    print("Controller switched successfully!")

```

```

while not nav.is_localized():
    nav.relocalize(init_pose)
    time.sleep(0.5)

if nav.check_path_reachability(goal_pose, nav.get_current_pose()):
    nav.navigate_to_goal(
        goal_pose, enable_collision_check=True, is_blocking=True, timeout=30
    )
    print("Whether reached:", nav.check_goal_arrival())

nav.stop_navigation()
# Shutdown system
robot.request_shutdown()
robot.wait_for_shutdown()
robot.destroy()

```

类：GalbotPerception

高精度双目深度估计，单次推理并保存伪彩色深度图

适用场景：触发一次双目深度估计推理，并获取推理结果

```

examples/python/galbot_perception/foundation_stereo_example.py
"""Foundation stereo depth example (G1): single run_once, save a pseudo-color depth
image."""

import time

import cv2
import numpy as np

try:
    from galbot_sdk import (
        GalbotPerception,
        GalbotRobot,
        MachineType,
        PerceptionModule,
    )
except ImportError:
    print("Failed to import galbot_sdk, please install it first or check if it is in
PYTHONPATH")
    raise

OUTPUT_IMAGE_PATH = "foundation_stereo_depth.png"

def main():
    robot = GalbotRobot.get_instance(MachineType.G1)
    if not robot.init():
        print("Robot init failed")
        return
    print("Robot init OK")

    perception = GalbotPerception.get_instance(MachineType.G1)
    if not perception.init({PerceptionModule.FOUNDATION_STEREO,
PerceptionModule.LIGHT_STEREO}):
        print("Perception init failed")
        return

```



```

print("Perception init OK")

time.sleep(12) # Wait for perception models to load
print("Triggering single inference...")

if not perception.run_once(PerceptionModule.FOUNDATION_STEREO):
    print("run_once failed to send command")
    return

print("Waiting for inference result...")
if not perception.wait_for_new_result(
    PerceptionModule.FOUNDATION_STEREO, timeout_s=6.0
):
    print("Timed out waiting for inference result")
    return

ok, result = perception.get_latest_result(PerceptionModule.FOUNDATION_STEREO)
if not ok:
    print("get_latest_result failed")
    return

print(result.get_result_info())

depth_map = result.instance_mask
if depth_map is None:
    print("No depth map (instance_mask is empty)")
    return

print(f"Depth map shape: {depth_map.shape}, dtype: {depth_map.dtype}")
print(f"Depth value range: [{np.nanmin(depth_map)}, {np.nanmax(depth_map)}]")

depth_f = depth_map.astype(np.float32)
valid = depth_f[depth_f > 0]
if valid.size > 0:
    vmin, vmax = np.percentile(valid, [1, 99])
    normalized = np.clip((depth_f - vmin) / (vmax - vmin + 1e-6), 0, 1)
else:
    normalized = np.zeros_like(depth_f)
colored = cv2.applyColorMap(
    (normalized * 255).astype(np.uint8), cv2.COLORMAP_TURBO
)

if cv2.imwrite(OUTPUT_IMAGE_PATH, colored):
    print(f"Saved: {OUTPUT_IMAGE_PATH}")
else:
    print(f"Failed to save: {OUTPUT_IMAGE_PATH}")

if __name__ == "__main__":
    main()
    GalbotRobot.get_instance(MachineType.G1).request_shutdown()
    GalbotRobot.get_instance(MachineType.G1).wait_for_shutdown()
    GalbotRobot.get_instance(MachineType.G1).destroy()

```

类：Parameter

适用场景：创建运动规划参数对象，用于配置运动规划的各种参数如速度限制、加速度限制、规划时间等。

```
examples/python/galbot_motion/create_parameter.py
from galbot_sdk.g1 import Parameter, create_parameter, G1JointGroup

# Create Parameter via constructor and set options
p = Parameter()
p.set_blocking(True)
p.set_check_collision(False)
p.set_timeout(5.0)
p.set_actuate('with_chain_only')
p.set_tool_pose(False)
p.set_reference_frame('base_link')

p.joint_state = {
    G1JointGroup.left_arm: [0.0] * 7,
    # Can add others if needed:
    # G1JointGroup.right_arm: [0.0] * 7,
    # G1JointGroup.LEG: [0.0] * 4,
}

print('blocking:', p.get_blocking())
print('collision check:', p.get_check_collision())
print('timeout:', p.get_timeout())

# Or use factory function to quickly create Parameter
p2 = create_parameter(direct_execute=False, blocking=True, timeout=3.0,
    actuate='with_chain_only', tool_pose=False, check_collision=True)
print('Factory-created timeout:', p2.get_timeout())
```

辅助函数

check_motion_status

适用场景：检查运动规划执行结果状态，判断规划是否成功，打印错误信息。

```
examples/python/galbot_motion/check_motion_status.py
from galbot_sdk.g1 import MotionStatus, check_motion_status

status_str = check_motion_status(MotionStatus.SUCCESS)
print('MotionStatus string:', status_str)
```

create_joint_state / create_pose_state

适用场景：快速创建关节状态和位姿状态对象，方便运动规划接口调用。

```
examples/python/galbot_motion/create_pose_state.py
from galbot_sdk.g1 import create_joint_state, create_pose_state, JointStates, PoseState, Pose

# Create helper objects using the factory function
js = create_joint_state()
ps = create_pose_state()
```

```
# Fill example fields
js.chain_name = 'left_arm'
js.joint_positions = [0.0] * 7

ps.chain_name = 'left_arm'
# 7D vector: x, y, z, qx, qy, qz, qw
ps.pose = Pose([1.0, 0.0, 0.0, 0.0, 0.0, 0.0, 1.0])

print(type(js), js.chain_name)
print(type(ps), ps.chain_name)
```

15. C++ 示例

本文件为 API 中公开的函数与类型提供简短示例，演示如何使用这些接口。

机器人关节名称 (G1 2.2)

关节组列表

机器人关节组名称包括: ["head", "left_arm", "right_arm", "leg", "left_gripper", "right_gripper", "left_suction_cup", "right_suction_cup"]

各关节组详细信息

关节组名称	关节组英文名	关节数量	关节名称列表
头部	head	2	head_joint1, head_joint2
腿部	leg	5	leg_joint1, leg_joint2, leg_joint3, leg_joint4, leg_joint5
左臂	left_arm	7	left_arm_joint1, left_arm_joint2, left_arm_joint3, left_arm_joint4, left_arm_joint5, left_arm_joint6, left_arm_joint7
右臂	right_arm	7	right_arm_joint1, right_arm_joint2, right_arm_joint3, right_arm_joint4, right_arm_joint5, right_arm_joint6, right_arm_joint7
左夹爪	left_gripper	1	left_gripper_joint1
右夹爪	right_gripper	1	right_gripper_joint1
左吸盘	left_suction_cup	1	left_suction_cup_joint1
右吸盘	right_suction_cup	1	right_suction_cup_joint1

如何正确使用 joint_groups

joint_groups 的设计单位是“关节组（运动链）”，而不是单个关节。这样设计是为了保证：

- 运动链一致性：头/臂/腿等关节按链路整体校验和控制。
- 指令顺序确定性：joint_groups 会按传入顺序展开为具体关节。
- 组级约束校验：执行前会检查关节组的 active/passive 属性及输入合法性。

传参规则：

- 1. 需要控制整条链路时，优先使用 joint_groups (head/arm/leg/gripper/suction cup) 。
- 1. 当 joint_names 非空时，优先使用 joint_names，会覆盖 joint_groups。
- 1. 当 joint_groups 和 joint_names 都为空时，SDK 默认控制所有 active body 关节组。
- 1. 建议先调用 get_joint_group_names() 和 get_joint_names(true, groups)，避免手写字符串出错。

G1 关节组典型场景

关节组	典型场景
head	头部朝向/相机对准
left_arm / right_arm	机械臂抓取与操作
leg	下肢姿态调整
left_gripper / right_gripper	夹爪开合宽度控制
left_suction_cup / right_suction_cup	吸盘吸放作业

传感器类型与 Frame 说明

获取传感器数据时 (get_rgb_data、get_depth_data、get_imu_data、get_lidar_data) ，通过 SensorType 枚举指定要查询的传感器。可用的 SensorType 枚举值详见：C++ API 参考 > 传感器类型。

获取传感器外参有两种方式：

- get_sensor_extrinsic(): SDK 内部已映射 frame ID，直接传入 SensorType 即可
- get_transform(): 需要显式传入 frame 名称，表格第二列为各传感器对应的 Frame ID

SensorType	Frame ID
HEAD_LEFT_CAMERA	head_left_camera_color_optical_frame
HEAD_RIGHT_CAMERA	head_right_camera_color_optical_frame
LEFT_ARM_CAMERA	left_arm_camera_color_optical_frame
LEFT_ARM_DEPTH_CAMERA	left_arm_camera_color_optical_frame
RIGHT_ARM_CAMERA	right_arm_camera_color_optical_frame
RIGHT_ARM_DEPTH_CAMERA	right_arm_camera_color_optical_frame
BASE_LIDAR	lidar_base_link
TORSO_IMU	imu_base_link
BASE_ULTRASONIC	— (无 TF frame)
LEFT_FRONT_SURROUND_CAMERA	left_front_surround_color_optical_frame
RIGHT_FRONT_SURROUND_CAMERA	right_front_surround_color_optical_frame
LEFT_REAR_SURROUND_CAMERA	left_rear_surround_color_optical_frame
RIGHT_REAR_SURROUND_CAMERA	right_rear_surround_color_optical_frame

注意：调用 get_frame_names() 可获取当前所有可用的坐标系名称。

类：GalbotRobot

tips: 程序启动后立刻就 get, 数据可能不会立刻就绪, 可适当 sleep 几秒

获取实例并初始化 (get_instance && init)

适用场景: 程序启动阶段, 获取机器人单例并完成 SDK 初始化。必须在调用其他 API 前执行。

```
examples/cpp/galbot_robot/src/get_instance_init_example.cpp

#include <iostream>
#include <vector>
#include <array>
#include <memory>
#include <thread>

#include "galbot_robot.hpp"

using namespace galbot::sdk;

int main() {
    // Get object instance
    auto& robot = GalbotRobot::get_instance(MachineType::G1);

    // Initialize singleton instance
    if (robot.init()) {
        std::cout << "System initialized successfully!" << std::endl;
    } else {
        std::cerr << "System initialization failed!" << std::endl;
        return -1;
    }

    // Check whether it is in the running state
    while (robot.is_running()) {
        // do something
        std::cout << "System is running." << std::endl;
        break;
    }

    // Register an exit callback (optional; automatically triggered when an exit signal is
    received)
    robot.register_exit_callback([]() {
        std::cout << "System is exiting..." << std::endl;
    });
    std::cout << "System exit callback registered successfully" << std::endl;

    // Send exit signal
    robot.request_shutdown();
    // Wait until entering shutdown state
    robot.wait_for_shutdown();
    // Release SDK related resources
    robot.destroy();
    std::cout << "Program finished" << std::endl;

    return 0;
}
```

释放 SDK 相关资源并退出程序

适用场景：程序退出前释放 SDK 占用的所有资源，正常关闭系统。

```
examples/cpp/galbot_robot/src/stop_base_example.cpp
#include <iostream>
#include <vector>
#include <chrono>
#include <thread>
#include <string>

#include "galbot_robot.hpp"

using namespace galbot::sdk;

int main() {
    // Get object instance
    auto& robot = GalbotRobot::get_instance(MachineType::G1);

    // Initialize system
    if (robot.init()) {
        std::cout << "System initialized successfully!" << std::endl;
    } else {
        std::cerr << "System initialization failed!" << std::endl;
        return -1;
    }

    // Program started, waiting for data
    std::this_thread::sleep_for(std::chrono::milliseconds(2000));

    // Stop chassis motion
    while (true) {
        ControlStatus status = robot.stop_base();
        if (status == ControlStatus::SUCCESS) {
            std::cout << "Chassis motion has been stopped successfully!" << std::endl;
            break;
        } else {
            std::cerr << "Chassis stop motion failed, retrying..." << std::endl;
        }
    }

    // Exit system and release SDK resources
    robot.request_shutdown();
    robot.wait_for_shutdown();
    robot.destroy();

    return 0;
}
```

日志函数

适用场景：使用 SDK 内置的日志系统输出日志，统一日志级别和格式。

```
examples/cpp/galbot_robot/src/logger_example.cpp
#include <iostream>

#include "galbot_robot.hpp"
#include "galbot_sdk_logger.hpp"
```

```

using namespace galbot::sdk;

int main() {
    // Get and initialize the GalbotRobot singleton
    auto& robot = GalbotRobot::get_instance(MachineType::G1);
    robot.init();
    // =====
    // 1. Initialize SDK logging
    // =====
    LoggerConfig logger_config;
    // Log generation path; if not filled, defaults to ~/galbot_sdk_log/user_log
    logger_config.path = "";
    // Log file name; if not filled, defaults to
    <process_name>_<current_time>_<pid>_<thread_id>.log
    logger_config.file_name = "";
    // Maximum bytes per single log file
    logger_config.file_max_size = 1024 * 1024 * 10; // 10MB
    // Maximum number of rotating log files; creates new file when full, retaining up to
    file_max_num files
    logger_config.file_max_num = 5;
    // Minimum log output level
    logger_config.level = LogLevel::DEBUG;
    // Whether to output to terminal; default is false
    logger_config.console_output = false;

    if (!init_galbot_sdk_logger(logger_config)) {
        std::cerr << "failed to init galbot sdk logger" << std::endl;
        return -1;
    } else {
        std::cout << "galbot sdk logger initialized successfully" << std::endl;
    }

    // =====
    // 2. Print logs at different levels
    // =====
    GALBOT_SDK_LOG_TRACE << "this is trace log";
    GALBOT_SDK_LOG_DEBUG << "this is debug log";
    GALBOT_SDK_LOG_INFO << "this is info log";
    GALBOT_SDK_LOG_WARN << "this is warning log";
    GALBOT_SDK_LOG_ERROR << "this is error log";
    GALBOT_SDK_LOG_CRITICAL << "this is critical log";

    // =====
    // 3. Supports chained << style
    // =====
    int robot_id = 3;
    GALBOT_SDK_LOG_INFO << "robot_id = " << robot_id;

    GALBOT_SDK_LOG_INFO << "example program finished";

    // Exit the system and release SDK and logger resources
    robot.request_shutdown();
    robot.wait_for_shutdown();
    robot.destroy();
    return 0;
}

```


设置关节角度 (set_joint_positions)

适用场景：单次点位移动，低频率的位置控制任务。该接口内部会对目标位置进行速度受限的插值规划，适合一次性移动到目标关节角度的场景。

WARNING: 此接口**不适合**模型推理输出的高频关节控制场景！该接口每次调用都会产生一次轨迹插值，连续调用会导致运动不连续且有延迟。

如果您在做**模型推理**场景，请使用 set_joint_commands 或 set_joint_commands_batch 直接下发关节指令。

```
examples/cpp/galbot_robot/src/set_joint_positions_example.cpp

#include <iostream>
#include <vector>
#include <chrono>
#include <thread>
#include <string>

#include "galbot_robot.hpp"

using namespace galbot::sdk;

int main() {
    // Get object instance
    auto& robot = GalbotRobot::get_instance(MachineType::G1);

    // Initialize system
    if (robot.init()) {
        std::cout << "System initialized successfully!" << std::endl;
    } else {
        std::cerr << "System initialization failed!" << std::endl;
        return -1;
    }

    // Program started, waiting for data
    std::this_thread::sleep_for(std::chrono::milliseconds(2000));

    // Get specified joint names; joint groups include ["leg", "head", "left_arm",
    "right_arm"]
    std::vector<std::string> joint_groups = {"head"};
    bool only_active_joint = true; // Get active joints
    auto head_joint_names_vec =
        robot.get_joint_names(only_active_joint, joint_groups);
    std::cout << "Head joint names:" << std::endl;
    for (size_t i = 0; i < head_joint_names_vec.size(); ++i) {
        std::cout << i << ": " << head_joint_names_vec[i] << std::endl;
    }
    // Passing an empty array returns all joint group information by default
    std::vector<std::string> null_vec = {};
    auto all_joint_names_vec =
        robot.get_joint_names(only_active_joint, null_vec);
    std::cout << "All joint names:" << std::endl;
    for (size_t i = 0; i < all_joint_names_vec.size(); ++i) {
        std::cout << i << ": " << all_joint_names_vec[i] << std::endl;
    }

    // Joint groups to control; passing an empty array controls leg, head, left_arm, and
```

```

right_arm by default
    joint_groups = {"head"};
    // Specific joints to control; if provided, this overrides the joint_groups parameter
    std::vector<std::string> joint_names = {};
    // Joint positions; head joint group contains two joints
    std::vector<double> joint_pos = {0.2, 0.2};
    // Whether to block and wait for joint angles to reach position or timeout
    bool is_block = true;
    // Maximum joint speed (rad/s)
    double speed_rad_s = 0.1;
    // Maximum wait time (seconds)
    double timeout_s = 10.0;

    // Set joint positions
    ControlStatus joint_execution_status =
        robot.set_joint_positions(joint_pos, joint_groups, joint_names,
                                is_block, speed_rad_s, timeout_s);

    if (joint_execution_status == ControlStatus::SUCCESS) {
        std::cout << "Joint command set successfully!" << std::endl;
    } else {
        std::cerr << "Failed to set joint command!" << std::endl;
    }

    // Query joint positions by group; empty array defaults to leg, head, dual-arm groups.
    Second parameter specifies joint names, which overrides joint_groups if provided.
    auto ret_positions = robot.get_joint_positions(joint_groups, {});
    for (auto position : ret_positions) {
        std::cout << "joint positions is " << position << std::endl;
    }

    // Use specific joint names for control; this parameter overrides joint_groups
    joint_names = {"head_joint1", "head_joint2"};
    joint_pos = {0.0, 0.0};

    // Set joint positions
    joint_execution_status = robot.set_joint_positions(joint_pos, joint_groups, joint_names,
        is_block, speed_rad_s, timeout_s);

    if (joint_execution_status == ControlStatus::SUCCESS) {
        std::cout << "Joint command set successfully!" << std::endl;
    } else {
        std::cerr << "Failed to set joint command!" << std::endl;
    }

    // Query joint positions by group; empty array defaults to leg, head, dual-arm groups.
    Second parameter specifies joint names, which overrides joint_groups if provided.
    ret_positions = robot.get_joint_positions(joint_groups, {});
    for (auto position : ret_positions) {
        std::cout << "joint positions is " << position << std::endl;
    }

    // Exit system and release SDK resources
    robot.request_shutdown();
    robot.wait_for_shutdown();
    robot.destroy();

    return 0;
}

```

设置夹爪指令 (set_gripper_command)

适用场景：控制夹爪开合。支持位置控制和力矩控制两种模式。

```
examples/cpp/galbot_robot/src/set_gripper_command_example.cpp
#include <iostream>
#include <vector>
#include <chrono>
#include <thread>
#include <string>

#include "galbot_robot.hpp"

using namespace galbot::sdk;

void print_gripper_state(std::shared_ptr<GripperState> gripper_state) {
    std::cout << "Timestamp (ns): " << gripper_state->timestamp_ns << std::endl;

    std::cout << " width " << gripper_state->width << " velocity " << gripper_state->velocity
    << " effort " << gripper_state->effort << " is moving "
    << gripper_state->is_moving << std::endl;
}

int main() {
    // Get object instance
    auto& robot = GalbotRobot::get_instance(MachineType::G1);

    // Initialize system
    if (robot.init()) {
        std::cout << "System initialized successfully!" << std::endl;
    } else {
        std::cerr << "System initialization failed!" << std::endl;
        return -1;
    }

    // Program started, waiting for data
    std::this_thread::sleep_for(std::chrono::milliseconds(2000));

    // Gripper width (m)
    double width_m = 0.02;
    // Gripper speed (m/s)
    double velocity_mps = 0.05;
    // Gripper torque (N·m)
    double effort = 10;
    // Whether to block until execution completes
    bool is_blocking = true;
    // Set left gripper width to 0.02m, speed 0.05m/s, torque 10, block until execution
    // completes
    ControlStatus gripper_execution_status =
        robot.set_gripper_command("left_gripper", width_m, velocity_mps,
                                   effort, is_blocking);

    if (gripper_execution_status == ControlStatus::SUCCESS) {
        std::cout << "Gripper command set successfully!" << std::endl;
    } else {
        std::cerr << "Failed to set gripper command!" << std::endl;
    }
}
```

```

// Get gripper state
JointStateMessage joint_state;
auto gripper_state_ptr = robot.get_gripper_state("left_gripper");

if (gripper_state_ptr == nullptr) {
    std::cerr << "get gripper state error" << std::endl;
} else {
    print_gripper_state(gripper_state_ptr);
}

// Gripper width (m)
width_m = 0.1;
// Gripper speed (m/s)
velocity_mps = 0.05;
// Gripper torque (N·m)
effort = 10;
// Whether to block until execution completes
is_blocking = false;
// Set left gripper width to 0.1m, speed 0.05m/s, torque 10, block until execution
completes
gripper_execution_status =
    robot.set_gripper_command("left_gripper", width_m, velocity_mps,
                              effort, is_blocking);

if (gripper_execution_status == ControlStatus::SUCCESS) {
    std::cout << "Gripper command set successfully!" << std::endl;
} else {
    std::cerr << "Failed to set gripper command!" << std::endl;
}

// Exit system and release SDK resources
robot.request_shutdown();
robot.wait_for_shutdown();
robot.destroy();

return 0;
}

```

设置吸盘指令 (set_suction_cup_command)

适用场景：控制吸盘吸放物体，获取吸盘当前的开启/关闭状态。

```

examples/cpp/galbot_robot/src/set_suction_cup_command_example.cpp

#include <iostream>
#include <vector>
#include <chrono>
#include <thread>
#include <string>

#include "galbot_robot.hpp"

using namespace galbot::sdk;

int main() {
    // Get object instance
    auto& robot = GalbotRobot::get_instance(MachineType::G1);

    // Initialize system
    if (robot.init()) {

```

```

        std::cout << "System initialized successfully!" << std::endl;
    } else {
        std::cerr << "System initialization failed!" << std::endl;
        return -1;
    }

    // Program started, waiting for data
    std::this_thread::sleep_for(std::chrono::milliseconds(2000));

    // Activate suction cup
    if (robot.set_suction_cup_command("right_suction_cup", true) == ControlStatus::SUCCESS)
    {
        std::cout << "Suction cup activation command sent successfully" << std::endl;

    } else {
        std::cerr << "Suction cup activation command failed to send!" << std::endl;
    }
    std::this_thread::sleep_for(std::chrono::milliseconds(1000));
    // Deactivate suction cup
    if (robot.set_suction_cup_command("right_suction_cup", false) ==
ControlStatus::SUCCESS) {
        std::cout << "Suction cup deactivation command sent successfully" << std::endl;

    } else {
        std::cerr << "Suction cup deactivation command failed to send" << std::endl;
    }

    // Exit system and release SDK resources
    robot.request_shutdown();
    robot.wait_for_shutdown();
    robot.destroy();

    return 0;
}

```

停止轨迹执行 (stop_trajectory_execution)

适用场景：停止当前正在执行的关节轨迹，中断正在进行的运动。

```

examples/cpp/galbot_robot/src/stop_trajectory_execution_example.cpp

#include <iostream>
#include <vector>
#include <chrono>
#include <thread>
#include <string>

#include "galbot_robot.hpp"

using namespace galbot::sdk;

int main() {
    // Get object instance
    auto& robot = GalbotRobot::get_instance(MachineType::G1);

    // Initialize system
    if (robot.init()) {
        std::cout << "System initialized successfully!" << std::endl;
    } else {
        std::cerr << "System initialization failed!" << std::endl;
    }
}

```

```

        return -1;
    }

    // Program started, waiting for data
    std::this_thread::sleep_for(std::chrono::milliseconds(2000));

    // Stop trajectory execution
    while(true) {
        ControlStatus joint_execution_status =
            robot.stop_trajectory_execution();

        // Check execution results
        if (joint_execution_status == ControlStatus::SUCCESS) {
            std::cout << "Trajectory stop command sent successfully" << std::endl;
            break;
        } else {
            std::cerr << "Failed to send trajectory stop command, retrying..." <<
std::endl;
        }
    }

    // Exit system and release SDK resources
    robot.request_shutdown();
    robot.wait_for_shutdown();
    robot.destroy();

    return 0;
}

```

设置轨迹 (execute_joint_trajectory)

适用场景：执行一条预定义的关节空间轨迹，包含多个路点。SDK 会按照时间节点执行整条轨迹。

```

examples/cpp/galbot_robot/src/execute_joint_trajectory_example.cpp

#include <iostream>
#include <vector>
#include <chrono>
#include <thread>
#include <string>

#include "galbot_robot.hpp"

using namespace galbot::sdk;

double g_target_time = 10;
double g_start_time = 10;

std::string trajectory_status_to_string(TrajectoryControlStatus status) {
    switch (status) {
        case TrajectoryControlStatus::INVALID_INPUT:
            return "INVALID_INPUT";
        case TrajectoryControlStatus::RUNNING:
            return "RUNNING";
        case TrajectoryControlStatus::COMPLETED:
            return "COMPLETED";
        case TrajectoryControlStatus::STOPPED_UNREACHED:
            return "STOPPED_UNREACHED";
        case TrajectoryControlStatus::ERROR:
            return "ERROR";
    }
}

```

```

    case TrajectoryControlStatus::DATA_FETCH_FAILED:
        return "DATA_FETCH_FAILED";
    case TrajectoryControlStatus::STATUS_NUM:
        return "STATUS_NUM";
    default:
        return "UNKNOWN_STATUS";
    }
}

void wait_for_traj_reached(const std::vector<std::string> &joint_groups) {
    std::vector<TrajectoryControlStatus> traj_exec_states;
    int count = 0;
    bool all_reached = false;
    while (count++ < 150) {
        std::this_thread::sleep_for(std::chrono::milliseconds(100));
        all_reached = true;
        traj_exec_states = GalbotRobot::get_instance(MachineType::G1)
            .check_trajectory_execution_status(joint_groups);
        if (traj_exec_states.size() != joint_groups.size()) {
            std::cout << "traj_exec_states size != joint_groups size" << std::endl;
        }
        for (int i = 0; i < joint_groups.size(); ++i) {
            std::cout << joint_groups[i] << " exec state is "
                << trajectory_status_to_string(traj_exec_states[i])
                << std::endl;
            if (traj_exec_states[i] != TrajectoryControlStatus::COMPLETED) {
                all_reached = false;
            }
        }

        if (all_reached) {
            std::cout << "all reached" << std::endl;
            break;
        }
    }
    for (const auto &status : traj_exec_states) {
        std::cout << "done reached state is " << trajectory_status_to_string(status)
            << std::endl;
    }
}

std::vector<TrajectoryPoint>
generate_target_trajectory(int32_t joint_size, double ampl = 0.2,
    double cycle = 10) {
    double amplitude = -ampl;
    double frequency = 1.0 / cycle;
    double phase = -M_PI / 2;
    double offset = amplitude;
    double dt = 0.004;
    int step = g_target_time / dt;

    std::vector<TrajectoryPoint> trajectory_data_vec;
    trajectory_data_vec.resize(step + 1);
    // Create a RobotCommand trajectory
    for (int i = 0; i <= step; ++i) {
        double t = i * dt;
        trajectory_data_vec[i].time_from_start_second = g_start_time + t;
        trajectory_data_vec[i].joint_command_vec.resize(joint_size);
        // Joint command
        for (int j = 0; j < joint_size; ++j) {

```

```

        trajectory_data_vec[i].joint_command_vec[j].position =
            offset + amplitude * std::sin(2 * M_PI * frequency * t + phase);
        trajectory_data_vec[i].joint_command_vec[j].velocity =
            amplitude * 2 * M_PI * frequency *
            std::cos(2 * M_PI * frequency * t + phase);
    }
}

return trajectory_data_vec;
}

int main() {
    // Get object instance
    auto& robot = GalbotRobot::get_instance(MachineType::G1);

    // Initialize system
    if (robot.init()) {
        std::cout << "System initialized successfully!" << std::endl;
    } else {
        std::cerr << "System initialization failed!" << std::endl;
        return -1;
    }

    // Program started, waiting for data
    std::this_thread::sleep_for(std::chrono::milliseconds(2000));

    // Execute joint trajectory
    Trajectory trajectory;
    // Enter joint group name to control, including ["leg", "head", "left_arm",
    "right_arm", "left_gripper", "right_gripper"]
    trajectory.joint_groups = {"head"};
    // Fill this field to control specific joint angles, which will override joint_groups
    if provided
    trajectory.joint_names = {};
    // Generate trajectory
    trajectory.points = generate_target_trajectory(2);
    // Whether to block until trajectory execution completes; when false, you can use
    bool is_traj_block = false;

    // Wait for trajectory execution to complete; this function wraps
    check_trajectory_execution_status to check trajectory execution status
    wait_for_traj_reached(trajectory.joint_groups);

    robot.request_shutdown();
    robot.wait_for_shutdown();
    robot.destroy();

    return 0;
}

```

设置关节指令 (set_joint_commands)

适用场景：高频关节控制，例如**模型推理输出**（每步推理输出一帧关节指令）、自定义轨迹跟踪。直接下发关节指令给底层控制器，不做额外插值。

examples/cpp/galbot_robot/src/set_joint_command_example.cpp


```

#include <iostream>
#include <vector>
#include <chrono>
#include <thread>
#include <string>

#include "galbot_robot.hpp"

using namespace galbot::sdk;

void print_joint_positions(const std::vector<double>& positions) {
    std::cout << "Current joint positions:" << std::endl;
    for (size_t i = 0; i < positions.size(); ++i) {
        std::cout << "  joint " << i << ": " << positions[i] << " rad" << std::endl;
    }
    std::cout << std::endl;
}

std::string execution_status_to_string(ControlStatus status) {
    switch (status) {
        case ControlStatus::SUCCESS:
            return "SUCCESS";
        case ControlStatus::TIMEOUT:
            return "TIMEOUT";
        case ControlStatus::FAULT:
            return "FAULT";
        case ControlStatus::INVALID_INPUT:
            return "INVALID_INPUT";
        case ControlStatus::INIT_FAILED:
            return "INIT_FAILED";
        case ControlStatus::IN_PROGRESS:
            return "IN_PROGRESS";
        case ControlStatus::STOPPED_UNREACHED:
            return "STOPPED_UNREACHED";
        case ControlStatus::DATA_FETCH_FAILED:
            return "DATA_FETCH_FAILED";
        case ControlStatus::PUBLISH_FAIL:
            return "PUBLISH_FAIL";
        case ControlStatus::COMM_DISCONNECTED:
            return "COMM_DISCONNECTED";
        default:
            return "UNKNOWN_STATUS";
    }
}

int main() {
    // Get object instance
    auto& robot = GalbotRobot::get_instance(MachineType::G1);

    // Initialize system
    if (robot.init()) {
        std::cout << "System initialized successfully!" << std::endl;
    } else {
        std::cerr << "System initialization failed!" << std::endl;
        return -1;
    }

    // data
    std::this_thread::sleep_for(std::chrono::milliseconds(2000));
}

```

```

std::vector<std::string> joint_groups = {"head"};
std::vector<std::string> joint_names = {};

std::vector<JointCommand> joint_commands(2);
joint_commands[0].position = 0.2;
joint_commands[1].position = 0.2;
// Set head joint angles to 0.3 0.3
ControlStatus execution_status =
    robot.set_joint_commands(joint_commands, joint_groups, joint_names);
if (execution_status != ControlStatus::SUCCESS) {
    std::cout << "Joint angle command sending failed" << std::endl;
} else {
    std::cout << "Joint angle command sent successfully" << std::endl;
}

// , waitexecute
std::this_thread::sleep_for(std::chrono::milliseconds(10000));

// Query joint positions
auto ret_positions = robot.get_joint_positions(joint_groups, {});
print_joint_positions(ret_positions);

// Step 2: Return to initial position — set both head joints to 0.0 rad
joint_commands[0].position = 0.0;
joint_commands[1].position = 0.0;

// Set joint commands by joint names; setting joint_names overrides joint_groups
joint_groups = {"");
joint_names = {"head_joint1", "head_joint2"};
execution_status =
    robot.set_joint_commands(joint_commands, joint_groups, joint_names);
if (execution_status != ControlStatus::SUCCESS) {
    std::cout << "Joint angle command sending failed" << std::endl;
} else {
    std::cout << "Joint angle command sent successfully" << std::endl;
}

// Exit system and release SDK resources
robot.request_shutdown();
robot.wait_for_shutdown();
robot.destroy();

std::cout << "\nProgram exited" << std::endl;
return 0;
}

```

批量设置关节指令 (set_joint_commands_batch)

适用场景：批量下发未来多帧关节指令，用于运动预测模型一次性输出多步结果的场景，可以提高控制频率和连续性。

```

examples/cpp/galbot_robot/src/set_joint_commands_batch_example.cpp

#include <chrono>
#include <cmath>
#include <iostream>
#include <string>
#include <thread>

```

```

#include <vector>

#include "galbot_robot.hpp"

using namespace galbot::sdk;

std::string control_status_to_string(ControlStatus status) {
    switch (status) {
        case ControlStatus::SUCCESS:
            return "SUCCESS";
        case ControlStatus::INVALID_INPUT:
            return "INVALID_INPUT";
        case ControlStatus::INIT_FAILED:
            return "INIT_FAILED";
        case ControlStatus::COMM_DISCONNECTED:
            return "COMM_DISCONNECTED";
        case ControlStatus::FAULT:
            return "FAULT";
        case ControlStatus::PUBLISH_FAIL:
            return "PUBLISH_FAIL";
        default:
            return "UNKNOWN_STATUS";
    }
}

std::vector<TrajectoryPoint> generate_batch_trajectory(int32_t joint_size, double ampl =
0.2, double cycle = 10,
                                                    int num_points = 10) {

    double amplitude = -ampl;
    double frequency = 1.0 / cycle;
    double phase = -M_PI / 2;
    double offset = amplitude;
    double dt = cycle / num_points;

    std::vector<TrajectoryPoint> trajectory_data_vec;
    trajectory_data_vec.resize(num_points);
    // Create batch trajectory points
    for (int i = 0; i < num_points; ++i) {
        double t = i * dt;
        trajectory_data_vec[i].time_from_start_second = t;
        trajectory_data_vec[i].joint_command_vec.resize(joint_size);
        // Joint command
        for (int j = 0; j < joint_size; ++j) {
            trajectory_data_vec[i].joint_command_vec[j].position =
                offset + amplitude * std::sin(2 * M_PI * frequency * t + phase);
            trajectory_data_vec[i].joint_command_vec[j].velocity =
                amplitude * 2 * M_PI * frequency * std::cos(2 * M_PI * frequency * t + phase);
            // trajectory_data_vec[i].joint_command_vec[j].acceleration = 0.0;
            // trajectory_data_vec[i].joint_command_vec[j].effort = 0.0;
        }
    }

    return trajectory_data_vec;
}

int main() {
    // Get object instance
    auto& robot = GalbotRobot::get_instance(MachineType::G1);

    // Initialize system

```

```

if (robot.init()) {
    std::cout << "System initialized successfully!" << std::endl;
} else {
    std::cerr << "System initialization failed!" << std::endl;
    return -1;
}

// Program started, waiting for data
std::this_thread::sleep_for(std::chrono::milliseconds(2000));

// Batch set joint commands
Trajectory trajectory;
// Enter joint group names to control, including ["leg", "head", "left_arm", "right_arm",
"left_gripper", "right_gripper"]
trajectory.joint_groups = {"head"};
// Fill this field to control specific joint angles, which will override joint_groups if
provided
trajectory.joint_names = {};
// Generate batched trajectory points (joint commands at multiple time points)
trajectory.points = generate_batch_trajectory(2, 0.2, 10.0, 10);

// Batch set joint commands (non-blocking, returns immediately)
ControlStatus status = robot.set_joint_commands_batch(trajectory);
std::cout << "Batch joint command status: " << control_status_to_string(status) <<
std::endl;

if (status == ControlStatus::SUCCESS) {
    std::cout << "Batch commands submitted, executing in background (non-blocking)" <<
std::endl;
} else {
    std::cerr << "Failed to submit batch commands!" << std::endl;
}

// Wait for a while to let the command execute
std::this_thread::sleep_for(std::chrono::milliseconds(1000));

// Exit system and release SDK resources
robot.request_shutdown();
robot.wait_for_shutdown();
robot.destroy();

return 0;
}

```

直接发布原始目标 (PublishTarget)

适用场景：高级用户直接构造 SingoriXTarget 并通过 publish 通道下发到底层 WBCS，适合高频原始 target 发送与 joint/task mixed target 一次性 dispatch。

examples/cpp/galbot_robot/src/publish_target_example.cpp

```

/**
 * @file publish_target_example.cpp
 * @brief G1 PublishTarget menu example for SDK mirror SingoriXTarget.
 *
 * This example shows how to construct `SingoriXTarget` directly at the SDK layer
 * and send it to the low-level WBCS through `PublishTarget()`.
 *
 * 1. Joint-space commands are written into `target_group_trajectory_map`
 *    - Typical for head / arm style joint-space control

```

```

*   - Each group corresponds to one `TargetGroupTrajectory`
*   - Each trajectory point is described by `GroupCommand + JointCommand`
*
* 2. Chassis pose / twist style task-space commands are written into
*   `target_task_trajectory_map`
*   - Typical for chassis pose / chassis twist control
*   - Each task corresponds to one `TargetTaskTrajectory`
*   - Each trajectory point is described by `TaskCommand + FrameTriad`
*
* 3. One `SingoriXTarget` can contain both joint trajectory and task trajectory
*   - This supports one-shot dispatch for whole-body / mixed control
*
* 4. The current SDK does not automatically switch the chassis controller when
*   calling `PublishTarget()` / `RequestTarget()`
*   - Therefore this example explicitly calls
*     `switch_controller(G1ControllerName::CHASSIS_POSE_CTRL)` or
*     `switch_controller(G1ControllerName::CHASSIS_TWIST_CTRL)` before
*     chassis pose / twist / mixed scenes
*
* 5. For the base twist scene, this example automatically sends a zero-twist
*   target after the configured duration to stop the chassis.
*/

```

```

#include <chrono>
#include <cstdint>
#include <iostream>
#include <string>
#include <thread>
#include <unordered_map>
#include <vector>

#include "galbot_robot.hpp"

using namespace galbot::sdk;

namespace {

constexpr const char* kChassisTaskName = "chassis";
constexpr const char* kChassisSubtaskPose = "chassis_pose";
constexpr const char* kChassisSubtaskTwist = "chassis_twist";

int64_t now_ns() {
    return std::chrono::duration_cast<std::chrono::nanoseconds>(
        std::chrono::system_clock::now().time_since_epoch())
        .count();
}

const char* to_string(ControlStatus status) {
    switch (status) {
        case ControlStatus::SUCCESS:
            return "SUCCESS";
        case ControlStatus::TIMEOUT:
            return "TIMEOUT";
        case ControlStatus::FAULT:
            return "FAULT";
        case ControlStatus::INVALID_INPUT:
            return "INVALID_INPUT";
        case ControlStatus::INIT_FAILED:
            return "INIT_FAILED";
        case ControlStatus::IN_PROGRESS:

```

```

        return "IN_PROGRESS";
    case ControlStatus::STOPPED_UNREACHED:
        return "STOPPED_UNREACHED";
    case ControlStatus::DATA_FETCH_FAILED:
        return "DATA_FETCH_FAILED";
    case ControlStatus::PUBLISH_FAIL:
        return "PUBLISH_FAIL";
    case ControlStatus::COMM_DISCONNECTED:
        return "COMM_DISCONNECTED";
    default:
        return "UNKNOWN_STATUS";
    }
}

Quaternion yaw_to_quaternion(double yaw) {
    Quaternion q;
    q.z = std::sin(yaw * 0.5);
    q.w = std::cos(yaw * 0.5);
    return q;
}

SingorIXTarget make_empty_target() {
    SingorIXTarget target;
    target.header.timestamp_ns = now_ns();
    target.header.frame_id = "base_link";
    return target;
}

TargetConfig make_group_target_config() {
    TargetConfig config;
    config.target_data = TARGET_DATA_DEFAULT;
    config.target_type = TARGET_TYPE_PROVERRIDE;
    config.target_sampling = TargetSampling::TARGET_SAMPLING_DEFAULT;
    config.target_priority = 1;
    return config;
}

TargetConfig make_pose_target_config() {
    TargetConfig config;
    config.target_data = TARGET_DATA_FRAME_POSE;
    config.target_type = TARGET_TYPE_PROVERRIDE;
    config.target_sampling = TargetSampling::TARGET_SAMPLING_LINEAR_INTERPOLATE;
    config.target_priority = 1;
    return config;
}

TargetConfig make_twist_target_config() {
    TargetConfig config;
    config.target_data = TARGET_DATA_FRAME_TWIST;
    config.target_type = TARGET_TYPE_OVERRIDE;
    config.target_sampling = TargetSampling::TARGET_SAMPLING_DIRECT_PASS;
    config.target_priority = 1;
    return config;
}

SingorIXTarget build_joint_target(const std::string& group_name,
                                   const std::vector<std::string>& joint_names,
                                   const std::vector<double>& positions,
                                   double time_from_start_s) {
    SingorIXTarget target = make_empty_target();

```

```

    auto& group_traj = target.target_group_trajectory_map[group_name];
    group_traj.target_config = make_group_target_config();
    group_traj.joint_names = joint_names;

    GroupCommand command;
    command.time_from_start_s = time_from_start_s;
    command.joint_commands.resize(joint_names.size());
    for (size_t i = 0; i < joint_names.size(); ++i) {
        command.joint_commands[i].position = positions[i];
        command.joint_commands[i].velocity = 0.0;
        command.joint_commands[i].acceleration = 0.0;
        command.joint_commands[i].effort = 0.0;
    }
    group_traj.group_commands.push_back(command);
    return target;
}

SingoriXTarget build_chassis_pose_target(double x,
                                         double y,
                                         double yaw,
                                         double time_from_start_s,
                                         const std::string& frame_id = "odom",
                                         const std::string& reference_frame_id = "odom") {
    SingoriXTarget target = make_empty_target();
    auto& task_traj = target.target_task_trajectory_map[kChassisTaskName];
    task_traj.target_config = make_pose_target_config();
    task_traj.group_names = {G1JointGroup::CHASSIS};
    task_traj.subtask_names = {kChassisSubtaskPose};

    TaskCommand command;
    command.time_from_start_s = time_from_start_s;

    FrameTriad triad;
    triad.header.timestamp_ns = now_ns();
    triad.header.frame_id = frame_id;
    triad.body_frame_id = "base_link";
    triad.reference_frame_id = reference_frame_id;
    triad.pose = Pose();
    triad.pose->position.x = x;
    triad.pose->position.y = y;
    triad.pose->position.z = 0.0;
    triad.pose->orientation = yaw_to_quaternion(yaw);
    command.subtask_commands.push_back(triad);

    task_traj.task_commands.push_back(command);
    return target;
}

SingoriXTarget build_chassis_twist_target(double vx,
                                         double vy,
                                         double wz,
                                         double time_from_start_s) {
    SingoriXTarget target = make_empty_target();
    auto& task_traj = target.target_task_trajectory_map[kChassisTaskName];
    task_traj.target_config = make_twist_target_config();
    task_traj.group_names = {G1JointGroup::CHASSIS};
    task_traj.subtask_names = {kChassisSubtaskTwist};

    TaskCommand command;
    command.time_from_start_s = time_from_start_s;

```

```

FrameTriad triad;
triad.header.timestamp_ns = now_ns();
triad.header.frame_id = "base_link";
triad.body_frame_id = "base_link";
triad.reference_frame_id = "base_link";
triad.twist = Twist();
triad.twist->linear = Vector3{vx, vy, 0.0};
triad.twist->angular = Vector3{0.0, 0.0, wz};
command.subtask_commands.push_back(triad);

task_traj.task_commands.push_back(command);
return target;
}

SingoriXTarget build_stop_twist_target() {
    return build_chassis_twist_target(0.0, 0.0, 0.0, 0.1);
}

SingoriXTarget merge_targets(const std::vector<SingoriXTarget>& targets) {
    SingoriXTarget merged = make_empty_target();
    for (const auto& target : targets) {
        for (const auto& [group_name, group_traj] : target.target_group_trajectory_map) {
            merged.target_group_trajectory_map[group_name] = group_traj;
        }
        for (const auto& [task_name, task_traj] : target.target_task_trajectory_map) {
            merged.target_task_trajectory_map[task_name] = task_traj;
        }
    }
    return merged;
}

void print_menu() {
    std::cout << "\nAvailable commands:\n"
        << "  joint      - publish a joint-only head target\n"
        << "  base_pose   - publish a chassis pose target\n"
        << "  base_twist  - publish a chassis twist target with auto stop\n"
        << "  mixed_pose  - publish head + left_arm + chassis pose in one target\n"
        << "  mixed_twist - publish head + left_arm + chassis twist in one target\n"
        << "  quit        - exit example\n"
        << std::endl;
}

ControlStatus ensure_controller(GalbotRobot& robot, const std::string& controller_name) {
    const auto status = robot.switch_controller(controller_name);
    std::cout << "switch_controller(" << controller_name << "): " << to_string(status) <<
    std::endl;
    return status;
}

void print_result(const std::string& scene_name, ControlStatus status) {
    std::cout << scene_name << " PublishTarget status: " << to_string(status) << std::endl;
}

ControlStatus run_twist_scene(GalbotRobot& robot,
                             const std::string& scene_name,
                             const SingoriXTarget& target,
                             double twist_duration_s) {
    std::cout << scene_name << ": start moving for " << twist_duration_s << " seconds" <<
    std::endl;
}

```



```

const auto motion_status = robot.PublishTarget(target);
print_result(scene_name, motion_status);
if (motion_status != ControlStatus::SUCCESS) {
    return motion_status;
}

std::this_thread::sleep_for(std::chrono::duration<double>(twist_duration_s));
std::cout << scene_name << ": send stop twist target" << std::endl;
const auto stop_status = robot.PublishTarget(build_stop_twist_target());
std::cout << scene_name << " stop status: " << to_string(stop_status) << std::endl;
return stop_status;
}

} // namespace

int main() {
    auto& robot = GalbotRobot::get_instance(MachineType::G1);
    if (!robot.init()) {
        std::cerr << "robot init failed" << std::endl;
        return -1;
    }

    std::this_thread::sleep_for(std::chrono::seconds(2));

    const auto head_joint_names = robot.get_joint_names(true, {G1JointGroup::HEAD});
    const auto left_arm_joint_names = robot.get_joint_names(true, {G1JointGroup::LEFT_ARM});
    if (head_joint_names.empty() || left_arm_joint_names.empty()) {
        std::cerr << "failed to fetch active head/left_arm joints" << std::endl;
        robot.request_shutdown();
        robot.wait_for_shutdown();
        robot.destroy();
        return -1;
    }

    const std::vector<std::string> head_single_joint = {head_joint_names.front()};
    const std::vector<std::string> arm_single_joint = {left_arm_joint_names.front()};
    constexpr double kJointTimeS = 3.0;
    constexpr double kPoseTimeS = 4.0;
    constexpr double kTwistCommandTimeS = 0.2;
    constexpr double kTwistDurationS = 2.0;

    print_menu();

    std::string command;
    while (true) {
        std::cout << "Enter command: ";
        if (!(std::cin >> command)) {
            break;
        }
        if (command == "quit") {
            break;
        }

        if (command == "joint") {
            const auto target = build_joint_target(G1JointGroup::HEAD, head_single_joint, {0.2},
kJointTimeS);
            print_result("joint", robot.PublishTarget(target));
            continue;
        }
    }
}

```

```

    if (command == "base_pose") {
        if (ensure_controller(robot, G1ControllerName::CHASSIS_POSE_CTRL) !=
ControlStatus::SUCCESS) {
            continue;
        }
        const auto target = build_chassis_pose_target(0.2, 0.0, 0.0, kPoseTimeS);
        print_result("base_pose", robot.PublishTarget(target));
        continue;
    }

    if (command == "base_twist") {
        if (ensure_controller(robot, G1ControllerName::CHASSIS_TWIST_CTRL) !=
ControlStatus::SUCCESS) {
            continue;
        }
        const auto target = build_chassis_twist_target(0.05, 0.0, 0.0, kTwistCommandTimeS);
        run_twist_scene(robot, "base_twist", target, kTwistDurationS);
        continue;
    }

    if (command == "mixed_pose") {
        if (ensure_controller(robot, G1ControllerName::CHASSIS_POSE_CTRL) !=
ControlStatus::SUCCESS) {
            continue;
        }
        const auto target = merge_targets({
            build_joint_target(G1JointGroup::HEAD, head_single_joint, {0.15}, kJointTimeS),
            build_chassis_pose_target(0.1, 0.0, 0.0, kPoseTimeS),
        });
        print_result("mixed_pose", robot.PublishTarget(target));
        continue;
    }

    if (command == "mixed_twist") {
        if (ensure_controller(robot, G1ControllerName::CHASSIS_TWIST_CTRL) !=
ControlStatus::SUCCESS) {
            continue;
        }
        const auto target = merge_targets({
            build_joint_target(G1JointGroup::HEAD, head_single_joint, {-0.15}, kJointTimeS),
            build_chassis_twist_target(0.05, 0.0, 0.0, kTwistCommandTimeS),
        });
        run_twist_scene(robot, "mixed_twist", target, kTwistDurationS);
        continue;
    }

    std::cout << "Unknown command: " << command << std::endl;
    print_menu();
}

robot.request_shutdown();
robot.wait_for_shutdown();
robot.destroy();
return 0;
}

```

直接请求原始目标 (RequestTarget)

适用场景：高级用户直接构造 SingoriXTarget 并通过 request 通道下发到底层 WBCS，适合需要获取服务端错误返回的 raw target 场景。

examples/cpp/galbot_robot/src/request_target_example.cpp

```
/**
 * @file request_target_example.cpp
 * @brief G1 RequestTarget menu example for SDK mirror SingoriXTarget.
 *
 * This example shows how to construct `SingoriXTarget` directly at the SDK layer
 * and send it to the low-level WBCS through `RequestTarget()`.
 *
 * 1. Joint-space commands are written into `target_group_trajectory_map`
 *    - Typical for head / arm style joint-space control
 *    - Each group corresponds to one `TargetGroupTrajectory`
 *    - Each trajectory point is described by `GroupCommand + JointCommand`
 *
 * 2. Chassis pose / twist style task-space commands are written into
 *    `target_task_trajectory_map`
 *    - Typical for chassis pose / chassis twist control
 *    - Each task corresponds to one `TargetTaskTrajectory`
 *    - Each trajectory point is described by `TaskCommand + FrameTriad`
 *
 * 3. One `SingoriXTarget` can contain both joint trajectory and task trajectory
 *    - This supports one-shot dispatch for whole-body / mixed control
 *
 * 4. The current SDK does not automatically switch the chassis controller when
 *    calling `PublishTarget()` / `RequestTarget()`
 *    - Therefore this example explicitly calls
 *      `switch_controller(G1ControllerName::CHASSIS_POSE_CTRL)` or
 *      `switch_controller(G1ControllerName::CHASSIS_TWIST_CTRL)` before
 *      chassis pose / twist / mixed scenes
 *
 * 5. For the base twist scene, this example automatically sends a zero-twist
 *    target after the configured duration to stop the chassis.
 */

#include <chrono>
#include <cstdint>
#include <iostream>
#include <memory>
#include <string>
#include <thread>
#include <unordered_map>
#include <vector>

#include "galbot_robot.hpp"

using namespace galbot::sdk;

namespace {

constexpr const char* kChassisTaskName = "chassis";
constexpr const char* kChassisSubtaskPose = "chassis_pose";
constexpr const char* kChassisSubtaskTwist = "chassis_twist";

int64_t now_ns() {
```

```

    return std::chrono::duration_cast<std::chrono::nanoseconds>(
        std::chrono::system_clock::now().time_since_epoch())
        .count();
}

Quaternion yaw_to_quaternion(double yaw) {
    Quaternion q;
    q.z = std::sin(yaw * 0.5);
    q.w = std::cos(yaw * 0.5);
    return q;
}

SingorIXTarget make_empty_target() {
    SingorIXTarget target;
    target.header.timestamp_ns = now_ns();
    target.header.frame_id = "base_link";
    return target;
}

TargetConfig make_group_target_config() {
    TargetConfig config;
    config.target_data = TARGET_DATA_DEFAULT;
    config.target_type = TARGET_TYPE_PROVERRIDE;
    config.target_sampling = TargetSampling::TARGET_SAMPLING_DEFAULT;
    config.target_priority = 1;
    return config;
}

TargetConfig make_pose_target_config() {
    TargetConfig config;
    config.target_data = TARGET_DATA_FRAME_POSE;
    config.target_type = TARGET_TYPE_PROVERRIDE;
    config.target_sampling = TargetSampling::TARGET_SAMPLING_LINEAR_INTERPOLATE;
    config.target_priority = 1;
    return config;
}

TargetConfig make_twist_target_config() {
    TargetConfig config;
    config.target_data = TARGET_DATA_FRAME_TWIST;
    config.target_type = TARGET_TYPE_OVERRIDE;
    config.target_sampling = TargetSampling::TARGET_SAMPLING_DIRECT_PASS;
    config.target_priority = 1;
    return config;
}

SingorIXTarget build_joint_target(const std::string& group_name,
                                  const std::vector<std::string>& joint_names,
                                  const std::vector<double>& positions,
                                  double time_from_start_s) {
    SingorIXTarget target = make_empty_target();
    auto& group_traj = target.target_group_trajectory_map[group_name];
    group_traj.target_config = make_group_target_config();
    group_traj.joint_names = joint_names;

    GroupCommand command;
    command.time_from_start_s = time_from_start_s;
    command.joint_commands.resize(joint_names.size());
    for (size_t i = 0; i < joint_names.size(); ++i) {
        command.joint_commands[i].position = positions[i];
    }
}

```

```

        command.joint_commands[i].velocity = 0.0;
        command.joint_commands[i].acceleration = 0.0;
        command.joint_commands[i].effort = 0.0;
    }
    group_traj.group_commands.push_back(command);
    return target;
}

SingoriXTarget build_chassis_pose_target(double x,
                                         double y,
                                         double yaw,
                                         double time_from_start_s,
                                         const std::string& frame_id = "odom",
                                         const std::string& reference_frame_id = "odom") {
    SingoriXTarget target = make_empty_target();
    auto& task_traj = target.target_task_trajectory_map[kChassisTaskName];
    task_traj.target_config = make_pose_target_config();
    task_traj.group_names = {G1JointGroup::CHASSIS};
    task_traj.subtask_names = {kChassisSubtaskPose};

    TaskCommand command;
    command.time_from_start_s = time_from_start_s;

    FrameTriad triad;
    triad.header.timestamp_ns = now_ns();
    triad.header.frame_id = frame_id;
    triad.body_frame_id = "base_link";
    triad.reference_frame_id = reference_frame_id;
    triad.pose = Pose();
    triad.pose->position.x = x;
    triad.pose->position.y = y;
    triad.pose->position.z = 0.0;
    triad.pose->orientation = yaw_to_quaternion(yaw);
    command.subtask_commands.push_back(triad);

    task_traj.task_commands.push_back(command);
    return target;
}

SingoriXTarget build_chassis_twist_target(double vx,
                                         double vy,
                                         double wz,
                                         double time_from_start_s) {
    SingoriXTarget target = make_empty_target();
    auto& task_traj = target.target_task_trajectory_map[kChassisTaskName];
    task_traj.target_config = make_twist_target_config();
    task_traj.group_names = {G1JointGroup::CHASSIS};
    task_traj.subtask_names = {kChassisSubtaskTwist};

    TaskCommand command;
    command.time_from_start_s = time_from_start_s;

    FrameTriad triad;
    triad.header.timestamp_ns = now_ns();
    triad.header.frame_id = "base_link";
    triad.body_frame_id = "base_link";
    triad.reference_frame_id = "base_link";
    triad.twist = Twist();
    triad.twist->linear = Vector3{vx, vy, 0.0};
    triad.twist->angular = Vector3{0.0, 0.0, wz};

```

```

    command.subtask_commands.push_back(triad);

    task_traj.task_commands.push_back(command);
    return target;
}

SingoriXTarget build_stop_twist_target() {
    return build_chassis_twist_target(0.0, 0.0, 0.0, 0.1);
}

SingoriXTarget merge_targets(const std::vector<SingoriXTarget>& targets) {
    SingoriXTarget merged = make_empty_target();
    for (const auto& target : targets) {
        for (const auto& [group_name, group_traj] : target.target_group_trajectory_map) {
            merged.target_group_trajectory_map[group_name] = group_traj;
        }
        for (const auto& [task_name, task_traj] : target.target_task_trajectory_map) {
            merged.target_task_trajectory_map[task_name] = task_traj;
        }
    }
    return merged;
}

void print_menu() {
    std::cout << "\nAvailable commands:\n"
        << "  joint      - request a joint-only head target\n"
        << "  base_pose  - request a chassis pose target\n"
        << "  base_twist - request a chassis twist target with auto stop\n"
        << "  mixed_pose - request head + left_arm + chassis pose in one target\n"
        << "  mixed_twist - request head + left_arm + chassis twist in one target\n"
        << "  quit      - exit example\n"
        << std::endl;
}

void print_error_info(const std::string& scene_name, const std::shared_ptr<ErrorInfo>&
error_info) {
    if (error_info == nullptr) {
        std::cout << scene_name << ": RequestTarget returned nullptr" << std::endl;
        return;
    }
    if (error_info->error_vec.empty()) {
        std::cout << scene_name << ": RequestTarget success, service returned no errors" <<
std::endl;
        return;
    }

    std::cout << scene_name << ": RequestTarget returned " << error_info->error_vec.size()
<< " error entries:"
        << std::endl;
    for (const auto& error : error_info->error_vec) {
        std::cout << "  component=" << error.component << ", code=" << error.error_code
            << ", description=" << error.description << std::endl;
    }
}

ControlStatus ensure_controller(GalbotRobot& robot, const std::string& controller_name) {
    const auto status = robot.switch_controller(controller_name);
    std::cout << "switch_controller(" << controller_name << "): ";
    std::cout << (status == ControlStatus::SUCCESS ? "SUCCESS" : "FAILED") << std::endl;
    return status;
}

```

```

}

void run_twist_scene(GalbotRobot& robot,
                    const std::string& scene_name,
                    const SingoriXTarget& target,
                    double twist_duration_s) {
    std::cout << scene_name << ": start moving for " << twist_duration_s << " seconds" <<
    std::endl;
    print_error_info(scene_name, robot.RequestTarget(target));
    std::this_thread::sleep_for(std::chrono::duration<double>(twist_duration_s));
    std::cout << scene_name << ": send stop twist target" << std::endl;
    print_error_info(scene_name + "_stop", robot.RequestTarget(build_stop_twist_target()));
}

} // namespace

int main() {
    auto& robot = GalbotRobot::get_instance(MachineType::G1);
    if (!robot.init()) {
        std::cerr << "robot init failed" << std::endl;
        return -1;
    }

    std::this_thread::sleep_for(std::chrono::seconds(2));

    const auto head_joint_names = robot.get_joint_names(true, {G1JointGroup::HEAD});
    if (head_joint_names.empty()) {
        std::cerr << "failed to fetch active head joints" << std::endl;
        robot.request_shutdown();
        robot.wait_for_shutdown();
        robot.destroy();
        return -1;
    }

    const std::vector<std::string> head_single_joint = {head_joint_names.front()};
    constexpr double kJointTimeS = 3.0;
    constexpr double kPoseTimeS = 4.0;
    constexpr double kTwistCommandTimeS = 0.2;
    constexpr double kTwistDurationS = 2.0;

    print_menu();

    std::string command;
    while (true) {
        std::cout << "Enter command: ";
        if (!(std::cin >> command)) {
            break;
        }
        if (command == "quit") {
            break;
        }

        if (command == "joint") {
            const auto target = build_joint_target(G1JointGroup::HEAD, head_single_joint, {0.2},
            kJointTimeS);
            print_error_info("joint", robot.RequestTarget(target));
            continue;
        }

        if (command == "base_pose") {

```

```

        if (ensure_controller(robot, G1ControllerName::CHASSIS_POSE_CTRL) !=
ControlStatus::SUCCESS) {
            continue;
        }
        const auto target = build_chassis_pose_target(0.2, 0.0, 0.0, kPoseTimeS);
        print_error_info("base_pose", robot.RequestTarget(target));
        continue;
    }

    if (command == "base_twist") {
        if (ensure_controller(robot, G1ControllerName::CHASSIS_TWIST_CTRL) !=
ControlStatus::SUCCESS) {
            continue;
        }
        const auto target = build_chassis_twist_target(0.05, 0.0, 0.0, kTwistCommandTimeS);
        run_twist_scene(robot, "base_twist", target, kTwistDurationS);
        continue;
    }

    if (command == "mixed_pose") {
        if (ensure_controller(robot, G1ControllerName::CHASSIS_POSE_CTRL) !=
ControlStatus::SUCCESS) {
            continue;
        }
        const auto target = merge_targets({
            build_joint_target(G1JointGroup::HEAD, head_single_joint, {0.15}, kJointTimeS),
            build_chassis_pose_target(0.1, 0.0, 0.0, kPoseTimeS),
        });
        print_error_info("mixed_pose", robot.RequestTarget(target));
        continue;
    }

    if (command == "mixed_twist") {
        if (ensure_controller(robot, G1ControllerName::CHASSIS_TWIST_CTRL) !=
ControlStatus::SUCCESS) {
            continue;
        }
        const auto target = merge_targets({
            build_joint_target(G1JointGroup::HEAD, head_single_joint, {-0.15}, kJointTimeS),
            build_chassis_twist_target(0.05, 0.0, 0.0, kTwistCommandTimeS),
        });
        run_twist_scene(robot, "mixed_twist", target, kTwistDurationS);
        continue;
    }

    std::cout << "Unknown command: " << command << std::endl;
    print_menu();
}

robot.request_shutdown();
robot.wait_for_shutdown();
robot.destroy();
return 0;
}

```

停止底盘运动 (stop_base)

适用场景：立即停止底盘的所有运动，紧急停止时使用。


```

examples/cpp/galbot_robot/src/stop_base_example.cpp
#include <iostream>
#include <vector>
#include <chrono>
#include <thread>
#include <string>

#include "galbot_robot.hpp"

using namespace galbot::sdk;

int main() {
    // Get object instance
    auto& robot = GalbotRobot::get_instance(MachineType::G1);

    // Initialize system
    if (robot.init()) {
        std::cout << "System initialized successfully!" << std::endl;
    } else {
        std::cerr << "System initialization failed!" << std::endl;
        return -1;
    }

    // Program started, waiting for data
    std::this_thread::sleep_for(std::chrono::milliseconds(2000));

    // Stop chassis motion
    while (true) {
        ControlStatus status = robot.stop_base();
        if (status == ControlStatus::SUCCESS) {
            std::cout << "Chassis motion has been stopped successfully!" << std::endl;
            break;
        } else {
            std::cerr << "Chassis stop motion failed, retrying..." << std::endl;
        }
    }

    // Exit system and release SDK resources
    robot.request_shutdown();
    robot.wait_for_shutdown();
    robot.destroy();

    return 0;
}

```

设置底盘速度 (set_base_velocity)

适用场景：控制移动底盘的线速度和角速度，用于导航或远程遥控。

```

examples/cpp/galbot_robot/src/set_base_velocity_example.cpp
#include <iostream>
#include <vector>
#include <chrono>
#include <thread>
#include <string>

#include "galbot_robot.hpp"

using namespace galbot::sdk;

```

```

int main() {
    // Get object instance
    auto& robot = GalbotRobot::get_instance(MachineType::G1);

    // Initialize system
    if (robot.init()) {
        std::cout << "System initialized successfully!" << std::endl;
    } else {
        std::cerr << "System initialization failed!" << std::endl;
        return -1;
    }

    // Program started, waiting for data
    std::this_thread::sleep_for(std::chrono::milliseconds(2000));

    // Please confirm the surrounding environment before chassis testing
    // Set chassis speed, linear_velocity first two fields are x and y velocities,
    angular_velocity third field is z rotation speed
    std::array<double, 3> linear_velocity = {0.05, 0.0, 0.0};    // 0.05 m/s
    std::array<double, 3> angular_velocity = {0.0, 0.0, 0.1};    // 0.1 rad/s
    double duration_s = 2.0; // Automatically stop after 2 seconds

    if (robot.set_base_velocity(linear_velocity, angular_velocity, duration_s) ==
        ControlStatus::SUCCESS) {
        std::cout << "Chassis speed set successfully; will stop in " << duration_s << "
seconds then auto-stop." << std::endl;
    } else {
        std::cerr << "Set chassis speed failed." << std::endl;
    }

    // Wait for auto-stop to complete (with small buffer time)
    std::this_thread::sleep_for(std::chrono::duration<double>(duration_s + 0.5));

    // Exit system and release SDK resources
    robot.request_shutdown();
    robot.wait_for_shutdown();
    robot.destroy();

    return 0;
}

```

获取关节状态 (get_joint_states)

适用场景：获取所有关节的完整状态信息，包括位置、速度、电流等。适用于需要完整关节信息的算法（如动力学计算、状态估计）。

```

examples/cpp/galbot_robot/src/get_joint_states_example.cpp

#include <iostream>
#include <vector>
#include <chrono>
#include <thread>
#include <string>

#include "galbot_robot.hpp"

using namespace galbot::sdk;

void print_joint_states(const std::vector<JointState>& joint_states) {

```

```

    for (const auto& states : joint_states) {
        std::cout << "--- Joint State ---" << std::endl;
        std::cout << "Position:      " << states.position      << " rad" << std::endl;
        std::cout << "Velocity:      " << states.velocity      << " rad/s" << std::endl;
        std::cout << "Acceleration: " << states.acceleration << " rad/s^2" << std::endl;
        std::cout << "Effort:        " << states.effort        << " Nm" << std::endl;
        std::cout << "Current:       " << states.current       << " A" << std::endl;
        std::cout << "-----" << std::endl;
    }
}

int main() {
    // Get object instance
    auto& robot = GalbotRobot::get_instance(MachineType::G1);

    // Initialize system
    if (robot.init()) {
        std::cout << "System initialized successfully!" << std::endl;
    } else {
        std::cerr << "System initialization failed!" << std::endl;
        return -1;
    }

    // Program started, waiting for data
    std::this_thread::sleep_for(std::chrono::milliseconds(2000));

    // Get joint states by joint group names; returns all joints if empty
    std::vector<std::string> joint_groups = {"left_arm"};
    auto ret_states = robot.get_joint_states(joint_groups, {});
    print_joint_states(ret_states);

    // Get specified joint states; if provided, overrides joint group input
    std::vector<std::string> joint_names = {"left_arm_joint1", "left_arm_joint2"};
    ret_states = robot.get_joint_states(joint_groups, joint_names);
    print_joint_states(ret_states);

    // Exit system and release SDK resources
    robot.request_shutdown();
    robot.wait_for_shutdown();
    robot.destroy();

    return 0;
}

```

获取关节位置 (get_joint_positions)

适用场景：仅获取当前关节位置。当你只需要关节位置信息时使用此接口，比 `get_joint_states` 更轻量。

```

examples/cpp/galbot_robot/src/get_joint_positions_example.cpp

#include <iostream>
#include <vector>
#include <chrono>
#include <thread>
#include <string>

#include "galbot_robot.hpp"

using namespace galbot::sdk;

```

```

void print_joint_positions(const std::vector<double>& positions) {
    for (const auto& pos : positions) {
        std::cout << "joint positions is " << pos << std::endl;
    }
    std::cout << std::endl;
}

int main() {
    // Get object instance
    auto& robot = GalbotRobot::get_instance(MachineType::G1);

    // Initialize system
    if (robot.init()) {
        std::cout << "System initialized successfully!" << std::endl;
    } else {
        std::cerr << "System initialization failed!" << std::endl;
        return -1;
    }

    // Program started, waiting for data
    std::this_thread::sleep_for(std::chrono::milliseconds(2000));

    // Get joint positions by joint group names; returns all joints if empty
    std::vector<std::string> joint_groups = {"left_arm"};
    auto ret_positions = robot.get_joint_positions(joint_groups, {});
    std::cout << "Left arm joint positions:" << std::endl;
    print_joint_positions(ret_positions);

    // Get specified joint positions; if provided, overrides joint group input
    std::vector<std::string> joint_names = {"left_arm_joint1", "left_arm_joint2"};
    ret_positions = robot.get_joint_positions({joint_groups}, joint_names);
    std::cout << "Positions of left arm joint1 and joint2:" << std::endl;
    print_joint_positions(ret_positions);

    // Exit system and release SDK resources
    robot.request_shutdown();
    robot.wait_for_shutdown();
    robot.destroy();

    return 0;
}

```

获取关节名称 (get_joint_names)

适用场景：获取机器人所有关节的名称列表，用于迭代遍历关节或配置文件生成。

```

examples/cpp/galbot_robot/src/get_joint_names_example.cpp

#include <iostream>
#include <vector>
#include <chrono>
#include <thread>
#include <string>

#include "galbot_robot.hpp"

using namespace galbot::sdk;

int main() {

```

```

// Get object instance
auto& robot = GalbotRobot::get_instance(MachineType::G1);

// Initialize system
if (robot.init()) {
    std::cout << "System initialized successfully!" << std::endl;
} else {
    std::cerr << "System initialization failed!" << std::endl;
    return -1;
}

// Program started, waiting for data
std::this_thread::sleep_for(std::chrono::milliseconds(2000));

// Get specified joint names; joint groups include ["leg", "head", "left_arm",
"right_arm"]
std::vector<std::string> joint_groups = {"head"};
bool only_active_joint = true; // Get active joints
auto head_joint_names_vec =
    robot.get_joint_names(only_active_joint, joint_groups);
std::cout << "Head joint names:" << std::endl;
for (size_t i = 0; i < head_joint_names_vec.size(); ++i) {
    std::cout << i << ": " << head_joint_names_vec[i] << std::endl;
}

// Passing an empty array returns all joint group information by default
std::vector<std::string> null_vec = {};
auto all_joint_names_vec =
    robot.get_joint_names(only_active_joint, null_vec);
std::cout << "All joint names:" << std::endl;
for (size_t i = 0; i < all_joint_names_vec.size(); ++i) {
    std::cout << i << ": " << all_joint_names_vec[i] << std::endl;
}

// Exit system and release SDK resources
robot.request_shutdown();
robot.wait_for_shutdown();
robot.destroy();

return 0;
}

```

获取夹爪状态 (get_gripper_state)

适用场景：获取夹爪当前位置和状态，判断夹爪当前是打开还是闭合。

```

examples/cpp/galbot_robot/src/get_gripper_state_example.cpp

#include <iostream>
#include <vector>
#include <chrono>
#include <thread>
#include <string>

#include "galbot_robot.hpp"

using namespace galbot::sdk;

void print_gripper_state(std::shared_ptr<GripperState> gripper_state) {
    std::cout << "Timestamp (ns): " << gripper_state->timestamp_ns << std::endl;
}

```

```

        std::cout << " width " << gripper_state->width << " velocity " << gripper_state-
>velocity
                << " effort " << gripper_state->effort << " is moving "
                << gripper_state->is_moving << std::endl;
    }

int main() {
    // Get object instance
    auto& robot = GalbotRobot::get_instance(MachineType::G1);

    // Initialize system
    if (robot.init()) {
        std::cout << "System initialized successfully!" << std::endl;
    } else {
        std::cerr << "System initialization failed!" << std::endl;
        return -1;
    }

    // Program started, waiting for data
    std::this_thread::sleep_for(std::chrono::milliseconds(2000));

    // Get gripper state
    auto gripper_state_ptr = robot.get_gripper_state("left_gripper");

    if (gripper_state_ptr == nullptr) {
        std::cerr << "get gripper state error" << std::endl;
    } else {
        std::cout << "Left gripper state:" << std::endl;
        print_gripper_state(gripper_state_ptr);
    }

    // Exit system and release SDK resources
    robot.request_shutdown();
    robot.wait_for_shutdown();
    robot.destroy();

    return 0;
}

```

获取吸盘状态 (get_suction_cup_state)

适用场景：获取吸盘当前是否处于吸合状态，以及是否成功检测到物体。

```

examples/cpp/galbot_robot/src/get_suction_cup_state_example.cpp

#include <iostream>
#include <vector>
#include <chrono>
#include <thread>
#include <string>

#include "galbot_robot.hpp"

using namespace galbot::sdk;

void print_suction_cup_state(std::shared_ptr<SuctionCupState> suction_cup_state) {
    std::cout << "Timestamp (ns): " << suction_cup_state->timestamp_ns << std::endl;
    std::cout << "Activation: " << suction_cup_state->activation << std::endl;
    std::cout << "Pressure: " << suction_cup_state->pressure << " Pa" << std::endl;
}

```

```

        std::cout << "Action State: " << int(suction_cup_state->action_state) << std::endl;
    }

int main() {
    // Get object instance
    auto& robot = GalbotRobot::get_instance(MachineType::G1);

    // Initialize system
    if (robot.init()) {
        std::cout << "System initialized successfully!" << std::endl;
    } else {
        std::cerr << "System initialization failed!" << std::endl;
        return -1;
    }

    // Program started, waiting for data
    std::this_thread::sleep_for(std::chrono::milliseconds(2000));

    // Get suction cup state
    auto suction_cup_state_ptr = robot.get_suction_cup_state("right_suction_cup");

    if (suction_cup_state_ptr == nullptr) {
        std::cerr << "get suction cup state error" << std::endl;
    } else {
        std::cout << "Right suction cup status:" << std::endl;
        print_suction_cup_state(suction_cup_state_ptr);
    }

    // Exit system and release SDK resources
    robot.request_shutdown();
    robot.wait_for_shutdown();
    robot.destroy();

    return 0;
}

```

获取坐标变换 (get_transform)

适用场景：获取两个坐标系之间的变换关系（位姿），用于机械臂抓取、视觉定位等场景。

```

examples/cpp/galbot_robot/src/get_transform_example.cpp

#include <iostream>
#include <vector>
#include <chrono>
#include <thread>
#include <string>

#include "galbot_robot.hpp"

using namespace galbot::sdk;

void print_pose_vec(const std::vector<double> &pose_vec) {
    // Output pose_vec
    std::cout << "pose_vec = [";
    for (size_t i = 0; i < pose_vec.size(); ++i) {
        std::cout << pose_vec[i];
        if (i + 1 < pose_vec.size())
            std::cout << ", ";
    }
}

```

```

        std::cout << "]" << std::endl;
    }

int main() {
    // Get object instance
    auto& robot = GalbotRobot::get_instance(MachineType::G1);

    // Initialize system
    if (robot.init()) {
        std::cout << "System initialized successfully!" << std::endl;
    } else {
        std::cerr << "System initialization failed!" << std::endl;
        return -1;
    }

    // Program started, waiting for data
    std::this_thread::sleep_for(std::chrono::milliseconds(2000));

    // Get coordinate transform
    std::pair<std::vector<double>, int64_t> tf_ret = robot.get_transform("left_arm_link1",
"left_arm_link7", 0);

    if (tf_ret.first.empty()) {
        std::cout << "get_transform error" << std::endl;
    } else {
        std::cout << "tf_timestamp_ns: " << tf_ret.second << std::endl;
        print_pose_vec(tf_ret.first);
    }

    // Exit system and release SDK resources
    robot.request_shutdown();
    robot.wait_for_shutdown();
    robot.destroy();

    return 0;
}

```

获取 IMU 数据 (get_imu_data)

适用场景：获取机器人基座 IMU 的加速度、角速度和姿态信息，用于状态估计、运动分析。

```

examples/cpp/galbot_robot/src/get_imu_data_example.cpp

#include <iostream>
#include <vector>
#include <chrono>
#include <thread>
#include <string>

#include "galbot_robot.hpp"

using namespace galbot::sdk;

void print_imu_data(const std::shared_ptr<ImuData>& imu_data) {
    if (!imu_data) {
        std::cerr << "IMU data is empty" << std::endl;
        return;
    }

    std::cout << "Timestamp (ns): " << imu_data->timestamp_ns << std::endl;

```



```

std::cout << "Accelerometer: "
    << "x=" << imu_data->accel.x << ", "
    << "y=" << imu_data->accel.y << ", "
    << "z=" << imu_data->accel.z << std::endl;

std::cout << "Gyroscope: "
    << "x=" << imu_data->gyro.x << ", "
    << "y=" << imu_data->gyro.y << ", "
    << "z=" << imu_data->gyro.z << std::endl;

std::cout << "Magnetometer: "
    << "x=" << imu_data->magnet.x << ", "
    << "y=" << imu_data->magnet.y << ", "
    << "z=" << imu_data->magnet.z << std::endl;
}

int main() {
    // Get object instance
    auto& robot = GalbotRobot::get_instance(MachineType::G1);

    // Initialize system
    if (robot.init()) {
        std::cout << "System initialized successfully!" << std::endl;
    } else {
        std::cerr << "System initialization failed!" << std::endl;
        return -1;
    }

    // Program started, waiting for data
    std::this_thread::sleep_for(std::chrono::milliseconds(2000));

    // Get IMU data
    std::shared_ptr<ImuData> imu_data = robot.get_imu_data(SensorType::TORSO_IMU);
    if (imu_data) {
        std::cout << "IMU data retrieved successfully!" << std::endl;
        print_imu_data(imu_data);
    } else {
        std::cerr << "Failed to get IMU data!" << std::endl;
    }

    // Exit system and release SDK resources
    robot.request_shutdown();
    robot.wait_for_shutdown();
    robot.destroy();

    return 0;
}

```

获取电池信息 (get_bms_information)

适用场景：获取电池电压、电量等信息，用于低电量检测和状态监控。

```

examples/cpp/galbot_robot/src/get_bms_information_example.cpp

#include <iostream>
#include <chrono>
#include <thread>
#include <string>

```

```

#include "galbot_robot.hpp"

using namespace galbot::sdk;

void print_bms_information(const std::shared_ptr<BmsInfo>& bms_info) {
    if (!bms_info) {
        std::cerr << "BMS info is empty" << std::endl;
        return;
    }

    std::cout << "Voltage (V): " << bms_info->voltage << std::endl;
    std::cout << "Current (A): " << bms_info->current << std::endl;
    std::cout << "Battery level (%): " << bms_info->battery_level << std::endl;
    std::cout << "Temperature (C): " << bms_info->temperature << std::endl;
    std::cout << "Charging status: " << std::boolalpha << bms_info->charging_status
        << std::noboolalpha << std::endl;
    std::cout << "Health status: " << std::boolalpha << bms_info->health_status
        << std::noboolalpha << std::endl;
    std::cout << "Capacity (Ah): " << bms_info->capacity << std::endl;
}

int main() {
    // Get object instance
    auto& robot = GalbotRobot::get_instance(MachineType::G1);

    // Initialize system
    if (robot.init()) {
        std::cout << "System initialized successfully!" << std::endl;
    } else {
        std::cerr << "System initialization failed!" << std::endl;
        return -1;
    }

    // Program started, waiting for data
    std::this_thread::sleep_for(std::chrono::milliseconds(3000));

    // Get BMS information
    auto bms_info = robot.get_bms_information();
    if (bms_info) {
        std::cout << "BMS info retrieved successfully!" << std::endl;
        print_bms_information(bms_info);
    } else {
        std::cerr << "Failed to get BMS info!" << std::endl;
    }

    // Exit system and release SDK resources
    robot.request_shutdown();
    robot.wait_for_shutdown();
    robot.destroy();

    return 0;
}

```

获取设备信息 (get_device_information)

适用场景：获取机器人硬件版本、固件版本等设备信息，用于调试和兼容性检查。

examples/cpp/galbot_robot/src/get_device_information_example.cpp

```

#include <chrono>
#include <iostream>
#include <string>
#include <thread>
#include <vector>

#include "galbot_robot.hpp"

using namespace galbot::sdk;

void print_device_info(const std::shared_ptr<DeviceInfo>& device_info) {
    if (!device_info) {
        std::cerr << "Device information is empty" << std::endl;
        return;
    }

    std::cout << "Device information:" << std::endl;
    std::cout << "  Model: " << (device_info->model.empty() ? "N/A" : device_info->model) <<
std::endl;
    std::cout << "  Serial number: " << (device_info->serial_number.empty() ? "N/A" :
device_info->serial_number) << std::endl;
    std::cout << "  Firmware version: " << (device_info->firmware_version.empty() ? "N/A" :
device_info->firmware_version)
        << std::endl;
    std::cout << "  Hardware version: " << (device_info->hardware_version.empty() ? "N/A" :
device_info->hardware_version)
        << std::endl;
    std::cout << "  : " << (device_info->manufacturer.empty() ? "N/A" : device_info-
>manufacturer) << std::endl;
}

int main() {
    // Get object instance
    auto& robot = GalbotRobot::get_instance(MachineType::G1);

    // Initialize system
    if (robot.init()) {
        std::cout << "System initialized successfully!" << std::endl;
    } else {
        std::cerr << "System initialization failed!" << std::endl;
        return -1;
    }

    // Program started, waiting for data
    std::this_thread::sleep_for(std::chrono::milliseconds(1000));

    // Get device information
    std::shared_ptr<DeviceInfo> device_info = robot.get_device_information();
    if (device_info) {
        std::cout << "Device information retrieved successfully!" << std::endl;
        print_device_info(device_info);
    } else {
        std::cerr << "Failed to get device information!" << std::endl;
    }

    // Exit system and release SDK resources
    robot.request_shutdown();
    robot.wait_for_shutdown();
    robot.destroy();
}

```

```
    return 0;
}
```

获取相机图像数据 (get_rgb_data && get_depth_data) (get_camera_data)

适用场景：获取 RGB 图像和深度图像数据，用于视觉感知、目标检测、SLAM 等任务。

```
examples/cpp/galbot_robot/src/get_camera_data_example.cpp

#include <iostream>
#include <vector>
#include <memory>
#include <unordered_set>
#include <chrono>
#include <thread>

#include "galbot_robot.hpp"
#include "opencv2/opencv.hpp"

using namespace galbot::sdk;

void print_rgb_data(const std::shared_ptr<RgbData> &rgb_data) {
    if (rgb_data == nullptr) {
        std::cout << "rgb_data is nullptr" << std::endl;
        return;
    }

    std::cout << "Camera image timestamp: "
                << rgb_data->header.timestamp_ns
                << std::endl;
    std::cout << "format is " << rgb_data->format << std::endl;
    std::cout << "frame_id is " << rgb_data->header.frame_id << std::endl;
    std::cout << "data size is " << rgb_data->data.size() << std::endl;

    std::cout << "show image:";

    std::shared_ptr<cv::Mat> img = rgb_data->convert_to_cv2_mat();

    cv::imwrite("result_image.jpg", *img);

    std::cout << "Image saved to result_image.jpg" << std::endl;
}

void print_depth_data(const std::shared_ptr<DepthData> depth_data_ptr) {
    if (depth_data_ptr == nullptr) {
        std::cout << "depth_data_ptr is nullptr" << std::endl;
        return;
    }

    std::cout << "Camera image timestamp: "
                << depth_data_ptr->header.timestamp_ns
                << std::endl;
    std::cout << "format is " << depth_data_ptr->format << std::endl;
    std::cout << "frame_id is " << depth_data_ptr->header.frame_id << std::endl;
    std::cout << "data size is " << depth_data_ptr->data.size() << std::endl;

    std::shared_ptr<cv::Mat> img = depth_data_ptr->convert_to_cv2_mat();

    if (img && !img->empty()) {
        cv::Mat img_vis;
```

```

// Image information normalization
cv::normalize(*img, img_vis, 0, 255, cv::NORM_MINMAX, CV_8UC1);

// Pseudo-color enhancement
cv::Mat img_color;
cv::applyColorMap(img_vis, img_color, cv::COLORMAP_JET);

// save
cv::imwrite("check_raw_data.png", *img);
cv::imwrite("check_visual_view.jpg", img_color);

std::cout << "Image saved: \n"
            << "1. check_raw_data.png -> A fully black image containing real physical
depth\n"
            << "2. check_visual_view.jpg -> A colorized image where object contours are
visible" << std::endl;
    }
}

int main() {
    // Get object instance
    auto& robot = GalbotRobot::get_instance(MachineType::G1);

    // Initialize sensors; only cameras and LiDAR sensors passed during initialization can
retrieve data
    std::unordered_set<SensorType> sensor_types = {
        SensorType::HEAD_LEFT_CAMERA,    // Head left camera
        SensorType::LEFT_ARM_DEPTH_CAMERA, // Left arm depth camera
    };

    // Initialize system
    if (robot.init(sensor_types)) {
        std::cout << "System initialized successfully!" << std::endl;
    } else {
        std::cerr << "System initialization failed!" << std::endl;
        return -1;
    }

    // Wait for camera data ready
    std::this_thread::sleep_for(std::chrono::milliseconds(2000));

    // Get RGB image data
    std::shared_ptr<RgbData> rgb_data = robot.get_rgb_data(SensorType::HEAD_LEFT_CAMERA);
    if (rgb_data) {
        std::cout << "RGB image data retrieved successfully!" << std::endl;
        print_rgb_data(rgb_data);
    } else {
        std::cerr << "Failed to get RGB image data!" << std::endl;
    }

    // Get depth image data
    std::shared_ptr<DepthData> depth_data =
robot.get_depth_data(SensorType::LEFT_ARM_DEPTH_CAMERA);
    if (depth_data) {
        std::cout << "Depth image data retrieved successfully!" << std::endl;
        print_depth_data(depth_data);
    } else {
        std::cerr << "Failed to get depth image data!" << std::endl;
    }
}

```

```

    // Exit system and release SDK resources
    robot.request_shutdown();
    robot.wait_for_shutdown();
    robot.destroy();

    return 0;
}

```

获取传感器内外参 (get_camera_intrinsic && get_sensor_extrinsic)

适用场景：获取相机内参和传感器相对于机器人基座的外参，用于计算机视觉计算和 3D 重建。

```

examples/cpp/galbot_robot/src/get_camera_params.cpp

#include <iostream>
#include <vector>
#include <memory>
#include <unordered_set>
#include <chrono>
#include <thread>

#include "galbot_robot.hpp"
#include "opencv2/opencv.hpp"

using namespace galbot::sdk;

void print_rgb_data(
    const std::shared_ptr<RgbData> &rgb_data) {
    if (rgb_data == nullptr) {
        std::cout << "rgb_data is nullptr" << std::endl;
        return;
    }

    std::cout << "Camera image timestamp: "
                << rgb_data->header.timestamp_ns
                << std::endl;
    std::cout << "format is " << rgb_data->format << std::endl;
    std::cout << "frame_id is " << rgb_data->header.frame_id << std::endl;
    std::cout << "data size is " << rgb_data->data.size() << std::endl;

    std::cout << "show image:";

    std::shared_ptr<cv::Mat> img = rgb_data->convert_to_cv2_mat();

    cv::imwrite("result_image.jpg", *img);

    std::cout << "Image saved to result_image.jpg" << std::endl;
}

void print_depth_data(
    const std::shared_ptr<DepthData>
        depth_data_ptr) {
    if (depth_data_ptr == nullptr) {
        std::cout << "depth_data_ptr is nullptr" << std::endl;
        return;
    }

    std::cout << "Camera image timestamp: "
                << depth_data_ptr->header.timestamp_ns
                << std::endl;

```

```

std::cout << "format is " << depth_data_ptr->format << std::endl;
std::cout << "frame_id is " << depth_data_ptr->header.frame_id << std::endl;
std::cout << "data size is " << depth_data_ptr->data.size() << std::endl;

std::shared_ptr<cv::Mat> img = depth_data_ptr->convert_to_cv2_mat();

if (img && !img->empty()) {
    cv::Mat img_vis;

    // Image information normalization
    cv::normalize(*img, img_vis, 0, 255, cv::NORM_MINMAX, CV_8UC1);

    // Pseudo-color enhancement
    cv::Mat img_color;
    cv::applyColorMap(img_vis, img_color, cv::COLORMAP_JET);

    // save
    cv::imwrite("check_raw_data.png", *img);
    cv::imwrite("check_visual_view.jpg", img_color);

    std::cout << "Image saved: \n"
                << "1. check_raw_data.png -> A fully black image containing real physical
depth\n"
                << "2. check_visual_view.jpg -> A colorized image where object contours are
visible" << std::endl;
    }
}

void print_camera_info(const std::shared_ptr<CameraInfo>& camerainfo){
    if (camerainfo == nullptr) {
        std::cout << "camerainfo is nullptr" << std::endl;
        return;
    }

    std::cout << "camera info:" << std::endl;
    std::cout << "header {" << std::endl;
    std::cout << "  timestamp_ns: " << camerainfo->header.timestamp_ns << std::endl;
    std::cout << "  frame_id: " << camerainfo->header.frame_id << std::endl;
    std::cout << "}" << std::endl;
    std::cout << "width: " << camerainfo->width << std::endl;
    std::cout << "height: " << camerainfo->height << std::endl;
    std::cout << "distortion_model: " << camerainfo->distortion_model << std::endl;
    std::cout << "d: ";
    for (auto& d_i : camerainfo->d) {
        std::cout << d_i << " ";
    }
    std::cout << std::endl;
    std::cout << "k: ";
    for (auto& k_i : camerainfo->k) {
        std::cout << k_i << " ";
    }
    std::cout << std::endl;
    std::cout << "r: ";
    for (auto& r_i : camerainfo->r) {
        std::cout << r_i << " ";
    }
    std::cout << std::endl;
    std::cout << "p: ";
    for (auto& p_i : camerainfo->p) {
        std::cout << p_i << " ";
    }
}

```

```

    }
    std::cout << std::endl;
    std::cout << "T: ";
    for (auto& t_i : camerainfo->T) {
        std::cout << t_i << " ";
    }
    std::cout << std::endl;
    std::cout << "roi {" << std::endl;
    std::cout << "  width: " << camerainfo->roi.width << std::endl;
    std::cout << "  height: " << camerainfo->roi.height << std::endl;
    std::cout << "}" << std::endl;
    std::cout << "camera_type: " << camerainfo->camera_type << std::endl;
}

int main() {
    // Get object instance
    auto& robot = GalbotRobot::get_instance(MachineType::G1);

    // Initialize sensors; only cameras and LiDAR sensors passed during initialization can
    retrieve data
    std::unordered_set<SensorType> sensor_types = {
        SensorType::HEAD_LEFT_CAMERA,      // Head left camera
        SensorType::LEFT_ARM_DEPTH_CAMERA,  // Left arm depth camera
    };

    // Initialize system
    if (robot.init(sensor_types)) {
        std::cout << "System initialized successfully!" << std::endl;
    } else {
        std::cerr << "System initialization failed!" << std::endl;
        return -1;
    }
    // Wait for camera data ready
    std::this_thread::sleep_for(std::chrono::milliseconds(2000));

    // Get RGB image data
    std::shared_ptr<RgbData> rgb_data = robot.get_rgb_data(SensorType::HEAD_LEFT_CAMERA);
    if (rgb_data) {
        std::cout << "RGB image data retrieved successfully!" << std::endl;
        print_rgb_data(rgb_data);
    } else {
        std::cerr << "Failed to get RGB image data!" << std::endl;
    }

    // Get camera parameters
    std::shared_ptr<CameraInfo> rgb_camerainfo =
robot.get_camera_intrinsic(SensorType::LEFT_ARM_DEPTH_CAMERA);
    if (rgb_camerainfo) {
        std::cout << "Camera parameters retrieved successfully!" << std::endl;
        print_camera_info(rgb_camerainfo);
    } else {
        std::cerr << "Failed to get camera parameters!" << std::endl;
    }

    // Get depth image data
    std::shared_ptr<DepthData> depth_data =
robot.get_depth_data(SensorType::LEFT_ARM_DEPTH_CAMERA);
    if (depth_data) {
        std::cout << "Depth image data retrieved successfully!" << std::endl;
        print_depth_data(depth_data);
    }
}

```



```

    } else {
        std::cerr << "Failed to get depth image data!" << std::endl;
    }

    // Get sensor extrinsics
    std::pair<std::vector<double>, int64_t> sensor_extrinsic =
robot.get_sensor_extrinsic(SensorType::LEFT_ARM_DEPTH_CAMERA);
    if (!sensor_extrinsic.first.empty()) {
        std::cout << "Sensor extrinsics retrieved successfully!" << std::endl;
        std::cout << "transform: ";
        for (auto& t_i : sensor_extrinsic.first) {
            std::cout << t_i << " ";
        }
        std::cout << std::endl;
        std::cout << "timestamp: " << sensor_extrinsic.second << std::endl;
    } else {
        std::cerr << "Failed to get sensor extrinsics!" << std::endl;
    }

    // Exit system and release SDK resources
    robot.request_shutdown();
    robot.wait_for_shutdown();
    robot.destroy();

    return 0;
}

```

获取雷达数据 (get_lidar_data)

适用场景：获取激光雷达点云数据，用于导航、避障、环境建模。

```

examples/cpp/galbot_robot/src/get_lidar_data_example.cpp
#include <iostream>
#include <vector>
#include <string>
#include <fstream>
#include <cmath>
#include <chrono>
#include <thread>
#include <memory>
#include <unordered_set>

#include "galbot_robot.hpp"

using namespace galbot::sdk;

// Define point structure
struct Point3D {
    float x, y, z;
};

/**
 * Directly extract the XYZ array from LidarData via pointer operations
 */
std::vector<Point3D> get_xyz_points(const std::shared_ptr<LidarData>& cloud, bool
remove_nan = false) {
    std::vector<Point3D> points;
    if (!cloud || cloud->data.empty()) return points;

```

```

// 1. Find the offsets of x, y, z fields (offset)
// In the Python version, fields are indexed by field name; here we precompute offsets
for better performance
int32_t off_x = -1, off_y = -1, off_z = -1;
for (const auto& f : cloud->fields) {
    if (f.name == "x") off_x = f.offset;
    else if (f.name == "y") off_y = f.offset;
    else if (f.name == "z") off_z = f.offset;
}

if (off_x == -1 || off_y == -1 || off_z == -1) {
    std::cerr << "Error: point cloud data is missing required xyz fields" << std::endl;
    return points;
}

uint32_t num_points = cloud->width * cloud->height;
points.reserve(num_points);

const uint8_t* raw_data = cloud->data.data();
uint32_t point_step = cloud->point_step;

// 2. Read directly via pointer (core zero-copy logic)
for (uint32_t i = 0; i < num_points; ++i) {
    // Calculate starting pointer for current point
    const uint8_t* pt_ptr = raw_data + (i * point_step);

    // Use reinterpret_cast to directly cast pointer types and read memory
    // Assume LiDAR data is float32 (F), which is the most common format
    float x = *reinterpret_cast<const float*>(pt_ptr + off_x);
    float y = *reinterpret_cast<const float*>(pt_ptr + off_y);
    float z = *reinterpret_cast<const float*>(pt_ptr + off_z);

    // Handle NaN values (corresponds to Python remove_nan logic)
    if (remove_nan) {
        if (std::isnan(x) || std::isnan(y) || std::isnan(z)) {
            continue;
        }
    }

    points.push_back({x, y, z});
}

return points;
}

/**
 * Save to a PCD file
 */
void save_xyz_to_pcd(const std::vector<Point3D>& points, const std::string& filename) {
    std::ofstream fs(filename);
    if (!fs.is_open()) {
        std::cerr << "Unable to open file for writing: " << filename << std::endl;
        return;
    }

    // PCD 0.7 Header
    fs << "# .PCD v0.7 - Point Cloud Data file format\n"
        << "VERSION 0.7\n"
        << "FIELDS x y z\n"
        << "SIZE 4 4 4\n"

```

```

        << "TYPE F F F\n"
        << "COUNT 1 1 1\n"
        << "WIDTH " << points.size() << "\n"
        << "HEIGHT 1\n"
        << "VIEWPOINT 0 0 0 1 0 0 0\n"
        << "POINTS " << points.size() << "\n"
        << "DATA ascii\n";

    // Data
    for (const auto& p : points) {
        fs << p.x << " " << p.y << " " << p.z << "\n";
    }

    fs.close();
    std::cout << "Saved " << points.size() << " points to " << filename << std::endl;
}

int main() {
    // Get object instance
    auto& robot = GalbotRobot::get_instance(MachineType::G1);

    // Initialize sensors; only cameras and LiDAR sensors passed during initialization can
    retrieve data
    std::unordered_set<SensorType> sensor_types = {
        SensorType::BASE_LIDAR          // Chassis lidar
    };

    // Initialize system
    if (robot.init(sensor_types)) {
        std::cout << "System initialized successfully!" << std::endl;
    } else {
        std::cerr << "System initialization failed!" << std::endl;
        return -1;
    }

    // Program started, waiting for data
    std::this_thread::sleep_for(std::chrono::milliseconds(2000));

    // Get LiDAR data
    std::shared_ptr<LidarData> lidar_data = robot.get_lidar_data(SensorType::BASE_LIDAR);
    if (lidar_data) {
        std::cout << "Lidar data retrieved successfully!" << std::endl;
        std::vector<Point3D> xyz_points = get_xyz_points(lidar_data, false);
        if (!xyz_points.empty()) {
            save_xyz_to_pcd(xyz_points, "output_xyz.pcd");
        }
    } else {
        std::cerr << "Failed to get lidar data!" << std::endl;
    }

    // Exit system and release SDK resources
    robot.request_shutdown();
    robot.wait_for_shutdown();
    robot.destroy();

    return 0;
}

```

一键回零-odom 坐标系 (whole_body_reset_zero_odom)

适用场景：快速将全身上下所有关节运动到零位（初始姿态），基于 odom 坐标系。通常用于程序启动后的初始化。

```
examples/cpp/galbot_robot/src/whole_body_reset_zero_odom_example.cpp
#include <iostream>

#include "galbot_motion.hpp"
#include "galbot_robot.hpp"

using namespace galbot::sdk;

namespace {
const char* control_status_to_string(ControlStatus status) {
    switch (status) {
        case ControlStatus::SUCCESS:
            return "SUCCESS";
        case ControlStatus::TIMEOUT:
            return "TIMEOUT";
        case ControlStatus::FAULT:
            return "FAULT";
        case ControlStatus::INVALID_INPUT:
            return "INVALID_INPUT";
        case ControlStatus::INIT_FAILED:
            return "INIT_FAILED";
        case ControlStatus::IN_PROGRESS:
            return "IN_PROGRESS";
        case ControlStatus::STOPPED_UNREACHED:
            return "STOPPED_UNREACHED";
        case ControlStatus::DATA_FETCH_FAILED:
            return "DATA_FETCH_FAILED";
        case ControlStatus::PUBLISH_FAIL:
            return "PUBLISH_FAIL";
        case ControlStatus::COMM_DISCONNECTED:
            return "COMM_DISCONNECTED";
        default:
            return "UNKNOWN_STATUS";
    }
}
} // namespace

int main() {
    auto& robot = GalbotRobot::get_instance(MachineType::G1);
    auto& motion = GalbotMotion::get_instance(MachineType::G1);

    if (!robot.init()) {
        std::cerr << "GalbotRobot init failed." << std::endl;
        return -1;
    }
    if (!motion.init()) {
        std::cerr << "GalbotMotion init failed." << std::endl;
        return -1;
    }

    // Optional frames (frame_id: base_link/odom/map, reference_frame_id: odom/map)
    const std::string frame_id = "base_link";
    const std::string reference_frame_id = "odom";
```

```

// reset to zero
auto result = robot.zero_whole_body_and_base(
    frame_id,
    reference_frame_id,
    /*is_blocking=*/true,
    /*leg_head_speed_rad_s=*/0.2,
    /*leg_head_timeout_s=*/15.0,
    /*params=*/default_param);

std::cout << "Zero joint status: " << motion.status_to_string(result.first) <<
std::endl;
std::cout << "Zero base status: " << control_status_to_string(result.second) <<
std::endl;

robot.request_shutdown();
robot.wait_for_shutdown();
robot.destroy();

return 0;
}

```

一键回零-map 坐标系 (whole_body_reset_zero_map)

适用场景：快速将全身上下所有关节运动到零位（初始姿态），基于 map 坐标系。

examples/cpp/galbot_robot/src/whole_body_reset_zero_map_example.cpp

```

#include <iostream>

#include "galbot_motion.hpp"
#include "galbot_robot.hpp"

using namespace galbot::sdk;

namespace {
const char* control_status_to_string(ControlStatus status) {
    switch (status) {
        case ControlStatus::SUCCESS:
            return "SUCCESS";
        case ControlStatus::TIMEOUT:
            return "TIMEOUT";
        case ControlStatus::FAULT:
            return "FAULT";
        case ControlStatus::INVALID_INPUT:
            return "INVALID_INPUT";
        case ControlStatus::INIT_FAILED:
            return "INIT_FAILED";
        case ControlStatus::IN_PROGRESS:
            return "IN_PROGRESS";
        case ControlStatus::STOPPED_UNREACHED:
            return "STOPPED_UNREACHED";
        case ControlStatus::DATA_FETCH_FAILED:
            return "DATA_FETCH_FAILED";
        case ControlStatus::PUBLISH_FAIL:
            return "PUBLISH_FAIL";
        case ControlStatus::COMM_DISCONNECTED:
            return "COMM_DISCONNECTED";
        default:
            return "UNKNOWN_STATUS";
    }
}

```

```

    }
}
} // namespace

int main() {
    auto& robot = GalbotRobot::get_instance(MachineType::G1);
    auto& motion = GalbotMotion::get_instance(MachineType::G1);

    if (!robot.init()) {
        std::cerr << "GalbotRobot init failed." << std::endl;
        return -1;
    }
    if (!motion.init()) {
        std::cerr << "GalbotMotion init failed." << std::endl;
        return -1;
    }

    // Optional frames (frame_id: base_link/odom/map, reference_frame_id: odom/map)
    const std::string frame_id = "base_link";
    const std::string reference_frame_id = "map";

    // reset to zero
    auto result = robot.zero_whole_body_and_base(
        frame_id,
        reference_frame_id,
        /*is_blocking=*/true,
        /*leg_head_speed_rad_s=*/0.2,
        /*leg_head_timeout_s=*/15.0,
        /*params=*/default_param);

    std::cout << "Zero joint status: " << motion.status_to_string(result.first) <<
std::endl;
    std::cout << "Zero base status: " << control_status_to_string(result.second) <<
std::endl;

    robot.request_shutdown();
    robot.wait_for_shutdown();
    robot.destroy();

    return 0;
}

```

类：GalbotMotion

获取实例并初始化 (get_instance && init)

适用场景：获取运动规划模块单例并初始化。必须在使用运动规划功能前调用。

```

examples/cpp/galbot_motion/src/get_instance_init_example.cpp
#include <iostream>
#include <set>
#include <string>

#include "galbot_motion.hpp"
#include "galbot_robot.hpp"

using namespace galbot::sdk;

```

```

int main() {

    auto& planner = GalbotMotion::get_instance(MachineType::G1);
    auto& robot = GalbotRobot::get_instance(MachineType::G1);

    if (planner.init()) {
        std::cout << "Planner initialized successfully!" << std::endl;
    } else {
        std::cerr << "Planner initialization failed!" << std::endl;
        return -1;
    }

    if (robot.init()) {
        std::cout << "System initialized successfully!" << std::endl;
    } else {
        std::cerr << "System initialization failed!" << std::endl;
        return -1;
    }

    // You can still manage the robot lifecycle through GalbotRobot
    robot.request_shutdown();
    robot.wait_for_shutdown();
    robot.destroy();

    return 0;
}

```

正运动学（使用当前状态或指定 RobotStates）（forward_kinematics）

适用场景：根据给定的关节角度计算末端执行器的位姿。用于机械臂任务验证和状态分析。

```

examples/cpp/galbot_motion/src/forward_kinematics_example.cpp

#include <algorithm>
#include <chrono>
#include <iostream>
#include <map>
#include <string>
#include <thread>
#include <tuple>
#include <vector>

#include "galbot_motion.hpp"
#include "galbot_robot.hpp"

using namespace galbot::sdk;

// Helper print function
void print_pose(const std::string& label, const std::tuple<MotionStatus,
std::vector<double>>& res,
               GalbotMotion& planner) {
    std::cout << "[" << label << "]" Status: " << planner.status_to_string(std::get<0>(res))
<< std::endl;

    if (std::get<0>(res) == MotionStatus::SUCCESS) {
        std::cout << "End-effector pose: ";
        for (double v : std::get<1>(res)) {
            std::cout << v << " ";
        }
        std::cout << "\n" << std::endl;
    }
}

```

```

    } else {
        std::cout << "Calculation failed!" << std::endl;
    }
}

int main() {
    auto& planner = GalbotMotion::get_instance(MachineType::G1);
    auto& robot = GalbotRobot::get_instance(MachineType::G1);

    if (planner.init()) {
        std::cout << "Planner initialized successfully!" << std::endl;
    } else {
        std::cerr << "Planner initialization failed!" << std::endl;
        return -1;
    }

    if (robot.init()) {
        std::cout << "System initialized successfully!" << std::endl;
    } else {
        std::cerr << "System initialization failed!" << std::endl;
        return -1;
    }

    // Program started, waiting for data
    std::this_thread::sleep_for(std::chrono::milliseconds(3000));

    std::map<std::string, std::vector<double>> chain_joints = {
        {"leg", {0.4992, 1.4991, 1.0005, 0.0000}},
        {"head", {0.0000, 0.0}},
        {"left_arm", {1.9999, -1.6000, -0.5999, -1.6999, 0.0000, -0.7999, 0.0000}},
        {"right_arm", {-2.0000, 1.6001, 0.6001, 1.7000, 0.0000, 0.8000, 0.0000}}};

    std::vector<double> whole_body_joint;
    std::vector<std::string> keys = {"leg", "head", "left_arm", "right_arm"};
    for (const auto& key : keys) {
        whole_body_joint.insert(whole_body_joint.end(), chain_joints[key].begin(),
chain_joints[key].end());
    }

    std::vector<double> base_state = {0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 1.0};
    std::string end_link = "left_arm_end_effector_mount_link";
    std::string reference_frame = "base_link";

    // --- test 1: defaultparameters (current status) ---
    try {
        std::cout << ">> Executing: Basic forward kinematics..." << std::endl;
        auto res1 = planner.forward_kinematics(end_link, reference_frame);
        print_pose("Basic version", res1, planner);
    } catch (const std::exception& e) {
        std::cerr << "✗ Basic version exception: " << e.what() << std::endl;
    }

    // --- test 2: jointstatus + parameters ---
    try {
        std::cout << ">> Executing: Forward kinematics with custom joints..." << std::endl;

        std::unordered_map<std::string, std::vector<double>> custom_joint_state = {"left_arm",
chain_joints["left_arm"]}};
        auto custom_param_ptr = std::make_shared<Parameter>();
        auto res2 = planner.forward_kinematics(end_link, reference_frame, custom_joint_state,

```



```

custom_param_ptr);

    print_pose("Custom parameters", res2, planner);
} catch (const std::exception& e) {
    std::cerr << "✘ Custom-parameter exception: " << e.what() << std::endl;
}

// --- test 3: RobotStates forward kinematics(current status, planning)---
try {
    std::cout << ">> Executing: Forward kinematics based on RobotStates..." << std::endl;

    RobotStates current_state = planner.get_robot_states();
    if (current_state.whole_body_joint.empty()) {
        std::cerr << "✘ RobotStates-based: Unable to get current body joint states; ensure
WBC/sensors are ready." << std::endl;
    } else {
        auto ref_robot_state_ptr = std::make_shared<RobotStates>(std::move(current_state));
        auto res3 = planner.forward_kinematics_by_state(end_link, ref_robot_state_ptr,
reference_frame,

                                                                std::make_shared<Parameter>());
        print_pose("Based on RobotStates", res3, planner);
    }
} catch (const std::exception& e) {
    std::cerr << "✘ RobotStates-based exception: " << e.what() << std::endl;
}

robot.request_shutdown();
robot.wait_for_shutdown();
robot.destroy();

return 0;
}

```

获取连杆名称 (get_link_names)

适用场景：获取机器人模型中所有连杆的名称列表，用于碰撞检测和运动规划调试。

```

examples/cpp/galbot_motion/src/get_link_names_example.cpp

#include <chrono>
#include <iostream>
#include <string>
#include <thread>
#include <vector>

#include "galbot_motion.hpp"
#include "galbot_robot.hpp"

using namespace galbot::sdk;

void print_link_names(const std::vector<std::string>& link_names, const std::string& title)
{
    std::cout << title << " (total " << link_names.size() << " items):" << std::endl;
    for (size_t i = 0; i < link_names.size(); ++i) {
        std::cout << "   " << (i + 1) << ". " << link_names[i] << std::endl;
    }
}

int main() {
    // Get object instance

```

```

auto& motion = GalbotMotion::get_instance(MachineType::G1);
auto& robot = GalbotRobot::get_instance(MachineType::G1);

// Initialize system
if (!motion.init()) {
    std::cerr << "GalbotMotion initialization failed!" << std::endl;
    return -1;
}
if (!robot.init()) {
    std::cerr << "GalbotRobot initialization failed!" << std::endl;
    return -1;
}

std::cout << "System initialized successfully!" << std::endl;

// Program started, waiting for data
std::this_thread::sleep_for(std::chrono::milliseconds(2000));

try {
    // Get all link names
    std::vector<std::string> all_link_names = motion.get_link_names(false);
    print_link_names(all_link_names, "\nAll link names");

    // getend effectorexecute link
    std::vector<std::string> ee_link_names = motion.get_link_names(true);
    print_link_names(ee_link_names, "\nEnd-effector link names");
} catch (const std::exception& e) {
    std::cerr << "Get link name exception: " << e.what() << std::endl;
}

// Exit system and release SDK resources
robot.request_shutdown();
robot.wait_for_shutdown();
robot.destroy();

return 0;
}

```

逆运动学（基础与基于 RobotStates）(inverse_kinematics)

适用场景：根据期望的末端位姿求解所需的关节角度。是机械臂抓取等操作的核心步骤。

```

examples/cpp/galbot_motion/src/inverse_kinematics_example.cpp

#include <algorithm>
#include <chrono>
#include <iostream>
#include <memory>
#include <stdexcept>
#include <string>
#include <thread>
#include <tuple>
#include <unordered_map>
#include <vector>

#include "galbot_motion.hpp"
#include "galbot_robot.hpp"

using namespace galbot::sdk;

```

```

// Helper function: print inverse kinematics result
void print_ik_result(const std::string& label,
                    const std::tuple<MotionStatus, std::unordered_map<std::string,
std::vector<double>>>& res,
                    GalbotMotion& planner) {
    auto status = std::get<0>(res);
    auto joint_map = std::get<1>(res);

    std::cout << "[" << label << "]" Status feedback: " << planner.status_to_string(status)
<< std::endl;

    if (status == MotionStatus::SUCCESS) {
        std::cout << "✓ IK computation succeeded! Joint angles obtained:" << std::endl;
        for (const auto& [name, joints] : joint_map) {
            std::cout << "  - Chain [" << name << "]: ";
            for (double v : joints)
                std::cout << v << " ";
            std::cout << std::endl;
        }
    } else {
        std::cout << "✗ inverse kinematics failed, checkinput targetpose." << std::endl;
    }
    std::cout << "-----" << std::endl;
}

int main() {
    auto& planner = GalbotMotion::get_instance(MachineType::G1);
    auto& robot = GalbotRobot::get_instance(MachineType::G1);

    if (planner.init()) {
        std::cout << "Planner initialized successfully!" << std::endl;
    } else {
        std::cerr << "Planner initialization failed!" << std::endl;
        return -1;
    }

    if (robot.init()) {
        std::cout << "System initialized successfully!" << std::endl;
    } else {
        std::cerr << "System initialization failed!" << std::endl;
        return -1;
    }

    std::this_thread::sleep_for(std::chrono::milliseconds(1000));

    // Joint state definition
    std::unordered_map<std::string, std::vector<double>> chain_joints = {
        {"leg", {0.4992, 1.4991, 1.0005, 0.0000, -0.0004}},
        {"head", {0.0000, 0.0}},
        {"left_arm", {1.9999, -1.6000, -0.5999, -1.6999, 0.0000, -0.7999, 0.0000}},
        {"right_arm", {-2.0000, 1.6001, 0.6001, 1.7000, 0.0000, 0.8000, 0.0000}};

    // Target pose definition (x, y, z, qx, qy, qz, qw)
    std::unordered_map<std::string, std::vector<double>> chain_pose_baselink = {
        {"left_arm", {0.1267, 0.2342, 0.7356, 0.0220, 0.0127, 0.0343, 0.9991}},
        {"right_arm", {0.1267, -0.2345, 0.7358, -0.0225, 0.0126, -0.0343, 0.9991}};

    // Whole-body joint vector concatenation (Leg -> Head -> Left Arm -> Right Arm)
    std::vector<double> whole_body_joint;
    std::vector<std::string> key_order = {"leg", "head", "left_arm", "right_arm"};

```

```

    for (const auto& key : key_order) {
        whole_body_joint.insert(whole_body_joint.end(), chain_joints[key].begin(),
chain_joints[key].end());
    }

    // General configuration
    std::string reference_frame = "base_link";
    std::string target_frame = "EndEffector";
    std::string target_chain = "left_arm";
    auto params = std::make_shared<Parameter>();

    // Scenario 1: Single-chain inverse kinematics
    try {
        std::cout << ">> Running Scenario 1: Single-chain IK test..." << std::endl;
        std::vector<std::string> one_chain = {target_chain};

        auto res = planner.inverse_kinematics(chain_pose_baselink[target_chain], // 1.
target_pose
                                                one_chain // 2.
chain_names
                                                // target_frame, // 3.
target_frame
                                                // reference_frame, // 4.
reference_frame
                                                // {}, // 5.
initial_joint_positions (empty)
                                                // false, // 6.
enable_collision_check (bool)
                                                // params // 7.
params (shared_ptr)
        );
        print_ik_result("Single-chain inverse kinematics", res, planner);
        std::this_thread::sleep_for(std::chrono::milliseconds(800));
    } catch (const std::exception& e) {
        std::cerr << "Scenario 1 exception: " << e.what() << std::endl;
    }

    // Scenario 2: Arm chain + torso inverse kinematics
    try {
        std::cout << ">> Running Scenario 2: Arm chain + torso IK test..." << std::endl;
        std::vector<std::string> chain_with_torso = {target_chain, "torso"};

        auto res = planner.inverse_kinematics(chain_pose_baselink[target_chain],
chain_with_torso, target_frame,
                                                reference_frame, {}, false, params);
        print_ik_result("Arm + torso inverse kinematics", res, planner);
        std::this_thread::sleep_for(std::chrono::milliseconds(800));
    } catch (const std::exception& e) {
        std::cerr << "Scenario 2 exception: " << e.what() << std::endl;
    }

    // Scenario 3: invalid chain combination
    try {
        std::cout << ">> Running Scenario 3: Invalid chain-combination test..." << std::endl;
        std::vector<std::string> error_chains = {target_chain, "torso", "head"};

        auto res = planner.inverse_kinematics(chain_pose_baselink[target_chain], error_chains,
target_frame,
                                                reference_frame, {}, false, params);
        print_ik_result("Invalid chain combination detection", res, planner);
    }

```

```

    } catch (const std::exception& e) {
        std::cerr << "Scenario 3 exception: " << e.what() << std::endl;
    }

    // Scenario 4: use reference joints (initial_joint_positions can specify chain joints as
    // IK references; unspecified chain joints are filled with whole-body joints)
    try {
        std::cout << ">> Running Scenario 4: IK test with initial reference values..." <<
std::endl;
        std::vector<std::string> one_chain = {target_chain};

        auto res = planner.inverse_kinematics(chain_pose_baselink[target_chain], one_chain,
target_frame, reference_frame,
                                chain_joints, false, params);
        print_ik_result("Inverse kinematics with reference values", res, planner);
        std::this_thread::sleep_for(std::chrono::milliseconds(800));
    } catch (const std::exception& e) {
        std::cerr << "Scenario 4 exception: " << e.what() << std::endl;
    }

    // scenario 5: RobotStates inverse kinematics
    try {
        std::cout << ">> Running Scenario 5: IK test based on RobotStates..." << std::endl;

        // Construct RobotStates smart pointer
        auto ref_state = std::make_shared<RobotStates>();
        ref_state->chain_name = target_chain;
        ref_state->whole_body_joint = whole_body_joint;
        ref_state->base_state = {0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 1.0};

        std::vector<std::string> one_chain = {target_chain};

        auto res = planner.inverse_kinematics_by_state(chain_pose_baselink[target_chain], //
1. target_pose
                                one_chain, //
2. chain_names
                                target_frame, //
3. target_frame
                                reference_frame, //
4. reference_frame
                                ref_state, // 5.
reference_robot_states (shared_ptr)
                                false, // 6.
enable_collision_check (bool)
                                params // 7. params (shared_ptr)
        );
        print_ik_result("RobotStates inverse kinematics", res, planner);
    } catch (const std::exception& e) {
        std::cerr << "Scenario 5 exception: " << e.what() << std::endl;
    }

    robot.request_shutdown();
    robot.wait_for_shutdown();
    robot.destroy();
    std::cout << "Resources released." << std::endl;

    return 0;
}

```

获取与设置末端位姿(get_set_end_effort_pos)

适用场景：直接获取当前末端位姿，或将末端移动到指定位姿。集成了逆运动学计算和执行。

examples/cpp/galbot_motion/src/get_set_end_effort_pos_example.cpp

```
#include <iostream>
#include <vector>
#include <string>
#include <unordered_map>
#include <thread>
#include <chrono>
#include <tuple>
#include <memory>
#include <stdexcept>

#include "galbot_motion.hpp"
#include "galbot_robot.hpp"

using namespace galbot::sdk;

// Helper function: print pose information
void print_pose_info(const std::string& label, const std::vector<double>& pose) {
    if (pose.size() == 7) {
        std::cout << "[" << label << "]" Pose: "
            << "pos(" << pose[0] << ", " << pose[1] << ", " << pose[2] << ")," << "
            << "ori(" << pose[3] << ", " << pose[4] << ", " << pose[5] << ", " <<
pose[6] << ")"
            << std::endl;
    }
}

int main() {

    auto& planner = GalbotMotion::get_instance(MachineType::G1);
    auto& robot = GalbotRobot::get_instance(MachineType::G1);

    if (!planner.init()) {
        std::cerr << "GalbotMotion initialization failed" << std::endl;
        return -1;
    }
    if (!robot.init()) {
        std::cerr << "GalbotRobot initialization failed" << std::endl;
        return -1;
    }

    std::this_thread::sleep_for(std::chrono::milliseconds(1000));

    std::unordered_map<std::string, std::vector<double>> chain_pose_baselink = {
        {"leg", {0.0596, -0.0000, 1.0327, 0.5000, 0.5003, 0.4997, 0.5000}},
        {"head", {0.0599, 0.0002, 1.4098, -0.7072, 0.0037, 0.0037, 0.7069}},
        {"left_arm", {0.1267, 0.2342, 0.7356, 0.0220, 0.0127, 0.0343, 0.9991}},
        {"right_arm", {0.097768, -0.226021, 0.8, -0.0117403, -0.0098713, 0.0157502,
0.999758}}
    };

    std::string reference_frame = "base_link";
    std::string target_frame = "EndEffector";
    std::string target_chain = "right_arm";
    auto custom_param = std::make_shared<Parameter>();
```

```

// --- Scenario 1: Get end-effector pose (basic version) ---
try {
    std::cout << ">> Scenario 1: Getting the basic end-effector pose..." << std::endl;
    std::string end_ee_link = "right_arm_end_effector_mount_link";

    auto res = planner.get_end_effector_pose(end_ee_link, reference_frame);

    MotionStatus status = std::get<0>(res);
    std::vector<double> pose = std::get<1>(res);

    std::cout << "Execution status: " << planner.status_to_string(status) << std::endl;
    if (status == MotionStatus::SUCCESS) {
        print_pose_info("Basic version", pose);
    }
    std::this_thread::sleep_for(std::chrono::milliseconds(800));
} catch (const std::exception& e) {
    std::cerr << "✗ Scenario 1 exception: " << e.what() << std::endl;
}

// --- Scenario 2: Get end-effector pose by specified chain name + custom frame ---
try {
    std::cout << ">> Scenario 2: Getting pose by specified chain name..." << std::endl;

    auto res = planner.get_end_effector_pose_on_chain(target_chain, target_frame,
reference_frame);

    MotionStatus status = std::get<0>(res);
    std::vector<double> pose = std::get<1>(res);

    std::cout << "Execution status: " << planner.status_to_string(status) << std::endl;
    if (status == MotionStatus::SUCCESS) {
        print_pose_info("Specified chain name version", pose);
    }
    std::this_thread::sleep_for(std::chrono::milliseconds(800));
} catch (const std::exception& e) {
    std::cerr << "✗ Scenario 2 exception: " << e.what() << std::endl;
}

// --- Scenario 3: Set end-effector pose ---
try {
    std::cout << ">> Scenario 3: Setting end-effector pose..." << std::endl;

    std::string ee_frame = "right_arm";
    std::vector<double> target_pose = chain_pose_baselink[ee_frame];

    MotionStatus status = planner.set_end_effector_pose(
        target_pose,          // 1
        ee_frame,             // 2
        reference_frame,      // 3
        nullptr,              // 4. Important: pass nullptr if no specific reference
state is used
        false,                // 5. enable_collision_check
        true,                 // 6. is_blocking
        5.0,                  // 7. timeout
        custom_param          // 8. params
    );

    std::cout << "Set status: " << planner.status_to_string(status) << std::endl;
    if (status == MotionStatus::SUCCESS) {

```

```

        std::cout << "✔ Command sent successfully (blocking wait mode)" << std::endl;
    }
} catch (const std::exception& e) {
    std::cerr << "✘ Pose-setting exception: " << e.what() << std::endl;
}

// --- Scenario 4: Get end-effector pose again after execution ---
try {
    std::cout << ">> Scenario 4: Getting the basic end-effector pose..." << std::endl;
    std::string end_ee_link = "right_arm_end_effector_mount_link";

    auto res = planner.get_end_effector_pose(end_ee_link, reference_frame);

    MotionStatus status = std::get<0>(res);
    std::vector<double> pose = std::get<1>(res);

    std::cout << "Execution status: " << planner.status_to_string(status) << std::endl;
    if (status == MotionStatus::SUCCESS) {
        print_pose_info("Basic version", pose);
    }
    std::this_thread::sleep_for(std::chrono::milliseconds(800));
} catch (const std::exception& e) {
    std::cerr << "✘ Scenario 4 exception: " << e.what() << std::endl;
}

robot.request_shutdown();
robot.wait_for_shutdown();
robot.destroy();

return 0;
}

```

单点运动规划（关节空间与笛卡尔空间）（single_point_planning）

适用场景：规划从当前位姿到单个目标位姿的无碰撞运动轨迹。适用于单点目标到达任务。

```

examples/cpp/galbot_motion/src/single_point_planning_example.cpp

#include <iostream>
#include <vector>
#include <string>
#include <unordered_map>
#include <thread>
#include <chrono>
#include <tuple>
#include <memory>
#include <stdexcept>

#include "galbot_motion.hpp"
#include "galbot_robot.hpp"

using namespace galbot::sdk;

// Define a trajectory return type to simplify the code
using TrajResult = std::tuple<MotionStatus, std::unordered_map<std::string,
std::vector<std::vector<double>>>>;

/**
 * Helper function: print planning result
 */

```



```

void print_plan_info(const std::string& label, const TrajResult& res, const std::string&
chain_name, GalbotMotion& planner) {
    auto status = std::get<0>(res);
    auto traj_map = std::get<1>(res);

    std::cout << "[" << label << "]" Status: " << planner.status_to_string(status) <<
std::endl;
    if (status == MotionStatus::SUCCESS) {
        if (traj_map.count(chain_name) && !traj_map[chain_name].empty()) {
            std::cout << "✓ Planning succeeded: trajectory points = " <<
traj_map[chain_name].size() << std::endl;
        } else {
            std::cout << " Status is SUCCESS but trajectory is empty (possibly already at
target)" << std::endl;
        }
    } else {
        std::cout << "✗ Planning failed" << std::endl;
    }
    std::cout << "-----" << std::endl;
}

int main() {

    auto& planner = GalbotMotion::get_instance(MachineType::G1);
    auto& robot = GalbotRobot::get_instance(MachineType::G1);

    if (!planner.init()) {
        std::cerr << "GalbotMotion init FAILED" << std::endl;
        return -1;
    }
    if (!robot.init()) {
        std::cerr << "GalbotRobot init FAILED" << std::endl;
        return -1;
    }

    std::this_thread::sleep_for(std::chrono::milliseconds(1000));

    std::unordered_map<std::string, std::vector<double>> chain_joints = {
        {"leg",          {0.4992, 1.4991, 1.0005, 0.0000, -0.0004}},
        {"head",         {0.0000, 0.0}},
        {"left_arm",     {1.9999, -1.6000, -0.5999, -1.6999, 0.0000, -0.7999, 0.0000}},
        {"right_arm",    {-2.0000, 1.6001, 0.6001, 1.7000, 0.0000, 0.8000, 0.0000}}
    };

    std::unordered_map<std::string, std::vector<double>> chain_pose_baselink = {
        {"leg",          {0.0596, -0.0000, 1.0327, 0.5000, 0.5003, 0.4997, 0.5000}},
        {"head",         {0.0599, 0.0002, 1.4098, -0.7072, 0.0037, 0.0037, 0.7069}},
        {"left_arm",     {0.1267, 0.2342, 0.7356, 0.0220, 0.0127, 0.0343, 0.9991}},
        {"right_arm",    {0.1267, -0.2345, 0.7358, -0.0225, 0.0126, -0.0343, 0.9991}}
    };

    auto params = std::make_shared<Parameter>();
    std::string target_chain = "left_arm";

    // NOTE:
    // - GalbotMotion does NOT provide real-time obstacle perception / automatic
environment updates today.
    // - When enable_collision_check=true, collision checking uses self-collision +
objects you load manually via
    //   add_obstacle/attach_target_object (including point clouds if you load them

```

explicitly).

```
// Scenario 1: joint-space planning, target type = joint state
try {
    std::cout << ">> Scenario 1: joint-space planning (joint target)..." << std::endl;

    // Construct target joint state
    auto target_joint = std::make_shared<JointStates>();
    target_joint->chain_name = target_chain;
    target_joint->joint_positions = chain_joints[target_chain];

    auto res = planner.motion_plan(
        target_joint,    // 1. target
        nullptr,        // 2. start (nullptr means start from current state)
        nullptr,        // 3. reference_robot_states
        false,          // 4. enable_collision_check
        params           // 5. params
    );

    print_plan_info("Joint-space planning (joint target)", res, target_chain, planner);
} catch (const std::exception& e) { std::cerr << e.what() << std::endl; }

// Scenario 2: joint-space planning, target type = end-effector pose (Cartesian)
try {
    std::cout << ">> Scenario 2: joint-space planning (pose target)..." << std::endl;

    // Construct target pose state
    auto target_pose = std::make_shared<PoseState>();
    target_pose->chain_name = target_chain;
    target_pose->frame_id = "EndEffector";
    target_pose->reference_frame = "base_link";
    target_pose->pose = chain_pose_baselink[target_chain];

    auto res = planner.motion_plan(
        target_pose,    // 1. target
        nullptr,        // 2. start
        nullptr,        // 3. reference_robot_states
        false,          // 4. enable_collision_check
        params           // 5. params
    );

    print_plan_info("Joint-space planning (pose target)", res, target_chain, planner);
    std::this_thread::sleep_for(std::chrono::milliseconds(800));
} catch (const std::exception& e) { std::cerr << e.what() << std::endl; }

// Scenario 3: joint-space planning with an explicit start state
try {
    std::cout << ">> Scenario 3: joint-space planning (explicit start)..." <<
std::endl;

    auto target_joint = std::make_shared<JointStates>();
    target_joint->chain_name = target_chain;
    target_joint->joint_positions = chain_joints[target_chain];

    auto start_joint = std::make_shared<JointStates>();
    start_joint->chain_name = target_chain;
    start_joint->joint_positions = std::vector<double>(7, 0.0); // Use seven zeros as
the starting point

    auto res = planner.motion_plan(
```

```

        target_joint,
        start_joint,    // 2. Specify the start point
        nullptr,
        false,
        params
    );

    print_plan_info("Joint-space planning (explicit start)", res, target_chain,
planner);
} catch (const std::exception& e) { std::cerr << e.what() << std::endl; }

robot.request_shutdown();
robot.wait_for_shutdown();
robot.destroy();
return 0;
}

```

多点轨迹规划(multi_point_planning)

适用场景：规划经过多个路径点的连续运动轨迹，适用于需要经过多个中间点的复杂运动。

```

examples/cpp/galbot_motion/src/multi_point_planning_example.cpp
#include <iostream>
#include <vector>
#include <string>
#include <unordered_map>
#include <thread>
#include <chrono>
#include <tuple>
#include <memory>
#include <stdexcept>
#include <algorithm>

#include "galbot_motion.hpp"
#include "galbot_robot.hpp"

using namespace galbot::sdk;

// Trajectory return type definition
using TrajResult = std::tuple<MotionStatus, std::unordered_map<std::string,
std::vector<std::vector<double>>>>;

/**
 * Helper function: print planning result
 */
void print_multi_plan_result(const std::string& label, const TrajResult& res, const
std::string& chain_name, GalbotMotion& planner) {
    auto status = std::get<0>(res);
    auto traj_map = std::get<1>(res);

    std::cout << "[" << label << "]" Status feedback: " << planner.status_to_string(status)
<< std::endl;
    if (status == MotionStatus::SUCCESS) {
        if (traj_map.count(chain_name) && !traj_map[chain_name].empty()) {
            std::cout << "✔ Multi-point planning succeeded: total trajectory points = "
<< traj_map[chain_name].size() << std::endl;
        } else {
            std::cout << " Status is SUCCESS but trajectory is empty; target may overlap
current pose." << std::endl;

```

```

    }
} else {
    std::cout << "✗ Multi-point planning failed." << std::endl;
}
std::cout << "-----" << std::endl;
}

int main() {
    auto& planner = GalbotMotion::get_instance(MachineType::G1);
    auto& robot = GalbotRobot::get_instance(MachineType::G1);

    if (!planner.init()) {
        std::cerr << "GalbotMotion initialization failed" << std::endl;
        return -1;
    }
    if (!robot.init()) {
        std::cerr << "GalbotRobot initialization failed" << std::endl;
        return -1;
    }

    std::this_thread::sleep_for(std::chrono::seconds(2));

    std::unordered_map<std::string, std::vector<double>> chain_joints = {
        {"leg", {0.4992, 1.4991, 1.0005, 0.0000, -0.0004}},
        {"head", {0.0000, 0.0}},
        {"left_arm", {1.9999, -1.6000, -0.5999, -1.6999, 0.0000, -0.7999, 0.0000}},
        {"right_arm", {-2.0000, 1.6001, 0.6001, 1.7000, 0.0000, 0.8000, 0.0000}}
    };

    std::vector<double> whole_body_joint;
    std::vector<std::string> keys = {"leg", "head", "left_arm", "right_arm"};
    for (const auto& key : keys) {
        whole_body_joint.insert(whole_body_joint.end(), chain_joints[key].begin(),
chain_joints[key].end());
    }

    auto params = std::make_shared<Parameter>();
    std::string target_chain = "left_arm";

    // --- Scenario 1: Multi-waypoint planning in Cartesian space (PoseState target) ---
    try {
        std::cout << ">> Running Scenario 1: Multi-waypoint planning in Cartesian
space..." << std::endl;

        auto target_pose_state = std::make_shared<PoseState>();
        target_pose_state->chain_name = target_chain;

        // Construct waypoints (3 intermediate poses)
        std::vector<std::vector<double>> waypoint_poses = {
            {0.1267, 0.2342, 0.7356, 0.0220, 0.0127, 0.0343, 0.9991},
            {0.2267, 0.2342, 0.7356, 0.0220, 0.0127, 0.0343, 0.9991},
            {0.3267, 0.2342, 0.7356, 0.0220, 0.0127, 0.0343, 0.9991},
            {0.4267, 0.2342, 0.7356, 0.0220, 0.0127, 0.0343, 0.9991}
        };

        auto res = planner.motion_plan_multi_waypoints(
            target_pose_state,
            waypoint_poses,
            nullptr, // start
            nullptr, // reference_robot_states

```

```

        false,    // enable_collision_check
        params    // params
    );

    print_multi_plan_result("Cartesian multi-waypoint single-chain planning", res,
target_chain, planner);
    std::this_thread::sleep_for(std::chrono::milliseconds(800));
    } catch (const std::exception& e) { std::cerr << "Scenario 1 exception: " << e.what()
<< std::endl; }

    // --- Scenario 2: Multi-waypoint planning in joint space (JointStates target) ---
    try {
        std::cout << ">> Running Scenario 2: Multi-waypoint planning in joint space..." <<
std::endl;

        auto target_joint = std::make_shared<JointStates>();
        target_joint->chain_name = target_chain;

        // Construct waypoints (3 intermediate poses)
        std::vector<std::vector<double>> waypoints = {
            {0.1267, 0.2342, 0.7356, 0.0220, 0.0127, 0.0343, 0.9991},
            {0.2267, 0.4342, 0.7356, 0.0220, 0.0127, 0.0343, 0.9991},
            {0.3267, 0.6342, 0.7356, 0.0220, 0.0127, 0.0343, 0.9991},
            {0.4267, 0.8342, 0.7356, 0.0220, 0.0127, 0.0343, 0.9991}
        };

        auto res = planner.motion_plan_multi_waypoints(
            target_joint,
            waypoints,
            nullptr,
            nullptr,
            false,
            params
        );

        print_multi_plan_result("Joint-space multi-waypoint", res, target_chain, planner);
    } catch (const std::exception& e) { std::cerr << "Scenario 2 exception: " << e.what()
<< std::endl; }

    // 4. Clean up resources
    robot.request_shutdown();
    robot.wait_for_shutdown();
    robot.destroy();
    return 0;
}

```

碰撞检测(collision_detection)

适用场景：检查给定关节状态是否会发生自碰撞或与环境障碍物碰撞。用于运动规划前的可行性检查。

```

examples/cpp/galbot_motion/src/collision_detection_example.cpp
#include <iostream>
#include <vector>
#include <string>
#include <unordered_map>
#include <thread>
#include <chrono>
#include <tuple>

```

```

#include <memory>
#include <stdexcept>
#include <algorithm>

#include "galbot_motion.hpp"
#include "galbot_robot.hpp"
#include "galbot_navigation.hpp"

using namespace galbot::sdk;

int main() {

    auto& planner = GalbotMotion::get_instance(MachineType::G1);
    auto& robot = GalbotRobot::get_instance(MachineType::G1);
    auto& navigation = GalbotNavigation::get_instance(MachineType::G1);

    // NOTE:
    // - GalbotNavigation (galbotNav) may use real-time obstacle perception/avoidance
    // during navigation (deployment dependent).
    // - GalbotMotion does NOT provide real-time obstacle perception today; Motion
    // collision uses self-collision +
    // manually loaded obstacles (add_obstacle/attach_target_object).

    if (!planner.init()) {
        std::cerr << "GalbotMotion init FAILED" << std::endl;
        return -1;
    }
    if (!robot.init()) {
        std::cerr << "GalbotRobot init FAILED" << std::endl;
        return -1;
    }
    if (!navigation.init()) {
        std::cerr << "GalbotNavigation init FAILED" << std::endl;
        return -1;
    }

    std::unordered_map<std::string, std::vector<double>> chain_joints = {
        {"leg", {0.4992, 1.4991, 1.0005, 0.0000, -0.0004}},
        {"head", {0.0000, 0.0}},
        {"left_arm", {1.9999, -1.6000, -0.5999, -1.6999, 0.0000, -0.7999, 0.0000}},
        {"right_arm", {-2.0000, 1.6001, 0.6001, 1.7000, 0.0000, 0.8000, 0.0000}}
    };

    std::vector<double> whole_body_joint;
    std::vector<std::string> keys = {"leg", "head", "left_arm", "right_arm"};
    for (const auto& key : keys) {
        whole_body_joint.insert(whole_body_joint.end(), chain_joints[key].begin(),
chain_joints[key].end());
    }

    std::vector<double> base_state = {0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 1.0};
    std::vector<double> bad_left_arm_joint = {1.99995, -1.60004, 0.599905, -1.69994, 0, -
0.799924, 0};
    auto custom_param = std::make_shared<Parameter>();

    try {
        std::cout << ">> Running collision check..." << std::endl;

        // Construct a RobotStates list for collision checking
        std::vector<std::shared_ptr<RobotStates>> check_states;
    }
}

```

```

// status 0: status (RobotStates)
auto state0 = std::make_shared<RobotStates>();
state0->whole_body_joint = whole_body_joint;
state0->base_state = base_state;
check_states.push_back(state0);

// status 1: status (JointStates RobotStates)
auto state1 = std::make_shared<JointStates>();
state1->chain_name = "left_arm";
state1->joint_positions = bad_left_arm_joint;
check_states.push_back(state1);

// Call collision detection interface
auto res = planner.check_collision(check_states, true, custom_param);

MotionStatus status = std::get<0>(res);
std::vector<bool> collision_results = std::get<1>(res);

std::cout << "Status: " << planner.status_to_string(status) << std::endl;

if (status == MotionStatus::SUCCESS) {
    std::cout << "OK: collision check finished (false=no collision,
true=collision):" << std::endl;
    for (size_t i = 0; i < collision_results.size(); ++i) {
        std::cout << "  - status [" << i << "]: "
            << (collision_results[i] ? "COLLISION" : "NO COLLISION")
            << std::endl;
    }
} else {
    std::cerr << "ERROR: collision check returned failure." << std::endl;
}

} catch (const std::exception& e) {
    std::cerr << "ERROR: collision check exception: " << e.what() << std::endl;
}

robot.request_shutdown();
robot.wait_for_shutdown();
robot.destroy();

return 0;
}

```

附加工具(attach_tool)

适用场景：在机器人末端添加工具（如夹具、吸盘等），更新运动规划模型，以便考虑工具质量和碰撞体积。

```

examples/cpp/galbot_motion/src/attach_tool_example.cpp

#include <iostream>
#include <string>
#include <thread>
#include <chrono>
#include <stdexcept>

#include "galbot_motion.hpp"
#include "galbot_robot.hpp"

```

```

using namespace galbot::sdk;

int main() {

    auto& planner = GalbotMotion::get_instance(MachineType::G1);
    auto& robot = GalbotRobot::get_instance(MachineType::G1);

    if (!planner.init()) {
        std::cerr << "GalbotMotion initialization failed" << std::endl;
        return -1;
    }
    if (!robot.init()) {
        std::cerr << "GalbotRobot initialization failed" << std::endl;
        return -1;
    }

    // Program started, waiting for data
    std::this_thread::sleep_for(std::chrono::seconds(2));

    // --- Execute tool-attach operation ---
    try {
        std::string chain_name = "left_arm";
        std::string tool_name = "suction_cup";

        MotionStatus status = planner.attach_tool(chain_name, tool_name);

        std::cout << "Execution status feedback: " << planner.status_to_string(status) <<
std::endl;

        if (status == MotionStatus::SUCCESS) {
            std::cout << "✔ Tool attached successfully: " << tool_name << std::endl;
        } else {
            std::cerr << "✘ Tool attachment failed. Please check whether the tool name is
defined in the configuration file." << std::endl;
        }

    } catch (const std::exception& e) {
        std::cerr << "✘ Exception occurred during runtime: " << e.what() << std::endl;
    }

    robot.request_shutdown();
    robot.wait_for_shutdown();
    robot.destroy();
    return 0;
}

```

卸载工具(detach_tool)

适用场景：从机器人末端移除已附加的工具，恢复原始机器人模型。

examples/cpp/galbot_motion/src/detach_tool_example.cpp

```

#include <iostream>
#include <string>
#include <thread>
#include <chrono>
#include <stdexcept>

#include "galbot_motion.hpp"
#include "galbot_robot.hpp"

```



```

using namespace galbot::sdk;

int main() {

    auto& planner = GalbotMotion::get_instance(MachineType::G1);
    auto& robot = GalbotRobot::get_instance(MachineType::G1);

    if (!planner.init()) {
        std::cerr << "GalbotMotion initialization failed" << std::endl;
        return -1;
    }
    if (!robot.init()) {
        std::cerr << "GalbotRobot initialization failed" << std::endl;
        return -1;
    }

    // Program started, waiting for data
    std::this_thread::sleep_for(std::chrono::seconds(2));

    // --- Execute tool-detach operation ---
    try {
        std::string chain_name = "left_arm";
        MotionStatus status = planner.detach_tool(chain_name);

        std::cout << "Execution status feedback: " << planner.status_to_string(status) <<
std::endl;

        if (status == MotionStatus::SUCCESS) {
            std::cout << "✓ Tool detached successfully." << std::endl;
        } else {
            std::cerr << "✗ Tool detachment failed." << std::endl;
        }

    } catch (const std::exception& e) {
        std::cerr << "✗ Exception occurred during runtime: " << e.what() << std::endl;
    }

    robot.request_shutdown();
    robot.wait_for_shutdown();
    robot.destroy();
    return 0;
}

```

加载/移除环境碰撞体(env_colider_operation)

适用场景：向运动规划环境中添加障碍物或移除已添加的障碍物，使运动规划考虑环境障碍。

```

examples/cpp/galbot_motion/src/env_colider_operation_example.cpp

#include <iostream>
#include <vector>
#include <string>
#include <array>
#include <thread>
#include <chrono>
#include <memory>
#include <stdexcept>

#include "galbot_motion.hpp"

```

```

#include "galbot_robot.hpp"

using namespace galbot::sdk;

/*
    NOTE:
    - GalbotMotion does NOT provide real-time obstacle perception / automatic environment
    updates today.
    - To make Motion collision checking consider environmental obstacles, you must load
    them manually
      (e.g., box/mesh/point_cloud via add_obstacle/attach_target_object).
    - Real-time perception / navigation-style obstacle updates in Motion is a planned
    future feature.
*/

int main() {
    auto& planner = GalbotMotion::get_instance(MachineType::G1);
    auto& robot = GalbotRobot::get_instance(MachineType::G1);

    if (!planner.init()) {
        std::cerr << "GalbotMotion init FAILED" << std::endl;
        return -1;
    }
    if (!robot.init()) {
        std::cerr << "GalbotRobot init FAILED" << std::endl;
        return -1;
    }

    std::this_thread::sleep_for(std::chrono::seconds(2));

    // Scenario 1: add a Box collision object into Motion environment.
    try {
        std::cout << ">> Scenario 1: add obstacle..." << std::endl;

        std::string obstacle_id = "box_test_1";
        std::string obj_type = "box";
        std::vector<double> obj_pose = {1.0, 0.0, 1.0, 0.0, 0.0, 0.0, 1.0};
        std::array<double, 3> obj_scale = {1.0, 1.0, 1.0};
        std::string target_frame = "world";

        MotionStatus status = planner.add_obstacle(
            obstacle_id,      // 1. ID
            obj_type,         // 2. Type
            obj_pose,         // 3. Pose
            obj_scale,        // 4. Scale (array)
            "",               // 5. key
            target_frame,     // 6. target_frame
            "",               // 7. ee_frame
            {},               // 8. ref_joints
            {},               // 9. ref_base
            {},               // 10. ignore_links
            0.0,              // 11. safe_margin
            0.0               // 12. resolution
        );

        std::cout << "Status: " << planner.status_to_string(status) << std::endl;

        planner.clear_obstacle();

        if (status == MotionStatus::SUCCESS) {

```

```

        std::cout << "OK: obstacle added" << std::endl;
    }
} catch (const std::exception& e) { std::cerr << e.what() << std::endl; }

// Scenario 2: add a duplicate ID (expected to fail).
try {
    std::cout << "\n>> Scenario 2: test duplicate ID..." << std::endl;

    std::string obstacle_id = "box_test_2";
    std::string obj_type = "box";
    std::vector<double> obj_pose = {1.0, 0.0, 1.0, 0.0, 0.0, 0.0, 1.0};
    std::array<double, 3> obj_scale = {1.0, 1.0, 1.0};
    std::string target_frame = "world";

    // First addition
    planner.add_obstacle(obstacle_id, obj_type, obj_pose, obj_scale, "", target_frame,
"", {}, {}, {}, 0.0, 0.0);

    // Second addition with same ID
    MotionStatus status = planner.add_obstacle(obstacle_id, obj_type, obj_pose,
obj_scale, "", target_frame, "", {}, {}, {}, 0.0, 0.0);

    std::cout << "Status: " << planner.status_to_string(status) << std::endl;

    planner.clear_obstacle();

    if (status == MotionStatus::FAULT) {
        std::cout << "OK: duplicate ID rejected (expected)" << std::endl;
    }
} catch (const std::exception& e) { std::cerr << e.what() << std::endl; }

robot.request_shutdown();
robot.wait_for_shutdown();
robot.destroy();
return 0;
}

```

类：GalbotNavigation

获取实例 / 初始化(get_instance_init)

适用场景：获取导航模块单例并初始化。必须在使用导航功能前调用。

```

examples/cpp/galbot_navigation/src/get_instance_init_example.cpp
#include "galbot_navigation.hpp"
#include "galbot_robot.hpp"

#include <iostream>
#include <string>
#include <vector>
#include <thread>
#include <thread>

using namespace galbot::sdk;

int main() {
    auto& navigation = GalbotNavigation::get_instance(MachineType::G1);
    auto& robot = GalbotRobot::get_instance(MachineType::G1);
}

```

```

// Initialize system
if (!robot.init()) {
    std::cerr << "Base instance initialization failed!" << std::endl;
    return -1;
}
if (!navigation.init()) {
    std::cerr << "Navigation instance initialization failed!" << std::endl;
    return -1;
}

std::cout << "Initialization successful!" << std::endl;

robot.request_shutdown();
robot.wait_for_shutdown();
robot.destroy();

return 0;
}

```

重定位 / 是否已定位 / 获取当前位姿(relocation)

适用场景：触发重定位，检查机器人是否已经成功定位，并获取机器人在地图中的当前位姿。

```

examples/cpp/galbot_navigation/src/relocation_example.cpp
#include "galbot_navigation.hpp"
#include "galbot_robot.hpp"

#include <iostream>
#include <string>
#include <vector>
#include <thread>
#include <thread>

using namespace galbot::sdk;

int main() {
    auto& navigation = GalbotNavigation::get_instance(MachineType::G1);
    auto& robot = GalbotRobot::get_instance(MachineType::G1);

    // Initialize system
    if (robot.init()) {
        std::cout << "Base instance initialized successfully!" << std::endl;
    } else {
        std::cerr << "Base instance initialization failed!" << std::endl;
        return -1;
    }
    if (navigation.init()) {
        std::cout << "Navigation instance initialized successfully!" << std::endl;
    } else {
        std::cerr << "Navigation instance initialization failed!" << std::endl;
        return -1;
    }

    Pose init_pose(std::vector<double>{0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 1.0});

    // checkrelocalize success
    int count_relocalize = 0;
    while (!navigation.is_localized() && count_relocalize < 20) {

```

```

        navigation.relocalize(init_pose);
        std::this_thread::sleep_for(std::chrono::milliseconds(500));
        std::cout << "is relocalizing" << std::endl;
        count_relocalize++;
    }

    if (navigation.is_localized()) {
        std::cout << "relocalization success." << std::endl;

        // Get current pose
        Pose current_pose = navigation.get_current_pose();
        std::cout << "Current pose: Position(" << current_pose.position.x << ", "
                    << current_pose.position.y << ", " << current_pose.position.z
                    << "), orientation(" << current_pose.orientation.x << ", "
                    << current_pose.orientation.y << ", " << current_pose.orientation.z
                    << ", " << current_pose.orientation.w << ")" << std::endl;

        robot.request_shutdown();
        robot.wait_for_shutdown();
    } else {
        std::cout << "relocalization failed, cannot proceed with navigation." << std::endl;
    }

    robot.destroy();

    return 0;
}

```

检查路径可达性并阻塞导航到目标(blocked_navigation)

适用场景：检查目标点是否可达，如果可达则阻塞等待导航完成到达目标。简单场景使用方便。

```

examples/cpp/galbot_navigation/src/blocked_navigation_example.cpp
#include "galbot_navigation.hpp"
#include "galbot_robot.hpp"

#include <iostream>
#include <string>
#include <vector>
#include <thread>
#include <thread>

using namespace galbot::sdk;

int main() {
    auto& navigation = GalbotNavigation::get_instance(MachineType::G1);
    auto& robot = GalbotRobot::get_instance(MachineType::G1);

    // Initialize system
    if (robot.init()) {
        std::cout << "Base instance initialized successfully!" << std::endl;
    } else {
        std::cerr << "Base instance initialization failed!" << std::endl;
        return -1;
    }

    if (navigation.init()) {
        std::cout << "Navigation instance initialized successfully!" << std::endl;
    } else {
        std::cerr << "Navigation instance initialization failed!" << std::endl;
    }
}

```

```

        return -1;
    }
    auto res = robot.switch_controller(G1ControllerName::CHASSIS_POSE_CTRL);
    if (res != ControlStatus::SUCCESS) {
        std::cerr << "Failed to switch controller!" << std::endl;
        return -1;
    }
    Pose init_pose(std::vector<double>{0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 1.0});
    Pose goal_pose(std::vector<double>{0.3, 0.0, 0.0, 0.0, 0.0, 0.0, 1.0});

    // checkrelocalize success
    int count_relocalize = 0;
    while (!navigation.is_localized() && count_relocalize < 20) {
        navigation.relocalize(init_pose);
        std::this_thread::sleep_for(std::chrono::milliseconds(500));
        std::cout << "is relocalizing" << std::endl;
        count_relocalize++;
    }

    if (navigation.is_localized()) {
        std::cout << "relocalization success." << std::endl;

        // Get current pose
        Pose current_pose = navigation.get_current_pose();
        std::cout << "Current pose: Position(" << current_pose.position.x << ", "
            << current_pose.position.y << ", " << current_pose.position.z
            << "), orientation(" << current_pose.orientation.x << ", "
            << current_pose.orientation.y << ", " << current_pose.orientation.z
            << ", " << current_pose.orientation.w << ")" << std::endl;

        std::this_thread::sleep_for(std::chrono::milliseconds(2000));

        // Whether to enable obstacle checking (can be set to true in open environments)
        bool enable_collision_check = false;
        // Whether to block and wait for arrival
        bool is_blocking = true;
        // Maximum wait time to reach position
        float timeout_s = 20;

        // Navigate 3 times in loop
        int count = 0;
        while (count++ < 3) {
            std::cout << "No. " << count << " navigation(s)" << std::endl;
            // checkpath navigation target
            if (navigation.check_path_reachability(goal_pose, init_pose)) {
                std::cout << "Path reachable, navigating to target point" << std::endl;
                NavigationStatus status = navigation.navigate_to_goal(
                    goal_pose, enable_collision_check, is_blocking, timeout_s);
                if (status == NavigationStatus::SUCCESS) {
                    std::cout << "Target point reached" << std::endl;
                } else {
                    std::cout << "navigationfailed, status: " << static_cast<int>(status)
<< std::endl;
                }
            } else {
                std::cout << "Path unreachable, cannot navigate to target point" <<
std::endl;
            }
            // Check path reachability and return to start point
            if (navigation.check_path_reachability(init_pose, goal_pose)) {

```

```

        std::cout << "Path reachable, navigating to start point" << std::endl;
        NavigationStatus status = navigation.navigate_to_goal(
            init_pose, enable_collision_check, is_blocking, timeout_s);
        if (status == NavigationStatus::SUCCESS) {
            std::cout << "Returned to start point" << std::endl;
        } else {
            std::cout << "navigationfailed, status: " << static_cast<int>(status)
<< std::endl;
        }
    } else {
        std::cout << "Path unreachable, cannot navigate to start point" <<
std::endl;
    }
}

// Stop navigation
navigation.stop_navigation();

// Get current pose again
current_pose = navigation.get_current_pose();
std::cout << "Current pose: Position(" << current_pose.position.x << ", "
    << current_pose.position.y << ", " << current_pose.position.z
    << "), orientation(" << current_pose.orientation.x << ", "
    << current_pose.orientation.y << ", " << current_pose.orientation.z
    << ", " << current_pose.orientation.w << ")" << std::endl;

} else {
    std::cout << "relocalization failed." << std::endl;
}

robot.request_shutdown();
robot.wait_for_shutdown();
robot.destroy();

return 0;
}

```

非阻塞导航 + 轮询判断到达(non_blocking_navigation)

适用场景：启动导航但不阻塞当前线程，可以在导航过程中做其他处理，需要轮询判断是否到达目标。适合需要异步处理的场景。

```

examples/cpp/galbot_navigation/src/non_blocking_navigation_example.cpp
#include "galbot_navigation.hpp"
#include "galbot_robot.hpp"

#include <iostream>
#include <string>
#include <vector>
#include <thread>
#include <thread>

using namespace galbot::sdk;

int main() {
    auto& navigation = GalbotNavigation::get_instance(MachineType::G1);
    auto& robot = GalbotRobot::get_instance(MachineType::G1);

```

```

// Initialize system
if (robot.init()) {
    std::cout << "Base instance initialized successfully!" << std::endl;
} else {
    std::cerr << "Base instance initialization failed!" << std::endl;
    return -1;
}

if (navigation.init()) {
    std::cout << "Navigation instance initialized successfully!" << std::endl;
} else {
    std::cerr << "Navigation instance initialization failed!" << std::endl;
    return -1;
}

auto res = robot.switch_controller(G1ControllerName::CHASSIS_POSE_CTRL);
if (res != ControlStatus::SUCCESS) {
    std::cerr << "Failed to switch controller!" << std::endl;
    return -1;
}

Pose init_pose(std::vector<double>{0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 1.0});
Pose goal_pose(std::vector<double>{0.3, 0.0, 0.0, 0.0, 0.0, 0.0, 1.0});

// checkrelocalize success
int count_relocalize = 0;
while (!navigation.is_localized() && count_relocalize < 20) {
    navigation.relocalize(init_pose);
    std::this_thread::sleep_for(std::chrono::milliseconds(5000));
    std::cout << "is relocalizing" << std::endl;
    count_relocalize++;
}
if (navigation.is_localized()) {
    std::cout << "relocalization success." << std::endl;

    // Get current pose
    Pose current_pose = navigation.get_current_pose();
    std::cout << "Current pose: Position(" << current_pose.position.x << ", "
        << current_pose.position.y << ", " << current_pose.position.z
        << "), orientation(" << current_pose.orientation.x << ", "
        << current_pose.orientation.y << ", " << current_pose.orientation.z
        << ", " << current_pose.orientation.w << ")" << std::endl;

    std::this_thread::sleep_for(std::chrono::milliseconds(2000));

    // Whether to enable obstacle checking (can be set to true in open environments)
    bool enable_collision_check = false;
    // Whether to block and wait for arrival
    bool is_blocking = false;

    // Navigate 2 times in loop, non-blocking wait for arrival
    int count = 0;
    while (count++ < 2) {
        std::cout << "No. " << count << " navigation(s)" << std::endl;
        // checkpath navigation target
        if (navigation.check_path_reachability(goal_pose, init_pose)) {
            std::cout << "Path reachable, navigating to target point" << std::endl;
            NavigationStatus status = navigation.navigate_to_goal(
                goal_pose, enable_collision_check, is_blocking);
            // wait
            int count_arrival = 0;

```



```

        while (!navigation.check_goal_arrival()) {
            std::cout << "navigate has not arrived" << std::endl;
            std::this_thread::sleep_for(std::chrono::milliseconds(500));
            if (++count_arrival > 10) {
                break;
            }
        }
        if (navigation.check_goal_arrival()) {
            std::cout << "Target point reached" << std::endl;
        } else {
            std::cout << "Navigation failed; target point not reached" <<
std::endl;
        }
    } else {
        std::cout << "Path unreachable, cannot navigate to target point" <<
std::endl;
    }
    // Check path reachability and return to start point
    if (navigation.check_path_reachability(init_pose, goal_pose)) {
        std::cout << "Path reachable, navigating to start point" << std::endl;
        NavigationStatus status = navigation.navigate_to_goal(
            init_pose, enable_collision_check, is_blocking);
        // wait
        int count_arrival = 0;
        while (!navigation.check_goal_arrival()) {
            std::cout << "navigate has not arrived" << std::endl;
            std::this_thread::sleep_for(std::chrono::milliseconds(1000));
            if (++count_arrival > 10) {
                break;
            }
        }
        if (navigation.check_goal_arrival()) {
            std::cout << "Target point reached" << std::endl;
        } else {
            std::cout << "Navigation failed; target point not reached" <<
std::endl;
        }
    } else {
        std::cout << "Path unreachable, cannot navigate to start point" <<
std::endl;
    }
    std::this_thread::sleep_for(std::chrono::milliseconds(1000));
}

// Stop navigation
navigation.stop_navigation();

// Get current pose
current_pose = navigation.get_current_pose();
std::cout << "Current pose: Position(" << current_pose.position.x << ", "
    << current_pose.position.y << ", " << current_pose.position.z
    << "), orientation(" << current_pose.orientation.x << ", "
    << current_pose.orientation.y << ", " << current_pose.orientation.z
    << ", " << current_pose.orientation.w << ")" << std::endl;
} else {
    std::cout << "relocalization failed, cannot proceed with navigation." << std::endl;
}

robot.request_shutdown();
robot.wait_for_shutdown();

```

```

robot.destroy();

return 0;
}

```

获取导航状态 + 轮询 SUCCESS/FAILED 或超时 (get_navigation_status_example)

适用场景：获取当前导航任务的执行状态，用于非阻塞导航时轮询。

```

examples/cpp/galbot_navigation/src/get_navigation_status_example.cpp
/**
 * example: navigation get_navigation_status, SUCCESS/FAILED timeout exit,
 * Avoid deadlock and execute error logic.
 */
#include "galbot_navigation.hpp"
#include "galbot_robot.hpp"

#include <chrono>
#include <iostream>
#include <string>
#include <thread>
#include <vector>

using namespace galbot::sdk;

int main() {
    auto& navigation = GalbotNavigation::get_instance(MachineType::G1);
    auto& robot = GalbotRobot::get_instance(MachineType::G1);

    if (!robot.init()) {
        std::cerr << "Base instance initialization failed!" << std::endl;
        return -1;
    }
    if (!navigation.init()) {
        std::cerr << "Navigation instance initialization failed!" << std::endl;
        return -1;
    }
    auto res = robot.switch_controller(G1ControllerName::CHASSIS_POSE_CTRL);
    if (res != ControlStatus::SUCCESS) {
        std::cerr << "Failed to switch controller!" << std::endl;
        return -1;
    }
    Pose goal_pose(std::vector<double>{0.5, 0.0, 0.0, 0.0, 0.0, 0.0, 1.0});
    const double timeout_s = 20.0;
    const double poll_interval_s = 0.5;

    // Non-blocking navigation
    navigation.navigate_to_goal(goal_pose, true, false, static_cast<float>(timeout_s));
    auto start = std::chrono::steady_clock::now();

    while (true) {
        NavigationTaskStatus status = navigation.get_navigation_status();
        auto elapsed = std::chrono::duration<double>(std::chrono::steady_clock::now() -
start).count();

        if (status == NavigationTaskStatus::SUCCESS) {
            std::cout << "Target reached" << std::endl;
            break;

```

```

    }
    if (status == NavigationTaskStatus::FAILED) {
        std::cout << "Navigation failed; exit error-handling logic promptly" <<
std::endl;
        break;
    }
    if (elapsed >= timeout_s) {
        std::cout << "navigationtimeout, exit" << std::endl;
        break;
    }

    if (status == NavigationTaskStatus::RUNNING) {
        std::cout << "Navigating... Status: RUNNING, elapsed: " << elapsed << "s" <<
std::endl;
    } else {
        std::cout << "Status: UNKNOWN, : " << elapsed << "s" << std::endl;
    }

    std::this_thread::sleep_for(std::chrono::milliseconds(static_cast<int>(poll_interval_s *
1000)));
}

robot.request_shutdown();
robot.wait_for_shutdown();
robot.destroy();
std::cout << "Resources released successfully" << std::endl;
return 0;
}

```

直线移动到目标 / 停止导航(linear_movement)

适用场景：让机器人沿直线移动到目标点，或停止当前正在进行的导航任务。

```

examples/cpp/galbot_navigation/src/linear_movement_example.cpp
#include "galbot_navigation.hpp"
#include "galbot_robot.hpp"

#include <iostream>
#include <string>
#include <vector>
#include <thread>
#include <chrono>
#include <iomanip>
#include <sstream>

using namespace galbot::sdk;

// Helper function: get current time string [HH:MM:SS.ms]
std::string get_timestamp() {
    auto now = std::chrono::system_clock::now();
    auto ms =
std::chrono::duration_cast<std::chrono::milliseconds>(now.time_since_epoch()) % 1000;
    std::time_t t = std::chrono::system_clock::to_time_t(now);
    std::tm tm = *std::localtime(&t);

    std::stringstream ss;
    ss << "[" << std::put_time(&tm, "%H:%M:%S") << "."
    << std::setfill('0') << std::setw(3) << ms.count() << "] ";
}

```

```

    return ss.str();
}

int main() {
    auto& navigation = GalbotNavigation::get_instance(MachineType::G1);
    auto& robot = GalbotRobot::get_instance(MachineType::G1);

    // Initialize system
    std::cout << get_timestamp() << "Starting call to robot.init()..." << std::endl;
    if (robot.init()) {
        std::cout << get_timestamp() << "Base instance initialized successfully!" <<
std::endl;
    } else {
        std::cerr << get_timestamp() << "Base instance initialization failed!" <<
std::endl;
        return -1;
    }

    std::cout << get_timestamp() << "Starting call to navigation.init()..." << std::endl;
    if (navigation.init()) {
        std::cout << get_timestamp() << "Navigation instance initialized successfully!" <<
std::endl;
    } else {
        std::cerr << get_timestamp() << "Navigation instance initialization failed!" <<
std::endl;
        return -1;
    }

    auto res = robot.switch_controller(G1ControllerName::CHASSIS_POSE_CTRL);
    if (res != ControlStatus::SUCCESS) {
        std::cerr << "Failed to switch controller!" << std::endl;
        return -1;
    }
    Pose init_pose(std::vector<double>{0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 1.0});
    Pose goal_pose(std::vector<double>{0.3, 0.0, 0.0, 0.0, 0.0, 0.0, 1.0});

    // checkrelocalize success
    std::cout << get_timestamp() << "Enter relocalization check loop..." << std::endl;
    int count_relocalize = 0;
    while (!navigation.is_localized() && count_relocalize < 20) {
        std::cout << get_timestamp() << "Calling navigation.relocalize()..." << std::endl;
        navigation.relocalize(init_pose);
        std::this_thread::sleep_for(std::chrono::milliseconds(500));
        std::cout << get_timestamp() << "is relocalizing" << std::endl;
        count_relocalize++;
    }

    if (navigation.is_localized()) {
        std::cout << get_timestamp() << "relocalization success." << std::endl;

        // Get current pose
        std::cout << get_timestamp() << "Calling navigation.get_current_pose()..." <<
std::endl;
        Pose current_pose = navigation.get_current_pose();
        std::cout << get_timestamp() << "Current pose: Position(" <<
current_pose.position.x << ", "
        << current_pose.position.y << ", " << current_pose.position.z
        << "), orientation(" << current_pose.orientation.x << ", "
        << current_pose.orientation.y << ", " << current_pose.orientation.z
        << ", " << current_pose.orientation.w << ")" << std::endl;
    }
}

```

```

// Whether to block and wait for arrival
bool is_blocking = true;
// Maximum wait time to reach position
float timeout_s = 20;

// --- ---
std::cout << "-----" << std::endl;
std::cout << get_timestamp() << "Preparing to execute move_straight_to (linear
movement)..." << std::endl;

// Record start time
auto start_move = std::chrono::system_clock::now();

// Execute move
NavigationStatus status = navigation.move_straight_to(goal_pose, is_blocking,
timeout_s);

// Record end time and calculate elapsed time
auto end_move = std::chrono::system_clock::now();
auto duration = std::chrono::duration_cast<std::chrono::milliseconds>(end_move -
start_move).count();

if (status == NavigationStatus::SUCCESS) {
    std::cout << get_timestamp() << "Target point reached (elapsed: " << duration
<< "ms)" << std::endl;
} else {
    std::cout << get_timestamp() << "navigationfailed, status: " <<
static_cast<int>(status)
        << " (elapsed: " << duration << "ms)" << std::endl;
}
std::cout << "-----" << std::endl;

// Stop navigation
std::cout << get_timestamp() << "Calling navigation.stop_navigation()..." <<
std::endl;
navigation.stop_navigation();

// Get current pose again
std::cout << get_timestamp() << "Calling navigation.get_current_pose()..." <<
std::endl;
current_pose = navigation.get_current_pose();
std::cout << get_timestamp() << "Current pose: Position(" <<
current_pose.position.x << ", "
        << current_pose.position.y << ", " << current_pose.position.z
        << "), orientation(" << current_pose.orientation.x << ", "
        << current_pose.orientation.y << ", " << current_pose.orientation.z
        << ", " << current_pose.orientation.w << ")" << std::endl;
} else {
    std::cout << get_timestamp() << "Relocalization failed, cannot continue
navigation." << std::endl;
}

std::cout << get_timestamp() << "Executing shutdown..." << std::endl;
robot.request_shutdown();
robot.wait_for_shutdown();
robot.destroy();
std::cout << get_timestamp() << "Program finished." << std::endl;

```

```
    return 0;
}
```

完整运行示例（简单流程）(complete_running_example)

适用场景：展示导航功能的完整使用流程，包括初始化、重定位、导航到目标等步骤，供参考。

```
examples/cpp/galbot_navigation/src/complete_running_example.cpp
#include "galbot_navigation.hpp"
#include "galbot_robot.hpp"

#include <iostream>
#include <string>
#include <vector>
#include <thread>
#include <thread>

using namespace galbot::sdk;

int main() {
    auto& navigation = GalbotNavigation::get_instance(MachineType::G1);
    auto& robot = GalbotRobot::get_instance(MachineType::G1);

    // Initialize system
    if (!robot.init()) {
        std::cerr << "Base instance initialization failed!" << std::endl;
        return -1;
    }
    if (!navigation.init()) {
        std::cerr << "Navigation instance initialization failed!" << std::endl;
        return -1;
    }

    auto res = robot.switch_controller(G1ControllerName::CHASSIS_POSE_CTRL);
    if (res != ControlStatus::SUCCESS) {
        std::cerr << "Failed to switch controller!" << std::endl;
        return -1;
    }
    Pose init_pose(std::vector<double>{0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 1.0});
    Pose goal_pose(std::vector<double>{0.5, 0.0, 0.0, 0.0, 0.0, 0.0, 1.0});

    // checkrelocalize success
    int count_relocalize = 0;
    while (!navigation.is_localized() && count_relocalize < 20) {
        navigation.relocalize(init_pose);
        std::this_thread::sleep_for(std::chrono::milliseconds(500));
        std::cout << "is relocalizing" << std::endl;
        count_relocalize++;
    }

    if (navigation.is_localized()) {
        std::cout << "Relocalization successful!" << std::endl;

        // Get current pose
        Pose current_pose = navigation.get_current_pose();
        std::cout << "Current pose: Position(" << current_pose.position.x << ", "
                    << current_pose.position.y << ", " << current_pose.position.z
                    << "), orientation(" << current_pose.orientation.x << ", "
                    << current_pose.orientation.y << ", " << current_pose.orientation.z
```

```

        << ", " << current_pose.orientation.w << ")" << std::endl;

// Whether to enable obstacle checking (can be set to true in open environments)
bool enable_collision_check = false;
// Whether to block and wait for arrival
bool is_blocking = true;
// Maximum wait time to reach position
float timeout_s = 20;

// checkpath navigation target
if (navigation.check_path_reachability(goal_pose, init_pose)) {
    std::cout << "Path reachable, navigating to target point" << std::endl;
    // navigation, obstaclecheck, wait, wait 20
    NavigationStatus status = navigation.navigate_to_goal(
        goal_pose, enable_collision_check, is_blocking, timeout_s);
    if (status == NavigationStatus::SUCCESS) {
        std::cout << "Target point reached" << std::endl;
    } else {
        std::cout << "navigationfailed, status: " << static_cast<int>(status) <<
std::endl;
    }
} else {
    std::cout << "Path unreachable, cannot navigate to target point" << std::endl;
}

// Check path reachability and return to start point
if (navigation.check_path_reachability(init_pose, goal_pose)) {
    std::cout << "Path reachable, navigating to start point" << std::endl;
    // Navigate to the target waypoint with obstacle checking disabled and non-
blocking wait for arrival
    is_blocking = false;
    NavigationStatus status = navigation.navigate_to_goal(
        init_pose, enable_collision_check, is_blocking);
    // wait
    int count_arrival = 0;
    while (!navigation.check_goal_arrival()) {
        std::cout << "navigate has not arrived" << std::endl;
        std::this_thread::sleep_for(std::chrono::milliseconds(1000));
        if (++count_arrival > 10) {
            break;
        }
    }
    if (navigation.check_goal_arrival()) {
        std::cout << "Target point reached" << std::endl;
    } else {
        std::cout << "Navigation failed; target point not reached" << std::endl;
    }
} else {
    std::cout << "Path unreachable, cannot navigate to start point" << std::endl;
}

// checkpath navigation target
if (navigation.check_path_reachability(goal_pose, init_pose)) {
    std::cout << "Path reachable, navigating to target point" << std::endl;
    // target, wait, wait 10
    is_blocking = true;
    NavigationStatus status = navigation.move_straight_to(
        goal_pose, is_blocking, timeout_s);

    if (status == NavigationStatus::SUCCESS) {

```

```

        std::cout << "Target point reached" << std::endl;
    } else {
        std::cout << "navigationfailed, status: " << static_cast<int>(status) <<
std::endl;
    }
    } else {
        std::cout << "Path unreachable, cannot navigate to target point" << std::endl;
    }

    // Stop navigation
    navigation.stop_navigation();

    } else {
        std::cout << "Relocalization failed!" << std::endl;
    }

    robot.request_shutdown();
    robot.wait_for_shutdown();
    robot.destroy();

    return 0;
}

```

类: GalbotPerception

高精度双目深度估计，单次推理并保存伪彩色深度图

适用场景：触发一次双目深度估计推理，并获取推理结果

```

examples/cpp/galbot_perception/src/foundation_stereo_run_once_example.cpp
/**
 * Foundation stereo depth example: single run_once + wait_for_new_result to fetch one
inference result.
 */

#include <iostream>
#include <thread>
#include <chrono>

#include "galbot_robot.hpp"
#include "galbot_perception.hpp"
#include "galbot_sdk_type.hpp"
#include "opencv2/opencv.hpp"

using namespace galbot::sdk;

bool run_foundation_stereo_once(GalbotPerception& perception) {
    if (!perception.run_once(PerceptionModule::FOUNDATION_STEREO)) {
        std::cerr << "run_once failed to send command" << std::endl;
        return false;
    }

    std::cout << "Waiting for inference result..." << std::endl;
    if (!perception.wait_for_new_result(PerceptionModule::FOUNDATION_STEREO, 5.0)) {
        std::cerr << "Timed out waiting for inference result" << std::endl;
        return false;
    }
}

```



```

DetectionResult result;
if (!perception.get_latest_result(PerceptionModule::FOUNDATION_STEREO, result)) {
    std::cerr << "get_latest_result failed" << std::endl;
    return false;
}

if (!result.instanceMask.empty()) {
    cv::Mat depth_map = result.instanceMask;
    std::cout << "Depth map size: " << depth_map.cols << "x" << depth_map.rows
        << ", type: " << depth_map.type() << std::endl;

    double min_val, max_val;
    cv::minMaxLoc(depth_map, &min_val, &max_val);
    std::cout << "Depth value range: [" << min_val << ", " << max_val << "]" <<
std::endl;

    cv::Mat depth_f;
    depth_map.convertTo(depth_f, CV_32F);
    cv::Mat mask = (depth_f > 0);

    cv::Mat normalized;
    cv::normalize(depth_f, normalized, 0, 255, cv::NORM_MINMAX, CV_8UC1, mask);

    cv::Mat colored;
    cv::applyColorMap(normalized, colored, cv::COLORMAP_TURBO);

    cv::imwrite("foundation_stereo_depth.jpg", colored);
    std::cout << "Depth map saved to foundation_stereo_depth.jpg" << std::endl;
} else {
    std::cout << "No depth map (instanceMask is empty)" << std::endl;
}

return true;
}

int main() {
    auto& robot = GalbotRobot::get_instance(MachineType::G1);
    robot.init();

    auto& perception = GalbotPerception::get_instance(MachineType::G1);
    perception.init({PerceptionModule::FOUNDATION_STEREO});

    // Wait for perception models to load
    std::this_thread::sleep_for(std::chrono::seconds(12));
    std::cout << "Init OK, sending single inference request..." << std::endl;

    run_foundation_stereo_once(perception);

    robot.request_shutdown();
    robot.wait_for_shutdown();
    robot.destroy();

    return 0;
}

```

类：Parameter

创建并使用 Parameter(parameter_use)

适用场景：创建运动规划参数对象，用于配置运动规划的各种参数如速度限制、加速度限制、规划时间等。

```
examples/cpp/galbot_motion/src/parameter_use_example.cpp
#include <iostream>
#include <vector>
#include <string>
#include <memory>

#include "galbot_motion.hpp"

using namespace galbot::sdk;

int main() {
    // Create Parameter via constructor and set options
    auto p = std::make_shared<Parameter>();

    p->set_blocking(true);           // Set whether to block execution
    p->set_check_collision(false);    // Disable collision detection
    p->set_timeout(5.0);              // Set timeout (seconds)
    p->set_actuate("with_chain_only");// Set drive mode
    p->set_tool_pose(false);          // Whether to consider tool pose
    p->set_reference_frame("base_link");

    std::cout << "--- Parameter p ---" << std::endl;
    std::cout << "blocking: " << (p->get_blocking() ? "True" : "False") << std::endl;
    std::cout << "collision check: " << (p->get_check_collision() ? "True" : "False") <<
std::endl;
    std::cout << "timeout: " << p->get_timeout() << "s" << std::endl;

    return 0;
}
```

辅助函数

status_to_string / JointStates / PoseState (auxi_fun_use)

适用场景：展示如何使用辅助函数转换状态为字符串，创建关节状态和位姿状态对象。

```
examples/cpp/galbot_motion/src/function_use_example.cpp
#include <iostream>
#include <string>

#include "galbot_motion.hpp"

using namespace galbot::sdk;

int main() {

    auto& planner = GalbotMotion::get_instance(MachineType::G1);

    MotionStatus status = MotionStatus::SUCCESS;
```

```

std::string status_str = planner.status_to_string(status);

std::cout << "MotionStatus string: " << status_str << std::endl;

auto js = std::make_shared<JointStates>();
auto ps = std::make_shared<PoseState>();

js->chain_name = "left_arm";
js->joint_positions = std::vector<double>(7, 0.0);

ps->chain_name = "left_arm";
ps->pose = Pose(std::vector<double>{1.0, 0.0, 0.0, 0.0, 0.0, 0.0, 1.0});
ps->reference_frame = "base_link";
ps->frame_id = "EndEffector";

std::cout << "--- JointStates ---" << std::endl;
std::cout << "Type: " << typeid(*js).name() << std::endl; // Print class name
std::cout << "Chain Name: " << js->chain_name << std::endl;
std::cout << "Joints size: " << js->joint_positions.size() << std::endl;

std::cout << "\n--- PoseState ---" << std::endl;
std::cout << "Type: " << typeid(*ps).name() << std::endl;
std::cout << "Chain Name: " << ps->chain_name << std::endl;
std::cout << "Pose Z: " << ps->pose.position.z << std::endl;

return 0;
}

```

16. Python API 参考 - G1 机器

- **GalbotRobot**: 核心机器人控制模块。用于机器人连接、生命周期管理、关节控制、传感器数据查询和硬件状态监控。
- **GalbotMotion**: 运动规划与执行模块。用于笛卡尔空间/关节空间运动、轨迹规划、逆运动学和全身控制。
- **GalbotNavigation**: 移动导航模块。用于移动底盘定位、地图构建、路径规划和自主移动。
- **GalbotPerception**: 端侧感知模块（仅 G1）。加载视觉模型、运行推理，并读取结构化结果（如立体深度）；与 GalbotRobot 传感器 API 配合使用。
- **Types & Enums**: 数据结构、枚举与状态类型。本节用于查询类型定义、传感器类型、错误码及其他模块使用的数据结构。

核心机器人控制

Galbot 人形机器人的主机器人控制界面。

此类提供了用于控制 Galbot 机器人的单例接口。它支持： 关节位置和轨迹控制末端执行器控制（夹具和吸盘） 移动基座速度控制传感器数据采集（IMU、相机、LiDAR、超声波） 坐标系转换系统生命周期管理使用 GalbotRobot::get_instance(MachineType) 获取特定平台 (G1/S1) 的参考。除非另有说明，所有角度均以弧度为单位。除非另有说明，所有线性距离均以米为单位。除非另有说明，所有时间戳均以纳秒为单位。

获取控制器权限

```
def acquire_controller(controller_name: str) -> ControlStatus
```

获取硬件控制权限。

请求指定控制器接管硬件控制。该操作与 switch_controller 使用同一套 WBCS 切换请求机制。

参数

名称	类型	默认值	描述
controller_name	str	需要传参	控制器名称字符串，例如 "left_arm_pvt_ctrl"。

返回值

类型	描述
----	----

ControlStatus	控制状态，指示获取操作的成功或失败
---------------	-------------------

检查轨迹执行状态

```
def check_trajectory_execution_status(  
    joint_groups: Sequence[str] = []  
) -> list[TrajectoryControlStatus]
```

获取指定关节组的轨迹执行状态。

查询指定关节组的轨迹当前执行状态。这对于在非阻塞执行模式下监视轨迹进度很有用。

参数

名称	类型	默认值	描述
joint_groups	Sequence[str]	[]	关节组列表

返回值

类型	描述
list[TrajectoryControlStatus]	轨迹控制状态列表：轨迹执行状态列表。

清除末端执行器命令

```
def clear_end_effector_command() -> ControlStatus
```

清除 WBC 末端执行器任务命令。

清除已发布到 WBC 通道的末端执行器任务轨迹命令。

返回值

类型	描述
ControlStatus	控制状态，表示命令发布结果。

销毁

```
def destroy() -> None
```

清理系统资源。

执行机器人控制系统资源的最终清理，包括中间件连接、传感器接口和通信通道。这是关闭序列的最后一步：request_shutdown() -> wait_for_shutdown() -> destroy()。

此方法应在程序结束时调用一次。调用 destroy() 后，SDK 进入终态，无法在同一进程重新初始化。如需再次使用，请退出当前进程并启动新进程。

执行关节轨迹

```
def execute_joint_trajectory(trajecory: Trajectory, is_blocking: bool = True) ->
ControlStatus
```

执行预先计划的关节轨迹。

执行由具有关联关节位置、速度和计时信息的航点组成的轨迹。轨迹控制器在路点之间进行插值以生成平滑的运动。

参数

名称	类型	默认值	描述
trajectory	Trajectory	需要传参	轨迹数据结构，包含路点和时序信息。必须指定 trajectory.joint_groups 或 trajectory.joint_names；如果两者都为空，则返回 INVALID_INPUT。
is_blocking	bool	True	是否阻塞等待完成

返回值

类型	描述
ControlStatus	控制状态，指示轨迹执行/提交的成功或失败

警告

每个 TrajectoryPoint.joint_command_vec 的顺序必须与 trajectory.joint_names 定义的关节顺序或 trajectory.joint_groups 的展开顺序一致。

警告

对于逐帧模型推理输出，建议使用命令流式接口（set_joint_commands / set_joint_commands_batch），而不是反复重新提交完整轨迹。

获取当前控制器

```
def get_active_controller(group_name: str) -> str
```

获取指定关节组的活动控制器名称。

返回当前 SDK 实例记录的该关节组控制器。该值会以模型默认控制器初始化，并在 SDK 控制器管理接口成功调用后更新。若控制已释放或该关节组无已知控制器，则返回 "CONTROLLER_NAME_NUM"。

参数

名称	类型	默认值	描述
group_name	str	需要传参	要查询的关节组名称。

返回值

类型	描述
str	std::string：当前 SDK 实例已知的活动控制器名称。

获取电池管理信息

```
def get_bms_information() -> dict
```

获取 BMS（电池管理系统）信息。

返回值

类型	描述
dict	共享指针，包含电池信息

获取相机内参

```
def get_camera_intrinsic(camera_id: SensorType) -> dict
```

获取相机内参。

获取指定相机的内参，包括焦距、主点、畸变系数等。

参数

名称	类型	默认值	描述
camera_id	SensorType	需要传参	RGB 相机 ID

返回值

类型	描述
dict	字典：包含相机内参。 - header: 消息头（带时间戳和帧信息） - height: 图像高度（像素） - width: 图像宽度（像素） - distortion_model: 畸变模型，如 "plumb_bob" - D: 畸变系数（浮点数列表） - K: 相机内参矩阵（9 个浮点数列表） - binning_x: 水平像素合并因子 - binning_y: 垂直像素合并因子 - roi: 感兴趣区域（整数列表） - camera_type: 相机类型。失败时返回空字典。

注意

相机传感器必须在初始化期间通过 enable_sensor_set 启用。

获取深度图像数据

```
def get_depth_data(camera_id: SensorType) -> dict
```

从指定相机获取最新的深度图像。

检索指定深度相机捕获的最新深度图像。深度值通常表示距相机传感器的距离。

参数

名称	类型	默认值	描述
camera_id	SensorType	需要传参	RGB 相机 ID

返回值

类型	描述
dict	字典：包含以下键： - header: 消息头（带时间戳和帧信息） - format: 图像格式，如 "depth16" 或其他 - depth_scale: 深度缩放因子 - height: 图像高度（像素） - width: 图像宽度（像素） - data: 压缩深度图像二进制数据（字节）。失败时返回空字典。

注意

相机传感器必须在初始化期间通过 enable_sensor_set 启用。

注意

深度值通常以毫米（mm）或米（m）为单位。

获取设备信息

```
def get_device_information() -> dict
```

获取设备信息。

检索基本设备信息，包括设备型号、序列号、固件版本、硬件版本和制造商。此信息用于设备管理、版本控制、系统诊断和设备识别。

返回值

类型	描述
dict	返回设备信息的共享指针，包含型号、序列号、固件版本、硬件版本和厂商信息。若读取失败，返回空指针。

获取灵巧手状态

```
def get_dexhand_state(end_effector: str, dexhand_type: DexHandType = ...) -> Any
```

获取当前灵巧手状态。

检索灵巧手反馈到 dexhand_state。对于 INSPIRE 和 BRAINCO，仅填充 dexhand_state.joint_state，force_sensor_map 为空。对于 SHARPA，填充完整关节状态以及可用的命名力传感器数据。

参数

名称	类型	默认值	描述
end_effector	str	需要传参	灵巧手名称，例如 "left_dexhand" 或 "right_dexhand"。
dexhand_type	DexHandType	...	灵巧手型号类型（可选，默认：INSPIRE）。

返回值

类型	描述
Any	DexhandState（灵巧手状态）

获取力传感器数据

```
def get_force_sensor_data(sensor_type: GalbotOneFoxtrotSensor) -> dict
```

获取力/扭矩传感器数据。

从指定的力/扭矩传感器检索最新测量值。这些传感器通常安装在手腕或末端执行器上，用于接触力监测和合规性控制。

参数

名称	类型	默认值	描述
sensor_type	GalbotOneFoxtrotSensor	需要传参	力传感器类型

返回值

类型	描述
dict	返回力传感器数据的共享指针，包含力向量 [fx, fy, fz]、力矩向量 [tx, ty, tz] 以及时间戳。若传感器未启用或读取失败，返回空指针。

注意

力传感器必须在初始化期间通过 enable_sensor_set 启用。

获取坐标系名称

```
def get_frame_names() -> list[str]
```

获取 TF 树中所有可用的坐标系名称。

返回值

类型	描述
list[str]	所有可用坐标系名称列表。

获取夹爪状态

```
def get_gripper_state(end_effector: str) -> GripperState
```

获取指定夹爪的当前状态。

返回夹爪的位置、速度、受力以及运动状态估计信息。

GripperState.is_moving 基于时间窗口判定：若在内部窗口内未检测到有效开合变化，则该值会变为 false。

参数

名称	类型	默认值	描述
end_effector	str	需要传参	要查询的夹爪关节组名称（例如 left_gripper、right_gripper）。

返回值

类型	描述
GripperState	返回夹爪状态的共享指针；若读取失败，返回 nullptr。

获取 IMU 数据

```
def get_imu_data(sensor_id: SensorType) -> dict
```

获取 IMU（惯性测量单元）传感器数据。

检索最新的 IMU 测量值，包括线性加速度、角速度和方向估计。

参数

名称	类型	默认值	描述
sensor_id	SensorType	需要传参	传感器 ID

返回值

类型	描述
dict	字典：包含以下键： - timestamp_ns: 时间戳（纳秒） - accel: 加速度 Vector3 {"x": float, "y": float, "z": float} - gyro: 陀螺仪 Vector3 {"x": float, "y": float, "z": float} - magnet: 磁力计 Vector3 {"x": float, "y": float, "z": float}。

	失败时返回空字典。
--	-----------

注意

IMU 传感器必须在初始化期间通过 `enable_sensor_set` 启用。

注意

加速度以米每二次方秒 (m/s^2) 为单位。

注意

角速度以弧度每秒 (rad/s) 为单位。

获取关节组名称

```
def get_joint_group_names() -> list[str]
```

获取机器人可用的关节组名称。

检索机器人运动学配置中定义的所有关节组名称。这对于在运行时发现可用的控制组很有用。

返回值

类型	描述
<code>list[str]</code>	关节组名称向量，或如果检索失败则为空向量

获取关节名称

```
def get_joint_names(only_active_joint: bool = True, joint_groups: Sequence[str] = []) -> list[str]
```

通过组名获取机器人关节名称。

检索属于指定关节组的关节名称。这对于在设置关节位置时确定正确的顺序很有用。

参数

名称	类型	默认值	描述
<code>only_active_joint</code>	<code>bool</code>	<code>True</code>	是否仅返回主动关节
<code>joint_groups</code>	<code>Sequence[str]</code>	<code>[]</code>	关节组列表

返回值

类型	描述
<code>list[str]</code>	返回关节名称列表，顺序取决于参数配置

获取关节位置

```
def get_joint_positions(  
    joint_groups: Sequence[str] = [],
```

```
joint_names: Sequence[str] = []
) -> list[float]
```

获取关节位置

参数

名称	类型	默认值	描述
joint_groups	Sequence[str]	[]	关节组列表
joint_names	Sequence[str]	[]	关节名称列表

返回值

类型	描述
list[float]	当前关节角度向量（弧度）

注意

返回值顺序：指定 joint_names 时按 joint_names 顺序；仅指定 joint_groups 时按组定义顺序；都为空时按 chassis、head、left_arm、right_arm、leg 顺序返回身体关节

获取关节状态

```
def get_joint_states(
    joint_groups: Sequence[str] = [],
    joint_names: Sequence[str] = []
) -> list[JointState]
```

获取关节状态

参数

名称	类型	默认值	描述
joint_groups	Sequence[str]	[]	关节组名称列表
joint_names	Sequence[str]	[]	具体关节名称列表，优先于 joint_groups

返回值

类型	描述
list[JointState]	返回关节状态列表，包含位置、速度、加速度、力矩等信息

注意

返回值顺序：指定 joint_names 时按 joint_names 顺序；仅指定 joint_groups 时按组定义顺序；都为空时按 chassis、head、left_arm、right_arm、leg 顺序返回身体关节

获取激光雷达数据

```
def get_lidar_data(sensor_id: SensorType) -> dict
```

获取最新的激光雷达点云数据。

检索指定 LiDAR 传感器捕获的最新 3D 点云。每个点通常包含 (x, y, z) 坐标和可选的强度值。

参数

名称	类型	默认值	描述
sensor_id	SensorType	需要传参	传感器 ID

返回值

类型	描述
dict	字典：包含点云数据字段和二进制点数据。失败时返回空字典。

注意

LiDAR 传感器必须在初始化期间通过 enable_sensor_set 启用。

获取日志信息

```
def get_log_information(timewindow_s: SupportsInt, log_level: LogLevel) -> dict
```

获取日志信息。

参数

名称	类型	默认值	描述
timewindow_s	SupportsInt	需要传参	时间窗口（秒）
log_level	LogLevel	需要传参	日志级别

返回值

类型	描述
dict	共享指针，包含日志信息

获取里程计数据

```
def get_odom() -> dict
```

获取机器人里程信息。

从里程计系统检索机器人当前的姿态和速度估计。里程计通常融合车轮编码器、IMU 和其他本体感觉传感器。

返回值

类型	描述
dict	返回里程计数据的共享指针，包含位置、姿态、线速度、角速度与时间戳。若里程计不可用，返回空指针。

获取 RGB 图像数据

```
def get_rgb_data(camera_id: SensorType) -> dict
```

从指定相机获取最新的 RGB 图像。

检索指定 RGB 相机捕获的最新彩色图像。

参数

名称	类型	默认值	描述
camera_id	SensorType	需要传参	RGB 相机 ID

返回值

类型	描述
dict	字典：包含以下键：- header: 消息头（带时间戳和帧信息）- format: 图像格式，如 "jpeg" 或 "png" - data: 压缩图像二进制数据（字节）。失败时返回空字典。

注意

相机传感器必须在初始化期间通过 enable_sensor_set 启用。

获取传感器外参

```
def get_sensor_extrinsic(sensor_id: SensorType, reference_frame: str = 'base_link') -> tuple
```

获取传感器外参。

检索指定传感器的外部参数，包括相对于机器人基础坐标系的旋转和平移向量。

参数

名称	类型	默认值	描述
sensor_id	SensorType	需要传参	传感器 ID
reference_frame	str	'base_link'	参考坐标系

返回值

类型	描述
tuple	包含以下内容的 Pair：- 7 个 double 的向量，表示变换

	[x, y, z, qx, qy, qz, qw], 其中 (x, y, z) 为平移 (米) , (qx, qy, qz, qw) 为四元数方向 - 变换有效时的时间戳 (纳秒) 。如果检索失败则返回空向量, 时间戳为 0。
--	---

注意

传感器必须在初始化期间通过 enable_sensor_set 启用。

获取吸盘状态

```
def get_suction_cup_state(end_effector: str) -> SuctionCupState
```

获取当前吸盘状态。

检索指定吸盘的当前状态, 包括激活状态和真空压力测量。

参数

名称	类型	默认值	描述
end_effector	str	需要传参	吸盘末端执行器

返回值

类型	描述
SuctionCupState	返回吸盘状态的共享指针; 若读取失败, 返回 nullptr。

获取坐标变换

```
def get_transform(
    target_frame: str,
    source_frame: str,
    timestamp_ns: SupportsInt = 0,
    timeout_ms: SupportsInt = 100
) -> tuple
```

查询坐标系变换 (TF) 查询机器人 TF 树中两个坐标系之间的变换。

这用于在不同参考坐标系之间转换姿态和位置 (例如, 从相机坐标系到基坐标系, 从末端执行器到世界坐标系) 。

参数

名称	类型	默认值	描述
target_frame	str	需要传参	目标坐标系
source_frame	str	需要传参	源坐标系
timestamp_ns	SupportsInt	0	时间戳 (纳秒)
timeout_ms	SupportsInt	100	超时时间 (毫秒)

返回值

类型	描述
tuple	包含以下内容的 Pair: - 7 个 double 的向量, 表示变换 [x, y, z, qx, qy, qz, qw], 其中 (x, y, z) 为平移 (米), (qx, qy, qz, qw) 为四元数方向 - 变换有效时的时间戳 (纳秒)。如果检索失败或超时则返回空向量, 时间戳为 0。

获取超声波数据

```
def get_ultrasonic_data(ultrasonic_type: UltrasonicType) -> dict
```

从指定的超声波传感器获取距离测量值。

从超声波距离传感器之一检索最新的距离测量值。机器人通常在其周边布置有多个超声波传感器。

参数

名称	类型	默认值	描述
ultrasonic_type	UltrasonicType	需要传参	超声波传感器类型

返回值

类型	描述
dict	返回超声波测距数据的共享指针 (单位: 米); 若传感器未启用或读取失败, 返回 nullptr。

注意

超声波传感器必须在初始化期间通过 enable_sensor_set 启用。

获取音量

```
def get_volume() -> float
```

获取当前系统全局卷值。

返回值

类型	描述
float	float: 返回当前音量值, 范围为 0.0 到 100.0

获取 WBC 末端执行器位姿

```
def get_wbc_end_effector_poses() -> dict[str, list[float]]
```

获取 WBC 末端执行器位姿。

返回 WBC 各末端执行器 (左臂、右臂、头部) 的当前位姿。

返回值

类型	描述
dict[str, list[float]]	字典映射，包含：- "lee_pose": 左臂末端位姿 [x, y, z, qx, qy, qz, qw]- "ree_pose": 右臂末端位姿 [x, y, z, qx, qy, qz, qw]- "head_pose": 头部末端位姿 [x, y, z, qx, qy, qz, qw]当失败或缺少键时，可能返回空条目或部分数据。

初始化

```
def init(enable_sensor_set: Set[SensorType] = ...) -> bool
```

初始化机器人控制系统。

初始化机器人硬件通信、中间件和传感器接口。为了优化资源使用，只有在 enable_sensor_set 中指定的传感器才会被初始化并可用于数据读取。

此方法应在程序启动时调用一次。在未调用 destroy() 的情况下多次调用不会报错，但只有第一次调用生效。

参数

名称	类型	默认值	描述
enable_sensor_set	Set[SensorType]	...	要启用的传感器集合，如果为空，则启用默认的传感器集合，仅指定所需传感器可减少启动时间和资源占用

返回值

类型	描述
bool	初始化成功返回 true，否则返回 false。

检查是否运行中

```
def is_running() -> bool
```

检查机器人控制系统是否正在运行。

查询机器人控制系统是否仍处于活动状态，或者是否已收到关闭信号（例如，SIGINT、SIGTERM）。

返回值

类型	描述
bool	系统正常运行时返回 true，否则返回 false。

发布目标

```
def publish_target(target: SingoriXTarget) -> ControlStatus
```

将目标发布到机器人控制系统，使目标可被运动规划模块使用。这是 Python 绑定版本。

参数

名称	类型	默认值	描述
target	SingoriXTarget	需要传参	目标对象

返回值

类型	描述
ControlStatus	无返回值

释放控制器权限

```
def release_controller(group_name: str = 'all') -> ControlStatus
```

释放硬件权限。

控制硬件，释放关节。与 acquire_controller 相反。如果运行则隐式停止执行。

参数

名称	类型	默认值	描述
group_name	str	'all'	要释放的关节组名称，支持的组：chassis（底盘）、legs（腿部）、head（头部）、left_arm（左臂）、right_arm（右臂）、gripper（夹爪）、suction_cup（吸盘）或 "all"（释放所有控制器）

返回值

类型	描述
ControlStatus	控制状态，指示释放操作的成功或失败

重新加载控制器

```
def reload_controller(group_name: str = 'all') -> ControlStatus
```

重新加载控制器。

重新初始化控制器。相当于一个完整的重启循环：停止->重置->启动。对于错误恢复或应用配置更改很有用。

参数

名称	类型	默认值	描述
group_name	str	'all'	要重新加载的控制器组名称

返回值

类型	描述
ControlStatus	控制状态，指示重新加载操作的成功或失败

请求关机

```
def request_shutdown() -> None
```

请求系统关闭。

发送关闭信号以启动系统正常关闭。这会触发已注册的退出回调并开始资源清理。作为关闭序列的第一步调用：request_shutdown() -> wait_for_shutdown() -> destroy()。

请求目标

```
def request_target(target: SingorIXTarget) -> ErrorInfo
```

向机器人控制系统请求一个目标，用于运动规划和执行。这是 Python 绑定版本。

参数

名称	类型	默认值	描述
target	SingorIXTarget	需要传参	目标对象

返回值

类型	描述
ErrorInfo	无返回值

设置底盘姿态

```
def set_base_pose(
    base_pose: Pose,
    is_blocking: bool = True,
    timeout_s: SupportsFloat = 15.0
) -> ControlStatus
```

设置移动基础位姿命令。

命令机器人的移动底座移动到你参考坐标系中的指定姿态。这使用底盘姿态控制器 (CHASSIS_POSE_CTRL)。当完整的 3D 姿态 (位置 + 四元数方向) 已经可用时, 使用此重载。

参数

名称	类型	默认值	描述
base_pose	Pose	需要传参	底盘目标姿态
is_blocking	bool	True	是否阻塞等待完成
timeout_s	SupportsFloat	15.0	超时时间 (秒)。

返回值

类型	描述
ControlStatus	控制状态, 指示命令传输的成功或失败

设置底盘姿态

```
def set_base_pose(
    x: SupportsFloat,
    y: SupportsFloat,
    yaw: SupportsFloat,
    frame_id: str = 'odom',
    reference_frame_id: str = 'odom',
    is_blocking: bool = True,
    timeout_s: SupportsFloat = 15.0
) -> ControlStatus
```

使用可选坐标系设置移动基础姿态 (x、y、航向角)。

将此重载用于由所选坐标系中的 x/y/航向角定义的平面 2D 目标命令。

参数

名称	类型	默认值	描述
x	SupportsFloat	需要传参	X 坐标 (米)
y	SupportsFloat	需要传参	Y 坐标 (米)
yaw	SupportsFloat	需要传参	航向角 (弧度)
frame_id	str	'odom'	坐标系 ID
reference_frame_id	str	'odom'	参考坐标系 ID
is_blocking	bool	True	是否阻塞等待完成
timeout_s	SupportsFloat	15.0	超时时间 (秒)。

返回值

类型	描述
ControlStatus	控制状态, 指示命令传输的成功或失败

设置底盘姿态

```
def set_base_pose(
    x: SupportsFloat,
    y: SupportsFloat,
    yaw: SupportsFloat,
    frame_id: str,
    reference_frame_id: str,
    time_from_start_s: SupportsFloat,
    is_blocking: bool = True,
    timeout_s: SupportsFloat = 15.0
) -> ControlStatus
```

使用明确的插值时间设置移动基础姿态（x、y、航向角）。
当必须通过 time_from_start_s 协调到达时间时，请使用此重载。

参数

名称	类型	默认值	描述
x	SupportsFloat	需要传参	X 坐标（米）
y	SupportsFloat	需要传参	Y 坐标（米）
yaw	SupportsFloat	需要传参	航向角（弧度）
frame_id	str	需要传参	坐标系 ID
reference_frame_id	str	需要传参	参考坐标系 ID
time_from_start_s	SupportsFloat	需要传参	预期到达时间（秒）
is_blocking	bool	True	是否阻塞等待完成
timeout_s	SupportsFloat	15.0	超时时间（秒）。

返回值

类型	描述
ControlStatus	控制状态，指示命令传输的成功或失败

设置底盘速度

```
def set_base_velocity(
    linear_velocity: list[float],
    angular_velocity: list[float],
    duration_s: SupportsFloat = 0.0
) -> ControlStatus
```

设置移动基础速度命令。
命令机器人的移动底座以指定的线速度和角速度移动。速度在机器人的基坐标系中表示。

参数

名称	类型	默认值	描述
linear_velocity	list[float]	需要传参	线速度（米/秒），在底座坐标系中表示。顺序：{vx,

			vy, vz}- vx: X 方向线速度（前后） - vy: Y 方向线速度（左右） - vz: Z 方向线速度（垂直）
angular_velocity	list[float]	需要传参	角速度（弧度/秒），在底座坐标系中表示。顺序： {wx, wy, wz}- wx: 绕 X 轴角速度（翻滚） - wy: 绕 Y 轴角速度（俯仰） - wz: 绕 Z 轴角速度（偏航）
duration_s	SupportsFloat	0.0	持续时间（秒）

返回值

类型	描述
ControlStatus	控制状态，指示命令传输的成功或失败

设置灵巧手命令

```
def set_dexhand_command(
    end_effector: str,
    dexhand_command: Sequence[JointCommand],
    dexhand_type: DexHandType = ...,
    is_blocking: bool = True
) -> ControlStatus
```

通过关节命令控制灵巧手。

使用关节命令向量控制灵巧手（位置、速度、力矩等）。

参数

名称	类型	默认值	描述
end_effector	str	需要传参	要控制的灵巧手名称（例如 left_dexhand、right_dexhand）。
dexhand_command	Sequence[JointCommand]	需要传参	每个灵巧手关节的命令向量。Inspire: [位置, 速度, 加速度, 力矩] 范围 [0-1000, 0-1000, 0-1000]。BrainCo: [位置, 速度, 加速度, 力矩] 范围 [0-100, -100-100, ,]。Sharpa: 22 个关节命令 [位置, 速度, 加速度, 力矩]。
dexhand_type	DexHandType	...	灵巧手型号。
is_blocking	bool	True	是否阻塞等待动作完成。

返回值

类型	描述
ControlStatus	控制状态，表示命令发布结果。

设置末端执行器命令

```
def set_end_effector_command(  
    poses: Sequence[list[float]],  
    end_effector_frames: Sequence[str],  
    reference_frames: Sequence[str] = [],  
    time_from_start_s: SupportsFloat = 0.0  
) -> ControlStatus
```

设置 WBC 末端执行器位姿命令（任务轨迹发布）。

poses 中每一行为 [x, y, z, qx, qy, qz, qw]（单位：米，四元数顺序 xyzw）。poses 与 end_effector_frames 的长度必须一致。

参数

名称	类型	默认值	描述
poses	Sequence[list[float]]	需要传参	每个末端执行器对应一个位姿；每行格式为 [x, y, z, qx, qy, qz, qw]（单位：米，四元数顺序 xyzw）。
end_effector_frames	Sequence[str]	需要传参	每个位姿对应的目标坐标系 id（例如连杆名称）。
reference_frames	Sequence[str]	[]	每个位姿对应的参考坐标系。省略或传入 [] 时默认对所有位姿使用 "world"。否则长度必须与 poses 一致。常用值："world"（默认）。
time_from_start_s	SupportsFloat	0.0	从轨迹起点开始的时间（秒，可选，默认 0.0）。

返回值

类型	描述
ControlStatus	控制状态，表示命令发布结果。

设置夹爪命令

```
def set_gripper_command(  
    end_effector: str,  
    width_m: SupportsFloat,  
    velocity_mps: SupportsFloat = 0.03,
```

```
    effort: SupportsFloat = 30,  
    is_blocking: bool = True  
) -> ControlStatus
```

控制夹具张开宽度和力度。

命令夹具以受控的速度和最大夹持力移动到指定的开口宽度。

参数

名称	类型	默认值	描述
end_effector	str	需要传参	要控制的夹爪关节组名称（例如 left_gripper、right_gripper）。
width_m	SupportsFloat	需要传参	目标开口宽度（米）
velocity_mps	SupportsFloat	0.03	夹爪运动速度（米/秒）
effort	SupportsFloat	30	最大夹持力（牛顿·米，N·m）。
is_blocking	bool	True	是否阻塞等待完成

返回值

类型	描述
ControlStatus	控制状态，指示夹爪命令的成功或失败

设置关节命令

```
def set_joint_commands(  
    joint_commands: Sequence[JointCommand],  
    joint_groups: Sequence[str] = [],  
    joint_names: Sequence[str] = [],  
    time_from_start_s: SupportsFloat = 10.0  
) -> ControlStatus
```

设置低级关节命令以进行高频流控制。

适用于高频命令流（例如，每帧模型推理输出）。

此 API 不会从当前/开始位置插值到第一个目标。控制器尽快将关节驱动到每个命令目标以满足 time_from_start_s（预期到达时间）。

对于标准关节（头、腿、手臂），当前版本仅 JointCommand::position 有效；

目前忽略速度、加速度和作用力。对于夹具关节，位置字段代表夹具宽度，并且速度和作用力字段均受到支持且有效。夹具运动使用指定速度和 time_from_start_s 之间较慢的一个。因此，在设置夹爪速度时，可以将 time_from_start_s 设置为 0（最快到达），夹爪将直接由指定的速度控制。

参数

名称	类型	默认值	描述
----	----	-----	----

joint_commands	Sequence[JointCommand]	需要传参	关节命令列表
joint_groups	Sequence[str]	[]	要控制的关节组。支持的组：legs（腿部）、head（头部）、left_arm（左臂）、right_arm（右臂）、gripper（夹爪）、suction_cup（吸盘）。如果 joint_names 也为空，则不能为空，否则返回 INVALID_INPUT。
joint_names	Sequence[str]	[]	要控制的具体关节名称。此参数优先于 joint_groups。如果提供了 joint_names，则忽略 joint_groups。如果 joint_groups 也为空，则不能为空，否则返回 INVALID_INPUT。
time_from_start_s	SupportsFloat	10.0	预期到达时间（秒）

返回值

类型	描述
ControlStatus	控制状态，指示命令传输的成功或失败

警告

尤其在第一条命令下发时，请避免当前关节角与目标关节角差值过大。角度突变可能导致运动过快并带来安全风险。

批量设置关节命令

```
def set_joint_commands_batch(trajecory: Trajectory) -> ControlStatus
```

以批处理模式设置关节命令（非阻塞） 实时控制模式下设置多个关节命令轨迹点，支持一次性提交多个时间点的轨迹控制命令。

提供非阻塞高频轨迹执行接口。与 set_joint_commands 类似，但支持批量轨迹控制，适用于 VLA 推理批量输出等场景。

参数

名称	类型	默认值	描述
trajectory	Trajectory	需要传参	包含关节命令路点的轨迹数据结构。必须指定 trajectory.joint_groups 或 trajectory.joint_names；如

			果两者都为空，则返回 INVALID_INPUT。每个 TrajectoryPoint 包含 time_from_start 和关节命令列表。关节命令包括位置（弧度）、速度（弧度/秒）、加速度（弧度/秒 ² ）、作用力（牛·米）、Kp（位置增益）和 Kd（速度增益）。
--	--	--	---

返回值

类型	描述
ControlStatus	控制状态，指示命令提交的成功或失败。立即返回，不等待执行完成（非阻塞）。

警告

每个 TrajectoryPoint.joint_command_vec 的顺序必须与 trajectory.joint_names 定义的关节顺序或 trajectory.joint_groups 的展开顺序一致。

设置关节位置

```
def set_joint_positions(
    joint_positions: list[float],
    joint_groups: Sequence[str] = [],
    joint_names: Sequence[str] = [],
    is_blocking: bool = True,
    speed_rad_s: SupportsFloat = 0.2,
    timeout_s: SupportsFloat = 15.0
) -> ControlStatus
```

按名称设置指定关节组的目标关节位置（用于低频关键帧/姿态转换） 命令机器人将指定关节移动到目标位置。

该运动以具有可配置速度限制的平滑轨迹执行。

参数

名称	类型	默认值	描述
joint_positions	list[float]	需要传参	目标关节位置
joint_groups	Sequence[str]	[]	要控制的关节组名称。支持的组：腿部、头部、左臂、右臂。如果 joint_names 也为空，则不能为空，否则返回 INVALID_INPUT。
joint_names	Sequence[str]	[]	要控制的具体关节名称。

			此参数优先于 joint_groups。如果提供了 joint_names，则忽略 joint_groups。如果 joint_groups 也为空，则不能为空，否则返回 INVALID_INPUT。
is_blocking	bool	True	是否阻塞等待完成。true：阻塞直到完成或超时；false：发送后立即返回。
speed_rad_s	SupportsFloat	0.2	最大运动速度（弧度/秒）
timeout_s	SupportsFloat	15.0	超时时间（秒）。

返回值

类型	描述
ControlStatus	控制状态，指示运动命令的成功或失败

警告

此 API 不适合高频率的逐帧运动控制。

设置吸盘命令

```
def set_suction_cup_command(end_effector: str, activate: bool) -> ControlStatus
```

控制吸盘激活状态。

激活或停用指定的吸盘末端执行器。

参数

名称	类型	默认值	描述
end_effector	str	需要传参	吸盘末端执行器
activate	bool	需要传参	是否激活吸盘

返回值

类型	描述
ControlStatus	控制状态，指示命令传输的成功或失败

设置音量

```
def set_volume(volume: SupportsFloat) -> bool
```

设置系统全局音量值。

参数

名称	类型	默认值	描述
volume	SupportsFloat	需要传参	音量值 (0-100)

返回值

类型	描述
bool	返回音量设置结果：成功为 true，失败为 false。

启动控制器

```
def start_controller(group_name: str = 'all') -> ControlStatus
```

开始控制器执行。

激活控制器以开始发送命令。与 stop_controller 相反。需要事先获得硬件权限（获取）。

参数

名称	类型	默认值	描述
group_name	str	'all'	要启动的控制器组名称

返回值

类型	描述
ControlStatus	控制状态，指示启动操作的成功或失败

启动麦克风流输入

```
def start_microphone_stream_input(
    callback: Callable,
    chunk_size: SupportsInt = 2560,
    use_raw_audio: bool = False
) -> str
```

启动麦克风流音频输入。

参数

名称	类型	默认值	描述
callback	Callable	需要传参	音频数据回调函数
chunk_size	SupportsInt	2560	每个音频块的样本数
use_raw_audio	bool	False	是否使用原始音频。 - false: 使用处理后的音频（默认） - true: 输出原始 PCM 音频数据（目前不支持动态配置）

返回值

类型	描述
str	str：用于标识音频输入流的流 ID。

停止音频流输出

```
def stop_audio_stream_output(stream_id: str = '') -> None
```

停止指定音频输出流或所有活动音频输出流的播放。

参数

名称	类型	默认值	描述
stream_id	str	"	输出流 ID

紧急停止底盘

```
def stop_base() -> ControlStatus
```

紧急停止移动底座运动。

立即命令移动基地停止一切运动。这是一项安全功能，当需要立即停止基本运动时应使用。

返回值

类型	描述
ControlStatus	控制状态，指示命令传输的成功或失败

停止控制器

```
def stop_controller(group_name: str = 'all') -> ControlStatus
```

停止控制器执行。

停止命令执行但保留硬件权限。与 start_controller 相反。

参数

名称	类型	默认值	描述
group_name	str	'all'	要停止的控制器组名称

返回值

类型	描述
ControlStatus	控制状态，指示停止操作的成功或失败

停止麦克风流输入

```
def stop_microphone_stream_input(stream_id: str = '') -> None
```

停止指定的麦克风流式音频输入。

参数

名称	类型	默认值	描述
stream_id	str	"	输出流 ID

停止轨迹执行

```
def stop_trajectory_execution() -> ControlStatus
```

停止所有当前正在执行的关节轨迹。

立即停止执行所有关节组中的所有活动关节轨迹。停止后关节将保持当前位置。

返回值

类型	描述
ControlStatus	控制状态，指示命令传输的成功或失败

切换控制器

```
def switch_controller(controller_name: str) -> ControlStatus
```

切换活动控制器策略。

将硬件控制切换到新的策略。请求会发送到 WBCS 控制器管理器，由其执行安全切换，并在接受请求后启动目标控制器。

参数

名称	类型	默认值	描述
controller_name	str	需要传参	控制器名称字符串，例如 "chassis_pose_ctrl"。

返回值

类型	描述
ControlStatus	控制状态，指示切换操作的成功或失败

等待关机

```
def wait_for_shutdown() -> None
```

阻塞直到关闭完成。

阻塞调用线程直到所有模块都正常关闭完毕。作为关闭序列的第二步调用，在 request_shutdown() 之后、destroy() 之前。

注意

当 is_running() 变为 false 时此函数将返回。

写入音频流输出

```
def write_audio_stream_output(audio_chunk: str, stream_id: str = '') -> bool
```

将 PCM 格式的音频数据块写入音频输出流以进行实时播放。

参数

名称	类型	默认值	描述
audio_chunk	str	需要传参	音频数据块
stream_id	str	"	输出流 ID

返回值

类型	描述
bool	返回操作结果：音频写入并成功下发播放任务为 true，失败为 false。

全身与底盘归零

```
def zero_whole_body_and_base(
    base_zero_pose: Pose,
    is_blocking: bool = True,
    leg_head_speed_rad_s: SupportsFloat = 0.2,
    leg_head_timeout_s: SupportsFloat = 15.0,
    params: Parameter = None
) -> tuple[MotionStatus, ControlStatus]
```

一键归零：将全身关节回零，并将底盘位姿回到零位。

该接口会调用 move_whole_body_joint_zero 完成关节归零，并将底盘目标位姿设为零。若 params 为空指针，则使用 default_param。

参数

名称	类型	默认值	描述
base_zero_pose	Pose	需要传参	-
is_blocking	bool	True	-
leg_head_speed_rad_s	SupportsFloat	0.2	-
leg_head_timeout_s	SupportsFloat	15.0	-
params	Parameter	None	-

返回值

类型	描述
tuple[MotionStatus, ControlStatus]	-

全身与底盘归零

```
def zero_whole_body_and_base(  
    frame_id: str = 'odom',  
    reference_frame_id: str = 'odom',  
    is_blocking: bool = True,  
    leg_head_speed_rad_s: SupportsFloat = 0.2,  
    leg_head_timeout_s: SupportsFloat = 15.0,  
    params: Parameter = None  
) -> tuple[MotionStatus, ControlStatus]
```

一键归零：通过可选坐标系将全身关节归零，并将底座（x、y、航向角）归零。

参数

名称	类型	默认值	描述
frame_id	str	'odom'	坐标系 ID
reference_frame_id	str	'odom'	参考坐标系 ID
is_blocking	bool	True	是否阻塞等待完成
leg_head_speed_rad_s	SupportsFloat	0.2	腿部和头部关节速度（弧度/秒）
leg_head_timeout_s	SupportsFloat	15.0	腿部和头部运动超时（秒）
params	Parameter	None	额外参数

返回值

类型	描述
tuple[MotionStatus, ControlStatus]	-

运动规划与执行

Galbot 机器人的统一运动规划和控制接口。

该接口提供全面的机器人运动控制 API，包括： - 正向和逆向运动学计算 - 单链和多链轨迹规划 - 碰撞检测（自碰撞和环境） - 工具和障碍物管理 - 全身协调运动规划使用

GalbotMotion::get_instance(MachineType) 获取特定平台 (G1/S1) 的实例。所有角度单位为弧度，线性单位为米（SI 标准）。四元数必须归一化： $\sqrt{x^2 + y^2 + z^2 + w^2} = 1$ 。

添加障碍物

```
def add_obstacle(
    obstacle_id: str,
    obstacle_type: str,
    pose: list[float],
    scale: list[float] = [0.0, 0.0, 0.0],
    key: str = '',
    target_frame: str = 'world',
    ee_frame: str = 'ee_base',
    reference_joint_positions: list[float] = [],
    reference_base_pose: list[float] = [],
    ignore_collision_link_names: Sequence[str] = [],
    safe_margin: SupportsFloat = 0.0,
    resolution: SupportsFloat = 0.01
) -> MotionStatus
```

将碰撞对象加载到环境中。

将几何或基于网格的障碍物插入环境中以避免碰撞。障碍可以是静态的（世界坐标系下固定的）或与机器人相关的。支持原始形状、网格、点云和深度图像。

参数

名称	类型	默认值	描述
obstacle_id	str	需要传参	障碍物唯一标识符（场景中不得重复），后续可用于删除或更新。
obstacle_type	str	需要传参	障碍物几何类型
pose	list[float]	需要传参	障碍物位姿：[x, y, z, qx, qy, qz, qw]（米，四元数），相对于 target_frame。
scale	list[float]	[0.0, 0.0, 0.0]	几何尺寸（米）：- box: [长度, 宽度, 高度]- sphere: [半径, -, -]- cylinder: [半径, 高度, -]- mesh/point_cloud: 缩放因子
key	str	"	类型特定数据：- mesh/point_cloud: 文件路径（例如 "/path/to/model.stl"）- depth_image: 相机源数据- robot_state: 机器人状态数据
target_frame	str	'world'	位姿参考坐标系（默认 "world"），可为 "world"、"base_link" 或运动链名称（如 "left_arm"）。
ee_frame	str	'ee_base'	当 target_frame 为运动链

			名称时，指定链上参考帧（如 "ee_base"、"camera_base"、"camera_object"）。
reference_joint_positions	list[float]	[]	用于计算坐标变换的机器人关节状态（弧度）。为空时使用当前机器人状态。
reference_base_pose	list[float]	[]	地图坐标系下的机器人基座位姿：[x, y, z, qx, qy, qz, qw]。为空时使用当前定位。
ignore_collision_link_names	Sequence[str]	[]	忽略碰撞的连杆名称列表
safe_margin	SupportsFloat	0.0	安全距离缓冲（米）。当距离 < safe_margin 时判定碰撞，默认 0。
resolution	SupportsFloat	0.01	复杂几何体（mesh/point cloud/depth image）的离散化分辨率（米），默认 0.01。

返回值

类型	描述
MotionStatus	运动状态：- 成功：障碍物添加成功 - 无效输入：无效的障碍物 ID（重复）、类型或参数 - 故障：处理几何形状或添加到场景失败

注意

点云说明：point_cloud 指通过此 API 显式加载的点云障碍物（通常来自文件/离线数据）。它与导航系统维护的点云地图不同，GalbotMotion 不会自动订阅或与 galbotNav 的点云地图同步碰撞检测。

注意

障碍物会持续存在，直到显式删除或清空。

注意

对于移动障碍物，请在新位姿处删除并重新添加（当前无更新方法）。

警告

较大的 safe_margin（安全边距）值可能会过度约束规划空间，导致无法找到可行解，请谨慎使用。

附加目标对象

```
def attach_target_object(
    obstacle_id: str,
    obstacle_type: str,
    pose: list[float],
    scale: list[float] = [0.0, 0.0, 0.0],
    key: str = '',
    target_frame: str = 'world',
    ee_frame: str = 'ee_base',
    reference_joint_positions: list[float] = [],
    reference_base_pose: list[float] = [],
    ignore_collision_link_names: Sequence[str] = [],
    safe_margin: SupportsFloat = 0.0,
    resolution: SupportsFloat = 0.01
) -> MotionStatus
```

将碰撞物体附加到机器人上（例如，抓取的物体）。与 `add_obstacle()` 类似，但对象随机器人移动（连接到连杆/运动链），用于表示抓取的物体、传感器或有效载荷。

在运动过程中，对象相对于连接坐标系的姿态保持不变。

参数

名称	类型	默认值	描述
obstacle_id	str	需要传参	要移除的障碍物 ID
obstacle_type	str	需要传参	障碍物几何类型
pose	list[float]	需要传参	物体位姿：[x, y, z, qx, qy, qz, qw]（米，四元数），在附着时相对于 target_frame。
scale	list[float]	[0.0, 0.0, 0.0]	几何尺寸（米）：box 为 [length, width, height]；sphere 为 [radius, -, -]；cylinder 为 [radius, height, -]。
key	str	''	障碍物唯一标识符
target_frame	str	'world'	附着参考坐标系（默认 "world"）；抓取物体场景通常填写运动链名称（如 "left_arm"）。
ee_frame	str	'ee_base'	当 target_frame 为运动链名称时，指定链上参考帧（如 "ee_base"、"camera_base" 等）。
reference_joint_positions	list[float]	[]	用于计算附着变换的机器人关节状态（弧度）。为空时使用当前机器人状态。
reference_base_pose	list[float]	[]	地图坐标系下的机器人基

			座位姿：[x, y, z, qx, qy, qz, qw]。为空时使用当前定位。
ignore_collision_link_names	Sequence[str]	[]	忽略碰撞的连杆名称列表
safe_margin	SupportsFloat	0.0	安全距离缓冲（米），默认 0。
resolution	SupportsFloat	0.01	复杂几何体的离散化分辨率（米），默认 0.01。

返回值

类型	描述
MotionStatus	运动状态：SUCCESS（成功）、INVALID_INPUT（无效输入）或 FAULT（故障）

注意

点云说明：与 add_obstacle() 相同。此处的 point_cloud 是显式加载的点云对象，不会自动与任何导航端点云地图同步。

注意

附加的对象随机器人移动；其碰撞几何体自动更新。

注意

通常用于抓取和放置：抓取后附加目标对象，释放后分离。

警告

请确保 ignore_collision_link_names 包含抓取连杆，以避免误判为碰撞。

附加工具

```
def attach_tool(chain: str, tool: str) -> MotionStatus
```

将工具连接到末端执行器。

将工具（夹具、相机、定制末端执行器）加载到运动链上。更新运动学模型和碰撞几何体以包含该工具。

参数

名称	类型	默认值	描述
chain	str	需要传参	用于挂载工具的运动链（例如 left_arm、right_arm）。
tool	str	需要传参	工具信息

返回值

类型	描述
MotionStatus	运动状态： - 成功：附加工具成功 - 无效输入：无效的运动链或工具名称 - 故障：附加工具失败

注意

工具变换和碰撞几何体必须预先在机器人描述中配置。

注意

附加新工具会自动分离该运动链上先前附加的任何工具。

警告

运动学和碰撞检查将反映附加的工具；请相应更新计划。

检查碰撞

```
def check_collision(
    start: Sequence[RobotStates],
    enable_collision_check: bool = True,
    params: Parameter = ...
) -> tuple[MotionStatus, list[bool]]
```

检查机器人状态是否存在碰撞。

检查自身碰撞和与环境障碍物的碰撞，支持批量处理。因此，如果需要 Motion 考虑环境障碍物（包括点云），必须显式加载障碍物地图/对象（例如，obstacle_type = point_cloud 且在 key 中提供文件路径）。

注意：将实时感知集成到 galbotMotion 是未来计划，目前内部验证有限。

验证给定机器人配置是否无碰撞。

参数

名称	类型	默认值	描述
start	Sequence[RobotStates]	需要传参	起始机器人状态
enable_collision_check	bool	True	是否启用碰撞检查
params	Parameter	...	额外参数

返回值

类型	描述
tuple[MotionStatus, list[bool]]	元组（状态，碰撞结果）： - 状态：检测完成时为 MotionStatus::SUCCESS，否则为错误码- 碰撞结果：布尔向量（与 start 等长），true 表示检测到碰撞，false 表示无碰撞

注意

用于验证计划轨迹或基于采样的规划器。

注意

尊重先前添加的障碍物中的 `safe_margin`（安全边距）设置。

注意

尊重先前添加的障碍物中的 `safe_margin`（安全边距）设置。

清空障碍物

```
def clear_obstacle() -> MotionStatus
```

从规划场景中移除所有碰撞障碍物。

清除整个障碍物集，将规划场景重置为空（机器人几何体除外）。

返回值

类型	描述
MotionStatus	-

注意

附加的对象（参见 `attach_target_object`）不会被清除。

注意

即使场景已为空，也可以安全调用。

分离目标对象

```
def detach_target_object(obstacle_id: str) -> MotionStatus
```

从机器人上分离物体（例如，释放后）。从机器人上移除附着的物体。通常在释放抓取的对象后调用。

该对象被完全从规划场景中移除（不转换为静态障碍物）。

参数

名称	类型	默认值	描述
obstacle_id	str	需要传参	-

返回值

类型	描述
MotionStatus	-

注意

如需在移除后将对象作为静态障碍物保留在场景中，请使用 `clear_obstacle()`。

分离工具

```
def detach_tool(chain: str) -> MotionStatus
```

将当前工具与末端执行器分离。

相应地更新运动学模型和碰撞几何体。

参数

名称	类型	默认值	描述
chain	str	需要传参	要分离工具的运动链

返回值

类型	描述
MotionStatus	运动状态：- 成功：分离工具成功 - 无效输入：无效的运动链名称或未附加工具 - 故障：分离工具失败

注意

如果没有工具附加，则操作成功但无效果。

正向运动学

```
def forward_kinematics(  
    target_frame: str,  
    reference_frame: str = 'base_link',  
    joint_state: dict] = {},  
    params: Parameter = ...  
) -> tuple[MotionStatus, list[float]]
```

计算目标连杆的正向运动学。

计算给定关节配置的指定连杆的笛卡尔姿态。对于确定末端执行器位置、验证配置或计算中间连杆姿态很有用。

参数

名称	类型	默认值	描述
target_frame	str	需要传参	目标坐标系
reference_frame	str	'base_link'	参考坐标系
joint_state	dict]	{}	关节名到关节位置的映射；默认空字典表示使用当前机器人关节状态。
params	Parameter	...	额外参数

返回值

类型	描述
tuple[MotionStatus, list[float]]	元组（状态，位姿向量）：- 状态：成功时为 MotionStatus::SUCCESS，否则为错误码- 位姿向量：[x, y, z, qx, qy, qz, qw]（米，四元数），失败时为空

注意

关节角度以弧度为单位，输出位姿以米为单位的单位四元数表示。

警告

target_frame 必须是 URDF 模型中的有效连杆。

基于状态的正向运动学

```
def forward_kinematics_by_state(  
    target_frame: str,  
    reference_robot_states: RobotStates = None,  
    reference_frame: str = 'base_link',  
    params: Parameter = ...  
) -> tuple[MotionStatus, list[float]]
```

使用完整的机器人状态计算正向运动学。

与 forward_kinematics() 类似，但接受 RobotStates 对象来指定完整的机器人配置（全身关节+底座姿态）。

参数

名称	类型	默认值	描述
target_frame	str	需要传参	目标坐标系
reference_robot_states	RobotStates	None	参考机器人起始状态
reference_frame	str	'base_link'	参考坐标系
params	Parameter	...	额外参数

返回值

类型	描述
tuple[MotionStatus, list[float]]	元组（状态，位姿向量）：- 状态：成功时为 MotionStatus::SUCCESS，否则为错误码- 位姿向量：[x, y, z, qx, qy, qz, qw]（米，四元数），失败时为空

注意

在不修改实际状态的情况下为假设状态计算正向运动学时非常有用。

获取已构建障碍物列表

```
def get_built_obstacles_list() -> list[str]
```


获取当前加载的障碍物 ID 列表。

返回值

类型	描述
list[str]	-

获取链关节状态

```
def get_chain_joint_state() -> dict[str, list[float]]
```

获取所有运动链的当前关节配置。

检索每条链的关节状态，将整体配置分解为各个链的贡献。

返回值

类型	描述
dict[str, list[float]]	-

注意

关节向量大小因链的自由度而异。

获取末端执行器姿态

```
def get_end_effector_pose(
    end_effector_frame: str,
    reference_frame: str = 'base_link'
) -> tuple[MotionStatus, list[float]]
```

从机器人状态获取当前末端执行器姿态。

查询 TF（变换）树以检索指定末端执行器连杆的当前笛卡尔姿态。需要在机器人的 URDF 模型中定义该连杆。

参数

名称	类型	默认值	描述
end_effector_frame	str	需要传参	末端执行器坐标系
reference_frame	str	'base_link'	参考坐标系

返回值

类型	描述
tuple[MotionStatus, list[float]]	元组 (状态, 位姿向量): - 状态: 成功时为 MotionStatus::SUCCESS, 错误码说明: - DATA_FETCH_FAILED: TF 查找失败 - INVALID_INPUT: 无效的帧名称- 位姿向量: [x, y, z, qx, qy, qz, qw] (单位: 米, 四元数), 失败时为空

注意

反映当前实际的机器人状态（非计划状态）。

警告

需要 TF 树正确发布且为最新状态。

获取指定链末端执行器姿态

```
def get_end_effector_pose_on_chain(
    chain_name: str,
    frame_id: str = 'EndEffector',
    reference_frame: str = 'base_link'
) -> tuple[MotionStatus, list[float]]
```

获取特定运动链的当前末端执行器姿态。

通过链名称和坐标系类型检索末端执行器姿态的便捷方法，无需知道 URDF 中的确切连杆名称。

参数

名称	类型	默认值	描述
chain_name	str	需要传参	链名称
frame_id	str	'EndEffector'	坐标系 ID
reference_frame	str	'base_link'	参考坐标系

返回值

类型	描述
tuple[MotionStatus, list[float]]	元组 (状态, 位姿向量)：状态（成功时为 MotionStatus::SUCCESS，否则为错误码），位姿向量 [x, y, z, qx, qy, qz, qw]（单位：米，四元数），失败时为空

注意

内部将 chain_name + frame_id 映射到实际的 URDF 连杆名称。

获取雅可比矩阵

```
def get_jacobian(
    chain_name: str,
    target_frame: str = 'EndEffector',
    reference_frame: str = 'base_link',
    joint_state: dict = {},
    params: Parameter = ...
) -> tuple[MotionStatus, list[list[float]]]
```

计算指定运动链的雅可比矩阵

参数

名称	类型	默认值	描述
chain_name	str	需要传参	链名称
target_frame	str	'EndEffector'	计算雅可比的目标连杆： "EndEffector"（法兰盘）或 "Tool"（工具中心点）。默 认："EndEffector"
reference_frame	str	'base_link'	参考坐标系："base_link" （机器人基座坐标系）或 "world"（世界坐标系）。 默认："base_link"
joint_state	dict]	{}	关节状态对象，包含当前 关节位置
params	Parameter	...	运动学参数配置

返回值

类型	描述
tuple[MotionStatus, list[list[float]]]	返回 numpy 数组，形状为 [6 x DOF]，表示末端执行器 在参考坐标系下的雅可比矩阵

注意

返回雅可比矩阵，形状为 [6 x DOF]，表示关节空间到任务空间的线性映射

注意

用于运动学分析、力控、奇异值分解等场景

通过状态获取雅可比矩阵

```
def get_jacobian_by_state(
    chain_name: str,
    target_frame: str = 'EndEffector',
    reference_frame: str = 'base_link',
    reference_robot_states: RobotStates = None,
    params: Parameter = ...
) -> tuple[MotionStatus, list[list[float]]]
```

通过给定的机器人状态计算指定运动链的雅可比矩阵

参数

名称	类型	默认值	描述
chain_name	str	需要传参	链名称
target_frame	str	'EndEffector'	计算雅可比的目标连杆。 默认："EndEffector"
reference_frame	str	'base_link'	参考坐标系。默认： "base_link"
reference_robot_states	RobotStates	None	机器人状态，包含关节位

			置信息
params	Parameter	...	运动学参数配置

返回值

类型	描述
tuple[MotionStatus, list[list[float]]]	元组（状态，雅可比矩阵）：- 状态：成功时为 MotionStatus::SUCCESS，否则为错误码- 雅可比矩阵：6xN 矩阵（N 为链自由度），失败时为空

注意

返回雅可比矩阵，形状为 [6 x DOF]

获取连杆名称

```
def get_link_names(only_end_effector: bool = False) -> list[str]
```

从运动学模型中获取机器人连杆名称。
检索机器人 URDF 模型中定义的连杆名称列表，可用于过滤末端执行器连杆或返回所有连杆。

参数

名称	类型	默认值	描述
only_end_effector	bool	False	是否仅返回末端执行器连杆，true 仅返回末端执行器/工具连杆，false 返回所有连杆（包括底座、中间连杆等），默认 false（所有连杆）

返回值

类型	描述
list[str]	连杆名称字符串向量（如果检索失败则为空）

注意

基于连杆没有子连杆来检测末端执行器。

注意

用于正向运动学查询或 TF 帧验证。

获取运动规划配置

```
def get_motion_plan_config() -> tuple[MotionStatus, MotionPlanConfig]
```

获取当前运动规划配置。

检索活动规划器配置，包括速度/加速度限制和规划算法参数。

返回值

类型	描述
tuple[MotionStatus, MotionPlanConfig]	-

注意

用于检查当前限制或保存/恢复配置。

获取机器人状态

```
def get_robot_states() -> RobotStates
```

获取当前完整的机器人状态。

检索当前全身关节配置和移动底座姿态。代表机器人的完整运动状态。

返回值

类型	描述
RobotStates	-

注意

反映实际机器人状态（来自传感器反馈/状态估计）。

注意

用作规划操作的种子/参考。

获取支持的链

```
def get_supported_chains() -> set[str]
```

获取支持的运动链名称列表（例如 left_arm、right_arm）。

返回值

类型	描述
set[str]	-

获取支持的末端执行器坐标系

```
def get_supported_ee_frames() -> set[str]
```

获取支持的末端执行器坐标系标识符集。

返回值

类型	描述
set[str]	-

获取支持的坐标系

```
def get_supported_frames() -> set[str]
```

获取支持的参考坐标系名称集。

返回值

类型	描述
set[str]	-

获取支持的连杆

```
def get_supported_links() -> set[str]
```

获取支持的连杆名称列表（正向运动学/逆向运动学的 URDF 连杆名称）。

返回值

类型	描述
set[str]	-

获取支持的障碍物类型

```
def get_supported_obstacle_types() -> set[str]
```

获取支持的障碍物类型集（例如盒子、球体、圆柱体、网格）。

返回值

类型	描述
set[str]	-

获取支持的工具列表

```
def get_supported_tool_list() -> set[str]
```

获取 attach_tool 支持的工具名称列表。

返回值

类型	描述
set[str]	-

初始化

```
def init() -> bool
```

初始化运动规划系统和通信接口。

必须在任何其他 API 函数之前调用。初始化内部通信中间件，加载机器人运动学模型，并建立与控制服务的连接。

返回值

类型	描述
bool	-

注意

可安全多次调用；成功后的后续调用返回第一次初始化的结果。

警告

如果 `init()` 返回 `false`，则所有其他 API 调用将失败。

逆向运动学

```
def inverse_kinematics(  
    target_pose: list[float],  
    chain_names: Sequence[str],  
    target_frame: str = 'EndEffector',  
    reference_frame: str = 'base_link',  
    initial_joint_positions: dict = {},  
    enable_collision_check: bool = True,  
    params: Parameter = ...  
) -> tuple[MotionStatus, dict[str, list[float]]]
```

计算目标笛卡尔姿态的逆运动学。

求解实现指定末端执行器姿态的关节配置。支持单链 IK（仅手臂）或协调多链 IK（手臂 + 躯干/腿）。

参数

名称	类型	默认值	描述
target_pose	list[float]	需要传参	目标笛卡尔位姿：[x, y, z, qx, qy, qz, qw]（米，四元数）。
chain_names	Sequence[str]	需要传参	要解决逆运动学的运动链名称列表
target_frame	str	'EndEffector'	目标坐标系
reference_frame	str	'base_link'	参考坐标系
initial_joint_positions	dict	{}	初始关节位置猜测
enable_collision_check	bool	True	是否启用碰撞检查

params	Parameter	...	额外参数
--------	-----------	-----	------

返回值

类型	描述
tuple[MotionStatus, dict[str, list[float]]]	元组（状态，解映射）：- 状态：若可求解则为 MotionStatus::SUCCESS，否则为错误码- 解映射：{chain_name -> joint_angles}（弧度），失败时空

注意

IK 可能有多个解；返回第一个有效的解。

注意

种子配置影响收敛速度和哪个解被选中。

警告

如果目标在工作空间之外或处于奇异配置，则无法保证有解。

基于状态的逆向运动学

```
def inverse_kinematics_by_state(
    target_pose: list[float],
    chain_names: Sequence[str],
    target_frame: str = 'EndEffector',
    reference_frame: str = 'base_link',
    reference_robot_states: RobotStates = None,
    enable_collision_check: bool = True,
    params: Parameter = ...
) -> tuple[MotionStatus, dict[str, list[float]]]
```

使用完整的机器人状态作为种子计算逆运动学。

与 inverse_kinematics() 类似，但接受 RobotStates 来指定种子配置，从而允许精确控制整个机器人状态。

参数

名称	类型	默认值	描述
target_pose	list[float]	需要传参	目标笛卡尔位姿：[x, y, z, qx, qy, qz, qw]（米，四元数）。
chain_names	Sequence[str]	需要传参	要解决逆运动学的运动链名称列表
target_frame	str	'EndEffector'	目标坐标系
reference_frame	str	'base_link'	参考坐标系
reference_robot_states	RobotStates	None	参考机器人起始状态
enable_collision_check	bool	True	是否启用碰撞检查

params	Parameter	...	额外参数
--------	-----------	-----	------

返回值

类型	描述
tuple[MotionStatus, dict[str, list[float]]]	元组（状态，解映射）：- 状态：若可求解则为 MotionStatus::SUCCESS，否则为错误码- 解映射：{chain_name -> joint_angles}（弧度），失败时为空

注意

在不修改实际状态的情况下为假设状态离线规划时非常有用。

轨迹规划

```
def motion_plan(
    target: RobotStates,
    start: RobotStates = None,
    reference_robot_states: RobotStates = None,
    enable_collision_check: bool = True,
    params: Parameter = ...
) -> tuple[MotionStatus, dict[str, list[list[float]]]]
```

规划单个运动链的轨迹。

参数

名称	类型	默认值	描述
target	RobotStates	需要传参	目标位姿
start	RobotStates	None	起始机器人状态
reference_robot_states	RobotStates	None	参考机器人起始状态
enable_collision_check	bool	True	是否启用碰撞检查
params	Parameter	...	额外参数

返回值

类型	描述
tuple[MotionStatus, dict[str, list[list[float]]]]	元组（状态，轨迹映射）：- 状态：规划成功时为 MotionStatus::SUCCESS，否则为错误码- 轨迹映射：{chain_name -> waypoint_list}，其中 waypoint_list 为关节配置序列（弧度）

注意

碰撞语义：GalbotMotion 不提供实时障碍物订阅；规划期间仅考虑通过 API 添加的障碍物。

注意

轨迹时间参数化，具有速度/加速度约束。

注意

对于直接执行 (params->is_direct_execute=true) , 轨迹会自动下发到机器人执行。

警告

target 必须是 PoseState 或 JointStates; 传递 base RobotStates 将导致 INVALID_INPUT 错误。

多路点轨迹规划

```
def motion_plan_multi_waypoints(  
    target: RobotStates,  
    waypoint_poses: Sequence[list[float]],  
    start: RobotStates = None,  
    reference_robot_states: RobotStates = None,  
    enable_collision_check: bool = True,  
    params: Parameter = ...  
) -> tuple[MotionStatus, dict[str, list[list[float]]]]
```

通过多个链的路径点规划协调轨迹。

通过路径点序列实现协调的多臂或全身运动。每个链都可以有自己的路点序列，以同步方式执行。

参数

名称	类型	默认值	描述
target	RobotStates	需要传参	目标位姿
waypoint_poses	Sequence[list[float]]	需要传参	路点位姿列表
start	RobotStates	None	起始机器人状态
reference_robot_states	RobotStates	None	参考机器人起始状态
enable_collision_check	bool	True	是否启用碰撞检查
params	Parameter	...	额外参数

返回值

类型	描述
tuple[MotionStatus, dict[str, list[list[float]]]]	运动规划成功返回 true，失败返回 false。

注意

所有链的轨迹在时间上同步以实现协调运动。

注意

用于双臂操作或移动操作任务。

多路点轨迹规划

```
def motion_plan_multi_waypoints(  
    targets: dict[],  
    start: Sequence[RobotStates] = [],  
    reference_robot_states: RobotStates = None,  
    enable_collision_check: bool = True,
```

```
    params: Parameter = ...
) -> tuple[MotionStatus, dict[str, list[list[float]]]]
```

通过多个链的路径点规划协调轨迹。

通过路径点序列实现协调的多臂或全身运动。每个链都可以有自己的路点序列，以同步方式执行。

参数

名称	类型	默认值	描述
targets	dict[]	需要传参	多路点目标位姿字典（每个运动链对应一个路点序列）
start	Sequence[RobotStates]	[]	起始机器人状态
reference_robot_states	RobotStates	None	参考机器人起始状态
enable_collision_check	bool	True	是否启用碰撞检查
params	Parameter	...	额外参数

返回值

类型	描述
tuple[MotionStatus, dict[str, list[list[float]]]]	元组（状态，轨迹映射）：- 状态：规划成功时为 MotionStatus::SUCCESS，否则为错误码- 轨迹映射：{chain_name -> waypoint_list}（所有运动链）

注意

所有链的轨迹在时间上同步以实现协调运动。

注意

用于双臂操作或移动操作任务。

移动全身关节到零位

```
def move_whole_body_joint_zero(
    is_blocking: bool = True,
    leg_head_speed_rad_s: SupportsFloat = 0.2,
    leg_head_timeout_s: SupportsFloat = 15.0,
    params: Parameter = ...
) -> MotionStatus
```

将全身关节移动到预定义的零（家庭）配置。

腿部和头部关节通过 GalbotRobot（直接关节控制）进行控制，而左/右臂则通过启用碰撞检查的运动规划器进行规划。

零配置的联合顺序遵循 SDK 约定：leg(5) + head(2) + left_arm(7) + right_arm(7)。

参数

名称	类型	默认值	描述
is_blocking	bool	True	-
leg_head_speed_rad_s	SupportsFloat	0.2	-
leg_head_timeout_s	SupportsFloat	15.0	-
params	Parameter	...	-

返回值

类型	描述
MotionStatus	-

移除障碍物

```
def remove_obstacle(obstacle_id: str) -> MotionStatus
```

从规划场景中移除碰撞障碍物。

参数

名称	类型	默认值	描述
obstacle_id	str	需要传参	-

返回值

类型	描述
MotionStatus	-

注意

移除不存在的障碍物返回 INVALID_INPUT（非 NO_ERROR）。

设置末端执行器姿态

```
def set_end_effector_pose(
    target_pose: list[float],
    end_effector_frame: str,
    reference_frame: str = 'base_link',
    reference_robot_states: RobotStates = None,
    enable_collision_check: bool = True,
    is_blocking: bool = True,
    timeout: SupportsFloat = -1.0,
    params: Parameter = ...
) -> MotionStatus
```

命令末端执行器移动到目标笛卡尔姿态。

笛卡尔运动命令的高级接口。在内部执行 IK、规划轨迹并可选择执行运动。支持阻塞（等待完成）和非阻塞（立即返回）模式。

参数

名称	类型	默认值	描述
target_pose	list[float]	需要传参	目标笛卡尔位姿：[x, y, z, qx, qy, qz, qw]（米，四元数）。
end_effector_frame	str	需要传参	末端执行器坐标系
reference_frame	str	'base_link'	参考坐标系
reference_robot_states	RobotStates	None	参考机器人起始状态
enable_collision_check	bool	True	是否启用碰撞检查
is_blocking	bool	True	是否阻塞等待完成
timeout	SupportsFloat	-1.0	阻塞模式超时（秒）。当 timeout < 0 且 is_blocking=true 时，使用 params->timeout_second。
params	Parameter	...	额外参数

返回值

类型	描述
MotionStatus	MotionStatus：- SUCCESS：阻塞模式下表示运动完成；非阻塞模式下表示命令发送成功。- TIMEOUT：运动超过超时时间。- INVALID_INPUT：输入参数无效。- FAULT：规划或执行失败。

注意

运动类型（直线/关节空间）由 params->move_line 控制。

注意

对于直接执行（params->is_direct_execute=true），请避免传入 reference_robot_states。

警告

阻塞调用将暂停执行直到运动完成；建议在非阻塞模式下使用进度监控。

设置运动规划配置

```
def set_motion_plan_config(config: MotionPlanConfig) -> MotionStatus
```

设置全局运动规划配置。

更新规划器设置，例如速度/加速度限制、规划算法参数和优化目标。影响后续所有的计划操作。

参数

名称	类型	默认值	描述
----	----	-----	----

config	MotionPlanConfig	需要传参	-
--------	------------------	------	---

返回值

类型	描述
MotionStatus	-

注意

更改会持续存在，直到显式重置或进程重启。

注意

有关可用参数，请参阅 MotionPlanConfig 文档。

状态转字符串

```
def status_to_string(status: MotionStatus) -> str
```

将 MotionStatus 枚举转换为人类可读的字符串。

将状态代码映射到用于日志记录、错误报告或 UI 显示的描述性字符串。

参数

名称	类型	默认值	描述
status	MotionStatus	需要传参	-

返回值

类型	描述
str	-

注意

使用 status_string_map_ 进行查找；如果状态未知则返回 UNKNOWN。

移动导航

移动机器人底盘导航与定位接口。

该类提供线程安全的单例接口，用于控制移动底盘导航系统。支持二维位姿估计、重定位、带动态避障的目标导航与路径规划。导航系统在全局地图坐标系下工作，支持阻塞与非阻塞两种导航模式，并兼容差分驱动和全向底盘。除非另有说明，所有位姿参数均以地图坐标系表示。

检查目标到达

```
def check_goal_arrival() -> bool
```

检查机器人是否成功达到当前目标。

该方法查询导航系统以确定机器人是否已在可接受的位置和方向公差内到达目标位姿。当使用非阻塞导航模式轮询完成情况时，这特别有用。

返回值

类型	描述
bool	若机器人在容差阈值内到达目标则返回 true；若仍在导航、无活动目标或尚未到达则返回 false。

注意

在非阻塞导航场景中最有用。

注意

到达的公差阈值由导航模块的内部参数定义（通常在 YAML 配置文件中设置）。

注意

如果没有激活的导航命令，此方法返回 false。

检查路径可达性

```
def check_path_reachability(goal_pose: numpy.ArrayLike, start_pose: numpy.ArrayLike) -> bool
```

检查地图中是否存在从起点到目标的无碰撞路径。

此方法查询全局路径规划器以确定指定的起始姿态和目标姿态之间是否存在有效的无碰撞路径。这对于在尝试导航之前验证目标姿态或多目标路径规划非常有用。

参数

名称	类型	默认值	描述
goal_pose	numpy.ArrayLike	需要传参	目标位姿（地图坐标系）
start_pose	numpy.ArrayLike	需要传参	起始位姿

返回值

类型	描述
bool	若起点到目标存在无碰撞路径则返回 true；若未找到有效路径则返回 false。

注意

此方法仅基于地图检查静态障碍物。

注意

路径计算可能根据距离需要一些时间。

注意

返回 true 并不保证成功导航。

获取当前姿态

```
def get_current_pose() -> list[float]
```

获取地图坐标系中机器人底盘的当前估计位姿。

该方法返回来自定位系统的最新姿态估计。位姿表示机器人的 base_link 坐标系相对于地图坐标系原点的位置和方向。

返回值

类型	描述
list[float]	结构包含以下字段： - x: X 坐标（米，地图坐标系） - y: Y 坐标（米，地图坐标系） - theta: 方向（弧度，地图坐标系，逆时针从 x 轴算起）

注意

仅当 is_localized() 返回 true 时返回的位姿才有效。

注意

该位姿表示机器人底盘接地轮廓（base footprint）的中心。

获取导航状态

```
def get_navigation_status() -> NavigationTaskStatus
```

获取当前导航任务状态。

返回导航系统报告的最新任务状态（未知、正在运行、成功或失败）。在运行非阻塞导航时使用此功能可以轮询状态并在失败或超时时及时退出错误逻辑，从而避免死锁或无限期等待。

返回值

类型	描述
NavigationTaskStatus	导航任务状态 当前任务状态。如果尚无状态则为 UNKNOWN；导航时为 RUNNING；任务完成时为 SUCCESS 或 FAILED。

注意

在非阻塞导航中很有用：在 `get_navigation_status()` 上循环检查进度。

初始化

```
def init() -> bool
```

初始化导航子系统及其依赖项。

在使用任何其他导航功能之前必须调用此方法。它初始化通信通道、加载地图、启动定位模块并准备路径规划器。

返回值

类型	描述
bool	初始化成功返回 true，否则返回 false。

注意

此方法应在获取单例实例后仅调用一次。

注意

后续调用将返回第一次初始化的结果。

警告

在成功初始化前调用导航方法将导致错误。

检查是否已定位

```
def is_localized() -> bool
```

检查机器人当前是否在地图中定位。

该方法查询定位系统以确定机器人是否具有足够置信度的有效姿态估计。未定位的机器人不应执行导航任务。

返回值

类型	描述
bool	若机器人已可靠定位则返回 true；若定位丢失或不确定则返回 false。

注意

建议在发出导航命令前检查定位状态。

注意

如果返回 false，请考虑调用 `relocalize()` 并提供初始猜测。

直线移动到目标

```
def move_straight_to(
    goal_pose: numpy.ArrayLike,
    is_blocking: bool = True,
    timeout: SupportsFloat = 8
) -> tuple
```

将机器人移动到里程计坐标系中的相对目标位姿。

此方法命令机器人移动到相对于其在里程计 (odom) 坐标系中的当前位置指定的姿态。这对于不需要基于地图的规划的短距离、精确的运动非常有用。与 `navigate_to_goal()` 不同，此方法不执行动态障碍物检测或全局路径规划。它使用全向运动规划来直接移动到目标。

参数

名称	类型	默认值	描述
goal_pose	numpy.ArrayLike	需要传参	相对于当前机器人底座帧的目标位姿。包含：- x: 前后位移（米）- y: 左右位移（米）- theta: 转向角度（弧度）
is_blocking	bool	True	执行模式标志。- true (阻塞): 阻塞直到运动完成、失败或超时- false (非阻塞): 发送导航命令后立即返回
timeout	SupportsFloat	8	阻塞模式最大等待时间（秒），默认 8.0 秒；仅在 is_blocking 为 true 时生效。

返回值

类型	描述
tuple	导航状态指示结果：- 非阻塞模式：命令接受状态 - 阻塞模式：最终运动结果（成功、失败、超时）

注意

此方法不检查障碍物或碰撞。请先检查路径可达性。

注意

此方法使用里程计帧，不需要地图帧。

注意

适用于小的精确调整，如最终接近。

警告

由于禁用了碰撞检查，请确保路径无障。

警告

长距离下里程计漂移可能影响精度。建议缩短单次移动距离或定期重定位。

导航到目标

```
def navigate_to_goal(  
    goal_pose: numpy.ArrayLike,  
    enable_collision_check: bool = True,  
    is_blocking: bool = False,  
    timeout: SupportsFloat = 8,  
    omni_plan: bool = True  
) -> tuple
```

将机器人导航至地图坐标系中的目标位姿。

该方法命令移动底座使用全局路径规划器和局部轨迹控制器导航到指定的目标位姿。如果启用了碰撞检查，规划器将计算从当前姿态到目标的无碰撞路径，同时考虑静态地图障碍物和动态障碍物。

参数

名称	类型	默认值	描述
goal_pose	numpy.ArrayLike	需要传参	目标位姿（地图坐标系）
enable_collision_check	bool	True	是否启用碰撞检查
is_blocking	bool	False	执行模式标志。- true（阻塞）：阻塞直到到达目标、失败或超时。- false（非阻塞）：发送导航命令后立即返回。
timeout	SupportsFloat	8	阻塞模式下的最大等待时间（秒），默认值：8.0秒，仅在 is_blocking 为 true 时相关，如果在时间内未到达目标，则返回超时状态
omni_plan	bool	True	运动规划模式标志。- true: 启用全向运动规划（各向同性移动）- false: 使用有限方向的运动规划

返回值

类型	描述
tuple	导航状态指示结果：- 非阻塞模式：命令接受状态 - 阻塞模式：最终导航结果（成功、失败、超时）

注意

机器人必须先定位（`is_localized()` 返回 `true`），才能开始导航。

注意

对于阻塞模式，调用线程将阻塞直到导航完成或超时。

注意

实际导航时间可能超过超时值（阻塞模式下）。

警告

长距离导航可能需要较长时间；建议使用非阻塞模式或监控进度。

重定位

```
def relocate(init_pose: numpy.ArrayLike) -> tuple
```

执行重新定位以重新估计机器人在地图坐标系中的姿态。

该方法重置定位滤波器并提供初始姿态估计，以帮助机器人在已知地图中重新建立其位置。当机器人失去定位或手动将机器人放置在已知位置时，这非常有用。

参数

名称	类型	默认值	描述
<code>init_pose</code>	<code>numpy.ArrayLike</code>	需要传参	初始姿态估计

返回值

类型	描述
<code>tuple</code>	导航状态，指示重定位请求的结果。详见 <code>NavigationStatus</code> 枚举的取值说明。

注意

重定位时机器人应保持静止以获得最佳效果。

注意

调用此方法后，使用 `is_localized()` 验证成功。

停止导航

```
def stop_navigation() -> tuple
```

停止当前的导航任务并使机器人停止。

此方法立即取消任何正在进行的导航命令（来自 `navigate_to_goal()`或 `move_straight_to()`）并命令机器人停止。机器人将根据其运动学约束减速并安全停止。

返回值

类型	描述
tuple	导航状态指示是否成功将停止命令发送到导航系统。

注意

可在导航期间随时调用。

注意

停止后，机器人的位置可能与原始位置不同。

注意

机器人将尝试根据其加速度限制平滑停止。

感知模块接口

感知模块接口；通过 `get_instance(MachineType)` 获取单例实例。

G1 平台专用：`get_instance(MachineType::S1)` 会抛出 `std::runtime_error`。

获取模块的最新缓存结果

```
def get_latest_result(module: PerceptionModule) -> tuple
```

返回模块的最新缓存结果，不阻塞。

参数

名称	类型	默认值	描述
module	PerceptionModule	需要传参	感知模块。

返回值

类型	描述
tuple	如果有结果可用返回 <code>true</code> ，否则返回 <code>false</code> 。

初始化感知模块并加载模型

```
def init(enabled_modules: Set[PerceptionModule]) -> bool
```

初始化感知模块并加载指定模块的模型。

参数

名称	类型	默认值	描述
enabled_modules	Set[PerceptionModule]	需要传参	要启用的感知模块集合。

返回值

类型	描述
bool	如果所有请求的模块都成功加载则返回 true。

为指定模块运行一次推理

```
def run_once(module: PerceptionModule) -> bool
```

为指定模块运行一次推理。

参数

名称	类型	默认值	描述
module	PerceptionModule	需要传参	感知模块。

返回值

类型	描述
bool	成功返回 true，失败返回 false。

注意

初始化后，等待约 10 秒让模型准备就绪后再调用 run_once。

等待模块产生新结果或超时

```
def wait_for_new_result(module: PerceptionModule, timeout_s: SupportsFloat = 5.0) -> bool
```

阻塞直到模块产生新结果或超时。与 run_once 配合使用获取最新输出。

参数

名称	类型	默认值	描述
module	PerceptionModule	需要传参	感知模块。
timeout_s	SupportsFloat	5.0	超时时间（秒）。

返回值

类型	描述
bool	成功返回 true，超时返回 false。

类型与枚举

音频数据

音频数据结构用于封装音频数据。

Member Variables

Name	Type	Description
data	list[int]	二进制数据包，具体格式由 format 字段指定。- "pcm" 格式: PCM 音频数据（小端序，16 位整数）- "json" 格式: UTF-8 编码的 JSON 文本
format	str	音频数据格式说明，例如: "pcm": 采样率 16000 Hz，位深度 16 bit，单声道; "json": UTF-8 编码的 JSON 文本
header	Header	消息头
type	str	音频数据类型标识符。- "waken_up": 唤醒事件（格式为 JSON）- "speech": 语音数据（格式为 PCM）

碰撞检查选项

碰撞检测启用/禁用配置。

该结构在运动规划和执行期间提供对碰撞检查的细粒度控制。它支持独立切换自碰撞检测（机器人连杆相互碰撞）和环境碰撞检测（机器人与障碍物或工作空间边界碰撞）。禁用碰撞检查可以提高计算性能，但可能会导致不安全的轨迹。在受控环境中谨慎使用。

检查环境碰撞检测是否禁用

```
def get_disable_env_collision_check() -> bool
```

检查环境碰撞检测是否禁用

返回值

类型	描述
bool	-

检查自碰撞检测是否禁用

```
def get_disable_self_collision_check() -> bool
```

检查自碰撞检测是否禁用

返回值

类型	描述
bool	-

打印碰撞检测配置到标准输出

```
def print() -> None
```

打印碰撞检测配置到标准输出

启用或禁用环境碰撞检测

```
def set_disable_env_collision_check(disable: bool) -> None
```

设置是否禁用环境碰撞检测

参数

名称	类型	默认值	描述
disable	bool	需要传参	-

警告

禁用环境检查可能导致与障碍物发生碰撞。

启用或禁用自碰撞检测

```
def set_disable_self_collision_check(disable: bool) -> None
```

true 禁用自碰撞检测，false 启用

参数

名称	类型	默认值	描述
disable	bool	需要传参	-

警告

禁用自碰撞检查可能导致物理上不可行的配置。

控制状态

控制命令执行状态枚举。

表示机器人控制命令的执行状态，包括关节控制、末端执行器控制等运动控制操作。

Enum Value	Description
COMM_DISCONNECTED	通信断开

DATA_FETCH_FAILED	数据获取失败
FAULT	故障
INIT_FAILED	初始化失败
INVALID_INPUT	无效输入
IN_PROGRESS	进行中
PUBLISH_FAIL	发布失败
STOPPED_UNREACHED	已停止但未到达
SUCCESS	成功
TIMEOUT	超时

深度数据

包含来自深度相机或 RGB-D 传感器的压缩深度图像数据。与 ROS 2 的 `sensor_msgs/compressedImage` 消息类型兼容（支持深度扩展）。

Member Variables

Name	Type	Description
data	list[int]	包含原始或压缩深度图像数据的二进制数据块
depth_scale	int	深度缩放因子，用于将像素值转换为实际深度（米）， $\text{真实深度} = \text{像素值} / \text{depth_scale}$ ，例如： $\text{depth_scale} = 1000$ 表示像素值单位为毫米
format	str	指定深度编码和压缩格式，例如： "16UC1; compressedDepth png" (16 位无符号整数，PNG 压缩深度)
header	Header	包含采集时间戳和相机坐标系
height	int	深度图像的行数
width	int	深度图像的列数

检测与分割结果

单对象检测或实例分割结果（2D 框、类别、可选掩码/关键点）。

Member Variables

Name	Type	Description
bbox	tuple[int, int, int, int]	Bounding box as (x, y, width, height)
class_index	int	Class index

class_name	str	Class name
confidence	float	Confidence score
keypoints	list[tuple[float, float]]	Keypoints as list of (x, y) tuples

检测结果

单个模块周期的聚合感知输出（图像、掩码、姿态、点云等）。

Member Variables

Name	Type	Description
bounding_boxes	list[tuple[int, int, int, int]]	Bounding boxes as list of (x, y, width, height)
class_indices	list[int]	List of class indices
class_names	list[str]	List of class names
confidences	list[float]	List of confidences
detection_results	list[DetectionAndSegmentationResult]	List of DetectionAndSegmentationResult
grasp_pose_result	list[list[float]]	Grasp pose results
instance_mask	Any	Instance mask as numpy array (HxW or HxWxC), or None if empty
ocr_string	list[str]	OCR results
point_clouds	list	Point clouds as list of Nx3 numpy arrays
running_info	str	Running info string
sensor_name	str	Sensor name
target_point_poses	list[numpy.ndarray[numpy.float32]]	4x4 poses from perception proto field target_point_poses (same buffer as target_poses here)
target_poses	list[numpy.ndarray[numpy.float32]]	List of 4x4 target pose matrices (C++ targetPoses; perception proto target_point_poses fills this)
timestamp_ns	int	Timestamp in nanoseconds

清空结果

```
def clear() -> None
```

清空所有存储的结果。

获取结果信息

```
def get_result_info() -> str
```

获取结果摘要字符串。

返回值

类型	描述
str	-

灵巧手状态

包含带时间戳的关节反馈；并在可用时（例如 Sharpa）包含来自灵巧手力反馈话题的逐传感器力/力矩测量。Sharpa 每只手按 22 关节建模。

Member Variables

Name	Type	Description
force_sensor_map	dict[str, EffortInfo]	命名力传感器映射（Sharpa；其他为空）
joint_state	JointStateMessage	灵巧手关节状态消息
timestamp_ns	int	状态时间戳（自纪元以来的纳秒数）

灵巧手类型

SDK 使用此枚举将灵巧手命令和状态查询路由到正确的实现。Inspire 和 BrainCo 灵巧手共享标准关节命令/状态路径。Sharpa 灵巧手使用专用的 22 关节话题接口；完整状态在 DexhandState 中返回（包括力传感器）。

Enum Value	Description
BRAINCO	BrainCo 灵巧手
INSPIRE	Inspire 灵巧手
SHARPA	Sharpa 灵巧手

力矩信息

表示通常由力/扭矩传感器测量的 6 自由度 (6-DOF) 力螺旋（力和扭矩）。也称为空间力或广义力。

Member Variables

Name	Type	Description
force	Vector3	力向量（牛顿）：[fx, fy, fz]- fx: X 方向力- fy: Y 方向力- fz: Z 方向力
timestamp_ns	int	测量时间戳（自纪元以来的纳秒数）
torque	Vector3	力矩向量（牛顿·米）：[tx, ty, tz]- tx: 绕 X 轴力矩- ty: 绕 Y 轴力矩- tz: 绕 Z 轴力矩

错误

描述单个模块或组件的错误，包括错误代码和用于调试和诊断的人类可读描述。

Member Variables

Name	Type	Description
commpent	str	组件
description	str	错误描述
error_code	int	用于程序化错误处理的数值错误代码

错误信息

包含来自多个模块或组件的带时间戳的错误消息集合。

Member Variables

Name	Type	Description
error_vec	list[Error]	误差向量
timestamp_ns	int	收集错误的时间戳（自纪元以来的纳秒数）

力数据

包含来自 6 轴力/扭矩传感器的带有时间戳的力和扭矩测量值，通常安装在机器人手腕或工具端接口处。

Member Variables

Name	Type	Description
force	Vector3	力
timestamp_ns	int	测量时间戳（自纪元以来的纳秒数）
torque	Vector3	力矩向量（牛顿·米）：[tx, ty, tz]

坐标系

表示坐标系三个正交轴的可视化表示，通常用于显示坐标系的方向和姿态。

Member Variables

Name	Type	Description
body_frame_id	str	Body frame id

header	Header	Message header
pose	Pose	None
reference_frame_id	str	Reference frame id
twist	Twist	None
wrench	Wrench	None

G1 控制器名称

G1 控制器名称的字符串常量。

定义 G1 机器人型号支持的控制器名称。

Enum Value	Description
CHASSIS_POSE_CTRL	底盘姿态控制器
CHASSIS_TWIST_CTRL	底盘速度控制器
CONTROLLER_NAME_NUM	控制器名称数量
HEAD_PVT_BYPASS_CTRL	头部位置-速度-力矩旁路控制器
HEAD_PVT_CTRL	头部位置-速度-力矩控制器
LEFT_ARM_PVT_BYPASS_CTRL	左臂位置-速度-力矩旁路控制器
LEFT_ARM_PVT_CTRL	左臂位置-速度-力矩控制器
LEFT_DEXHAND_CTRL	左手灵巧手控制器
LEFT_GRIPPER_CTRL	左夹爪控制器
LEG_PVT_BYPASS_CTRL	腿部位置-速度-力矩旁路控制器
LEG_PVT_CTRL	腿部位置-速度-力矩控制器
RIGHT_ARM_PVT_BYPASS_CTRL	右臂位置-速度-力矩旁路控制器
RIGHT_ARM_PVT_CTRL	右臂位置-速度-力矩控制器
RIGHT_DEXHAND_CTRL	右手灵巧手控制器
RIGHT_GRIPPER_CTRL	右夹爪控制器

G1 关节组

"关节组"是 SDK 的主要控制/规划单元，而非单个关节： - **运动一致控制**：每个链/末端执行器组都会验证和执行命令 - **确定性命令排序**：joint_groups 按组顺序扩展为具体的 joint_names - **组级行为**：每个组都有自己的主动/被动属性和执行容忍度 **推荐用法**：在填充 API 参数（例如 joint_groups）时，使用此结构体中的常量。如果需要精确的关节名称，请在运行时通过 get_joint_names(true, {group_name}) 查询，而非硬编码。在同时接受 joint_groups 和 joint_names 的 API 中，joint_names 优先。

Enum Value	Description
chassis	底盘机构组（关节位置控制中为被动），默认关节：

	chassis_joint1 ... chassis_joint4, 典型用途: 底盘状态分组; 底座运动应使用底座 API
head	头部 2 自由度链条, 默认关节: head_joint1、head_joint2, 典型用途: 注视/相机朝向控制
left_arm	左臂 7 自由度链条, 默认关节: left_arm_joint1 ... left_arm_joint7, 典型用途: 左臂 Reach/操作
left_dexhand	左手灵巧手组, 默认关节: left_dexhand_joint1 ... left_dexhand_joint6, 典型用途: 多指灵巧操作 (左)
left_gripper	左夹爪链条, 默认关节: left_gripper_joint1, 典型用途: 左夹爪开合和夹持宽度控制
left_suction_cup	左吸盘末端执行器组, 默认关节: left_suction_cup_joint1, 典型用途: 左臂真空拾取/放置
leg	腿部链条, 默认关节: leg_joint1 ... leg_joint5, 典型用途: 下半身姿态/运动相关控制
right_arm	右臂 7 自由度链条, 默认关节: right_arm_joint1 ... right_arm_joint7, 典型用途: 右臂 Reach/操作
right_dexhand	右手灵巧手组, 默认关节: right_dexhand_joint1 ... right_dexhand_joint6, 典型用途: 多指灵巧操作 (右)
right_gripper	右夹爪链条, 默认关节: right_gripper_joint1, 典型用途: 右夹爪开合和夹持宽度控制
right_suction_cup	右吸盘末端执行器组, 默认关节: right_suction_cup_joint1, 典型用途: 右臂真空拾取/放置

Galbot One Foxtrot 传感器类型

描述机器人手腕力传感器的力传感器枚举。

识别安装在机器人腕关节处的力/扭矩传感器, 用于力控制操作和接触检测。

Enum Value	Description
LEFT_WRIST_FORCE	左腕力传感器
RIGHT_WRIST_FORCE	右腕力传感器

夹爪状态

表示平行爪夹具的当前状态, 包括张开宽度、运动状态和抓取力。

Member Variables

Name	Type	Description
effort	float	力矩

is_moving	bool	运动标志（来自运动窗口）， false 表示在配置的时间窗口内未检测到有效运动， true 表示检测到有效运动
joint_positions	list[float]	夹爪关节位置（弧度）， 通常 0-0.04 米
timestamp_ns	int	状态时间戳（自纪元以来的纳秒数）
velocity	float	夹爪开合速度（米/秒）。
width	float	夹爪开口宽度（米）， 距离

关节组命令

用于控制一组关节的命令结构，包含多个关节的目标位置和参数。

Member Variables

Name	Type	Description
joint_commands	list[JointCommand]	Joint commands at this point
time_from_start_s	float	Time from trajectory start in seconds

消息头

标准消息头包含时间戳和坐标系信息。时间戳以纳秒的形式存储（与其他传感器类型统一）。

Member Variables

Name	Type	Description
frame_id	str	标识数据所在的坐标系，例如： "base_link"、"world"、 "camera_optical_frame"、 "lidar_link"、"map"
timestamp_ns	int	数据采集的时间戳（自纪元以来的纳秒数）， 记录数据被捕获或生成的时间

IK 求解器配置

逆运动学 (IK) 解算器配置参数。

该结构配置数值逆运动学解算器，用于计算实现所需末端执行器姿态的关节配置。它支持具有可配置种子策略、收敛容差、关节限制处理和超时参数的碰撞感知 IK。IK 求解是一种迭代数值优化过程，可以受益于多次随机初始化来找到可行的无碰撞解决方案。

获取关节限制安全裕度

```
def get_col_aware_ik_joint_limit_bias() -> float
```

获取碰撞感知逆运动学关节限制偏差。较大的偏差会使求解器更倾向于避开碰撞区域。

返回值

类型	描述
float	-

获取碰撞感知 IK 求解器超时

```
def get_col_aware_ik_timeout() -> float
```

获取碰撞感知逆运动学求解超时时间（秒）

返回值

类型	描述
float	-

检查是否启用碰撞检查日志

```
def get_enable_collision_check_log() -> bool
```

获取是否启用碰撞检测日志输出

返回值

类型	描述
bool	-

获取方向误差容差

```
def get_rotation_eps() -> list[float]
```

获取旋转收敛容差（弧度）

返回值

类型	描述
list[float]	-

获取 IK 求解器种子生成策略

```
def get_seed_type() -> SeedType
```

获取逆运动学求解种子类型

返回值

类型	描述
SeedType	-

获取笛卡尔位置误差容差

```
def get_translation_eps() -> list[float]
```

获取平移收敛容差（米）

返回值

类型	描述
list[float]	-

打印 IK 求解器配置到标准输出

```
def print() -> None
```

打印逆运动学求解器配置到标准输出

设置关节位置限制的安全裕度

```
def set_col_aware_ik_joint_limit_bias(bias: SupportsFloat) -> None
```

设置碰撞感知逆运动学关节限制偏差

参数

名称	类型	默认值	描述
bias	SupportsFloat	需要传参	-

注意

防止 IK 求解器提议在奇点附近的配置。

设置碰撞感知 IK 求解器超时

```
def set_col_aware_ik_timeout(timeout: SupportsFloat) -> None
```

设置碰撞感知逆运动学求解超时时间（秒）

参数

名称	类型	默认值	描述
timeout	SupportsFloat	需要传参	-

注意

较长的超时允许更多种子尝试但会延迟规划。

启用或禁用碰撞检测日志

```
def set_enable_collision_check_log(enable: bool) -> None
```

设置是否启用碰撞检测日志输出

参数

名称	类型	默认值	描述
enable	bool	需要传参	-

注意

用于调试由于碰撞约束导致的 IK 失败。

设置方向误差容差

```
def set_rotation_eps(eps: list[float]) -> None
```

设置旋转收敛容差（弧度）

参数

名称	类型	默认值	描述
eps	list[float]	需要传参	-

注意

当方向误差在此容差内时接受 IK 解。

设置种子类型

```
def set_seed_type(type: SeedType) -> None
```

设置逆运动学求解种子类型

参数

名称	类型	默认值	描述
type	SeedType	需要传参	-

设置笛卡尔位置误差容差

```
def set_translation_eps(eps: list[float]) -> None
```

设置平移收敛容差（米）

参数

名称	类型	默认值	描述
eps	list[float]	需要传参	-

注意

当位置误差在此容差内时接受 IK 解。

IMU 数据

包含来自惯性测量单元 (IMU) 的带时间戳的数据，包括加速度计、陀螺仪和磁力计测量值。

Member Variables

Name	Type	Description
accel	Vector3	线加速度 (米/秒 ²) : [ax, ay, az]。
gyro	Vector3	陀螺仪数据
magnet	Vector3	磁力计数据
timestamp_ns	int	测量时间戳 (自纪元以来的纳秒数)

关节命令

指定轨迹或控制命令中单个机器人关节所需的运动参数。

Member Variables

Name	Type	Description
acceleration	float	加速度
effort	float	期望的关节力矩 (牛顿·米)
position	float	期望的关节位置 (弧度)
velocity	float	期望的关节速度 (弧度/秒)

单关节状态

单关节状态结构。

表示单个机器人关节的完整实时状态，包含运动学量（位置、速度、加速度）以及动力学量（力矩与电机电流）。

Member Variables

Name	Type	Description
acceleration	float	-
current	float	-
effort	float	-
position	float	-
velocity	float	-

关节状态消息

多个关节的关节状态的时间戳集合，通常表示机器人在某一时刻的完整关节配置的快照。

Member Variables

Name	Type	Description
joint_state_vec	list[JointState]	关节状态向量
timestamp_ns	int	采集时间戳（自纪元以来的纳秒数）

关节状态

表示运动链的目标关节配置。扩展 RobotStates 以指定基于关节的运动目标。用于关节轨迹规划和正向运动学计算。所有关节角度必须以弧度为单位。矢量大小必须与指定运动链的自由度匹配。

Member Variables

Name	Type	Description
joint_positions	list[float]	-

获取类型

```
def get_type() -> RobotStatesType
```

获取状态类型，指示这是关节空间目标

返回值

类型	描述
RobotStatesType	-

设置关节

```
def set_joint(index: SupportsInt, val: SupportsInt) -> None
```

设置指定索引的关节值

参数

名称	类型	默认值	描述
index	SupportsInt	需要传参	-
val	SupportsInt	需要传参	-

注意

函数执行边界检查；无效索引被静默忽略。

警告

越界访问不会返回错误；请确保索引有效。

设置关节位置

```
def set_joint_positions(joints: list[float]) -> None
```

设置所有关节位置

参数

名称	类型	默认值	描述
joints	list[float]	需要传参	-

注意

向量大小应等于链中驱动关节的数量。

运动学边界

机器人运动链关节的运动边界参数。

该结构定义了机器人运动链（例如，机械臂、移动底座或腿链）的运动学约束。它指定了链条中每个关节的位置、速度、加速度和加加速度限制。这些边界对于确保轨迹规划和执行过程中安全且物理上可行的运动至关重要。每个向量应包含运动链中每个关节的一个值。所有关节空间量均以弧度或每单位时间的弧度指定。

获取下限加速度限制

```
def get_acc_lower_limit() -> list[float]
```

获取加速度下限（rad/s²）

返回值

类型	描述
list[float]	-

获取上限加速度限制

```
def get_acc_upper_limit() -> list[float]
```

获取加速度上限（rad/s²）

返回值

类型	描述
list[float]	-

获取运动链名称

```
def get_chain_name() -> str
```

获取该边界约束所属的运动链名称

返回值

类型	描述
str	-

获取下限加加速度限制

```
def get_jerk_lower_limit() -> list[float]
```

获取加加速度下限 (rad/s³)

返回值

类型	描述
list[float]	-

获取上限加加速度限制

```
def get_jerk_upper_limit() -> list[float]
```

获取加加速度上限 (rad/s³)

返回值

类型	描述
list[float]	-

获取下限位置限制

```
def get_lower_limit() -> list[float]
```

获取关节位置下限 (rad)

返回值

类型	描述
list[float]	-

获取上限位置限制

```
def get_upper_limit() -> list[float]
```

获取关节位置上限 (rad)

返回值

类型	描述
list[float]	-

获取下限速度限制

```
def get_vel_lower_limit() -> list[float]
```

获取速度下限 (rad/s)

返回值

类型	描述
list[float]	-

获取上限速度限制

```
def get_vel_upper_limit() -> list[float]
```

获取速度上限 (rad/s)

返回值

类型	描述
list[float]	-

打印输出

```
def print() -> None
```

打印运动学边界约束信息到标准输出

设置下限加速度限制

```
def set_acc_lower_limit(limits: list[float]) -> None
```

设置加速度下限

参数

名称	类型	默认值	描述
limits	list[float]	需要传参	-

注意

用于轨迹优化和平滑约束。

设置上限加速度限制

```
def set_acc_upper_limit(limits: list[float]) -> None
```

设置加速度上限

参数

名称	类型	默认值	描述
limits	list[float]	需要传参	-

注意

用于轨迹优化和平滑约束。

设置运动链名称

```
def set_chain_name(name: str) -> None
```

设置该边界约束所属的运动链名称

参数

名称	类型	默认值	描述
name	str	需要传参	-

设置下限加加速度限制

```
def set_jerk_lower_limit(limits: list[float]) -> None
```

设置加加速度下限

参数

名称	类型	默认值	描述
limits	list[float]	需要传参	-

注意

加加速度约束可提高运动平滑度并减少机械应力。

设置上限加加速度限制

```
def set_jerk_upper_limit(limits: list[float]) -> None
```

设置加加速度上限

参数

名称	类型	默认值	描述
limits	list[float]	需要传参	-

注意

加加速度约束可提高运动平滑度并减少机械应力。

设置下限位置限制

```
def set_lower_limit(limits: list[float]) -> None
```

设置关节位置下限

参数

名称	类型	默认值	描述
limits	list[float]	需要传参	-

注意

向量大小必须等于链中的关节数。

设置上限位置限制

```
def set_upper_limit(limits: list[float]) -> None
```

设置关节位置上限

参数

名称	类型	默认值	描述
limits	list[float]	需要传参	-

注意

向量大小必须等于链中的关节数。

设置下限速度限制

```
def set_vel_lower_limit(limits: list[float]) -> None
```

设置速度下限

参数

名称	类型	默认值	描述
limits	list[float]	需要传参	-

注意

双向关节通常为负值。

设置上限速度限制

```
def set_vel_upper_limit(limits: list[float]) -> None
```

设置速度上限

参数

名称	类型	默认值	描述
limits	list[float]	需要传参	-

注意

双向关节通常为正值。

激光雷达数据

与 ROS 2sensor_msgs/PointCloud2 兼容的通用 N 维点云结构。将点数据存储为二进制 blob，并使用定义数据布局的字段描述符。支持有序（结构化）和无序（非结构化）点云。

Member Variables

Name	Type	Description
data	list[int]	包含所有点数据的二进制数据块（按行优先顺序排列），大小应等于 row_step × height 字节，每个点占用 point_step 字节，布局根据 fields 描述符确定
fields	list[PointField]	描述每个点中存在的数据通道（x, y, z, intensity, rgb 等）及其二进制布局。
header	Header	消息头
height	int	点云高度，无序点云 height = 1（单行），有序点云 height = 行数（例如，来自旋转 LiDAR 或深度相机）
is_bigendian	bool	数据的字节序，true 表示大端序，false 表示小端序（x86/ARM 系统通常为小端序）
is_dense	bool	是否所有点都有效，true 表示所有点都有效且没有 NaN 或 Inf 值，false 表示点云可能包含无效点（NaN 或 Inf）
point_step	int	单个点结构的总字节大小，包括所有字段和填充，必须 ≥ 所有字段大小的总和，可能包含对齐填充
row_step	int	行行间距
width	int	点云宽度，无序点云 width = 点的总数，有序点云 width = 每行的点数（列数），总点数 = height × width

直线轨迹碰撞检测基元

用于笛卡尔线性轨迹验证的几何基元配置。

该结构配置笛卡尔空间中线性末端执行器轨迹的碰撞检测几何表示。它支持两种基本类型：无限细线和扫掠体积圆柱体。选择适当的原语会影响碰撞检测的保守性和计算成本。圆柱体基元可以更准确地对机器人的实际扫描体积进行建模，但需要更昂贵的几何查询。

获取圆柱体基元半径

```
def get_cylinder_prim_radius() -> float
```

获取圆柱基元半径（米）

返回值

类型	描述
float	-

获取直线检查基元类型

```
def get_line_check_primitive_type() -> PrimitiveType
```

获取直线检查基元类型

返回值

类型	描述
PrimitiveType	-

获取直线基元曲率

```
def get_line_prim_curvature() -> float
```

获取直线基元曲率

返回值

类型	描述
float	-

打印输出

```
def print() -> None
```

打印直线轨迹检查基元信息到标准输出

设置圆柱体基元半径

```
def set_cylinder_prim_radius(radius: SupportsFloat) -> None
```

设置圆柱基元半径（米）

参数

名称	类型	默认值	描述
radius	SupportsFloat	需要传参	-

注意

较大的半径增加安全裕度但可能过于保守。

注意

仅当基元类型为 CYLINDER 时适用。

设置直线检查基元类型

```
def set_line_check_primitive_type(type: PrimitiveType) -> None
```

设置直线检查基元类型

参数

名称	类型	默认值	描述
type	PrimitiveType	需要传参	-

注意

推荐 CYLINDER 用于安全关键应用。

设置直线基元曲率

```
def set_line_prim_curvature(curvature: SupportsFloat) -> None
```

设置直线基元曲率

参数

名称	类型	默认值	描述
curvature	SupportsFloat	需要传参	-

注意

控制如何将弯曲路径离散为直线段。

注意

较低的值提高精度但增加计算成本。

日志级别

表示日志消息的严重级别。

Enum Value	Description
CRITICAL	严重日志
DEBUG	调试日志
ERROR	错误日志
INFO	信息日志
TRACE	跟踪日志
WARN	警告日志

机器类型

此枚举定义了 Galbot SDK 支持的不同机器人平台或机器类型。客户端可以使用这些值来指定他们正在使用的机器人模型，特别是对于返回特定于平台的实现的工厂方法。将枚举保留在通用类型定义中可确保整个 SDK 的一致性，同时隐藏各个模块中的实现细节。

Enum Value	Description
G1	G1 机器人
S1	S1 机器人

运动规划配置

全面的运动规划配置管理。

MotionPlanConfig 充当所有运动规划子系统的集中配置容器。它聚合了采样策略、轨迹生成参数、逆运动学解算器设置、碰撞检测选项、可行性验证标准和运动学约束边界。此类提供了用于配置复杂运动规划管道的统一接口，支持简单的机械臂规划和具有多个运动链的全身人形运动生成。配置对象通过共享指针进行延迟初始化和 管理，以优化内存使用并支持可选功能配置。

创建碰撞检查选项

```
def create_collision_check_option() -> CollisionCheckOption
```

创建碰撞检测选项并返回

返回值

类型	描述
CollisionCheckOption	-

创建 IK 求解器配置

```
def create_ik_solver_config() -> IKSolverConfig
```

创建逆运动学求解器配置并返回

返回值

类型	描述
IKSolverConfig	-

创建直线轨迹检查基元

```
def create_line_traj_check_primitive() -> LineTrajCheckPrimitive
```

创建直线轨迹检查基元并返回

返回值

类型	描述
LineTrajCheckPrimitive	-

创建采样器配置

```
def create_sampler_config() -> SamplerConfig
```

创建采样器配置并返回

返回值

类型	描述
SamplerConfig	-

创建轨迹可行性检查选项

```
def create_trajectory_feasibility_check_option() -> TrajectoryFeasibilityCheckOption
```

创建轨迹可行性检查选项并返回

返回值

类型	描述
TrajectoryFeasibilityCheckOption	-

创建轨迹规划配置

```
def create_trajectory_plan_config() -> TrajectoryPlanConfig
```

创建轨迹规划配置并返回

返回值

类型	描述
TrajectoryPlanConfig	-

获取碰撞检查选项

```
def get_collision_check_option() -> CollisionCheckOption
```

获取碰撞检测选项

返回值

类型	描述
CollisionCheckOption	-

注意

使用 create_collision_check_option() 确保有效的配置。

获取碰撞检查选项引用

```
def get_collision_check_option_ref() -> CollisionCheckOption
```

获取碰撞检测选项引用

返回值

类型	描述
CollisionCheckOption	-

获取可行性边界

```
def get_feasibility_boundary() -> list[KinematicsBoundary]
```

获取可行性边界约束

返回值

类型	描述
list[KinematicsBoundary]	-

获取硬关约束

```
def get_hard_joint_limit() -> list[KinematicsBoundary]
```

获取硬关节位置限制边界

返回值

类型	描述
----	----

list[KinematicsBoundary]	-
--------------------------	---

获取 IK 关约束

```
def get_ik_joint_limit() -> list[KinematicsBoundary]
```

获取逆运动学关节位置限制边界

返回值

类型	描述
list[KinematicsBoundary]	-

获取 IK 求解器配置

```
def get_ik_solver_config() -> IKSolverConfig
```

获取逆运动学求解器配置

返回值

类型	描述
IKSolverConfig	-

注意

使用 create_ik_solver_config() 确保有效的配置。

获取 IK 求解器配置引用

```
def get_ik_solver_config_ref() -> IKSolverConfig
```

获取逆运动学求解器配置引用

返回值

类型	描述
IKSolverConfig	-

获取直线轨迹检查基元

```
def get_line_traj_check_primitive() -> LineTrajCheckPrimitive
```

获取直线轨迹检查基元

返回值

类型	描述
LineTrajCheckPrimitive	-

注意

使用 `create_line_traj_check_primitive()` 确保有效的配置。

获取直线轨迹检查基元引用

```
def get_line_traj_check_primitive_ref() -> LineTrajCheckPrimitive
```

获取直线轨迹检查基元引用

返回值

类型	描述
LineTrajCheckPrimitive	-

获取是否恢复 IK 关节限制

```
def get_revert_ik_joint_limit() -> bool
```

获取是否恢复逆运动学关节限制

返回值

类型	描述
bool	-

获取需要恢复 IK 关节限制的运动链

```
def get_revert_ik_joint_limit_chains() -> list[str]
```

获取需要恢复逆运动学关节限制的运动链列表

返回值

类型	描述
list[str]	-

获取采样器配置

```
def get_sampler_config() -> SamplerConfig
```

获取采样器配置

返回值

类型	描述
SamplerConfig	-

注意

使用 `create_sampler_config()` 确保有效的配置。

获取采样器配置引用

```
def get_sampler_config_ref() -> SamplerConfig
```

获取采样器配置引用

返回值

类型	描述
SamplerConfig	-

获取采样关节约束

```
def get_sampler_joint_limit() -> list[KinematicsBoundary]
```

获取采样器关节位置限制边界

返回值

类型	描述
list[KinematicsBoundary]	-

获取轨迹可行性检查选项

```
def get_trajectory_feasibility_check_option() -> TrajectoryFeasibilityCheckOption
```

获取轨迹可行性检查选项

返回值

类型	描述
TrajectoryFeasibilityCheckOption	-

注意

使用 create_trajectory_feasibility_check_option() 确保有效的配置。

获取轨迹可行性检查选项引用

```
def get_trajectory_feasibility_check_option_ref() -> TrajectoryFeasibilityCheckOption
```

获取轨迹可行性检查选项引用

返回值

类型	描述
TrajectoryFeasibilityCheckOption	-

获取轨迹规划配置

```
def get_trajectory_plan_config() -> TrajectoryPlanConfig
```

获取轨迹规划配置

返回值

类型	描述
TrajectoryPlanConfig	-

注意

使用 create_trajectory_plan_config() 确保有效的配置。

获取轨迹规划配置引用

```
def get_trajectory_plan_config_ref() -> TrajectoryPlanConfig
```

获取轨迹规划配置引用

返回值

类型	描述
TrajectoryPlanConfig	-

获取更新时间

```
def get_update_time() -> int
```

获取更新时间参数

返回值

类型	描述
int	-

打印输出

```
def print() -> None
```

将完整的运动规划配置打印到标准输出。
以人类可读格式输出所有子配置参数和运动学边界信息，便于调试、日志记录以及配置状态校验。

设置碰撞检查选项

```
def set_collision_check_option(option: CollisionCheckOption) -> None
```

设置碰撞检测选项

参数

名称	类型	默认值	描述
option	CollisionCheckOption	需要传参	-

设置可行性边界

```
def set_feasibility_boundary(boundary: Sequence[KinematicsBoundary]) -> None
```

设置可行性边界约束

参数

名称	类型	默认值	描述
boundary	Sequence[KinematicsBoundary]	需要传参	-

注意

这些边界用于一般轨迹可行性检查。

设置硬关节约束

```
def set_hard_joint_limit(boundary: Sequence[KinematicsBoundary]) -> None
```

设置硬关节位置限制边界

参数

名称	类型	默认值	描述
boundary	Sequence[KinematicsBoundary]	需要传参	-

注意

硬限制必须永不违反；通常对应于物理极限。

设置 IK 关节约束

```
def set_ik_joint_limit(boundary: Sequence[KinematicsBoundary]) -> None
```

设置逆运动学关节位置限制边界

参数

名称	类型	默认值	描述
boundary	Sequence[KinematicsBoundary]	需要传参	-

注意

IK 限制可能比硬限制更紧以改善收敛性。

设置 IK 求解器配置

```
def set_ik_solver_config(config: IKSolverConfig) -> None
```

设置逆运动学求解器配置

参数

名称	类型	默认值	描述
config	IKSolverConfig	需要传参	-

设置直线轨迹检查基元

```
def set_line_traj_check_primitive(primitive: LineTrajCheckPrimitive) -> None
```

设置直线轨迹检查基元

参数

名称	类型	默认值	描述
primitive	LineTrajCheckPrimitive	需要传参	-

设置是否恢复 IK 关节限制

```
def set_revert_ik_joint_limit(flag: bool) -> None
```

设置是否恢复逆运动学关节限制

参数

名称	类型	默认值	描述
flag	bool	需要传参	-

注意

用于通过暂时恢复极端限制来从受限配置中恢复。

设置需要恢复 IK 关节限制的运动链

```
def set_revert_ik_joint_limit_chains(chains: Sequence[str]) -> None
```

设置需要恢复逆运动学关节限制的运动链列表

参数

名称	类型	默认值	描述
chains	Sequence[str]	需要传参	-

注意

如果非空，自动启用 revert_ik_joint_limit 标志。

注意

空向量禁用选择性恢复（适用于所有链）。

设置采样器配置

```
def set_sampler_config(config: SamplerConfig) -> None
```

设置采样器配置

参数

名称	类型	默认值	描述
config	SamplerConfig	需要传参	-

设置采样关节约束

```
def set_sampler_joint_limit(boundary: Sequence[KinematicsBoundary]) -> None
```

设置采样器关节位置限制边界

参数

名称	类型	默认值	描述
boundary	Sequence[KinematicsBoundary]	需要传参	-

注意

采样限制定义了探索配置空间的有效范围。

设置轨迹可行性检查选项

```
def set_trajectory_feasibility_check_option(option: TrajectoryFeasibilityCheckOption) -> None
```

设置轨迹可行性检查选项

参数

名称	类型	默认值	描述
option	TrajectoryFeasibilityCheckOption	需要传参	-

设置轨迹规划配置

```
def set_trajectory_plan_config(config: TrajectoryPlanConfig) -> None
```

设置轨迹规划配置

参数

名称	类型	默认值	描述
config	TrajectoryPlanConfig	需要传参	-

设置更新时间

```
def set_update_time(t: SupportsInt) -> None
```

设置更新时间参数

参数

名称	类型	默认值	描述
t	SupportsInt	需要传参	-

注意

用于配置版本控制和缓存失效。

运动状态

机器人动作执行状态枚举。

表示机器人运动命令的执行状态，包括轨迹跟随、位姿到达等运动规划操作。

Enum Value	Description
COMM_DISCONNECTED	通信断开
DATA_FETCH_FAILED	数据获取失败
FAULT	故障
INIT_FAILED	初始化失败
INVALID_INPUT	无效输入
IN_PROGRESS	进行中
PUBLISH_FAIL	发布失败
STATUS_NUM	状态数量
STOPPED_UNREACHED	已停止但未到达
SUCCESS	成功
TIMEOUT	超时
UNSUPPORTED_FUNCRION	不支持的函数

导航任务状态

导航任务当前状态枚举。

表示导航系统报告的活动或已完成的导航任务的当前状态。用于非阻塞导航时轮询，及时检测 RUNNING、SUCCESS 或 FAILED 并退出错误逻辑。

Enum Value	Description
FAILED	失败
RUNNING	运行中
SUCCESS	导航任务成功完成
UNKNOWN	未知

里程计数据

里程计数据。 包含来自里程计源（轮式编码器、IMU 融合等）的机器人位姿和速度估计。用于机器人定位和导航。

Member Variables

Name	Type	Description
angular_velocity	list[float]	角速度 $[\omega_x, \omega_y, \omega_z]$ (弧度/秒)
linear_velocity	list[float]	线速度 $[v_x, v_y, v_z]$ (米/秒)
orientation	list[float]	四元数方向 $[q_x, q_y, q_z, q_w]$
position	list[float]	位置 $[x, y, z]$ (米)
timestamp_ns	int	里程计时间戳 (自纪元以来的纳秒数)

参数

运动规划参数配置类。

此类扩展了 PlannerConfig，为全身运动规划和执行提供全面的配置选项。它封装了执行模式、驱动类型、工具架处理、碰撞检查和坐标系规范。所有角度参数均以弧度为单位，线性参数以米（SI 单位）为单位。

Member Variables

Name	Type	Description
joint_state	dict[str, list[float]]	-
timeout_second	float	-

获取驱动类型

```
def get_actuate_type() -> str
```

获取驱动类型

返回值

类型	描述
str	-

获取是否阻塞

```
def get_blocking() -> bool
```


获取是否阻塞执行

返回值

类型	描述
bool	-

获取是否检测碰撞

```
def get_check_collision() -> bool
```

获取是否进行碰撞检测

返回值

类型	描述
bool	-

获取是否直接执行

```
def get_direct_execute() -> bool
```

获取是否直接执行

返回值

类型	描述
bool	-

获取参考帧

```
def get_reference_frame() -> str
```

获取参考坐标系

返回值

类型	描述
str	-

获取超时

```
def get_timeout() -> float
```

获取超时时间（秒）

返回值

类型	描述
float	-

获取工具位姿

```
def get_tool_pose() -> bool
```

获取目标工具位姿

返回值

类型	描述
bool	-

设置驱动类型

```
def set_actuate(actuate: str) -> None
```

设置驱动类型

参数

名称	类型	默认值	描述
actuate	str	需要传参	-

警告

actuate 必须为受支持的取值 ("with_chain_only"、"with_torso"、"with_leg") ， 否则行为未定义。

设置是否阻塞

```
def set_blocking(blocking: bool) -> None
```

设置是否阻塞执行

参数

名称	类型	默认值	描述
blocking	bool	需要传参	-

设置是否检测碰撞

```
def set_check_collision(check_collision: bool) -> None
```

设置是否进行碰撞检测

参数

名称	类型	默认值	描述
check_collision	bool	需要传参	-

警告

禁用碰撞检查可能会导致不安全的轨迹。

设置是否直接执行

```
def set_direct_execute(direct_execute: bool) -> None
```

设置是否直接执行

参数

名称	类型	默认值	描述
direct_execute	bool	需要传参	-

设置是否移动直线

```
def set_move_line(move_line: bool) -> None
```

设置是否为直线运动

参数

名称	类型	默认值	描述
move_line	bool	需要传参	-

注意

直线运动提供可预测的笛卡尔路径，但可能在狭窄空间中受限制。

设置参考帧

```
def set_reference_frame(frame: str) -> None
```

设置参考坐标系

参数

名称	类型	默认值	描述
frame	str	需要传参	-

注意

必须是机器人 TF 树中的有效帧。

设置超时

```
def set_timeout(timeout: SupportsFloat) -> None
```

设置运动执行超时时间（秒）

参数

名称	类型	默认值	描述
timeout	SupportsFloat	需要传参	-

注意

仅在阻塞模式启用时适用。

设置工具位姿

```
def set_tool_pose(tool_pose: bool) -> None
```

设置目标工具位姿

参数

名称	类型	默认值	描述
tool_pose	bool	需要传参	-

感知模块

启用的感知管道（初始化时加载的模型集）。

Enum Value	Description
FOUNDATION_STEREO	高精度立体深度；用于需要高精度的任务（如搬箱子场景）。
LIGHT_STEREO	轻量级立体深度；用于精度要求不高的场景（如迎宾场景）。此版本不支持。

点

表示三维笛卡尔空间中的位置。

Member Variables

Name	Type	Description
x	float	-
y	float	-
z	float	-

点云字段

描述 PointCloud2 点结构中的一个数据字段，定义其名称、类型、偏移量和计数。与 ROS 2sensor_msgs/PointField 兼容。

Member Variables

Name	Type	Description
------	------	-------------

count	int	此字段的数组长度，标量字段（x, y, z, intensity）通常为 1，数组字段可能大于 1（例如 count=3 表示 3 元素向量）
datatype	...	数据类型
offset	int	此字段相对于点数据结构起始位置的字节偏移量，例如：对于点布局 [x(float32), y(float32), z(float32), intensity(float32)]，x 的偏移为 0，y 的偏移为 4，z 的偏移为 8，intensity 的偏移为 12

点云字段数据类型

数据类型枚举。

定义点云字段的原始数据类型，确定每个字段值的字节大小和解释方法。

Enum Value	Description
FLOAT32	32 位浮点数
FLOAT64	64 位浮点数
INT16	16 位整数
INT32	32 位整数
INT8	8 位整数
UINT16	16 位无符号整数
UINT32	32 位无符号整数
UINT8	8 位无符号整数
UNKNOWN	未知

姿态

姿态（位置+方向）结构。

表示 3D 空间中的完整 6-DOF（自由度）姿态，结合位置（平移）和方向（旋转）信息。常用于机器人末端执行器姿态、物体姿态和坐标系变换。

姿态状态

表示笛卡尔空间 (SE(3)) 中的目标末端执行器姿态。扩展 RobotStates 为运动链指定基于姿态的运动目标。用于逆运动学和笛卡尔轨迹规划。姿态值：以米为单位的位置，以四元数为单位的方向。坐标系必须存在于机器人的 TF 树中。

获取类型

```
def get_type() -> RobotStatesType
```

获取位姿状态类型

返回值

类型	描述
RobotStatesType	-

几何基元类型

原始类型相关说明。

Enum Value	Description
CYLINDER	圆柱体
LINE	直线

四元数

使用四元数表示 (x, y, z, w) 表示 3D 旋转。单位四元数的大小为 1，表示有效旋转。

Member Variables

Name	Type	Description
w	float	-
x	float	-
y	float	-
z	float	-

RGB 数据

包含来自 RGB 相机的压缩彩色图像数据。兼容 ROS 2sensor_msgs/CompressedImage 格式。

Member Variables

Name	Type	Description
data	list[int]	包含压缩图像数据的二进制数据块
format	str	指定压缩格式和编码
header	Header	包含采集时间戳和相机坐标系

机器人状态

封装机器人的完整运动状态，包括全身关节配置和移动底座位姿。此类作为更专业的状态表示（PoseState、JointStates）的基础，并在整个规划和控制管道中用于状态规范和反馈。所有角度值均以弧度为单位，线性值以米（SI 单位）为单位。基本姿态使用四元数表示方向（x、y、z、qx、qy、qz、qw）。

Member Variables

Name	Type	Description
base_state	list[float]	-
whole_body_joint	list[float]	-

获取类型

```
def get_type() -> RobotStatesType
```

获取机器人状态类型

返回值

类型	描述
RobotStatesType	-

设置基座状态

```
def set_base_state(base_pose: Pose) -> None
```

设置底座状态（位姿）

参数

名称	类型	默认值	描述
base_pose	Pose	需要传参	-

注意

四元数必须单位归一化（ $x^2 + y^2 + z^2 + w^2 = 1$ ）。

设置全身关节

```
def set_whole_body_joint(joint_positions: list[float]) -> None
```

设置全身关节位置

参数

名称	类型	默认值	描述
joint_positions	list[float]	需要传参	-

注意

向量大小应等于总驱动关节数。

机器人状态类型

用于区分派生状态类型的枚举。

用于 RobotStates 派生类的运行时类型识别。

Enum Value	Description
JOINT	关节
POSE	姿态
ROBOT_STATES	机器人状态数量

采样器配置

基于采样的运动规划器的配置参数。

该结构配置基于采样的规划算法（例如，RRT、RRT*）。它控制状态空间采样分辨率、插值设置、路径简化和规划终止条件。基于采样的规划器通过随机采样状态并将它们连接起来构建运动计划图来探索配置空间。

获取是否插值

```
def get_interpolate() -> bool
```

获取是否启用插值

返回值

类型	描述
bool	-

获取插值数量

```
def get_interpolation_cnt() -> int
```

获取插值点数量

返回值

类型	描述
int	-

获取最大规划时间

```
def get_max_planning_time() -> float
```

获取最大规划时间（秒）

返回值

类型	描述
float	-

获取最大简化时间

```
def get_max_simplification_time() -> float
```

获取最大简化时间（秒）

返回值

类型	描述
float	-

获取是否简化

```
def get_simplify() -> bool
```

获取是否启用简化

返回值

类型	描述
bool	-

获取状态检查分辨率

```
def get_state_check_resolution() -> float
```

获取状态检查分辨率

返回值

类型	描述
float	-

获取状态检查类型

```
def get_state_check_type() -> StateCheckType
```

获取状态检查类型

返回值

类型	描述
StateCheckType	-

获取终止条件类型

```
def get_termination_condition_type() -> TerminationConditionType
```

获取终止条件类型

返回值

类型	描述
TerminationConditionType	-

打印输出

```
def print() -> None
```

打印采样器配置信息到标准输出

设置是否插值

```
def set_interpolate(enable: bool) -> None
```

设置是否启用插值

参数

名称	类型	默认值	描述
enable	bool	需要传参	-

注意

插值可提高轨迹平滑度和碰撞检测覆盖率。

设置插值数量

```
def set_interpolation_cnt(cnt: SupportsInt) -> None
```

设置插值点数量

参数

名称	类型	默认值	描述
cnt	SupportsInt	需要传参	-

注意

较高的计数可改善碰撞检测但增加计算成本。

设置最大规划时间

```
def set_max_planning_time(time: SupportsFloat) -> None
```

设置最大规划时间（秒）

参数

名称	类型	默认值	描述
time	SupportsFloat	需要传参	-

注意

如果找到精确解，规划可能提前终止（取决于终止条件）。

设置最大简化时间

```
def set_max_simplification_time(time: SupportsFloat) -> None
```

设置最大简化时间（秒）

参数

名称	类型	默认值	描述
time	SupportsFloat	需要传参	-

注意

较长的简化时间可能产生更短、更平滑的路径。

设置是否简化

```
def set_simplify(enable: bool) -> None
```

设置是否启用简化

参数

名称	类型	默认值	描述
enable	bool	需要传参	-

注意

简化减少路点并提高轨迹效率。

设置状态检查分辨率

```
def set_state_check_resolution(resolution: SupportsFloat) -> None
```

设置状态检查分辨率

参数

名称	类型	默认值	描述
resolution	SupportsFloat	需要传参	-

注意

较低的值增加规划精度但可能减慢计算速度。

设置状态检查类型

```
def set_state_check_type(type: StateCheckType) -> None
```

设置状态检查类型

参数

名称	类型	默认值	描述
type	StateCheckType	需要传参	-

设置终止条件类型

```
def set_termination_condition_type(type: TerminationConditionType) -> None
```

设置终止条件类型

参数

名称	类型	默认值	描述
type	TerminationConditionType	需要传参	-

随机种子类型

指定逆运动学 (IK) 解算器的初始化策略。不同的种子类型会影响收敛速度和解的质量。

Enum Value	Description
RANDOM_PROGRESSIVE_SEED	随机渐进种子
RANDOM_SEED	随机种子
USER_DEFINED_SEED	用户自定义种子

传感器类型

描述机器人上各种传感器的传感器类型枚举。

识别机器人上可用于感知、定位和操作任务的不同传感器类型。

Enum Value	Description
BASE_LIDAR	基座激光雷达
BASE_ULTRASONIC	基座超声波
HEAD_LEFT_CAMERA	左眼相机
HEAD_RIGHT_CAMERA	右眼相机
LEFT_ARM_CAMERA	左臂相机
LEFT_ARM_DEPTH_CAMERA	左臂深度相机
LEFT_FRONT_SURROUND_CAMERA	左前方环视相机
LEFT_REAR_SURROUND_CAMERA	左后方环视相机
RIGHT_ARM_CAMERA	右臂相机
RIGHT_ARM_DEPTH_CAMERA	右臂深度相机
RIGHT_FRONT_SURROUND_CAMERA	右前方环视相机
RIGHT_REAR_SURROUND_CAMERA	右后方环视相机
TORSO_IMU	躯干 IMU

SingoriX 目标

SingoriX 控制器的目标表示，用于与 SingoriX 外部控制器通信。

Member Variables

Name	Type	Description
header	Header	Message header
target_group_trajectory_map	dict[str, TargetGroupTrajectory]	Joint-space trajectory map
target_task_trajectory_map	dict[str, TargetTaskTrajectory]	Task-space trajectory map

状态检查类型

状态检查类型相关说明。

Enum Value	Description
EUCLIDEAN_DISTANCE	欧氏距离
RADIAN_DISTANCE	弧度距离

吸盘动作状态

代表真空吸盘末端执行器的运行状态，跟踪从空闲到成功或失败的抽吸过程。

Enum Value	Description
FAILED	失败
IDLE	空闲
SUCCESS	吸盘动作成功
SUCKING	吸取中

吸盘状态

包含真空吸盘夹具的当前状态，包括激活状态、压力读数和动作状态。

Member Variables

Name	Type	Description
action_state	SUCTION_ACTION_STATE	动作状态
activation	bool	激活
pressure	float	压力
timestamp_ns	int	状态时间戳（自纪元以来的纳秒数）

目标配置

目标的配置参数，包含目标类型、采样方式、约束条件等设置。

Member Variables

Name	Type	Description
target_data	int	Target data bitmask
target_id	str	Target identifier
target_priority	int	Target priority
target_sampling	TargetSampling	Sampling strategy
target_ts	Timestamp	Target timestamp
target_type	int	Target type bitmask

目标关节组轨迹

目标在关节组空间中的轨迹，包含一系列途经点和时间信息。

Member Variables

Name	Type	Description
group_commands	list[GroupCommand]	Trajectory points
joint_names	list[str]	Joint names
target_config	TargetConfig	Target configuration

目标采样

目标轨迹的采样方法，用于在路径点之间生成中间点。

Enum Value	Description
TARGET_SAMPLING_B_SPLINES	B 样条采样
TARGET_SAMPLING_CUBIC_SPLINES	三次样条采样
TARGET_SAMPLING_CUSTOM	自定义采样
TARGET_SAMPLING_DEFAULT	默认采样
TARGET_SAMPLING_DIRECT_PASS	直接传递
TARGET_SAMPLING_LINEAR_INTERPOLATE	线性插值采样
TARGET_SAMPLING_QUINTIC_SPLINES	五次样条采样
TARGET_SAMPLING_S_CURVE_PROFILE	S 曲线轮廓采样
TARGET_SAMPLING_TRAPEZOIDAL_PROFILE	梯形轮廓采样

目标任务轨迹

目标在任务空间中的轨迹，包含任务级别的约束和要求。

Member Variables

Name	Type	Description
group_names	list[str]	Related group names
joint_names	list[str]	Related joint names
subtask_names	list[str]	Subtask names
target_config	TargetConfig	Target configuration
task_commands	list[TaskCommand]	Trajectory points

任务命令

任务级别的命令结构，包含一个或多个目标和执行参数。

Member Variables

Name	Type	Description
------	------	-------------

subtask_commands	list[FrameTriad]	Subtask commands at this point
time_from_start_s	float	Time from trajectory start in seconds

终止条件类型

终止条件类型相关说明。

Enum Value	Description
TIMEOUT	仅当超过最大规划时间时终止
TIMEOUT_AND_EXACT_SOLUTION	超时或精确解

时间戳

表示具有秒和纳秒分量的高精度时间点。兼容 ROS 2builtin_interfaces/Time 和 std_msgs/Header 时间戳格式。

Member Variables

Name	Type	Description
nanosec	int	纳秒
sec	int	秒

轨迹

表示一个完整的机器人轨迹，包含多个随时间变化的路点。joint_groups 和 joint_names 不能同时为空。如果同时为空，函数将返回 ControlStatus::INVALID_INPUT。此限制确保了关节顺序的确定性，防止因隐式内部默认顺序（可能在 SDK 版本之间变化）导致的难以察觉的 bug。使用方法：推荐设置具有语义组名的 joint_groups 或 joint_names。如果同时设置，joint_names 优先于 joint_groups

Member Variables

Name	Type	Description
joint_groups	list[str]	关节组名称列表，如 ["left_arm", "right_arm"]。注意：joint_groups 和 joint_names 不能同时为空
joint_names	list[str]	显式关节名称。当非空时，优先于 joint_groups，并且仅验证为活动关节。当 joint_groups 也为空时，不能同时为空。
points	list[TrajectoryPoint]	有序的轨迹路点列表。每个点的

		joint_command_vec 必须与 joint_names 或 joint_groups 定义的关节顺序匹配。关节顺序的确定方式：(1) 如果 joint_names 非空，则使用 joint_names；(2) 否则使用 joint_groups 按顺序展开（例如，{"head","left_arm"} 展开为 head_joint1, head_joint2, left_arm_joint1, ... left_arm_joint7）。
--	--	--

轨迹控制状态

机器人轨迹执行状态枚举。

表示机器人沿着由多个路点组成的预先规划的轨迹运行时的实时执行状态。

Enum Value	Description
COMPLETED	已完成
DATA_FETCH_FAILED	数据获取失败
ERROR	错误日志
INVALID_INPUT	无效输入
RUNNING	运行中
STOPPED_UNREACHED	轨迹执行过程中停止但未达到终点

轨迹可行性检查选项

轨迹验证和可行性检查配置。

该结构提供了对在轨迹验证期间强制执行可行性约束的细粒度控制。它支持碰撞检测、关节限制合规性和速度剖面可行性的独立切换。有选择地禁用检查可以提高调试、模拟或保证满足某些约束的场景的计算性能。禁用可行性检查可能会产生不安全或物理上无法实现的轨迹。仅当通过其他方式验证约束时才应谨慎使用。

获取是否禁用碰撞检查

```
def get_disable_collision_check() -> bool
```

获取是否禁用碰撞检测

返回值

类型	描述
bool	-

获取是否禁用关节限制检查

```
def get_disable_joint_limit_check() -> bool
```

获取是否禁用关节限制检查

返回值

类型	描述
bool	-

获取是否禁用速度可行性检查

```
def get_disable_velocity_feasibility_check() -> bool
```

获取是否禁用速度可行性检查

返回值

类型	描述
bool	-

打印输出

```
def print() -> None
```

打印轨迹可行性检查选项到标准输出

设置是否禁用碰撞检查

```
def set_disable_collision_check(disable: bool) -> None
```

设置是否禁用碰撞检测

参数

名称	类型	默认值	描述
disable	bool	需要传参	-

 **警告**

禁用碰撞检查可能导致轨迹与环境发生碰撞

设置是否禁用关节限制检查

```
def set_disable_joint_limit_check(disable: bool) -> None
```

设置是否禁用关节限制检查

参数

名称	类型	默认值	描述
disable	bool	需要传参	-

警告

禁用关节限制检查可能导致轨迹超出关节限位

设置是否禁用速度可行性检查

```
def set_disable_velocity_feasibility_check(disable: bool) -> None
```

设置是否禁用速度可行性检查

参数

名称	类型	默认值	描述
disable	bool	需要传参	-

注意

速度可行性确保轨迹可以被机器人执行。

轨迹规划配置

轨迹规划和参数化配置。

该结构配置轨迹生成参数，用于将离散运动计划转换为平滑的时间参数化轨迹。它支持单段和多路点轨迹规划。轨迹规划涉及计算沿几何路径的速度和加速度分布，同时尊重运动学约束。

获取最小移动时间

```
def get_min_move_time() -> float
```

获取最小运动时间（秒）

返回值

类型	描述
float	-

获取直线插值中间点

```
def get_move_line_intermediate_point() -> float
```

获取直线运动中间点设置

返回值

类型	描述
----	----

float	-
-------	---

获取路点规划预期时间

```
def get_way_point_plan_expected_time() -> float
```

获取路点规划期望时间

返回值

类型	描述
float	-

打印输出

```
def print() -> None
```

打印轨迹规划配置信息到标准输出

设置最小移动时间

```
def set_min_move_time(time: SupportsFloat) -> None
```

设置最小运动时间（秒）

参数

名称	类型	默认值	描述
time	SupportsFloat	需要传参	-

注意

非零值防止运动过快；0.0 允许在运动学限制内的最高速度

设置直线插值中间点

```
def set_move_line_intermediate_point(value: SupportsFloat) -> None
```

设置直线运动中间点设置

参数

名称	类型	默认值	描述
value	SupportsFloat	需要传参	-

注意

较高的值提高笛卡尔路径精度但增加计算成本。

设置路点规划预期时间

```
def set_way_point_plan_expected_time(time: SupportsFloat) -> None
```

设置路点规划期望时间

参数

名称	类型	默认值	描述
time	SupportsFloat	需要传参	-

注意

用作时间最优轨迹生成算法的提示。

轨迹点

表示机器人轨迹中的路点，指定特定时间的关节状态。joint_command_vec 的顺序必须与 Trajectory.joint_names 或 Trajectory.joint_groups（按顺序展开）定义的关节顺序一致。顺序不匹配将导致未定义行为。

Member Variables

Name	Type	Description
joint_command_vec	list[JointCommand]	该路点所有关节的关节命令列表。顺序必须与 joint_names 或 joint_groups 展开顺序一致。
time_from_start_second	float	起始时间偏移（秒）

扭曲速度

表示刚体的线速度和角速度组合，遵循 ROS 标准消息格式。

Member Variables

Name	Type	Description
angular	Vector3	Angular velocity vector
linear	Vector3	Linear velocity vector

超声波数据

包含带有时间戳的单个超声波距离测量。

Member Variables

Name	Type	Description
distance	float	到最近障碍物的测量距离（米）。
timestamp_ns	int	超声波测量时间戳（纳秒）

超声波类型

底盘超声波传感器探头枚举（8 个方向）

识别布置在移动底盘周围的各个超声波传感器，用于全向障碍物检测和接近感测。

Enum Value	Description
BACK_LEFT	左后超声波
BACK_RIGHT	右后超声波
FRONT_LEFT	左前超声波
FRONT_RIGHT	右前超声波
LEFT_LEFT	左侧左超声波
LEFT_RIGHT	左侧右超声波
RIGHT_LEFT	右侧左超声波
RIGHT_RIGHT	右侧右超声波

三维向量

表示三维矢量，用于力、扭矩、速度、加速度和其他矢量。

Member Variables

Name	Type	Description
x	float	X 分量
y	float	Y 分量
z	float	Z 分量

全身控制异常

将 MotionStatus 枚举值转换为字符串。

力与力矩

力与力矩相关说明。

Member Variables

Name	Type	Description
force	Vector3	Force vector
torque	Vector3	Torque vector

检查运动状态

```
def check_motion_status(status: MotionStatus) -> str
```

检查当前运动状态，验证运动是否满足约束条件。

参数

名称	类型	默认值	描述
status	MotionStatus	需要传参	运动状态

返回值

类型	描述
str	str: 运动状态的字符串表示。

创建参数

```
def create_parameter(
    direct_execute: bool,
    blocking: bool,
    timeout: SupportsFloat,
    actuate: str,
    tool_pose: bool,
    check_collision: bool,
    frame: str = 'base_link'
) -> Parameter
```

创建运动命令参数，设置执行方式、超时、碰撞检查等选项。

参数

名称	类型	默认值	描述
direct_execute	bool	需要传参	是否直接执行该运动。
blocking	bool	需要传参	是否阻塞执行直到完成。
timeout	SupportsFloat	需要传参	等待运动完成的最长时间。
actuate	str	需要传参	驱动类型（位置/速度/力矩）。
tool_pose	bool	需要传参	该运动是否针对工具位姿。
check_collision	bool	需要传参	是否进行碰撞检测。
frame	str	'base_link'	运动使用的坐标系，默认

			值为 "base_link"。
--	--	--	-----------------

返回值

类型	描述
Parameter	Parameter: 新的 Parameter 实例。

雪山混混

17. C++ API 参考 - G1 机器人

- **GalbotRobot**: 核心机器人控制模块。用于机器人连接、生命周期管理、关节控制、传感器数据查询和硬件状态监控。
- **GalbotMotion**: 运动规划与执行模块。用于笛卡尔空间/关节空间运动、轨迹规划、逆运动学和全身控制。
- **GalbotNavigation**: 移动导航模块。用于移动底盘定位、地图构建、路径规划和自主移动。
- **GalbotPerception**: 端侧感知模块（仅 G1）。加载视觉模型、运行推理，并读取结构化结果（如立体深度）；与 GalbotRobot 传感器 API 配合使用。
- **Types & Enums**: 数据结构、枚举与状态类型。本节用于查询类型定义、传感器类型、错误码及其他模块使用的数据结构。

核心机器人控制

Galbot 人形机器人的主机器人控制界面。

此类提供了用于控制 Galbot 机器人的单例接口。它支持：关节位置和轨迹控制末端执行器控制（夹具和吸盘）移动基座速度控制传感器数据采集（IMU、相机、LiDAR、超声波）坐标系转换系统生命周期管理使用 GalbotRobot::get_instance(MachineType) 获取特定平台 (G1/S1) 的参考。除非另有说明，所有角度均以弧度为单位。除非另有说明，所有线性距离均以米为单位。除非另有说明，所有时间戳均以纳秒为单位。

析构函数

```
virtual galbot::sdk::GalbotRobot::~GalbotRobot()=default
```

初始化

```
virtual bool galbot::sdk::GalbotRobot::init(const std::unordered_set< SensorType >
&enable_sensor_set={})=0
```

初始化机器人控制系统。

初始化机器人硬件通信、中间件和传感器接口。为了优化资源使用，只有在 enable_sensor_set 中指定的传感器才会被初始化并可用于数据读取。

此方法应在程序启动时调用一次。在未调用 destroy() 的情况下多次调用不会报错，但只有第一次调用生效。

参数

Name	Type	Description
enable_sensor_set	const std::unordered_set< SensorType > &	要启用的传感器集合，如果为空，则启用默认的传感器集合，仅指定所需传感器可减少启动时间和资源占用

返回值

Type	Description
bool	初始化成功返回 true，否则返回 false。

发布目标

```
virtual ControlStatus galbot::sdk::GalbotRobot::PublishTarget(const SingoriXTarget &target)=0
```

将目标发布到机器人控制系统，使目标可被运动规划模块使用。

参数

Name	Type	Description
target	const SingoriXTarget &	目标对象

返回值

Type	Description
ControlStatus	无返回值

请求目标

```
virtual std::shared_ptr< ErrorInfo > galbot::sdk::GalbotRobot::RequestTarget(const SingoriXTarget &target)=0
```

向机器人控制系统请求一个目标，用于运动规划和执行。

参数

Name	Type	Description
target	const SingoriXTarget &	目标对象

返回值

Type	Description
std::shared_ptr< ErrorInfo >	返回共享指针：本地预检查失败或服务响应结果。若客户端不可用、连接断开、超时或返回空响应，则返回 nullptr。

设置关节命令

```
virtual ControlStatus galbot::sdk::GalbotRobot::set_joint_commands(  
    const std::vector< JointCommand > &joint_commands,  
    const std::vector< std::string > &joint_groups={},  
    const std::vector< std::string > &joint_names={},  
    const double time_from_start_s=10  
) =0
```

设置低级关节命令以进行高频流控制。

适用于高频命令流（例如，每帧模型推理输出）。

此 API 不会从当前/开始位置插值到第一个目标。控制器尽快将关节驱动到每个命令目标以满足 time_from_start_s（预期到达时间）。

对于标准关节（头、腿、手臂），当前版本仅 JointCommand::position 有效；

目前忽略速度、加速度和作用力。对于夹具关节，位置字段代表夹具宽度，并且速度和作用力字段均受到支持且有效。夹具运动使用指定速度和 time_from_start_s 之间较慢的一个。因此，在设置夹爪速度时，可以将 time_from_start_s 设置为 0（最快到达），夹爪将直接由指定的速度控制。

参数

Name	Type	Description
joint_commands	const std::vector< JointCommand > &	关节命令列表
joint_groups	const std::vector< std::string > &	要控制的关节组。支持的组：legs（腿部）、head（头部）、left_arm（左臂）、right_arm（右臂）、gripper（夹爪）、suction_cup（吸盘）。如果 joint_names 也为空，则不能为空，否则返回 INVALID_INPUT。
joint_names	const std::vector< std::string > &	要控制的具体关节名称。此参数优先于 joint_groups。如果提供了 joint_names，则忽略 joint_groups。如果 joint_groups 也为空，则不能为空，否则返回 INVALID_INPUT。
time_from_start_s	const double	预期到达时间（秒）

返回值

Type	Description
ControlStatus	控制状态，指示命令传输的成功或失败

警告

尤其在第一条命令下发时，请避免当前关节角与目标关节角差值过大。角度突变可能导致运动过快并带来安全风险。

设置关节位置

```
virtual ControlStatus galbot::sdk::GalbotRobot::set_joint_positions(  
    const std::vector< double > &joint_positions,  
    const std::vector< std::string > &joint_groups={},  
    const std::vector< std::string > &joint_names={},  
    const bool is_blocking=true,  
    const double speed_rad_s=0.2,  
    const double timeout_s=15  
) =0
```

按名称设置指定关节组的目标关节位置（用于低频关键帧/姿态转换） 命令机器人将指定关节移动到目标位置。

该运动以具有可配置速度限制的平滑轨迹执行。

参数

Name	Type	Description
joint_positions	const std::vector< double > &	目标关节位置
joint_groups	const std::vector< std::string > &	要控制的关节组名称。支持的组：腿部、头部、左臂、右臂。如果 joint_names 也为空，则不能为空，否则返回 INVALID_INPUT。
joint_names	const std::vector< std::string > &	要控制的具体关节名称。此参数优先于 joint_groups。如果提供了 joint_names，则忽略 joint_groups。如果 joint_groups 也为空，则不能为空，否则返回 INVALID_INPUT。
is_blocking	const bool	是否阻塞等待完成。true：阻塞直到完成或超时；false：发送后立即返回。
speed_rad_s	const double	最大运动速度（弧度/秒）
timeout_s	const double	超时时间（秒）

返回值

Type	Description
ControlStatus	控制状态，指示运动命令的成功或失败

警告

此 API 不适合高频率的逐帧运动控制。

检查轨迹执行状态

```
virtual std::vector< TrajectoryControlStatus >
galbot::sdk::GalbotRobot::check_trajectory_execution_status(
    std::vector< std::string > joint_groups
) =0
```

获取指定关节组的轨迹执行状态。

查询指定关节组的轨迹当前执行状态。这对于在非阻塞执行模式下监视轨迹进度很有用。

参数

Name	Type	Description
joint_groups	std::vector< std::string >	关节组列表

返回值

Type	Description
std::vector< TrajectoryControlStatus >	轨迹控制状态列表：轨迹执行状态列表。

执行关节轨迹

```
virtual ControlStatus galbot::sdk::GalbotRobot::execute_joint_trajectory(
    Trajectory trajectory,
    bool is_blocking=true
) =0
```

执行预先计划的关节轨迹。

执行由具有关联关节位置、速度和计时信息的航点组成的轨迹。轨迹控制器在路点之间进行插值以生成平滑的运动。

参数

Name	Type	Description
trajectory	Trajectory	轨迹数据结构，包含路点和时序信息。必须指定 trajectory.joint_groups 或 trajectory.joint_names；如果两者都为空，则返回 INVALID_INPUT。
is_blocking	bool	是否阻塞等待完成

返回值

Type	Description
ControlStatus	控制状态，指示轨迹执行/提交的成功或失败

 **警告**

每个 TrajectoryPoint.joint_command_vec 的顺序必须与 trajectory.joint_names 定义的关节顺序或 trajectory.joint_groups 的展开顺序一致。

警告

对于逐帧模型推理输出，建议使用命令流式接口（set_joint_commands / set_joint_commands_batch），而不是反复重新提交完整轨迹。

批量设置关节命令

```
virtual ControlStatus galbot::sdk::GalbotRobot::set_joint_commands_batch(const Trajectory
&trajectory)=0
```

以批处理模式设置关节命令（非阻塞） 实时控制模式下设置多个关节命令轨迹点，支持一次性提交多个时间点的轨迹控制命令。

提供非阻塞高频轨迹执行接口。与 set_joint_commands 类似，但支持批量轨迹控制，适用于 VLA 推理批量输出等场景。

参数

Name	Type	Description
trajectory	const Trajectory &	包含关节命令路点的轨迹数据结构。必须指定 trajectory.joint_groups 或 trajectory.joint_names；如果两者都为空，则返回 INVALID_INPUT。每个 TrajectoryPoint 包含 time_from_start 和关节命令列表。关节命令包括位置（弧度）、速度（弧度/秒）、加速度（弧度/秒²）、作用力（牛·米）、Kp（位置增益）和 Kd（速度增益）。

返回值

Type	Description
ControlStatus	控制状态，指示命令提交的成功或失败。立即返回，不等待执行完成（非阻塞）。

警告

每个 TrajectoryPoint.joint_command_vec 的顺序必须与 trajectory.joint_names 定义的关节顺序或 trajectory.joint_groups 的展开顺序一致。

设置吸盘命令

```
virtual ControlStatus galbot::sdk::GalbotRobot::set_suction_cup_command(
    const std::string &end_effector,
```

```
bool activate
) =0
```

控制吸盘激活状态。

激活或停用指定的吸盘末端执行器。

参数

Name	Type	Description
end_effector	const std::string &	吸盘末端执行器
activate	bool	是否激活吸盘

返回值

Type	Description
ControlStatus	控制状态，指示命令传输的成功或失败

支持平台: G1

设置夹爪命令

```
virtual ControlStatus galbot::sdk::GalbotRobot::set_gripper_command(
    const std::string &end_effector,
    double width_m,
    double velocity_mps=0.03,
    double effort=30,
    bool is_blocking=true
) =0
```

控制夹具张开宽度和力度。

命令夹具以受控的速度和最大夹持力移动到指定的开口宽度。

参数

Name	Type	Description
end_effector	const std::string &	要控制的夹爪关节组名称（例如 left_gripper、right_gripper）。
width_m	double	目标开口宽度（米）
velocity_mps	double	夹爪运动速度（米/秒）
effort	double	最大夹持力（牛顿·米，N·m）。
is_blocking	bool	是否阻塞等待完成

返回值

Type	Description
ControlStatus	控制状态，指示夹爪命令的成功或失败

获取夹爪状态

```
virtual std::shared_ptr< GripperState > galbot::sdk::GalbotRobot::get_gripper_state(
    const std::string &end_effector
) =0
```

获取指定夹爪的当前状态。

返回夹爪的位置、速度、受力以及运动状态估计信息。

GripperState.is_moving 基于时间窗口判定：若在内部窗口内未检测到有效开合变化，则该值会变为 false。

参数

Name	Type	Description
end_effector	const std::string &	要查询的夹爪关节组名称（例如 left_gripper、right_gripper）。

返回值

Type	Description
std::shared_ptr< GripperState >	返回夹爪状态的共享指针；若读取失败，返回 nullptr。

获取吸盘状态

```
virtual std::shared_ptr< SuctionCupState >
galbot::sdk::GalbotRobot::get_suction_cup_state(
    const std::string &end_effector
) =0
```

获取当前吸盘状态。

检索指定吸盘的当前状态，包括激活状态和真空压力测量。

参数

Name	Type	Description
end_effector	const std::string &	吸盘末端执行器

返回值

Type	Description
std::shared_ptr< SuctionCupState >	返回吸盘状态的共享指针；若读取失败，返回 nullptr。

支持平台: G1

获取灵巧手状态

```
virtual ControlStatus galbot::sdk::GalbotRobot::get_dexhand_state(
    const std::string &end_effector,
```



```

    DexhandState &dexhand_state,
    DexHandType dexhand_type=DexHandType::INSPIRE
) =0

```

获取当前灵巧手（dexhand）状态。

将灵巧手反馈写入 dexhand_state。对于 INSPIRE 和 BRAINCO，仅填充 dexhand_state.joint_state，force_sensor_map 为空；对于 SHARPA，会填充完整关节状态，并在可用时返回命名力传感器数据。

参数

Name	Type	Description
end_effector	const std::string &	要查询的灵巧手关节组名称（例如 left_dexhand、right_dexhand）。
dexhand_state	DexhandState &	输出：灵巧手状态（joint_state 及可选 force_sensor_map）。
dexhand_type	DexHandType	灵巧手型号（INSPIRE、BRAINCO 或 SHARPA）。

返回值

Type	Description
ControlStatus	控制状态，指示成功或失败。

设置灵巧手命令

```

virtual ControlStatus galbot::sdk::GalbotRobot::set_dexhand_command(
    const std::string &end_effector,
    const std::vector< JointCommand > &dexhand_command,
    DexHandType dexhand_type=DexHandType::INSPIRE,
    bool is_blocking=true
) =0

```

通过关节命令控制灵巧手。

使用关节命令向量控制灵巧手（位置、速度、力矩等）。

参数

Name	Type	Description
end_effector	const std::string &	要控制的灵巧手名称（例如 left_dexhand、right_dexhand）。
dexhand_command	const std::vector< JointCommand > &	每个灵巧手关节的命令向量。Inspire: [位置, 速度, 加速度, 力矩] 范围 [0-1000, 0-1000, , 0-1000]。BrainCo: [位置, 速度, 加速度, 力矩] 范围 [0-100, -100-100, ,]。Sharpa: 22 个关节命令 [位置, 速度, 加速度, 力矩]。
dexhand_type	DexHandType	灵巧手型号。
is_blocking	bool	是否阻塞等待动作完成。

返回值

Type	Description
ControlStatus	控制状态，表示命令发布结果。

获取关节位置

```
virtual std::vector< double > galbot::sdk::GalbotRobot::get_joint_positions(  
    const std::vector< std::string > &joint_groups,  
    const std::vector< std::string > &joint_names  
) =0
```

获取关节位置

参数

Name	Type	Description
joint_groups	const std::vector< std::string > &	关节组列表
joint_names	const std::vector< std::string > &	关节名称列表

返回值

Type	Description
std::vector< double >	当前关节角度向量（弧度）

注意

返回值顺序：指定 joint_names 时按 joint_names 顺序；仅指定 joint_groups 时按组定义顺序；都为空时按 chassis、head、left_arm、right_arm、leg 顺序返回身体关节

获取关节组名称

```
virtual std::vector< std::string > galbot::sdk::GalbotRobot::get_joint_group_names()=0
```

获取机器人可用的关节组名称。

检索机器人运动学配置中定义的所有关节组名称。这对于在运行时发现可用的控制组很有用。

返回值

Type	Description
std::vector< std::string >	关节组名称向量，或如果检索失败则为空向量

获取关节名称

```
virtual std::vector< std::string > galbot::sdk::GalbotRobot::get_joint_names(  
    bool only_active_joint=true,  
    const std::vector< std::string > &joint_groups={}  
) =0
```

通过组名获取机器人关节名称。

检索属于指定关节组的关节名称。这对于在设置关节位置时确定正确的顺序很有用。

参数

Name	Type	Description
only_active_joint	bool	是否仅返回主动关节
joint_groups	const std::vector< std::string > &	关节组列表

返回值

Type	Description
std::vector< std::string >	返回关节名称列表，顺序取决于参数配置

获取关节状态

```
virtual std::vector< JointState > galbot::sdk::GalbotRobot::get_joint_states(  
    const std::vector< std::string > &joint_group_vec,  
    const std::vector< std::string > &joint_names_vec={}  
) =0
```

通过组名称获取实时关节状态。

检索指定关节的综合状态信息，包括位置、速度、加速度、力（扭矩）和其他反馈数据。

参数

Name	Type	Description
joint_group_vec	const std::vector< std::string > &	关节组向量
joint_names_vec	const std::vector< std::string > &	关节名称向量

返回值

Type	Description
std::vector< JointState >	结构向量，包含每个关节的当前状态

注意

返回值顺序：指定 joint_names 时按 joint_names 顺序；仅指定 joint_groups 时按组定义顺序；都为空时按 chassis、head、left_arm、right_arm、leg 顺序返回身体关节

设置底盘速度

```
virtual ControlStatus galbot::sdk::GalbotRobot::set_base_velocity(  
    const std::array< double, 3 > &linear_velocity,  
    const std::array< double, 3 > &angular_velocity,  
    double duration_s=0.0  
) =0
```

设置移动基础速度命令。

命令机器人的移动底座以指定的线速度和角速度移动。速度在机器人的基坐标系中表示。

参数

Name	Type	Description
linear_velocity	const std::array< double, 3 > &	线速度（米/秒），在底座坐标系中表示。顺序：{vx, vy, vz}- vx: X 方向线速度（前后）- vy: Y 方向线速度（左右）- vz: Z 方向线速度（垂直）
angular_velocity	const std::array< double, 3 > &	角速度（弧度/秒），在底座坐标系中表示。顺序：{wx, wy, wz}- wx: 绕 X 轴角速度（翻滚）- wy: 绕 Y 轴角速度（俯仰）- wz: 绕 Z 轴角速度（偏航）
duration_s	double	持续时间（秒）

返回值

Type	Description
ControlStatus	控制状态，指示命令传输的成功或失败

设置底盘姿态

```
virtual ControlStatus galbot::sdk::GalbotRobot::set_base_pose(  
    const Pose &base_pose,  
    bool is_blocking=true,  
    double timeout_s=15.0  
) =0
```

设置移动基础位姿命令。

命令机器人的移动底座移动到其参考坐标系中的指定姿态。这使用底盘姿态控制器（CHASSIS_POSE_CTRL）。当完整的 3D 姿态（位置 + 四元数方向）已经可用时，使用此重载。

参数

Name	Type	Description
base_pose	const Pose &	底盘目标姿态
is_blocking	bool	是否阻塞等待完成
timeout_s	double	超时时间（秒）

返回值

Type	Description
ControlStatus	控制状态，指示命令传输的成功或失败

设置底盘姿态

```
virtual ControlStatus galbot::sdk::GalbotRobot::set_base_pose(
    double x,
    double y,
    double yaw,
    const std::string &frame_id="odom",
    const std::string &reference_frame_id="odom",
    bool is_blocking=true,
    double timeout_s=15.0
) =0
```

使用可选坐标系设置移动基础姿态（x、y、航向角）。

将此重载用于由所选坐标系中的 x/y/航向角定义的平面 2D 目标命令。

参数

Name	Type	Description
x	double	X 坐标（米）
y	double	Y 坐标（米）
yaw	double	航向角（弧度）
frame_id	const std::string &	坐标系 ID
reference_frame_id	const std::string &	参考坐标系 ID
is_blocking	bool	是否阻塞等待完成
timeout_s	double	超时时间（秒）

返回值

Type	Description
ControlStatus	控制状态，指示命令传输的成功或失败

设置底盘姿态

```
virtual ControlStatus galbot::sdk::GalbotRobot::set_base_pose(
    double x,
    double y,
    double yaw,
    const std::string &frame_id,
    const std::string &reference_frame_id,
    double time_from_start_s,
    bool is_blocking=true,
    double timeout_s=15.0
) =0
```

使用明确的插值时间设置移动基础姿态（x、y、航向角）。

当必须通过 time_from_start_s 协调到达时间时，请使用此重载。

参数

Name	Type	Description
------	------	-------------

x	double	X 坐标 (米)
y	double	Y 坐标 (米)
yaw	double	航向角 (弧度)
frame_id	const std::string &	坐标系 ID
reference_frame_id	const std::string &	参考坐标系 ID
time_from_start_s	double	预期到达时间 (秒)
is_blocking	bool	是否阻塞等待完成
timeout_s	double	超时时间 (秒)

返回值

Type	Description
ControlStatus	控制状态，指示命令传输的成功或失败

设置末端执行器命令

```
virtual ControlStatus galbot::sdk::GalbotRobot::set_end_effector_command(  
    const std::vector< std::vector< double > > &poses,  
    const std::vector< std::string > &end_effector_frames,  
    const std::vector< std::string > &reference_frames={},  
    double time_from_start_s=0.0  
) =0
```

设置 WBC 末端执行器位姿命令（任务轨迹发布）。

poses 中每一行为 [x, y, z, qx, qy, qz, qw]（单位：米，四元数顺序 xyzw）。poses 与 end_effector_frames 的长度必须一致。

参数

Name	Type	Description
poses	const std::vector< std::vector< double > > &	每个末端执行器对应一个位姿；每行格式为 [x, y, z, qx, qy, qz, qw]（单位：米，四元数顺序 xyzw）。
end_effector_frames	const std::vector< std::string > &	每个位姿对应的目标坐标系 id（例如连杆名称）。
reference_frames	const std::vector< std::string > &	每个位姿对应的参考坐标系。省略或传入 [] 时默认对所有位姿使用 "world"。否则长度必须与 poses 一致。常用值："world"（默认）。
time_from_start_s	double	从轨迹起点开始的时间（秒，可选，默认 0.0）。

返回值

Type	Description
ControlStatus	控制状态，表示命令发布结果。

清除末端执行器命令

```
virtual ControlStatus galbot::sdk::GalbotRobot::clear_end_effector_command()=0
```

清除 WBC 末端执行器任务命令。
清除已发布到 WBC 通道的末端执行器任务轨迹命令。

返回值

Type	Description
ControlStatus	控制状态，表示命令发布结果。

获取 WBC 末端执行器位姿

```
virtual std::unordered_map< std::string, std::vector< double > >  
galbot::sdk::GalbotRobot::get_wbc_end_effector_poses() =0
```

获取 WBC 末端执行器位姿。
返回 WBC 各末端执行器（左臂、右臂、头部）的当前位姿。

返回值

Type	Description
std::unordered_map< std::string, std::vector< double > >	字典映射，包含：- "lee_pose": 左臂末端位姿 [x, y, z, qx, qy, qz, qw]- "ree_pose": 右臂末端位姿 [x, y, z, qx, qy, qz, qw]- "head_pose": 头部末端位姿 [x, y, z, qx, qy, qz, qw]当失败或缺少键时，可能返回空条目或部分数据。

紧急停止底盘

```
virtual ControlStatus galbot::sdk::GalbotRobot::stop_base()=0
```

紧急停止移动底座运动。
立即命令移动基地停止一切运动。这是一项安全功能，当需要立即停止基本运动时应使用。

返回值

Type	Description
ControlStatus	控制状态，指示命令传输的成功或失败

全身与底盘归零

```
virtual std::pair< MotionStatus, ControlStatus >  
galbot::sdk::GalbotRobot::zero_whole_body_and_base(  
    const Pose &base_zero_pose,  
    bool is_blocking=true,  
    double leg_head_speed_rad_s=0.2,
```

```
double leg_head_timeout_s=15.0,
std::shared_ptr< Parameter > params=nullptr
) =0
```

一键归零：将全身关节归零，并将底盘位姿回到零位。

该接口会调用 `move_whole_body_joint_zero` 完成关节归零，并将底盘目标位姿设为零。若 `params` 为空指针，则使用 `default_param`。

参数

Name	Type	Description
base_zero_pose	const Pose &	底盘零位姿态
is_blocking	bool	是否阻塞等待完成
leg_head_speed_rad_s	double	腿部和头部关节速度（弧度/秒）
leg_head_timeout_s	double	腿部和头部运动超时（秒）
params	std::shared_ptr< Parameter >	额外参数

返回值

Type	Description
std::pair< MotionStatus, ControlStatus >	元组 (运动状态（关节），控制状态（底座）)

全身与底盘归零

```
virtual std::pair< MotionStatus, ControlStatus >
galbot::sdk::GalbotRobot::zero_whole_body_and_base(
    const std::string &frame_id="odom",
    const std::string &reference_frame_id="odom",
    bool is_blocking=true,
    double leg_head_speed_rad_s=0.2,
    double leg_head_timeout_s=15.0,
    std::shared_ptr< Parameter > params=nullptr
) =0
```

一键归零：通过可选坐标系将全身关节归零，并将底座（x、y、航向角）归零。

参数

Name	Type	Description
frame_id	const std::string &	坐标系 ID
reference_frame_id	const std::string &	参考坐标系 ID
is_blocking	bool	是否阻塞等待完成
leg_head_speed_rad_s	double	腿部和头部关节速度（弧度/秒）
leg_head_timeout_s	double	腿部和头部运动超时（秒）
params	std::shared_ptr< Parameter >	额外参数

返回值

Type	Description
------	-------------

std::pair< MotionStatus, ControlStatus >	元组 (运动状态 (关节) , 控制状态 (底座))
--	-----------------------------

停止轨迹执行

```
virtual ControlStatus galbot::sdk::GalbotRobot::stop_trajectory_execution()=0
```

停止所有当前正在执行的关节轨迹。

立即停止执行所有关节组中的所有活动关节轨迹。停止后关节将保持当前位置。

返回值

Type	Description
ControlStatus	控制状态，指示命令传输的成功或失败

重新加载控制器

```
virtual ControlStatus galbot::sdk::GalbotRobot::reload_controller(const std::string &group_name="all")=0
```

重新加载控制器。

重新初始化控制器。相当于一个完整的重启循环：停止->重置->启动。对于错误恢复或应用配置更改很有用。

参数

Name	Type	Description
group_name	const std::string &	要重新加载的控制器组名称

返回值

Type	Description
ControlStatus	控制状态，指示重新加载操作的成功或失败

切换控制器

```
virtual ControlStatus galbot::sdk::GalbotRobot::switch_controller(const std::string &controller_name)=0
```

切换活动控制器策略。

将硬件控制切换到新的策略。请求会发送到 WBCS 控制器管理器，由其执行安全切换，并在接受请求后启动目标控制器。

参数

Name	Type	Description
controller_name	const std::string &	控制器名称字符串，例如 "chassis_pose_ctrl"。

返回值

Type	Description
ControlStatus	控制状态，指示切换操作的成功或失败

获取当前控制器

```
virtual std::string galbot::sdk::GalbotRobot::get_active_controller(const std::string
&group_name)=0
```

获取指定关节组的活动控制器名称。

返回当前 SDK 实例记录的该关节组控制器。该值会以模型默认控制器初始化，并在 SDK 控制器管理接口成功调用后更新。若控制已释放或该关节组无已知控制器，则返回 "CONTROLLER_NAME_NUM"。

参数

Name	Type	Description
group_name	const std::string &	要查询的关节组名称。

返回值

Type	Description
std::string	std::string：当前 SDK 实例已知的活动控制器名称。

获取控制器权限

```
virtual ControlStatus galbot::sdk::GalbotRobot::acquire_controller(const std::string
&controller_name)=0
```

获取硬件控制权限。

请求指定控制器接管硬件控制。该操作与 switch_controller 使用同一套 WBCS 切换请求机制。

参数

Name	Type	Description
controller_name	const std::string &	控制器名称字符串，例如 "left_arm_pvt_ctrl"。

返回值

Type	Description
ControlStatus	控制状态，指示获取操作的成功或失败

释放控制器权限

```
virtual ControlStatus galbot::sdk::GalbotRobot::release_controller(const std::string
&group_name="all")=0
```

释放硬件权限。

控制硬件，释放关节。与 acquire_controller 相反。如果运行则隐式停止执行。

参数

Name	Type	Description
group_name	const std::string &	要释放的关节组名称，支持的组：chassis（底盘）、legs（腿部）、head（头部）、left_arm（左臂）、right_arm（右臂）、gripper（夹爪）、suction_cup（吸盘）或 "all"（释放所有控制器）

返回值

Type	Description
ControlStatus	控制状态，指示释放操作的成功或失败

启动控制器

```
virtual ControlStatus galbot::sdk::GalbotRobot::start_controller(const std::string
&group_name="all")=0
```

开始控制器执行。

激活控制器以开始发送命令。与 stop_controller 相反。需要事先获得硬件权限（获取）。

参数

Name	Type	Description
group_name	const std::string &	要启动的控制器组名称

返回值

Type	Description
ControlStatus	控制状态，指示启动操作的成功或失败

停止控制器

```
virtual ControlStatus galbot::sdk::GalbotRobot::stop_controller(const std::string
&group_name="all")=0
```

停止控制器执行。

停止命令执行但保留硬件权限。与 start_controller 相反。

参数

Name	Type	Description
group_name	const std::string &	要停止的控制器组名称

返回值

Type	Description
ControlStatus	控制状态，指示停止操作的成功或失败

获取 IMU 数据

```
virtual std::shared_ptr< ImuData > galbot::sdk::GalbotRobot::get_imu_data(SensorType imu_type)=0
```

获取 IMU（惯性测量单元）传感器数据。

检索最新的 IMU 测量值，包括线性加速度、角速度和方向估计。

参数

Name	Type	Description
imu_type	SensorType	IMU 传感器类型

返回值

Type	Description
std::shared_ptr< ImuData >	返回 IMU 数据结构的共享指针；若传感器未启用或读取失败，返回 nullptr。

注意

IMU 传感器必须在初始化期间通过 enable_sensor_set 启用。

注意

加速度以米每二次方秒（m/s²）为单位。

注意

角速度以弧度每秒（rad/s）为单位。

获取里程计数据

```
virtual std::shared_ptr< OdomData > galbot::sdk::GalbotRobot::get_odom()=0
```

获取机器人里程信息。

从里程计系统检索机器人当前的姿态和速度估计。里程计通常融合车轮编码器、IMU 和其他本体感觉传感器。

返回值

Type	Description
std::shared_ptr< OdomData >	返回里程计数据的共享指针，包含位置、姿态、线速度、角速度与时间戳。若里程计不可用，返回空指针。

获取设备信息

```
virtual std::shared_ptr< DeviceInfo > galbot::sdk::GalbotRobot::get_device_information()=0
```

获取设备信息。

检索基本设备信息，包括设备型号、序列号、固件版本、硬件版本和制造商。此信息用于设备管理、版本控制、系统诊断和设备识别。

返回值

Type	Description
std::shared_ptr< DeviceInfo >	返回设备信息的共享指针，包含型号、序列号、固件版本、硬件版本和厂商信息。若读取失败，返回空指针。

获取 RGB 图像数据

```
virtual std::shared_ptr< RgbData > galbot::sdk::GalbotRobot::get_rgb_data(const SensorType rgb_camera)=0
```

从指定相机获取最新的 RGB 图像。

检索指定 RGB 相机捕获的最新彩色图像。

参数

Name	Type	Description
rgb_camera	const SensorType	RGB 相机名称

返回值

Type	Description
std::shared_ptr< RgbData >	返回 RGB 图像数据的共享指针，包含图像缓冲区、分辨率、编码与时间戳。若相机未启用或读取失败，返回空指针。

注意

相机传感器必须在初始化期间通过 enable_sensor_set 启用。

获取深度图像数据

```
virtual std::shared_ptr< DepthData > galbot::sdk::GalbotRobot::get_depth_data(const SensorType depth_camera)=0
```

从指定相机获取最新的深度图像。

检索指定深度相机捕获的最新深度图像。深度值通常表示距相机传感器的距离。

参数

Name	Type	Description
depth_camera	const SensorType	深度相机名称

返回值

Type	Description
std::shared_ptr< DepthData >	返回深度图像数据的共享指针，包含图像缓冲区、分辨率、编码与时间戳。若相机未启用或读取失败，返回空指针。

注意

相机传感器必须在初始化期间通过 enable_sensor_set 启用。

注意

深度值通常以毫米（mm）或米（m）为单位。

获取激光雷达数据

```
virtual std::shared_ptr< LidarData > galbot::sdk::GalbotRobot::get_lidar_data(const
SensorType lidar)=0
```

获取最新的激光雷达点云数据。

检索指定 LiDAR 传感器捕获的最新 3D 点云。每个点通常包含 (x, y, z) 坐标和可选的强度值。

参数

Name	Type	Description
lidar	const SensorType	激光雷达名称

返回值

Type	Description
std::shared_ptr< LidarData >	返回 PointCloud2 点云数据的共享指针（坐标单位为米，参考 LiDAR 坐标系）。若 LiDAR 未启用或读取失败，返回空指针。

注意

LiDAR 传感器必须在初始化期间通过 enable_sensor_set 启用。

获取超声波数据

```
virtual std::shared_ptr< UltrasonicData > galbot::sdk::GalbotRobot::get_ultrasonic_data(  
    const UltrasonicType ultrasonic_type  
) =0
```

从指定的超声波传感器获取距离测量值。

从超声波距离传感器之一检索最新的距离测量值。机器人通常在其周边布置有多个超声波传感器。

参数

Name	Type	Description
ultrasonic_type	const UltrasonicType	超声波传感器类型

返回值

Type	Description
std::shared_ptr< UltrasonicData >	返回超声波测距数据的共享指针（单位：米）；若传感器未启用或读取失败，返回 nullptr。

注意

超声波传感器必须在初始化期间通过 enable_sensor_set 启用。

支持平台: G1

获取相机内参

```
virtual std::shared_ptr< CameraInfo > galbot::sdk::GalbotRobot::get_camera_intrinsic(  
    const SensorType camera  
) =0
```

获取相机内参。

获取指定相机的内参，包括焦距、主点、畸变系数等。

参数

Name	Type	Description
camera	const SensorType	相机名称

返回值

Type	Description
std::shared_ptr< CameraInfo >	返回相机内参数据的共享指针，包含焦距、主点坐标、畸变系数等参数。若相机未启用或读取失败，返回空指针。

注意

相机传感器必须在初始化期间通过 enable_sensor_set 启用。

获取坐标变换

```
virtual std::pair< std::vector< double >, int64_t >
galbot::sdk::GalbotRobot::get_transform(
    const std::string &target_frame,
    const std::string &source_frame,
    int64_t timestamp_ns=0,
    int64_t timeout_ms=100
) =0
```

查询坐标系变换（TF） 查询机器人 TF 树中两个坐标系之间的变换。

这用于在不同参考坐标系之间转换姿态和位置（例如，从相机坐标系到基坐标系，从末端执行器到世界坐标系）。

参数

Name	Type	Description
target_frame	const std::string &	目标坐标系
source_frame	const std::string &	源坐标系
timestamp_ns	int64_t	时间戳（纳秒）
timeout_ms	int64_t	超时时间（毫秒）

返回值

Type	Description
std::pair< std::vector< double >, int64_t >	包含以下内容的 Pair: - 7 个 double 的向量，表示变换 [x, y, z, qx, qy, qz, qw]，其中 (x, y, z) 为平移（米），(qx, qy, qz, qw) 为四元数方向 - 变换有效时的时间戳（纳秒）。如果检索失败或超时则返回空向量，时间戳为 0。

获取坐标系名称

```
virtual std::vector< std::string > galbot::sdk::GalbotRobot::get_frame_names()=0
```

获取 TF 树中所有可用的坐标系名称。

返回值

Type	Description
std::vector< std::string >	所有可用坐标系名称列表。

获取传感器外参

```
virtual std::pair< std::vector< double >, int64_t >
galbot::sdk::GalbotRobot::get_sensor_extrinsic(
    const SensorType sensor_id,
    const std::string &reference_frame="base_link"
) =0
```


获取传感器外参。

检索指定传感器的外部参数，包括相对于机器人基础坐标系的旋转和平移向量。

参数

Name	Type	Description
sensor_id	const SensorType	传感器 ID
reference_frame	const std::string &	参考坐标系

返回值

Type	Description
std::pair< std::vector< double >, int64_t >	包含以下内容的 Pair： - 7 个 double 的向量，表示变换 [x, y, z, qx, qy, qz, qw]，其中 (x, y, z) 为平移（米），(qx, qy, qz, qw) 为四元数方向 - 变换有效时的时间戳（纳秒）。如果检索失败则返回空向量，时间戳为 0。

注意

传感器必须在初始化期间通过 enable_sensor_set 启用。

获取力传感器数据

```
virtual std::shared_ptr< ForceData > galbot::sdk::GalbotRobot::get_force_sensor_data(
    const GalbotOneFoxtrotSensor sensor_type
) =0
```

获取力/扭矩传感器数据。

从指定的力/扭矩传感器检索最新测量值。这些传感器通常安装在手腕或末端执行器上，用于接触力监测和合规性控制。

参数

Name	Type	Description
sensor_type	const GalbotOneFoxtrotSensor	力传感器类型

返回值

Type	Description
std::shared_ptr< ForceData >	返回力传感器数据的共享指针，包含力向量 [fx, fy, fz]、力矩向量 [tx, ty, tz] 以及时间戳。若传感器未启用或读取失败，返回空指针。

注意

力传感器必须在初始化期间通过 enable_sensor_set 启用。

支持平台: G1

启动麦克风流输入

```
virtual std::string galbot::sdk::GalbotRobot::start_microphone_stream_input(
    std::function< void(const std::shared_ptr< AudioData >)> callback,
    int chunk_size=2560,
    bool use_raw_audio=false
) =0
```

启动麦克风流音频输入。

参数

Name	Type	Description
callback	std::function< void(const std::shared_ptr< AudioData >)>	音频数据回调函数
chunk_size	int	每个音频块的样本数
use_raw_audio	bool	是否使用原始音频。 - false: 使用处理后的音频（默认） - true: 输出原始 PCM 音频数据（目前不支持动态配置）

返回值

Type	Description
std::string	str：用于标识音频输入流的流 ID。

支持平台: G1

停止麦克风流输入

```
virtual void galbot::sdk::GalbotRobot::stop_microphone_stream_input(const std::string
&stream_id="")=0
```

停止指定的麦克风流式音频输入。

参数

Name	Type	Description
stream_id	const std::string &	输出流 ID

支持平台: G1

写入音频流输出

```
virtual bool galbot::sdk::GalbotRobot::write_audio_stream_output(
    const std::string &audio_chunk,
    const std::string &stream_id=""
) =0
```

将 PCM 格式的音频数据块写入音频输出流以进行实时播放。

参数

Name	Type	Description
audio_chunk	const std::string &	音频数据块
stream_id	const std::string &	输出流 ID

返回值

Type	Description
bool	返回操作结果：音频写入并成功下发播放任务为 true，失败为 false。

支持平台: G1

停止音频流输出

```
virtual void galbot::sdk::GalbotRobot::stop_audio_stream_output(const std::string &stream_id="")=0
```

停止指定音频输出流或所有活动音频输出流的播放。

参数

Name	Type	Description
stream_id	const std::string &	输出流 ID

支持平台: G1

获取音量

```
virtual float galbot::sdk::GalbotRobot::get_volume()=0
```

获取当前系统全局卷值。

返回值

Type	Description
float	float：返回当前音量值，范围为 0.0 到 100.0

支持平台: G1

设置音量

```
virtual bool galbot::sdk::GalbotRobot::set_volume(float volume)=0
```

设置系统全局音量值。

参数

Name	Type	Description
volume	float	音量值 (0-100)

返回值

Type	Description
bool	返回音量设置结果：成功为 true，失败为 false。

支持平台: G1

检查是否运行中

```
virtual bool galbot::sdk::GalbotRobot::is_running()=0
```

检查机器人控制系统是否正在运行。

查询机器人控制系统是否仍处于活动状态，或者是否已收到关闭信号（例如，SIGINT、SIGTERM）。

返回值

Type	Description
bool	系统正常运行时返回 true，否则返回 false。

请求关机

```
virtual void galbot::sdk::GalbotRobot::request_shutdown()=0
```

请求系统关闭。

发送关闭信号以启动系统正常关闭。这会触发已注册的退出回调并开始资源清理。作为关闭序列的第一步调用：request_shutdown() -> wait_for_shutdown() -> destroy()。

等待关机

```
virtual void galbot::sdk::GalbotRobot::wait_for_shutdown()=0
```

阻塞直到关闭完成。

阻塞调用线程直到所有模块都正常关闭完毕。作为关闭序列的第二步调用，在 request_shutdown() 之后、destroy() 之前。

注意

当 is_running() 变为 false 时此函数将返回。

销毁

```
virtual void galbot::sdk::GalbotRobot::destroy()=0
```

清理系统资源。

执行机器人控制系统资源的最终清理，包括中间件连接、传感器接口和通信通道。这是关闭序列的最后一步：request_shutdown() -> wait_for_shutdown() -> destroy()。

此方法应在程序结束时调用一次。调用 destroy() 后，SDK 进入终态，无法在同一进程重新初始化。如需再次使用，请退出当前进程并启动新进程。

注册退出回调

```
virtual void galbot::sdk::GalbotRobot::register_exit_callback(std::function< void()>
exit_function)=0
```

注册关闭事件的回调函数。

注册一个用户定义的回调函数，当收到关闭信号时将调用该函数。可以注册多个回调，并按照注册顺序执行。

参数

Name	Type	Description
exit_function	std::function< void()>	退出时执行的回调函数，签名：void()，用于在程序退出时执行清理操作（例如保存数据、释放资源）

注意

回调函数应快速完成以避免延迟关闭。

获取电池管理信息

```
virtual std::shared_ptr< BmsInfo > galbot::sdk::GalbotRobot::get_bms_information()=0
```

获取 BMS（电池管理系统）信息。

返回值

Type	Description
std::shared_ptr< BmsInfo >	共享指针，包含电池信息

支持平台: G1

获取日志信息

```
virtual std::shared_ptr< LogInfo > galbot::sdk::GalbotRobot::get_log_information(
    uint64 timewindow_s,
    LogLevel log_level
) =0
```

获取日志信息。

参数

Name	Type	Description
timewindow_s	uint64	
log_level	LogLevel	

返回值

Type	Description
std::shared_ptr< LogInfo >	共享指针，包含日志信息

获取实例

```
static GalbotRobot & galbot::sdk::GalbotRobot::get_instance(MachineType m)
```

运行时工厂方法，用于选择具体机器人单例。

参数

Name	Type	Description
m	MachineType	机器类型

返回值

Type	Description
GalbotRobot &	指定机器类型的单例机器人接口引用。

运动规划与执行

Galbot 机器人的统一运动规划和控制接口。

该接口提供全面的机器人运动控制 API，包括： - 正向和逆向运动学计算 - 单链和多链轨迹规划 - 碰撞检测（自碰撞和环境） - 工具和障碍物管理 - 全身协调运动规划使用

GalbotMotion::get_instance(MachineType) 获取特定平台 (G1/S1) 的实例。所有角度单位为弧度，线性单位为米（SI 标准）。四元数必须归一化： $\sqrt{x^2 + y^2 + z^2 + w^2} = 1$ 。

析构函数

```
virtual galbot::sdk::GalbotMotion::~~GalbotMotion()=default
```

初始化

```
virtual bool galbot::sdk::GalbotMotion::init()=0
```

初始化运动规划系统和通信接口。

必须在任何其他 API 函数之前调用。初始化内部通信中间件，加载机器人运动学模型，并建立与控制服务的连接。

返回值

Type	Description
bool	初始化成功返回 true，否则返回 false。

注意

可安全多次调用；成功后的后续调用返回第一次初始化的结果。

警告

如果 init() 返回 false，则所有其他 API 调用将失败。

检查是否有效

```
virtual bool galbot::sdk::GalbotMotion::is_valid()=0
```

检查运动接口是否正确初始化和运行。

返回值

Type	Description
bool	若对象有效且可用则返回 true，否则返回 false。

注意

如果初始化状态不确定，应在关键操作前检查。

正向运动学

```
virtual std::tuple< MotionStatus, std::vector< double > >
galbot::sdk::GalbotMotion::forward_kinematics(
    const std::string &target_frame,
    const std::string &reference_frame="base_link",
    const std::unordered_map< std::string, std::vector< double > > &joint_state={},
    std::shared_ptr< Parameter > params=default_param
) =0
```

计算目标连杆的正向运动学。

计算给定关节配置的指定连杆的笛卡尔姿态。对于确定末端执行器位置、验证配置或计算中间连杆姿态很有用。

参数

Name	Type	Description
target_frame	const std::string &	目标坐标系
reference_frame	const std::string &	参考坐标系
joint_state	const std::unordered_map< std::string, std::vector< double > > &	关节名到关节位置的映射；默认空字典表示使用当前机器人关节状态。
params	std::shared_ptr< Parameter >	额外参数

返回值

Type	Description
std::tuple< MotionStatus, std::vector< double > >	元组（状态，位姿向量）：- 状态：成功时为 MotionStatus::SUCCESS，否则为错误码- 位姿向量：[x, y, z, qx, qy, qz, qw]（米，四元数），失败时为空

注意

关节角度以弧度为单位，输出位姿以米为单位的单位四元数表示。

警告

target_frame 必须是 URDF 模型中的有效连杆。

基于状态的正向运动学

```
virtual std::tuple< MotionStatus, std::vector< double > >
galbot::sdk::GalbotMotion::forward_kinematics_by_state(
    const std::string &target_frame,
    const std::shared_ptr< RobotStates > &reference_robot_states=nullptr,
    const std::string &reference_frame="base_link",
    std::shared_ptr< Parameter > params=default_param
) =0
```

使用完整的机器人状态计算正向运动学。

与 forward_kinematics() 类似，但接受 RobotStates 对象来指定完整的机器人配置（全身关节+底座姿态）。

参数

Name	Type	Description
target_frame	const std::string &	目标坐标系
reference_robot_states	const std::shared_ptr< RobotStates > &	参考机器人起始状态
reference_frame	const std::string &	参考坐标系
params	std::shared_ptr< Parameter >	额外参数

返回值

Type	Description
std::tuple< MotionStatus, std::vector< double > >	元组（状态，位姿向量）：- 状态：成功时为 MotionStatus::SUCCESS，否则为错误码- 位姿向量：[x, y, z, qx, qy, qz, qw]（米，四元数），失败时为空

注意

在不修改实际状态的情况下为假设状态计算正向运动学时非常有用。

逆向运动学

```
virtual std::tuple< MotionStatus, std::unordered_map< std::string, std::vector<
double > > > galbot::sdk::GalbotMotion::inverse_kinematics(
    const std::vector< double > &target_pose,
    const std::vector< std::string > &chain_names,
    const std::string &target_frame="EndEffector",
    const std::string &reference_frame="base_link",
    const std::unordered_map< std::string, std::vector< double > >
&initial_joint_positions={},
    const bool &enable_collision_check=true,
    std::shared_ptr< Parameter > params=default_param
) =0
```

计算目标笛卡尔姿态的逆运动学。

求解实现指定末端执行器姿态的关节配置。支持单链 IK（仅手臂）或协调多链 IK（手臂 + 躯干/腿）。

参数

Name	Type	Description
target_pose	const std::vector< double > &	目标笛卡尔位姿： [x, y, z, qx, qy, qz, qw]（米，四元数）。
chain_names	const std::vector< std::string > &	要解决逆运动学的运动链名称列表
target_frame	const std::string &	目标坐标系
reference_frame	const std::string &	参考坐标系
initial_joint_positions	const std::unordered_map< std::string, std::vector< double > > &	初始关节位置猜测
enable_collision_check	const bool &	是否启用碰撞检查
params	std::shared_ptr< Parameter >	额外参数

返回值

Type	Description
std::tuple< MotionStatus, std::unordered_map< std::string, std::vector< double > > >	元组（状态，解映射）： - 状态： 若可求解则为 MotionStatus::SUCCESS， 否则为错误码- 解映射： {chain_name -> joint_angles}（弧度）， 失败时为空

注意

IK 可能有多个解；返回第一个有效的解。

注意

种子配置影响收敛速度和哪个解被选中。

警告

如果目标在工作空间之外或处于奇异配置，则无法保证有解。

基于状态的逆向运动学

```
virtual std::tuple< MotionStatus, std::unordered_map< std::string, std::vector<
double > > > galbot::sdk::GalbotMotion::inverse_kinematics_by_state(
    const std::vector< double > &target_pose,
    const std::vector< std::string > &chain_names,
    const std::string &target_frame="EndEffector",
    const std::string &reference_frame="base_link",
    const std::shared_ptr< RobotStates > &reference_robot_states=nullptr,
    const bool &enable_collision_check=true,
    std::shared_ptr< Parameter > params=default_param
) =0
```

使用完整的机器人状态作为种子计算逆运动学。

与 inverse_kinematics() 类似，但接受 RobotStates 来指定种子配置，从而允许精确控制整个机器人状态。

参数

Name	Type	Description
target_pose	const std::vector< double > &	目标笛卡尔位姿： [x, y, z, qx, qy, qz, qw]（米，四元数）。
chain_names	const std::vector< std::string > &	要解决逆运动学的运动链名称列表
target_frame	const std::string &	目标坐标系
reference_frame	const std::string &	参考坐标系
reference_robot_states	const std::shared_ptr< RobotStates > &	参考机器人起始状态
enable_collision_check	const bool &	是否启用碰撞检查
params	std::shared_ptr< Parameter >	额外参数

返回值

Type	Description
std::tuple< MotionStatus, std::unordered_map< std::string, std::vector< double > > >	元组（状态，解映射）： - 状态： 若可求解则为 MotionStatus::SUCCESS， 否则为错误码- 解映射： {chain_name -> joint_angles}（弧度）， 失败时为空

注意

在不修改实际状态的情况下为假设状态离线规划时非常有用。

获取雅可比矩阵

```
virtual std::tuple< MotionStatus, std::vector< std::vector< double > > >
galbot::sdk::GalbotMotion::get_jacobian(
    const std::string &chain_name,
    const std::string &target_frame="EndEffector",
    const std::string &reference_frame="base_link",
    const std::unordered_map< std::string, std::vector< double > > &joint_state={},
    std::shared_ptr< Parameter > params=default_param
) =0
```

计算指定运动链的雅可比矩阵

参数

Name	Type	Description
chain_name	const std::string &	链名称
target_frame	const std::string &	计算雅可比的目标连杆： "EndEffector"（法兰盘）或 "Tool" （工具中心点）。默认： "EndEffector"
reference_frame	const std::string &	参考坐标系："base_link"（机器人基 座坐标系）或 "world"（世界坐标 系）。默认："base_link"
joint_state	const std::unordered_map< std::string, std::vector< double > > &	关节状态对象，包含当前关节位置
params	std::shared_ptr< Parameter >	运动学参数配置

返回值

Type	Description
std::tuple< MotionStatus, std::vector< std::vector< double > > >	返回 numpy 数组，形状为 [6 x DOF]，表示末端执行器 在参考坐标系下的雅可比矩阵

注意

返回雅可比矩阵，形状为 [6 x DOF]，表示关节空间到任务空间的线性映射

注意

用于运动学分析、力控、奇异值分解等场景

通过状态获取雅可比矩阵

```
virtual std::tuple< MotionStatus, std::vector< std::vector< double > > >  
galbot::sdk::GalbotMotion::get_jacobian_by_state(  
    const std::string &chain_name,  
    const std::string &target_frame="EndEffector",  
    const std::string &reference_frame="base_link",  
    const std::shared_ptr< RobotStates > &reference_robot_states=nullptr,  
    std::shared_ptr< Parameter > params=default_param  
) =0
```

通过给定的机器人状态计算指定运动链的雅可比矩阵

参数

Name	Type	Description
chain_name	const std::string &	链名称
target_frame	const std::string &	计算雅可比的目标连杆。默认： "EndEffector"

reference_frame	const std::string &	参考坐标系。默认: "base_link"
reference_robot_states	const std::shared_ptr< RobotStates > &	机器人状态, 包含关节位置信息
params	std::shared_ptr< Parameter >	运动学参数配置

返回值

Type	Description
std::tuple< MotionStatus, std::vector< std::vector< double > > >	元组 (状态, 雅可比矩阵): - 状态: 成功时为 MotionStatus::SUCCESS, 否则为错误码- 雅可比矩阵: 6xN 矩阵 (N 为链自由度), 失败时为空

注意

返回雅可比矩阵, 形状为 [6 x DOF]

获取末端执行器姿态

```
virtual std::tuple< MotionStatus, std::vector< double > >
galbot::sdk::GalbotMotion::get_end_effector_pose(
    const std::string &end_effector_frame,
    const std::string &reference_frame="base_link"
) =0
```

从机器人状态获取当前末端执行器姿态。

查询 TF (变换) 树以检索指定末端执行器连杆的当前笛卡尔姿态。需要在机器人的 URDF 模型中定义该连杆。

参数

Name	Type	Description
end_effector_frame	const std::string &	末端执行器坐标系
reference_frame	const std::string &	参考坐标系

返回值

Type	Description
std::tuple< MotionStatus, std::vector< double > >	元组 (状态, 位姿向量): - 状态: 成功时为 MotionStatus::SUCCESS, 错误码说明: - DATA_FETCH_FAILED: TF 查找失败 - INVALID_INPUT: 无效的帧名称- 位姿向量: [x, y, z, qx, qy, qz, qw] (单位: 米, 四元数), 失败时为空

注意

反映当前实际的机器人状态 (非计划状态)。

警告

需要 TF 树正确发布且为最新状态。

获取指定链末端执行器姿态

```
virtual std::tuple< MotionStatus, std::vector< double > >
galbot::sdk::GalbotMotion::get_end_effector_pose_on_chain(
    const std::string &chain_name,
    const std::string frame_id="EndEffector",
    const std::string &reference_frame="base_link"
) =0
```

获取特定运动链的当前末端执行器姿态。

通过链名称和坐标系类型检索末端执行器姿态的便捷方法，无需知道 URDF 中的确切连杆名称。

参数

Name	Type	Description
chain_name	const std::string &	链名称
frame_id	const std::string	坐标系 ID
reference_frame	const std::string &	参考坐标系

返回值

Type	Description
std::tuple< MotionStatus, std::vector< double > >	元组 (状态, 位姿向量)：状态（成功时为 MotionStatus::SUCCESS，否则为错误码），位姿向量 [x, y, z, qx, qy, qz, qw]（单位：米，四元数），失败时空

注意

内部将 chain_name + frame_id 映射到实际的 URDF 连杆名称。

设置末端执行器姿态

```
virtual MotionStatus galbot::sdk::GalbotMotion::set_end_effector_pose(
    const std::vector< double > &target_pose,
    const std::string &end_effector_frame,
    const std::string &reference_frame="base_link",
    std::shared_ptr< RobotStates > reference_robot_states=nullptr,
    const bool &enable_collision_check=true,
    const bool &is_blocking=true,
    const double &timeout=-1.0,
    std::shared_ptr< Parameter > params=default_param
) =0
```

命令末端执行器移动到目标笛卡尔姿态。

笛卡尔运动命令的高级接口。在内部执行 IK、规划轨迹并可选择执行运动。支持阻塞（等待完成）和非阻塞（立即返回）模式。

参数

Name	Type	Description
target_pose	const std::vector< double > &	目标笛卡尔位姿：[x, y, z, qx, qy, qz, qw]（米，四元数）。
end_effector_frame	const std::string &	末端执行器坐标系
reference_frame	const std::string &	参考坐标系
reference_robot_states	std::shared_ptr< RobotStates >	参考机器人起始状态
enable_collision_check	const bool &	是否启用碰撞检查
is_blocking	const bool &	是否阻塞等待完成
timeout	const double &	阻塞模式超时（秒）。当 timeout < 0 且 is_blocking=true 时，使用 params->timeout_second。
params	std::shared_ptr< Parameter >	额外参数

返回值

Type	Description
MotionStatus	MotionStatus：- SUCCESS：阻塞模式下表示运动完成；非阻塞模式下表示命令发送成功。- TIMEOUT：运动超过超时时间。- INVALID_INPUT：输入参数无效。- FAULT：规划或执行失败。

注意

运动类型（直线/关节空间）由 params->move_line 控制。

注意

对于直接执行（params->is_direct_execute=true），请避免传入 reference_robot_states。

警告

阻塞调用将暂停执行直到运动完成；建议在非阻塞模式下使用进度监控。

轨迹规划

```
virtual std::tuple< MotionStatus, std::unordered_map< std::string, std::vector<
std::vector< double > > > > galbot::sdk::GalbotMotion::motion_plan(
    std::shared_ptr< RobotStates > target,
    std::shared_ptr< RobotStates > start=nullptr,
    std::shared_ptr< RobotStates > reference_robot_states=nullptr,
    bool enable_collision_check=true,
    std::shared_ptr< Parameter > params=default_param
) =0
```

规划单个运动链的轨迹。

参数

Name	Type	Description
------	------	-------------

target	std::shared_ptr< RobotStates >	目标位姿
start	std::shared_ptr< RobotStates >	起始机器人状态
reference_robot_states	std::shared_ptr< RobotStates >	参考机器人起始状态
enable_collision_check	bool	是否启用碰撞检查
params	std::shared_ptr< Parameter >	额外参数

返回值

Type	Description
std::tuple< MotionStatus, std::unordered_map< std::string, std::vector< std::vector< double > > > >	元组（状态，轨迹映射）：- 状态：规划成功时为 MotionStatus::SUCCESS，否则为错误码- 轨迹映射：{chain_name -> waypoint_list}，其中 waypoint_list 为关节配置序列（弧度）

注意

碰撞语义：GalbotMotion 不提供实时障碍物订阅；规划期间仅考虑通过 API 添加的障碍物。

注意

轨迹时间参数化，具有速度/加速度约束。

注意

对于直接执行（params->is_direct_execute=true），轨迹会自动下发到机器人执行。

警告

target 必须是 PoseState 或 JointStates；传递 base RobotStates 将导致 INVALID_INPUT 错误。

单链多路点轨迹规划

```
virtual std::tuple< MotionStatus, std::unordered_map< std::string, std::vector<
std::vector< double > > > > galbot::sdk::GalbotMotion::motion_plan_multi_waypoints(
    std::shared_ptr< RobotStates > target,
    std::vector< std::vector< double > > targets,
    std::shared_ptr< RobotStates > start=nullptr,
    std::shared_ptr< RobotStates > reference_robot_states=nullptr,
    bool enable_collision_check=true,
    std::shared_ptr< Parameter > params=default_param
) =0
```

为单条运动链规划多路点轨迹。

参数

Name	Type	Description
target	std::shared_ptr< RobotStates >	定义路点类型的模板状态（PoseState 或 JointStates）并指定运动链名称，状态类型决定了路点数据的格式
targets	std::vector< std::vector< double > >	多路点目标位姿字典（每个运动链对

		应一个路点序列)
start	std::shared_ptr< RobotStates >	可选起始状态 (JointStates) 。为 nullptr 时使用当前机器人状态。警告：直接执行时请保持为 nullptr。
reference_robot_states	std::shared_ptr< RobotStates >	参考机器人起始状态
enable_collision_check	bool	是否启用碰撞检查
params	std::shared_ptr< Parameter >	额外参数

返回值

Type	Description
std::tuple< MotionStatus, std::unordered_map< std::string, std::vector< std::vector< double > > >	元组 (状态, 轨迹映射)：状态 (规划成功时为 MotionStatus::SUCCESS, 否则为错误码), 轨迹映射 (运动链名 -> 路点列表), 平滑轨迹通过所有路点

注意

碰撞语义：与 motion_plan() 相同。碰撞工作区间在路点处用 get_support_links() 定义。

注意

规划器确保路点处的 C1 连续性 (连续速度) 。

注意

中间路点可能无法精确到达 (插值导致) 。

多路点轨迹规划

```
virtual std::tuple< MotionStatus, std::unordered_map< std::string, std::vector<
std::vector< double > > > galbot::sdk::GalbotMotion::motion_plan_multi_waypoints(
    std::unordered_map< std::shared_ptr< RobotStates >, std::vector< std::vector<
double > > > targets,
    std::vector< std::shared_ptr< RobotStates > > start={},
    std::shared_ptr< RobotStates > reference_robot_states=nullptr,
    bool enable_collision_check=true,
    std::shared_ptr< Parameter > params=default_param
) =0
```

通过多个链的路径点规划协调轨迹。

通过路径点序列实现协调的多臂或全身运动。每个链都可以有自己的路点序列，以同步方式执行。

参数

Name	Type	Description
targets	std::unordered_map< std::shared_ptr< RobotStates >, std::vector< std::vector< double > > >	多路点目标位姿字典 (每个运动链对应一个路点序列)
start	std::vector< std::shared_ptr< RobotStates > >	起始机器人状态

reference_robot_states	std::shared_ptr< RobotStates >	参考机器人起始状态
enable_collision_check	bool	是否启用碰撞检查
params	std::shared_ptr< Parameter >	额外参数

返回值

Type	Description
std::tuple< MotionStatus, std::unordered_map< std::string, std::vector< std::vector< double > > >	元组（状态，轨迹映射）：- 状态：规划成功时为 MotionStatus::SUCCESS，否则为错误码- 轨迹映射：{chain_name -> waypoint_list}（所有运动链）

注意

所有链的轨迹在时间上同步以实现协调运动。

注意

用于双臂操作或移动操作任务。

移动全身关节到零位

```
virtual MotionStatus galbot::sdk::GalbotMotion::move_whole_body_joint_zero(
    bool is_blocking=true,
    double leg_head_speed_rad_s=0.2,
    double leg_head_timeout_s=15.0,
    std::shared_ptr< Parameter > params=default_param
) =0
```

将全身关节移动到预定义的零（家庭）配置。

腿部和头部关节通过 GalbotRobot（直接关节控制）进行控制，而左/右臂则通过启用碰撞检查的运动规划器进行规划。

零配置的联合顺序遵循 SDK 约定：leg(5) + head(2) + left_arm(7) + right_arm(7)。

参数

Name	Type	Description
is_blocking	bool	是否阻塞等待完成
leg_head_speed_rad_s	double	腿部和头部关节速度（弧度/秒）
leg_head_timeout_s	double	腿部和头部运动超时（秒）
params	std::shared_ptr< Parameter >	额外参数

返回值

Type	Description
MotionStatus	运动状态 总体执行状态

检查碰撞

```
virtual std::tuple< MotionStatus, std::vector< bool > >
galbot::sdk::GalbotMotion::check_collision(
    const std::vector< std::shared_ptr< RobotStates > > &start,
    bool enable_collision_check=true,
    std::shared_ptr< Parameter > params=default_param
) =0
```

检查机器人状态是否存在碰撞。

检查自身碰撞和与环境障碍物的碰撞，支持批量处理。因此，如果需要 Motion 考虑环境障碍物（包括点云），必须显式加载障碍物地图/对象（例如，obstacle_type = point_cloud 且在 key 中提供文件路径）。

注意：将实时感知集成到 galbotMotion 是未来计划，目前内部验证有限。

验证给定机器人配置是否无碰撞。

参数

Name	Type	Description
start	const std::vector< std::shared_ptr< RobotStates > > &	起始机器人状态
enable_collision_check	bool	是否启用碰撞检查
params	std::shared_ptr< Parameter >	额外参数

返回值

Type	Description
std::tuple< MotionStatus, std::vector< bool > >	元组（状态，碰撞结果）：- 状态：检测完成时为 MotionStatus::SUCCESS，否则为错误码- 碰撞结果：布尔向量（与 start 等长），true 表示检测到碰撞，false 表示无碰撞

注意

用于验证计划轨迹或基于采样的规划器。

注意

尊重先前添加的障碍物中的 safe_margin（安全边距）设置。

注意

尊重先前添加的障碍物中的 safe_margin（安全边距）设置。

附加工具

```
virtual MotionStatus galbot::sdk::GalbotMotion::attach_tool(
    const std::string &chain,
    const std::string &tool
) =0
```

将工具连接到末端执行器。

将工具（夹具、相机、定制末端执行器）加载到运动链上。更新运动学模型和碰撞几何体以包含该工具。

参数

Name	Type	Description
chain	const std::string &	用于挂载工具的运动链（例如 left_arm、right_arm）。
tool	const std::string &	工具信息

返回值

Type	Description
MotionStatus	运动状态：- 成功：附加工具成功 - 无效输入：无效的运动链或工具名称 - 故障：附加工具失败

注意

工具变换和碰撞几何体必须预先在机器人描述中配置。

注意

附加新工具会自动分离该运动链上先前附加的任何工具。

警告

运动学和碰撞检查将反映附加的工具；请相应更新计划。

分离工具

```
virtual MotionStatus galbot::sdk::GalbotMotion::detach_tool(const std::string &chain)=0
```

将当前工具与末端执行器分离。

相应地更新运动学模型和碰撞几何体。

参数

Name	Type	Description
chain	const std::string &	要分离工具的运动链

返回值

Type	Description
MotionStatus	运动状态：- 成功：分离工具成功 - 无效输入：无效的运动链名称或未附加工具 - 故障：分离工具失败

注意

如果没有工具附加，则操作成功但无效果。

添加障碍物

```
virtual MotionStatus galbot::sdk::GalbotMotion::add_obstacle(  
    const std::string &obstacle_id,  
    const std::string &obstacle_type,  
    const std::vector< double > &pose,  
    const std::array< double, 3 > &scale={},  
    const std::string &key="",  
    const std::string &target_frame="world",  
    const std::string &ee_frame="ee_base",  
    const std::vector< double > &reference_joint_positions={},  
    const std::vector< double > &reference_base_pose={},  
    const std::vector< std::string > &ignore_collision_link_names={},  
    const double &safe_margin=0,  
    const double &resolution=0.01  
) =0
```

将碰撞对象加载到环境中。

将几何或基于网格的障碍物插入环境中以避免碰撞。障碍可以是静态的（世界坐标系下固定的）或与机器人相关的。支持原始形状、网格、点云和深度图像。

参数

Name	Type	Description
obstacle_id	const std::string &	障碍物唯一标识符（场景中不得重复），后续可用于删除或更新。
obstacle_type	const std::string &	障碍物几何类型
pose	const std::vector< double > &	障碍物位姿：[x, y, z, qx, qy, qz, qw]（米，四元数），相对于target_frame。
scale	const std::array< double, 3 > &	几何尺寸（米）：- box: [长度, 宽度, 高度]- sphere: [半径, -, -]- cylinder: [半径, 高度, -]- mesh/point_cloud: 缩放因子
key	const std::string &	类型特定数据：- mesh/point_cloud: 文件路径（例如"/path/to/model.stl"）- depth_image: 相机源数据- robot_state: 机器人状态数据
target_frame	const std::string &	位姿参考坐标系（默认 "world"），可为 "world"、"base_link" 或运动链名称（如 "left_arm"）。
ee_frame	const std::string &	当 target_frame 为运动链名称时，指定链上参考帧（如 "ee_base"、"camera_base"、"camera_object"）。
reference_joint_positions	const std::vector< double > &	用于计算坐标变换的机器人关节状态

		(弧度)。为空时使用当前机器人状态。
reference_base_pose	const std::vector< double > &	地图坐标系下的机器人基座位姿: [x, y, z, qx, qy, qz, qw]。为空时使用当前定位。
ignore_collision_link_names	const std::vector< std::string > &	忽略碰撞的连杆名称列表
safe_margin	const double &	安全距离缓冲 (米)。当距离 < safe_margin 时判定碰撞, 默认 0。
resolution	const double &	复杂几何体 (mesh/point cloud/depth image) 的离散化分辨率 (米), 默认 0.01。

返回值

Type	Description
MotionStatus	运动状态: - 成功: 障碍物添加成功 - 无效输入: 无效的障碍物 ID (重复)、类型或参数 - 故障: 处理几何形状或添加到场景失败

注意

点云说明: point_cloud 指通过此 API 显式加载的点云障碍物 (通常来自文件/离线数据)。它与导航系统维护的点云地图不同, GalbotMotion 不会自动订阅或与 galbotNav 的点云地图同步碰撞检测。

注意

障碍物会持续存在, 直到显式删除或清空。

注意

对于移动障碍物, 请在新位姿处删除并重新添加 (当前无更新方法)。

警告

较大的 safe_margin (安全边距) 值可能会过度约束规划空间, 导致无法找到可行解, 请谨慎使用。

移除障碍物

```
virtual MotionStatus galbot::sdk::GalbotMotion::remove_obstacle(const std::string
&obstacle_id)=0
```

从规划场景中移除碰撞障碍物。

参数

Name	Type	Description
obstacle_id	const std::string &	要移除的障碍物 ID

返回值

Type	Description
MotionStatus	运动状态：SUCCESS（成功）、INVALID_INPUT（障碍物ID未找到）或FAULT（故障）

注意

移除不存在的障碍物返回 INVALID_INPUT（非 NO_ERROR）。

清空障碍物

```
virtual MotionStatus galbot::sdk::GalbotMotion::clear_obstacle()=0
```

从规划场景中移除所有碰撞障碍物。

清除整个障碍物集，将规划场景重置为空（机器人几何体除外）。

返回值

Type	Description
MotionStatus	运动状态：SUCCESS（成功）或FAULT（故障）

注意

附加的对象（参见 attach_target_object）不会被清除。

注意

即使场景已为空，也可以安全调用。

附加目标对象

```
virtual MotionStatus galbot::sdk::GalbotMotion::attach_target_object(
    const std::string &obstacle_id,
    const std::string &obstacle_type,
    const std::vector< double > &pose,
    const std::array< double, 3 > &scale={},
    const std::string &key="",
    const std::string &target_frame="world",
    const std::string &ee_frame="ee_base",
    const std::vector< double > &reference_joint_positions={},
    const std::vector< double > &reference_base_pose={},
    const std::vector< std::string > &ignore_collision_link_names={},
    const double &safe_margin=0,
    const double &resolution=0.01
) =0
```

将碰撞物体附加到机器人上（例如，抓取的物体）。与 add_obstacle() 类似，但对象随机器人移动（连接到连杆/运动链），用于表示抓取的物体、传感器或有效载荷。

在运动过程中，对象相对于连接坐标系的姿态保持不变。

参数

Name	Type	Description
obstacle_id	const std::string &	要移除的障碍物 ID
obstacle_type	const std::string &	障碍物几何类型
pose	const std::vector< double > &	物体位姿：[x, y, z, qx, qy, qz, qw]（米，四元数），在附着时相对于 target_frame。
scale	const std::array< double, 3 > &	几何尺寸（米）：box 为 [length, width, height]；sphere 为 [radius, -, -]；cylinder 为 [radius, height, -]。
key	const std::string &	障碍物唯一标识符
target_frame	const std::string &	附着参考坐标系（默认 "world"）；抓取物体场景通常填写运动链名称（如 "left_arm"）。
ee_frame	const std::string &	当 target_frame 为运动链名称时，指定链上参考帧（如 "ee_base"、"camera_base" 等）。
reference_joint_positions	const std::vector< double > &	用于计算附着变换的机器人关节状态（弧度）。为空时使用当前机器人状态。
reference_base_pose	const std::vector< double > &	地图坐标系下的机器人基座位姿：[x, y, z, qx, qy, qz, qw]。为空时使用当前定位。
ignore_collision_link_names	const std::vector< std::string > &	忽略碰撞的连杆名称列表
safe_margin	const double &	安全距离缓冲（米），默认 0。
resolution	const double &	复杂几何体的离散化分辨率（米），默认 0.01。

返回值

Type	Description
MotionStatus	运动状态：SUCCESS（成功）、INVALID_INPUT（无效输入）或 FAULT（故障）

注意

点云说明：与 add_obstacle() 相同。此处的 point_cloud 是显式加载的点云对象，不会自动与任何导航端点云地图同步。

注意

附加的对象随机器人移动；其碰撞几何体自动更新。

注意

通常用于抓取和放置：抓取后附加目标对象，释放后分离。

警告

请确保 `ignore_collision_link_names` 包含抓取连杆，以避免误判为碰撞。

分离目标对象

```
virtual MotionStatus galbot::sdk::GalbotMotion::detach_target_object(const std::string
&obstacle_id)=0
```

从机器人上分离物体（例如，释放后）。从机器人上移除附着的物体。通常在释放抓取的对象后调用。

该对象被完全从规划场景中移除（不转换为静态障碍物）。

参数

Name	Type	Description
obstacle_id	const std::string &	要移除的障碍物 ID

返回值

Type	Description
MotionStatus	运动状态：SUCCESS（成功）、INVALID_INPUT（障碍物 ID 未找到）或 FAULT（故障）

注意

如需在移除后将对象作为静态障碍物保留在场景中，请使用 `clear_obstacle()`。

设置运动规划配置

```
virtual MotionStatus galbot::sdk::GalbotMotion::set_motion_plan_config(
    std::shared_ptr< MotionPlanConfig > config
) =0
```

设置全局运动规划配置。

更新规划器设置，例如速度/加速度限制、规划算法参数和优化目标。影响后续所有的计划操作。

参数

Name	Type	Description
config	std::shared_ptr< MotionPlanConfig >	新的运动规划配置

返回值

Type	Description
MotionStatus	运动状态：- 成功：配置应用成功 - 无效输入：无效的配置参数 - 故障：配置更新失败

注意

更改会持续存在，直到显式重置或进程重启。

注意

有关可用参数，请参阅 MotionPlanConfig 文档。

获取运动规划配置

```
virtual std::tuple< MotionStatus, MotionPlanConfig >
galbot::sdk::GalbotMotion::get_motion_plan_config()=0
```

获取当前运动规划配置。

检索活动规划器配置，包括速度/加速度限制和规划算法参数。

返回值

Type	Description
std::tuple< MotionStatus, MotionPlanConfig >	元组 (状态, 配置): 状态 (成功时为 MotionStatus::SUCCESS, 否则为错误码), 配置 (当前的 MotionPlanConfig 对象, 失败时为默认构造)

注意

用于检查当前限制或保存/恢复配置。

获取连杆名称

```
virtual std::vector< std::string > galbot::sdk::GalbotMotion::get_link_names(bool
only_end_effector=false)=0
```

从运动学模型中获取机器人连杆名称。

检索机器人 URDF 模型中定义的连杆名称列表，可用于过滤末端执行器连杆或返回所有连杆。

参数

Name	Type	Description
only_end_effector	bool	是否仅返回末端执行器连杆，true 仅返回末端执行器/工具连杆，false 返回所有连杆 (包括底座、中间连杆等)，默认 false (所有连杆)

返回值

Type	Description
std::vector< std::string >	连杆名称字符串向量 (如果检索失败则为空)

注意

基于连杆没有子连杆来检测末端执行器。

注意

用于正向运动学查询或 TF 帧验证。

获取支持的连杆

```
virtual const std::set< std::string > & galbot::sdk::GalbotMotion::get_support_links()=0
```

获取机器人模型中有效的连杆名称列表。

返回加载的 URDF 运动学模型中定义的所有连杆名称，可用于验证正向运动学查询或 TF 查找。

返回值

Type	Description
const std::set< std::string > &	支持的连杆名称字符串集合的常量引用

注意

此集合在初始化期间从机器人 URDF 文件填充。

获取支持的链

```
virtual const std::set< std::string > & galbot::sdk::GalbotMotion::get_support_chains()=0
```

获取有效的运动链名称列表。

返回所有预定义运动链的名称（例如 "left_arm"、"right_arm"、"torso"、"left_leg"、"right_leg"）。链是被视为一个单元进行规划和控制的关节组。

返回值

Type	Description
const std::set< std::string > &	支持的运动链名称列表，例如：- "left_arm" - 左臂- "right_arm" - 右臂- "torso" - 躯干- "left_leg" - 左腿- "right_leg" - 右腿

注意

链定义在机器人配置文件中指定。

获取支持的坐标系

```
virtual const std::set< std::string > & galbot::sdk::GalbotMotion::get_support_frame()=0
```

获取有效的参考坐标系名称列表。

返回姿态规范支持的标准参考坐标系（例如 "base_link"、"world"、"odom"）。

返回值

Type	Description
const std::set< std::string > &	支持的参考坐标系，例如： - "base_link" - 机器人底座- "world" - 世界坐标系- "odom" - 里程计坐标系

注意

这些是全局/基座帧；各个连杆帧通过它们访问。

获取支持的末端执行器坐标系

```
virtual const std::set< std::string > &  
galbot::sdk::GalbotMotion::get_support_ee_frame()=0
```

获取有效的末端执行器坐标系类型列表。

返回可为末端执行器姿态指定的帧类型（例如 "EndEffector"、"Camera"、"TCP"）。

返回值

Type	Description
const std::set< std::string > &	支持的末端执行器帧类型，例如： - "EndEffector" - 默认末端执行器- "Camera" - 相机坐标系- "TCP" - 工具中心点

注意

实际连杆名称由链名称 + 帧类型组合派生。

获取支持的工具列表

```
virtual const std::set< std::string > &  
galbot::sdk::GalbotMotion::get_support_tool_list()=0
```

获取可附加的可用工具列表。

返回可通过 attach_tool() 连接的预定义工具（夹持器、传感器、自定义末端执行器）的名称。

返回值

Type	Description
const std::set< std::string > &	支持的工具名称列表，例如夹持器、传感器、自定义末端执行器。

注意

工具必须预先配置几何体和运动学偏移。

获取支持的障碍物类型

```
virtual std::set< std::string > galbot::sdk::GalbotMotion::get_support_obstacle_type()=0
```

获取支持的碰撞障碍物几何类型列表。

返回 add_obstacle() 和 attach_target_object() 支持的几何类型（例如 "box"、"sphere"、"cylinder"、"mesh"、"point_cloud"、"depth_image"）。

返回值

Type	Description
std::set< std::string >	支持的障碍物几何类型，例如： - "box" - 立方体 - "sphere" - 球体 - "cylinder" - 圆柱体 - "mesh" - 网格模型 - "point_cloud" - 点云 - "depth_image" - 深度图

注意

不同类型需要不同的缩放/键参数格式。

获取已构建对象列表

```
virtual std::vector< std::string > galbot::sdk::GalbotMotion::get_built_object_list()=0
```

获取规划场景中当前添加的障碍物列表。

返回场景中当前存在的所有障碍物（静态障碍物和附加对象）的障碍物 ID。

返回值

Type	Description
std::vector< std::string >	障碍物 ID 字符串向量（如果没有障碍物则为空）

注意

用于调试场景状态或防止重复 ID。

获取附加对象列表

```
virtual std::vector< std::string > galbot::sdk::GalbotMotion::get_attached_object_list()=0
```

获取机器人上当前附加的物体列表。

返回当前附加到机器人的所有障碍物 ID（通过 attach_target_object() 添加）。

返回值

Type	Description
std::vector< std::string >	附加对象 ID 字符串向量（如果没有附加对象则为空）

注意

用于追踪抓取的对象或负载。

检查连杆名称是否有效

```
virtual bool galbot::sdk::GalbotMotion::is_link_name_valid(
    const std::string &value,
```

```
    bool throw_exception=false
) =0
```

根据机器人模型验证连杆名称。

检查所提供的连杆名称是否存在于加载的 URDF 模型中。

参数

Name	Type	Description
value	const std::string &	要验证的全身关节状态
throw_exception	bool	验证失败时是否抛出异常

返回值

Type	Description
bool	若连杆名称有效则返回 true，否则返回 false。

注意

使用 get_support_links() 获取有效连杆名称。

检查链名称是否有效

```
virtual bool galbot::sdk::GalbotMotion::is_chain_name_valid(
    const std::string &value,
    bool throw_exception=false
) =0
```

验证运动链名称。

检查机器人配置中是否定义了提供的链名称。

参数

Name	Type	Description
value	const std::string &	要验证的全身关节状态
throw_exception	bool	验证失败时是否抛出异常

返回值

Type	Description
bool	若运动链名称有效则返回 true，否则返回 false。

注意

使用 get_support_chains() 获取有效链名称。

检查坐标系名称是否有效

```
virtual bool galbot::sdk::GalbotMotion::is_frame_name_valid(
    const std::string &value,
```

```
bool throw_exception=false
) =0
```

验证参考坐标系名称。

检查提供的坐标系名称是否是姿态规范的有效参考坐标系。

参数

Name	Type	Description
value	const std::string &	要验证的参考坐标系名称（如 "base_link"、"world"）。
throw_exception	bool	验证失败时是否抛出异常

返回值

Type	Description
bool	若坐标系名称有效则返回 true，否则返回 false。

注意

使用 get_support_frame() 获取有效帧名称。

检查末端执行器坐标系是否有效

```
virtual bool galbot::sdk::GalbotMotion::is_ee_frame_valid(
    const std::string &value,
    bool throw_exception=false
) =0
```

验证末端执行器坐标系类型。

检查提供的帧类型对于末端执行器坐标系规格是否有效。

参数

Name	Type	Description
value	const std::string &	要验证的末端执行器坐标系类型（如 "EndEffector"、"Camera"、"TCP"）。
throw_exception	bool	验证失败时是否抛出异常

返回值

Type	Description
bool	若末端执行器坐标系类型有效则返回 true，否则返回 false。

注意

使用 get_support_ee_frame() 获取有效帧类型。

检查工具名称是否有效

```
virtual bool galbot::sdk::GalbotMotion::is_tool_name_valid(  
    const std::string &value,  
    bool throw_exception=false  
) =0
```

验证工具名称。

检查提供的工具名称是否可用于附件。

参数

Name	Type	Description
value	const std::string &	要验证的全身关节状态
throw_exception	bool	验证失败时是否抛出异常

返回值

Type	Description
bool	若工具名称有效则返回 true，否则返回 false。

注意

使用 get_support_tool_list() 获取可用工具列表。

检查障碍物类型是否有效

```
virtual bool galbot::sdk::GalbotMotion::is_obstacle_type_valid(  
    const std::string &value,  
    bool throw_exception=false  
) =0
```

验证障碍物几何类型。

检查所提供的类型是否支持障碍物创建。

参数

Name	Type	Description
value	const std::string &	要验证的全身关节状态
throw_exception	bool	验证失败时是否抛出异常

返回值

Type	Description
bool	若障碍物类型有效则返回 true，否则返回 false。

注意

使用 get_support_obstacle_type() 获取支持的类型。

检查全身状态是否有效

```
virtual bool galbot::sdk::GalbotMotion::is_whole_body_state_valid(  
    const std::vector< double > &value,  
    bool throw_exception=false  
) =0
```

验证全身关节配置向量。

检查提供的矢量是否具有适合全身状态的正确尺寸（机器人自由度）。

参数

Name	Type	Description
value	const std::vector< double > &	要验证的全身关节配置向量（弧度）。
throw_exception	bool	验证失败时是否抛出异常

返回值

Type	Description
bool	若向量长度与机器人自由度匹配则返回 true，否则返回 false。

注意

预期大小：get_robot_dof() 个元素。

注意

不验证关节限制合规性，仅验证向量维度。

检查 7 维姿态是否有效

```
virtual bool galbot::sdk::GalbotMotion::is_pose_7d_valid(  
    const std::vector< double > &value,  
    bool throw_exception=false  
) =0
```

验证 7D 姿态向量（位置 + 四元数）。

检查提供的姿态向量是否恰好有 7 个元素：[x, y, z, qx, qy, qz, qw]。

参数

Name	Type	Description
value	const std::vector< double > &	要验证的 7D 位姿向量。
throw_exception	bool	验证失败时是否抛出异常

返回值

Type	Description
------	-------------

bool	若向量长度为 7 ([x,y,z,qx,qy,qz,qw]) 则返回 true, 否则返回 false。
------	--

注意

不强制四元数归一化; 仅检查维度。

警告

使用未归一化的四元数将导致运动学中的不确定行为。

检查链关节状态是否有效

```
virtual bool galbot::sdk::GalbotMotion::is_chain_joint_state_valid(  
    const std::unordered_map< std::string, std::vector< double > > &value,  
    bool throw_exception=false  
) =0
```

验证链索引关节配置图。

检查链名称是否有效以及每个链的关节向量是否具有正确的维度。

参数

Name	Type	Description
value	const std::unordered_map< std::string, std::vector< double > > &	要验证的全身关节状态
throw_exception	bool	验证失败时是否抛出异常

返回值

Type	Description
bool	若所有运动链名称和向量长度均有效则返回 true, 否则返回 false。

注意

验证链名称存在性和关节向量维度。

获取机器人自由度

```
virtual int galbot::sdk::GalbotMotion::get_robot_dof()=0
```

获取机器人总自由度 (DOF)。

返回完整机器人 (全身) 中驱动关节的数量, 用于验证关节状态向量维度。

返回值

Type	Description
int	机器人自由度 (驱动关节) 数量

注意

典型的人形移动机械臂：15-30 自由度。

获取机器人状态

```
virtual RobotStates galbot::sdk::GalbotMotion::get_robot_states()=0
```

获取当前完整的机器人状态。

检索当前全身关节配置和移动底座姿态。代表机器人的完整运动状态。

返回值

Type	Description
RobotStates	机器人状态对象包含：whole_body_joint（完整的关节配置，单位：弧度），base_state（移动底座位姿 [x, y, z, qx, qy, qz, qw]，单位：米，四元数）

注意

反映实际机器人状态（来自传感器反馈/状态估计）。

注意

用作规划操作的种子/参考。

获取全身状态

```
virtual std::vector< double > galbot::sdk::GalbotMotion::get_whole_body_state()=0
```

获取当前全身关节配置。

检索机器人中所有驱动关节的关节角度。

返回值

Type	Description
std::vector< double >	关节角度向量（弧度）

注意

关节排序由机器人 URDF/配置定义。

获取底盘状态

```
virtual std::vector< double > galbot::sdk::GalbotMotion::get_chassis_state()=0
```

获取当前移动基础姿态。

检索机器人移动底座在地图坐标系中的位置和方向。

返回值

Type	Description
std::vector< double >	基座位姿向量： [x, y, z, qx, qy, qz, qw] (米, 单位四元数)

注意

对于非移动机器人，通常返回单位位姿。

注意

位姿来自定位/里程计系统。

获取链关节状态

```
virtual std::unordered_map< std::string, std::vector< double > >
galbot::sdk::GalbotMotion::get_chain_joint_state() =0
```

获取所有运动链的当前关节配置。

检索每条链的关节状态，将整体配置分解为各个链的贡献。

返回值

Type	Description
std::unordered_map< std::string, std::vector< double > >	映射 {运动链名 -> 关节配置} (单位: 弧度) 对应每个运动链, 例如: {"left_arm" -> [7 个关节角度], "right_arm" -> [7 个关节角度]}

注意

关节向量大小因链的自由度而异。

获取链姿态状态

```
virtual std::unordered_map< std::string, std::vector< double > >
galbot::sdk::GalbotMotion::get_chain_pose_state(
    const std::string &frame="base_link"
) =0
```

获取所有运动链的当前笛卡尔姿态。

计算所有链末端执行器的正向运动学，返回它们在指定参考坐标系中的位姿。

参数

Name	Type	Description
frame	const std::string &	参考坐标系

返回值

Type	Description
std::unordered_map< std::string, std::vector< double > >	映射 {运动链名 -> 位姿向量}, 其中位姿向量是每个运动

	链末端执行器的 [x, y, z, qx, qy, qz, qw] (米, 四元数)
--	--

注意

比 `get_chain_joint_state()` 计算量更大, 可用于获取末端执行器相对于指定帧的位姿。

替换关节状态

```
virtual bool galbot::sdk::GalbotMotion::replace_joint_state(  
    const std::string &chain_name,  
    const std::vector< double > &chain_joint,  
    std::vector< double > &whole_body_joint  
) =0
```

更新全身配置中的特定链条关节。

用于修改完整的全身关节向量内的链的关节值, 同时保留其他链的状态的效用函数。

参数

Name	Type	Description
chain_name	const std::string &	链名称
chain_joint	const std::vector< double > &	要更新的链关节
whole_body_joint	std::vector< double > &	原始全身关节向量

返回值

Type	Description
bool	替换成功返回 true; 若运动链名称无效或尺寸不匹配则返回 false。

注意

`whole_body_joint` 就地修改; 请确保维度正确。

注意

用于离线状态操作或自定义规划种子。

状态转字符串

```
virtual std::string galbot::sdk::GalbotMotion::status_to_string(MotionStatus status)=0
```

将 `MotionStatus` 枚举转换为人类可读的字符串。

将状态代码映射到用于日志记录、错误报告或 UI 显示的描述性字符串。

参数

Name	Type	Description
status	MotionStatus	运动状态枚举值

返回值

Type	Description
std::string	描述性状态字符串（例如，"SUCCESS: 执行成功", "TIMEOUT: 执行超时"）

注意

使用 status_string_map_ 进行查找；如果状态未知则返回 UNKNOWN。

获取实例

```
static GalbotMotion & galbot::sdk::GalbotMotion::get_instance(MachineType m)
```

用于选择具体运动规划单例的运行时工厂。

参数

Name	Type	Description
m	MachineType	机器类型

返回值

Type	Description
GalbotMotion &	指定机器类型的单例运动接口引用。

移动导航

移动机器人底盘导航与定位接口。

该类提供线程安全的单例接口，用于控制移动底盘导航系统。支持二维位姿估计、重定位、带动态避障的目标导航与路径规划。导航系统在全局地图坐标系下工作，支持阻塞与非阻塞两种导航模式，并兼容差分驱动和全向底盘。除非另有说明，所有位姿参数均以地图坐标系表示。

析构函数

```
virtual galbot::sdk::GalbotNavigation::~GalbotNavigation()=default
```

初始化

```
virtual bool galbot::sdk::GalbotNavigation::init()=0
```

初始化导航子系统及其依赖项。

在使用任何其他导航功能之前必须调用此方法。它初始化通信通道、加载地图、启动定位模块并准备路径规划器。

返回值

Type	Description
bool	初始化成功返回 true，否则返回 false。

注意

此方法应在获取单例实例后仅调用一次。

注意

后续调用将返回第一次初始化的结果。

警告

在成功初始化前调用导航方法将导致错误。

重定位

```
virtual NavigationStatus galbot::sdk::GalbotNavigation::relocalize(const Pose &init_pose)=0
```

执行重新定位以重新估计机器人在地图坐标系中的姿态。
该方法重置定位滤波器并提供初始姿态估计，以帮助机器人在已知地图中重新建立其位置。当机器人失去定位或手动将机器人放置在已知位置时，这非常有用。

参数

Name	Type	Description
init_pose	const Pose &	初始姿态估计

返回值

Type	Description
NavigationStatus	导航状态，指示重定位请求的结果。详见 NavigationStatus 枚举的取值说明。

注意

重定位时机器人应保持静止以获得最佳效果。

注意

调用此方法后，使用 is_localized() 验证成功。

检查是否已定位

```
virtual bool galbot::sdk::GalbotNavigation::is_localized()=0
```

检查机器人当前是否在地图中定位。

该方法查询定位系统以确定机器人是否具有足够置信度的有效姿态估计。未定位的机器人不应执行导航任务。

返回值

Type	Description
bool	若机器人已可靠定位则返回 true；若定位丢失或不确定则返回 false。

注意

建议在发出导航命令前检查定位状态。

注意

如果返回 false，请考虑调用 `relocalize()` 并提供初始猜测。

获取当前姿态

```
virtual Pose galbot::sdk::GalbotNavigation::get_current_pose()=0
```

获取地图坐标系中机器人底盘的当前估计位姿。

该方法返回来自定位系统的最新姿态估计。位姿表示机器人的 `base_link` 坐标系相对于地图坐标系原点的位置和方向。

返回值

Type	Description
Pose	结构包含以下字段： - x: X 坐标（米，地图坐标系） - y: Y 坐标（米，地图坐标系） - theta: 方向（弧度，地图坐标系，逆时针从 x 轴算起）

注意

仅当 `is_localized()` 返回 true 时返回的位姿才有效。

注意

该位姿表示机器人底盘接地轮廓（base footprint）的中心。

导航到目标

```
virtual NavigationStatus galbot::sdk::GalbotNavigation::navigate_to_goal(
    const Pose &goal_pose,
    bool enable_collision_check=true,
    bool is_blocking=false,
    float timeout=8,
    bool omni_plan=true
) =0
```

将机器人导航至地图坐标系中的目标位姿。

该方法命令移动底座使用全局路径规划器和局部轨迹控制器导航到指定的目标位姿。如果启用了碰撞检查，规划器将计算从当前姿态到目标的无碰撞路径，同时考虑静态地图障碍物和动态障碍物。

参数

Name	Type	Description
goal_pose	const Pose &	目标位姿（地图坐标系）
enable_collision_check	bool	是否启用碰撞检查
is_blocking	bool	执行模式标志。- true（阻塞）：阻塞直到到达目标、失败或超时。- false（非阻塞）：发送导航命令后立即返回。
timeout	float	阻塞模式下的最大等待时间（秒），默认值：8.0 秒，仅在 is_blocking 为 true 时相关，如果在时间内未到达目标，则返回超时状态
omni_plan	bool	运动规划模式标志。- true: 启用全向运动规划（各向同性移动）- false: 使用有限方向的运动规划

返回值

Type	Description
NavigationStatus	导航状态指示结果：- 非阻塞模式：命令接受状态 - 阻塞模式：最终导航结果（成功、失败、超时）

注意

机器人必须先定位 (is_localized() 返回 true) ，才能开始导航。

注意

对于阻塞模式，调用线程将阻塞直到导航完成或超时。

注意

实际导航时间可能超过超时值（阻塞模式下）。

警告

长距离导航可能需要较长时间；建议使用非阻塞模式或监控进度。

直线移动到目标

```
virtual NavigationStatus galbot::sdk::GalbotNavigation::move_straight_to(
    const Pose &goal_pose,
    bool is_blocking=true,
    float timeout=8
) =0
```


将机器人移动到里程计坐标系中的相对目标位姿。

此方法命令机器人移动到相对于其在里程计 (odom) 坐标系中的当前位置指定的姿态。这对于不需要基于地图的规划的短距离、精确的运动非常有用。与 `navigate_to_goal()` 不同，此方法不执行动态障碍物检测或全局路径规划。它使用全向运动规划来直接移动到目标。

参数

Name	Type	Description
goal_pose	const Pose &	相对于当前机器人底座帧的目标位姿。包含：- x: 前后位移 (米) - y: 左右位移 (米) - theta: 转向角度 (弧度)
is_blocking	bool	执行模式标志。- true (阻塞): 阻塞直到运动完成、失败或超时- false (非阻塞): 发送导航命令后立即返回
timeout	float	阻塞模式最大等待时间 (秒)，默认 8.0 秒；仅在 is_blocking 为 true 时生效。

返回值

Type	Description
NavigationStatus	导航状态指示结果：- 非阻塞模式：命令接受状态 - 阻塞模式：最终运动结果 (成功、失败、超时)

注意

此方法不检查障碍物或碰撞。请先检查路径可达性。

注意

此方法使用里程计帧，不需要地图帧。

注意

适用于小的精确调整，如最终接近。

警告

由于禁用了碰撞检查，请确保路径无障。

警告

长距离下里程计漂移可能影响精度。建议缩短单次移动距离或定期重定位。

停止导航

```
virtual NavigationStatus galbot::sdk::GalbotNavigation::stop_navigation()=0
```

停止当前的导航任务并使机器人停止。

此方法立即取消任何正在进行的导航命令（来自 `navigate_to_goal()`或 `move_straight_to()`）并命令机器人停止。机器人将根据其运动学约束减速并安全停止。

返回值

Type	Description
NavigationStatus	导航状态指示是否成功将停止命令发送到导航系统。

注意

可在导航期间随时调用。

注意

停止后，机器人的位置可能与原始位置不同。

注意

机器人将尝试根据其加速度限制平滑停止。

检查路径可达性

```
virtual bool galbot::sdk::GalbotNavigation::check_path_reachability(
    const Pose &goal_pose,
    const Pose &start_pose
) =0
```

检查地图中是否存在从起点到目标的无碰撞路径。

此方法查询全局路径规划器以确定指定的起始姿态和目标姿态之间是否存在有效的无碰撞路径。这对于在尝试导航之前验证目标姿态或多目标路径规划非常有用。

参数

Name	Type	Description
goal_pose	const Pose &	目标位姿（地图坐标系）
start_pose	const Pose &	起始位姿

返回值

Type	Description
bool	若起点到目标存在无碰撞路径则返回 true；若未找到有效路径则返回 false。

注意

此方法仅基于地图检查静态障碍物。

注意

路径计算可能根据距离需要一些时间。

注意

返回 true 并不保证成功导航。

检查目标到达

```
virtual bool galbot::sdk::GalbotNavigation::check_goal_arrival()=0
```

检查机器人是否成功达到当前目标。

该方法查询导航系统以确定机器人是否已在可接受的位置和方向公差内到达目标位姿。当使用非阻塞导航模式轮询完成情况时，这特别有用。

返回值

Type	Description
bool	若机器人在容差阈值内到达目标则返回 true；若仍在导航、无活动目标或尚未到达则返回 false。

注意

在非阻塞导航场景中最有用。

注意

到达的公差阈值由导航模块的内部参数定义（通常在 YAML 配置文件中设置）。

注意

如果没有激活的导航命令，此方法返回 false。

获取导航状态

```
virtual NavigationTaskStatus galbot::sdk::GalbotNavigation::get_navigation_status()=0
```

获取当前导航任务状态。

返回导航系统报告的最新任务状态（未知、正在运行、成功或失败）。在运行非阻塞导航时使用此功能可以轮询状态并在失败或超时及时退出错误逻辑，从而避免死锁或无限期等待。

返回值

Type	Description
NavigationTaskStatus	导航任务状态 当前任务状态。如果尚无状态则为 UNKNOWN；导航时为 RUNNING；任务完成时为 SUCCESS 或 FAILED。

注意

在非阻塞导航中很有用：在 get_navigation_status() 上循环检查进度。

获取实例

```
static GalbotNavigation & galbot::sdk::GalbotNavigation::get_instance(MachineType m)
```

用于选择具体导航单例的运行时工厂。

这种静态工厂方法允许在运行时根据机器人机器类型选择导航实现。方法声明驻留在接口标头中以实现编译时可用性，而实际的实现逻辑（包括特定于平台的 include 和 switch 语句）包含在相应的 .cpp 文件中。这种设计保持接口干净，同时启用特定于平台的实例化，而无需暴露实现细节。

参数

Name	Type	Description
m	MachineType	机器类型

返回值

Type	Description
GalbotNavigation &	指定机器类型的单例导航接口引用。

注意

添加新机器类型支持需要更新此类。

感知模块接口

感知模块接口；通过 `get_instance(MachineType)` 获取单例实例。

G1 平台专用：`get_instance(MachineType::S1)` 会抛出 `std::runtime_error`。

析构函数

```
virtual galbot::sdk::GalbotPerception::~~GalbotPerception()=default
```

初始化感知模块并加载模型

```
virtual bool galbot::sdk::GalbotPerception::init(const std::set< PerceptionModule >
&enabled_modules)=0
```

初始化感知模块并加载指定模块的模型。

参数

Name	Type	Description
enabled_modules	const std::set< PerceptionModule > &	要启用的感知模块集合。

返回值

Type	Description
bool	如果所有请求的模块都成功加载则返回 true。

为指定模块运行一次推理

```
virtual bool galbot::sdk::GalbotPerception::run_once(PerceptionModule module)=0
```

为指定模块运行一次推理。

参数

Name	Type	Description
module	PerceptionModule	要运行的感知模块。

注意

初始化后，等待约 10 秒让模型准备就绪后再调用 run_once。

等待模块产生新结果或超时

```
virtual bool galbot::sdk::GalbotPerception::wait_for_new_result(
    PerceptionModule module,
    double timeout_s=5.0
) =0
```

阻塞直到模块产生新结果或超时。与 run_once 配合使用获取最新输出。

参数

Name	Type	Description
module	PerceptionModule	感知模块。
timeout_s	double	超时时间（秒）。

获取模块的最新缓存结果

```
virtual bool galbot::sdk::GalbotPerception::get_latest_result(
    PerceptionModule module,
    DetectionResult &result
) =0
```

返回模块的最新缓存结果，不阻塞。

参数

Name	Type	Description
module	PerceptionModule	感知模块。
result	DetectionResult &	输出检测结果。

返回值

Type	Description
bool	如果有结果可用返回 true，否则返回 false。

获取感知模块单例实例

```
static GalbotPerception & galbot::sdk::GalbotPerception::get_instance(MachineType m)
```

获取 Perception 模块单例实例。

参数

Name	Type	Description
m	MachineType	机器类型（如 MachineType::G1）。MachineType::S1 不支持，会抛出异常。

返回值

Type	Description
GalbotPerception &	指定机器类型的单例实例引用。

类型与枚举

驱动类型

指定在运动规划和执行期间应启动哪些运动链。这控制机器人是否仅使用目标手臂，或者还涉及躯干或腿部运动。

Enum Value	Description
ACTUATE_WITH_CHAIN_ONLY	仅执行目标运动链
ACTUATE_WITH_TORSO	执行目标运动链和躯干
ACTUATE_WITH_LEG	执行目标运动链和腿部
ACTUATE_TYPE_NUM	驱动类型数量

音频数据

音频数据结构用于封装音频数据。

成员变量

Name	Type	Description
------	------	-------------

header	Header	消息头
type	std::string	音频数据类型标识符。 - "waken_up": 唤醒事件（格式为 JSON） - "speech": 语音数据（格式为 PCM）
format	std::string	音频数据格式说明，例如："pcm": 采样率 16000 Hz，位深度 16 bit，单声道；"json": UTF-8 编码的 JSON 文本
data	std::vector< uint8_t >	二进制数据包，具体格式由 format 字段指定。 - "pcm" 格式：PCM 音频数据（小端序，16 位整数） - "json" 格式：UTF-8 编码的 JSON 文本

轴对齐 2D 边界框

轴对齐 2D 边界框，带有类别 ID 和置信度分数辅助功能。

获取左上角 x 坐标

```
int BBox::x1() const
```

获取左上角 y 坐标

```
int BBox::y1() const
```

获取右下角 x 坐标

```
int BBox::x2() const
```

获取右下角 y 坐标

```
int BBox::y2() const
```

计算边界框面积

```
float BBox::area() const
```

计算边界框中心点

```
cv::Point2f BBox::center() const
```

成员变量

Name	Type	Description
rect	cv::Rect	
cls	std::pair< int, float >	

电池管理信息

计算边界框中心点。

成员变量

Name	Type	Description
voltage	float	电压
current	float	电流 (A)
battery_level	float	电池电量
temperature	float	温度
charging_status	bool	充电状态
health_status	bool	健康状态
capacity	float	容量

相机信息

完整的相机标定数据，包括内参、畸变系数、校正和投影矩阵。与 ROS 2 的 sensor_msgs/CameraInfo 消息类型兼容。

成员变量

Name	Type	Description
header	Header	包含时间戳和相机坐标系 ID（例如 "camera_optical_frame"）
height	uint32_t	校准期间此相机生成的图像垂直分辨率
width	uint32_t	校准期间此相机生成的图像水平分辨率
distortion_model	std::string	指定使用的镜头畸变模型。常见值：- "plumb_bob": Brown-Conrady 模型（径向 k1/k2/k3 和切向 p1/p2 畸变）- "rational_polynomial": 扩展模型（包含额外参数）- "equidistant": 鱼眼镜头模型
d	std::vector< double >	畸变参数向量，大小和解释取决于畸变模型。例如 "plumb_bob" 模型：[k1, k2, p1, p2, k3]- k1, k2, k3: 径向畸变系数- p1, p2: 切向畸变系数
k	std::array< double, 9 >	3×3 矩阵（行优先顺序），将相机坐标系中的 3D 点映射到 2D 像素坐标：fx 0 cx 0 fy cy 0 1fx, fy: 焦距（像素单位）cx, cy: 主点（光学中心）坐标（像素）

r	std::array< double, 9 >	3×3 旋转矩阵（行优先顺序）。对于立体相机：旋转左/右相机图像平面使其共面。
p	std::array< double, 12 >	3×4 投影矩阵（行优先顺序），将 3D 点投影到校正后的图像坐标。fx', fy': 校正后的焦距 cx', cy': 校正后的主点 Tx, Ty: 平移分量（立体相机）
binning_x	uint32_t	水平方向像素合并数。- 0 或 1: 不合并- 2: 每 2 个像素合并为 1 个输出像素- 4: 每 4 个像素合并为 1 个输出像素
binning_y	uint32_t	垂直方向像素合并数。- 0 或 1: 不合并- 2: 每 2 个像素合并为 1 个输出像素- 4: 每 4 个像素合并为 1 个输出像素
roi	RegionOfInterest	感兴趣区域
camera_type	std::string	可选字段，指定相机类型或型号，例如："monocular"、"stereo_left"、"stereo_right"、"depth"
T	std::vector< double >	可选的变换矩阵，用于厂商特定或扩展的标定数据。大小和解释取决于具体实现。

运动链类型

识别机器人结构中的不同运动链，以进行正向/反向运动学计算和运动规划。

Enum Value	Description
HEAD	头部
LEFT_ARM	左臂
RIGHT_ARM	右臂
LEG	腿部
TORSO	躯干
CHAIN_NUM	运动链数量

碰撞检查选项

碰撞检测启用/禁用配置。

该结构在运动规划和执行期间提供对碰撞检查的细粒度控制。它支持独立切换自碰撞检测（机器人连杆相互碰撞）和环境碰撞检测（机器人与障碍物或工作空间边界碰撞）。禁用碰撞检查可以提高计算性能，但可能会导致不安全的轨迹。在受控环境中谨慎使用。

启用或禁用自碰撞检测

```
void galbot::sdk::CollisionCheckOption::set_disable_self_collision_check(bool disable)
```

true 禁用自碰撞检测，false 启用

参数

Name	Type	Description
disable	bool	传入 true 表示禁用自碰撞检测，传入 false 表示启用。

警告

禁用自碰撞检查可能导致物理上不可行的配置。

启用或禁用环境碰撞检测

```
void galbot::sdk::CollisionCheckOption::set_disable_env_collision_check(bool disable)
```

设置是否禁用环境碰撞检测

参数

Name	Type	Description
disable	bool	是否禁用碰撞检测

警告

禁用环境检查可能导致与障碍物发生碰撞。

检查自碰撞检测是否禁用

```
bool galbot::sdk::CollisionCheckOption::get_disable_self_collision_check() const
```

检查自碰撞检测是否禁用

返回值

Type	Description
bool	若当前已禁用自碰撞检测，则返回 true；否则返回 false。

检查环境碰撞检测是否禁用

```
bool galbot::sdk::CollisionCheckOption::get_disable_env_collision_check() const
```

检查环境碰撞检测是否禁用

返回值

Type	Description
bool	若当前已禁用环境碰撞检测，则返回 true；否则返回 false。

打印碰撞检测配置到标准输出

```
void galbot::sdk::CollisionCheckOption::print() const
```

打印碰撞检测配置到标准输出

控制状态

控制命令执行状态枚举。

表示机器人控制命令的执行状态，包括关节控制、末端执行器控制等运动控制操作。

Enum Value	Description
SUCCESS	成功
TIMEOUT	超时
FAULT	故障
INVALID_INPUT	无效输入
INIT_FAILED	初始化失败
IN_PROGRESS	进行中
STOPPED_UNREACHED	已停止但未到达
DATA_FETCH_FAILED	数据获取失败
PUBLISH_FAIL	发布失败
COMM_DISCONNECTED	通信断开
STATUS_NUM	状态数量

深度数据

包含来自深度相机或 RGB-D 传感器的压缩深度图像数据。与 ROS 2 的 sensor_msgs/compressedImage 消息类型兼容（支持深度扩展）。

深度图转 OpenCV Mat

```
std::shared_ptr< cv::Mat > galbot::sdk::DepthData::convert_to_cv2_mat()
```

将深度数据解码并转换为 OpenCV Mat 格式以进行处理。

返回值

Type	Description
std::shared_ptr< cv::Mat >	解码成功时返回 std::shared_ptr<cv::Mat> 智能指针；解码失败时返回空指针。

成员变量

Name	Type	Description
header	Header	包含采集时间戳和相机坐标系
height	uint32_t	深度图像的行数
width	uint32_t	深度图像的列数
format	std::string	指定深度编码和压缩格式，例如： "16UC1; compressedDepth png" (16 位无符号整数，PNG 压缩深度)
data	std::vector< uint8_t >	包含原始或压缩深度图像数据的二进制数据块
depth_scale	uint32_t	深度缩放因子，用于将像素值转换为实际深度（米），真实深度 = 像素值 / depth_scale，例如： depth_scale = 1000 表示像素值单位为毫米

检测与分割结果

单对象检测或实例分割结果（2D 框、类别、可选掩码/关键点）。

传感器数据指针类型定义

```
DetectionAndSegmentationResult::SENSORDATA_POINTER_TYPEDEFS(DetectionAndSegmentationResult)
```

检测与分割结果

```
DetectionAndSegmentationResult::DetectionAndSegmentationResult()
```

构造函数

```
DetectionAndSegmentationResult::DetectionAndSegmentationResult(  
    const cv::Rect &box,  
    const std::string &name,  
    const int index,  
    const float conf,  
    const std::vector< cv::Point2f > &kps=std::vector< cv::Point2f >(),  
    const std::vector< cv::Point > &poly=std::vector< cv::Point >()  
)
```

参数

Name	Type	Description
box	const cv::Rect &	
name	const std::string &	
index	const int	
conf	const float	
kps	const std::vector< cv::Point2f > &	
poly	const std::vector< cv::Point > &	

打印结果信息

```
void DetectionAndSegmentationResult::printInfo(std::ostream &os, bool showPolygon=false) const
```

参数

Name	Type	Description
os	std::ostream &	
showPolygon	bool	

成员变量

Name	Type	Description
bbox	cv::Rect	
className	std::string	
classIndex	int	
confidence	float	
keypoints	std::vector< cv::Point2f >	
polygon	std::vector< cv::Point >	

检测结果

打印检测和分割结果信息。

传感器数据指针类型定义

```
DetectionResult::SENSORDATA_POINTER_TYPEDEFS(DetectionResult)
```

添加检测结果

```
void DetectionResult::addResult(
    const cv::Rect &box,
    const std::string &className,
    int classIndex,
    float confidence
)
```

参数

Name	Type	Description
box	const cv::Rect &	
className	const std::string &	
classIndex	int	
confidence	float	

添加带关键点的检测结果

```
void DetectionResult::addResult(  
    const cv::Rect &box,  
    const std::vector< cv::Point2f > &kps,  
    const std::string &className,  
    int classIndex,  
    float confidence  
)
```

参数

Name	Type	Description
box	const cv::Rect &	
kps	const std::vector< cv::Point2f > &	
className	const std::string &	
classIndex	int	
confidence	float	

添加姿态

```
void DetectionResult::addPose(const Eigen::Matrix4f &pose)
```

参数

Name	Type	Description
pose	const Eigen::Matrix4f &	

添加关键点

```
void DetectionResult::addKeypoints(const std::vector< cv::Point2f > &kps)
```

参数

Name	Type	Description
kps	const std::vector< cv::Point2f > &	

添加点云

```
void DetectionResult::addPointCloud(const PointCloudPtr &cloud)
```

参数

Name	Type	Description
------	------	-------------

cloud	const PointCloudPtr &	
-------	-----------------------	--

设置运行信息

```
void DetectionResult::setRunningInfo(const std::string &info)
```

参数

Name	Type	Description
info	const std::string &	

添加分类掩码

```
void DetectionResult::addClassMask(const cv::Mat &mask)
```

参数

Name	Type	Description
mask	const cv::Mat &	

添加实例掩码

```
void DetectionResult::addInstanceMask(const cv::Mat &mask)
```

参数

Name	Type	Description
mask	const cv::Mat &	

清空结果

```
void DetectionResult::clear()
```

获取结果信息

```
std::string DetectionResult::getResultInfo()
```

从其他结果复制

```
void DetectionResult::copyFrom(const DetectionResult &other)
```

参数

Name	Type	Description
other	const DetectionResult &	

构造函数

```
DetectionResult::DetectionResult()
```

成员变量

Name	Type	Description
timestamp_ns	EIGEN_MAKE_ALIGNED_OPERATOR_NEW int64_t	
sensorName	std::string	
resultImage	cv::Mat	
resultPointCloud	std::vector< PointCloudPtr >	
detectionAndSegmentationResults	std::vector< DetectionAndSegmentationResult >	
boundingBoxes	std::vector< cv::Rect >	
classNames	std::vector< std::string >	
classIndices	std::vector< int >	
confidences	std::vector< float >	
ocrString	std::vector< std::string >	
targetPoses	std::vector< Eigen::Matrix4f >	
keypoints	std::vector< std::vector< cv::Point2f > >	
runningInfo	std::string	
classMask	cv::Mat	
instanceMask	cv::Mat	
garbage_result	GarbageResult::Ptr	
object6DPoses	std::map< std::string, std::vector< Eigen::Matrix4f > >	
grasp_pose_result	std::vector< std::vector< float > >	
targetPointPoses	std::vector< Eigen::Matrix4f >	
preTargetPointPoses	std::vector< Eigen::Matrix4f >	
ocrPointPoses	std::vector< Eigen::Matrix4f >	
ocrLabelImage	std::map< std::string, std::vector< cv::Mat > >	
ocrResults	std::map< std::string, Eigen::Matrix4f >	

设备信息

构造函数，初始化检测结果。

成员变量

Name	Type	Description
model	std::string	设备型号名称或标识符
serial_number	std::string	设备序列号
firmware_version	std::string	系统固件版本字符串
hardware_version	std::string	硬件版本或修订号
manufacturer	std::string	制造商名称或公司标识符

灵巧手状态

包含带时间戳的关节反馈；并在可用时（例如 Sharpa）包含来自灵巧手力反馈话题的逐传感器力/力矩测量。Sharpa 每只手按 22 关节建模。

成员变量

Name	Type	Description
timestamp_ns	int64_t	状态时间戳（自纪元以来的纳秒数）
joint_state	JointStateMessage	灵巧手关节状态消息
force_sensor_map	std::unordered_map< std::string, EffortInfo >	命名力传感器映射（Sharpa；其他为空）

灵巧手类型

SDK 使用此枚举将灵巧手命令和状态查询路由到正确的实现。Inspire 和 BrainCo 灵巧手共享标准关节命令/状态路径。Sharpa 灵巧手使用专用的 22 关节话题接口；完整状态在 DexhandState 中返回（包括力传感器）。

Enum Value	Description
INSPIRE	Inspire 灵巧手
BRAINCO	BrainCo 灵巧手
SHARPA	Sharpa 灵巧手

力矩信息

表示通常由力/扭矩传感器测量的 6 自由度 (6-DOF) 力螺旋（力和扭矩）。也称为空间力或广义力。

成员变量

Name	Type	Description
timestamp_ns	int64_t	测量时间戳（自纪元以来的纳秒数）
force	Vector3	力向量（牛顿）：[fx, fy, fz]- fx: X 方向力- fy: Y 方向力- fz: Z 方向力
torque	Vector3	力矩向量（牛顿·米）：[tx, ty, tz]- tx: 绕 X 轴力矩- ty: 绕 Y 轴力矩- tz: 绕 Z 轴力矩

错误

描述单个模块或组件的错误，包括错误代码和用于调试和诊断的人类可读描述。

构造函数

```
galbot::sdk::Error::Error(std::string compment_input, int error_code_input, std::string description_input)
```

构造函数

参数

Name	Type	Description
commpent_input	std::string	组件名称
error_code_input	int	错误代码
description_input	std::string	错误描述

成员变量

Name	Type	Description
commpent	std::string	组件
error_code	uint64_t	用于程序化错误处理的数值错误代码
description	std::string	错误描述

错误信息

包含来自多个模块或组件的带时间戳的错误消息集合。

成员变量

Name	Type	Description
timestamp_ns	int64_t	收集错误的时间戳（自纪元以来的纳秒数）
error_vec	std::vector< Error >	误差向量

力数据

包含来自 6 轴力/扭矩传感器的带有时间戳的力和扭矩测量值，通常安装在机器人手腕或工具端接口处。

成员变量

Name	Type	Description
timestamp_ns	int64_t	测量时间戳（自纪元以来的纳秒数）

force	Vector3	力
torque	Vector3	力矩向量（牛顿·米）：[tx, ty, tz]

坐标系

表示坐标系三个正交轴的可视化表示，通常用于显示坐标系的方向和姿态。

成员变量

Name	Type	Description
header	Header	
body_frame_id	std::string	
reference_frame_id	std::string	
pose	std::optional< Pose >	
twist	std::optional< Twist >	
wrench	std::optional< Wrench >	

G1 控制器名称

G1 控制器名称的字符串常量。

定义 G1 机器人型号支持的控制器名称。

成员变量

Name	Type	Description
CHASSIS_POSE_CTRL	constexpr Name	底盘姿态控制器
CHASSIS_TWIST_CTRL	constexpr Name	底盘速度控制器
LEG_PVT_BYPASS_CTRL	constexpr Name	腿部位置-速度-力矩旁路控制器
LEG_PVT_CTRL	constexpr Name	腿部位置-速度-力矩控制器
HEAD_PVT_BYPASS_CTRL	constexpr Name	头部位置-速度-力矩旁路控制器
HEAD_PVT_CTRL	constexpr Name	头部位置-速度-力矩控制器
LEFT_ARM_PVT_BYPASS_CTRL	constexpr Name	左臂位置-速度-力矩旁路控制器
LEFT_ARM_PVT_CTRL	constexpr Name	左臂位置-速度-力矩控制器
RIGHT_ARM_PVT_BYPASS_CTRL	constexpr Name	右臂位置-速度-力矩旁路控制器
RIGHT_ARM_PVT_CTRL	constexpr Name	右臂位置-速度-力矩控制器
LEFT_GRIPPER_CTRL	constexpr Name	左夹爪控制器
RIGHT_GRIPPER_CTRL	constexpr Name	右夹爪控制器
LEFT_DEXHAND_CTRL	constexpr Name	左手灵巧手控制器
RIGHT_DEXHAND_CTRL	constexpr Name	右手灵巧手控制器
CONTROLLER_NAME_NUM	constexpr Name	控制器名称数量

G1 关节组

"关节组"是 SDK 的主要控制/规划单元，而非单个关节： - **运动一致控制**：每个链/末端执行器组都会验证和执行命令 - **确定性命令排序**：joint_groups 按组顺序扩展为具体的 joint_names - **组级行为**：每个组都有自己的主动/被动属性和执行容忍度 **推荐用法**：在填充 API 参数（例如 joint_groups）时，使用此结构体中的常量。如果需要精确的关节名称，请在运行时通过 get_joint_names(true, {group_name}) 查询，而非硬编码。在同时接受 joint_groups 和 joint_names 的 API 中，joint_names 优先。

成员变量

Name	Type	Description
HEAD	constexpr Name	头部 2 自由度链条，默认关节：head_joint1、head_joint2，典型用途：注视/相机朝向控制
LEFT_ARM	constexpr Name	左臂 7 自由度链条，默认关节：left_arm_joint1 ... left_arm_joint7，典型用途：左臂 Reach/操作
RIGHT_ARM	constexpr Name	右臂 7 自由度链条，默认关节：right_arm_joint1 ... right_arm_joint7，典型用途：右臂 Reach/操作
LEFT_GRIPPER	constexpr Name	左夹爪链条，默认关节：left_gripper_joint1，典型用途：左夹爪开合和夹持宽度控制
RIGHT_GRIPPER	constexpr Name	右夹爪链条，默认关节：right_gripper_joint1，典型用途：右夹爪开合和夹持宽度控制
LEG	constexpr Name	腿部链条，默认关节：leg_joint1 ... leg_joint5，典型用途：下半身姿态/运动相关控制
CHASSIS	constexpr Name	底盘机构组（关节位置控制中为被动），默认关节：chassis_joint1 ... chassis_joint4，典型用途：底盘状态分组；底座运动应使用底座 API
LEFT_SUCTION_CUP	constexpr Name	左吸盘末端执行器组，默认关节：left_suction_cup_joint1，典型用途：左臂真空拾取/放置
RIGHT_SUCTION_CUP	constexpr Name	右吸盘末端执行器组，默认关节：right_suction_cup_joint1，典型用途：右臂真空拾取/放置
LEFT_DEXHAND	constexpr Name	左手灵巧手组，默认关节：left_dexhand_joint1 ...

		left_dexhand_joint6, 典型用途: 多指灵巧操作 (左)
RIGHT_DEXHAND	constexpr Name	右手灵巧手组, 默认关节: right_dexhand_joint1 ... right_dexhand_joint6, 典型用途: 多指灵巧操作 (右)

Galbot One Foxtrot 传感器类型

描述机器人手腕力传感器的力传感器枚举。

识别安装在机器人腕关节处的力/扭矩传感器，用于力控制操作和接触检测。

Enum Value	Description
LEFT_WRIST_FORCE	左腕力传感器
RIGHT_WRIST_FORCE	右腕力传感器
FORCE_NUM	力传感器数量

SDK 日志流

构造函数

```
galbot::sdk::GalbotSdkLogStream::GalbotSdkLogStream(  
    LogLevel level,  
    const char *file,  
    const char *func,  
    int line  
)
```

参数

Name	Type	Description
level	LogLevel	
file	const char *	
func	const char *	
line	int	

operator<<

```
GalbotSdkLogStream & galbot::sdk::GalbotSdkLogStream::operator<<(const T &v)
```

参数

Name	Type	Description
v	const T &	

operator<<

```
GalbotSdkLogStream & galbot::sdk::GalbotSdkLogStream::operator<<(std::ostream
&(*manip)(std::ostream &))
```

参数

Name	Type	Description
manip	std::ostream &(*)(std::ostream &)	

析构函数

```
galbot::sdk::GalbotSdkLogStream::~GalbotSdkLogStream() noexcept
```

垃圾检测结果

析构函数，销毁日志流对象

传感器数据指针类型定义

```
GarbageResult::SENSORDATA_POINTER_TYPEDEFS(GarbageResult)
```

添加垃圾检测结果

```
void GarbageResult::addResult(
    const std::string &uuid,
    Eigen::Vector3f box_shape,
    const Eigen::Vector3f &box_center_pos,
    const cv::Rect &box
)
```

参数

Name	Type	Description
uuid	const std::string &	
box_shape	Eigen::Vector3f	
box_center_pos	const Eigen::Vector3f &	
box	const cv::Rect &	

添加点云（按 UUID）

```
void GarbageResult::addPointCloud(const std::string &uuid, const PointCloudPtr
&bbox_pointcloud)
```

参数

Name	Type	Description
uuid	const std::string &	
bbox_pointcloud	const PointCloudPtr &	

成员变量

Name	Type	Description
boxes_shape	EIGEN_MAKE_ALIGNED_OPERATOR_NEW std::map< std::string, Eigen::Vector3f >	
boxes_center_pos	std::map< std::string, Eigen::Vector3f >	
bboxes	std::map< std::string, cv::Rect >	
bboxes_pointcloud	std::map< std::string, PointCloudPtr >	

夹爪状态

为指定 UUID 添加边界框点云数据。

成员变量

Name	Type	Description
timestamp_ns	int64_t	状态时间戳（自纪元以来的纳秒数）
width	double	夹爪开口宽度（米）， 距离
velocity	double	夹爪开合速度（米/秒）。
effort	double	力矩
is_moving	bool	运动标志（来自运动窗口）， false 表示在配置的时间窗口内未检测到有效运动， true 表示检测到有效运动
joint_positions	std::vector< double >	夹爪关节位置（弧度）， 通常 0-0.04 米

关节组命令

用于控制一组关节的命令结构， 包含多个关节的目标位置和参数。

成员变量

Name	Type	Description
time_from_start_s	double	
joint_commands	std::vector< JointCommand >	

消息头

标准消息头包含时间戳和坐标系信息。时间戳以纳秒的形式存储（与其他传感器类型统一）。

成员变量

Name	Type	Description
------	------	-------------

timestamp_ns	int64_t	数据采集的时间戳（自纪元以来的纳秒数），记录数据被捕获或生成的时间
frame_id	std::string	标识数据所在的坐标系，例如： "base_link"、"world"、 "camera_optical_frame"、 "lidar_link"、"map"

IK 求解器配置

逆运动学 (IK) 解算器配置参数。

该结构配置数值逆运动学解算器，用于计算实现所需末端执行器姿态的关节配置。它支持具有可配置种子策略、收敛容差、关节限制处理和超时参数的碰撞感知 IK。IK 求解是一种迭代数值优化过程，可以受益于多次随机初始化来找到可行的无碰撞解决方案。

设置碰撞感知 IK 求解器超时

```
void galbot::sdk::IKSolverConfig::set_col_aware_ik_timeout(double timeout)
```

设置碰撞感知逆运动学求解超时时间（秒）

参数

Name	Type	Description
timeout	double	求解超时（毫秒）

注意

较长的超时允许更多种子尝试但会延迟规划。

设置种子类型

```
void galbot::sdk::IKSolverConfig::set_seed_type(SeedType type)
```

设置逆运动学求解种子类型

参数

Name	Type	Description
type	SeedType	种子生成方法

设置关节位置限制的安全裕度

```
void galbot::sdk::IKSolverConfig::set_col_aware_ik_joint_limit_bias(double bias)
```

设置碰撞感知逆运动学关节限制偏差

参数

Name	Type	Description
bias	double	关节限制禁忌区域的距离阈值（单位：弧度）

注意

防止 IK 求解器提议在奇点附近的配置。

设置笛卡尔位置误差容差

```
void galbot::sdk::IKSolverConfig::set_translation_eps(const std::array< double, 3 > &eps)
```

设置平移收敛容差（米）

参数

Name	Type	Description
eps	const std::array< double, 3 > &	每轴位置误差容差 {x, y, z}（单位：米）

注意

当位置误差在此容差内时接受 IK 解。

设置方向误差容差

```
void galbot::sdk::IKSolverConfig::set_rotation_eps(const std::array< double, 3 > &eps)
```

设置旋转收敛容差（弧度）

参数

Name	Type	Description
eps	const std::array< double, 3 > &	按轴给出的方向误差容差 {roll, pitch, yaw}（单位：弧度）

注意

当方向误差在此容差内时接受 IK 解。

启用或禁用碰撞检测日志

```
void galbot::sdk::IKSolverConfig::set_enable_collision_check_log(bool enable)
```

设置是否启用碰撞检测日志输出

参数

Name	Type	Description
enable	bool	true 以输出碰撞检测日志，false 以

		抑制日志
--	--	------

注意

用于调试由于碰撞约束导致的 IK 失败。

获取碰撞感知 IK 求解器超时

```
double galbot::sdk::IKSolverConfig::get_col_aware_ik_timeout() const
```

获取碰撞感知逆运动学求解超时时间（秒）

返回值

Type	Description
double	超时持续时间（单位：毫秒）

获取 IK 求解器种子生成策略

```
SeedType galbot::sdk::IKSolverConfig::get_seed_type() const
```

获取逆运动学求解种子类型

返回值

Type	Description
SeedType	当前种子生成方法

获取关节限制安全裕度

```
double galbot::sdk::IKSolverConfig::get_col_aware_ik_joint_limit_bias() const
```

获取碰撞感知逆运动学关节限制偏差。较大的偏差会使求解器更倾向于避开碰撞区域。

返回值

Type	Description
double	关节限制偏置距离（单位：弧度）

获取笛卡尔位置误差容差

```
const std::array< double, 3 > & galbot::sdk::IKSolverConfig::get_translation_eps() const
```

获取平移收敛容差（米）

返回值

Type	Description
const std::array< double, 3 > &	每轴位置误差容差 {x, y, z}（单位：米）

获取方向误差容差

```
const std::array< double, 3 > & galbot::sdk::IKSolverConfig::get_rotation_eps() const
```

获取旋转收敛容差（弧度）

返回值

Type	Description
const std::array< double, 3 > &	按轴给出的姿态容差 {roll, pitch, yaw}（单位：弧度）。

检查是否启用碰撞检查日志

```
bool galbot::sdk::IKSolverConfig::get_enable_collision_check_log() const
```

获取是否启用碰撞检测日志输出

返回值

Type	Description
bool	启用碰撞检测日志时返回 true，禁用时返回 false。

打印 IK 求解器配置到标准输出

```
void galbot::sdk::IKSolverConfig::print() const
```

打印逆运动学求解器配置到标准输出

嵌套枚举

随机种子类型

逆运动学求解种子类型枚举

Enum Value	Description
RANDOM_SEED	随机种子
RANDOM_PROGRESSIVE_SEED	随机渐进种子
USER_DEFINED_SEED	用户自定义种子

IMU 数据

包含来自惯性测量单元 (IMU) 的带时间戳的数据，包括加速度计、陀螺仪和磁力计测量值。

成员变量

Name	Type	Description
timestamp_ns	int64_t	测量时间戳（自纪元以来的纳秒数）
accel	Vector3	线加速度（米/秒²）：[ax, ay, az]。

gyro	Vector3	陀螺仪数据
magnet	Vector3	磁力计数据

关节命令

指定轨迹或控制命令中单个机器人关节所需的运动参数。

成员变量

Name	Type	Description
position	double	期望的关节位置（弧度）
velocity	double	期望的关节速度（弧度/秒）
acceleration	double	加速度
effort	double	期望的关节力矩（牛顿·米）

单关节状态

单关节状态结构。

表示单个机器人关节的完整实时状态，包含运动学量（位置、速度、加速度）以及动力学量（力矩与电机电流）。

单关节状态

```
galbot::sdk::JointState::JointState()=default
```

单关节状态结构。

表示单个机器人关节的完整实时状态，包含运动学量（位置、速度、加速度）以及动力学量（力矩与电机电流）。

构造函数

```
galbot::sdk::JointState::JointState(  
    double position_input,  
    double velocity_input,  
    double acceleration_input,  
    double effort_input,  
    double current_input  
)
```

构造函数,

使用输入参数初始化关节状态

参数

Name	Type	Description
position_input	double	位置输入
velocity_input	double	速度输入
acceleration_input	double	加速度输入
effort_input	double	关节力矩/effort（牛顿·米，N·m）。
current_input	double	电流输入

成员变量

Name	Type	Description
position	double	关节角位置（弧度）
velocity	double	关节角速度（弧度/秒）
acceleration	double	关节角加速度（弧度/秒 ² ）
effort	double	关节力矩（牛顿·米，N·m）。
current	double	关节电流（安培）

关节状态消息

多个关节的关节状态的时间戳集合，通常表示机器人在某一时刻的完整关节配置的快照。

成员变量

Name	Type	Description
timestamp_ns	int64_t	采集时间戳（自纪元以来的纳秒数）
joint_state_vec	std::vector< JointState >	关节状态向量

关节状态

表示运动链的目标关节配置。扩展 RobotStates 以指定基于关节的运动目标。用于关节轨迹规划和正向运动学计算。所有关节角度必须以弧度为单位。矢量大小必须与指定运动链的自由度匹配。

获取类型

```
Type galbot::sdk::JointStates::get_type() const override
```

获取状态类型，指示这是关节空间目标

返回值

Type	Description
Type	， 指示这是关节空间目标

设置关节位置

```
void galbot::sdk::JointStates::set_joint_positions(const std::vector< double > &joints)
```

设置所有关节位置

参数

Name	Type	Description
joints	const std::vector< double > &	关节状态列表

注意

向量大小应等于链中驱动关节的数量。

设置关节

```
void galbot::sdk::JointStates::set_joint(int index, int val)
```

设置指定索引的关节值

参数

Name	Type	Description
index	int	关节索引
val	int	关节值数组

注意

函数执行边界检查；无效索引被静默忽略。

警告

越界访问不会返回错误；请确保索引有效。

成员变量

Name	Type	Description
joint_positions	std::vector< double >	链的目标关节配置

运动学边界

机器人运动链关节的运动边界参数。

该结构定义了机器人运动链（例如，机械臂、移动底座或腿链）的运动学约束。它指定了链条中每个关节的位置、速度、加速度和加加速度限制。这些边界对于确保轨迹规划和执行过程中安全且物理上可行的运动至关重要。每个向量应包含运动链中每个关节的一个值。所有关节空间量均以弧度或每单位时间的弧度指定。

设置运动链名称

```
void galbot::sdk::KinematicsBoundary::set_chain_name(const std::string &name)
```

设置该边界约束所属的运动链名称

参数

Name	Type	Description
name	const std::string &	边界名称

设置下限位置限制

```
void galbot::sdk::KinematicsBoundary::set_lower_limit(const std::vector< double > &limits)
```

设置关节位置下限

参数

Name	Type	Description
limits	const std::vector< double > &	每个关节的下限位置限制向量

注意

向量大小必须等于链中的关节数。

设置上限位置限制

```
void galbot::sdk::KinematicsBoundary::set_upper_limit(const std::vector< double > &limits)
```

设置关节位置上限

参数

Name	Type	Description
limits	const std::vector< double > &	每个关节的上限位置限制向量

注意

向量大小必须等于链中的关节数。

设置下限速度限制

```
void galbot::sdk::KinematicsBoundary::set_vel_lower_limit(const std::vector< double > &limits)
```

设置速度下限

参数

Name	Type	Description
limits	const std::vector< double > &	每个关节的速度下限向量（单位：rad/s）。

注意

双向关节通常为负值。

设置上限速度限制

```
void galbot::sdk::KinematicsBoundary::set_vel_upper_limit(const std::vector< double >
&limits)
```

设置速度上限

参数

Name	Type	Description
limits	const std::vector< double > &	每个关节的速度上限向量（单位：rad/s）。

注意

双向关节通常为正值。

设置下限加速度限制

```
void galbot::sdk::KinematicsBoundary::set_acc_lower_limit(const std::vector< double >
&limits)
```

设置加速度下限

参数

Name	Type	Description
limits	const std::vector< double > &	每个关节的加速度下限向量（单位：rad/s ² ）。

注意

用于轨迹优化和平滑约束。

设置上限加速度限制

```
void galbot::sdk::KinematicsBoundary::set_acc_upper_limit(const std::vector< double >
&limits)
```

设置加速度上限

参数

Name	Type	Description
limits	const std::vector< double > &	每个关节的加速度上限向量（单位：rad/s ² ）。

注意

用于轨迹优化和平滑约束。

设置下限加加速度限制

```
void galbot::sdk::KinematicsBoundary::set_jerk_lower_limit(const std::vector< double >
&limits)
```

设置加加速度下限

参数

Name	Type	Description
limits	const std::vector< double > &	每个关节的加加速度下限向量（单位：rad/s ³ ）。

注意

加加速度约束可提高运动平滑度并减少机械应力。

设置上限加加速度限制

```
void galbot::sdk::KinematicsBoundary::set_jerk_upper_limit(const std::vector< double >
&limits)
```

设置加加速度上限

参数

Name	Type	Description
limits	const std::vector< double > &	每个关节的加加速度上限向量（单位：rad/s ³ ）。

注意

加加速度约束可提高运动平滑度并减少机械应力。

获取运动链名称

```
const std::string & galbot::sdk::KinematicsBoundary::get_chain_name() const
```

获取该边界约束所属的运动链名称

返回值

Type	Description
const std::string &	运动链名称字符串的常量引用

获取下限位置限制

```
const std::vector< double > & galbot::sdk::KinematicsBoundary::get_lower_limit() const
```

获取关节位置下限 (rad)

返回值

Type	Description
const std::vector< double > &	向量下限位置限制 (每个关节, 单位: 弧度)

获取上限位置限制

```
const std::vector< double > & galbot::sdk::KinematicsBoundary::get_upper_limit() const
```

获取关节位置上限 (rad)

返回值

Type	Description
const std::vector< double > &	向量上限位置限制 (每个关节, 单位: 弧度)

获取下限速度限制

```
const std::vector< double > & galbot::sdk::KinematicsBoundary::get_vel_lower_limit() const
```

获取速度下限 (rad/s)

返回值

Type	Description
const std::vector< double > &	向量下限速度限制 (每个关节, 单位: 弧度/秒)

获取上限速度限制

```
const std::vector< double > & galbot::sdk::KinematicsBoundary::get_vel_upper_limit() const
```

获取速度上限 (rad/s)

返回值

Type	Description
const std::vector< double > &	向量上限速度限制 (每个关节, 单位: 弧度/秒)

获取下限加速度限制

```
const std::vector< double > & galbot::sdk::KinematicsBoundary::get_acc_lower_limit() const
```

获取加速度下限 (rad/s²)

返回值

Type	Description
const std::vector< double > &	常量引用, 向量下限加速度限制 (单位: rad/s²)

获取上限加速度限制

```
const std::vector< double > & galbot::sdk::KinematicsBoundary::get_acc_upper_limit() const
```

获取加速度上限 (rad/s²)

返回值

Type	Description
const std::vector< double > &	常量引用：每个关节的加速度上限向量（单位：rad/s²）。

获取下限加加速度限制

```
const std::vector< double > & galbot::sdk::KinematicsBoundary::get_jerk_lower_limit() const
```

获取加加速度下限 (rad/s³)

返回值

Type	Description
const std::vector< double > &	向量下限加加速度限制（每个关节，单位：rad/s³）

获取上限加加速度限制

```
const std::vector< double > & galbot::sdk::KinematicsBoundary::get_jerk_upper_limit() const
```

获取加加速度上限 (rad/s³)

返回值

Type	Description
const std::vector< double > &	向量上限加加速度限制（每个关节，单位：rad/s³）

打印输出

```
void galbot::sdk::KinematicsBoundary::print() const
```

打印运动学边界约束信息到标准输出

激光雷达数据

与 ROS 2sensor_msgs/PointCloud2 兼容的通用 N 维点云结构。将点数据存储为二进制 blob，并使用定义数据布局的字段描述符。支持有序（结构化）和无序（非结构化）点云。

成员变量

Name	Type	Description
------	------	-------------

header	Header	消息头
height	uint32_t	点云高度，无序点云 height = 1（单行），有序点云 height = 行数（例如，来自旋转 LiDAR 或深度相机）
width	uint32_t	点云宽度，无序点云 width = 点的总数，有序点云 width = 每行的点数（列数），总点数 = height × width
fields	std::vector< PointField >	描述每个点中存在的数据通道（x, y, z, intensity, rgb 等）及其二进制布局。
is_bigendian	bool	数据的字节序，true 表示大端序，false 表示小端序（x86/ARM 系统通常为小端序）
point_step	uint32_t	单个点结构的总字节大小，包括所有字段和填充，必须 >= 所有字段大小的总和，可能包含对齐填充
row_step	uint32_t	行行间距
data	std::vector< uint8_t >	包含所有点数据的二进制数据块（按行优先顺序排列），大小应等于 row_step × height 字节，每个点占用 point_step 字节，布局根据 fields 描述符确定
is_dense	bool	是否所有点都有效，true 表示所有点都有效且没有 NaN 或 Inf 值，false 表示点云可能包含无效点（NaN 或 Inf）

直线轨迹碰撞检测基元

用于笛卡尔线性轨迹验证的几何基元配置。

该结构配置笛卡尔空间中线性末端执行器轨迹的碰撞检测几何表示。它支持两种基本类型：无限细线和扫掠体积圆柱体。选择适当的原语会影响碰撞检测的保守性和计算成本。圆柱体基元可以更准确地对机器人的实际扫描体积进行建模，但需要更昂贵的几何查询。

设置直线检查基元类型

```
void galbot::sdk::LineTrajCheckPrimitive::set_line_check_primitive_type(PrimitiveType type)
```

设置直线检查基元类型

参数

Name	Type	Description
------	------	-------------

type	PrimitiveType	直线轨迹类型
------	---------------	--------

注意

推荐 CYLINDER 用于安全关键应用。

设置圆柱体基元半径

```
void galbot::sdk::LineTrajCheckPrimitive::set_cylinder_prim_radius(double radius)
```

设置圆柱基元半径（米）

参数

Name	Type	Description
radius	double	轨迹曲率半径

注意

较大的半径增加安全裕度但可能过于保守。

注意

仅当基元类型为 CYLINDER 时适用。

设置直线基元曲率

```
void galbot::sdk::LineTrajCheckPrimitive::set_line_prim_curvature(double curvature)
```

设置直线基元曲率

参数

Name	Type	Description
curvature	double	曲线曲率

注意

控制如何将弯曲路径离散为直线段。

注意

较低的值提高精度但增加计算成本。

获取直线检查基元类型

```
PrimitiveType galbot::sdk::LineTrajCheckPrimitive::get_line_check_primitive_type() const
```

获取直线检查基元类型

返回值

Type	Description
------	-------------

PrimitiveType	当前基元类型 (LINE 或 CYLINDER)
---------------	--------------------------

获取圆柱体基元半径

```
double galbot::sdk::LineTrajCheckPrimitive::get_cylinder_prim_radius() const
```

获取圆柱基元半径 (米)

返回值

Type	Description
double	圆柱体基元半径 (单位: 米)

获取直线基元曲率

```
double galbot::sdk::LineTrajCheckPrimitive::get_line_prim_curvature() const
```

获取直线基元曲率

返回值

Type	Description
double	直线基元曲率

打印输出

```
void galbot::sdk::LineTrajCheckPrimitive::print() const
```

打印直线轨迹检查基元信息到标准输出

嵌套枚举

几何基元类型

直线轨迹检查基元类型枚举

Enum Value	Description
LINE	直线
CYLINDER	圆柱体

日志配置

定义日志系统的配置参数, 包括文件设置、日志级别和输出选项。

成员变量

Name	Type	Description
path	std::string	路径

file_name	std::string	文件名
file_max_size	uint64_t	最大文件大小
file_max_num	uint64_t	最大文件数
level	LogLevel	级别
console_output	bool	控制台输出

日志信息

日志信息。

成员变量

Name	Type	Description
level	std::string	
message	std::string	消息

日志级别

表示日志消息的严重级别。

Enum Value	Description
TRACE	跟踪日志
DEBUG	调试日志
INFO	信息日志
WARN	警告日志
ERROR	错误日志
CRITICAL	严重日志

机器类型

此枚举定义了 Galbot SDK 支持的不同机器人平台或机器类型。客户端可以使用这些值来指定他们正在使用的机器人模型，特别是对于返回特定于平台的实现的工厂方法。将枚举保留在通用类型定义中可确保整个 SDK 的一致性，同时隐藏各个模块中的实现细节。

Enum Value	Description
G1	G1 机器人

运动规划配置

全面的运动规划配置管理。

MotionPlanConfig 充当所有运动规划子系统的集中配置容器。它聚合了采样策略、轨迹生成参数、逆运动学解算器设置、碰撞检测选项、可行性验证标准和运动学约束边界。此类提供了用于配置复杂运动规划管道的统一接口，支持简单的机械臂规划和具有多个运动链的全身人形运动生成。配置对象通过共享指针进行延迟初始化和 管理，以优化内存使用并支持可选功能配置。

构造函数

```
galbot::sdk::MotionPlanConfig::MotionPlanConfig()=default
```

运动规划配置类

设置更新时间

```
void galbot::sdk::MotionPlanConfig::set_update_time(int64_t t)
```

设置更新时间参数

参数

Name	Type	Description
t	int64_t	时间参数

注意

用于配置版本控制和缓存失效。

获取更新时间

```
int64_t galbot::sdk::MotionPlanConfig::get_update_time()
```

获取更新时间参数

返回值

Type	Description
int64_t	更新时间（单位：秒）

创建采样器配置

```
std::shared_ptr< SamplerConfig > galbot::sdk::MotionPlanConfig::create_sampler_config()
```

创建采样器配置并返回

返回值

Type	Description
std::shared_ptr< SamplerConfig >	采样器配置的共享指针

创建轨迹规划配置

```
std::shared_ptr< TrajectoryPlanConfig >  
galbot::sdk::MotionPlanConfig::create_trajectory_plan_config()
```

创建轨迹规划配置并返回

返回值

Type	Description
std::shared_ptr< TrajectoryPlanConfig >	轨迹规划配置的共享指针

创建 IK 求解器配置

```
std::shared_ptr< IKSolverConfig > galbot::sdk::MotionPlanConfig::create_ik_solver_config()
```

创建逆运动学求解器配置并返回

返回值

Type	Description
std::shared_ptr< IKSolverConfig >	IK 求解器配置的共享指针

创建碰撞检查选项

```
std::shared_ptr< CollisionCheckOption >  
galbot::sdk::MotionPlanConfig::create_collision_check_option()
```

创建碰撞检测选项并返回

返回值

Type	Description
std::shared_ptr< CollisionCheckOption >	碰撞检查选项的共享指针

创建轨迹可行性检查选项

```
std::shared_ptr< TrajectoryFeasibilityCheckOption >  
galbot::sdk::MotionPlanConfig::create_trajectory_feasibility_check_option()
```

创建轨迹可行性检查选项并返回

返回值

Type	Description
std::shared_ptr< TrajectoryFeasibilityCheckOption >	轨迹可行性检查选项的共享指针

创建直线轨迹检查基元

```
std::shared_ptr< LineTrajCheckPrimitive >
galbot::sdk::MotionPlanConfig::create_line_traj_check_primitive()
```

创建直线轨迹检查基元并返回

返回值

Type	Description
std::shared_ptr< LineTrajCheckPrimitive >	直线轨迹检查基元的共享指针

设置采样器配置

```
void galbot::sdk::MotionPlanConfig::set_sampler_config(const std::shared_ptr<
SamplerConfig > &config)
```

设置采样器配置

参数

Name	Type	Description
config	const std::shared_ptr< SamplerConfig > &	采样器配置的共享指针；传入 nullptr 清除配置。

设置轨迹规划配置

```
void galbot::sdk::MotionPlanConfig::set_trajectory_plan_config(
    const std::shared_ptr< TrajectoryPlanConfig > &config
)
```

设置轨迹规划配置

参数

Name	Type	Description
config	const std::shared_ptr< TrajectoryPlanConfig > &	新的运动规划配置

设置 IK 求解器配置

```
void galbot::sdk::MotionPlanConfig::set_ik_solver_config(const std::shared_ptr<
IKSolverConfig > &config)
```

设置逆运动学求解器配置

参数

Name	Type	Description
config	const std::shared_ptr< IKSolverConfig > &	IK 求解器配置的共享指针；传入 nullptr 清除配置。

设置碰撞检查选项

```
void galbot::sdk::MotionPlanConfig::set_collision_check_option(
    const std::shared_ptr< CollisionCheckOption > &option
)
```

设置碰撞检测选项

参数

Name	Type	Description
option	const std::shared_ptr< CollisionCheckOption > &	碰撞检查选项的共享指针；传入 nullptr 清除选项。

设置轨迹可行性检查选项

```
void galbot::sdk::MotionPlanConfig::set_trajectory_feasibility_check_option(
    const std::shared_ptr< TrajectoryFeasibilityCheckOption > &option
)
```

设置轨迹可行性检查选项

参数

Name	Type	Description
option	const std::shared_ptr< TrajectoryFeasibilityCheckOption > &	选项

设置可行性边界

```
void galbot::sdk::MotionPlanConfig::set_feasibility_boundary(
    const std::vector< KinematicsBoundary > &boundary
)
```

设置可行性边界约束

参数

Name	Type	Description
boundary	const std::vector< KinematicsBoundary > &	运动学边界向量，每个运动链一个

注意

这些边界用于一般轨迹可行性检查。

设置直线轨迹检查基元

```
void galbot::sdk::MotionPlanConfig::set_line_traj_check_primitive(
    const std::shared_ptr< LineTrajCheckPrimitive > &primitive
)
```

设置直线轨迹检查基元

参数

Name	Type	Description
primitive	const std::shared_ptr<LineTrajCheckPrimitive > &	运动基元

设置 IK 关约束

```
void galbot::sdk::MotionPlanConfig::set_ik_joint_limit(const std::vector<KinematicsBoundary > &boundary)
```

设置逆运动学关节位置限制边界

参数

Name	Type	Description
boundary	const std::vector<KinematicsBoundary > &	IK 求解器关节约束的运动学边界向量

注意

IK 限制可能比硬限制更紧以改善收敛性。

设置采样关约束

```
void galbot::sdk::MotionPlanConfig::set_sampler_joint_limit(const std::vector<KinematicsBoundary > &boundary)
```

设置采样器关节位置限制边界

参数

Name	Type	Description
boundary	const std::vector<KinematicsBoundary > &	采样算法的运动学边界向量

注意

采样限制定义了探索配置空间的有效范围。

设置硬关约束

```
void galbot::sdk::MotionPlanConfig::set_hard_joint_limit(const std::vector<KinematicsBoundary > &boundary)
```

设置硬关节位置限制边界

参数

Name	Type	Description
boundary	const std::vector<KinematicsBoundary > &	边界约束

注意

硬限制必须永不违反；通常对应于物理极限。

设置是否恢复 IK 关节限制

```
void galbot::sdk::MotionPlanConfig::set_revert_ik_joint_limit(bool flag)
```

设置是否恢复逆运动学关节限制

参数

Name	Type	Description
flag	bool	标志位

注意

用于通过暂时恢复极端限制来从受限配置中恢复。

设置需要恢复 IK 关节限制的运动链

```
void galbot::sdk::MotionPlanConfig::set_revert_ik_joint_limit_chains(const std::vector<std::string > &chains)
```

设置需要恢复逆运动学关节限制的运动链列表

参数

Name	Type	Description
chains	const std::vector< std::string > &	运动链列表

注意

如果非空，自动启用 revert_ik_joint_limit 标志。

注意

空向量禁用选择性恢复（适用于所有链）。

获取采样器配置

```
std::shared_ptr< SamplerConfig > galbot::sdk::MotionPlanConfig::get_sampler_config() const
```

获取采样器配置

返回值

Type	Description
std::shared_ptr< SamplerConfig >	采样器配置的共享指针；未设置时返回 nullptr。

注意

使用 create_sampler_config() 确保有效的配置。

获取轨迹规划配置

```
std::shared_ptr< TrajectoryPlanConfig >  
galbot::sdk::MotionPlanConfig::get_trajectory_plan_config() const
```

获取轨迹规划配置

返回值

Type	Description
std::shared_ptr< TrajectoryPlanConfig >	轨迹规划配置的共享指针；未设置时返回 nullptr。

注意

使用 create_trajectory_plan_config() 确保有效的配置。

获取 IK 求解器配置

```
std::shared_ptr< IKSolverConfig > galbot::sdk::MotionPlanConfig::get_ik_solver_config()  
const
```

获取逆运动学求解器配置

返回值

Type	Description
std::shared_ptr< IKSolverConfig >	IK 求解器配置的共享指针；未设置时返回 nullptr。

注意

使用 create_ik_solver_config() 确保有效的配置。

获取碰撞检查选项

```
std::shared_ptr< CollisionCheckOption >  
galbot::sdk::MotionPlanConfig::get_collision_check_option() const
```

获取碰撞检测选项

返回值

Type	Description
std::shared_ptr< CollisionCheckOption >	碰撞检查选项的共享指针；未设置时返回 nullptr。

注意

使用 create_collision_check_option() 确保有效的配置。

获取轨迹可行性检查选项

```
std::shared_ptr< TrajectoryFeasibilityCheckOption >  
galbot::sdk::MotionPlanConfig::get_trajectory_feasibility_check_option() const
```

获取轨迹可行性检查选项

返回值

Type	Description
std::shared_ptr< TrajectoryFeasibilityCheckOption >	轨迹可行性检查选项的共享指针；未设置时返回 nullptr。

注意

使用 create_trajectory_feasibility_check_option() 确保有效的配置。

获取直线轨迹检查基元

```
std::shared_ptr< LineTrajCheckPrimitive >  
galbot::sdk::MotionPlanConfig::get_line_traj_check_primitive() const
```

获取直线轨迹检查基元

返回值

Type	Description
std::shared_ptr< LineTrajCheckPrimitive >	直线轨迹检查基元的共享指针；未设置时返回 nullptr。

注意

使用 create_line_traj_check_primitive() 确保有效的配置。

获取采样器配置引用

```
SamplerConfig & galbot::sdk::MotionPlanConfig::get_sampler_config_ref()
```

获取采样器配置引用

返回值

Type	Description
SamplerConfig &	采样器配置的可修改引用（保证非空）。

获取轨迹规划配置引用

```
TrajectoryPlanConfig & galbot::sdk::MotionPlanConfig::get_trajectory_plan_config_ref()
```

获取轨迹规划配置引用

返回值

Type	Description
TrajectoryPlanConfig &	轨迹规划配置的可修改引用（保证非空）。

获取 IK 求解器配置引用

```
IKSolverConfig & galbot::sdk::MotionPlanConfig::get_ik_solver_config_ref()
```

获取逆运动学求解器配置引用

返回值

Type	Description
IKSolverConfig &	IK 求解器配置的可修改引用（保证非空）。

获取碰撞检查选项引用

```
CollisionCheckOption & galbot::sdk::MotionPlanConfig::get_collision_check_option_ref()
```

获取碰撞检测选项引用

返回值

Type	Description
CollisionCheckOption &	碰撞检查选项的可修改引用（保证非空）。

获取轨迹可行性检查选项引用

```
TrajectoryFeasibilityCheckOption &  
galbot::sdk::MotionPlanConfig::get_trajectory_feasibility_check_option_ref()
```

获取轨迹可行性检查选项引用

返回值

Type	Description
TrajectoryFeasibilityCheckOption &	轨迹可行性检查选项的可修改引用（保证非空）。

获取直线轨迹检查基元引用

```
LineTrajCheckPrimitive & galbot::sdk::MotionPlanConfig::get_line_traj_check_primitive_ref()
```

获取直线轨迹检查基元引用

返回值

Type	Description
LineTrajCheckPrimitive &	直线轨迹检查基元的可修改引用（保证非空）。

获取可行性边界

```
const std::vector< KinematicsBoundary > &  
galbot::sdk::MotionPlanConfig::get_feasibility_boundary() const
```

获取可行性边界约束

返回值

Type	Description
------	-------------

<code>const std::vector< KinematicsBoundary > &</code>	运动学可行性边界向量的可修改引用。
--	-------------------

获取可行性边界

```
std::vector< KinematicsBoundary > &  
galbot::sdk::MotionPlanConfig::get_feasibility_boundary()
```

获取可行性边界约束

返回值

Type	Description
<code>std::vector< KinematicsBoundary > &</code>	运动学可行性边界向量的可修改引用。

获取 IK 关约束

```
const std::vector< KinematicsBoundary > &  
galbot::sdk::MotionPlanConfig::get_ik_joint_limit() const
```

获取逆运动学关节位置限制边界

返回值

Type	Description
<code>const std::vector< KinematicsBoundary > &</code>	IK 关节限制边界向量的可修改引用。

获取 IK 关约束

```
std::vector< KinematicsBoundary > & galbot::sdk::MotionPlanConfig::get_ik_joint_limit()
```

获取逆运动学关节位置限制边界

返回值

Type	Description
<code>std::vector< KinematicsBoundary > &</code>	IK 关节限制边界向量的可修改引用。

获取采样关约束

```
const std::vector< KinematicsBoundary > &  
galbot::sdk::MotionPlanConfig::get_sampler_joint_limit() const
```

获取采样器关节位置限制边界

返回值

Type	Description
<code>const std::vector< KinematicsBoundary > &</code>	采样关节限制边界向量的可修改引用。

获取采样关约束

```
std::vector< KinematicsBoundary > &
galbot::sdk::MotionPlanConfig::get_sampler_joint_limit()
```

获取采样器关节位置限制边界

返回值

Type	Description
std::vector< KinematicsBoundary > &	采样关节限制边界向量的可修改引用。

获取硬关约束

```
const std::vector< KinematicsBoundary > &
galbot::sdk::MotionPlanConfig::get_hard_joint_limit() const
```

获取硬关节位置限制边界

返回值

Type	Description
const std::vector< KinematicsBoundary > &	硬关节限制边界向量的可修改引用。

注意

硬限制代表绝对的机械/安全约束，必须永不违反。

获取硬关约束

```
std::vector< KinematicsBoundary > & galbot::sdk::MotionPlanConfig::get_hard_joint_limit()
```

获取硬关节位置限制边界

返回值

Type	Description
std::vector< KinematicsBoundary > &	硬关节限制边界向量的可修改引用。

获取需要恢复 IK 关节限制的运动链

```
const std::vector< std::string > &
galbot::sdk::MotionPlanConfig::get_revert_ik_joint_limit_chains() const
```

获取需要恢复逆运动学关节限制的运动链列表

返回值

Type	Description
const std::vector< std::string > &	需要恢复 IK 关节限制的运动链名称向量的可修改引用（保证非空）。

注意

空向量表示在所有链上恢复（如果标志启用）。

获取需要恢复 IK 关节限制的运动链

```
std::vector< std::string > &  
galbot::sdk::MotionPlanConfig::get_revert_ik_joint_limit_chains()
```

获取需要恢复逆运动学关节限制的运动链列表

返回值

Type	Description
std::vector< std::string > &	需要恢复 IK 关节限制的运动链名称向量的可修改引用（保证非空）。

获取是否恢复 IK 关节限制

```
bool galbot::sdk::MotionPlanConfig::get_revert_ik_joint_limit()
```

获取是否恢复逆运动学关节限制

返回值

Type	Description
bool	true：将 IK 关节限制恢复为硬限制；false：使用默认限制。

打印输出

```
void galbot::sdk::MotionPlanConfig::print() const
```

将完整的运动规划配置打印到标准输出。

以人类可读格式输出所有子配置参数和运动学边界信息，便于调试、日志记录以及配置状态校验。

运动状态

机器人动作执行状态枚举。

表示机器人运动命令的执行状态，包括轨迹跟随、位姿到达等运动规划操作。

Enum Value	Description
SUCCESS	成功
TIMEOUT	超时
FAULT	故障
INVALID_INPUT	无效输入
INIT_FAILED	初始化失败

IN_PROGRESS	进行中
STOPPED_UNREACHED	已停止但未到达
DATA_FETCH_FAILED	数据获取失败
PUBLISH_FAIL	发布失败
COMM_DISCONNECTED	通信断开
STATUS_NUM	状态数量
UNSUPPORTED_FUNCRIION	不支持的函数

导航状态

导航任务执行状态枚举。

定义执行导航命令（例如移动到目标位置或遵循路径）时可能的结果。

Enum Value	Description
SUCCESS	导航任务成功完成
FAIL	失败
TIMEOUT	执行超时，任务未在允许的时间内完成
INVALID_INPUT	无效输入
MODE_ERR	模式错误
COMM_ERR	通信错误
WAIT_INITIALIZED	等待初始化

导航任务状态

导航任务当前状态枚举。

表示导航系统报告的活动或已完成的导航任务的当前状态。用于非阻塞导航时轮询，及时检测 RUNNING、SUCCESS 或 FAILED 并退出错误逻辑。

Enum Value	Description
UNKNOWN	未知
RUNNING	运行中
SUCCESS	导航任务成功完成
FAILED	失败

里程计数据

里程计数据。 包含来自里程计源（轮式编码器、IMU 融合等）的机器人位姿和速度估计。用于机器人定位和导航。

成员变量

Name	Type	Description
timestamp_ns	int64_t	里程计时间戳（自纪元以来的纳秒数）
position	std::array< double, 3 >	位置 [x, y, z]（米）
orientation	std::array< double, 4 >	四元数方向 [qx, qy, qz, qw]
linear_velocity	std::array< double, 3 >	线速度 [vx, vy, vz]（米/秒）
angular_velocity	std::array< double, 3 >	角速度 [wx, wy, wz]（弧度/秒）

参数

运动规划参数配置类。

此类扩展了 PlannerConfig，为全身运动规划和执行提供全面的配置选项。它封装了执行模式、驱动类型、工具架处理、碰撞检查和坐标系规范。所有角度参数均以弧度为单位，线性参数以米（SI 单位）为单位。

参数

```
galbot::sdk::Parameter::Parameter()=default
```

运动规划参数配置类。

此类扩展了 PlannerConfig，为全身运动规划和执行提供全面的配置选项。它封装了执行模式、驱动类型、工具架处理、碰撞检查和坐标系规范。所有角度参数均以弧度为单位，线性参数以米（SI 单位）为单位。

构造函数

```
galbot::sdk::Parameter::Parameter(
    bool direct_execute,
    bool blocking,
    double timeout,
    std::string actuate,
    bool tool_pose,
    bool check_collision,
    const std::string &frame="base_link"
)
```

全身运动规划配置的参数化构造函数。

参数

Name	Type	Description
direct_execute	bool	true 立即执行计划的运动，false 不执行
blocking	bool	是否阻塞等待完成
timeout	double	运动执行允许的最大时间（秒）
actuate	std::string	驱动模式字符串。可选值： "with_chain_only"（仅目标链/手臂）、 "with_torso"（手臂+躯干）、 "with_leg"（手臂+腿）。
tool_pose	bool	true 目标姿态相对于工具帧，false 相对于末端执行器
check_collision	bool	如果为 true，则在规划期间启用碰撞检测；如果为 false，则跳过碰撞检测
frame	const std::string &	参考坐标系

注意

仅当 blocking 为 true 时 timeout 才相关。

注意

无效的 actuate 字符串会导致未定义行为；确保使用有效键。

设置是否直接执行

```
void galbot::sdk::Parameter::set_direct_execute(bool direct_execute)
```

设置是否直接执行

参数

Name	Type	Description
direct_execute	bool	是否直接执行

设置是否阻塞

```
void galbot::sdk::Parameter::set_blocking(bool blocking)
```

设置是否阻塞执行

参数

Name	Type	Description
blocking	bool	是否阻塞

设置超时

```
void galbot::sdk::Parameter::set_timeout(double timeout)
```

设置运动执行超时时间（秒）

参数

Name	Type	Description
timeout	double	运动执行允许的最大时间（秒，必须为正值）。

注意

仅在阻塞模式启用时适用。

设置驱动类型

```
void galbot::sdk::Parameter::set_actuate(const std::string &actuate)
```

设置驱动类型

参数

Name	Type	Description
actuate	const std::string &	驱动类型

警告

actuate 必须为受支持的取值（"with_chain_only"、"with_torso"、"with_leg"），否则行为未定义。

设置工具位姿

```
void galbot::sdk::Parameter::set_tool_pose(bool tool_pose)
```

设置目标工具位姿

参数

Name	Type	Description
tool_pose	bool	工具位姿

设置是否检测碰撞

```
void galbot::sdk::Parameter::set_check_collision(bool check_collision)
```

设置是否进行碰撞检测

参数

Name	Type	Description
check_collision	bool	如果为 true，规划器会对自碰撞和环境障碍物进行碰撞检测；如果为 false，则跳过碰撞检查

警告

禁用碰撞检查可能会导致不安全的轨迹。

设置参考帧

```
void galbot::sdk::Parameter::set_reference_frame(const std::string &frame)
```

设置参考坐标系

参数

Name	Type	Description
frame	const std::string &	参考帧名称（例如： "base_link"、 "world"、 "odom"）

注意

必须是机器人 TF 树中的有效帧。

设置是否使用直线运动

```
void galbot::sdk::Parameter::set_move_line(bool flag)
```

设置是否为直线运动

参数

Name	Type	Description
flag	bool	标志位

注意

直线运动提供可预测的笛卡尔路径，但可能在狭窄空间中受限制。

获取是否直接执行

```
bool galbot::sdk::Parameter::get_direct_execute() const
```

获取是否直接执行

返回值

Type	Description
bool	true 直接执行计划的运动， false 不执行。

获取是否阻塞

```
bool galbot::sdk::Parameter::get_blocking() const
```

获取是否阻塞执行

返回值

Type	Description
bool	true 阻塞直到运动完成或超时，false 不阻塞。

获取超时

```
double galbot::sdk::Parameter::get_timeout() const
```

获取超时时间（秒）

返回值

Type	Description
double	超时时长（秒，正值）。

获取驱动类型

```
std::string galbot::sdk::Parameter::get_actuate_type() const
```

获取驱动类型

返回值

Type	Description
std::string	驱动类型字符串（"with_chain_only"、"with_torso"、"with_leg"或"unknown"（如果未找到））

获取工具位姿

```
bool galbot::sdk::Parameter::get_tool_pose() const
```

获取目标工具位姿

返回值

Type	Description
bool	true 目标姿态相对于工具帧，false 相对于末端执行器。

获取是否检测碰撞

```
bool galbot::sdk::Parameter::get_check_collision() const
```

获取是否进行碰撞检测

返回值

Type	Description
bool	如果启用碰撞检测则为 true，否则为 false

获取参考帧

```
std::string galbot::sdk::Parameter::get_reference_frame() const
```

获取参考坐标系

返回值

Type	Description
std::string	参考帧名称

是否使用直线运动

```
bool galbot::sdk::Parameter::is_move_line()
```

判断是否为直线运动

返回值

Type	Description
bool	true 使用直线（笛卡尔）运动，false 使用标准轨迹。

感知模块

启用的感知管道（初始化时加载的模型集）。

Enum Value	Description
FOUNDATION_STEREO	
LIGHT_STEREO	
STATUS_NUM	

规划器配置

运动规划配置结构。

用于机器人运动规划和执行的全面配置，控制规划模式、碰撞检查、参考坐标系和执行参数等行为。

成员变量

Name	Type	Description
is_direct_execute	bool	是否在规划后立即执行轨迹，true 表示一次性完成规划和执行，false 表示仅规划轨迹（用于预览或验证）
is_blocking	bool	是否同步等待操作完成，true 表示阻塞直到规划/执行完成或超时发生，false 表示立即返回（异步模式）

timeout_second	double	阻塞操作的超时时间（秒），等待规划或执行完成的最大时间，默认值：20 秒
actuate_type	ActuateType	驱动类型（运动链选择）。 - "with_chain_only": 仅使用指定的运动链- "with_torso": 包含躯干- "with_leg": 包含腿部- "unknown": 未知类型
is_tool_pose	bool	目标是否指定为工具中心点 (TCP) 位姿，true 表示目标是末端执行器 TCP 位姿（笛卡尔空间），false 表示目标是关节空间配置
is_relative_pose	bool	目标位姿是否相对于当前位姿，true 表示目标位姿是相对于当前位置的位移，false 表示目标位姿是绝对位姿
is_check_collision	bool	是否在规划期间启用碰撞检测，true 表示检查与障碍物和自碰撞，false 表示跳过碰撞检测（请谨慎使用）
reference_frame	std::string	目标位姿的参考坐标系。指定目标位姿所在的坐标系，例如： - "base_link": 机器人底座坐标系- "world": 世界坐标系- "odom": 里程计坐标系
joint_state	std::unordered_map< std::string, std::vector< double > >	指定规划的起始关节配置。 - 键（key）： 关节名称- 值（value）： 关节角度（弧度） - 空字典表示使用当前机器人状态
move_line	bool	是否规划直线运动，true 表示在笛卡尔空间中规划直线运动（末端执行器沿直线移动），false 表示在关节空间中规划标准轨迹（可能不是笛卡尔直线）

点

表示三维笛卡尔空间中的位置。

成员变量

Name	Type	Description
x	double	X 坐标（米）
y	double	Y 坐标（米）
z	double	Z 坐标（米）

点云字段

描述 PointCloud2 点结构中的一个数据字段，定义其名称、类型、偏移量和计数。与 ROS 2sensor_msgs/PointField 兼容。

成员变量

Name	Type	Description
name	std::string	此数据通道的标识符。标准字段名称包括：- "x", "y", "z": 3D 笛卡尔坐标- "intensity": 强度- "rgb" 或 "rgba": 颜色- "timestamp": 时间戳
offset	uint32_t	此字段相对于点数据结构起始位置的字节偏移量，例如：对于点布局 [x(float32), y(float32), z(float32), intensity(float32)], x 的偏移为 0, y 的偏移为 4, z 的偏移为 8, intensity 的偏移为 12
datatype	DataType	数据类型
count	uint32_t	此字段的数组长度，标量字段 (x, y, z, intensity) 通常为 1，数组字段可能大于 1（例如 count=3 表示 3 元素向量）

嵌套枚举

数据类型

点字段数据类型枚举

Enum Value	Description
UNKNOWN	未知
INT8	8 位整数
UINT8	8 位无符号整数
INT16	16 位整数
UINT16	16 位无符号整数
INT32	32 位整数
UINT32	32 位无符号整数
FLOAT32	32 位浮点数
FLOAT64	64 位浮点数

姿态

姿态（位置+方向）结构。

表示 3D 空间中的完整 6-DOF（自由度）姿态，结合位置（平移）和方向（旋转）信息。常用于机器人末端执行器姿态、物体姿态和坐标系变换。

姿态

```
galbot::sdk::Pose::Pose()=default
```

姿态（位置+方向）结构。

表示 3D 空间中的完整 6-DOF（自由度）姿态，结合位置（平移）和方向（旋转）信息。常用于机器人末端执行器姿态、物体姿态和坐标系变换。

构造函数

```
galbot::sdk::Pose::Pose(const T &pos, const T &quat)
```

构造函数，使用位置和四元数初始化位姿

参数

Name	Type	Description
pos	const T &	三维位置向量 [x, y, z]，单位：米，长度必须为 3。
quat	const T &	四元数

构造函数

```
galbot::sdk::Pose::Pose(const T &vec)
```

构造函数，使用 7 维向量初始化位姿

参数

Name	Type	Description
vec	const T &	7 维向量： [x, y, z, qx, qy, qz, qw]，前三项为位置（米），后四项为四元数

成员变量

Name	Type	Description
position	Point	三维位置 (x, y, z) ，单位：米。
orientation	Quaternion	方向

姿态状态

表示笛卡尔空间 (SE(3)) 中的目标末端执行器姿态。扩展 RobotStates 为运动链指定基于姿态的运动目标。用于逆运动学和笛卡尔轨迹规划。姿态值：以米为单位的位置，以四元数为单位的方向。坐标系必须存在于机器人的 TF 树中。

获取类型

```
Type galbot::sdk::PoseState::get_type() const override
```

获取位姿状态类型

返回值

Type	Description
Type	笛卡尔位姿目标

成员变量

Name	Type	Description
frame_id	std::string	运动链上的目标帧（例如："left_ee"）
reference_frame	std::string	参考坐标系（例如："base_link"）
pose	Pose	位姿

四元数

使用四元数表示 (x, y, z, w) 表示 3D 旋转。单位四元数的大小为 1，表示有效旋转。

成员变量

Name	Type	Description
x	double	X 坐标
y	double	Y 坐标
z	double	Z 坐标
w	double	四元数 w 分量

参考坐标系

指定表达姿态、位置或轨迹的坐标系。注意：这是一个用于 C 风格兼容性的普通枚举（不是枚举类）。

Enum Value	Description
------------	-------------

FRAME_WORLD	世界坐标系
FRAME_BASE	基座坐标系

感兴趣区域

定义图像内的矩形子区域以进行选择处理。与 ROS 2sensor_msgs/RegionOfInterest 兼容。

成员变量

Name	Type	Description
x_offset	uint32_t	X 偏移
y_offset	uint32_t	Y 偏移
height	uint32_t	感兴趣区域的像素行数
width	uint32_t	感兴趣区域的像素列数
do_rectify	bool	是否校正

RGB 数据

包含来自 RGB 相机的压缩彩色图像数据。兼容 ROS 2sensor_msgs/CompressedImage 格式。

RGB 图像转 OpenCV Mat

```
std::shared_ptr< cv::Mat > galbot::sdk::RgbData::convert_to_cv2_mat()
```

将 RGB 图像数据解码并转换为 OpenCV

Mat 格式以进行处理

返回值

Type	Description
std::shared_ptr< cv::Mat >	解码成功时返回 std::shared_ptr<cv::Mat> 智能指针；解码失败时返回空指针。

成员变量

Name	Type	Description
header	Header	包含采集时间戳和相机坐标系
format	std::string	指定压缩格式和编码
data	std::vector< uint8_t >	包含压缩图像数据的二进制数据块

机器人状态

封装机器人的完整运动状态，包括全身关节配置和移动底座位姿。此类作为更专业的状态表示（PoseState、JointStates）的基础，并在整个规划和控制管道中用于状态规范和反馈。所有角度值均以弧度为单位，线性值以米（SI 单位）为单位。基本姿态使用四元数表示方向（x、y、z、qx、qy、qz、qw）。

获取类型

```
virtual Type galbot::sdk::RobotStates::get_type() const
```

获取机器人状态类型

返回值

Type	Description
Type	类型枚举值，基类为 ROBOT_STATES

机器人状态

```
galbot::sdk::RobotStates::RobotStates()=default
```

封装机器人的完整运动状态，包括全身关节配置和移动底座位姿。此类作为更专业的状态表示（PoseState、JointStates）的基础，并在整个规划和控制管道中用于状态规范和反馈。所有角度值均以弧度为单位，线性值以米（SI 单位）为单位。基本姿态使用四元数表示方向（x、y、z、qx、qy、qz、qw）。

设置全身关节

```
void galbot::sdk::RobotStates::set_whole_body_joint(const std::vector< double > &joint_positions)
```

设置全身关节位置

参数

Name	Type	Description
joint_positions	const std::vector< double > &	目标关节位置

注意

向量大小应等于总驱动关节数。

设置基座状态

```
void galbot::sdk::RobotStates::set_base_state(const Pose &base_pose)
```

设置底座状态（位姿）

参数

Name	Type	Description
base_pose	const Pose &	SE(3)中的基座姿态：位置（米）和方向（四元数）

注意

四元数必须单位归一化 ($x^2 + y^2 + z^2 + w^2 = 1$) 。

构造函数

```
galbot::sdk::RobotStates::RobotStates(  
    const std::string &chain,  
    const std::vector< double > &whole_joint,  
    const Pose &base_pose  
)
```

构造函数，使用运动链、全身关节和底座位姿初始化机器人状态

参数

Name	Type	Description
chain	const std::string &	要分离工具的运动链
whole_joint	const std::vector< double > &	全身关节
base_pose	const Pose &	移动基座位姿：位置（米）+ 姿态（单位四元数）。

成员变量

Name	Type	Description
chain_name	std::string	链名称
whole_body_joint	std::vector< double >	全身关节
base_state	std::vector< double >	移动基座位姿：[x, y, z, qx, qy, qz, qw]（米，四元数）。

嵌套枚举

类型

机器人状态类型枚举

Used for runtime type identification of RobotStates-derived classes.

Enum Value	Description
POSE	姿态
JOINT	关节
ROBOT_STATES	机器人状态数量

采样器配置

基于采样的运动规划器的配置参数。

该结构配置基于采样的规划算法（例如，RRT、RRT*）。它控制状态空间采样分辨率、插值设置、路径简化和规划终止条件。基于采样的规划器通过随机采样状态并将它们连接起来构建运动计划图来探索配置空间。

设置状态检查类型

```
void galbot::sdk::SamplerConfig::set_state_check_type(StateCheckType type)
```

设置状态检查类型

参数

Name	Type	Description
type	StateCheckType	评估状态相似性的距离度量类型

设置状态检查分辨率

```
void galbot::sdk::SamplerConfig::set_state_check_resolution(double resolution)
```

设置状态检查分辨率

参数

Name	Type	Description
resolution	double	认为两个状态不同的最小距离阈值 (单位：弧度或无量纲)

注意

较低的值增加规划精度但可能减慢计算速度。

设置是否插值

```
void galbot::sdk::SamplerConfig::set_interpolate(bool enable)
```

设置是否启用插值

参数

Name	Type	Description
enable	bool	true 启用插值，false 仅使用采样路点

注意

插值可提高轨迹平滑度和碰撞检测覆盖率。

设置插值数量

```
void galbot::sdk::SamplerConfig::set_interpolation_cnt(int cnt)
```

设置插值点数量

参数

Name	Type	Description
cnt	int	要插入的中间路点数量（0 = 使用自动计算）

注意

较高的计数可改善碰撞检测但增加计算成本。

设置是否简化

```
void galbot::sdk::SamplerConfig::set_simplify(bool enable)
```

设置是否启用简化

参数

Name	Type	Description
enable	bool	true 启用路径简化和平滑，false 使用原始规划路径

注意

简化减少路点并提高轨迹效率。

设置最大简化时间

```
void galbot::sdk::SamplerConfig::set_max_simplification_time(double time)
```

设置最大简化时间（秒）

参数

Name	Type	Description
time	double	最大简化持续时间（单位：秒）；负值表示无时间限制

注意

较长的简化时间可能产生更短、更平滑的路径。

设置终止条件类型

```
void galbot::sdk::SamplerConfig::set_termination_condition_type(TerminationConditionType type)
```

设置终止条件类型

参数

Name	Type	Description
type	TerminationConditionType	终止条件类型（仅超时或超时/精确解）

设置最大规划时间

```
void galbot::sdk::SamplerConfig::set_max_planning_time(double time)
```

设置最大规划时间（秒）

参数

Name	Type	Description
time	double	最大规划持续时间（单位：秒）

注意

如果找到精确解，规划可能提前终止（取决于终止条件）。

获取状态检查类型

```
StateCheckType galbot::sdk::SamplerConfig::get_state_check_type() const
```

获取状态检查类型

返回值

Type	Description
StateCheckType	评估状态相似性的距离度量类型

获取状态检查分辨率

```
double galbot::sdk::SamplerConfig::get_state_check_resolution() const
```

获取状态检查分辨率

返回值

Type	Description
double	分辨率值（单位：弧度或无量纲，取决于状态检查类型）

获取是否插值

```
bool galbot::sdk::SamplerConfig::get_interpolate() const
```

获取是否启用插值

返回值

Type	Description
bool	如果启用插值则为 true，否则为 false

获取插值数量

```
int galbot::sdk::SamplerConfig::get_interpolation_cnt() const
```

获取插值点数量

返回值

Type	Description
int	要插入的中间路点数量

获取是否简化

```
bool galbot::sdk::SamplerConfig::get_simplify() const
```

获取是否启用简化

返回值

Type	Description
bool	如果启用路径简化则为 true，否则为 false

获取最大简化时间

```
double galbot::sdk::SamplerConfig::get_max_simplification_time() const
```

获取最大简化时间（秒）

返回值

Type	Description
double	最大简化持续时间（单位：秒）

获取终止条件类型

```
TerminationConditionType galbot::sdk::SamplerConfig::get_termination_condition_type() const
```

获取终止条件类型

返回值

Type	Description
TerminationConditionType	终止条件类型

获取最大规划时间

```
double galbot::sdk::SamplerConfig::get_max_planning_time() const
```

获取最大规划时间（秒）

返回值

Type	Description
double	最大规划持续时间（单位：秒）

打印输出

```
void galbot::sdk::SamplerConfig::print() const
```

打印采样器配置信息到标准输出

嵌套枚举

状态检查类型

状态检查类型枚举

Enum Value	Description
EUCLIDEAN_DISTANCE	欧氏距离
RADIAN_DISTANCE	弧度距离

终止条件类型

终止条件类型枚举

Enum Value	Description
TIMEOUT	仅当超过最大规划时间时终止
TIMEOUT_AND_EXACT_SOLUTION	超时或精确解

随机种子类型

指定逆运动学 (IK) 解算器的初始化策略。不同的种子类型会影响收敛速度和解的质量。

Enum Value	Description
RANDOM_SEED	随机种子
RANDOM_PROGRESSIVE_SEED	随机渐进种子
USER_DEFINED_SEED	用户自定义种子
SEED_TYPE_NUM	种子类型数量

传感器状态

表示传感器数据采集和处理操作的状态，适用于相机、激光雷达、IMU、力传感器和其他传感器类型。

Enum Value	Description
SUCCESS	成功
TIMEOUT	超时
FAULT	故障
INVALID_INPUT	无效输入
INIT_FAILED	初始化失败
IN_PROGRESS	进行中
STOPPED_UNREACHED	已停止但未到达
DATA_FETCH_FAILED	数据获取失败，传感器可能已断开连接或损坏
PUBLISH_FAIL	发布失败
COMM_DISCONNECTED	通信断开
STATUS_NUM	状态数量

传感器类型

描述机器人上各种传感器的传感器类型枚举。

识别机器人上可用于感知、定位和操作任务的不同传感器类型。

Enum Value	Description
HEAD_LEFT_CAMERA	左眼相机
HEAD_RIGHT_CAMERA	右眼相机
LEFT_ARM_CAMERA	左臂相机
RIGHT_ARM_CAMERA	右臂相机
LEFT_ARM_DEPTH_CAMERA	左臂深度相机
RIGHT_ARM_DEPTH_CAMERA	右臂深度相机
BASE_LIDAR	基座激光雷达
TORSO_IMU	躯干 IMU
BASE_ULTRASONIC	基座超声波
LEFT_FRONT_SURROUND_CAMERA	左前方环视相机
RIGHT_FRONT_SURROUND_CAMERA	右前方环视相机
LEFT_REAR_SURROUND_CAMERA	左后方环视相机
RIGHT_REAR_SURROUND_CAMERA	右后方环视相机
SENSOR_NUM	传感器数量

SingoriX 目标

SingoriX 控制器的目标表示，用于与 SingoriX 外部控制器通信。

成员变量

Name	Type	Description
header	Header	
target_group_trajectory_map	std::unordered_map< std::string, TargetGroupTrajectory >	
target_task_trajectory_map	std::unordered_map< std::string, TargetTaskTrajectory >	

吸盘动作状态

代表真空吸盘末端执行器的运行状态，跟踪从空闲到成功或失败的抽吸过程。

Enum Value	Description
suction_action_idle	吸取动作空闲
suction_action_sucking	吸取动作吸取中
suction_action_success	吸取动作成功
suction_action_failed	吸取动作失败

吸盘状态

包含真空吸盘夹具的当前状态，包括激活状态、压力读数和动作状态。

成员变量

Name	Type	Description
timestamp_ns	int64_t	状态时间戳（自纪元以来的纳秒数）
activation	bool	激活
pressure	double	压力
action_state	SUCTION_ACTION_STATE	动作状态

目标配置

目标的配置参数，包含目标类型、采样方式、约束条件等设置。

成员变量

Name	Type	Description
target_data	int32_t	

target_type	int32_t	
target_sampling	TargetSampling	
target_priority	int32_t	
target_id	std::string	
target_ts	Timestamp	

目标数据位

用于表示目标数据类型的位掩码，标识目标包含哪些数据分量。

Enum Value	Description
TARGET_DATA_NONE	
TARGET_DATA_JOINT_POS	
TARGET_DATA_JOINT_VEL	
TARGET_DATA_JOINT_ACC	
TARGET_DATA_JOINT_EFF	
TARGET_DATA_FRAME_POSE	
TARGET_DATA_FRAME_TWIST	
TARGET_DATA_FRAME_WRENCH	
TARGET_DATA_DEFAULT	

目标关节组轨迹

目标在关节组空间中的轨迹，包含一系列途经点和时间信息。

成员变量

Name	Type	Description
target_config	TargetConfig	
joint_names	std::vector< std::string >	
group_commands	std::vector< GroupCommand >	

目标采样

目标轨迹的采样方法，用于在路径点之间生成中间点。

Enum Value	Description
TARGET_SAMPLING_DEFAULT	
TARGET_SAMPLING_DIRECT_PASS	
TARGET_SAMPLING_LINEAR_INTERPOLATE	
TARGET_SAMPLING_TRAPEZOIDAL_PROFILE	

TARGET_SAMPLING_S_CURVE_PROFILE	
TARGET_SAMPLING_CUBIC_SPLINES	
TARGET_SAMPLING_QUINTIC_SPLINES	
TARGET_SAMPLING_B_SPLINES	
TARGET_SAMPLING_CUSTOM	

目标任务轨迹

目标在任务空间中的轨迹，包含任务级别的约束和要求。

成员变量

Name	Type	Description
target_config	TargetConfig	
group_names	std::vector< std::string >	
joint_names	std::vector< std::string >	
subtask_names	std::vector< std::string >	
task_commands	std::vector< TaskCommand >	

目标类型位

用于表示目标类型的位掩码，标识目标是哪种类型（关节空间、任务空间等）。

Enum Value	Description
TARGET_TYPE_NONE	
TARGET_TYPE_TOUCH	
TARGET_TYPE_CLEAR	
TARGET_TYPE_PREPENDNOW	
TARGET_TYPE_APPEND	
TARGET_TYPE_OVERRIDE	
TARGET_TYPE_PROVERRIDE	
TARGET_TYPE_DEFAULT	

任务命令

任务级别的命令结构，包含一个或多个目标和执行参数。

成员变量

Name	Type	Description
time_from_start_s	double	

subtask_commands	std::vector< FrameTriad >	
------------------	---------------------------	--

时间戳

表示具有秒和纳秒分量的高精度时间点。兼容 ROS 2builtin_interfaces/Time 和 std_msgs/Header 时间戳格式。

成员变量

Name	Type	Description
sec	int64_t	秒
nanosec	uint32_t	纳秒

轨迹

表示一个完整的机器人轨迹，包含多个随时间变化的路点。joint_groups 和 joint_names 不能同时为空。如果同时为空，函数将返回 ControlStatus::INVALID_INPUT。此限制确保了关节顺序的确定性，防止因隐式内部默认顺序（可能在 SDK 版本之间变化）导致的难以察觉的 bug。使用方法：推荐设置具有语义组名的 joint_groups 或 joint_names。如果同时设置，joint_names 优先于 joint_groups

成员变量

Name	Type	Description
points	std::vector< TrajectoryPoint >	有序的轨迹路点列表。每个点的 joint_command_vec 必须与 joint_names 或 joint_groups 定义的关节顺序匹配。关节顺序的确定方式：(1) 如果 joint_names 非空，则使用 joint_names；(2) 否则使用 joint_groups 按顺序展开（例如，{"head","left_arm"} 展开为 head_joint1, head_joint2, left_arm_joint1, ... left_arm_joint7）。
joint_groups	std::vector< std::string >	关节组名称列表，如 ["left_arm", "right_arm"]。注意：joint_groups 和 joint_names 不能同时为空
joint_names	std::vector< std::string >	显式关节名称。当非空时，优先于 joint_groups，并且仅验证为活动关节。当 joint_groups 也为空时，不能同时为空。

轨迹控制状态

机器人轨迹执行状态枚举。

表示机器人沿着由多个路点组成的预先规划的轨迹运行时的实时执行状态。

Enum Value	Description
INVALID_INPUT	无效输入
RUNNING	运行中
COMPLETED	已完成
STOPPED_UNREACHED	轨迹执行过程中停止但未达到终点
ERROR	错误日志
DATA_FETCH_FAILED	数据获取失败
STATUS_NUM	状态数量

轨迹可行性检查选项

轨迹验证和可行性检查配置。

该结构提供了对在轨迹验证期间强制执行可行性约束的细粒度控制。它支持碰撞检测、关节限制合规性和速度剖面可行性的独立切换。有选择地禁用检查可以提高调试、模拟或保证满足某些约束的场景的计算性能。禁用可行性检查可能会产生不安全或物理上无法实现的轨迹。仅当通过其他方式验证约束时才应谨慎使用。

设置是否禁用碰撞检查

```
void galbot::sdk::TrajectoryFeasibilityCheckOption::set_disable_collision_check(bool disable)
```

设置是否禁用碰撞检测

参数

Name	Type	Description
disable	bool	是否禁用碰撞检测

警告

禁用碰撞检查可能导致轨迹与环境发生碰撞

设置是否禁用关节限制检查

```
void galbot::sdk::TrajectoryFeasibilityCheckOption::set_disable_joint_limit_check(bool disable)
```

设置是否禁用关节限制检查

参数

Name	Type	Description
disable	bool	传入 true 表示跳过关节限位验证，传入 false 表示强制执行位置限位验证。

警告

禁用关节限制检查可能导致轨迹超出关节限位

设置是否禁用速度可行性检查

```
void
galbot::sdk::TrajectoryFeasibilityCheckOption::set_disable_velocity_feasibility_check(bool
disable)
```

设置是否禁用速度可行性检查

参数

Name	Type	Description
disable	bool	true 跳过速度限制验证，false 强制执行速度验证。

注意

速度可行性确保轨迹可以被机器人执行。

获取是否禁用碰撞检查

```
bool galbot::sdk::TrajectoryFeasibilityCheckOption::get_disable_collision_check() const
```

获取是否禁用碰撞检测

返回值

Type	Description
bool	若当前已禁用碰撞检查则返回 true；否则返回 false。

获取是否禁用关节限制检查

```
bool galbot::sdk::TrajectoryFeasibilityCheckOption::get_disable_joint_limit_check() const
```

获取是否禁用关节限制检查

返回值

Type	Description
bool	若当前已禁用关节限位检查则返回 true；否则返回

	false。
--	--------

获取是否禁用速度可行性检查

```
bool
galbot::sdk::TrajectoryFeasibilityCheckOption::get_disable_velocity_feasibility_check()
const
```

获取是否禁用速度可行性检查

返回值

Type	Description
bool	若当前已禁用速度可行性检查则返回 true；否则返回 false。

打印输出

```
void galbot::sdk::TrajectoryFeasibilityCheckOption::print() const
```

打印轨迹可行性检查选项到标准输出

轨迹规划配置

轨迹规划和参数化配置。
该结构配置轨迹生成参数，用于将离散运动计划转换为平滑的时间参数化轨迹。它支持单段和多路点轨迹规划。轨迹规划涉及计算沿几何路径的速度和加速度分布，同时尊重运动学约束。

设置最小移动时间

```
void galbot::sdk::TrajectoryPlanConfig::set_min_move_time(double time)
```

设置最小运动时间（秒）

参数

Name	Type	Description
time	double	最小轨迹执行时间（单位：秒）

注意

非零值防止运动过快；0.0 允许在运动学限制内的最高速度

设置直线插值中间点

```
void galbot::sdk::TrajectoryPlanConfig::set_move_line_intermediate_point(double value)
```

设置直线运动中间点设置

参数

Name	Type	Description
value	double	线性路径段的中间路点数量（无量纲）

注意

较高的值提高笛卡尔路径精度但增加计算成本。

设置路点规划预期时间

```
void galbot::sdk::TrajectoryPlanConfig::set_way_point_plan_expected_time(double time)
```

设置路点规划期望时间

参数

Name	Type	Description
time	double	路点序列的预期轨迹执行时间（单位：秒）

注意

用作时间最优轨迹生成算法的提示。

获取最小移动时间

```
double galbot::sdk::TrajectoryPlanConfig::get_min_move_time() const
```

获取最小运动时间（秒）

返回值

Type	Description
double	最小轨迹执行时间（单位：秒）

获取直线插值中间点

```
double galbot::sdk::TrajectoryPlanConfig::get_move_line_intermediate_point() const
```

获取直线运动中间点设置

返回值

Type	Description
double	线性路径段的中间路点数量（无量纲）

获取路点规划预期时间

```
double galbot::sdk::TrajectoryPlanConfig::get_way_point_plan_expected_time() const
```

获取路点规划期望时间

返回值

Type	Description
double	路点规划的预期轨迹执行时间（单位：秒）

打印输出

```
void galbot::sdk::TrajectoryPlanConfig::print() const
```

打印轨迹规划配置信息到标准输出

轨迹点

表示机器人轨迹中的路点，指定特定时间的关节状态。joint_command_vec 的顺序必须与 Trajectory.joint_names 或 Trajectory.joint_groups（按顺序展开）定义的关节顺序一致。顺序不匹配将导致未定义行为。

成员变量

Name	Type	Description
time_from_start_second	double	起始时间偏移（秒）
joint_command_vec	std::vector< JointCommand >	该路点所有关节的关节命令列表。顺序必须与 joint_names 或 joint_groups 展开顺序一致。

坐标变换消息

表示带时间戳的坐标系变换，包括平移和旋转。常用于机器人技术中的 TF（变换）树。

成员变量

Name	Type	Description
timestamp_ns	int64_t	变换时间戳（自纪元以来的纳秒数）
translation	Point	平移向量（米）。
rotation	Quaternion	旋转

扭曲速度

表示刚体的线速度和角速度组合，遵循 ROS 标准消息格式。

成员变量

Name	Type	Description
linear	Vector3	
angular	Vector3	

超声波数据

包含带有时间戳的单个超声波距离测量。

成员变量

Name	Type	Description
timestamp_ns	int64_t	超声波测量时间戳（纳秒）
distance	double	到最近障碍物的测量距离（米）。

超声波类型

底盘超声波传感器探头枚举（8 个方向）

识别布置在移动底盘周围的各个超声波传感器，用于全向障碍物检测和接近感测。

Enum Value	Description
FRONT_LEFT	左前超声波
FRONT_RIGHT	右前超声波
RIGHT_LEFT	右侧左超声波
RIGHT_RIGHT	右侧右超声波
BACK_LEFT	左后超声波
BACK_RIGHT	右后超声波
LEFT_LEFT	左侧左超声波
LEFT_RIGHT	左侧右超声波
ULTRASONIC_NUM	超声波传感器数量

三维向量

表示三维矢量，用于力、扭矩、速度、加速度和其他矢量。

成员变量

Name	Type	Description
x	double	X 分量
y	double	Y 分量
z	double	Z 分量

力与力矩

力与力矩相关说明。

成员变量

Name	Type	Description
force	Vector3	
torque	Vector3	

雪山混混

18. 故障排除

这份文档整理了 SDK 使用中常见的问题。排查时请先做“通用检查”，确认机器人处于可工作前提，再进入状态/版本确认和具体问题定位。

快速定位

如果你的问题暂未在下方列表中找到，可直接跳转到文末的仍未解决？获取进一步支持渠道。

建议先按这个统一顺序排查：

1. 通用检查项：先确认急停已松开，再确认当前工作模式是否进入 ABNORMAL，确保机器人处于正常工作模式（这是所有功能正常工作的前提）。
1. 系统与配置：在确认工作模式正常后，再查看机器人与 SDK 版本信息是否对应。
1. 具体故障现象：最后进入对应问题章节做专项排查。

如果你是“想确认机器状态或版本”，看这里：

- 如何查看当前工作模式
- 如何查看机器人系统版本
- 如何检查机器人系统版本与 SDK 兼容性
- 机器人紧急开关（急停按钮）说明
- 机器人机载算力单元介绍（HPU + XCU）
- 如何进入 HPU 与 XCU
- 如何查看机器 SN 号
- 如何检查关节是否正常

如果你是“机器人不动/不响应”，再看这里：

- 发送控制指令无响应
- 机器人上运行 SDK 程序可用，但有线连接不行
- 机器人关节无法运动
- 机器人进入异常模式，重启服务
- 机器人进入异常模式：定位服务异常
- 机器人底盘无法移动
- 末端执行器控制失败
- 机器人扬声器无声音

如果你是“系统配置相关”，看这里：

- 如何启用末端实时控制指令

- 关节限位在哪里配置
- 启用/禁用某个系统服务
- 重启后 embosa IP 配置被改回去

如果你是“相机相关问题”，看这里：

- 如何修改相机分辨率
- 如何查看相机帧率
- 如何查看相机内外参和标定信息

如果你是“其他问题”，看这里：

- SDK log 在哪里
- 如何开启 core 文件并调试
- 在 Robot 上执行 Python 找不到 module
- 执行 Python 示例报错：numpy 没有 typeDict
- SDK Docker 启动方式
- 如何使用感知服务（GBS 1.15.16+）

开始前先确认

- 日志和进程命令要在对应端执行，先判断问题属于哪一类服务：
 - HPU：定位、导航、规划、感知等服务（例如 localization、navigation、mps、perception）。
 - XCU：运动控制相关服务（例如 singorix、末端控制、模式管理等）。
 - 快速判断方法：机器人“能连上但不动/动作异常”优先查 XCU；“能动但地图、路径、避障、感知结果异常”优先查 HPU。
- 路径、服务名、配置字段区分大小写。
- 如果是 PC 端远程部署，先确认网络和 IP 配置已正确生效。

通用检查项（建议先做）

1. 急停状态：先确认机器人背部红色急停按钮已松开/已弹出。
1. 工作模式：在 XCU 执行 `/data/bin/sys_tool`，确认当前模式不是 ABNORMAL。
1. 异常服务定位：若当前模式是 ABNORMAL，先查看 `/userdata/log/mission_manager/mission_manager.INFO` 并搜索 `abnormal_service`，恢复到正常模式后再继续后续排查。
1. 状态/版本确认：仅在模式正常后，再检查 SN、系统版本、当前模式详情等基础信息。

1. 服务状态：按问题类型检查 singorix、galbot_chassis_service 等关键服务是否在线。

控制与运动

发送控制指令无响应

先按上面的通用检查项走完，再继续看控制链路。

先看主日志：

```
tail -f /userdata/log/SingoriX/info/singorix_wbcs_main.INFO
```

再检查 singorix 进程：

```
ps -ef | grep singorix | grep -v grep
```

如果 singorix 在，但控制还是没响应，再看底盘相关进程：

```
ps -ef | grep chassis
```

若是 PC 端部署，重启后配置容易被改回去，这时要检查 HPU 和 XCU 上 /data/config/system.cfg 里的 enable_modify_embosa_cfg。它为 false 时，embosa_ip_config.json 才不会在重启流程里被覆盖。

```
[20260114 11:43:43.242373 3359 wbcs_interface.cpp:589] wbcs_error_writer_ SENT 125 datas, time: 3854.9
[20260114 11:43:43.242452 3359 wbcs_interface.cpp:592] Errors Timestamp : 1768362223.241983456
[20260114 11:43:43.242594 3359 wbcs_interface.cpp:595] Errors component: right_arm_driver_state, error code: 46, description: DriverEstop
[20260114 11:43:43.242686 3359 wbcs_interface.cpp:595] Errors component: chassis_driver_state, error code: 46, description: DriverEstop
[20260114 11:43:43.242983 3359 wbcs_interface.cpp:595] Errors component: chassis_max_error_level, error code: 30000, description: ErrorSafetyMedium
[20260114 11:43:43.243088 3359 wbcs_interface.cpp:595] Errors component: left_arm_driver_state, error code: 46, description: DriverEstop
[20260114 11:43:43.243170 3359 wbcs_interface.cpp:595] Errors component: leg_driver_state, error code: 46, description: DriverEstop
[20260114 11:43:43.243314 3359 wbcs_interface.cpp:595] Errors component: left_arm_max_error_level, error code: 30000, description: ErrorSafetyMedium
[20260114 11:43:43.243405 3359 wbcs_interface.cpp:595] Errors component: head_max_error_level, error code: 30000, description: ErrorSafetyMedium
[20260114 11:43:43.243495 3359 wbcs_interface.cpp:595] Errors component: head_driver_state, error code: 46, description: DriverEstop
[20260114 11:43:43.243585 3359 wbcs_interface.cpp:595] Errors component: right_arm_max_error_level, error code: 30000, description: ErrorSafetyMedium
[20260114 11:43:43.243673 3359 wbcs_interface.cpp:595] Errors component: leg_max_error_level, error code: 30000, description: ErrorSafetyMedium
[20260114 11:43:43.557313 3277 main.cpp:85] Thread pool Pending tasks: 0

root@galbot-xcu:~# ps -ef | grep singorix | grep -v grep
root 18751 18679 57 11:50 ? 00:09:47 /data/bin/singorix_wbcs_main /data/bin/./config/SingoriX/galbot_one_golf/real/robot_config.toml
root@galbot-xcu:~#

[20260114 11:46:28.992761 29762 wbcs_interface.cpp:589] wbcs_error_writer_ SENT 1 datas, time: 4020.65
[20260114 11:46:28.992838 29762 wbcs_interface.cpp:592] Errors Timestamp : 1768362388.992542143
[20260114 11:46:28.992900 29762 wbcs_interface.cpp:595] Errors component: chassis_driver_state, error code: 40, description: DriverInit
[20260114 11:46:28.992950 29762 wbcs_interface.cpp:595] Errors component: chassis_max_error_level, error code: 30000, description: ErrorSafetyMedium
[20260114 11:46:29.993495 29762 wbcs_interface.cpp:618] Errors Timestamp : 1768362389.993191809
[20260114 11:46:29.993748 29762 wbcs_interface.cpp:620] Error component: WBCSManager, error code: 20, description: Sync chassis ...
[20260114 11:46:29.993964 29762 wbcs_interface.cpp:589] wbcs_error_writer_ SENT 1 datas, time: 4021.65
```

机器人上运行 SDK 程序可用，但有线连接不行

问题现象：SDK 程序在机器人本机运行正常，但通过 PC 有线连接运行失败。

常见原因：embosa_ip 未正确配置，或配置被重置。

处理建议：

1. 先检查网络配置是否正确，重点核对 PC、XCU、HPU 的 IP 与 embosa_ip_config.json。

1. 配置参考文档：

PC 端部署 - Ubuntu / 2. 配置网络

1. 如果已经配置正确但再次检查发现被重置，按以下小节处理：

重启后 embosa IP 配置被改回去

机器人关节无法运动

本问题适用于头部、腿部或手臂的某个关节无响应的情况。

先查看 singorix 日志确认具体错误类型：

```
tail -f /userdata/log/SingoriX/info/singorix_wbcs_main.INFO
```

可能原因一：关节超出运动范围限制

关节位置可能已达到或超出其机械限位。此问题在手臂关节上较为常见，可通过目视观察到关节姿态异常，同时 singorix 会输出相应报错信息。

```
E20260114 10:45:39.889472 3264 wbcs_interface.cpp:737] Error component: WBCSManager, error code: 20, description: Sync leg ...
I20260114 10:45:39.890435 3264 wbcs_interface.cpp:706] wbcs_error_writer SENT 1 datas, time: 42620.2
E20260114 10:45:39.890522 3264 wbcs_interface.cpp:709] Errors Timestamp : 1768358739.889566412
E20260114 10:45:39.890702 3264 wbcs_interface.cpp:712] Errors component: right_arm_driver_state, error code: 41, description: DriverConstructed
E20260114 10:45:39.890815 3264 wbcs_interface.cpp:712] Errors component: right_arm_joint6_custom_error, error code: 1, description: Not yet identified
E20260114 10:45:39.890906 3264 wbcs_interface.cpp:712] Errors component: leg_driver_state, error code: 41, description: DriverConstructed
E20260114 10:45:39.890995 3264 wbcs_interface.cpp:712] Errors component: chassis_max_error_level, error code: 30000, description: ErrorSafetyMedium
E20260114 10:45:39.891083 3264 wbcs_interface.cpp:712] Errors component: chassis_driver_state, error code: 41, description: DriverConstructed
E20260114 10:45:39.891170 3264 wbcs_interface.cpp:712] Errors component: right_arm_max_error_level, error code: 30000, description: ErrorSafetyMedium
E20260114 10:45:39.891256 3264 wbcs_interface.cpp:712] Errors component: right_arm_joint7_custom_error, error code: 1, description: Not yet identified
E20260114 10:45:39.891343 3264 wbcs_interface.cpp:712] Errors component: right_arm_joint4_custom_error, error code: 1, description: Not yet identified
E20260114 10:45:39.891430 3264 wbcs_interface.cpp:712] Errors component: head_max_error_level, error code: 30000, description: ErrorSafetyMedium
E20260114 10:45:39.891517 3264 wbcs_interface.cpp:712] Errors component: right_arm_joint1_custom_error, error code: 1, description: Not yet identified
E20260114 10:45:39.891605 3264 wbcs_interface.cpp:712] Errors component: right_arm_joint2_custom_error, error code: 1, description: Not yet identified
E20260114 10:45:39.891693 3264 wbcs_interface.cpp:712] Errors component: right_arm_joint3_custom_error, error code: 59, description: Not yet identified
E20260114 10:45:39.891780 3264 wbcs_interface.cpp:712] Errors component: left_arm_driver_state, error code: 41, description: DriverConstructed
E20260114 10:45:39.891868 3264 wbcs_interface.cpp:712] Errors component: leg_max_error_level, error code: 30000, description: ErrorSafetyMedium
E20260114 10:45:39.891955 3264 wbcs_interface.cpp:712] Errors component: left_arm_max_error_level, error code: 30000, description: ErrorSafetyMedium
E20260114 10:45:39.892088 3264 wbcs_interface.cpp:712] Errors component: right_arm_joint5_custom_error, error code: 1, description: Not yet identified
E20260114 10:45:39.892216 3264 wbcs_interface.cpp:712] Errors component: head_driver_state, error code: 41, description: DriverConstructed
E20260114 10:45:39.892305 3264 wbcs_interface.cpp:712] Errors component: right_arm_joint3_error, error code: 200, description: Not yet identified
E20260114 10:45:39.893378 3264 wbcs_interface.cpp:735] Errors Timestamp : 1768358739.892447746
```

关节超限处理

建议按以下顺序处理：

1. 先尝试按住示教按钮，直接拖动机械臂回到关节限位范围内。
1. 如果无法拖动，先观察是哪个关节超限；二、三关节位置有隐藏抱闸按钮，可按下述方法处理。
 1. 拔下活塞，按下急停按钮，然后戳下隐藏抱闸按钮并拖动机械臂回到限位内。
 1. 旋开急停按钮复位，观察关节是否恢复正常。



如果超限关节没有隐藏抱闸按钮，使用方法 2：

1. 通过 service_display 进入后台：连续点击机器人胸前屏幕右下角，直到出现后台登录窗口，输入密码 996996。

1. 登录后进入：左臂电机或右臂电机 -> 电机操作（见图示例）。

1. 在电机操作页面的第一个输入框输入对应关节序号（例如 4 号关节）。

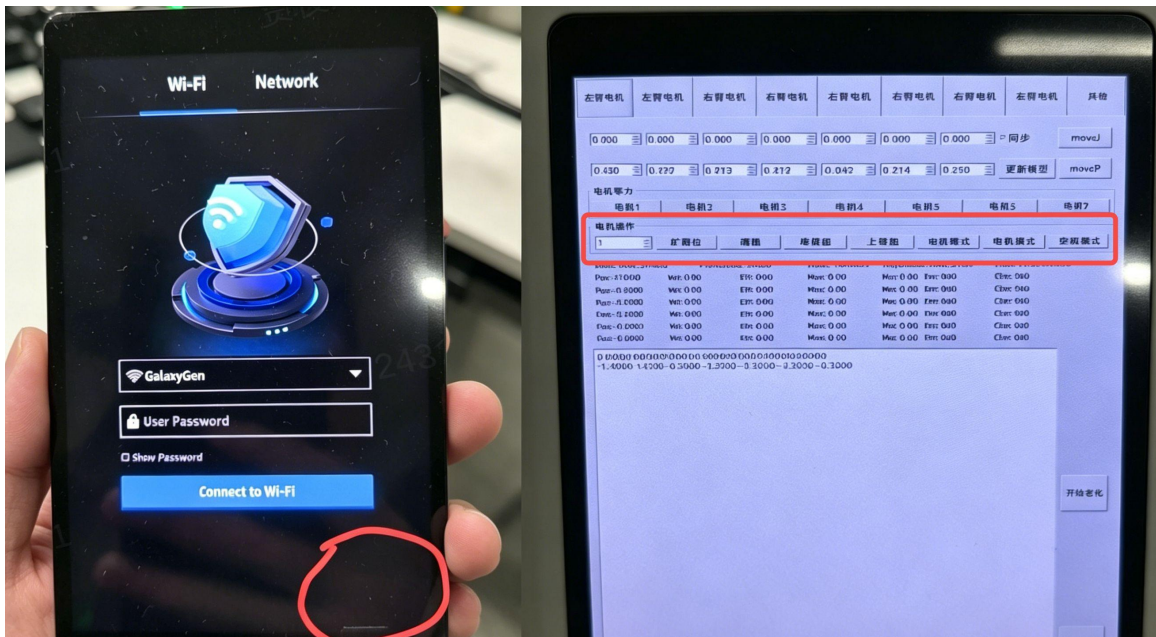
1. 拍下急停按钮，再旋开急停按钮，等待 10s。

1. 依次点击：空闲模式 -> 扩限位 -> 清错 -> 清错 -> 清错 -> 清错 -> 上使能 -> 电流模式（每个按钮间隔至少 1s）。

1. 查看机械臂对应关节是否可转动；如果仍不行，重复步骤 4 和步骤 5。

1. 处理完成后，在 XCU 执行以下命令重启 launcher，退出该界面并恢复常规运行状态：

```
systemctl restart launcher
```

可能原因二：关节初始化失败

机器人启动过程中，关节未成功释放制动器（无抱闸释放声）。singorix 会输出初始化失败的错误信息。

以下截图为头部关节初始化失败的示例：

```
E20251102 21:21:32.943883 7102 wbc_interface.cpp:593 Errors component: head_max_error_level, error code: 30000, description: ErrorSafetyMedium
E20251102 21:21:32.943974 7102 wbc_interface.cpp:593 Errors component: head_joint1_custom_error, error code: 1, description: Not yet identified
E20251102 21:21:32.944060 7102 wbc_interface.cpp:593 Errors component: head_driver_state, error code: 43, description: DriverError
E20251102 21:21:32.944147 7102 wbc_interface.cpp:593 Errors component: head_joint2_custom_error, error code: 1, description: Not yet identified
```

处理建议：重启机器人并留意启动时是否有抱闸释放声。

可能原因三：MCU 故障

微控制单元（MCU）可能出现错误或停止响应。检查以下目录下的最新日志文件，确认日志是否持续更新且无错误信息输出：

- /userdata/log/mp_core/
- /userdata/log/secure_core/

机器人进入异常模式：定位服务异常

问题现象：机器人进入 ABNORMAL，并提示定位服务异常。

常见场景：新机器人首次使用时较常见。

常见原因：未完成建图，导致定位服务无法正常运行。

处理建议：

1. 确认当前是否已完成建图。
1. 若未建图，先完成建图与定位流程。

1. 建图完成后，在 HPU 执行以下命令重启服务：

```
sudo systemctl restart launcher.service
```

1. 完成后再次查看工作模式，确认是否恢复为 WORKING_MODE。

参考建图文档：

- 建图流程
- 定位流程

机器人进入异常模式，重启服务

如果机器人进入异常模式，尝试重启所有服务进行恢复，可以重启 launcher。

• XCU 执行：

```
systemctl restart launcher
```

• HPU 执行：

```
sudo systemctl restart launcher
```

机器人底盘无法移动

底盘不动，先按通用检查项确认急停和工作模式，再看服务和电量。

检查底盘服务：

```
ps -ef | grep galbot_chassis_service | grep -v grep
```

再看 SOC：

```
tail -f /userdata/log/galbot_chassis_service/galbot_chassis_service.INFO | grep soc
```

SOC 低于 25% 时，底盘会停机保护，先充电再试。

```
root@galbot-xcu:~# ps -ef | grep galbot_chassis_service | grep -v grep
root      18740 18679 18 11:50 ?                00:03:42 /data/bin/galbot_chassis_service

root@galbot-xcu:~# tail -f /userdata/log/galbot_chassis_service/galbot_chassis_service.INFO | grep soc
I20260114 12:12:20.317093 19086 main.cpp:495] [mcu]I:soc is 78.9, cnt send 2656, rev 19035, 1588
I20260114 12:12:32.367662 19086 main.cpp:495] [mcu]I:soc is 78.8, cnt send 2662, rev 19077, 1592
```

末端执行器控制失败

先按通用检查项确认急停和工作模式，再做末端专项判断：急停按下再释放，看看末端有没有最基本的响应。如果完全没有动作，先按硬件故障方向查。

如果有响应但行为不对，再看配置是否一致。配置文件在：

```
/data/config/SingoriX/galbot_one_golf/real/robot_config.toml
```

1.16 及以后版本使用 overlay，配置路径改为：

```
/data/config/default/SingoriX/galbot_g1-v2.2/robot_config.toml
```

默认是左手夹爪、右手吸盘。请确认实物安装和配置一致（例如 right_suction_cup 与 right_gripper），修改后重启 launcher 生效：

```
systemctl restart launcher
```

```
[groups]
group_lists = [ "chassis", "leg", "head", "left_arm", "right_arm",
               "left_key", "right_key", "left_effort", "right_effort",
               "left_gripper",          # Choose one of three: left_none / left_gripper / left_suction_cup
               "right_suction_cup",     # Choose one of three: right_none / right_gripper / right_suction_cup
               "torso_imu"]
```

机器人扬声器无声音/启用音频服务 galbot_vtn

在 Orin/HPU 上编辑:

```
/data/config/launcher.cfg
```

把对应 enable 从 false 改成 true, 然后:

```
sudo systemctl restart launcher
systemctl status launcher
```

```
{
  "binary_path": "/data/galbot/bin",
  "modules": [
    {
      "name": "audio_player",
      "enable": false,
      "relaunch_flag": true
    },
    {
      "name": "galbot_vtn_service",
      "enable": false,
      "relaunch_flag": true
    },
    {
      "name": "service_navigation_plan",
      "enable": true,
      "relaunch_flag": true
    },
  ],
}
```

系统与配置

如何查看当前工作模式

可通过 XCU 上的 sys_tool 查询当前整机工作模式。

1. 登录 XCU:

```
ssh root@192.168.xxx.xx
# password: 12345678
```

1. 进入工具目录并启动：

```
cd /data/bin  
./sys_tool
```

sys_tool 是 XCU 系统工具，用于查询基线版本、切换/查询整机模式。工具路径：
/data/bin/sys_tool。

启动后会进入交互式命令行，常用命令：

- v：查询基线软件版本信息（含序列号、软硬件版本）。
- q：查询当前整机模式（用于判断是否处于正常工作模式）。

模式判断：

- 正常情况下应看到 mode_type：WORKING_MODE。
- 若不是 WORKING_MODE（例如进入 ABNORMAL），先按通用检查项第 3 步定位 abnormal_service，恢复后再继续其他排查。

示例（q 查询关键字段）：

```
current mode:  
mode_type: WORKING_MODE  
mode_switch_state {  
  status: SUCCESS  
}
```

```

root@galbot-xcu:~# /data/bin/sys_tool
[folder_mkdirs]: folder [/userdata/log/sys_tool] has already exists
#####
m: 切换整机模式
q: 查询整机模式
o: 查询整机 Onboard Config 配置信息
v: 查询基线软件版本信息
p: 启动 pipeline 任务
c: 结束 pipeline 任务
cr: 触发回充
cu: 触发出桩
ci: 打断充电
pack: 切换打包姿态
exit: 退出
r: 重启 Launcher
#####
please input cmd_type: q
current mode: SN: "R001GBDDAAAE0632"
timestamp: 1778752654139
mode_type: WORKING_MODE
mode_switch_state {
    status: SUCCESS
}
monitor_metrics {
    bms_soc: 98
    bms_voltage: 53.900001525878906
    charging_type: INVALID_CHARGE
    bms_wireless_charge_current: 0
}
}
I20260514 15:56:38.998538 3256 main.cpp:532] [hpu_commond_callback]: Hpu commond request content: {
    "abnormal_service": "service_motion_plan",
    "mode_type": 7,
    "timestamp": 1778745398992319
}
I20260514 15:56:38.998790 3256 main.cpp:555] [hpu_commond_callback]: Has process hpu commond!
I20260514 15:56:39.428421 3267 mode_switch_state.cpp:455] [Transition]: Moving process: 31

```

如何查看机器人系统版本

推荐优先使用 SDK 命令快速查看：

```
galbot_sdk check-version
```

如果机器人使用非默认 IP 地址：

```
galbot_sdk check-version --robot-ip <机器人 IP>
```

如果当前环境没有该命令，再在 XCU 或 HPU 上执行：

```
cat /data/config/system.cfg
```

看 CUR_VERSION 字段即可。

```
galbot@galbot-echo:~$ cat /data/config/system.cfg
{
  "SN": "SN001",
  "HARDWARE_VERSION": "2.2",
  "enable_modify_embosa_cfg": true,
  "software_environment": {
    "system_version": "ubuntu20.04",
    "device_type": "hpu",
    "architecture": "aarch64"
  },
  "environment_name": "/var/maps/beijing1701/",
  "log": {
    "log_path": "/userdata/log",
    "log_storage_period_day": 5
  },
  "tcp": {
    "server": {
      "ip": "192.168.1.88",
      "port": [
        50050,
        50051
      ]
    }
  },
  "audio_player": {
    "volume": 100,
    "resource_path": "/data/galbot/resource/",
    "mixer_control": "default"
  },
  "CUR_VERSION": "GBS_1.15.2",
  "ROBOT_SERIES": "GI",
  "INTEGRATION_TIME": "2026-01-27",
  "APPLY_SCENE": "VLA-CAPSULE"
}
```

如何检查机器人系统版本与 SDK 兼容性

使用以下命令快速检查当前 SDK 与机器人系统版本（GBS）是否匹配：

```
galbot_sdk check-version
```

常用参数：

指定非默认机器人 IP 地址

```
galbot_sdk check-version --robot-ip <机器人 IP>
```

查看完整兼容性对照

```
galbot_sdk check-version --list
```

```
# 离线手工校验
galbot_sdk check-version --sdk-version 1.8.0 --robot-version GBS_1.16.0
```

兼容性对照（参考 output/check_robot_compat.py）：

SDK 版本范围	对应机器人系统版本（GBS）
1.5.0 ~ 1.7.99	GBS_1.15.x
1.8.0 ~ 1.8.99	GBS_1.16.x

如果检查结果不匹配，优先升级机器人系统版本到 SDK 要求范围；只有在必须保持当前机器人版本时，再选择降级 SDK。

机器人紧急开关（急停按钮）说明

急停按钮位于机器人本体背部醒目位置，为红色按钮，标有 EMERGENCY STOP 字样。





使用方法：

1. 一旦发现设备异常或人身危险，立即按下急停按钮。
1. 急停触发后，机器人即刻终止全部运动；设备不断电，仅锁定运动与执行机构。
1. 急停按钮按下后会自动自锁，需顺时针旋转完成解锁释放。
1. 解锁复位后，机器人方可重新接收控制指令，恢复正常运动操作。

机器人机载算力单元介绍 (HPU + XCU)

Galbot G1 的机载计算单元采用双计算单元协同设计，由 HPU (High-Performance Unit) 和 XCU (X Computing Unit) 共同组成。其中 HPU 主要负责视觉感知、AI 推理、运动规划、导航等高算力任务，XCU 主要负责底层电机控制、关节执行与实时控制任务。该设计遵循高可靠性、实时性与扩展性原则，是理解通信延迟、部署架构和故障排查的基础。

计算单元	全称	职责	典型任务
HPU	High-Performance Unit	高算力任务	视觉感知、AI 推理、运动规划、导航、SLAM
XCU	X Computing Unit	实时控制任务	底层电机控制、关节执行、实时控制

- 硬件配置：HPU 搭载 NVIDIA Jetson Orin 平台 (64GB, 275 TOPS) ，XCU 为实时控制侧处理器。
- 系统软件：运行定制化 Linux，集成实时内核补丁，保障运动控制的确定性执行。
- 架构注意：HPU 与 XCU 均为 ARM64 (aarch64) 架构，在这两个平台上运行的程序需使用 aarch64 工具链交叉编译。

为什么拆成两个计算单元？

- 实时性隔离：电机控制需要微秒级确定性响应，视觉推理可能一次性占用 GPU 几百毫秒；放在同一计算单元会互相抢占资源，导致关节卡顿甚至整机失控。
- 故障隔离：HPU 侧 AI 模型异常时，XCU 仍可维持底层控制与站立姿态，避免单点故障扩散为整机失效。
- 扩展性：后续升级算力时可优先替换 Orin 模块，XCU 的实时控制逻辑与固件保持稳定，降低硬件迭代成本。

如何进入 HPU 与 XCU

可通过以下方式获取登录 IP：

1. 在机器人 Network 界面查看当前无线 IP。
1. PC 与机器人有线直连时，使用固定 IP：

- XCU：192.168.1.66
- HPU (Orin) ：192.168.1.88

登录方式与账号密码如下：

- 登录 HPU (Orin) ：


```
ssh galbot@<Orin_IP>
# 密码: gb@2023
```

- 登录 XCU:

```
ssh root@<XCU_IP>
# 密码: 12345678
```

建议先 ping 验证连通性，再执行 ssh 登录。

如何查看机器 SN 号

在 XCU 上执行:

```
/data/bin/sys_tool
```

按工具提示查看设备信息。

如何检查关节是否正常

登录 XCU 后进入关节诊断工具，输入 r 看状态。

No Errors 一般就说明关节正常。若提示急停或关节速度过快相关错误，先复位再测。

```
Other info:
  left_key_pressed: 0
Errors Timestamp: 1774246512.716847769
Errors component: left_arm_driver_state, error code: 46, description: DriverEstop
Errors component: chassis_driver_state, error code: 46, description: DriverEstop
Errors component: left_arm_max_error_level, error code: 50000, description: ErrorSafetyCritical
Errors component: right_arm_driver_state, error code: 46, description: DriverEstop
Errors component: head_max_error_level, error code: 50000, description: ErrorSafetyCritical
Errors component: right_arm_max_error_level, error code: 50000, description: ErrorSafetyCritical
Errors component: leg_max_error_level, error code: 50000, description: ErrorSafetyCritical
Errors component: head_driver_state, error code: 46, description: DriverEstop
Errors component: chassis_max_error_level, error code: 50000, description: ErrorSafetyCritical
Errors component: leg_driver_state, error code: 46, description: DriverEstop

cartesian_tar_pos: [0, 0, 0, 0, 0, 0]
Errors Timestamp: 1772700041.042746484
Errors component: left_arm_driver_state, error code: 45, description: DriverError
Errors component: left_arm_joint3_custom_error, error code: 52, description: Not yet identified
Errors component: left_arm_joint5_custom_error, error code: 52, description: Not yet identified
Errors component: left_arm_max_error_level, error code: 30000, description: ErrorSafetyMedium
Errors component: left_arm_joint4_custom_error, error code: 52, description: Not yet identified
Errors component: left_arm_joint2_error, error code: 4, description: Not yet identified
Errors component: left_arm_joint4_error, error code: 4, description: Not yet identified
Errors component: left_arm_joint1_custom_error, error code: 52, description: Not yet identified
Errors component: left_arm_joint1_error, error code: 4, description: Not yet identified
Errors component: left_arm_joint7_error, error code: 4, description: Not yet identified
Errors component: left_arm_joint6_custom_error, error code: 52, description: Not yet identified
Errors component: left_arm_joint5_error, error code: 4, description: Not yet identified
Errors component: left_arm_joint7_custom_error, error code: 52, description: Not yet identified
Errors component: left_arm_joint3_error, error code: 4, description: Not yet identified
Errors component: left_arm_joint6_error, error code: 4, description: Not yet identified
Errors component: left_arm_joint2_custom_error, error code: 52, description: Not yet identified
```

如何启用末端实时控制指令

在 XCU 上把:

```
/data/config/default/SingoriX/galbot_g1-v2.2/robot_config.toml
```

里的 using_wbc 改成 true, 然后重启 launcher。

关节限位在哪里配置

IK、规划和控制用的是同一套关节限位, 来源是 URDF: galbot_one_golf.urdf。

路径如下:

- XCU: /data/config/galbot_description/galbot_one_golf_description
- HPU: /data/galbot/config/galbot_description/galbot_one_golf_description

启用/禁用某个系统服务

根据目标服务运行在 HPU 还是 XCU, 登录对应平台后编辑:

```
/userdata/user_config/mode_service_config_default.toml
```

注释或取消注释对应服务项后, 在对应平台重启 launcher:

```
systemctl restart launcher
```

```
root@galbot-xcu:/data/config# cat /data/config/mode_service_config_default.toml
# 整机模式配置文件

# 系统启动自动进入的模式: INVALID_MODE(无效模式)、WORKING_MODE、TELEOPERATION_MODE(遥控模式)、DEPLOYMENT_MODE(部署模式)、DATA_COLLECTION_MODE(数据采集)
[start_up_mode]
auto_enter_mode_type = "WORKING_MODE"

# 无效模式下动态启动的服务列表
# [[invalid_mode_config]]
# service_name = "localization_server"      # 服务名
# relaunch_flag = false                     # 是否允许重拉起, true-允许, false-不允许

# OTA动态启动的服务列表
# [[ota_mode_config]]
# service_name = "localization_server"      # 服务名
# relaunch_flag = false                     # 是否允许重拉起

# 异常模式下动态启动的服务列表
# [[abnormal_mode_config]]
# service_name = "localization_server"
# relaunch_flag = false

# 工作模式下动态启动的服务列表
[[working_mode_config]]
service_name = "localization_server"
relaunch_flag = false

[[working_mode_config]]
service_name = "relocalization_server"
relaunch_flag = false

[[working_mode_config]]
service_name = "service_livox_capture"
relaunch_flag = true
```

重启后 embosa IP 配置被改回去

在 XCU 和 HPU 上分别把 /data/config/system.cfg 里的 enable_modify_embosa_cfg 设为 false, 然后分别重启 launcher。

```
root@galbot-xcu: ~ 118x29
"APPLY_SCENE": "VLA-CAPSULE",
"CUR_VERSION": "GBS_1.16.0.1.rc1",
"HARDWARE_VERSION": "2.2",
"INTEGRATION_TIME": "20260401",
"ROBOT_SERIES": "G1",
"SN": "R001GBDDAAAE0879",
"application_scene": "mainline",
"auto_charging_mode_enable": false,
"enable_modify_embosa_cfg": true,
environment_name : /var/maps/beijing1701/",
"log": {
  "log_path": "/userdata/log",
  "log_storage_period_day": 5
},
"main_mission": {
  "enable_navigation_charging_mode": false
},
"ota": {
  "device": "XCU",
  "has_ota": false
},
"software_environment": {
  "architecture": "aarch64",
  "device_type": "xcu",
  "system_version": "ubuntu18.04"
},
"tcp": {
/data/config/system.cfg" [noeol] 38L, 964C
```

相机与标定信息

如何修改相机分辨率

在 Orin 上改这几个文件：

- /data/galbot/config/right_arm_camera.cfg
- /data/galbot/config/left_arm_camera.cfg
- /data/galbot/config/front_head_camera.cfg

改完重启机器就会生效。多传感器同时开的时候，适当降分辨率会稳一些。

```
galbot@galbot-ech:/data/galbot/config$ ls
aphrodm_mps_config      model_weights_config.json
aphrodm_pns_config      moduleConfig.yaml
cacert.pem               navigation_plan_config.pbtxt
cvs                      perception.cfg
data_backflow_service_install.sh  perceptionCoreconfig.ini
data_backflow_service.json  right_arm_camera.cfg
data_backflow_service.service  right_arm_camera_data_collection.cfg
data_backflow_service.sh     robot_body.toml
dev_manage_mqtt_install.sh    robot_offset.yaml
dev_manage_mqtt.json         robot_state_publisher_cfg.json
dev_manage_mqtt.service      ros_sensorConfig.ini
dev_manage_mqtt.sh          sensorConfig.ini
embosa_ip_config.json        singort
embosa_logger_cfg.json      stereo_camera_intrinsic.toml
embosa_sensorConfig.ini     surr_cam_params.yaml
front_head_camera.cfg       surround_cameras.cfg
front_head_camera_data_collection.cfg  surround_cameras_data_collection.cfg
galbot_description         swallows
galbot_fusion_params.yaml   swallows.json
launcher.cfg               sys_monitor.service
launcher.service           system.cfg
left_arm_camera.cfg         upload.toml
left_arm_camera_data_collection.cfg  urdf_parser_config
livox_mid360_config.json    vtn_param.cfg
mes
mic
```

```
camera_name = "left_arm_camera"
camera_type = "RealSense_D405"

color_width = 1280
color_height = 720
depth_width = 1280
depth_height = 720
encoding = "jpeg"
jpeg_qua = 60
frame_rate = 15
auto_exposure = 1
compress_enabled=false
compression_level=90
server_ip = "192.168.1.50"
ir_enabled = false
ir_width = 1280
ir_height = 720
~
```

如何查看相机帧率

可以先看 /data/galbot/config/xxx.cfg 里的配置，再用 HPU 上 /data/galbot/bin/embosa_topic_tool 统计话题频率。


```
galbot@galbot-echo:/data/galbot/config$ grep "frame_rate" *.cfg
front_head_camera_capture.cfg:frame_rate = 15
front_head_camera_capture_data_collection.cfg:frame_rate = 30
front_head_camera_capture_factory_s56.cfg:frame_rate = 15
front_head_camera_capture_factory_shitong.cfg:frame_rate = 15
front_head_camera.cfg:frame_rate = 15
front_head_camera_data_collection.cfg:frame_rate = 30
left_arm_camera_capture.cfg:frame_rate = 15
left_arm_camera_capture_data_collection.cfg:frame_rate = 30
left_arm_camera_capture_factory.cfg:frame_rate = 15
left_arm_camera.cfg:frame_rate = 15
left_arm_camera_data_collection.cfg:frame_rate = 30
right_arm_camera_capture.cfg:frame_rate = 15
right_arm_camera_capture_data_collection.cfg:frame_rate = 30
right_arm_camera_capture_factory.cfg:frame_rate = 15
right_arm_camera.cfg:frame_rate = 15
right_arm_camera_data_collection.cfg:frame_rate = 30
surround_cameras_capture.cfg:frame_rate = 10
surround_cameras_capture_data_collection.cfg:frame_rate = 10
surround_cameras_capture_factory.cfg:frame_rate = 10
surround_cameras.cfg:frame_rate = 10
surround_cameras_data_collection.cfg:frame_rate = 10

galbot@galbot-echo:/data/galbot/bin$ ./embosa_topic_tool hz /left_front_surround/color/image_raw
-----
Topic: /left_front_surround/color/image_raw
Message Type: galbot.sensor_proto.CompressedImage
-----
Max      Min      Cur
9.98     9.98     9.98
-----
Topic: /left_front_surround/color/image_raw
Message Type: galbot.sensor_proto.CompressedImage
-----
Max      Min      Cur
10.00    9.98     10.00
-----
Topic: /left_front_surround/color/image_raw
Message Type: galbot.sensor_proto.CompressedImage
-----
Max      Min      Cur
10.00    9.98     10.00
-----
Topic: /left_front_surround/color/image_raw
Message Type: galbot.sensor_proto.CompressedImage
-----
Max      Min      Cur
10.00    9.94     9.94
```

如何查看相机内外参和标定信息

XCU 上直接看文件：

```
cat /userdata/config/robot_config/eyehand_calib.toml
cat /userdata/config/robot_config/stereo_camera_intrinsic.toml
```

也可以直接用 SDK 接口读，内参和外参都能拿到。例如获取左臂相机外参：

```
transform = robot.get_transform(
    target_frame="base_link",
    source_frame="left_arm_camera_color_optical_frame",
)
```

或者：

```
transform = robot.get_sensor_extrinsic(sensor_id=SensorType.LEFT_ARM_CAMERA)
```

日志与调试

底层服务日志在哪里

底层服务按平台区分：

- HPU：定位、导航、规划、感知等服务。
- XCU：控制、底盘等服务。

对应日志都在本平台的 /userdata/log/<服务名>/ 下，例如：

- HPU： /userdata/log/localization_server/、
/userdata/log/service_navigation_plan/、 /userdata/log/service_motion_plan/、
/userdata/log/Perception_Algorithm_Service/
- XCU： /userdata/log/SingoriX/、 /userdata/log/galbot_chassis_service/、
/userdata/log/service_display/、 /userdata/log/mission_manager/

排查时先确认服务跑在哪个平台，再去对应平台查看对应服务目录。

SDK log 在哪里

PC 或 Robot 侧运行 SDK 后，日志默认在：

```
~/galbot_sdk_log/
```

如果在 PC 上看到 get info failed 或 PUBLISH FAIL，优先查两端的 embosa_ip_config.json。

如何开启 core 文件并调试

core 文件通常用来定位服务崩溃。开启 core、设置路径、重新登录 shell 之后，文件一般会落在：

```
/userdata/core
```

例如 mps 服务可以这样看：

```
gdb /data/galbot/bin/service_motion_plan [core_file]
```

进 gdb 后执行 bt -full。

```

40037537925855745, 1945608225577174814, 2834715, 9232388042942056448, 8796093022208, 9007199254740992, 9232388042939957248, 0, 0, 0, 0, 9232388042942056450}}, sa_f
lags = -2145384446, sa_restorer = 0xffff73df1d18 <abort@got.plt>}
sigs = {_val = {32, 0 <repeats 15 times>}}
#2 0x0000ffff73c1380c in _gnu_cxx::__verbose_terminate_handler() () at /usr/lib/aarch64-linux-gnu/libstdc++.so.6
#3 0x0000ffff73c1320c in () at /usr/lib/aarch64-linux-gnu/libstdc++.so.6
#4 0x0000ffff73c13044 in () at /usr/lib/aarch64-linux-gnu/libstdc++.so.6
#5 0x0000ffff73c13044 in _gxx_personality_v0 () at /usr/lib/aarch64-linux-gnu/libstdc++.so.6
#6 0x0000ffff73b4e504 in () at /usr/lib/aarch64-linux-gnu/libgcc_s.so.1
#7 0x0000ffff73b4eef4 in Unwind_Resume () at /usr/lib/aarch64-linux-gnu/libgcc_s.so.1
#8 0x0000ffff73b43db1f0 in () at /usr/lib/aarch64-linux-gnu/libnvinfer.so.8
#9 0x0000ffff73b43c4f7c in () at /usr/lib/aarch64-linux-gnu/libnvinfer.so.8
#10 0x0000ffff73b4355ee0 in () at /usr/lib/aarch64-linux-gnu/libnvinfer.so.8
#11 0x0000ffff73b4359db0 in () at /usr/lib/aarch64-linux-gnu/libnvinfer.so.8
#12 0x0000ffff73b41ba0 in easy_deploy::TrtInferCore::Inference(std::shared_ptr<easy_deploy::IPipelinePackage>, std::cxx11::basic_string<char>, std::char_traits<ch
ar>, std::allocator<char> >) () at /data/galbot/lib/libtrt_core.so
#13 0x0000ffff73e1c52c in easy_deploy::BaseInferCore::SyncInfer(easy_deploy::BlobsTensor*, std::cxx11::basic_string<char>, std::char_traits<char>, std::allocator<
char> >, int) () at /data/galbot/lib/libdeploy_core.so
#14 0x0000ffff73e333e0 in easy_deploy::BaseStereoMatchingModel::ComputeDisp(cv::Mat const&, cv::Mat const&, cv::Mat&, std::cxx11::basic_string<char>, std::char_tr
aits<char>, std::allocator<char> > const&) () at /data/galbot/lib/libdeploy_core.so
#15 0x0000ffff73c23b80 in FoundationStereo::run(std::vector<ImageFrameStamped>, std::allocator<ImageFrameStamped> > const&, DetectionResult&, std::cxx11::basic_st
ring<char>, std::char_traits<char>, std::allocator<char> >&) () at /data/galbot/lib/libfoundationStereo.so
#16 0x0000ffff739ca3f44 in PerceptionInterface::run(std::vector<ImageFrameStamped>, std::allocator<ImageFrameStamped> > const&, DetectionResult&) ()
    at /data/galbot/lib/libPerceptionInterface.so
#17 0x0000ffff739ca402c in PerceptionInterface::execute(PerceptionConfig const&, std::vector<ImageFrameStamped>, std::allocator<ImageFrameStamped> > const&, Detectio
nResult&) () at /data/galbot/lib/libPerceptionInterface.so
#18 0x00000000000049c7c in GalbotPerceptionServer::handleRunOnceCommand(ServerCommand const&):(lambda()#1)::operator()() const ()
#19 0x00000000000049d3c in std::Function_handler<void (), BS::thread_pool<(unsigned char)l>::submit_task<GalbotPerceptionServer::handleRunOnceCommand(ServerCommand
const&):(lambda()#1), void>(GalbotPerceptionServer::handleRunOnceCommand(ServerCommand const&):(lambda()#1)&&, signed char):(lambda()#1)>::M_invoke(std::Any_
data const&) ()
#20 0x00000000000049b3c in BS::thread_pool<(unsigned char)l>::worker(unsigned long) ()
--Type <RET> for more, q to quit, c to continue without paging--
#21 0x0000ffff73c0bf9c in () at /usr/lib/aarch64-linux-gnu/libstdc++.so.6
#22 0x0000ffff739cc6024 in start_thread (arg=0xffff73cdef80) at pthread_create.c:477
    ret = <optimized out>
    pd = <optimized out>
    unwind_buf =
        {cancel_jmp_buf = {{jmp_buf = {281470772194588, 281474144818704, 281474144818702, 281473262198784, 281474144818703, 281472624619392, 281470772195328,
281470772195336, 281473262202880, 281470772191456, 281470772191456, 9614905107694120012, 0, 9614905105888979376, 1065353216, 0, 0, 0, 0, 0, 0}, mask_was_saved
= 0}}, priv = {pad = {0x0, 0x0, 0x0, 0x0}, data = {prev = 0x0, cleanup = 0x0, canceltype = 0}}
    not_first_call = <optimized out>
#23 0x0000ffff73a9f52c in thread_start () at ../sysdeps/unix/sysv/linux/aarch64/clone.S:78
(qdb) Quit

```

在 Robot 上执行 Python 找不到 module

在 ~/.bashrc 里加一行：

```
export PYTHONPATH=/data/galbot/lib:$PYTHONPATH
```

重新登录后生效。

执行 Python 示例报错：numpy 没有 typeDict

一般是 scipy 版本太低，升级一下：

```
pip install --upgrade scipy
```

安装与环境

SDK Docker 启动方式

```
gunzip -c galbot_sdk_image_1.8.0.tar.gz | docker load
cd output/docker
./run.sh
```

如何使用感知服务（GBS 1.15.16+）

在 HPU 上修改：

```
/data/galbot/config/galbot_perception_moduleConfig.yaml
```

把 sensor_data_sync_max_diff_tolerance 改成 500000（纳秒），重启 launcher 后再跑 examples/g1/python/galbot_perception 下的示例验证。

仍未解决？

如果你的问题没有在本文档中列出，可以在 GitHub 提交 issue：

<https://github.com/GalaxyGeneralRobotics/GalbotSDK/issues>

提交时请标注你的 SDK/机器人版本和具体问题现象，我们会尽快协助恢复；也可以直接联系售后技术支持人员。

雪山混混

19. Galbot SDK 许可证

Copyright (c) 2023-2026 Galbot. All Rights Reserved.

This software contains confidential and proprietary information of Galbot, Inc. ("Confidential Information"). You shall not disclose such Confidential Information and shall use it only in accordance with the terms of the license agreement you entered into with Galbot, Inc.

UNAUTHORIZED COPYING, USE, OR DISTRIBUTION OF THIS SOFTWARE, OR ANY PORTION OR DERIVATIVE THEREOF, IS STRICTLY PROHIBITED. IF YOU HAVE RECEIVED THIS SOFTWARE IN ERROR, PLEASE NOTIFY GALBOT, INC. IMMEDIATELY AND DELETE IT FROM YOUR SYSTEM.

本项目中使用的第三方依赖库的许可证文件位于 licenses/ 目录中。

雪山混混